
ANALYST TO ARCHITECT · BOOK 4 OF 4

Be Interview Ready

Part 8. How data hiring works, the extra-points method, and a question bank for each skill and role, with take-home assignments and mock interviews.

Contents

Part 8 — Be Interview Ready	1
Chapter 68. How Data Hiring Works	2
68.1 The whole process, gate by gate	3
68.2 Cracking the resume screen: the software gate, then the human gate	4
68.3 What each round actually tests	6
68.4 LinkedIn hiring: how recruiter search actually works, and how to be found	7
68.5 Referrals, used properly	9
68.6 Pay: reading the numbers before the offer gate	9
68.7 Preparing in 30, 60, or 90 days	11
68.8 Worked examples, in three groups	12
68.9 Four resumes, matched to their JDs	17
Chapter 69. The Extra-Points Method	27
69.1 How interviewers actually score	28
69.2 The shape of a strong answer	29
69.3 The twelve extra-point moves	30
69.4 One question, all three tiers	34
69.5 The same moves, under different rounds	35
69.6 What not to do	35
Chapter 70. Excel, Google Sheets, VBA & BI Question Bank	45
70.0 The practice table, and how to use this bank	46
70.1 Formulas, lookups, and logic	47
70.2 Pivot tables and data analysis	50
70.3 Power Query	53
70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE	54
70.5 VBA and Excel macros	56
70.6 Google Apps Script	60
70.7 DAX and Power BI modeling	63
70.8 Dashboard critiques	66
Chapter 71. SQL Question Bank	71
71.1 Core concepts: SELECT, WHERE, and JOIN	72
71.2 Basic questions that are trickier than they look	75
71.3 Aggregation: GROUP BY and HAVING	82
71.4 Subqueries, CTEs, and EXISTS vs. IN	83
71.5 Window functions: the classics	86
71.6 Classic problems: duplicates, gaps-and-islands, retention	90
71.7 Schema, constraints, and transactions	94
71.8 PostgreSQL vs. MySQL: where the dialects actually differ	101
71.9 Query optimization and indexes	103
71.10 Live-coding walk-throughs	106
Chapter 72. Python & pandas Question Bank	115
72.1 Python fundamentals: the gotchas that catch experienced candidates	116
72.2 Predict the output: basic to advanced tricky questions	121
72.3 Functions, generators, and decorators	127
72.4 pandas fundamentals: Series, DataFrames, and selection	131
72.5 pandas: grouping, joining, and reshaping	134
72.6 pandas: performance, missing data, strings, and dates	138
72.7 Debugging and live-coding walk-throughs	142

Chapter 72A. Data Structures & Algorithms Question Bank	148
72A.1 Why data roles get asked this at all	149
72A.2 Big-O: reasoning about time and space	149
72A.3 Arrays, strings, and the two-pointer / sliding-window patterns	153
72A.4 Hash maps and sets	157
72A.5 Recursion and memoization	159
72A.6 Sorting and searching	162
72A.7 Linked lists	166
72A.8 Trees and graphs	169
72A.9 A live-coding walk-through: parentheses validation	174
Chapter 73. Statistics, Probability & Experimentation Bank	178
73.0 The setup cell, and how to use this bank	179
73.1 Core probability concepts	180
73.2 Distributions	184
73.3 Hypothesis testing fundamentals	186
73.4 A/B test design	189
73.5 A/B test debugging	191
73.6 Causal inference and Simpson's paradox	194
73.7 Live-coding and live-analysis walk-throughs	198
Chapter 74. Machine Learning Question Bank	204
74.1 Basic-but-tricky ML questions	205
74.2 Bias, variance, and the shape of a good model	207
74.3 Metrics, calibration, and thresholds	211
74.4 Data leakage	213
74.5 Feature engineering	217
74.6 Model debugging	219
74.7 ML case studies and live-coding walk-throughs	220
Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates	225
75.1 The general case framework	226
75.2 Diagnosing a metric that moved	228
75.3 Designing metrics and dashboards	230
75.4 Product sense and decision cases	233
75.5 Guesstimates: structured estimation	234
75.6 Full cases, talked through live	238
Chapter 76A. Data Analyst & Data Scientist Question Bank	242
76A.1 What the two roles actually do, and how interviewers probe the line between them	243
76A.2 Walking through a Data Analyst project, end to end	245
76A.3 Walking through a Data Science project, end to end	247
76A.4 Stakeholder communication for DA and DS specifically	248
76A.5 Portfolio deep-dives and handling scrutiny	250
76A.6 Full scenarios, talked through live	252
Chapter 76B. Business Analyst Question Bank	256
76B.1 Basic-but-tricky BA questions	257
76B.2 Requirements gathering	259
76B.3 User stories and acceptance criteria	261
76B.4 Process mapping	263
76B.5 SDLC and where a BA fits	265
76B.6 Gap analysis and UAT	267
76B.7 Stakeholder management and pushback	269

76B.8 Full scenarios, talked through live	270
Chapter 77. Data Engineering & Data System Design Bank	274
77.1 Pipeline fundamentals	275
77.2 Basic-but-tricky data engineering questions	277
77.3 Data modeling for pipelines and warehouses	280
77.4 Orchestration and scheduling	283
77.5 Scale, partitioning, and performance	288
77.6 Data quality and monitoring	291
77.7 Full system design walk-throughs	293
Chapter 78. Automation & Integration Question Bank	298
78.1 Choosing the right automation approach	299
78.2 Basic-but-tricky automation questions	300
78.3 Report and alert automation	302
78.4 APIs, webhooks, and reverse ETL	303
78.5 Failure handling and monitoring for automations	307
78.6 Full design case	311
Chapter 79. GenAI, LLM & MLOps Question Bank	316
79.1 LLM fundamentals	317
79.2 Basic-but-tricky GenAI questions	320
79.3 Retrieval-Augmented Generation (RAG)	322
79.4 Agents and tool use	325
79.5 Evaluating AI applications	327
79.6 MLOps: making models survive production	328
79.7 LLMOps	331
79.8 Full design cases	333
Chapter 80. Architecture & Leadership Question Bank	339
80.1 Distributed systems trade-offs	340
80.2 Basic-but-tricky architecture questions	341
80.3 Data architecture decisions	343
80.4 Security, governance, and cost	344
80.5 Leadership scenarios	348
80.6 Full architecture design cases	349
Chapter 81. Behavioral, HR & Offer Conversations	354
81.1 The STAR method	355
81.2 Building a story bank	356
81.3 Classic behavioral questions, worked in full	358
81.4 Basic-but-tricky HR questions	360
81.5 Questions to ask interviewers	362
81.6 Handling offers	363
81.7 Handling a hostile follow-up, live	366
Chapter 82. Take-Home Assignments & Mock Interviews	370
82.0 Before you start: is this a fair take-home?	371
82.1 Take-home assignment: Data Analyst	371
82.2 Take-home assignment: Data Scientist	376
82.3 Take-home assignment: Data Engineer	381
82.4 Mock interview: Data Analyst (entry level)	386
82.5 Mock interview: Data Scientist (mid level)	388
82.6 Mock interview: Data Engineer (mid level)	390

Part 8 — Be Interview Ready

How data hiring works, and question banks for each role. Use Part 8 when you apply for a job: read Chapters 68 and 69 first, then the banks for the role you want.

Chapter
68. How Data Hiring Works
69. The Extra-Points Method
70. Excel, Google Sheets, VBA & BI Question Bank
71. SQL Question Bank
72. Python & pandas Question Bank
72A. Data Structures & Algorithms Question Bank
73. Statistics, Probability & Experimentation Bank
74. Machine Learning Question Bank
75. Product Sense, Metrics, Case Studies & Guesstimates
76A. Data Analyst & Data Scientist Question Bank
76B. Business Analyst Question Bank
77. Data Engineering & Data System Design Bank
78. Automation & Integration Question Bank
79. GenAI, LLM & MLOps Question Bank
80. Architecture & Leadership Question Bank
81. Behavioral, HR & Offer Conversations
82. Take-Home Assignments & Mock Interviews

Chapter 68. How Data Hiring Works

Chapter at a glance

You will learn to: see the full hiring process from the other side of the table, so nothing in it surprises you · write a CV that passes both the screening software and the human who reads it next · know what each interview round actually tests, for whichever of the ten roles you're aiming at · build a portfolio and a LinkedIn profile that do real work for you, not just sit there · use referrals properly, without asking for a favor you haven't earned · read pay figures across roles before the offer conversation · run a focused 30/60/90-day preparation plan instead of "studying everything."

Before you start: Chapter 7 (the ten roles and tracks); Chapter 8, especially §8.5 (decoding a job description) and §8.6 (reading salary figures); and Chapter 9, especially §9.1 (estimating your timeline from Time needed lines) and §9.5 (the seven-part portfolio piece). This chapter doesn't repeat them; it builds on them.

Time needed: 2.5–3.5 hours to read; allow 3–5 hours more for the project. After that it's a reference you return to at each stage of a real search.

In this chapter

START	Why this matters · In plain English	2
LEARN	68.1 The whole process, gate by gate	3
	68.2 Cracking the resume screen: the software gate, then the human gate	4
	68.3 What each round actually tests	6
	68.4 LinkedIn hiring: how recruiter search actually works, and how to be found	7
	68.5 Referrals, used properly	9
	68.6 Pay: reading the numbers before the offer gate	9
	68.7 Preparing in 30, 60, or 90 days	11
	68.8 Worked examples, in three groups	12
	68.9 Four resumes, matched to their JDs	17
APPLY	Common mistakes · In the real world · Project	22
REVIEW	Recap · Key terms · Check yourself	23
PRACTISE	Exercises	24
	Answers	25
NEXT	Where this leads	26

START

Why this matters

Most candidates prepare for the *questions*. Far fewer prepare for the *process*: the sequence of filters between "I want this job" and "I have an offer," each with its own, different test. A brilliant answer to a system design question is wasted if your CV never reached a human, or if you skip the salary conversation entirely because nobody told you when it happens. This chapter is the map of that whole process, so every other chapter in this part lands where it actually matters.

In plain English

Think of hiring as a series of gates, not one big exam.

A CV goes through a gate that's often software, then a human skimming for thirty seconds, then a recruiter call, then one or more rounds that test different things, then a final conversation about money and start date. Each gate has a different judge and a different question in mind. Preparing brilliantly for gate four while walking into gate one unprepared means you never reach gate four at all. This chapter walks the gates in order.

LEARN

68.1 The whole process, gate by gate

For a typical data role, in order:

1. **Application.** Your CV (also called a *résumé*; in Indian hiring the two words mean the same thing, and this chapter uses both) goes in, often with a short form, usually through an applicant tracking system (**ATS**), software that stores, searches, and sometimes scores applications before a human sees them.
2. **Resume screen.** Either the ATS filters by keyword match, or a recruiter (sometimes both) skims for fit: right skills, right seniority, no obvious mismatch.
3. **Recruiter call** (15–20 minutes). Confirms basics: your notice period, your current CTC and expected CTC (as ranges), location, and whether the role genuinely matches what you're looking for. This is a filter, not a technical round; treat it seriously anyway.
4. **One or more technical rounds.** SQL, Python, statistics, a case, a system design conversation, whichever mix suits the role and level (section 68.3).
5. **A hiring-manager round.** The person you'd actually report to, testing fit with how the team works, not just raw skill.
6. **Sometimes a panel or a take-home.** More common at larger companies or for senior roles (Chapter 82 covers take-homes in full).
7. **Offer, then background verification.** Pay, level, and start date are settled here; the expected CTC you gave in the recruiter call is the opening number, not the final one. In Indian hiring, background verification (checks on your past employment and education) often runs after you accept the offer, and reference calls tend to be less common than in some other countries. §8.6 covers how to read salary numbers, and section 68.6 compares pay across roles; use that groundwork now.

Two things to notice about this list. First, **most attrition happens at gates 1 and 2**, long before anyone tests your SQL. A strong technical candidate with a CV that never gets read never gets a chance to prove it. Second, **each gate rewards different things**: gate 1 rewards keyword and format discipline; gate 4 rewards the extra-point moves from Chapter 69; gate 7 rewards knowing your numbers in advance. Prepare for the gate in front of you, not the one three steps away.

68.2 Cracking the resume screen: the software gate, then the human gate

Your CV is read twice, by two completely different readers, and it has to pass both.

How an ATS actually parses a resume

An **applicant tracking system (ATS)** is software that stores every application and, at many companies, does some combination of three things: parses your document into structured fields (name, dates, employer, skills), scores or ranks it against the JD's keywords, and lets a recruiter search and filter the stored pool. The common platforms behave slightly differently, but the failure modes are the same across all of them:

Platform (common in India-facing hiring)	Typical behavior
Workday	Strict parsing; known to struggle with non-standard date formats and merged cells
Greenhouse	Generally strong parsing; still fails on graphics-heavy templates
Lever	Similar to Greenhouse; keyword search is heavily used by recruiters on top of parsing
Taleo (older, still common at large enterprises)	Older parsing engine; generally reported as less forgiving than the other three with tables, columns, and headers/footers
Naukri, LinkedIn, and similar job boards	Not a true ATS, but their own search and "match score" features work on the same keyword logic

This changes; verify before you rely on it. Parsers are updated, and each company configures its ATS differently, so the same platform can behave differently at two employers. The table above is general guidance as reported when this chapter was written (September 2026), not a vendor specification. The self-test below works whatever system a company uses.

What actually breaks parsing, tested the way a recruiter would test it (copy your CV's text out of the PDF and see what comes out):

- **Multi-column layouts.** A parser reading left to right across the page, not down one column then the next, can interleave your job titles with the wrong dates or your skills list with your education.
- **Text inside tables, text boxes, or headers/footers.** Many parsers skip these entirely. A contact phone number in a header, or a skills list in a table, can simply vanish from the parsed version even though it looks fine on screen.
- **Icons and images standing in for words.** A phone icon instead of the word "Phone," or a bar-chart graphic showing "Excel: Advanced," carries no text a parser can read at all.
- **Unusual date formats.** "Jan '22 – Present" parses less reliably than "January 2022 – Present"; when in doubt, spell it out.
- **Non-standard section headings.** "My Journey" instead of "Experience," or "What I Bring" instead of "Skills," can fail to map to the field the ATS expects, which means that whole section may not be scored at all.

A five-minute self-test: open your CV, select all the text, copy it, and paste it into a plain text file. If what comes out is readable, in the right order, with nothing missing, your CV will survive most ATS parsing. If it comes out scrambled, so will the parsed version a recruiter or algorithm actually scores.

Beating the keyword match, honestly

Once parsed, many systems score your CV against the JD's own language before a human ever opens it. Three concrete moves:

- **Mirror the JD's exact terms**, not synonyms, wherever they're true of you. If the JD says "stakeholder management" and you wrote "worked closely with business teams," a keyword match can miss the connection a human would instantly see. Re-read the must-have list (Chapter 8, §8.5) and match its wording directly.
- **Use both the acronym and the full term at least once**, since different systems search for different forms: "Structured Query Language (SQL)" covers a search for either.
- **Repeat your strongest 3–4 keywords naturally across two sections** (say, once in a summary line and once in the relevant bullet), which raises your match score on systems that count frequency, without ever repeating a keyword so often it reads as stuffed.

The human gate, immediately after

Assuming it survives parsing, a recruiter or hiring manager typically spends well under a minute on a first human pass. What they're scanning for, roughly in this order: right role and level, right core skills, recent and relevant experience, no obvious red flags. Structure the page so that fast scan succeeds:

- **Lead with outputs, not duties.** "Built a weekly at-risk customer list that the sales team uses to prioritize calls" beats "Responsible for customer data analysis." Chapter 9's portfolio-piece structure (§9.5: question, data, approach, result, check, decision, next step) is exactly the shape a strong CV bullet compresses into one line.
- **Quantify wherever honestly possible.** Not every number needs to be a business metric; "automated a report that took 3 hours a week" is a real, specific claim a reader can picture.
- **One page for under 5 years' experience**, two at most beyond that. Length signals judgment about what matters, not effort.
- **Projects section, for freshers and career switchers especially.** This is where your portfolio (§9.5) earns its keep on the page itself, not just behind a link.
- **A skills line near the top, not buried at the bottom.** Both readers, software and human, look there first; don't make either one hunt for it.

Watch out: don't lie to the keyword scanner. Padding a CV with skills you can't actually discuss (because you listed them purely to pass a filter) gets caught the moment a live-coding round starts. It costs you the interview you worked to reach, which is a worse outcome than not reaching it at all.

The format rules, in one place

- One column. Standard fonts (Calibri, Arial, or similar; avoid anything decorative). Standard headings: Experience, Education, Skills, Projects.
- No tables, no text boxes, no headers/footers carrying content that matters, no icons standing in for words, no photo (standard for Indian data roles; a photo can also introduce unconscious bias a recruiter would rather avoid).
- Save as the format the posting requests, usually PDF; if none is stated, PDF is the safer default, since it preserves formatting across every reader's device even though a small number of older ATS platforms parse .docx marginally more reliably. If you're applying to a company known to use an older system (word of mouth, or a careers page that looks dated), .docx is worth trying as a fallback if a PDF application seems to disappear without any response.

68.3 What each round actually tests

Match your preparation to the round in front of you, and to the role and level you're targeting from Chapter 7's ten roles.

Round	What it's really testing	How to prepare
Recruiter call	Basic fit, clear communication, realistic expectations	Know your notice period, your current and expected CTC as ranges (§8.6 and section 68.6 help you set the expected range), and a one-minute answer to "tell me about yourself"
Spreadsheet / BI exercise	Accurate, well-built sheets and dashboards, and whether you can explain the numbers	Chapter 70's bank
SQL / live coding	Correctness, process, how you handle being watched while you think	Chapter 71's bank; think out loud, use Chapter 69's moves
Python / pandas live coding	Clean, correct data work, talked through as you go	Chapter 72's bank (and Chapter 72A for algorithm questions)
Case / business	Structured thinking under ambiguity	Chapter 75's cases; always clarify first
Statistics / ML technical	Depth of understanding, not memorized formulas	Chapters 73–74's banks; explain <i>why</i> , not just <i>what</i>
Portfolio / project deep-dive	Whether you did the work you claim, and understand your own choices	Chapter 76A (data analyst and data scientist)
BA case (requirements, process)	Turning a vague request into clear requirements and a workable process	Chapter 76B
Automation design	An automation that is reliable, owned, and safe when it fails	Chapter 78's bank
GenAI / MLOps	Running models and LLM features in production, evaluated honestly	Chapter 79's bank

Round	What it's really testing	How to prepare
System design (senior roles)	Trade-off thinking at scale	Chapter 77's design walk-throughs
Architecture & leadership (senior roles)	Judgment across a whole system, and across the people who run it	Chapter 80's bank, with Chapter 77 for the design half
Take-home assignment	Independent work quality, code you'd actually ship	Chapter 82's worked assignments
Hiring-manager round	Team fit, how you'd actually work day to day	Specific stories, honest questions about the role
Behavioral / HR	Past behavior as a signal for future behavior	Chapter 81's STAR-method bank

Different roles weight these rounds differently. A **data analyst** interview leans on SQL, Excel/Sheets, and a business case; a **data scientist** interview adds statistics and ML depth; a **data engineer** interview adds system design and pipeline debugging; an **architect** interview is almost entirely case and system design, with far less live coding. Check the JD's must-have list (§8.5) to predict which rounds you'll actually face: a posting heavy on "dashboards" and "stakeholder communication" is unlikely to include a deep ML round, whatever the title says.

68.4 LinkedIn hiring: how recruiter search actually works, and how to be found

Most CVs are still submitted through a company's own application page, but an enormous share of data-role hiring, especially at large and mid-sized companies, starts the other way round: a recruiter searches LinkedIn for candidates *before* a single application arrives. Understanding how that search works changes what you actually do with your profile.

How a recruiter finds you

LinkedIn Recruiter (the paid tool most companies' talent teams use, distinct from the free version you browse on) lets a recruiter filter candidates by fields such as job title, skills, location, and company, and apply Boolean logic (AND / OR / NOT, exact phrases in quotes) within those filters, mainly the Job Title, Company, and Keywords fields. A recruiter hunting for a data analyst might, for instance, set the Job Title filter to "data analyst" OR "business analyst" and add "SQL" and "Power BI" as required keywords, filtering out "intern." (LinkedIn has also begun layering AI-driven semantic search on top of this, which can match near-synonyms even without an exact keyword; Boolean keyword matching still does most of the work, and remains the safer thing to design your profile around.)

Three consequences follow directly from how that search works:

- **Your headline and "Skills" section are the highest-weighted fields.** They're what the keyword search scans first and most heavily. A headline of just "Analyst at Company X" is nearly invisible to a search for "SQL" or "Power BI," even if both appear elsewhere on your profile.
- **The exact words matter more than the truth behind them.** A recruiter searching "SQL" will not find a profile that only says "databases," exactly the same lesson as the ATS keyword match in §68.2, now applied to LinkedIn's own search instead of a company's application system.
- **Recency and activity are a filter too.** Recruiters commonly sort by "recently active" or filter to profiles updated in the last few months, on the theory that an actively maintained profile signals an actively job-seeking (or at least reachable) candidate.

Setting up a profile that gets found

- **Headline: role plus your top two or three searchable skills,** not a job title alone. "Data Analyst | SQL · Power BI · Python" is both more searchable and more informative at a glance than "Analyst at Company X."
- **Skills section: list the exact terms from JDs you're targeting,** in the same wording, not paraphrased. This is the single highest-leverage five minutes you can spend on the whole profile.
- **"Open to Work" (recruiter-only visibility mode).** This signals to recruiters that you're open to offers, and it's a filter many recruiters apply directly; leaving it off can mean a search that would have found you never surfaces you at all. LinkedIn tries to hide the signal from recruiters at your current company, but its help pages say it can't guarantee that. If being discovered would cause a problem at work, leave it off.
- **About section: three or four sentences,** written like the first paragraph of a portfolio piece: what you do, one concrete thing you've built, what you're looking for next.
- **Keep it current as you learn.** Chapter 9's advice to retire early portfolio pieces as better ones replace them applies here too: a stale profile from before you started this book undersells you, and (per the point above) a stale, inactive-looking profile is also less likely to surface in a search at all.

Easy Apply, InMail, and what actually gets a reply

- **"Easy Apply" postings go into the same pool as everything else,** not a lesser one, but they also draw far higher volumes; a generic, unmodified profile submitted via Easy Apply competes against hundreds of similar submissions. Treat it exactly as seriously as a company-site application: check your match against the JD (§8.5) before you click.
- **When a recruiter messages you first (InMail), reply within a day or two, and specifically.** A reply that just says "Interested!" wastes the opening; a reply that references one specific duty from their message performs far better ("Thanks for reaching out, the automation piece you mentioned is exactly what I've been building toward").
- **Add a profile photo and complete every section.** LinkedIn's own guidance encourages a photo, and profiles with one are widely reported to get more views and messages. (A CV is different: leave the photo off, for the reasons in §68.2. LinkedIn is a networking profile, where a photo is expected.)

This changes; verify before you rely on it. LinkedIn's search mechanics, filters, and Easy Apply behavior are updated by the platform regularly, and hiring practices shift with them. The mechanics above were checked in September 2026; search LinkedIn's own current Recruiter help pages, or a recent, reputable sourcing guide, before treating any specific detail here as current.

Portfolio, alongside LinkedIn

Chapter 9, §9.5 already covers how to *build* a portfolio piece. Two rules for using it inside the hiring process itself:

- **Link it from both your CV and your LinkedIn profile's "Featured" section**, not just a GitHub profile page. A reader on either surface shouldn't have to hunt.
- **Pick your best two or three pieces for a given application**, not all of them. A recruiter who opens a portfolio wants a quick, relevant look, not an archive; reference the most relevant piece by name in a cover note or application answer where it fits: "In the project linked below, I built an at-risk customer list on a similar problem to what this role's JD describes."

68.5 Referrals, used properly

A referral doesn't skip the technical bar; it skips the resume-screen gate, which, per section 68.1, is where most candidates are lost before anyone tests their skills. That's still a large, legitimate advantage, worth pursuing correctly.

- **Ask for a referral only after you've decoded the JD (§8.5) and know you clear most of the must-haves.** Asking a connection to refer you into a role you're not close to ready for spends their credibility, not just your time.
- **Make it easy for them.** Send your CV, the job link, and two or three sentences on why you're a fit, the same duties-to-outputs mapping §8.5 teaches, so they can forward it in under a minute.
- **Build the relationship before you need it.** A message asking a near-stranger for a referral the same day lands very differently from one to someone whose posts you've engaged with, or who you've had one real conversation with previously.
- **Thank them regardless of outcome**, and tell them what happened. People refer again for candidates who close the loop (Chapter 69's move 12, applied to your own professional relationships).

68.6 Pay: reading the numbers before the offer gate

Chapter 8 (§8.6) showed you how to read one salary row, the data analyst's. In the job search you need a little more. The recruiter call (gate 3) asks for your expected CTC, and the offer (gate 7) hands you a real number to weigh, so it helps to see how the roles compare. Here is the data analyst row again, with two more roles from the same source. **Base salary** and **total pay** mean what they meant in §8.6: fixed yearly pay, and fixed pay plus bonuses and similar extras.

Role (PayScale title)	Salary profiles	Average total pay, under 1 year	Average total pay, 1–4 years	Base salary, 10th–90th percentile	Median base salary
Data Analyst	2,389	₹4,13,462	₹5,65,999	about ₹2,89,000 to ₹10,00,000	about ₹5,77,000
Data Scientist	1,267	₹5,95,255	₹10,05,147	about ₹3,12,000 to ₹20,00,000	about ₹10,00,000
Data Engineer	1,284	₹5,18,398	₹8,10,624	about ₹4,10,000 to ₹20,00,000	about ₹9,75,000

Source: PayScale India job pages for each title. Data Scientist and Data Engineer pages updated 9–14 July 2026, retrieved 16 September 2026; the Data Analyst row is Chapter 8's (page updated 10 July 2026, retrieved September 2026). PayScale rounds its percentile and median figures (it shows "₹1m"), so the last two columns are approximate. For data analysts, PayScale's exact average base salary is ₹5,77,472 (§8.6); its chart marks the same value, rounded to ₹577k, as the median. Figures change often; check the live pages.

In Indian terms, a data scientist's average total pay in years one to four works out to about ₹10.1 lakh, and a data engineer's to about ₹8.1 lakh.

Reading the comparison

The pattern is clearer than the numbers. Roles that need more tiers of skill (science and engineering, in Chapter 8's career tree) show higher medians and faster early growth than the analyst role in this data. The data scientist median base salary is roughly 1.7 times the data analyst median: $₹10,00,000 \div ₹5,77,000 = 1.73$. Both figures are PayScale's rounded medians, which is why "roughly" stays.

Don't read growth as a promise. In Chapter 8's exercise 6 you worked out that average total pay for data analysts with 1–4 years is 36.9% higher than for those under a year: $(₹5,65,999 \div ₹4,13,462 - 1) \times 100 = 36.9\%$. The same sum for data scientists gives $(₹10,05,147 \div ₹5,95,255 - 1) \times 100 = 68.9\%$, and for data engineers $(₹8,10,624 \div ₹5,18,398 - 1) \times 100 = 56.4\%$. But each pair is *two different groups of people*, not the same people a few years later. People who left the field, or never reported their pay, aren't in the data.

Setting your expected CTC. Give the recruiter a range you can defend, not the top of a national table: two sources that use different methods, filtered to your city and experience where the site allows it (§8.6's five steps), and compared on fixed pay, not the CTC headline. Chapter 81 covers the offer conversation itself.

This changes; verify before you rely on it. Salary pages are updated every few months, and the figures above were retrieved in September 2026. Check the live pages, and a second source, before you name a number in a real conversation. This is general information, not financial advice.

68.7 Preparing in 30, 60, or 90 days

How much **runway** you have (the time before you need to be interviewing) changes what "prepared" should mean. Farah Khan, the Riverstone sales executive from Chapters 8 and 9 who's working toward a data analyst role, is a useful case: she scored 5 of 10 on the must-haves in Chapter 8's JD (§8.5): strong on spreadsheets, communication, and CRM experience; gaps in SQL, a BI tool, and a shipped automation. Here's how her plan would differ by runway.

Start with the arithmetic Chapter 9 (§9.1) taught: chapter hours from the Time needed lines, divided by the hours you can really give. By those lines, Farah's three gaps add up to about 104–127 hours of chapter work: Chapters 12 and 13 for SQL (22–26 and 15–20 hours), Chapter 16 for Power BI (22–26 hours), and Chapters 19 and 20 for automation (25–30 and 20–25 hours). Farah has a full-time sales job, so 6 hours a week is realistic, and at that pace the whole list takes $104 \div 6 = 17$ to $127 \div 6 = 21$ weeks.

Runway	Weeks	Hours at 6 a week
90 days	$90 \div 7 = 12.9$	about 77
60 days	$60 \div 7 = 8.6$	about 51
30 days	$30 \div 7 = 4.3$	about 26

None of the three runways covers all 104–127 hours, so every plan below involves some **triage**: deciding which gaps to close fully, which to take only as far as "can demonstrate", and which to name honestly as still open.

90 days: two gaps closed, one started

With about 77 hours, Farah can close two gaps and start the third:

- **SQL fully:** Chapters 12–13, 37–46 hours. It's the must-have that blocks the most roles.
- **One automation she can show:** the Monday email from Chapter 8, built with Chapter 19's Apps Script sections (sections 19.11–19.14, 6–8 hours). Chapter 20's automation is in Python, which needs Chapters 17–18 first, so it waits.
- **Power BI to "can demonstrate":** Chapter 16 up to section 16.4 (installing, getting the data in, the model, and DAX from zero), roughly half the chapter, so about 11–13 hours.

That's 54–67 hours of skills, which leaves roughly 10–23 hours for interview practice. The right order is **skills first, interview practice in the last four weeks**: practicing interview technique on skills you don't have yet wastes the technique.

- **Weeks 1–9:** the Monday email first (Chapter 19 comes before Chapter 12 in this book's reading order), then SQL: 43–54 hours, each gap ending in a portfolio piece (§9.5).
- **Weeks 10–13:** Power BI to "can demonstrate", alongside drilling the extra-point moves with Chapters 69–71 and the Data Analyst & Data Scientist Question Bank (Chapter 76A); add the Business Analyst Question Bank (Chapter 76B) if you're also targeting BA roles. Applications and referral outreach (section 68.5) start in week 10, alongside the drilling. From week 11, two mock interviews a week (Chapter 82).

60 days: one deep gap, plus interview technique

About 51 hours is enough for SQL (37–46 hours) and little else. So Farah closes her single biggest gap properly, builds the Monday email only if she has already started it, and names Power BI honestly in interviews as the gap she's closing next, with the plan to close it. Interview preparation starts around week 7, running alongside the last of the SQL work rather than strictly after it.

30 days: triage, don't spread thin

About 26 hours doesn't cover even both SQL chapters (37–46 hours), let alone three gaps from zero. The honest move is sharper **triage**: pick the one gap that blocks the most roles (usually SQL, per §7.2's "the one skill every role shares"), get it to a defensible level with Chapter 12 (22–26 hours), and lean hard on Chapter 69's method to make existing strengths (spreadsheets, communication, CRM experience, for Farah) score as high as they possibly can. A 30-day plan that tries to touch everything shallowly loses to one that closes a single real gap and interviews confidently on the rest.

Interview extra point. Section 8.5 suggested bringing your *decoded* job description to the interview: when asked "why this role," name two duties from the posting, say what you've built that matches each, and ask one question about the hidden skill ("How are definitions like 'active customer' agreed today?"). Here is why it lands. The hiring manager wrote those duties because they're the work they need done, so matching each one to something you built answers their real question (can you do this job?) in thirty seconds. The hidden-skill question shows you've spotted the part of the job that's hard, which is the "business judgment" dimension of Chapter 69's rubric, applied to the job itself rather than to a technical answer.

68.8 Worked examples, in three groups

Ten examples with full solutions, in three groups: CV bullets and JD scoring (Examples 1–5), referral messages (Examples 6–8), and preparation plans (Examples 9–10). Each group runs from easy to hard, so read a group in order even if you only need one example from it; the reasoning builds.

CV bullets and JD scoring

Example 1 (Easy): CV bullet, a fresher's single duty

Situation: an unpaid college project, one task: cleaning a messy public dataset.

Weak bullet: "Worked on data cleaning for a college project using Python."

Solution: apply 68.2's "output, not duty" rule. What was the question, what was the result, what would a reader picture?

"Cleaned a 40,000-row public transport dataset in pandas: standardized inconsistent city names and resolved missing values, cutting duplicate records from 12% to under 1% so route-level counts could be trusted."

Why it's stronger: a number (40,000 rows, 12% to under 1%) replaces a vague verb ("worked on"), it names the specific problems (inconsistent names, missing values) a reader in the field recognizes immediately, and it ends on the question the cleaning answered: can route-level counts be trusted?

Example 2 (Easy): CV bullet, translating a non-data job

Situation: two years as a store manager, no formal "data" title, moving into a BA or analyst track.

Weak bullet: "Managed daily store operations and staff."

Solution: find the data-shaped work hiding inside a non-data title. A store manager tracks stock, staffing, and sales numbers daily: that's reporting and process work by another name.

"Built a weekly Excel stock-and-sales tracker across 3 store locations, used by regional management to reallocate inventory; cut stockouts by an estimated 15%."

Why it's stronger: this is Example 1's rule applied to a harder case: the underlying achievement existed all along, it just needed translating from "manager" language into "output" language. This is exactly the translation §8.5 teaches for reading a JD, run in reverse, on your own history.

Example 3 (Medium): Scoring a Business Analyst JD

Situation: a posting for "Business Analyst, Operations," at a large company, requiring: process mapping, stakeholder interviews, SQL (basic), Excel (advanced), a ticketing tool (Jira preferred); nice-to-have: BPMN certification, exposure to Power BI.

Candidate: two years as a coordinator, no formal BA title. Self-scores (0/1/2, per §8.5): process mapping 1 (done informally, no training), stakeholder interviews 2 (ran vendor meetings weekly), SQL 0, Excel 2, ticketing tool 1 (used Trello, not Jira), BPMN 0, Power BI 0.

Solution:

Skill	Type	Score	Weight	Points
Process mapping	Must	1	2	2
Stakeholder interviews	Must	2	2	4
SQL (basic)	Must	0	2	0
Excel (advanced)	Must	2	2	4
Ticketing tool	Must	1	2	2
BPMN	Nice	0	1	0
Power BI	Nice	0	1	0
Total				12 / 24 (50%)

Maximum: 5 must-haves × 2 points × 2 = 20, plus 2 nice-to-haves × 2 points × 1 = 4, so 24. Must-have coverage: $(1+2+0+2+1) / 10 = 60\%$.

Reading it. Worth applying: 60% must-have coverage with one clean gap (SQL) and one near-miss (Trello, not Jira, which is usually close enough to mention honestly and offer to learn). The two zero-scores on nice-to-haves don't matter; §8.5 is explicit that nice-to-haves are tie-breakers, not

requirements.

Example 4 (Medium): Scoring a Data Scientist JD with an inflated tool list

Situation: "Data Scientist," must-haves: Python, statistics, SQL, "machine learning," and, unusually for a mid-sized company's posting, TensorFlow, Docker, Kubernetes, and Airflow all listed as required, not nice-to-have.

Solution: before scoring, apply §8.5's red-flag check: "a tool list longer than the duties" for a role whose actual duties (read further down the posting) are "build models to predict customer churn and present results to stakeholders." First, sort the tool list. TensorFlow is a machine-learning library, so it belongs with the core skills (and deep learning is rarely needed to predict churn). Docker, Kubernetes, and Airflow are infrastructure: tools for packaging, running, and scheduling software. Three infrastructure tools as hard requirements for a role that's really about modeling and communication is a mismatch, likely a copy-pasted posting rather than a true requirement list.

Practical scoring move: score the two groups separately. Core (Python, statistics, SQL, ML, with TensorFlow as a nice-to-have in practice): score normally. Infrastructure tools (Docker, Kubernetes, Airflow): treat as a soft signal, not a hard gate, and raise it directly in the recruiter call ("the posting lists Kubernetes as required: is that truly day-one, or does the team support ramping up on it?"). This is Move 1 from Chapter 69 (clarify first), applied to the application stage instead of an interview answer.

Reading it. A rigid score against the literal JD would wrongly disqualify a strong-fit candidate. The skill here isn't scoring faster; it's recognizing when the JD itself needs a clarifying question before the score means anything.

Example 5 (Hard): Scoring for a senior or architect role, with judgment calls

Situation: "Data Architect," 8+ years, must-haves include "proven experience designing systems for 10M+ daily events," "stakeholder alignment across engineering and business," and a named cloud platform.

Candidate: 6 years' experience, designed a system handling roughly 2M daily events, strong stakeholder track record, experience on a different cloud platform than the one named.

Solution: senior JDs reward honest judgment over literal score-counting. Three specific calls:

1. **"8+ years" against 6.** Chapter 8's own guidance treats years as a guide, not a hard wall, especially when depth compensates for tenure; score it 1, not 0, and address the gap directly if asked, rather than self-disqualifying before applying.
2. **"10M+ daily events" against 2M.** This is a genuine, material gap for an architect role specifically about scale; score it honestly at 0–1, and prepare a system-design answer (Chapter 77) that shows you understand *how* the design would change at 5x the scale, even without having run it.
3. **A different cloud platform.** Score close to full credit; core architectural thinking (Chapter 77's design walk-throughs) transfers across platforms far more than junior-level tool skills do, and most architect interviews test the thinking, not platform trivia.

Reading it. At senior levels, the scoring method from §8.5 still applies, but its main value shifts from "should I apply" (almost always yes, if two of three must-haves are close) to "which specific gap do I need a strong, rehearsed answer for in the interview."

Referral messages

Example 6 (Easy): Referral message, a warm connection

Situation: a former teammate now works at the target company; you've spoken in the last month.

Solution:

"Hi Ishaan, hope the new role's going well! I saw Northwind Distributors is hiring a Data Analyst (link) and it's a strong match for what I've been building toward, especially the automation piece. I've attached my CV, would you be open to referring me, or pointing me to whoever's hiring for it?"

Why it works: short, names the specific role, ties one concrete strength to one concrete duty (per 68.5), and asks a clear, low-effort question.

Example 7 (Medium): Referral message, a cold LinkedIn connection

Situation: no prior relationship; connecting with someone at the target company specifically to ask for a referral.

Weak version: "Hi, I saw you work at X. I'm looking for a job, could you refer me?" (No context, asks a stranger for a favor in the first message.)

Solution: two messages, not one.

Message 1 (connect, no ask): "Hi Tanvi, I've been following Meridian Logistics' work in [specific area] and would love to connect as I explore roles in data analytics."

Message 2, a few days later, only after they accept: "Thanks for connecting! I noticed the Data Analyst opening on your team (link). I've spent the last few months building SQL and Power BI skills specifically around sales analytics, including a project on at-risk customer scoring. Would you be open to a referral, or a quick pointer to who's hiring for it?"

Why it works: it earns the ask across two messages instead of spending a stranger's goodwill in the first line, exactly 68.5's "build the relationship before you need it" rule, compressed into the shortest version that still respects it.

Example 8 (Hard): Re-approaching after a past rejection, for a different role

Situation: rejected for a Data Analyst role at a company six months ago; a new Analytics Engineer posting has opened, and you've closed your dbt/SQL gap since then.

Solution:

"Hi Nikhil, we spoke a few months back about the Data Analyst role. I understand the fit wasn't quite right then. Since then I've focused specifically on SQL and dbt (project link), and I noticed the Analytics Engineer opening, which maps closely to that work. Would it be worth a quick conversation, or should I apply directly?"

Why it works: it names the past interaction honestly instead of pretending it didn't happen (which the interviewer will remember anyway), states exactly what changed, and offers the reader an easy way to redirect you if the timing still isn't right. This is Chapter 69's Move 10 (admit limits honestly) applied to your own hiring history, not a technical answer.

Preparation plans

Example 9 (Medium): A 90-day plan for a true fresher

Situation: unlike Farah Khan (section 68.7), a recent graduate with coursework but no work-relevant portfolio at all, and no existing professional network to draw referrals from.

Solution: the shape from section 68.7 still applies, but week 1 needs an extra step Farah didn't: choosing *which* track (Chapter 7, §7.2) to aim at, since a true fresher hasn't narrowed it the way someone already working toward a specific role has.

- **Week 1:** pick one track using Chapter 7's map; resist the urge to prepare for all ten roles at once.
- **Weeks 2–9:** the same skills-first shape as Farah's, but built from zero rather than from an existing job's worth of transferable experience. Do the hours check first. For a data analyst, Chapters 10–16 and 19 add up to about 169–209 hours by their Time needed lines. A fresher studying full-time (say 25 hours a week) has about $8 \times 25 = 200$ hours in these eight weeks: just enough, with no slack, and Chapter 20's Python automation (after Chapters 17–18) waits. Part-time, at 6 hours a week, the same eight weeks give about 48 hours, so cut the scope to SQL (Chapters 12–13, 37–46 hours) and say honestly what comes next.
- **Weeks 8–10 (overlapping, since there's no existing network to lean on early):** LinkedIn and portfolio go live in week 8, not held until "ready," since a fresher's biggest gate-1 risk is having nothing to link at all; referral outreach starts cold (Example 7's two-message pattern) rather than warm, since Example 6's shortcut isn't available yet.
- **Weeks 11–13:** drilling with Chapters 69–71 and Chapter 76A, two mock interviews a week (Chapter 82), and applications.

Reading it. The plan's shape doesn't change with experience level; what changes is which weeks get compressed and which need the extra step this example adds.

Example 10 (Hard): A 60-day plan for a lateral move, not a beginner

Situation: an experienced Business Analyst (4 years) moving toward Data Scientist, with real SQL and stakeholder skills already, but no ML background at all.

Solution: this is a harder planning problem than either Farah's or Example 9's, because the candidate isn't weak, they're **misfiled**: strong on one track's skills, applying to a different track's roles. Two adjustments to section 68.7's method:

1. **Don't re-learn what already transfers.** SQL and stakeholder communication (Chapter 7's BA row) map almost directly onto a DS role's needs; scoring them at 0 out of false modesty wastes runway that should go to the real gap.
2. **The real gap (statistics and ML, Chapters 21–22, 30, and 35–39) is deep, not shallow**, so 60 days is tight. The honest move, per section 68.7's 60-day guidance, is triage within the gap itself. First statistics and experiments, Chapters 21–22 and 30, the most interviewed statistics topics (Chapter 73's bank tests them heavily). Then the ML path in the order each chapter builds on the one before: Chapter 35 (the math), 36 (the ML workflow), 37 (the algorithms), and 39 (honest evaluation, which evaluates Chapter 37's models). Chapter 38 and the deep-learning material (Chapter 43) wait: they're tested far less often at this level and can be learned after an offer, not before it. Then the hours check. By their Time needed lines, those seven chapters are about 122–147 hours, which over 60 days (8.6 weeks) is $122 \div 8.6 = 14$ to $147 \div 8.6 = 17$ hours a week. That assumes Python (Chapters 17–18) is already in place. At 10–12 hours a week, 60 days reaches the end of Chapter 36 (86–103 hours), and Chapters 37 and 39 go into a longer runway, named honestly in interviews as the next step.

Reading it. A lateral move from an adjacent track is not "starting over"; treating it that way wastes the runway's most valuable resource, which is the skills you don't have to rebuild.

68.9 Four resumes, matched to their JDs

Four complete resumes, each paired with a condensed job description it was actually built to clear, across four different roles and levels. Read each JD first, then the resume, then the annotation, which points out exactly why the resume clears both the software gate and the human gate from §68.2.

Some terms in the data science and data engineering resumes (AUC, SHAP, calibration, Kafka, Great Expectations, and others) come from Parts 4 and 5. If you took a route through the book that skipped those parts, you don't need them to follow this section: look at how each bullet is built, not at the tools.

68.9.1 Fresher, Data Analyst

JD excerpt: Data Analyst (Fresher), TrueNorth Retail Pvt. Ltd.

0–1 years' experience; freshers with strong projects considered. Must have: SQL (joins, aggregation), Excel or Google Sheets (pivot tables, lookups), basic data visualization. Nice to have: Python, a BI tool.

Resume

Aditi Sharma Pune, Maharashtra · aditi.sharma.data@email.com · +91-98XXXXXXX1 · linkedin.com/in/aditisharma-data · github.com/aditisharma-data

Summary Data Analyst fresher with hands-on SQL, Excel, and Power BI project experience on realistic business datasets. Comfortable turning a business question into a clean, checked answer.

Skills SQL (joins, aggregation, window functions) · Excel and Google Sheets (pivot tables, lookups) · Power BI (data visualization, dashboards) · Python (pandas, basic)

Projects

- **At-risk customer list, retail dataset (SQL, Power BI).** Built a query identifying customers with no order in 90+ days, ranked by revenue; visualized in a Power BI dashboard used to prioritize sales follow-up. Validated totals reconciled to source data before presenting.
- **Sales data cleaning and analysis (Python, pandas).** Cleaned a 12,000-row dataset (missing values, duplicate records), then analyzed monthly revenue trends by category; found and flagged a data entry error affecting 3% of rows.
- **Automated weekly report (Google Sheets, Apps Script).** Replaced a 45-minute manual weekly task with a script triggered automatically every Monday morning.

Education B.Com, Savitribai Phule Pune University, 2022–2025

Certifications Google Data Analytics Professional Certificate, 2025

Why this clears both gates. Software gate: single column, standard headings, every must-have keyword from the JD (SQL, Excel, pivot tables, data visualization) appears in the Skills line in the JD's own words. Human gate: no prior job title to lean on, so every project bullet follows §68.2's "output, not duty" rule with a number attached, and the project section does the work a fresher's experience section can't.

68.9.2 Career switcher, Business Analyst

JD excerpt: Business Analyst, Operations, Meridian Logistics

2+ years in any operations, coordination, or analyst role. Must have: process mapping, stakeholder communication, SQL (basic), Excel (advanced). Nice to have: Jira or similar ticketing tool, BPMN exposure.

Resume

Rohan Verma Bengaluru, Karnataka · rohan.verma.ba@email.com · +91-98XXXXXXX2 · linkedin.com/in/rohanverma-ba

Summary Operations professional moving into business analysis, with 4 years managing supplier coordination and reporting for a multi-location retail operation. Strong stakeholder communication; building SQL and process-mapping skills alongside hands-on project work.

Skills Process mapping · Stakeholder communication · Excel (advanced: pivot tables, lookups, what-if analysis) · SQL (joins, aggregation) · Trello (ticketing)

Experience

Operations Coordinator, Bluepeak Retail Pvt. Ltd. | Bengaluru | 2022–Present

- Built a weekly Excel stock-and-sales tracker across 3 store locations, used by regional management to reallocate inventory; contributed to an estimated 15% reduction in stockouts.
- Ran weekly vendor coordination meetings across 8 suppliers, resolving delivery and quality issues; documented recurring issues into a process map that cut average resolution time by 2 days.
- Coordinated a system migration between two teams, translating operational needs into requirements for the technical team.

Projects

- **Requirements and process map, order fulfillment workflow (SQL, process mapping).** Documented the current order-to-delivery process end to end, identified 3 redundant approval steps, and wrote a query analyzing where orders were delayed most often.

Education BBA, Christ University, 2018–2021

Certifications ECBA (IIBA Entry Certificate in Business Analysis), in progress, 2026

Why this clears both gates. Software gate: "process mapping" and "stakeholder communication," the JD's own phrases, appear directly rather than as the paraphrase ("worked closely with business teams") a switcher's first instinct often produces. Human gate: the Experience section translates operations duties into outputs using exactly Example 2's method from §68.8, and the one Projects entry signals BA-specific intent (a formal process map and a query) beyond what the day job alone shows. The certification line is one a switcher can honestly hold: ECBA is the International Institute of Business Analysis's (IIBA's) entry-level certificate, open to people without business analysis work experience, while its senior certification, CBAP, requires years of it. Check IIBA's current requirements before you list either.

68.9.3 Mid-level, Data Scientist

JD excerpt: Data Scientist, 3–5 years, Northbridge Analytics

Must have: Python, statistics and hypothesis testing, SQL, machine learning (supervised learning, model evaluation). Nice to have: A/B testing experience, cloud platform exposure (any).

Resume

Kavya Nair Hyderabad, Telangana · kavya.nair.ds@email.com · +91-98XXXXXXX3 ·
linkedin.com/in/kavyanair-ds · github.com/kavyanair-ds

Summary Data Scientist with 5 years in data, 3 of them building and evaluating models for customer retention and demand forecasting, from business question through to a monitored result. Strong on honest evaluation, not just model accuracy.

Skills Python (pandas, scikit-learn, statsmodels) · SQL · Statistics and hypothesis testing · Machine learning (supervised learning: logistic regression, gradient boosting; model evaluation and calibration) · A/B testing · AWS (S3, basic)

Experience **Data Scientist, Solvix Technologies** | Hyderabad | 2023–Present

- Built a churn prediction model (gradient boosting) reaching 0.79 AUC, evaluated against a cost-based threshold rather than accuracy alone; the resulting call list protected an estimated ₹11 lakh more revenue than a probability-only ranking over 6 months.
- Designed and analyzed an A/B test on a pricing change, correcting a peeking problem (stopping the test as soon as it looked significant) that the initial design had missed before launch.
- Explained model outputs to non-technical stakeholders using SHAP-based summaries, translating "why did this account score high risk" into specific, actionable account notes.

Data Analyst, Kestrel Retail | Hyderabad | 2021–2023

- Built the SQL and Python pipeline behind a weekly demand forecast, cutting stockouts by 18% over the following two quarters.

Projects

- **Open-source demand forecasting comparison (Python, statsmodels, Prophet).** Backtested five forecasting methods on multi-year retail data; found the simplest method (seasonal naive) beat two more complex ones on the noisiest series, and wrote up why.

Education B.Tech, Computer Science, VNIT Nagpur, 2017–2021

Certifications None currently listed (deliberately: at this level, shipped project results carry more weight than certificates, and the space is better spent on them)

Why this clears both gates. Software gate: every must-have term (statistics, hypothesis testing, SQL, machine learning, supervised learning, model evaluation) appears in the Skills line in the JD's own words, with A/B testing and a cloud platform (the nice-to-haves) also present rather than omitted.

Human gate: every experience bullet and the project compress Chapter 9's seven-part portfolio shape (§9.5) into one line: the question, the approach, the result, and a check or caveat. That is exactly what separates a mid-level DS resume that gets a callback from one that reads as a list of tool names.

68.9.4 Mid-level, Data Engineer

JD excerpt: Data Engineer, 3+ years, Fernhill Systems

Must have: SQL, Python, pipeline orchestration (Airflow or similar), cloud data warehouse (any major platform). Nice to have: dbt, streaming/real-time experience, data quality frameworks.

Resume

Arjun Reddy Chennai, Tamil Nadu · arjun.reddy.de@email.com · +91-98XXXXXXX4 · linkedin.com/in/arjunreddy-de · github.com/arjunreddy-de

Summary Data Engineer with 3 years building and maintaining pipelines feeding a company's core reporting and ML systems, with a focus on pipelines that fail loudly and get fixed fast, not silently.

Skills SQL · Python · Pipeline orchestration (Apache Airflow) · Cloud data warehouse (Snowflake) · dbt · Data quality frameworks (Great Expectations) · Basic streaming (Kafka, exposure)

Experience Data Engineer, Vantara Systems | Chennai | 2023–Present

- Rebuilt a daily sales pipeline (Airflow, Snowflake, dbt) that previously failed silently roughly once a month; added automated data-quality checks and alerting, cutting undetected failures to zero over the following year.
- Migrated 40+ legacy SQL reports into tested dbt models with documentation, reducing a recurring "which number is right" dispute between two teams to effectively none.
- On-call rotation for pipeline incidents; wrote the team's first incident runbook after a recurring late-data issue, cutting average resolution time from 3 hours to 40 minutes.

Data Analyst → Junior Data Engineer, Kestrel Retail | Chennai | 2021–2023

- Started as a data analyst; moved into engineering after automating a reporting pipeline that then became the team's standard approach for two other reports.

Projects

- **Personal streaming pipeline (Python, Kafka, Docker).** Built a small real-time pipeline ingesting a public API, to learn streaming concepts not used day-to-day at work; documented the design trade-offs against the batch approach used at the job.

Education B.E., Information Technology, Anna University, 2017–2021

Why this clears both gates. Software gate: the must-have list (SQL, Python, pipeline orchestration, a cloud data warehouse) and two nice-to-haves (dbt, data quality frameworks) all appear in the exact JD wording. The tool names (Airflow, Snowflake) match most searches, but adding the JD's generic

phrase too covers a recruiter who searches the category instead. Human gate: the bullets emphasize reliability and incident response, not just pipelines built, which is precisely what a data engineering hiring manager's fast scan is checking for beyond the tool list, and the one side project shows initiative on a specific named gap (real-time/streaming) rather than a generic "I like building things" claim.

APPLY

Common mistakes

Mistake	Symptom	Fix
A CV formatted for humans, not software	Never hears back, even from roles you're qualified for	Standard single-column format; match the JD's exact keywords
Preparing only for technical rounds	Never reaches them, because the CV or recruiter call fails first	Prepare gate 1 and gate 3 as seriously as gate 4
A portfolio with no write-up	Reviewer can't tell what you actually did or decided	Use §9.5's seven-part structure; lead with the question and the decision
Asking for a referral cold, underqualified	Damages the relationship, rarely works anyway	Decode the JD first; only ask once you clear most must-haves
A 30-day plan that tries to cover everything	Shallow on every skill, confident on none	Triage to the one gap that blocks the most roles
Treating the recruiter call as a formality	Loses the role on basics before any technical round	Know your numbers and your one-minute pitch cold

In the real world: what Farah actually did

Farah stayed on the internal route from Chapter 8. Her must-have coverage there was 50%: strong on spreadsheets, communication, and CRM experience; gaps in SQL, a BI tool, and a shipped automation. Anita's posting was still open, and Anita would interview outside candidates on the same must-haves, so Farah prepared exactly as an outside candidate would: the same CV rules, the same gates, no shortcuts.

She didn't apply immediately. She spent twelve weeks on SQL (Chapter 9 tells that story, plateau included), and built her Monday email along the way (Chapter 8): six weeks after her talk with Anita, it was listing, every Monday, the customers who had gone quiet. Two portfolio pieces came out of those weeks: the Monday email itself, and her question about which hospitality customers had ordered before last year's wedding season but not this year, written up on one page (§9.5). She updated her CV and her LinkedIn headline the same week she finished the SQL chapters, not the week she felt "ready," on the theory that recruiters search continuously and a stale profile costs her nothing to fix early.

At week twelve, before she had started Power BI, she re-scored her must-haves the same honest way: SQL 2, spreadsheets 2, a BI tool 0, an automated report 2, written communication 2. That's $2 + 2 + 0 + 2 + 2 = 8$ of 10, or 80%. Power BI came next.

When she applied for Anita's role, there was no referral to ask for: she was already inside. Instead, Meera, who had seen Farah's Friday-afternoon work on the at-risk list, told Anita what Farah had built. That got Farah a seat at the table, but the technical round still tested exactly what it tested for the outside candidates, and she still needed Chapter 69's moves to turn a correct SQL answer into a strong one live. Meera's word bought her a seat at the table. It didn't do the sitting for her.

Project: your own gate-by-gate plan

Goal: a written plan you'd actually follow, not a vague intention to "get ready."

Tools you'll need

No software beyond what's already in your hands: the JD-scoring method from §8.5, the portfolio structure from §9.5, and whichever ATS your target companies use (worth a quick search for "how [Company]'s application system works," since a few, Workday and similar systems especially, have known formatting quirks worth checking before you apply).

Steps:

1. Find one real job posting for a role you want and score your fit against it using §8.5's method.
2. Identify your one or two biggest gaps, and your two or three strongest existing matches.
3. Decide your realistic runway: 30, 60, or 90 days, honestly.
4. Write a week-by-week plan using this chapter's shape for that runway, naming which chapters of this book close which gap. Do the hours check from section 68.7 first: add up those chapters' Time needed lines and divide by the hours you can really give each week.
5. Update your CV and LinkedIn headline now, this week, against the JD's actual language, whether or not you feel ready to apply yet.

REVIEW

Recap

- Hiring is a **sequence of gates**, each judged differently; most candidates are lost at the CV and recruiter-call gates, long before any technical round.
- A CV has to pass **software first, a human second**; format and keyword-matching decide whether a human ever sees it.
- **Different rounds test different things**, and different roles weight rounds differently; predict your rounds from the JD's must-have list.
- A **portfolio** (§9.5) and a current **LinkedIn profile** do real work in the process, not just decoration.
- A **referral** skips the resume screen, not the technical bar; ask only once you've honestly scored yourself against the JD.
- **Pay figures compare groups, not promises**. Set your expected CTC as a range from two sources, for your city and experience, before the recruiter call asks for it.

- **30/60/90-day plans** aren't the same plan run for different lengths of time: shorter runway means real triage to the one gap that blocks the most roles, not shallow coverage of everything.

Key terms

applicant tracking system (ATS) · resume screen · keyword match · recruiter call · expected CTC · hiring-manager round · take-home assignment · background verification · referral · must-have coverage · portfolio piece · runway · triage (in preparation)

Check yourself

- ☐ I can name every gate in the process, in order, and what each one actually tests.
- ☐ My CV is in a format that survives an ATS, using the target JD's own keywords honestly.
- ☐ I know which interview rounds to expect for my target role, and I'm not spending equal effort on all of them.
- ☐ My portfolio and LinkedIn profile are current, linked from my CV, and structured around outputs, not duties.
- ☐ I only ask for a referral once I've honestly scored myself against the JD.
- ☐ I can give an expected CTC range and say where it comes from.
- ☐ I have a written, runway-appropriate plan, checked against the chapters' hours, not just a study list.

PRACTISE

Exercises

1. Find a real job posting for a role on your track (Chapter 7, §7.2) and score your must-have coverage using §8.5's method.
2. List the three interview rounds you'd most likely face for that posting, and say why, based on its must-have list.
3. Write your CV's top bullet for your strongest project using the "output, not duty" rule from 68.2.
4. Draft the two-sentence referral request you'd actually send for that posting, once you clear most of its must-haves (section 68.5).
5. Write your own 30-, 60-, and 90-day plan for the same posting, and say which one matches your real, honest runway.
6. Using the table in section 68.6, calculate how much higher average total pay is for data engineers with 1–4 years than for those under a year, as a percentage. Then give one reason why this isn't the pay rise a new data engineer should expect after a few years.

Answers

Exercises 1, 2, and 5 are about your own posting and your own runway, so there's no single answer. Check yours against these lists.

1. You listed every must-have from the posting, scored each 0, 1, or 2 using §8.5's definitions (2 only for work done on real or realistic data without help), and computed must-have coverage as the sum of must-have scores divided by twice the number of must-haves. Nice-to-haves are scored but kept out of the coverage figure.
2. Each round you named is tied to a specific line of the posting (for example, "Power BI preferred" → a BI exercise, Chapter 70; "SQL: joins and aggregation" → a live SQL round, Chapter 71), and you've said which round you expect to be hardest for you. A list of rounds with no reason from the posting doesn't count.
3. A sample answer, for a fresher's SQL project: *"Built a SQL query listing customers with no order in 90+ days, ranked by last year's revenue, so the sales team could call the biggest accounts first; checked the list's totals against the source table before sharing it."* It starts with a verb that names an output (built), says what question it answered (who has gone quiet), gives a number (90+ days), says who used it and why, and ends with a check. "Responsible for SQL analysis of customer data" is the duty version to avoid.
4. A sample answer: *"Hi Ishaan, I saw the Data Analyst opening on your team (link); I've built an at-risk customer list in SQL and a weekly automated report, which match its first two duties. Would you be open to referring me? My CV is attached."* Two sentences, the specific role, one concrete match between your work and a duty, and a clear, easy question. If you're not yet clearing most of the must-haves, the right answer is not to send it yet.
5. Your plan starts from the hours check (section 68.7): the chapters that close your gaps, their Time needed lines added up, divided by the hours a week you really have. The runway you chose is the one those hours fit; if none fits, the plan triages (one gap closed fully, others to "can demonstrate" or named as next). Interview practice (Chapters 69–82) sits in the last weeks, overlapping the last skill, not after everything.
6. $(₹8,10,624 \div ₹5,18,398 - 1) \times 100 = 56.4\%$ higher. It isn't a promised rise because the two figures come from **different groups of people**, not the same people over time. The 1–4 year group also includes only people still in the role who chose to report, and pay depends heavily on city, company, and skills. Any one of these is a good reason.

Where this leads

- **Chapter 69, The Extra-Points Method**, is what you use once you're through the gates and into a live round.
- **Chapters 70 to 80, including 72A, 76A, and 76B**, are the question banks for each specific round this chapter maps out (section 68.3's table says which bank goes with which round).
- **Chapter 81, Behavioral, HR & Offer Conversations**, picks up exactly where this chapter's gate 7 leaves off: the offer conversation itself.
- **Chapter 82, Take-Home Assignments & Mock Interviews**, is where the 90-day plan's final weeks, and this chapter's whole gate sequence, get rehearsed end to end.

Chapter 69. The Extra-Points Method

Chapter at a glance

You will learn to: understand the five dimensions that interview rubrics score, whatever the question · give a "strong" answer instead of a merely correct one, and know the difference · use twelve specific moves that turn a strong answer into an outstanding one, each shown before and after on a short example · carry a question through all three answer tiers yourself, live, under time pressure · recognize the red flags and over-corrections that cost points even when the technical content is right.

Before you start: the method needs nothing technical: this chapter is about *how* you answer, not what you know. The exercises draw on Chapters 4 and 12–51; each one names where its content is taught, so skip any you haven't studied. The method works alongside whichever question bank you use next (Chapters 70–82).

Time needed: about 1½ hours to read; 4–6 hours to drill all twenty exercises (or 1–2 hours for the five nearest your role); return to it before every interview.

Who this is for: every role this book covers, from a first analyst screening call to an architect-level system design round.

In this chapter

START	Why this matters · In plain English	27
LEARN	69.1 How interviewers actually score	28
	69.2 The shape of a strong answer	29
	69.3 The twelve extra-point moves	30
	69.4 One question, all three tiers	34
	69.5 The same moves, under different rounds	35
	69.6 What not to do	35
APPLY	Common mistakes · In the real world · Project	36
REVIEW	Recap · Key terms · Check yourself	37
PRACTISE	Exercises	38
	Answers	39
NEXT	Where this leads	44

START

Why this matters

Two candidates are asked the same SQL question. Both write a query that returns the right rows. One gets an offer; the other doesn't. The difference wasn't correctness: it was *everything around* the correct answer, whether they checked what "right" meant before diving in, whether they mentioned the one edge case that breaks their own query, whether they tied it back to something a business would actually do with the result.

Interviewers rarely score on a pass/fail switch. They score on a spread, usually without writing it down anywhere you'll see, and the gap between "we can move this person forward" and "we're not sure" is almost never about knowing more facts. It's about a small set of habits, repeated consistently, that this chapter teaches directly rather than leaving you to notice by accident after your fifth rejected interview.

In plain English

Think of two students answering the same exam question: "What causes seasons?"

One writes: "The Earth's tilt." Correct. One sentence.

The other writes: "The Earth's axis is tilted about 23.5 degrees, so as it orbits the sun, each hemisphere leans toward the sun for part of the year and away for the rest, which is what causes summer and winter (not distance from the sun, a common misconception worth ruling out). Here's a quick sketch showing why the northern and southern hemispheres are always opposite."

Both are "correct." Only one shows the examiner that the student actually understands the topic, anticipated the obvious wrong turn, and could teach it to someone else. Interviews reward the second student, every time, and the extra sentences aren't padding: each one is doing a specific job. This chapter is a catalog of those jobs.

LEARN

69.1 How interviewers actually score

Across data roles (analyst, BA, BI, analytics engineer, data scientist, data engineer, ML engineer, architect), companies' written rubrics differ, but most score some version of these five dimensions. This book uses this table as its rubric:

Dimension	1 (Weak)	2 (Acceptable)	3 (Strong)	4 (Outstanding)
Correctness	Wrong or incomplete	Correct for the simple case	Correct, including common edge cases	Correct, and explains <i>why</i> alternatives fail
Structure & communication	Rambling, jumps around	Understandable	Clear steps, signposted	Simple first, then depth; easy to follow for any audience
Depth & edge cases	None considered	Mentions one when prompted	Raises the important ones unprompted	Tests them, and explains the risk each creates
Business judgment	Purely technical	Mentions the business	Ties the answer to a decision or metric	Quantifies impact; proposes next actions
Collaboration	Doesn't clarify; defensive with hints	Accepts hints	Asks good clarifying questions	Checks assumptions, adapts, summarizes agreement

Simplification. Real rubrics often end in a plain "hire / no hire" with written notes. The five dimensions are what those notes are about.

In this book an answer that **passes** scores mostly 2s, a **strong** answer mostly 3s, and an **extra-points** answer mostly 4s; a 1 is a fail and gets no tier. Every core question in the banks (Chapters 70–81) is written against this table in those three tiers; the rapid-fire tables give a one-line answer plus one extra point, and Chapter 82 scores whole mock interviews on the same five dimensions. When a bank labels an answer "strong" or "extra-points," it means the answer scores mostly 3s or mostly 4s on this rubric, not that it uses fancier words.

Notice that four of the five dimensions aren't about how much you know. Correctness is assumed to be necessary; the other four are what separate candidates who are all correct. The rubric measures what you do *with* what you know, which is exactly the part a candidate can improve in a week, unlike years of experience.

Interview extra point. If you only remember one thing from this chapter, make it this: **every dimension above except correctness can be practiced on a question you already know the answer to.** You don't need harder questions to get better at this. You need to notice, on questions you can already solve, that you're stopping at "correct" instead of continuing to "outstanding."

69.2 The shape of a strong answer

Before the twelve individual moves, one overall shape matters more than any of them: **simple first, depth second.**

Give the one-line answer immediately, the thing a manager skimming a transcript would want to read. Then, and only then, go deeper. Reversed, this fails badly: an interviewer who has to wait ninety seconds through background and caveats before hearing your actual answer starts wondering whether you *have* one.

Weak shape: "So there are a few ways to think about this, and it depends on the data, but generally speaking when we're dealing with duplicates there's a question of what counts as a duplicate in the first place, and... okay, so, to find duplicate customers I'd probably group by the fields that should be unique and then filter for count greater than one."

Strong shape: "I'd group by the fields that should be unique and filter for a count above one. [pause] The one thing that changes the answer is what 'duplicate' means here: same email, or same name and phone, or something fuzzier like similar names with typos. Which is Riverstone worried about?"

Same technical content. The second version leads with the answer, then earns its follow-up sentence by asking something specific. That's the shape every example in this chapter follows, and it's worth drilling on its own, separately from the twelve moves below: **answer first, in one sentence, every time**, even when you're about to say more.

69.3 The twelve extra-point moves

Each move below gets the same three lines on a realistic Riverstone-style question: the question, an answer without the move, and the same answer with it, so you see the exact sentence that earns the point, not just the name of the technique. Where an example leans on something technical, the chapter that teaches it is named in brackets. Practice each on a question of your own before moving to the next.

Move 1: Clarify first

Restate the question and ask the one or two questions that actually change the answer. Not every question needs a clarification; asking one when nothing hinges on it wastes time and reads as stalling.

Question: "Write a query to find our top customers."

Without the move: Starts writing a query ranking by total order count.

With the move: "Top by what: revenue, order count, or profit margin? And over what period? I'll assume revenue, all-time, unless you tell me otherwise." (*Then proceeds, having lost nothing.*)

Move 2: State assumptions out loud when you can't ask

In take-home assignments and some live-coding platforms, nobody's there to answer a clarifying question. Say what you assumed and why, in one line, rather than silently picking one interpretation.

Question (take-home): "Calculate Riverstone's 2025 revenue."

Without the move: Adds up every order and reports ₹43,98,121, silently counting the two cancelled orders.

With the move: "I'm treating a cancelled order as not a sale, since that matches how Riverstone's reports define revenue. That gives ₹43,35,471; counting the two cancelled orders (₹62,650) would give ₹43,98,121. I've noted this assumption in a comment in case that's wrong."

Move 3: Signpost your structure

Before diving in, say the shape of what's coming. This costs three seconds and saves the interviewer from wondering whether you have a plan.

Question: "How would you find churned customers?"

Without the move: Starts writing a query at once, and the interviewer has to guess where it's heading and whether "churned" means what they mean.

With the move: "I'll cover three things: how I'd define 'churned' here, the query itself, and one edge case that trips people up."

Move 4: Simple answer first, depth second

Covered in full in section 69.2. The most common single mistake candidates make is skipping this and going straight to the caveats.

Question: "How would you find duplicate customers?"

Without the move: Two sentences about what might count as a duplicate before any answer at all (section 69.2's weak shape).

With the move: "I'd group by the fields that should be unique and keep the groups with a count above one (Chapter 12, section 12.9). The thing that changes the answer is what 'duplicate' means here..."

Move 5: Raise edge cases

The recurring list, worth memorizing: **NULLs, duplicates, ties, empty inputs, time zones, outliers, late-arriving data**. You don't need to check all seven every time; you need to scan the list and mention the one or two that actually apply.

Question: "How would you calculate month-over-month growth?"

Without the move: Gives the formula: $(\text{this month} - \text{last month}) \div \text{last month} \times 100$ (Chapter 4, section 4.1; in SQL, Chapter 13, section 13.6).

With the move: Gives the formula, then: "One thing that breaks this: if last month was zero (a brand-new product line, say), the percentage is undefined or misleadingly huge. I'd flag those rows separately rather than let a division by zero either crash the report or silently show 'infinity%'."

Move 6: Offer trade-offs and alternatives

Name a second approach and say why you picked the one you did. This shows you're choosing, not guessing.

Question: "Find each customer's latest order."

Without the move: Writes a correlated subquery (Chapter 12, section 12.12) and stops.

With the move: "I could do this with a correlated subquery or a window function (Chapter 13, section 13.4). I'd use the window function: a correlated subquery can run once per row unless the database rewrites it; a window function reliably scans once, which matters once this table has millions of rows. I'd check the query plan to be sure (Chapter 28, section 28.5)."

Move 7: Validate the result

A sanity check, a hand calculation, or a reconciliation to a known total. This is the single most under-used move by otherwise-strong candidates, and one of the cheapest to add.

Question: "Show Riverstone's 2025 revenue by segment."

Without the move: Shows the three segment totals and stops.

With the move: "Before I trust these numbers, I'd check that the segments add up to the total: Retail plus Wholesale plus Hospitality should come to ₹43,35,471. If they don't, something's wrong with the join before I even look at the results."

Move 8: Connect to business impact

Say what decision the answer feeds, or what it's worth. A number with no owner and no use is a fact, not an insight.

Question: "Which accounts haven't ordered in the last 90 days?"

Without the move: Returns the list of names.

With the move: "At the end of 2025 this finds three accounts that ordered earlier in the year and then went quiet for more than 90 days, plus Home Plus, which never ordered at all. On its own that's a list. Paired with each account's revenue (the three bought about ₹1.7 lakh between them in 2025), it becomes a prioritized call list, which is what I'd actually hand to the sales head, not the raw rows."

Move 9: Think about scale and maintenance

Who runs this again next month? What happens when the table is 100 times bigger?

Question: "We'd like that at-risk query to run every Monday. Anything you'd change?"

Without the move: "No, it works."

With the move: "This query is fine at Riverstone's current size. If order volume grew 50x, I'd want an index on `order_date` and `customer_id` (Chapter 28, section 28.6), and I'd think about whether this needs to be a scheduled job with its own table (Chapter 46) instead of a query run fresh every time someone opens the dashboard."

Move 10: Admit limits honestly

Say what you don't know, and how you'd find out. Bluffing is far more damaging than a gap.

Question: "Have you used Spark?"

Without the move: "Yes, a bit," hoping there's no follow-up; or a flat "No," and silence.

With the move: "I haven't used Spark, but I've [split a large job into chunks, for example processed a year of order data month by month in Python]. The idea, splitting work so parts run in parallel, is the same. I'd expect to be productive within a couple of weeks, and I'd want to pair with someone on the team for the first real task."

Fill the brackets with something you have actually done. Only claim what you've done. (If you worked through Chapter 48, you've run Spark on one machine: say exactly that, and no more.)

Move 11: Use real evidence

A specific project from your own work beats a hypothetical every time it's available.

Question: "Have you built anything like this?"

Without the move: "Yes, similar things," with no detail; or a hypothetical "I would...".

With the move: "I built something close to this at my last company: a weekly report that flagged accounts with no order in 30 days. The tricky part wasn't the query; it was agreeing with sales on what 'no order' should exclude, mainly trial accounts. I'd expect the same conversation here."

Use a project you actually did. Your Chapter 27 portfolio project counts.

Move 12: Close the loop

Summarize your answer in one line at the end, and check it actually answered what was asked.

Interviews run long; a candidate who trails off without a clear ending makes the interviewer do the work of deciding whether you finished.

Question: "List customers who have never placed an order."

Without the move: Writes the query and stops talking, leaving the interviewer to ask "is that it?"

With the move: "So: left join from customers to orders, keep the rows where the order is NULL (Chapter 12, section 12.10), and watch out for cancelled orders depending on the definition. Does that answer what you were after, or did you want me to go further into the performance side?"

The twelve tags

When an answer labels its moves, as section 69.4 and the banks that follow do, it uses one short tag per move, always written the same way:

Move	Tag	Move	Tag
1 Clarify first	[+Clarify]	7 Validate the result	[+Validate]
2 State assumptions	[+Assume]	8 Business impact	[+Business]
3 Signpost your structure	[+Signpost]	9 Scale and maintenance	[+Scale]
4 Simple answer first	[+Simple first]	10 Admit limits honestly	[+Limits]
5 Raise edge cases	[+Edge cases]	11 Use real evidence	[+Evidence]
6 Trade-offs and alternatives	[+Trade-offs]	12 Close the loop	[+Close]

69.4 One question, all three tiers

Here's a single question carried through the full arc, so you can see seven of the twelve moves working together rather than as an isolated list. Simple first sits underneath all of them, and the other four (assumptions, trade-offs, limits, evidence) fit other kinds of questions, as section 69.5 shows. Every core question in the banks uses this three-tier format; the rapid-fire tables give the strong answer plus one extra point.

Q · Riverstone's monthly revenue dropped 12% last month. How would you investigate?

Answer that passes (scores ~2)

"I'd look at revenue by product and by region to see where the drop happened, then ask the sales team what changed."

Reasonable instinct, no structure, and it skips the question of whether the number is even real.

Strong answer (scores ~3)

"First I'd confirm the 12% is measuring what I think it's measuring, then break it down to find where it came from, then look for causes. Revenue is orders times average order value, so I'd check which of those two moved. Then I'd cut by segment, region, product category, and sales rep to find where the drop is concentrated. Once I know where, I'd look at likely causes: pricing changes, a lost account, a stock shortage, competitor activity, or plain seasonality."

Extra-points answer (scores ~4, moves labeled)

- **[+Clarify]** "Is that 12% against last month, the same month last year, or a target? And is it invoiced revenue or collected revenue? Those tell different stories."
- **[+Validate]** "Before I look for a business cause, I'd rule out a data problem: a failed pipeline load, a changed status mapping, orders stuck in 'Pending' that used to auto-close. A surprising share of scary-looking drops turn out to be exactly this."
- **[+Signpost]** "I'd break it into a tree: revenue is orders times average order value; orders is active customers times orders per customer; average order value is units per order times average price per unit, and average price moves with list prices, discounts and product mix. Then I'd check each branch, and within whichever branch moved, look for concentration: is 80% of the drop from one region or one big account? Concentrated points to something specific; spread-out points to something broad, like a pricing change."
- **[+Edge cases]** "I'd also check calendar effects, such as fewer working days last month or a festival period shifting order timing, before assuming anything's actually wrong."
- **[+Business]** "I'd size each candidate cause in rupees, rank them, and bring back the two or three that explain most of the gap, each with a recommended action and an owner, not just a diagnosis."
- **[+Scale]** "And I'd suggest an early-warning metric, weekly new orders say, so the next time this happens, we catch it inside a week instead of at month-end."
- **[+Close]** "So: confirm it's real, find where, then why, then act, and I'd want that whole loop closed inside two or three days, not weeks."

Likely follow-ups: You find one customer caused 60% of the drop: what next? How do you tell a data issue from a real one, fast? What would you put on a dashboard to catch this earlier next time?

Red flags: jumping straight to a cause · never questioning whether the data is even right · no numbers, only narrative · a structure so elaborate it never actually answers the question.

Learn it in: Chapter 23, sections 23.10 (KPI trees) and 23.11 (diagnosing a change).

69.5 The same moves, under different rounds

Not every move fits every round equally. Knowing which to lean on saves time you don't have.

Round	Moves that matter most	Why
Screening call	Simple first, close the loop	Fifteen minutes total; every question needs a fast, complete answer
Live coding	Clarify, edge cases, validate	The interviewer is watching your process, not just your final query
Case / business	Clarify, signpost, business impact	There's rarely one right answer; the structure you signpost is what's being graded
System design	Trade-offs, scale, admit limits	No design is free of trade-offs; pretending otherwise is the tell
Behavioral	Real evidence, close the loop	A concrete story beats a general philosophy every time

Rapid-fire, low-stakes questions in any round rarely need more than moves 3, 4, and 7: signpost, lead with the answer, validate. Save the full twelve-move treatment for the two or three questions per interview that are clearly the ones being weighed most heavily; you can usually tell, because the interviewer leans in, takes notes, or asks a follow-up.

69.6 What not to do

Extra points are easy to over-apply. Three common over-corrections cost points just as fast as saying nothing:

- **Clarifying everything.** Asking a clarifying question about a question with an obvious, single interpretation reads as stalling, not diligence. Reserve it for questions where the answer genuinely changes.
- **Caveat overload.** Hedging every sentence ("it depends," "in most cases," "generally speaking") without ever committing to an answer scores *worse* on correctness than a clean, slightly-too-simple answer, because the interviewer can't tell if you actually know it.
- **Padding with business talk that isn't specific.** "This is important for the business" is not the business-judgment move; naming the decision it feeds, or estimating its size, is. A vague gesture at "impact" with no number attached earns nothing.

The twelve moves are seasoning, not the whole meal. A technically wrong answer with six moves labeled on it still scores as wrong.

Common mistakes

Mistake	Symptom	Fix
Leading with caveats instead of the answer	Interviewer looks confused or impatient partway through	Answer first, in one sentence; depth after
Clarifying a question with an obvious meaning	Feels like stalling; wastes shared time	Reserve clarifying questions for genuine ambiguity
Listing edge cases with no explanation	Sounds like reciting a checklist	Explain <i>why</i> each edge case you raise actually matters here
Business talk with no number	"This helps the business" and nothing else	Attach a rough size, a decision, or an owner
No ending	Trails off; interviewer has to ask "is that it?"	Close the loop: one summary sentence, check you answered the actual question
Overusing every move on every question	Runs out of time; low-stakes questions take too long	Match the moves to the round (69.5); save full depth for the questions that matter most

In the real world: the mock interview that changed nothing technical

Meera runs a mock interview for a friend, Ishita, who's moving from a BA role toward data analytics. Ishita answers every technical question correctly: her SQL is clean, her statistics are sound. Meera's feedback afterward has nothing to do with any of that.

"You know all of this," Meera says. "What you're not doing is telling me you know it. You answer the question and stop. When I asked about duplicate customers, you wrote a correct query and waited. I had to ask you what happens if the join finds a NULL; you knew the answer the second I asked, which means you already knew it before I asked. Say it before I have to ask."

They redo three questions. Same technical answers, word for word on the first sentence. The only change: after each answer, Ishita adds one sentence: an edge case, a validation check, or what she'd actually do with the result. The mock interview "feels" completely different to both of them, even though Ishita learned nothing new about SQL that afternoon.

Two weeks later, in a real screening call with another company, an interviewer asks her to find a distributor's top customers by revenue. She answers, then adds: "Top by revenue over what period, I'll assume this year unless you say otherwise, and I'd want to check that a handful of large customers aren't a data error before presenting this as a ranked list." When the offer comes, the hiring manager tells her on the call what stood out: *she checked her own work without being asked*. Nothing there was new knowledge. It was the twelve moves, or rather, three of them, used at the right moment.

Project: score your own last answer

Goal: take an answer you've actually given, in a past interview, a work meeting, or a mock session, and rewrite it through all three tiers.

Tools you'll need

No software for this chapter. The only "tool" worth naming: **record yourself** answering three questions from any bank chapter out loud, on your phone, then listen back and mark where you stopped at "correct" instead of continuing to "strong." Most people are surprised by how often that's the very first sentence.

Steps:

1. Write down, as close to verbatim as you can recall, an answer you gave to a real technical or case question.
2. Score it against the five-dimension rubric (69.1), one to four on each dimension, honestly.
3. Identify which of the twelve moves (69.3) were present and which were missing.
4. Rewrite the answer as a "strong" tier answer, then again as "extra-points," labeling each move you add with its tag.
5. Time yourself saying the extra-points version out loud. If it's over two minutes, cut it: extra points that take too long lose the "structure & communication" dimension you just gained.

REVIEW

Recap

- Most interview rubrics score some version of **five dimensions** (correctness, structure, depth, business judgment, collaboration), not technical correctness alone.
- **Simple answer first, depth second** is the single most important shape, underneath every other move.
- The **twelve extra-point moves** (clarify, state assumptions, signpost, simple-first, edge cases, trade-offs, validate, business impact, scale, admit limits, real evidence, close the loop) are specific, practicable habits, not vague advice to "communicate better."
- Different **rounds reward different subsets** of the twelve moves; matching effort to round matters under time pressure.
- Every move can be **overused**: over-clarifying, caveat overload, and vague business talk all cost points just as surely as saying nothing.
- The banks that follow (Chapters 70–82) write their model answers against this same rubric and label moves with the twelve tags from section 69.3, e.g. [**+Edge cases**], so you'll see this pattern hundreds of times; the goal is for it to become automatic well before your next interview.

Key terms

extra-point moves · five-dimension rubric · correctness · structure & communication · depth & edge cases · business judgment · collaboration · simple-first · clarifying question · stated assumption · signposting · edge case · trade-off · validation (sanity check) · business impact · scale and maintenance · admitting limits · real evidence · closing the loop · move tag · red flag (interview) · over-correction

Check yourself

- ☐ I can name the five scoring dimensions without looking, and I know none of them is "technical knowledge" alone.
- ☐ I lead every answer with one sentence that actually answers the question, before any caveat.
- ☐ I can name all twelve moves, and I can give an example of each from memory.
- ☐ I know which moves matter most in which round, and I don't spend full-depth time on a fifteen-second warm-up question.
- ☐ I can catch myself over-clarifying or caveat-padding, and stop.
- ☐ I've rewritten at least one of my own real answers through all three tiers.

PRACTISE

Exercises

Each exercise below is a short question. Answer it three times (passing, strong, extra-points), labeling your moves on the third pass with the tags from section 69.3, exactly as section 69.4 modeled. After each question, in brackets, are the roles that most often hear it (DA data analyst, BA business analyst, BI BI developer, DS data scientist, DE data engineer; "all" means every role) and where its content is taught. Skip any whose chapter you haven't studied yet.

Definitions (Easy)

1. What's the difference between **WHERE** and **HAVING**? (DA, BI, DE · Chapter 12, section 12.9)
2. Explain a left join to someone non-technical. (DA, BA, BI · Chapter 12, section 12.10)
3. What's the difference between mean and median, and when would you use each? (*all* · Chapter 4, section 4.5; Chapter 21, section 21.1)
4. What's the difference between a data warehouse and a data lake? (DE, DS, architect · Chapter 49, sections 49.1 and 49.5)
5. How would you decide between a bar chart and a line chart for a given dataset? (DA, BI · Chapter 15, sections 15.3 and 15.4)
6. Explain overfitting to someone who's never trained a model. (DS · Chapter 37, sections 37.6 and 37.10)
7. What's the difference between correlation and causation, with a Riverstone-style example? (DA, DS · Chapter 22, section 22.5)

Diagnosis (Medium)

8. How would you find duplicate rows in a customer table? (DA, DE · Chapter 14, section 14.4)
9. How would you detect an outlier in monthly sales data? (DA, DS · Chapter 14, section 14.6; Chapter 21, section 21.4)
10. A dashboard number looks wrong. Walk me through how you'd check it. (DA, BI · Chapter 14, sections 14.10 and 14.11; Chapter 47, section 47.8)
11. Write a query to find each customer's most recent order. (DA, DE · Chapter 13, section 13.4. Sketch the query in words, then apply the moves to your explanation of it.)
12. What would you do if two source systems disagree about the same number? (DA, BA, DE · Chapter 14, section 14.11; Chapter 23, section 23.13; Chapter 51, section 51.5)
13. A scheduled report failed to send this morning. Walk me through how you'd find out why. (DA, BI, DE · Chapter 20, section 20.11; Chapter 46, section 46.6)

Design and judgment (Hard)

14. Riverstone wants a single "health score" per customer. How would you design it? (DA, DS, BA · Chapter 23, section 23.9; Chapter 51, section 51.1)
15. How would you explain a confidence interval to a sales manager? (DA, DS · Chapter 22, section 22.1)
16. Design a KPI dashboard for a customer support team. (DA, BI · Chapter 23, sections 23.8 and 23.12; Chapter 16, section 16.7)
17. How would you handle a stakeholder who wants a report you think is misleading? (all · Chapter 24, section 24.8; Chapter 15, section 15.12)
18. How would you prioritize which of ten dashboard requests to build first? (DA, BA, BI · Chapter 24, section 24.1; Chapter 25, section 25.12)
19. How would you onboard a new analyst onto a report you built and then left? (DA, BI · Chapter 20, section 20.13; Chapter 26, section 26.9)

Behavioral

20. Tell me about a time you disagreed with a decision at work. (all · Chapter 24, section 24.8. A behavioral question: apply moves 3, 11, and 12 especially.)

Answers

Five answers are worked in full, in all three tiers (exercises 1, 3, 10, 12 and 20: one from each group, plus a second diagnosis question). The other fifteen give the strong answer and the extra points most worth adding; write the other tiers yourself and compare. Every number below is quoted from the chapter named beside it. The Riverstone order figures come from the one-year database, `riverstone_2025` (Chapter 13, section 13.1), with cancelled orders left out unless the answer says otherwise, and were re-run for this chapter.

1. WHERE and HAVING (worked in full)

- **Passes:** "WHERE filters rows; HAVING filters groups."
- **Strong:** "WHERE filters individual rows before they're grouped; HAVING filters the groups after aggregation, so it's the only place you can test a count or a sum. To find customers with more than ten orders in 2025, the status filter goes in WHERE and `COUNT(*) > 10` goes in HAVING."
- **Extra points:**
 - **[+Simple first]** Lead with that one-line difference before any detail.
 - **[+Edge cases]** "WHERE can't use an aggregate, because at that step no counts exist yet: the database works through FROM, WHERE, GROUP BY, HAVING, then SELECT (Chapter 12, section 12.11)."
 - **[+Trade-offs]** "A condition on a plain column, such as the status, belongs in WHERE: rows are removed before the grouping work, and the intent is clearer."
 - **[+Validate]** "On Riverstone's 2025 data that query returns seven customers, led by Green Leaf Hotels with 17 orders. I'd check one of them by counting its orders directly."

2. Strong: "Start from a complete list and bring in matching details where they exist. Every customer stays on the list; a customer with no orders just shows blanks in the order columns." Extra points: **[+Business]** that's exactly how you find customers who have never ordered (Home Plus, in Riverstone's 2025 data); **[+Edge cases]** a customer with three orders appears three times, so count customers, not rows.

3. Mean and median (worked in full)

- **Passes:** "The mean is the average; the median is the middle value."
- **Strong:** "The mean adds everything up and divides by the count; the median is the middle value once the values are sorted. Use the median when a few large values pull the mean up, as with order values or salaries; use the mean when totals must work, because the mean times the count gives back the total."
- **Extra points:**
 - **[+Business]** "On Riverstone's 2025 orders the mean is ₹25,061 but the median is ₹21,375, and only 70 of the 173 orders are above the mean. Tell a sales executive 'the average order is ₹25,061' and most of their orders will feel below average (Chapter 4, section 4.5)."
 - **[+Edge cases]** "Check for outliers first: the largest order, ₹1,00,278, moves the mean but hardly moves the median."
 - **[+Trade-offs]** "For budgets and forecasts, use the mean, because it adds up; for 'what does a typical order look like', use the median."
 - **[+Close]** "So: median for typical, mean for totals, and report both when they disagree."

4. Strong: "A warehouse holds cleaned, structured tables for analysis, with a schema and transactions; a lake holds raw files of any format cheaply in object storage. A lakehouse puts a table format such as Delta Lake on top of the lake so the files behave like tables." Extra points: **[+Trade-offs]** a lake without discipline becomes a swamp, with no transactions, no schema and no history; **[+Business]** Riverstone keeps orders in the warehouse and its sensor archive as Parquet in object storage, because each suits its job (Chapter 49, section 49.1).

5. Strong: "A line for a sequence over time, where the trend is the message; bars for comparing separate categories. Monthly revenue is a line; revenue by segment is a bar." Extra points: **[+Edge cases]** a bar's axis must start at zero, because its length is its value; a line's axis needn't, if you label it clearly (Chapter 15, sections 15.3 and 15.4); **[+Clarify]** ask what the reader should notice first: change over time, or which is biggest.
6. Strong: "Overfitting is a model memorizing its training examples, noise included, instead of learning the pattern. It scores well on data it has seen and poorly on new data, like a student who memorized last year's answers." Extra points: **[+Validate]** you catch it by comparing training and validation scores: a big gap is the symptom (Chapter 37, section 37.10); **[+Trade-offs]** the fixes are more data, a simpler model, or regularization, each at some cost.
7. Strong: "Two things moving together doesn't mean one causes the other. Customers who get slower deliveries might order less, or the customers who order irregularly might get worse service: that's reverse causation. And a third thing can drive both, like summer driving ice-cream sales and drownings." Extra points: **[+Validate]** on Riverstone's 2025 customers, orders and average delivery days barely correlate ($r = -0.086$), which neither proves nor disproves that slow delivery costs orders (Chapter 22, section 22.5); **[+Business]** to test a cause, run an experiment or use a method from Chapter 31.
8. Strong: "Group by the fields that should be unique and keep the groups with `COUNT(*) > 1`; then decide which record in each group to keep." Extra points: **[+Clarify]** exact duplicates or fuzzy ones (same email, or similar names with typos)? (Chapter 14, section 14.4); **[+Edge cases]** trim spaces and match case before comparing, and remember that blank emails all group together.
9. Strong: "Plot the months first. Then flag any month outside the fences, $1.5 \times \text{IQR}$ beyond the quartiles, and find out whether each flagged month is an error or real." Extra points: **[+Edge cases]** a festive-season spike is real, not an error, so compare each month with the same month last year too (Chapter 14, section 14.6; Chapter 21, section 21.4); **[+Business]** never delete an outlier just because it's big: fix it if it's an error, explain it if it's real.
10. A dashboard number looks wrong (*worked in full*)
- **Passes:** "I'd rerun the query and see whether the number changes."
 - **Strong:** "First I'd pin down what 'wrong' means: wrong compared with what, and by how much. Then I'd reconcile the dashboard to its source one step at a time, with the same filters, dates and definition in a direct query, until the two agree or I find the step where they split."
 - **Extra points:**
 - **[+Clarify]** "Who noticed, and what number did they expect? A finance total and a sales total often differ by definition, not by error."
 - **[+Validate]** "I'd check against a known total: Riverstone's 2025 revenue is ₹43,35,471, and December's is ₹4,39,823.50."
 - **[+Edge cases]** "The usual suspects: cancelled orders counted as sales (that alone turns ₹43,35,471 into ₹43,98,121), a join that duplicates rows, a date or time-zone boundary, or a refresh that didn't run."
 - **[+Business]** "I'd tell the people who use the number what's wrong and when it'll be fixed, before they find out for themselves (Chapter 47, section 47.8)."

- **[+Scale]** "Once it's fixed, I'd add a check that would have caught it: a reconciliation that must return zero (Chapter 14, section 14.10)."
- **[+Close]** "So: define the gap, reconcile step by step, fix, tell people, and add the check."

11. Strong: "Number each customer's orders newest first with `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC)`, and keep row 1 (Chapter 13, section 13.4)." Extra points: **[+Edge cases]** two orders on the same day are a tie: add `order_id` to the `ORDER BY` so the choice is repeatable, or use `RANK()` to keep both; **[+Trade-offs]** a correlated subquery gives the same result; say why you chose the window function (Move 6).

12. Two source systems disagree (*worked in full*)

- **Passes:** "I'd use the system I trust more."
- **Strong:** "I'd find out why they differ before choosing one. I'd compare the definitions (what each counts, which statuses, which dates), then the timing (when each was last updated), then match the records one by one to find the rows that differ. Usually the gap is a definition or timing difference, not a bug."
- **Extra points:**
 - **[+Clarify]** "Which number is being used for a decision, and by whom? That tells me which gap matters."
 - **[+Signpost]** "Definitions first, then timing, then row-by-row matching."
 - **[+Validate]** "To match records, I'd join the two on their shared key with a full outer join (Chapter 12, section 12.10); rows that exist on only one side usually explain most of the gap (Chapter 14, section 14.11)."
 - **[+Business]** "Then I'd agree which system is the system of record for this number (Chapter 51, section 51.5) and write the definition down so both teams get the same number from now on (Chapter 23, section 23.13)."
 - **[+Close]** "So: explain the gap, fix the definition, and name one source of truth."

13. Strong: "Check the run log first: did the job start, did it fail, and at which step: getting the data, building the report, or sending it? Then fix it, resend, and tell the recipients." Extra points: **[+Edge cases]** "no data today" and "the job failed" need different fixes, and a rerun must not send the report twice (Chapter 20, section 20.11); **[+Scale]** an alert on failure and automatic retries, so next time you hear about it before the recipients do (Chapter 46, section 46.6); **[+Business]** for the Daily Sales Flash, due by 07:30 IST, tell the readers first, then investigate.

14. Strong: "Start from the decision it serves, such as which accounts the account managers call first. Pick a few signals that predict that outcome (order frequency against the customer's own usual rhythm, the recent revenue trend, support tickets, payment history), score each simply, combine them, and check the score against customers who actually stopped ordering." Extra points: **[+Simple first]** start with a rule-based score before any model; **[+Validate]** test it on last year: did the low scores really go quiet? (Chapter 13, section 13.8, pattern 6); **[+Scale]** write the score back into the CRM where account managers already work (Chapter 51).

15. Strong: "It's a range that probably contains the true value, and its width tells you how sure we are. '62% of customers are satisfied, give or take about five points' is the honest report from a survey of 400 customers (Chapter 22, section 22.1)." Extra points: **[+Business]** ask whether the decision changes anywhere in that range: if it doesn't, the width doesn't matter; **[+Edge cases]** the method is right 95% of the time, so about one interval in twenty misses the truth.

16. Strong: "Start from what the team is trying to achieve, then a handful of KPIs: tickets opened and closed, time to first response and to resolution (the median and the 90th percentile, not only the mean), the age of the open backlog, and customer satisfaction. One headline row, trends underneath, and a breakdown by channel or category." Extra points: **[+Clarify]** who looks at it, and what do they decide from it?; **[+Edge cases]** speed can be gamed by closing tickets too early, so pair it with the reopen rate (Chapter 23, section 23.12).

17. Strong: "Ask what decision the report is for. Show them, with the numbers side by side, why the requested version would mislead, and offer a version that answers their real question. If they still want it, label it clearly and put my concern in writing (Chapter 24, section 24.8)." Extra points: **[+Business]** name the decision that could go wrong and what it could cost; **[+Close]** agree the next step out loud.

18. Strong: "Score each request on value (which decision it feeds, how many people use it, how often), effort, and urgency; build the high-value, low-effort ones first; and agree the order with the people asking rather than deciding alone (Chapter 25, section 25.12)." Extra points: **[+Clarify]** turn each ask into its question first (Chapter 24, section 24.1): several requests often turn out to be the same question, or one an existing report already answers.

19. Strong: "Hand over a short written guide: what the report answers and for whom, where the data comes from, how and when it runs, the known quirks and checks, and who to ask. Then run it together once, and let them run it next time while I watch (Chapter 20, section 20.13)." Extra points: **[+Validate]** they run it alone and match the last output exactly; **[+Scale]** keep it in a repository a stranger can run (Chapter 26, section 26.9).

20. A time you disagreed (*worked in full*)

Use your own story; the one below only shows the shape.

- **Passes:** "My manager wanted a different approach from mine. I explained my view, and in the end we went with theirs." True, but vague: no specifics, no result.
- **Strong:** "In my last job, my manager wanted the monthly sales report to count cancelled orders as revenue, because that's what the order system showed. I disagreed, because finance left them out and the two reports never matched. I showed both totals side by side with the cancelled orders listed, and we agreed to report revenue without them and add a footnote. From then on, the two reports matched."
- **Extra points:**
 - **[+Signpost]** "I'll give you the situation, what I did, and how it turned out."
 - **[+Evidence]** Real, specific details: the report, the two totals, and what you did yourself, not what "we" did.
 - **[+Business]** Say what changed because of it: the reports matched, and the monthly argument about which number is right stopped.

- **[+Close]** "So: I disagreed with evidence rather than opinion, and I'd do it the same way again, though next time I'd raise it before the report went out, not after."

Chapter 81 turns this shape into the STAR method and works this same question in full.

NEXT

Where this leads

- **Chapters 70–82**, every bank in this part, write their core-question model answers against this chapter's rubric and label moves with the twelve tags introduced here.
- **Chapter 68, How Data Hiring Works**, covers what happens before you're in the room (screening, portfolios, referrals), which this chapter doesn't touch.
- **Chapter 82, Take-Home Assignments & Mock Interviews**, is where you apply this chapter under realistic time pressure, end to end.

Chapter 70. Excel, Google Sheets, VBA & BI Question Bank

Chapter at a glance

You will learn to: answer the spreadsheet, automation, and BI questions that come up across screening calls, live exercises, and case interviews for analyst, BA, BI, and automation-track roles · check every formula answer on a small practice table you can type in two minutes · read a broken macro and fix it live · critique a dashboard the way a hiring manager would · walk out with a 20-question final-week revision list.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 10, 11, 15, 16 and 19, with a few links to Chapter 12 (joins, UNION ALL, GROUP BY). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 4–6 hours for a first pass (about 5 minutes per core question, 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list.

How this chapter is built. Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, one per line, tagged the same way: [+Clarify], [+Edge cases], [+Validate] and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it. Every formula result comes from the practice table in section 70.0.

In this chapter

LEARN	70.0 The practice table, and how to use this bank	46
	70.1 Formulas, lookups, and logic	47
	70.2 Pivot tables and data analysis	50
	70.3 Power Query	53
	70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE	54
	70.5 VBA and Excel macros	56
	70.6 Google Apps Script	60
	70.7 DAX and Power BI modeling	63
	70.8 Dashboard critiques	66
APPLY	Common mistakes · In the real world · Project	68
REVIEW	Key terms · Final-week revision list	70
NEXT	Where this leads	70

70.0 The practice table, and how to use this bank

Every formula answer below was worked on this six-row table of Riverstone order lines. Type it into a blank sheet starting at cell **A1** (header in row 1, data in rows 2 to 7), or open companion/ch70/ch70_practice.xlsx (sheet **Orders**; ch70_practice.csv imports into Google Sheets). The products and list prices are the ones in Chapters 10 and 11.

Columns A to E:

Row	A order_id	B order_date	C customer_name	D category	E product_name
2	1001	2025-01-06	Sharma Hardware	Storage	Storage Box 10L
3	1002	2025-01-07	Metro Mart	Kitchen	Water Bottle 1L
4	1003	2025-01-09	Sharma Hardware	Storage	Stackable Bin
5	1004	2025-01-13	Coastal Foods	Industrial	Industrial Crate
6	1005	2025-01-20	Metro Mart	Storage	Storage Box 25L
7	1006	2025-02-03	Sharma Hardware	Storage	Storage Box 10L

Columns F to J, same rows:

Row	F quantity	G unit_price	H discount_pct	I net_revenue	J status
2	25	430	5	10212.5	Delivered
3	40	115	0	4600	Delivered
4	30	290	5	8265	Cancelled
5	20	1400	10	25200	Delivered
6	8	750	0	6000	Shipped
7	12	430	0	5160	Delivered

Row 1 holds the headers (the column names above, without the letters).

net_revenue is quantity × unit_price × (1 – discount_pct/100), as in Chapter 10: for order 1001, 25 × 430 × 0.95 = 10,212.50. The six lines add up to ₹59,437.50; without the cancelled order 1003 (₹8,265), ₹51,172.50. The book's revenue rule holds here too: **a cancelled order is not a sale**, so every revenue answer below leaves it out. Columns K and L are empty; the macro questions write into K.

Levels and roles. Each question carries a level and the roles that usually ask it:

- **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds.
- **DA** data analyst · **BA** business analyst · **BI** BI developer · **AUT** automation (macro and script) roles.

Within each section the core questions run from easier to harder. If you're preparing for a first analyst job, do every Fresher question first, then come back for the Mid ones.

70.1 Formulas, lookups, and logic

Q70-001 · Total revenue for one customer, with a minimum-quantity filter

Level: Fresher · **Roles:** DA, BA, BI

Remember it as: SUMIFS: sum range first, because the condition pairs can repeat after it. SUMIF: sum range last, because it's optional.

Answer in one line: SUMIFS with the sum range first, then one range-condition pair per condition, including the one that drops cancelled orders:

```
=SUMIFS(I2:I7, C2:C7, "Sharma Hardware", F2:F7, ">=20", J2:J7, "<>Cancelled") → 10212.5
```

By hand: Sharma Hardware has three lines. Order 1001 (25 units, Delivered) counts: ₹10,212.50. Order 1003 has 30 units but was cancelled, so "<>Cancelled" (not equal to Cancelled) drops it. Order 1006 has only 12 units, so ">=20" drops it. Without the status condition the formula returns 18477.5, which counts a cancelled order as revenue.

Tier	What to say
Passes	Filter by hand and add up, or use SUMIF (it takes only one condition)
Strong	SUMIFS handles several AND conditions; the sum range comes first, unlike SUMIF
Extra points	[+Clarify] "Total revenue including or excluding cancelled orders?" [+Validate] cross-check against a pivot table filtered the same way [+Scale] use a Table reference (Orders[net_revenue]) so new rows are picked up automatically

Likely follow-ups: Same with COUNTIFS? Different in Google Sheets? "Top customer," not a named one?

Red flag: filtering by hand instead of a formula; nesting SUMIFS for several conditions.

Learn it in: Chapter 10, section 10.8 (the IFS family); Chapter 11, section 11.2 (criteria in depth).

Q70-002 · XLOOKUP vs INDEX/MATCH vs VLOOKUP

Level: Fresher · **Roles:** DA, BA, BI

Remember it as: VLOOKUP only looks right and defaults to "close enough." XLOOKUP looks any direction and defaults to exact.

Answer in one line: VLOOKUP looks only to the right and uses approximate match unless you pass FALSE; INDEX/MATCH looks in any direction; XLOOKUP does INDEX/MATCH's job in one function, exact match by default.

```
=XLOOKUP("Storage Box 25L", E2:E7, G2:G7) → 750  
=INDEX(G2:G7, MATCH("Storage Box 25L", E2:E7, 0)) → 750  
=VLOOKUP("Storage Box 25L", E2:G7, 3, FALSE) → 750
```

MATCH's third argument, 0, asks for an exact match and returns the position, 5; INDEX returns the 5th price. VLOOKUP's 3 is the column number counted from E, and FALSE switches off approximate match.

Tier	What to say
Passes	"XLOOKUP is the newer, better version of VLOOKUP"
Strong	VLOOKUP breaks when a column is inserted (the column number is typed in); always pass FALSE for exact match
Extra points	[+Edge cases] VLOOKUP's silent approximate-match default is the most common source of wrong-but-plausible results [+Trade-offs] use INDEX/MATCH in files that may open in an Excel older than 2021 [+Validate] handle "not found" on purpose: XLOOKUP's if_not_found argument or IFNA, not a blanket IFERROR (see Q70-008)

Likely follow-ups: Duplicate lookup values? What does MATCH's third argument do? Can plain VLOOKUP look to the left?

Red flag: claiming VLOOKUP can look to the left; not knowing the approximate-match default.

Learn it in: Chapter 10, section 10.10 (a first lookup); Chapter 11, sections 11.3 (IFNA) and 11.4 (lookups in depth).

Q70-003 · Absolute vs. relative references

Level: Fresher · **Roles:** DA, BA, BI

Remember it as: \$ = stays put when copied. No \$ = shifts with the copy. F4 cycles through all four combinations.

Answer in one line: B2 shifts when copied (relative); \$B\$2 never moves (absolute); \$B2 and B\$2 lock only the column or only the row (mixed).

Tier	What to say
Passes	"\$ locks a reference"
Strong	A concrete failure: a tax-rate cell referenced as B2 instead of \$B\$2, copied down a column, silently multiplies by the wrong cell on every row, with no error
Extra points	[+Edge cases] copying across and down needs a mixed reference in each direction, not one absolute lock [+Validate] spot-check the last cell after copying, not just the first

Likely follow-ups: What does F4 do? Named ranges vs. absolute references? Table references?

Red flag: can't give a concrete example of what breaks.

Learn it in: Chapter 10, section 10.6.

Q70-004 · Classify each order into a size tier

Level: Fresher · **Roles:** DA, BA

Remember it as: Nested IFs: most restrictive condition first, or the wrong tier wins.

Answer in one line: check Large before Medium. In L2, filled down to L7:

```
=IF(I2>20000, "Large", IF(I2>5000, "Medium", "Small"))
```

On the practice table this gives Medium, Small, Medium, **Large**, Medium, Medium: order 1004 (₹25,200) is the only Large one, and order 1002 (₹4,600) the only Small one.

Tier	What to say
Common wrong answer	<code>=IF(I2>5000, "Medium", IF(I2>20000, "Large", "Small"))</code> : it never returns "Large", because anything over 20,000 is caught by <code>>5000</code> first. Order 1004 comes out Medium
Passes	The fixed order above, with no mention of what happens at the boundaries
Strong	The fixed order, explained: the first true test wins, so the most restrictive goes first
Extra points	[+Edge cases] what happens at exactly 20,000? <code>></code> excludes it; ask whether the boundary is inclusive [+Trade-offs] past two or three tiers, switch to IFS or a lookup table with approximate match

Likely follow-ups: Same with IFS? With a lookup table? The tier boundary changes next quarter?

Red flag: ordering the conditions least restrictive first (exactly the wrong answer above); not testing the boundaries.

Learn it in: Chapter 10, section 10.8 (IF, IFS); Chapter 11, section 11.4 (approximate match: bands and tiers).

Q70-005 • SUMPRODUCT, and when to reach for it over SUMIFS

Level: Mid · **Roles:** DA, BI

Remember it as: SUMPRODUCT multiplies row by row, then adds: $\text{quantity} \times \text{price} \times (1 - \text{discount})$ for every row, summed.

Answer in one line: it multiplies arrays item by item and adds the results, so it can total a calculation without a helper column:

<code>=SUMPRODUCT(F2:F7, G2:G7, 1-H2:H7/100)</code>	→ 59437.5
<code>=SUMPRODUCT(F2:F7 * G2:G7 * (1-H2:H7/100) * (J2:J7<>"Cancelled"))</code>	→ 51172.5

The first line works row by row: $25 \times 430 \times 0.95 = 10,212.50$ for order 1001, $40 \times 115 \times 1 = 4,600$ for order 1002, and so on, then adds all six: the same 59,437.50 as `=SUM(I2:I7)`. The second line multiplies by one more array, `(J2:J7<>"Cancelled")`, which is TRUE (1) for a valid line and FALSE (0) for the cancelled one, so order 1003 contributes nothing. It matches `=SUMIFS(I2:I7, J2:J7, "<>Cancelled")`, 51172.5. (If you've read Chapter 35: with two arrays, SUMPRODUCT is exactly a dot product.)

Tier	What to say
Passes	"It multiplies things and adds them up"
Strong	It's array arithmetic: no helper column needed, and the result reconciles to the helper-column total
Extra points	[+Trade-offs] for simple conditions SUMIFS is easier to read; SUMPRODUCT earns its place for weighted totals or OR across columns, which SUMIFS can't express [+Edge cases] every array must be the same size, or it returns #VALUE!

Likely follow-ups: A weighted average with SUMPRODUCT? The old Excel array-formula version (`{}`)? The Python equivalent?

Red flag: no mention of it being array arithmetic; a total that quietly includes cancelled orders.

Learn it in: Chapter 11, section 11.2 (SUMPRODUCT: conditions the IFS family can't express).

Rapid-fire, 70.1

Roles: DA, BA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-006	COUNT vs COUNTA vs COUNTBLANK?	Numeric cells only / any non-empty cell / empty cells	[+Edge cases] a formula returning "" looks empty but isn't; COUNTA still counts it	Fresher · 10.7
Q70-007	Formula for "days since this order"?	=TODAY() - B2, formatted as a number, not a date	[+Edge cases] if it shows as a date, reformat as General or Number	Fresher · 10.9
Q70-008	IFERROR vs IFNA?	Catches every error type / catches only #N/A	[+Business] IFERROR used broadly can hide a real bug behind a friendly blank	Fresher · 11.3
Q70-009	FILTER and UNIQUE together?	=UNIQUE(FILTER(C2:C7, D2:D7="Storage")) spills Sharma Hardware, Metro Mart	[+Edge cases] the cells it spills into must be empty, or #SPILL!	Fresher · 11.6
Q70-010	TEXT function, one use?	=TEXT(B2, "MMMM") → January	[+Edge cases] the result is text now: no more date arithmetic on it	Fresher · 10.9
Q70-011	Named range: why use one?	A reusable name for a cell or range, e.g. TaxRate for \$B\$2	[+Scale] name a Table column (Orders[net_revenue]) rather than a fixed range, so it grows with the data	Fresher · 10.6
Q70-012	A common cause of #REF!?	A formula refers to a cell, row or column that's since been deleted	[+Validate] trace precedents to find the source, not just the symptom	Fresher · 10.6
Q70-013	Flag near-duplicate names (spacing, case)?	A helper column of cleaned names, =TRIM(LOWER(C2)), then =COUNTIF(clean_range, TRIM(LOWER(C2)))>1	[+Business] near-duplicates quietly split one customer into two rows of a pivot	Mid · 10.9, 10.8
Q70-014	Conditional formatting "Stop If True"?	When a rule with Stop If True matches, the rules below it aren't applied to that cell (rule order goes beyond Chapter 10)	[+Edge cases] without it, formats that don't clash (fill and font colour) stack; when they clash, the rule higher in the list wins, so order the rules on purpose	Mid · 10.12
Q70-015	Table (Ctrl+T) vs. a plain range?	Formulas and formatting extend as rows are added; structured references	[+Scale] building dashboards on Tables, not fixed ranges, is the easiest way to survive new data	Fresher · 10.11

70.2 Pivot tables and data analysis

Q70-016 · Pivot grand total doesn't match a manual SUM

Level: Fresher · **Roles:** DA, BA, BI

Remember it as: Refresh, then filters, then aggregation type: in that order.

Answer in one line: three usual suspects, checked in order: the source range doesn't cover new rows → a hidden filter or slicer is active → the value field defaulted to Count instead of Sum (one stray text value in a numeric column does this).

Tier	What to say
Passes	"Check the source range"
Strong	The three suspects above, in order
Extra points	[+Validate] refresh first, then check for active filters, before assuming the data is wrong [+Scale] build the pivot on a Table to remove the first cause for good

Likely follow-ups: Change the default summary? Calculated field vs. calculated item? Show values as % of total?

Red flag: jumping to "the data must be wrong" without checking the pivot's own settings first.

Learn it in: Chapter 11, section 11.5 (Refresh, and the traps that come with it).

Q70-017 · Revenue by customer by month, with a running total

Level: Mid · **Roles:** DA, BI

Remember it as: "Show Values As → Running Total In": the pivot does it natively; don't build it by hand.

Answer in one line: customer in Rows, month (grouped from the date field) in Columns, net revenue in Values twice, the second set to **Show Values As → Running Total In** by month.

Tier	What to say
Passes	"Customer in Rows, months in Columns, revenue in Values" (misses the running total)
Strong	Add the field a second time and set Show Values As → Running Total In
Extra points	[+Edge cases] grouping by Months alone across two years merges January 2024 with January 2025 unless Years is grouped too [+Validate] the running total's last value should equal a plain SUMIFS for that customer

Likely follow-ups: A slicer for region? Grouping vs. a calculated field? % of the customer's own annual total instead?

Red flag: not knowing Show Values As exists; building the running total with manual formulas.

Learn it in: Chapter 11, section 11.5 (Grouping dates; More ways to show values).

Q70-018 · A calculated field's aggregation trap

Level: Mid · **Roles:** DA, BI

Remember it as: A calculated field runs AFTER the summing, not before. Averaging a ratio this way is almost always wrong.

Answer in one line: a calculated field's formula works on the already-summed totals in each cell, not row by row: fine for `net_revenue / quantity`, wrong if you expect it to average a percentage column.

Tier	What to say
Passes	"A formula field inside the pivot"
Strong	It computes on the totals, which is right for a ratio of sums and wrong for anything that must be worked out row by row first
Extra points	[+Edge cases] the classic case: the average of <code>discount_pct</code> across lines $\neq$ total discount $\div$ total list value [+Validate] check a ratio field's grand total against a hand calculation from the raw totals

Likely follow-ups: Calculated item vs. calculated field? Does DAX have the same trap? (Yes: it's *why CALCULATE and filter context matter*, section 70.7.)

Red flag: not knowing that calculated fields work on sums.

Learn it in: Chapter 11, section 11.5 (Calculated fields, and why ratios need care).

Rapid-fire, 70.2

Roles: DA, BA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-019	Slicer vs. filter?	A visual control that can drive several pivots and charts at once / affects only its own pivot	[+Business] slicers make a report usable for a non-technical viewer	Fresher · 11.5
Q70-020	Top 3 customers by revenue in a pivot?	Value Filters → Top 10... , changed to 3, on the summed field	[+Edge cases] customers tied at the cut-off are all included, so "Top 3" can show 4 rows; say how you'd break the tie (e.g. by order count)	Fresher · 11.5
Q70-021	Pivot chart vs. a normal chart on pivot data?	Linked, updates with the pivot / needs rebuilding if the pivot's shape changes	[+Trade-offs] pivot charts are easier to maintain, harder to customize fully	Mid · 11.5
Q70-022	Refresh all pivots at once?	Data → Refresh All ; in VBA, <code>ThisWorkbook.RefreshAll</code> (one line; not covered in Chapter 19)	[+Scale] set "Refresh data when opening the file" so nobody sees stale numbers	Mid · 11.5, 11.7
Q70-023	"(blank)" appears in the row labels: why?	Some source rows have an empty value in that field; the pivot groups them under (blank) instead of dropping them	[+Validate] count them and trace them to the source rows: a data-quality signal (Chapter 11's pivot found ₹2,04,502.50 of revenue with no rep this way)	Mid · 11.5
Q70-024	Each customer's revenue as a % of the grand total?	Add revenue again → Show Values As → % of Grand Total	[+Business] the fastest way to answer "who matters most" with no manual maths	Fresher · 11.5
Q70-025	A pivot average, with blanks in the data?	Blanks are left out of the count, not treated as zero	[+Edge cases] text mixed into a numeric column can flip the field to Count instead (Q70-016)	Mid · 11.5

#	Question	One-line answer	Extra point	Level · learn it in
Q70-026	"Days between first and most recent order" per customer?	Customer in Rows, order_date in Values twice (Min and Max), then Max – Min in a plain formula beside the pivot (GETPIVOTDATA); or skip the pivot: =MAXIFS(B2:B7, C2:C7, "Sharma Hardware") - MINIFS(B2:B7, C2:C7, "Sharma Hardware") → 28	[+Edge cases] a calculated field can't do this, because it works on sums (Q70-018). If you've read Part 4, it's a spreadsheet version of Chapter 36's recency features (section 36.6)	Mid · 11.2, 11.5

70.3 Power Query

Q70-027 · Combine 12 monthly files into one clean table

Level: Mid · **Roles:** DA, BI

Remember it as: "From Folder" once, refresh forever. Copy-paste once, redo forever.

Answer in one line: Get Data → From Folder → Combine & Transform stacks every file with matching columns into one table; clean once in the Power Query Editor, and every step replays on refresh, including on a 13th file dropped into the folder later.

Tier	What to say
Passes	"Copy-paste all 12 files into one sheet" (works once, not repeatable)
Strong	From Folder + Combine & Transform, with the cleaning steps recorded once
Extra points	[+Scale] the whole point: next month's refresh needs no manual work [+Edge cases] select columns by name, not position, so a reordered column in one file doesn't silently misalign [+Validate] check that the combined row count equals the sum of the 12 files' row counts, to catch a skipped file

Likely follow-ups: Different column orders across files? A file that fails to load? How does this compare with a VBA macro doing the same thing?

Red flag: manual copy-paste as the final answer, no mention of repeatability.

Learn it in: Chapter 11, section 11.7 (compare with Chapter 19, section 19.6, the VBA version).

Q70-028 · Power Query "Merge" vs. "Append"

Level: Mid · **Roles:** DA, BI

Remember it as: Append = SQL UNION ALL (stack rows, keep duplicates). Merge = SQL JOIN (add columns).

Answer in one line: Append stacks tables (same columns, more rows, every row kept); Merge joins tables side by side on a key (new columns), with the join kinds SQL has: left, right, inner, full outer, anti.

Tier	What to say
Passes	"They both combine tables"
Strong	Append = UNION ALL, same columns; Merge = join on a matching key, and its join-kind list is Chapter 12's SQL joins
Extra points	[+Edge cases] a Merge on a key that repeats on the right side fans rows out, exactly Chapter 12's fan-out trap [+Validate] check the row count after a Merge against what you expect

Likely follow-ups: An anti-join in Power Query? What does "expand" do after a Merge? The SQL equivalent, and why not plain UNION?

Red flag: confusing Merge and Append; saying Append is UNION (which removes duplicates).

Learn it in: Chapter 11, section 11.7 (Step 5, the merge; Power Query in depth, Append); Chapter 12, sections 12.10 (the fan-out trap) and 12.12 (UNION vs UNION ALL).

Rapid-fire, 70.3

Roles: DA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-029	What's a Power Query "step"?	A named, reorderable transformation recorded in Applied Steps; each step runs on the previous step's output	[+Edge cases] renaming a column, then referring to the old name in a later step, breaks the query	Fresher · 11.7
Q70-030	Split "City, State" into two columns?	Split Column → By Delimiter (comma)	[+Edge cases] when some values hold extra commas, choose Split at: Right-most delimiter so the extra commas stay in the first part; trim spaces first	Fresher · 11.7
Q70-031	What does "Unpivot Columns" do?	Wide data → long data; Power Query's version of pandas' melt	[+Business] exports arrive wide (for people); analysis wants long, so this is often the first real cleaning step	Fresher · 11.7, 18.8
Q70-032	Change a column's data type: why does it matter?	Right-click the header → Change Type ; later number or date steps fail on a column still typed as text	[+Validate] look for new errors on rows that didn't convert	Fresher · 11.7
Q70-033	"Close & Load" vs. "Close & Load To..."?	Loads a table to a new sheet / lets you choose: table, PivotTable, or connection only	[+Scale] "Only Create Connection" for staging queries keeps the workbook lean	Fresher · 11.7

70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE

Q70-034 · Total revenue by category, sorted, with QUERY

Level: Mid · **Roles:** DA, BA

Remember it as: QUERY is SQL in a spreadsheet: column letters instead of names, otherwise the same rules.

Answer in one line:

```
=QUERY(A1:J7,  
  "select D, sum(I)  
  where J <> 'Cancelled'  
  group by D  
  order by sum(I) desc  
  label sum(I) 'net_revenue'", 1)
```

The result spills into a small table under the headers `category` and `net_revenue`: Industrial 25200, Storage 21372.5, Kitchen 4600. Storage is orders 1001, 1005 and 1006 (10,212.50 + 6,000 + 5,160); the cancelled order 1003 is left out by `where`. The final `1` says the range has one header row. `group by` is required whenever you aggregate alongside a column you don't aggregate: the same rule as SQL's `GROUP BY` in Chapter 12.

Tier	What to say
Passes	"I'd build a pivot table instead" (dodges the <code>QUERY</code> question)
Strong	The formula above, explaining <code>select</code> , <code>where</code> , <code>group by</code> , <code>order by</code> and <code>label</code>
Extra points	[+Edge cases] columns are named by their letter in the range, not by header, and a column inserted later shifts the letters [+Trade-offs] a pivot is faster for a one-off; <code>QUERY</code> wins when the result must feed another formula or chart

Likely follow-ups: Add a date range to the `where`? The `LIMIT` equivalent? How is this different from a pivot's output?

Red flag: not knowing `QUERY` exists; confusing column letters with header names; a total that includes cancelled orders.

Learn it in: Chapter 11, section 11.10 (`QUERY`); Chapter 12, section 12.9 (the `GROUP BY` rule).

Q70-035 · What `ARRAYFORMULA` does, and when it's actually needed

Level: Mid · **Roles:** DA, BA, AUT

Remember it as: Functions that already return one answer from a whole range don't need it. Per-row formulas do.

Answer in one line: it makes a per-row formula (`IF`, `&`, arithmetic) work on whole ranges, so one cell fills a column; functions that already return one answer from a whole range (`SUM`, `SUMIFS`, `FILTER`, `UNIQUE`, `QUERY`) don't need it.

```
=ARRAYFORMULA(IF(D2:D="", "", IF(D2:D="Storage", "Yes", "No")))
```

In K2 this fills Yes, No, Yes, No, Yes, Yes down to row 7 and leaves every row below blank. The outer `IF(D2:D="", "", ...)` is Chapter 11's guard: without it, the open-ended range `D2:D` would write "No" down every empty row of the sheet.

Tier	What to say
Passes	"It applies a formula to a whole range"
Strong	Explains <i>why</i> : some functions already work on ranges; plain IF, & and arithmetic don't, unless wrapped
Extra points	[+Business] one ARRAYFORMULA over an open-ended range covers rows added later, which fixes the most common way a shared sheet quietly breaks: a new row below the last filled-down formula [+Edge cases] open-ended ranges can be slow on a very large sheet

Likely follow-ups: How is it different from Excel's dynamic arrays? Why doesn't SUMIFS expand inside ARRAYFORMULA, and what would you use instead (SUMIF, or QUERY)?

Red flag: not knowing why some formulas need it and others don't.

Learn it in: Chapter 11, section 11.10 (ARRAYFORMULA).

Rapid-fire, 70.4

Roles: DA, BA and AUT for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-036	IMPORTRANGE, and its setup trap?	Pulls a range from another Sheets file by URL; needs a one-time "Allow access" click	[+Edge cases] if access to the source is removed, every IMPORTRANGE that reads it breaks (#REF!)	Fresher · 11.10
Q70-037	Pull live external data into Sheets?	GOOGLEFINANCE for currencies and stocks; IMPORTXML/IMPORTHTML for public web pages; Apps Script's UrlFetchApp for an API that needs a key	[+Trade-offs] IMPORT... functions are simplest but break when the page changes its layout	Mid · 11.11, 19.13
Q70-038	FILTER vs. QUERY's where?	Filters rows with a test and combines with other array formulas / SQL-like, can filter, aggregate and sort in one formula	[+Trade-offs] FILTER alone is easier to read; QUERY avoids nesting two formulas for filter-plus-total	Mid · 11.6, 11.10
Q70-039	Protect a shared sheet from accidental overwrites?	Data → Protect sheets and ranges , limiting who can edit the formula cells	[+Business] the cheapest fix for the most common cause of "why are the numbers wrong" on a team sheet	Mid · 10.14

70.5 VBA and Excel macros

Q70-042 · The VBA object model

Level: Mid · **Roles:** AUT, DA, BA

Remember it as: Application → Workbooks → Worksheets → Range. Biggest to smallest, like folders inside folders.

Answer in one line: Application (Excel itself) → Workbooks (every open file) → one Workbook → Worksheets → one Worksheet → Range (one or more cells; Cells(row, column) is one cell by number). Full form: Application.Workbooks("Sales.xlsx").Worksheets("Orders").Range("A1"); VBA lets you drop the parts it can assume.

Tier	What to say
Passes	"They're all objects in VBA"
Strong	The nesting above, with a fully qualified example
Extra points	[+Edge cases] relying on "the active sheet" is a common cause of a macro that works when run by hand but breaks from a button on another tab [+Scale] in any macro you'll reuse, qualify Range and Cells with an explicit worksheet

Likely follow-ups: ActiveSheet vs. ThisWorkbook.Worksheets("Orders")? What does With do? Referring to a closed workbook?

Red flag: relying entirely on whatever sheet and workbook happen to be active.

Learn it in: Chapter 19, section 19.5.

Q70-040 · Find the last used row

Level: Mid · **Roles:** AUT, DA

Remember it as: Start from the bottom and go up. Starting from the top and going down stops at the first gap.

Answer in one line: go to the last row of the sheet in column A and press Ctrl+Up, in code:

```
Sub ShowLastRow()
    Dim lastRow As Long
    lastRow = Cells(Rows.Count, "A").End(xlUp).Row
    Debug.Print "last row: " & lastRow
End Sub
```

On the practice table, the Immediate window shows:

```
last row: 7
```

Rows.Count is the number of the sheet's last row; Cells(Rows.Count, "A") is that cell in column A; .End(xlUp) jumps up to the first filled cell; .Row is its row number. .End(xlDown) from A1 stops at the first blank cell instead, so any gap in the column gives too small a number.

Tier	What to say
Common wrong answer	Range("A1").End(xlDown).Row: wrong as soon as the column has a gap
Passes	The bottom-up line, without saying why it's safer
Strong	The bottom-up line, with the reason: a gap can't stop it
Extra points	[+Scale] Rows.Count works whatever the sheet size or file format [+Validate] Debug.Print lastRow on a small table whose answer you know, like this one

Likely follow-ups: The last used column instead? Data that might sit below a gap? The same thing in Python?

Red flag: the top-down version as the final answer, with no mention of the gap problem.

Learn it in: Chapter 19, section 19.5 (the five things you'll write constantly).

Q70-041 • Fix the broken macro (infinite loop)

Level: Mid · **Roles:** AUT, DA

Remember it as: A *Do While* loop with no increment is a promise to run forever.

The interviewer hands you this macro, meant to write "Large" in column K beside every order over ₹20,000:

```
' BROKEN: never finishes
Sub MarkLargeOrders()
  Dim i As Integer
  i = 2
  Do While Cells(i, 1).Value <> ""
    If Cells(i, 9).Value > 20000 Then
      Cells(i, 11).Value = "Large"
    End If
  Loop
End Sub
```

What's wrong: *i* never increases, so `Cells(i, 1)` is always A2, the loop's test never becomes false, and Excel hangs (**Ctrl+Break** stops it).

```
' FIXED
Sub MarkLargeOrders()
  Dim i As Long, lastRow As Long
  lastRow = Cells(Rows.Count, "A").End(xlUp).Row
  For i = 2 To lastRow
    If Cells(i, 9).Value > 20000 Then Cells(i, 11).Value = "Large"
  Next i
End Sub
```

Run on the practice table, it writes **Large** in K5 (order 1004, ₹25,200) and leaves the other cells of column K empty. `For i = 2 To lastRow ... Next i` adds 1 to *i* on every pass and stops after the last row, so it can't run forever.

Tier	What to say
Diagnosis	The missing increment inside the <i>Do While</i> loop
Fix	A bounded <i>For...Next</i> loop instead, with <code>lastRow</code> worked out first
Extra points	[+Edge cases] also switch <code>Integer</code> to <code>Long</code> : <code>Integer</code> stops at 32,767, and many files have more rows [+Scale] cell by cell is slow on big ranges; read into an array, loop in memory, write back once (Q70-043) [+Validate] test on the six-row practice table first, where you know the answer by hand

Likely follow-ups: Speed this up on 50,000 rows? The array version? What if column I holds text instead of a number?

Red flag: adding an increment but not checking that the loop stops at the last row (or can run past it).

Learn it in: Chapter 19, section 19.4 (loops; Long versus Integer).

Q70-043 · Making a VBA macro run faster

Level: Mid · **Roles:** AUT

Remember it as: Arrays not cells first (the biggest win), then screen updating off and calculation manual; restore both at the end.

Answer in one line: read the range into an array, loop in memory, and write back in one block; set `Application.ScreenUpdating = False` and `Application.Calculation = xlCalculationManual` at the start and restore both at the end; never `Select` or `Activate`.

Tier	What to say
Passes	"Avoid unnecessary loops" (no concrete technique)
Strong	The three changes above, biggest win first
Extra points	[+Scale] the array rewrite is often 10-100 times faster on tens of thousands of rows, not a marginal gain [+Validate] time it with <code>Timer</code> before and after each change: measured, not assumed [+Edge cases] restore screen updating and calculation even when an error stops the macro (an error handler), or the workbook is left stuck in manual calculation

Likely follow-ups: Show the read-loop-write pattern. What goes wrong if you forget to restore `ScreenUpdating`? How would pandas do it?

Red flag: no mention of arrays, screen updating or calculation mode.

Learn it in: Chapter 19, section 19.10 (Making macros fast).

Rapid-fire, 70.5

Roles: AUT for every row; DA and BA for the Mid rows.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-044	Personal Macro Workbook: why use it?	A hidden <code>PERSONAL.XLSB</code> that opens with Excel, so a macro runs from any file	[+Business] for your own tools used across many reports, not tied to one workbook	Mid · 19.2
Q70-045	On Error Resume Next: the risk?	Skips a line that errors and carries on	[+Edge cases] used broadly it swallows real bugs too; keep it to one line and follow it with <code>On Error GoTo 0</code>	Mid · 19.6, 19.9
Q70-046	A custom worksheet function (UDF): its limits?	A <code>Function</code> instead of a <code>Sub</code> , returning a value into the calling cell	[+Trade-offs] it can't change other cells or formatting; anything beyond a calculation needs a <code>Sub</code>	Senior · 19.8

#	Question	One-line answer	Extra point	Level · learn it in
Q70-047	Workbook-level vs. worksheet-level events?	Whole-file events (Workbook_Open, Workbook_BeforeSave) / one-sheet events (Worksheet_Change); mostly beyond Chapter 19	[+Edge cases] Worksheet_Change code that writes to its own sheet triggers itself again unless Application.EnableEvents = False	Senior · 19.10
Q70-048	Consolidate every sheet into one summary?	Loop ThisWorkbook.Worksheets with For Each, skip the summary sheet by name, append each sheet's used range	[+Scale] exclude by name, not by position: sheets get renamed and reordered	Senior · 19.4, 19.6
Q70-049	UserForm: when to build one?	A custom dialog for structured, validated input from a non-technical user	[+Business] trades build time for far fewer "typed the date in the wrong cell" requests	Senior · 19.8
Q70-050	Debug a VBA macro step by step?	Breakpoint (F9) → run (F5) → step (F8), watching the Locals window or Debug.Print	[+Signpost] describe it as a workflow in steps: where you stop, what you watch, what you expect to see	Senior · 19.9
Q70-051	Save the active sheet as PDF and email it through Outlook?	ActiveSheet.ExportAsFixedFormat Type:=xlTypePDF, Filename:=pdfPath, then CreateObject("Outlook.Application") → a MailItem → .Attachments.Add pdfPath → .Send	[+Business] one of the highest-value automations in a real job, and Chapter 19's own project	Senior · 19.6, 19.7
Q70-071	VBA or Office Scripts?	VBA: desktop Excel, the full object model, Outlook / Office Scripts: Excel on the web, TypeScript, runs in Power Automate flows	[+Trade-offs] pick by where the file lives and who runs it	Mid · 19.11

70.6 Google Apps Script

Q70-052 · Read a range and write results back, without looping cell by cell

Level: Mid · **Roles:** AUT

Remember it as: Every `getValue()` or `setValue()` is a round trip to Google's servers. Batch them, or the clock (and the quota) runs out.

The interviewer shows the slow version and asks for a faster one that does the same job:

```
// SLOW: one round trip per cell
function markLargeSlow() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Orders');
  for (let i = 2; i <= sheet.getLastRow(); i++) {
    if (sheet.getRange(i, 9).getValue() > 20000) sheet.getRange(i, 11).setValue('Large');
  }
}
```

On the practice table it reads column I six times, one cell at a time, and writes once, to K5.

```
// FAST: one read, work in memory, one write
function markLarge() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Orders');
  const lastRow = sheet.getLastRow();
  if (lastRow < 2) return; // header only: nothing to do
  const values = sheet.getRange(2, 9, lastRow - 1, 3).getValues(); // I2:K, one read
  const flags = values.map(row => [row[2]]); // start from what K already holds
  for (let r = 0; r < values.length; r++) {
    if (values[r][0] > 20000) flags[r][0] = 'Large';
  }
  sheet.getRange(2, 11, flags.length, 1).setValues(flags); // column K, one write
  Logger.log(`${flags.length} rows checked`);
}
```

The **Execution log** shows:

```
6 rows checked
```

and column K ends exactly as the slow version leaves it: **Large** in K5, the other cells unchanged.

- `getRange(2, 9, lastRow - 1, 3)` is row 2, column 9 (I), all the data rows, three columns: I, J and K. Reading K as well is what lets the fast version keep whatever K already holds, as the slow one does.
- `values.map(row => [row[2]])` makes a one-column copy of K: `row[2]` is the third column read, counting from 0.
- The loop changes only the rows over 20,000, and `setValues(flags)` writes column K once.
- `if (lastRow < 2) return;` stops on a sheet with only a header row, where `lastRow - 1` would ask for 0 rows and `getRange` would fail.

Tier	What to say
Passes	The slow version: correct, but on a large sheet it runs into the six-minute limit
Strong	The batch version: one <code>getValues()</code> , work in memory, one <code>setValues()</code>
Extra points	[+Scale] the Apps Script version of Q70-043's VBA array pattern, for the same reason: fewer round trips [+Edge cases] a batch write that sets "" in the rows it doesn't flag would wipe whatever column K held; and a header-only sheet needs the guard [+Validate] run both versions on the practice table and compare column K

Likely follow-ups: What's the execution time limit, and what if a job really needs longer? Run this every night?

Red flag: cell-by-cell reads and writes with no mention of the time limit.

Learn it in: Chapter 19, section 19.12 (Read and write in batches); `map` and `=>` in section 19.13.

Q70-053 · Fix the broken script (duplicate emails on every trigger run)

Level: Mid · **Roles:** AUT

Remember it as: No "already sent" flag means every run re-sends to everyone, forever.

The **Confirmations** sheet of the practice workbook has `order_id`, `customer_name`, `email` and `status` in columns A to D. A daily trigger runs this:

```
// BROKEN: no memory of what's already been sent
function sendConfirmations() {
  const data =
    SpreadsheetApp.getActive().getSheetByName('Confirmations').getDataRange().getValues();
  for (let i = 1; i < data.length; i++) {
    MailApp.sendEmail(data[i][2], 'Order confirmed', 'Thanks for your order!');
  }
}
```

What's wrong: nothing records which rows were already emailed, so every run emails every customer again.

```
// FIXED: a per-row "Sent" flag, checked and set
function sendConfirmations() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Confirmations');
  const data = sheet.getDataRange().getValues();
  const statusCol = 4; // column D
  let sent = 0;
  for (let i = 1; i < data.length; i++) { // row 0 is the header
    if (data[i][statusCol - 1] !== 'Sent') {
      MailApp.sendEmail(data[i][2], 'Order confirmed', 'Thanks for your order!');
      sheet.getRange(i + 1, statusCol).setValue('Sent');
      sent++;
    }
  }
  Logger.log(`${sent} emails sent`);
}
```

Run twice on the three practice rows, the Execution log shows:

```
3 emails sent
0 emails sent
```

`data[i][2]` is the email (column C, counted from 0); `data[i][statusCol - 1]` is column D. `getRange(i + 1, statusCol)` is the same cell in sheet numbering: grid row `i` is sheet row `i + 1`, because the grid counts from 0 and the sheet from 1.

Tier	What to say
Diagnosis	No record of rows already processed
Fix	A "Sent" flag, checked before sending and set straight after
Extra points	[+Validate] run it twice on test data: the second run must send zero [+Edge cases] a failure between <code>sendEmail</code> and <code>setValue</code> re-sends that one row next time, and two overlapping runs can both send; a lock (Apps Script's <code>LockService</code> , beyond this book) stops the overlap [+Business] duplicate customer emails are the automation bug customers notice

Likely follow-ups: Set up the time-driven trigger itself? What happens after a failure halfway through? `MailApp` or `GmailApp` here?

Red flag: not seeing the missing "already processed" record as the root cause.

Learn it in: Chapter 19, section 19.13 (Triggers; an acknowledgement email).

Rapid-fire, 70.6

Roles: AUT for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q70-054	MailApp vs. GmailApp?	Simple, send-only / can also read, search and label, as the user's own Gmail	[+Trade-offs] MailApp is enough for simple outgoing notifications	Mid · 19.13
Q70-055	Time-driven vs. "on edit" trigger?	Runs on a schedule / runs when a user edits a cell	[+Edge cases] the simple <code>onEdit(e)</code> can't send email; that needs an <i>installable</i> trigger	Mid · 19.13
Q70-056	Call an external API from Apps Script?	<code>UrlFetchApp.fetch(url, options)</code> , then <code>JSON.parse()</code> on the reply	[+Business] how a Sheet syncs with a CRM with no server of its own (Chapter 78 goes deeper)	Mid · 19.13
Q70-057	The six-minute limit: what if a job needs longer?	Save how far it got (e.g. the last row done) with <code>PropertiesService</code> , and let the next run carry on from there	[+Scale] process in batches that each finish well inside the limit	Mid · 19.12, 19.13
Q70-058	<code>SpreadsheetApp.flush()</code> : when is it needed?	It applies all pending changes now, instead of when the script ends (beyond Chapter 19)	[+Edge cases] use it when a user or another script must see a change mid-run, such as a progress cell	Mid · 19.12

70.7 DAX and Power BI modeling

Q70-059 · Calculated column vs. measure

Level: Fresher · Roles: BI, DA

Remember it as: *Column = stored, one value per row, computed at refresh. Measure = computed live, changes with whatever's filtered.*

Answer in one line: a calculated column is computed row by row and stored in the model; a measure is computed when a visual asks for it, under whatever filters (a slicer, a date range) apply at that moment.

Tier	What to say
Passes	"They're both formulas in Power BI"
Strong	Net revenue per order line (row-level, stored) is a column; total or average revenue across whatever's filtered is a measure
Extra points	[+Trade-offs] logic about one row → column; anything that adds up or responds to filters → measure [+Scale] calculated columns take memory on large tables, because every row stores a value; measures store nothing [+Business] a measure respects whatever the viewer has filtered, with no extra work

Likely follow-ups: Filter context vs. row context? What does `CALCULATE` do? Would you ever need both for the same value?

Red flag: treating them as interchangeable; not knowing the storage difference.

Learn it in: Chapter 16, section 16.4 (Measures and calculated columns).

Q70-060 · Revenue this month vs. the same month last year

Level: Mid · **Roles:** BI, DA

Remember it as: `CALCULATE` changes the filter context. Everything else in DAX time intelligence is `CALCULATE` wearing a costume.

Answer in one line: two measures, the same as Chapter 16's:

```
Revenue LY = CALCULATE ( [Net Revenue], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
```

```
YoY Growth % = DIVIDE ( [Net Revenue] - [Revenue LY], [Revenue LY] )
```

In Chapter 16's model, the January 2025 row reads Net Revenue ₹8,51,95,521, Revenue LY ₹6,40,63,515 (January 2024) and YoY Growth % **33.0%**, and the same sums in SQL on `riverstone_full` agree.

Tier	What to say
Passes	"You'd filter to last year and compare" (in words, no DAX)
Strong	The two measures above, explaining that <code>CALCULATE</code> changes the filter context and that time intelligence needs a marked date table
Extra points	[+Signpost] "Two measures: last year first, then the growth on top of it" [+Edge cases] <code>DIVIDE</code> , not <code>/</code> : Revenue LY is blank for the first year of data, and <code>DIVIDE</code> returns blank instead of an error [+Validate] check one month you can compute another way (SQL, a pivot) before trusting the whole matrix

Likely follow-ups: `SAMEPERIODLASTYEAR` vs. `DATEADD`? What does a date table need to work? What does `REMOVEFILTERS` do inside `CALCULATE` (Q70-064)?

Red flag: using `/` instead of `DIVIDE`; no mention of the date table; can't say what `CALCULATE` does.

Learn it in: Chapter 16, sections 16.5 (`CALCULATE`) and 16.6 (Time intelligence).

Q70-061 · Star schema, and why it beats one flat table

Level: Mid · **Roles:** BI, DA

Remember it as: One fact table (events), several dimension tables (who, what, when) around it, like a star.

Answer in one line: a central fact table (one row per event, holding the numbers and the keys) surrounded by dimension tables (Customer, Product, Date) in one-to-many relationships: smaller, faster for DAX, and one clear date table for time intelligence.

Tier	What to say
Passes	"It's a way of organizing tables with relationships"
Strong	The fact/dimension structure above, and why it beats a flat table
Extra points	[+Scale] a flat table repeats every attribute on every fact row, which bloats memory and muddles how filters flow [+Trade-offs] for a very small dataset, flat can be fine: the star's benefits grow with the data and the time-intelligence needs [+Evidence] from your own work, if it's true for you: "On my last report, moving from one flat table to a star schema made the pages load noticeably faster, with no measure rewritten"

Likely follow-ups: A snowflake schema: when? Many-to-many relationships? A bridge table?

Red flag: no mention of facts vs. dimensions; claiming flat is always simpler.

Learn it in: Chapter 16, section 16.3 (and Chapter 28, section 28.9, for the warehouse version of the same idea).

Q70-062 · Row-level security (RLS): each manager sees only their own region

Level: Senior · **Roles:** BI

Remember it as: RLS is enforced by the service, per signed-in user: not a filter you apply before publishing.

Answer in one line: under **Modeling** → **Manage roles**, define a role with a DAX filter such as `UserRegion[email] = USERPRINCIPALNAME ()` on a users-and-regions table; once published, each user sees the report filtered to their own identity.

Tier	What to say
Passes	"You'd filter the data by region before publishing" (that isn't RLS at all)
Strong	The role and its DAX filter, applied per signed-in user
Extra points	[+Validate] test with View as in Desktop (Other user + the role), then with a real test account after publishing [+Scale] dynamic RLS (one table of users and regions) needs no change when people join; a static role per region does [+Business] one published report, safely different data per viewer, instead of N copies to maintain

Likely follow-ups: Static vs. dynamic RLS? Why `USERPRINCIPALNAME` rather than `USERNAME`? (In Desktop, `USERNAME` returns `DOMAIN\user`, not the email, so tests in Desktop and the Service behave differently.) How would you test it before rollout?

Red flag: confusing RLS with a report-level filter.

Learn it in: Chapter 16, section 16.10 (Dynamic roles).

Rapid-fire, 70.7

Roles: BI for every row; DA for the Mid rows.

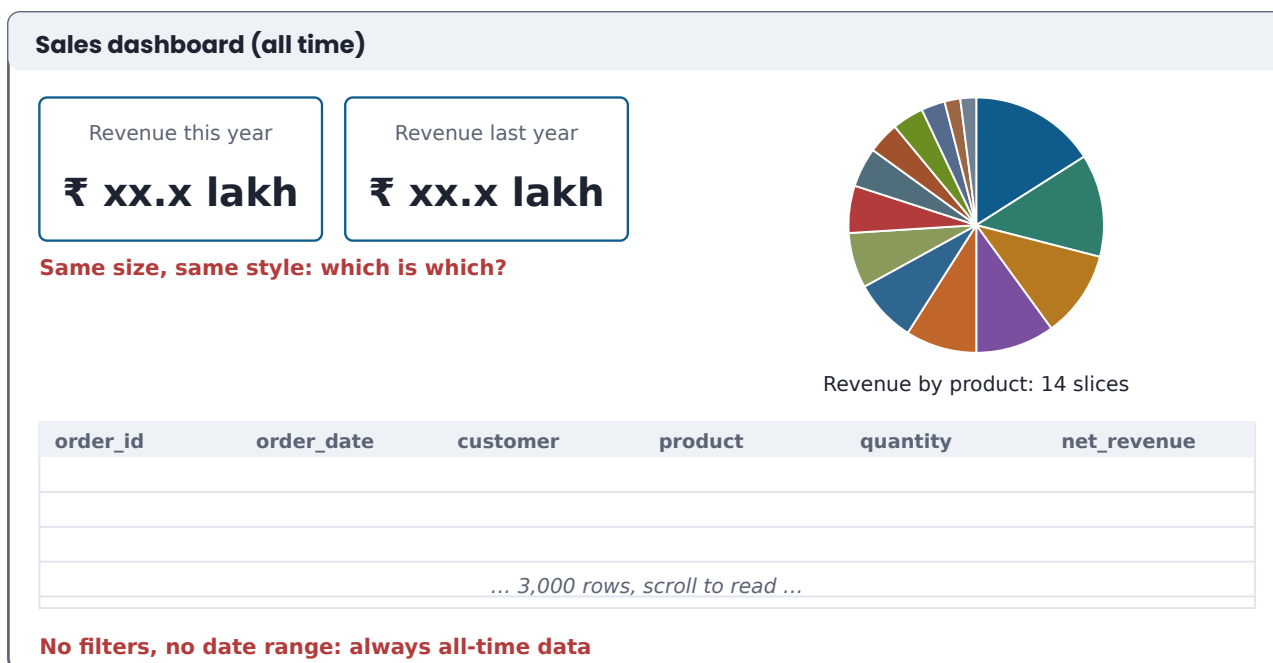
#	Question	One-line answer	Extra point	Level · learn it in
Q70-063	Import mode vs. DirectQuery?	A compressed copy in memory, fast, as fresh as the last refresh / a live query on every interaction, always current, slower	[+Trade-offs] default to Import; DirectQuery only when near-real-time matters more than speed	Mid · 16.2
Q70-064	What do REMOVEFILTERS / ALL do inside CALCULATE?	Remove filters, so the measure sees the unfiltered total; REMOVEFILTERS is the clearer name, ALL the older one you'll meet in inherited models	[+Edge cases] on a whole table they clear every filter on it; name one column (REMOVEFILTERS (Product[category])) to clear only that one	Mid · 16.5
Q70-065	Power BI dashboard vs. report?	Report: one or more interactive pages, built in Desktop / dashboard: one page of visuals pinned from several reports, in the Service	[+Business] an executive wants the dashboard glance; an analyst wants the report to dig into	Mid · 16.1, 16.9
Q70-066	Handle a many-to-many relationship in the model?	Introduce a bridge table, giving two clean one-to-many relationships (or mark the relationship many-to-many); beyond this book, but the idea is Chapter 12's bridge table	[+Trade-offs] a bridge table filters more predictably than a native many-to-many relationship	Senior · 12.1
Q70-067	Power Query vs. DAX: where does a transform belong?	Reshape and clean <i>before</i> loading, the same for every user / calculate <i>in</i> the model, responding to filters	[+Trade-offs] same every time → Power Query; depends on what's selected → DAX	Mid · 16.2, 16.4
Q70-068	Move a Power BI star schema and its measures to Tableau?	The model carries over; DAX measures become calculated fields; SAMEPERIODLASTYEAR-style time intelligence needs more date functions by hand	[+Trade-offs] the modelling idea is portable; measure syntax and time intelligence are what change	Senior · 16.12

70.8 Dashboard critiques

Q70-069 · Critique and rebuild: the "everything on one page, no filters" dashboard

Level: Mid · **Roles:** BI, DA, BA

The dashboard: one page. Two big numbers the same size (this year's revenue and last year's, nothing to tell them apart). A 14-slice pie chart of revenue by product. A 3,000-row raw table below. No filters, no date range: it always shows all-time data. Figure 70.1 is a sketch of it.



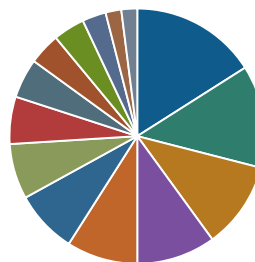
Revenue this year

₹ xx.x lakh

Revenue last year

₹ xx.x lakh

Same size, same style: which is which?



Revenue by product: 14 slices

order_id	order_date	customer	product	quantity	net_revenue
... 3,000 rows, scroll to read ...					

No filters, no date range: always all-time data

Figure 70.1 – The dashboard in Q70-069, as the interviewer describes it.

Problem	Why it matters	Fix
Two identical "big numbers"	The reader has to reread both labels every time to tell them apart	One large current number, with a small "+X% vs. last year" beside it
A 14-slice pie chart	Nobody can compare 14 angles; pies work only for a few parts (about five or six at most)	A sorted horizontal bar chart: the top 8-10 products plus "Other"
A 3,000-row table on the dashboard	A data dump, not a summary; it buries the answer in scrolling	Drill-through or export, not shown by default
No filters, no date range	It can never answer "how's this quarter?" without a rebuild	A date slicer that drives every visual on the page

Why it matters

Fix

Two identical numbers"

The reader has to reread both labels every time to tell them apart

One large current number, with a small "+X% vs. last year" beside it

A 14-slice pie chart

Nobody can compare 14 angles; pies work only for a few parts (about five or six at most)

A sorted horizontal bar chart: the top 8-10 products plus "Other"

A 3,000-row table on the dashboard

A data dump, not a summary; it buries the answer in scrolling

Drill-through or export, not shown by default

No filters, no date range

It can never answer "how's this quarter?" without a rebuild

A date slicer that drives every visual on the page

Extra-point moves shown: **[+Signpost]** organized by problem, not a stream of thoughts. **[+Business]** every fix ties to what the viewer needs to do. **[+Edge cases]** the "Other" bar and the drill-through both allow for more data than fits on one screen.

Learn it in: Chapter 15, sections 15.7 (pies and parts of a whole) and 15.11 (titles and clutter); Chapter 16, section 16.7 (designing the report page).

Q70-070 · Critique and rebuild: the "six gauges and a 3D pie, live on production" dashboard

Level: Senior · **Roles:** BI

The dashboard: six gauges across the top, one per region, showing progress to target. Below, a 3D exploded pie chart of revenue by sales rep. Every chart uses a different, unrelated colour palette. It refreshes live through DirectQuery against the production database. Figure 70.2 is a sketch of it.

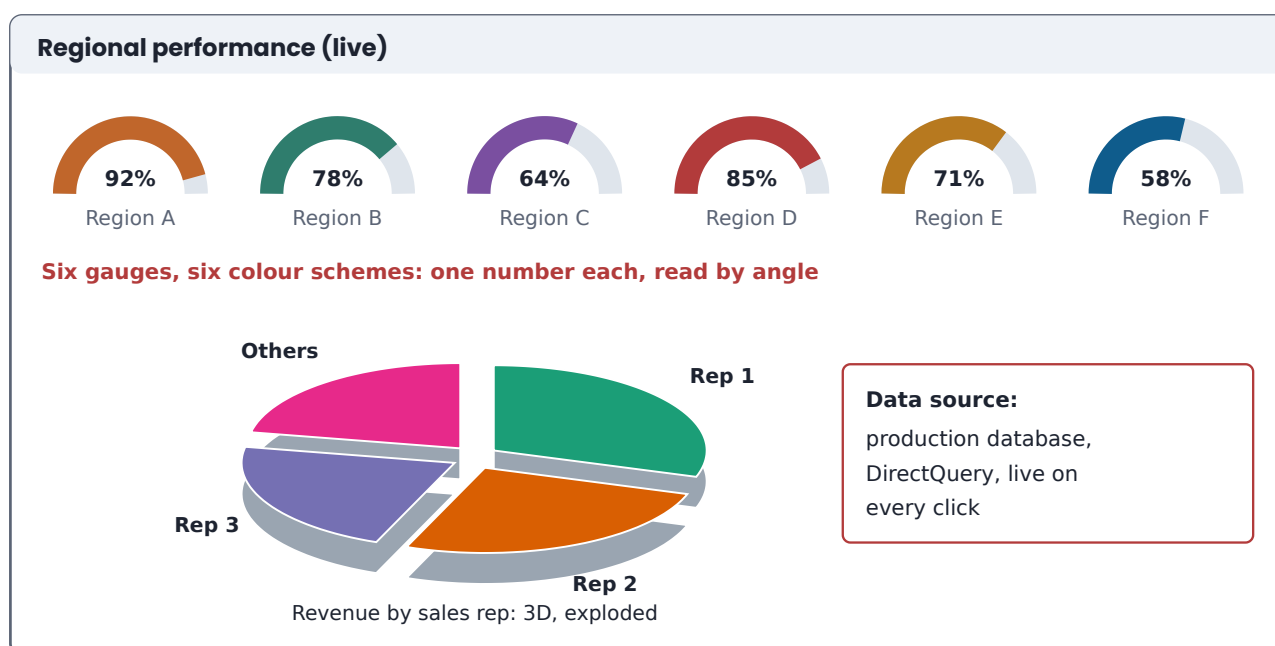


Figure 70.2 – The dashboard in Q70-070, as the interviewer describes it.

What's wrong, fast:

Problem	Why it matters	Fix
Six gauges	A lot of ink for one number each, shown by angle; harder to compare than bars on one axis	Six KPI cards, or one bar chart of % of target by region
A 3D exploded pie	Perspective makes the near slices look bigger than far slices of the same size; pulling slices apart makes it worse	A plain, sorted 2D bar chart
A different palette per chart	The reader must relearn what each colour means on every chart	One palette for the page: the same region in the same colour everywhere
Live DirectQuery for a daily check	Slows the production database for no real gain in freshness	Import mode with a scheduled refresh

Extra-point moves shown: **[+Evidence]** naming the specific, known distortion of 3D pies (Chapter 15's rule), not just "pies are bad". **[+Scale]** linking a chart setting to a real risk for the production system. **[+Business]** every fix asks what the viewer needs, not what "looks unprofessional".

Learn it in: Chapter 15, sections 15.7 (pies), 15.10 (colour with meaning) and 15.12 (misleading charts); Chapter 16, sections 16.2 (Import or DirectQuery) and 16.7 (gauges, and designing the page).

APPLY

Common mistakes

Mistake	Symptom	Fix
VL00KUP with the match type left out	Silent wrong results from approximate matching	Pass FALSE, or use XLOOKUP or INDEX/MATCH

Mistake	Symptom	Fix
Nested IFs in the wrong order	A middle tier "wins" over a higher one that should fire first	Most restrictive first; IFs past two levels
A revenue total that includes cancelled orders	The number is plausible, and too high	Add the status condition (" <code><>Cancelled</code> "), and say so out loud
Pivot source range misses new rows	The grand total quietly misses recent data	Build pivots on Tables, not fixed ranges
VBA looping cell by cell over a large range	The macro is slow or seems to hang	Array read → work in memory → array write
Apps Script looping <code>getValue()</code> / <code>setValue()</code>	The script hits the six-minute limit	Batch with <code>getValues()</code> / <code>setValues()</code>
A DAX calculated column where a measure was needed	The value doesn't respond to report filters	Rewrite it as a measure if it adds up or responds to filters
A dashboard with no date filter	Can't answer "how's this quarter?" without a rebuild	A date slicer driving every visual
A 3D or heavily decorated chart for simple data	It distorts the comparison it's meant to show	The plainest chart that shows the pattern

In the real world: the interview that turned on one broken macro

Arjun, a BA candidate, gets a live-coding round: a VBA macro meant to consolidate three regional files, which stops with "Subscript out of range." He's never seen this exact error before, and says so out loud, then reasons through it: "This usually means I'm referring to something, a sheet name, an array index, that doesn't exist. Let me check what the code asks for against what's actually there." The code refers to `Worksheets("Region2")`; the sheet is named "Region 2," with a space.

He fixes the typo, and then adds one more line, unprompted: a message box reporting how many rows came from each file, so a future user notices at once if a file fails to load. In the feedback call after the offer, the hiring manager singled out that line: "you diagnosed a real error methodically, then thought one step past 'it works now.'"

The lesson: a broken-macro round isn't testing whether you already know that error message. It's testing whether you have a *method* for one you've never seen, and whether you stop at "it runs" or keep going to "it runs, and it'll tell someone if it fails next time."

Project

Goal: one worked example per major topic in this chapter, on your own data or Riverstone's.

Tools you'll need

Excel and Google Sheets (both free at a basic level). **Power BI Desktop** (free to build; publishing has free and paid tiers). **VBA** ships inside Excel; **Google Apps Script** ships inside every Sheet (**Extensions** → **Apps Script**). Nothing beyond what Chapters 10, 11, 15, 16 and 19 already covered (plus Chapter 12 for the SQL comparisons). The practice table is in [companion/ch70/](#).

1. Write three SUMIFS / COUNTIFS formulas answering real questions, each checked against a pivot table built the slow way.
2. Build one Power Query that combines several files (or several sheets), naming every step for its purpose.
3. Write one VBA macro or Apps Script function that would take over ten minutes by hand; time it before and after.
4. Build one Power BI (or other) dashboard, then critique it yourself in this chapter's problem-why-fix table, honestly, before anyone else does.
5. Write one DAX measure (or a Sheets or Tableau equivalent) that responds correctly to a filter, and prove it by changing the filter.

REVIEW

Key terms

SUMIFS / COUNTIFS · XLOOKUP · INDEX / MATCH · absolute vs. relative reference · SUMPRODUCT · dynamic array (FILTER, UNIQUE) · pivot table · calculated field (pivot) · Power Query · Merge vs. Append · QUERY (Sheets) · ARRAYFORMULA · IMPORTRANGE · VBA object model · Personal Macro Workbook · On Error · UserForm · Office Scripts · Apps Script execution limit · getValues() / setValues() batching · installable trigger · calculated column vs. measure (DAX) · CALCULATE · filter context · star schema · row-level security (RLS) · Import mode vs. DirectQuery

Final-week revision list

Q70-001, Q70-002, Q70-003, Q70-005, Q70-016, Q70-018, Q70-027, Q70-028, Q70-034, Q70-035, Q70-040, Q70-041, Q70-043, Q70-052, Q70-053, Q70-059, Q70-060, Q70-061, Q70-062, Q70-069.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, picks up where section 70.3's Merge and Append and section 70.4's QUERY leave off: the same join and aggregation logic, in real SQL.
- **Chapter 78, Automation & Integration Question Bank**, goes further into API integration and scheduling than section 70.6's Apps Script questions.
- **Chapters 10, 11, 15, 16 and 19** teach every technique this bank draws on, in full; this chapter tests them, it doesn't re-teach them.

Chapter 71. SQL Question Bank

Chapter at a glance

You will learn to: answer the SQL questions that come up across screening calls, live-coding rounds, and take-home exercises for every data role · reason through the classic NULL and join traps that catch experienced candidates, not just beginners · write window-function solutions to the second-highest, top-N-per-group, running-total, streak and retention problems that recur across companies · know exactly where PostgreSQL and MySQL disagree · walk out with a 20-question final-week revision list.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point tags). The questions test Chapters 12 and 13 (SQL), Chapter 14 (cleaning), Chapter 27 (NTILE) and Chapter 28 (recursive CTEs, window frames, query plans, indexes, materialized views), plus Chapter 49, section 49.4 (ACID). This chapter tests those skills; it doesn't teach them again.

Time needed: 3–4 hours for a first pass (about 5 minutes per core question, 1 minute per rapid-fire row); 6–8 hours more to run every core question yourself in both databases; 1 hour for the final-week list.

How this chapter is built. Same format as every question bank in Part 8 (Chapters 70–82): every core question leads with a "**Remember it as...**" memory hook, a one-line answer, and a compact tier table, and every rapid-fire section is a scan table. After each question number comes its level: **Warm-up** (sections 71.1–71.3), **Core** (CTEs, window functions, the classic problems, schema and dialect questions) or **Advanced** (recursive CTEs, gaps-and-islands, window frames, transactions, indexes). Rapid-fire tables take the level of their section. Extra-point moves are labelled with Chapter 69's twelve tags, such as **[+Edge cases]**.

Every query is shown in PostgreSQL and was run on PostgreSQL 16. Where MySQL needs different syntax, a MySQL: block follows, and that version was run on MySQL 8.4 LTS. Every result printed below is the database's real answer, kept exactly as returned, including the surprising ones (an empty result, a tied ranking, an index the planner declines to use).

Which database. Most questions use `riverstone_2025`, the one-year practice database from Chapter 13, section 13.1. Question Q71-072 uses the mini database `riverstone` from Chapter 12, section 12.2, and says so. The questions that create tables (Q71-013 and section 71.7) run in `riverstone_lab`, the practice database you created in Chapter 12, section 12.13, and drop their tables at the end. Each **Verified** line names its database.

Learn it in pointers name the chapter and section that teaches the idea. Questions and rapid-fire rows marked **Beyond the book** go further than the teaching chapters; each of those carries its own short explanation.

In this chapter

LEARN	71.1 Core concepts: SELECT, WHERE, and JOIN	72
	71.2 Basic questions that are trickier than they look	75
	71.3 Aggregation: GROUP BY and HAVING	82
	71.4 Subqueries, CTEs, and EXISTS vs. IN	83
	71.5 Window functions: the classics	86
	71.6 Classic problems: duplicates, gaps-and-islands, retention	90
	71.7 Schema, constraints, and transactions	94
	71.8 PostgreSQL vs. MySQL: where the dialects actually differ	101
	71.9 Query optimization and indexes	103
	71.10 Live-coding walk-throughs	106
APPLY	Common mistakes · In the real world · Project	112
REVIEW	Key terms · Final-week revision list	113
NEXT	Where this leads	114

LEARN

71.1 Core concepts: SELECT, WHERE, and JOIN

Q71-001 · Find customers who have never placed an order · Warm-up

Remember it as: *LEFT JOIN + IS NULL, or NOT EXISTS. Never NOT IN if the other side might have a NULL.*

The query:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Verified (riverstone_2025): 1 of 24 customers has never ordered.

```
customer_id | customer_name
-----+-----
          16 | Home Plus
(1 row)
```

Tier	What to say
Passes	WHERE customer_id NOT IN (SELECT customer_id FROM orders): works on clean data, has a hidden flaw (Q71-002)
Strong	The LEFT JOIN ... IS NULL version above, filtering on the <i>right table's primary key</i> , not an arbitrary column
Extra points	[+Clarify] should a customer whose only order was cancelled count as "never ordered"? · [+Validate] total customers = customers with orders + customers without, checked: 24 = 23 + 1 ✓ · [+Business] add the signup date and sort by it, so sales can follow up the newest non-converters first

Likely follow-ups: No orders in the last 90 days, not ever? Products never sold? Rewrite without a join?

Red flag: `WHERE o.customer_id = NULL` (should be `IS NULL`); filtering a left join's right-table column in `WHERE` in a way that silently turns it into an inner join. The one deliberate exception is `IS NULL` on the right table's key: that is what makes this an anti-join (Chapter 12, section 12.10). Any other condition on the right table belongs in `ON`.

Learn it in: Chapter 12, section 12.10 (the anti-join).

Q71-002 · Why is `NOT IN` dangerous with a subquery that might contain a `NULL`? · Warm-up

Remember it as: One `NULL` in a `NOT IN` subquery poisons the whole comparison: the query returns nothing, silently.

Answer in one line: If the subquery's result contains even one `NULL`, `NOT IN` returns zero rows for every outer row, because SQL can't prove any value is "not equal to `NULL`": the comparison evaluates to `UNKNOWN`, not `TRUE`, and `NOT IN` needs every comparison to be `TRUE`. (Q71-009 shows the root cause: `NULL = NULL` isn't true either.)

Verified (riverstone_2025): the `orders.sales_rep_id` column has 11 `NULL` rows (unassigned orders). The trap, live:

```
SELECT count(*)
FROM employees
WHERE employee_id NOT IN (SELECT sales_rep_id FROM orders);
```

```
count
-----
      0
(1 row)
```

The same question, asked correctly:

```
SELECT count(*)
FROM employees
WHERE employee_id NOT IN (SELECT sales_rep_id
                          FROM orders
                          WHERE sales_rep_id IS NOT NULL);
```

```
count
-----
      2
(1 row)
```

Tier	What to say
Passes	"NOT IN can behave oddly with NULLs" (vague, no mechanism)
Strong	The UNKNOWN-comparison explanation above, with the fix (WHERE ... IS NOT NULL inside the subquery)
Extra points	[+Validate] the exact live numbers above: 0 rows (wrong) vs. 2 rows (right) on the same data, same question · [+Trade-offs] NOT EXISTS and LEFT JOIN ... IS NULL don't have this problem at all, since they test whether a matching row exists rather than comparing a value against a list that holds a NULL: prefer them by default

Likely follow-ups: Does IN (not NOT IN) have the same problem? (No: IN with a NULL in the list just doesn't match the NULL; it doesn't empty the whole result.) How would you find this bug in someone else's query?

Red flag: not knowing this is a real, live trap, not a theoretical one: 11 real NULL rows just demonstrated it.

Learn it in: Chapter 12, section 12.12 (the "NOT IN and NULLs" box).

Rapid-fire, §71.1 (Warm-up)

#	Question	One-line answer	Extra point
Q71-003	WHERE vs. HAVING?	Filters rows before grouping / filters groups after aggregation	[+Edge cases] you can't reference an aggregate function in WHERE: that's precisely what HAVING is for → Ch 12 §12.9
Q71-004	INNER JOIN vs. LEFT JOIN?	Only matching rows from both tables / all left-table rows, NULLs where there's no match	[+Business] a LEFT JOIN that unexpectedly shrinks the row count usually means a filter on the right table leaked into WHERE instead of ON → Ch 12 §12.10
Q71-005	What does SELECT DISTINCT actually do?	Removes duplicate <i>entire rows</i> from the result, not duplicates in one column alone	[+Edge cases] SELECT DISTINCT a, b removes duplicates of the combination of a and b, not of each column on its own → Ch 12 §12.5
Q71-006	UNION vs. UNION ALL?	UNION removes duplicate rows across the combined result / UNION ALL keeps everything, no duplicate check (verified: 2 identical rows → 2 with UNION ALL, 1 with UNION)	[+Scale] UNION ALL is faster, since it skips the duplicate check: use it whenever you know the two sets can't overlap → Ch 12 §12.12
Q71-007	BETWEEN: is it inclusive or exclusive?	Inclusive on both ends	[+Edge cases] BETWEEN '2025-01-01' AND '2025-01-31' on a TIMESTAMP column silently misses rows on the 31st after midnight: cast to DATE first, or use < '2025-02-01' → Ch 12 §12.6; the timestamp case is <i>beyond the book</i>
Q71-008	What's a correlated subquery?	A subquery that references a column from the outer query, re-evaluated once per outer row (e.g., each customer's own most recent order date)	[+Trade-offs] correlated subqueries can be slow at scale (one execution per row); a window function or a join often expresses the same logic faster → Ch 12 §12.12

71.2 Basic questions that are trickier than they look

Every one of these sounds like a warm-up question. Every one of them has caught experienced candidates, because the trap is in something they assumed rather than something they didn't know. They mix theory and practice, and each one is shown live wherever a query can demonstrate the point.

Q71-009 · Does `NULL = NULL` evaluate to true? · Warm-up

Remember it as: `NULL` isn't a value, it's the absence of one. You can't ask whether two absences are "equal."

Answer in one line: No: `NULL = NULL` evaluates to `NULL` (unknown), not `TRUE`, which is exactly why `WHERE column = NULL` never matches anything and `IS NULL` exists as a separate operator.

Verified (riverstone_2025):

```
SELECT (NULL = NULL) AS result,  
       (NULL = NULL) IS NULL AS is_this_null;
```

```
result | is_this_null  
-----+-----  
      | t  
(1 row)
```

PostgreSQL prints `NULL` as a blank and true as `t`. MySQL runs the same query and prints `NULL` and `1`, since it stores true as the number `1` (Q71-066).

Tier	What to say
Passes	"NULL equals NULL" (the actual trap, stated as fact)
Strong	Correctly says no, it evaluates to <code>NULL</code> /unknown, and explains why <code>= NULL</code> in a <code>WHERE</code> clause silently matches nothing
Extra points	[+Validate] the query above proves it two ways at once: the comparison itself shows blank (<code>NULL</code>), and wrapping it in <code>IS NULL</code> confirms that blank really is <code>NULL</code> , not empty text · [+Business] this is exactly the mechanism behind Q71-002's <code>NOT IN</code> trap: one root cause, two different symptoms

Likely follow-ups: How do you check for `NULL` correctly? What does `NULL AND FALSE` evaluate to? (`FALSE`: one known-false input is enough to decide an `AND`, even with an unknown on the other side.)

Red flag: confidently stating `NULL = NULL` is true.

Learn it in: Chapter 12, section 12.7.

Q71-010 · Can you use a column alias from `SELECT` inside the same query's `WHERE` clause? · Warm-up

Remember it as: `WHERE` runs before `SELECT` ever names anything. You can't filter on a name that doesn't exist yet.

Answer in one line: No: SQL's logical order runs `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`, so a `WHERE` clause runs before `SELECT` has assigned any alias, and referencing one errors.

Verified (riverstone_2025):

```
SELECT customer_name, city AS loc
FROM customers
WHERE loc = 'Mumbai';
```

ERROR: column "loc" does not exist

MySQL: the same query fails with its own wording:

```
SELECT customer_name, city AS loc
FROM customers
WHERE loc = 'Mumbai';
```

ERROR 1054 (42S22): Unknown column 'loc' in 'where clause'

Tier	What to say
Passes	"You can't use an alias in WHERE" (correct, no reason given)
Strong	+ explains the logical order, and that this is <i>why</i> the alias doesn't exist yet at the point WHERE runs
Extra points	[+Validate] the real error messages above, not a guess at what they might say · [+Edge cases] ORDER BY runs <i>after</i> SELECT, so aliases work fine there: the same alias that errors in WHERE is legal in ORDER BY, which is worth saying out loud since it looks inconsistent until you know the order

Likely follow-ups: Does GROUP BY allow an alias? (Yes, in PostgreSQL and MySQL, though not in every SQL engine.) How would you filter on a computed value if not in WHERE? (A CTE or subquery, filtering the outer layer: the same pattern Q71-033 and Q71-035 use for window functions.)

Red flag: not knowing SQL has a logical order that differs from the order it's written in.

Learn it in: Chapter 12, section 12.11.

Q71-011 · Why does sorting '9', '10', '2' as text give a different order than sorting them as numbers? · Warm-up

Remember it as: Text sorts character by character. "10" starts with "1," which comes before "2" and "9": the number itself is never considered.

Answer in one line: As text, comparison goes character by character, so '10' sorts before '2' because '1' < '2' as the very first character; as numbers, ordinary numeric comparison applies and 2 < 9 < 10 as expected.

Verified (riverstone_2025), the same three values two ways. As text:

```
SELECT val
FROM (VALUES ('9'), ('10'), ('2')) AS t(val)
ORDER BY val;
```

```

val
-----
10
2
9
(3 rows)

```

As integers:

```

SELECT val
FROM (VALUES (9), (10), (2)) AS t(val)
ORDER BY val;

```

```

val
-----
2
9
10
(3 rows)

```

MySQL: a list of literal rows is written `VALUES ROW(...)`, and its column is always called `column_0`:

```

SELECT column_0
FROM (VALUES ROW('9'), ROW('10'), ROW('2')) AS t
ORDER BY column_0;

```

```

+-----+
| column_0 |
+-----+
| 10      |
| 2       |
| 9       |
+-----+

```

Tier	What to say
Passes	"Text and numbers sort differently" (true, no mechanism)
Strong	The character-by-character explanation above, with the real output showing both orders on the same three values
Extra points	[+Business] this bug shows up whenever a numeric code gets stored as <code>VARCHAR</code> : order numbers, SKUs, PIN codes. A report silently sorts "wrong" with no error anywhere · [+Validate] cast to confirm: <code>ORDER BY val::int</code> on the text version recovers the numeric order (MySQL: <code>ORDER BY CAST(column_0 AS SIGNED)</code>), which is itself proof of what was happening

Likely follow-ups: How would you fix a column that's the wrong type after the fact? What about sorting mixed codes like "A10" vs "A9"?

Red flag: unable to explain *why* it happens, only that it does.

Learn it in: Chapter 12, section 12.1 (why types matter) and section 12.8 (`CAST` and `::`).

Q71-012 · What's the difference between DELETE, TRUNCATE, and DROP? · Warm-up

Remember it as: DELETE removes rows, one at a time, with a WHERE if you want. TRUNCATE empties the whole table, fast, no WHERE. DROP removes the table itself, structure and all.

Answer in one line: DELETE FROM t WHERE ... removes matching rows (or all rows with no WHERE), logged row by row, and can be rolled back inside a transaction; TRUNCATE TABLE t empties the entire table at once, much faster, minimally logged. In MySQL it also resets AUTO_INCREMENT; in PostgreSQL the identity or sequence keeps counting unless you write TRUNCATE TABLE t RESTART IDENTITY. DROP TABLE t removes the table's structure entirely, data and definition both gone.

Tier	What to say
Passes	"They all delete data"
Strong	The three distinct scopes and behaviors above
Extra points	[+Edge cases] TRUNCATE can't take a WHERE clause: it's all rows or nothing, by design · [+Business] the classic production incident is a DELETE whose WHERE got lost in a copy-paste, which empties the table. Inside BEGIN ... ROLLBACK you can undo it. TRUNCATE in MySQL commits immediately, so there is no undo · [+Trade-offs] DELETE fires row-level triggers (if any exist) and can be rolled back mid-transaction; TRUNCATE skips row triggers, and in MySQL it can't be rolled back

Likely follow-ups: Can TRUNCATE be rolled back? (In PostgreSQL, yes, inside a transaction; in MySQL, no: it commits implicitly.) Does TRUNCATE restart the ID counter? (MySQL: yes. PostgreSQL: only with RESTART IDENTITY; checked on both.) What happens to foreign-key-referenced rows on each?

Red flag: treating all three as interchangeable ways to "clear a table."

Learn it in: Chapter 12, section 12.13 (Step 8, transactions, and Step 11, TRUNCATE and DROP). RESTART IDENTITY is beyond the book.

Q71-013 · Does a UNIQUE constraint allow more than one NULL value in a column? · Warm-up

Remember it as: NULL never equals NULL (Q71-009): so a UNIQUE constraint, which is really an equality check, can't call two NULLs "duplicates" of each other.

Answer in one line: Yes, in both PostgreSQL and MySQL: a UNIQUE constraint allows any number of NULL values in that column, because uniqueness is checked by equality, and no two NULLs are ever equal to each other.

Verified, in riverstone_lab (same statements in both databases):

```
CREATE TABLE test_unique (id INT, email VARCHAR(50) UNIQUE);
INSERT INTO test_unique VALUES (1, NULL);
INSERT INTO test_unique VALUES (2, NULL);
SELECT * FROM test_unique;
```

```
id | email
----+-----
 1 | 
 2 | 
(2 rows)
```

MySQL:

```
+-----+-----+
| id  | email |
+-----+-----+
|  1  | NULL  |
|  2  | NULL  |
+-----+-----+
```

Both inserts succeeded: a surprising result to anyone who reads "unique" as "no duplicates, including blanks." Tidy up (same in both):

```
DROP TABLE test_unique;
```

Tier	What to say
Passes	"UNIQUE means no duplicates" (true for non-NULL values, misses the NULL exception entirely)
Strong	Correctly states multiple NULLs are allowed, and ties it back to <code>NULL = NULL</code> being unknown, not true
Extra points	[+Validate] the live proof above: two NULL emails, one UNIQUE constraint, zero errors · [+Business] this is why a "unique email" constraint alone doesn't stop two customer records both missing an email address from silently coexisting: a data-quality gap that looks like the schema should prevent it, and doesn't

Likely follow-ups: How would you actually prevent multiple blank emails if that's the real requirement? (Add `NOT NULL` alongside `UNIQUE`, or use a partial unique index. PostgreSQL 15 and later also offer `email VARCHAR(50) UNIQUE NULLS NOT DISTINCT`, which treats NULLs as equal for this one constraint, so the second NULL is rejected; checked on PostgreSQL 16.) Does a `PRIMARY KEY` allow NULLs the same way? (No: a primary key is `UNIQUE` plus `NOT NULL`, so it never allows even one NULL.)

Red flag: confidently claiming a `UNIQUE` column can never have more than one blank value.

Learn it in: Chapter 12, section 12.13 (Step 3, constraints) and section 12.7 (NULL). `NULLS NOT DISTINCT` is beyond the book.

Q71-014 · What happens if you self-join a table without giving it two different aliases? · Warm-up

Remember it as: The database can't tell which "employees" you mean if you never gave it two different names to tell them apart.

Answer in one line: It errors. PostgreSQL rejects the `FROM` clause itself (table name "employees" specified more than once); MySQL says `Not unique table/alias`. Without two aliases there's no way to say which copy each column comes from.

Verified (riverstone_2025):

```
SELECT employee_id
FROM employees
JOIN employees ON employee_id = manager_id;
```

```
ERROR:  table name "employees" specified more than once
```

MySQL:

```
SELECT employee_id
FROM employees
JOIN employees ON employee_id = manager_id;
```

```
ERROR 1066 (42000): Not unique table/alias: 'employees'
```

Tier	What to say
Passes	"You need aliases for a self-join" (correct, no example of what breaks without one)
Strong	The explanation above, plus the correctly aliased version (<code>FROM employees e JOIN employees m ON e.manager_id = m.employee_id</code>), which is exactly Chapter 12's self-join (section 12.10), and the base of Q71-029's hierarchy query
Extra points	[+Validate] the real error messages above, which are a different, earlier failure than the "ambiguous column" error a beginner might expect (and that some other engines, such as SQLite, do report): worth knowing so you recognize it instantly

Likely follow-ups: Would this also happen with a regular join between two *different* tables that share a column name? (Only for the shared column: you'd get an "ambiguous column" error on that reference, not on the table.)

Red flag: not immediately recognizing that a self-join needs two aliases.

Learn it in: Chapter 12, section 12.10 (the self-join).

Q71-015 · Why does `SELECT *` with a `GROUP BY` often error, even though it "looks" fine? · Warm-up

Remember it as: `GROUP BY` collapses many rows into one per group. `SELECT *` asks for every column, including ones that could hold many different values within that one group.

Answer in one line: Every non-aggregated column in `SELECT` must also appear in `GROUP BY`, because the database can't know which single value to show for a column that wasn't grouped and might differ across the rows being collapsed into one.

Verified (riverstone_2025):

```
SELECT * FROM orders GROUP BY customer_id;
```

```
ERROR: column "orders.order_id" must appear in the GROUP BY clause or be used in an aggregate function
```

MySQL:

```
SELECT * FROM orders GROUP BY customer_id;
```

```
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'riverstone_2025.orders.order_id' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

Tier	What to say
Passes	"SELECT * doesn't work well with GROUP BY" (true, no reason)
Strong	The collapsing-rows explanation above, plus the real error naming the exact offending column
Extra points	[+Edge cases] since MySQL 5.7.5, ONLY_FULL_GROUP_BY is on by default, so modern MySQL errors too; very old MySQL, or a server with that mode switched off, silently picks an arbitrary row's value instead, which is far more dangerous than an error. Both engines do accept ungrouped columns that depend on a grouped primary key: SELECT * FROM orders GROUP BY order_id runs in both (checked) · [+Business] a real reason SELECT * is a bad habit in any query with GROUP BY: list only the columns you need and can justify

Likely follow-ups: What SQL mode controls this behavior in MySQL? How would you fix this query to actually work?

Red flag: not knowing that a MySQL server with ONLY_FULL_GROUP_BY switched off returns a plausible-looking but unreliable answer instead of an error.

Learn it in: Chapter 12, section 12.9 (its dialect note on ONLY_FULL_GROUP_BY).

Rapid-fire, §71.2 (Warm-up): more basic-but-tricky theory and practice

#	Question	One-line answer	Extra point
Q71-016	Is SQL case-sensitive?	Keywords (SELECT, WHERE) are not; whether your <i>data</i> comparisons are case-sensitive depends on the column's collation, which differs by engine and setup	[+Edge cases] WHERE status = 'delivered' matches 'Delivered' in MySQL's default collation and not in PostgreSQL: never assume either way without checking → Ch 12 §12.6 and §12.16
Q71-017	CHAR vs. VARCHAR?	CHAR(n) is fixed-length, padded with spaces to exactly n characters; VARCHAR(n) is variable-length, up to n characters, no padding	[+Business] CHAR padding can break an exact-string comparison against a value that wasn't padded the same way in another system → <i>Beyond the book</i>
Q71-018	Do you need a semicolon at the end of a SQL statement?	Required to separate multiple statements in one script; often optional for a single statement in an interactive client	[+Edge cases] in a multi-statement script a missing semicolon usually makes the next statement a syntax error, and the error points at the wrong line, which makes it confusing to debug → Ch 12 §12.4
Q71-019	Is a FOREIGN KEY required for a JOIN to work?	No: a JOIN is just a condition in ON; it works with or without a declared foreign-key constraint	[+Business] a missing foreign key doesn't stop joins; it stops the database from <i>rejecting</i> orphaned data before the join ever runs: the constraint is a data-quality guard, not a join requirement → Ch 12 §12.10 and §12.13
Q71-020	Can two rows be "duplicates" if they have different primary keys?	Yes: a primary key only guarantees that column is unique; every other column can be identical across two rows	[+Edge cases] this is why "remove duplicates" almost always means "duplicates ignoring the ID column," which needs to be said explicitly → Ch 14 §14.4
Q71-021	Does LIKE '%text%' match case-sensitively?	Depends on the collation, same root cause as Q71-016; PostgreSQL offers ILIKE for a guaranteed case-insensitive match	[+Edge cases] when in doubt, wrap both sides in LOWER() for a comparison that ignores case under any collation → Ch 12 §12.6

#	Question	One-line answer	Extra point
Q71-022	How does an empty string ('') compare with NULL?	They are not the same thing: '' IS NULL is false, and '' = NULL is (per Q71-009) unknown, never true	[+Edge cases] a form that saves a blank text field as '' instead of NULL will silently slip past every IS NULL check written to catch missing data → Ch 12 §12.7

71.3 Aggregation: GROUP BY and HAVING

Q71-023 · Total quantity sold per category, only categories over 500 units · Warm-up

Remember it as: WHERE filters rows before the grouping happens. HAVING filters groups after.

The query:

```
SELECT p.category, SUM(oi.quantity) AS total_qty
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
HAVING SUM(oi.quantity) > 500
ORDER BY total_qty DESC;
```

Verified (riverstone_2025): three of the four categories pass; Furniture sold 30 units and is correctly left out.

```
category | total_qty
-----+-----
Storage  |      5350
Kitchen  |      3675
Industrial |       625
(3 rows)
```

Tier	What to say
Passes	The correct GROUP BY + HAVING query, but can't explain why WHERE can't do the same job
Strong	The query above, plus the reason: WHERE runs before the groups exist, so there is no SUM yet to test
Extra points	[+Validate] run it once without HAVING: four categories, Furniture at 30 units, so its absence is the filter working, not a bug · [+Business] this shape (group, sum, filter on the sum) is the core of almost every "which segments matter" business question

Likely follow-ups: Same, but only the top 2 categories regardless of a fixed threshold? Add a customer-segment breakdown too?

Red flag: trying to filter an aggregate in WHERE (WHERE SUM(quantity) > 500 errors, because aggregates can't appear in WHERE).

Learn it in: Chapter 12, section 12.9.

Rapid-fire, §71.3 (Warm-up)

#	Question	One-line answer	Extra point
Q71-024	Does COUNT(*) differ from COUNT(column)?	COUNT(*) counts all rows / COUNT(column) counts only the non-NULL values in that column	[+Edge cases] on riverstone_2025, COUNT(*) on orders is 175 and COUNT(sales_rep_id) is 164: the 11 NULL rows are silently left out (verified) → Ch 12 §12.9
Q71-025	Can you GROUP BY a column not in the SELECT list?	Yes, in both PostgreSQL and MySQL: you can group by something you don't display	[+Edge cases] the reverse is stricter: every non-aggregated SELECT column must appear in GROUP BY, or the query errors (Q71-015) → Ch 12 §12.9
Q71-026	What does GROUP BY 1, 2 mean?	Groups by the 1st and 2nd columns of the SELECT list, by position, not by name	[+Trade-offs] convenient for a long expression, but fragile: reordering the SELECT columns silently changes what's grouped → <i>Beyond the book</i>
Q71-027	How would you count DISTINCT customers per month?	SELECT DATE_TRUNC('month', order_date)::date AS month, COUNT(DISTINCT customer_id) FROM orders GROUP BY 1 (MySQL: DATE_FORMAT(order_date, '%Y-%m-01') in place of DATE_TRUNC('month', order_date)::date); both give 5, 8 and 10 customers for January to March 2025	[+Business] distinct customers per month, not the order count, is the number that answers "is our customer base growing" → Ch 12 §12.8 (DATE_TRUNC) and §12.9 (COUNT(DISTINCT ...))

71.4 Subqueries, CTEs, and EXISTS vs. IN

Q71-028 · Rewrite a nested subquery as a CTE, and explain why you would · Core

Remember it as: A CTE is a subquery with a name and a place to breathe. Same result, easier to read and debug one step at a time.

Answer in one line: WITH name AS (...) SELECT ... FROM name names an intermediate result so it can be read top to bottom and reused, instead of nesting parentheses inside parentheses.

The query:

```
WITH monthly_revenue AS (  
  SELECT DATE_TRUNC('month', o.order_date)::date AS month,  
         ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue  
  FROM orders o  
  JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status <> 'Cancelled'  
  GROUP BY 1  
)  
SELECT month, revenue  
FROM monthly_revenue  
ORDER BY month  
LIMIT 3;
```

Verified (riverstone_2025), the first three months:

```
month | revenue
-----+-----
2025-01-01 | 202640.00
2025-02-01 | 253664.00
2025-03-01 | 278007.50
(3 rows)
```

MySQL: replace the month line with `DATE_FORMAT(o.order_date, '%Y-%m-01') AS month`; the rest is unchanged, and it returns the same three rows.

```
WITH monthly_revenue AS (
  SELECT DATE_FORMAT(o.order_date, '%Y-%m-01') AS month,
         ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY 1
)
SELECT month, revenue
FROM monthly_revenue
ORDER BY month
LIMIT 3;
```

Tier	What to say
Passes	A nested subquery achieving the same result: correct, harder to read past two levels
Strong	The CTE version above, one logical step at a time
Extra points	[+Validate] the three months match Chapter 13's monthly revenue (₹2,02,640, ₹2,53,664, ₹2,78,007.50) · [+Scale] several CTEs can chain (<code>WITH a AS (...), b AS (...)</code>), each building on the last, which is how a long query stays readable (Q71-042 uses three)

Likely follow-ups: Recursive CTE: when would you need one? (Q71-029.) Does a CTE get computed once, or folded into the main query, and does that matter for performance?

Red flag: nesting subqueries three or four levels deep when a CTE would make the same logic readable.

Learn it in: Chapter 13, section 13.2.

Q71-029 · Write a recursive CTE to show Riverstone's full management hierarchy · Advanced

Remember it as: A recursive CTE is a loop: start with the base case (the top of the tree), then repeatedly join back to itself for the next level down.

The query:

```
WITH RECURSIVE hierarchy AS (  
  SELECT employee_id, employee_name, manager_id, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL          -- anchor: the top of the tree  
  UNION ALL  
  SELECT e.employee_id, e.employee_name, e.manager_id, h.level + 1  
  FROM employees e  
  JOIN hierarchy h ON e.manager_id = h.employee_id -- recursive step  
  WHERE h.level < 20                -- safety stop  
)  
SELECT employee_name, level  
FROM hierarchy  
ORDER BY level, employee_name;
```

Verified (riverstone_2025; the mini database has the same five employees):

```
employee_name | level  
-----+-----  
Anita Rao     |     1  
Farah Khan    |     2  
Vikram Singh  |     2  
Neha Kulkarni |     3  
Rahul Mehta   |     3  
(5 rows)
```

MySQL 8 runs the same query unchanged and returns the same five rows.

Tier	What to say
Passes	Writes the recursive CTE with help, but can't name the anchor and recursive parts or the stopping condition
Strong	The structure above: an anchor query, UNION ALL, and a recursive part that joins back to the CTE itself, stopping when a level finds no new rows
Extra points	[+Validate] the output is a correct three-level tree: Anita Rao at the top, her two direct reports, and Vikram Singh's two reports below him · [+Edge cases] a cycle in the data (A manages B, B manages A) makes the recursion run forever. MySQL stops a runaway recursion at <code>cte_max_recursion_depth</code> (1,000 by default); PostgreSQL has no limit, so add a cap such as <code>WHERE h.level < 20</code>

Likely follow-ups: Find everyone reporting up to a specific manager, at any depth? What if the hierarchy has a cycle by mistake?

Red flag: not knowing the anchor / recursive-part structure at all.

Learn it in: Chapter 28, section 28.2.

Rapid-fire, §71.4 (Core)

#	Question	One-line answer	Extra point
Q71-030	EXISTS vs. IN: when do they differ in behavior?	IN and EXISTS return the same rows, even with NULLs. They differ when negated: NOT IN returns nothing if the subquery holds a NULL (Q71-002); NOT EXISTS doesn't. Verified: EXISTS and IN both give 23 customers	[+Trade-offs] EXISTS can stop at the first match for each row, which is often faster on a large subquery; prefer NOT EXISTS to NOT IN by default → Ch 12 §12.12

#	Question	One-line answer	Extra point
Q71-031	What's a scalar subquery?	A subquery that returns exactly one row and one column, used anywhere a single value is expected	[+Edge cases] if it returns more than one row, the query errors when it runs, not when you write it → Ch 12 §12.12
Q71-032	Each customer's most recent order date, without a GROUP BY?	A correlated scalar subquery: <code>SELECT c.customer_name, (SELECT MAX(order_date) FROM orders o WHERE o.customer_id = c.customer_id) FROM customers c</code>	[+Trade-offs] correct and simple to read; a <code>LEFT JOIN</code> to a pre-aggregated subquery is usually faster on a large table, since it avoids one execution per outer row → Ch 12 §12.12

71.5 Window functions: the classics

Q71-033 · Find the second-highest-priced product · Core

Remember it as: `LIMIT/OFFSET` breaks on ties. `DENSE_RANK` doesn't.

The query:

```
SELECT product_name, unit_price
FROM (
  SELECT product_name, unit_price,
         DENSE_RANK() OVER (ORDER BY unit_price DESC) AS rnk
  FROM products
) t
WHERE rnk = 2;
```

Verified (riverstone_2025):

```
product_name | unit_price
-----+-----
Garden Chair |    1150.00
(1 row)
```

Tier	What to say
Passes	<code>ORDER BY unit_price DESC LIMIT 1 OFFSET 1</code> : gives the same right answer here, and silently breaks if two products tie for first place (<code>OFFSET 1</code> would then return one of the <i>tied firsts</i> , not the second-highest price)
Strong	The <code>DENSE_RANK</code> version above, which is safe on ties
Extra points	[+Edge cases] <code>RANK</code> vs. <code>DENSE_RANK</code> matters here: <code>RANK</code> skips a number after a tie (1, 1, 3), so <code>rnk = 2</code> could return zero rows if two products tied for first; <code>DENSE_RANK</code> (1, 1, 2) has no gap · [+Validate] on Riverstone's real (untied) top prices both approaches agree, which is exactly why this bug hides until the data changes

Likely follow-ups: Third-highest? Second-highest *per category*, not overall? What if there's a tie for second place itself: should both show?

Red flag: using `LIMIT/OFFSET` with no acknowledgment that it breaks on ties.

Learn it in: Chapter 13, section 13.4. (`OFFSET` itself is *beyond the book*.)

Q71-034 · ROW_NUMBER, RANK, and DENSE_RANK: demonstrate the difference on real tied data · Core

Remember it as: ROW_NUMBER never ties (1,2,3). RANK ties and skips (1,1,3). DENSE_RANK ties and doesn't skip (1,1,2).

Verified (riverstone_2025), on two customers tied at 16 orders each:

```
SELECT c.customer_name, COUNT(*) AS order_count,
       ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC, c.customer_name) AS rn,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS drnk
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY order_count DESC, c.customer_name
LIMIT 4;
```

customer_name	order_count	rn	rnk	drnk
Green Leaf Hotels	17	1	1	1
Metro Mart	16	2	2	2
Sharma Hardware	16	3	2	2
Northgate Distributors	13	4	4	3

(4 rows)

The ORDER BY inside ROW_NUMBER ends with c.customer_name, a **tie-breaker**: without it, which of the two tied customers gets 2 and which gets 3 is up to the database. Grouping by c.customer_id as well as the name keeps two customers who happen to share a name apart.

Tier	What to say
Passes	Names all three functions, can't explain how they differ on a tie
Strong	Reads the table above correctly: RANK gives both tied customers rank 2, then <i>skips</i> to 4; DENSE_RANK gives them both 2, then continues at 3 with no gap
Extra points	[+Validate] this is real tied data, not a made-up example: Metro Mart and Sharma Hardware really do have 16 orders each · [+Edge cases] ROW_NUMBER on a tie is non-deterministic unless you add a tie-breaker; two runs can disagree · [+Business] "top 3 customers" means something different depending on which function you pick when there's a tie at the boundary: worth clarifying with whoever asked for the list

Likely follow-ups: Which one would you use for "top N per group," and why? What does NTILE do? (Q71-039.)

Red flag: claiming they're interchangeable, or unable to predict what happens at a tie without running it.

Learn it in: Chapter 13, section 13.4.

Q71-035 · Top 2 products by revenue, within each category · Core

Remember it as: PARTITION BY is GROUP BY for window functions: it restarts the ranking at each group boundary instead of collapsing rows.

The query:

```
SELECT category, product_name, revenue, rn
FROM (
  SELECT p.category, p.product_name,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue,
    ROW_NUMBER() OVER (
      PARTITION BY p.category
      ORDER BY SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) DESC
    ) AS rn
  FROM order_items oi
  JOIN products p ON oi.product_id = p.product_id
  JOIN orders o ON oi.order_id = o.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY p.category, p.product_name
) t
WHERE rn <= 2
ORDER BY category, rn;
```

Verified (riverstone_2025):

category	product_name	revenue	rn
Furniture	Garden Chair	33637.50	1
Industrial	Industrial Crate	795830.00	1
Kitchen	Lunch Box Set	546630.00	1
Kitchen	Food Container Set	523745.00	2
Storage	Storage Box 25L	908212.50	1
Storage	Storage Box 10L	860946.00	2

(6 rows)

Note: **Furniture and Industrial show only one row each**, not two. That's real data, not a bug: Furniture and Industrial each have only one product in the catalogue.

Tier	What to say
Passes	A correlated subquery per category finding the top product, then a second one for the runner-up: works, ugly, doesn't generalize past "top 2"
Strong	The PARTITION BY version above: one query, any N
Extra points	[+Edge cases] exactly this: two categories return fewer than 2 rows because they have only one product; <code>rn <= 2</code> never guarantees exactly 2 rows per group, and it's worth saying so before the result surprises someone · [+Validate] run the inner query on its own: 8 rows, one per product, and they add up to ₹43,35,471, Riverstone's 2025 revenue, so nothing was lost or double-counted before the ranking

Likely follow-ups: Same but by revenue *share* within category, not rank? What changes using `RANK` instead of `ROW_NUMBER` here?

Red flag: not expecting that some groups may have fewer members than N.

Learn it in: Chapter 13, section 13.7 (Pattern 1, top N per group).

Q71-036 · Month-over-month revenue change with LAG · Core

Remember it as: LAG looks *backward* one row. LEAD looks *forward* one row. Both need an ORDER BY to mean anything.

The query:

```
WITH monthly_revenue AS (  
  SELECT DATE_TRUNC('month', o.order_date)::date AS month,  
         ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue  
  FROM orders o  
  JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status <> 'Cancelled'  
  GROUP BY 1  
)  
SELECT month, revenue,  
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change  
FROM monthly_revenue  
ORDER BY month  
LIMIT 4;
```

Verified (riverstone_2025), the first four months:

month	revenue	mom_change
2025-01-01	202640.00	
2025-02-01	253664.00	51024.00
2025-03-01	278007.50	24343.50
2025-04-01	210281.50	-67726.00

(4 rows)

MySQL: the same DATE_FORMAT swap as in Q71-028.

Tier	What to say
Passes	A self-join on month = previous month (date arithmetic): correct, even with gaps, but verbose
Strong	LAG as above, noting that the first row's change is NULL because there's no earlier month, plus a note that LAG needs a gap-free month series: build one first (Chapter 13, section 13.8, Pattern 5, fill in the missing months), or check that month - LAG(month) is exactly one month
Extra points	[+Validate] the real fall in April (−₹67,726) is kept as it is, a real month-over-month decline, rather than picking a friendlier month to show · [+Edge cases] a month with no orders has no row at all, so without a gap-free series LAG silently compares with a month that isn't the previous one

Likely follow-ups: LAG two periods back instead of one? Percentage change, not absolute? LEAD for "days until next order"?

Red flag: forgetting ORDER BY inside the window (the function becomes meaningless without a defined row order).

Learn it in: Chapter 13, section 13.5.

Rapid-fire, \$71.5 (Core)

#	Question	One-line answer	Extra point
Q71-037	What does SUM(x) OVER (ORDER BY month) compute, with no PARTITION BY?	A running total across all rows, in that order	[+Edge cases] without a frame clause, the default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, so rows that tie on the ORDER BY value get the same running total. Write ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW for a strict row-by-row running total → Ch 13 §13.6

#	Question	One-line answer	Extra point
Q71-038	What's the difference between a window function and GROUP BY?	GROUP BY collapses rows into one per group; a window function keeps every row and adds a calculated column beside it	[+Business] this is <i>why</i> window functions exist for reporting: you can show a customer's single order next to their running total, in the same row → Ch 13 §13.3
Q71-039	What's NTILE(4) for?	Splits ordered rows into 4 roughly equal-sized groups (quartiles)	[+Business] the SQL way to build a quartile or decile segmentation directly in a query → Ch 27 §27.4 (and Ch 28 §28.3)
Q71-040	Can you use a window function's result in the same query's WHERE clause?	No: window functions are calculated after WHERE; wrap the query in a subquery or CTE and filter the outer layer	[+Edge cases] this is exactly why Q71-033 and Q71-035 wrap the window function in a subquery before filtering, and Q71-074 shows the real error → Ch 13 §13.3

Q71-037, run (riverstone_2025). The running total over Q71-028's CTE:

```
WITH monthly_revenue AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS month,
         ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY 1
)
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month) AS running_total
FROM monthly_revenue
ORDER BY month
LIMIT 3;
```

month	revenue	running_total
2025-01-01	202640.00	202640.00
2025-02-01	253664.00	456304.00
2025-03-01	278007.50	734311.50

(3 rows)

[+Validate] the running total reconciles: $2,02,640 + 2,53,664 = 4,56,304$ ✓, and $+ 2,78,007.50 = 7,34,311.50$ ✓. Each month appears once here, so **RANGE** and **ROWS** give the same answer; they part company only when two rows share a month.

71.6 Classic problems: duplicates, gaps-and-islands, retention

Q71-041 • Find any duplicate customer names • Core

Remember it as: GROUP BY the thing that should be unique, HAVING COUNT > 1.

```
SELECT customer_name, COUNT(*)
FROM customers
GROUP BY customer_name
HAVING COUNT(*) > 1;
```

Verified (riverstone_2025): 0 rows. Riverstone's customer table has no exact-name duplicates.

```
customer_name | count
-----+-----
(0 rows)
```

Tier	What to say
Passes	The query above, correctly, with no result to show
Strong	+ says that an empty result here is the <i>real</i> answer on this data, not a broken query, and how they'd confirm it (run the same query on a column known to have duplicates, such as <code>city</code>)
Extra points	[+Edge cases] exact-name matching misses near-duplicates: trailing spaces, case differences, "Sharma Hardware" vs. "Sharma Hardware Pvt Ltd". Those need <code>TRIM/LOWER</code> first, or fuzzy matching, which is a different, harder problem · [+Business] near-duplicate customer records are one of the most common real data-quality issues in a CRM, and this exact-match query is only the first, cheapest check

Likely follow-ups: How would you catch near-duplicates, not just exact ones? What would you do once you found real duplicates: merge them how?

Red flag: assuming an empty result means the query is wrong, rather than considering that it might be the honest answer.

Learn it in: Chapter 12, section 12.9; Chapter 14, section 14.4 (exact and fuzzy duplicates).

Q71-042 · Find every customer's longest streak of consecutive order-days · Advanced

Remember it as: *The gaps-and-islands trick: subtract a row number from the date. Rows in the same streak land on the exact same result.*

The query:

```
WITH daily AS (
  SELECT DISTINCT customer_id, order_date
  FROM orders
),
numbered AS (
  SELECT customer_id, order_date,
         order_date - CAST(ROW_NUMBER() OVER (PARTITION BY customer_id
                                              ORDER BY order_date) AS INT) AS grp
  FROM daily
),
streaks AS (
  SELECT customer_id,
         MIN(order_date) AS streak_start,
         MAX(order_date) AS streak_end,
         COUNT(*)        AS streak_len
  FROM numbered
  GROUP BY customer_id, grp
)
SELECT customer_id, streak_start, streak_end, streak_len
FROM (
  SELECT s.*,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                          ORDER BY streak_len DESC, streak_start) AS rn
  FROM streaks s
) t
WHERE rn = 1 AND streak_len >= 2
ORDER BY streak_len DESC, customer_id;
```

Verified (riverstone_2025): two customers have a streak of two consecutive order-days; no streak reaches three.

customer_id	streak_start	streak_end	streak_len
5	2025-01-12	2025-01-13	2
15	2025-06-09	2025-06-10	2

(2 rows)

MySQL: a date minus a number is written `DATE_SUB`; the rest is unchanged, and it returns the same two rows:

```
WITH daily AS (
  SELECT DISTINCT customer_id, order_date
  FROM orders
),
numbered AS (
  SELECT customer_id, order_date,
         DATE_SUB(order_date,
                   INTERVAL ROW_NUMBER() OVER (PARTITION BY customer_id
                                                ORDER BY order_date) DAY) AS grp
  FROM daily
),
streaks AS (
  SELECT customer_id,
         MIN(order_date) AS streak_start,
         MAX(order_date) AS streak_end,
         COUNT(*) AS streak_len
  FROM numbered
  GROUP BY customer_id, grp
)
SELECT customer_id, streak_start, streak_end, streak_len
FROM (
  SELECT s.*,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                             ORDER BY streak_len DESC, streak_start) AS rn
  FROM streaks s
) t
WHERE rn = 1 AND streak_len >= 2
ORDER BY streak_len DESC, customer_id;
```

Tier	What to say
Passes	Solves it with <code>LAG</code> : flag a row when the gap to the previous date is more than 1 day, then a running <code>SUM</code> of the flags numbers the streaks. Correct, but longer
Strong	The row-number-minus-date query above, with the mechanism explained: within one streak the date goes up by 1 each row and so does the row number, so their difference stays constant; a gap breaks that and starts a new group. The last step keeps each customer's <i>longest</i> streak, not every streak
Extra points	[+Validate] the real result, honestly reported: only 2-day streaks exist in this dataset, nothing longer, rather than a more impressive-looking (fake) example · [+Signpost] name the pattern: this "constant difference" trick solves any gaps-and-islands problem: consecutive login days, consecutive shipped orders, consecutive price rises

Likely follow-ups: Same, but for consecutive *calendar weeks*, not days? What's the equivalent trick if the sequence isn't dates but plain row order? What changes if `order_date` were a `TIMESTAMP`? (Cast it to `DATE` first, or the subtraction and the grouping both break.)

Red flag: reaching for a procedural loop (a cursor, or code outside SQL) as the only solution they can think of.

Learn it in: Chapter 13, section 13.8 (Pattern 8, streaks).

Q71-043 · Which customers ordered in both January and February 2025? (a simple retention check) · Core

Remember it as: Two EXISTS clauses, ANDed together: "ordered in month A" AND "ordered in month B."

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id
                    AND o.order_date BETWEEN '2025-01-01' AND '2025-01-31')
AND EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id
                    AND o.order_date BETWEEN '2025-02-01' AND '2025-02-28')
ORDER BY c.customer_id;
```

Verified (riverstone_2025): 4 customers ordered in both months.

```
customer_id | customer_name
-----+-----
          1 | Sharma Hardware
          3 | Green Leaf Hotels
          4 | Coastal Foods
          5 | Metro Mart
(4 rows)
```

Tier	What to say
Passes	Two separate queries (January customers, February customers), compared by eye: works for a one-off check, doesn't scale to "for every pair of consecutive months"
Strong	The double-EXISTS version above, or two conditional counts compared in one row
Extra points	[+Signpost] this is retention at its simplest: full cohort retention extends it into a month-by-month grid, built with conditional aggregation (SUM(CASE WHEN ... THEN 1 ELSE 0 END)) rather than repeated EXISTS (Chapter 13's Pattern 7 builds the full grid) · [+Business] January had only 5 active customers (Q71-027), so 4 of them returning in February is a strong start; say what you'd compare it with before calling it good or bad

Likely follow-ups: Build a full 12-month retention grid? What's the difference between this and a rolling "active in the last 30 days" count?

Red flag: confusing "ordered in both months" with "ordered at all in the date range" (missing the AND of two EXISTS).

Learn it in: Chapter 12, section 12.12 (EXISTS); Chapter 13, section 13.8 (Pattern 7, cohort retention).

Rapid-fire, §71.6 (Core)

#	Question	One-line answer	Extra point
Q71-044	Find products that have never been sold?	LEFT JOIN products to order_items WHERE oi.product_id IS NULL, or NOT EXISTS	[+Validate] verified on riverstone_2025: 0 rows, because all 8 products have sold. On the mini database riverstone the same query returns Garden Chair: check which database you're on → Ch 12 §12.10
Q71-045	Find the Nth highest value in general, not just the 2nd?	DENSE_RANK() OVER (ORDER BY x DESC), then filter WHERE rnk = N in an outer query	[+Edge cases] the same tie-safety reasoning as Q71-033, for any N → Ch 13 §13.4
Q71-046	Compute a percentage of total per row?	100.0 * x / SUM(x) OVER (): no GROUP BY needed, since a window with no PARTITION BY sees every row	[+Edge cases] multiply by 100.0 first: in PostgreSQL an integer column divided by an integer total gives 0 (verified: quantity / SUM(quantity) OVER () * 100 is 0 on every order line). MySQL's / always returns a decimal, so the bug only shows when you port the query → Ch 13 §13.3

71.7 Schema, constraints, and transactions

The first three questions build small tables of their own. Run them in `riverstone_lab`, the practice database from Chapter 12, section 12.13; every demo drops its tables at the end. The foreign keys are written on their own line, as Chapter 12's dialect note in section 12.13 recommends: MySQL 8.4 and earlier silently ignore the shorter inline REFERENCES form, and InnoDB doesn't support foreign keys on temporary tables at all.

Q71-047 • Demonstrate the difference between ON DELETE CASCADE and the default (NO ACTION) behavior • Core

Remember it as: CASCADE says "delete the children too." The default says "refuse to delete the parent while children still exist."

Answer in one line: Without ON DELETE CASCADE, deleting a parent row is refused while any child rows still reference it; with ON DELETE CASCADE, deleting the parent automatically deletes those child rows too.

Verified, in riverstone_lab. With CASCADE (same statements in both databases):

```
CREATE TABLE parent_t (id INT PRIMARY KEY);
CREATE TABLE child_t (
  id INT,
  parent_id INT,
  FOREIGN KEY (parent_id) REFERENCES parent_t (id) ON DELETE CASCADE
);
INSERT INTO parent_t VALUES (1), (2);
INSERT INTO child_t VALUES (10, 1), (11, 1), (12, 2);
DELETE FROM parent_t WHERE id = 1;
SELECT * FROM child_t;
```

```

id | parent_id
----+-----
12 |         2
(1 row)

```

MySQL:

```

+-----+-----+
| id   | parent_id |
+-----+-----+
| 12   |         2 |
+-----+-----+

```

Rows 10 and 11, both children of parent 1, are gone too. **Without CASCADE, the default:**

```

CREATE TABLE parent_r (id INT PRIMARY KEY);
CREATE TABLE child_r (
    id INT,
    parent_id INT,
    FOREIGN KEY (parent_id) REFERENCES parent_r (id)
);
INSERT INTO parent_r VALUES (1);
INSERT INTO child_r VALUES (10, 1);
DELETE FROM parent_r WHERE id = 1;

```

```

ERROR: update or delete on table "parent_r" violates foreign key constraint "child_r_parent_id_fkey" on table
"child_r"
DETAIL: Key (id)=(1) is still referenced from table "child_r".

```

MySQL:

```

ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
(`riverstone_lab`.`child_r`, CONSTRAINT `child_r_ibfk_1` FOREIGN KEY (`parent_id`) REFERENCES `parent_r`
(`id`))

```

Tidy up (same in both):

```

DROP TABLE child_t, parent_t, child_r, parent_r;

```

Tier	What to say
Passes	"CASCADE deletes related rows too" (correct, no comparison with the default)
Strong	Both behaviors above, contrasted: cascade silently removes children; the default blocks the delete with an error
Extra points	[+Validate] both real outcomes on parallel tables: 2 rows silently gone with CASCADE, 1 delete flatly refused without it, in both databases · [+Edge cases] PostgreSQL's default is NO ACTION, checked at the end of the statement (and deferrable); RESTRICT checks immediately. In MySQL the two are identical · [+Business] CASCADE is powerful and dangerous: deleting one customer with CASCADE on their orders would silently delete their whole order history. Keep it for truly dependent data (an order's own line items), not for a parent whose "children" are valuable records in their own right

Likely follow-ups: What's `ON DELETE SET NULL`? When would you prefer it to `CASCADE`? What about `ON UPDATE CASCADE`?

Red flag: treating `CASCADE` as a safe default rather than a deliberate, case-by-case decision.

Learn it in: Chapter 12, section 12.13 (Step 6, the `ON DELETE` options, and the dialect note in Step 3). `ON UPDATE CASCADE` and deferrable constraints are *beyond the book*.

Q71-048 · Write a CHECK constraint enforcing that a discount percentage is between 0 and 100, and show it rejecting bad data · Core

Remember it as: A `CHECK` constraint is a rule the database enforces on every write, not a rule you hope the application remembers.

Verified, in `riverstone_lab` (same statements in both databases):

```
CREATE TABLE tx_check (  
  id          INT,  
  discount_pct NUMERIC(5,2),  
  CONSTRAINT chk_discount CHECK (discount_pct BETWEEN 0 AND 100)  
);  
INSERT INTO tx_check VALUES (1, 50);  
INSERT INTO tx_check VALUES (2, 150);
```

```
ERROR:  new row for relation "tx_check" violates check constraint "chk_discount"  
DETAIL:  Failing row contains (2, 150.00).
```

MySQL:

```
ERROR 3819 (HY000): Check constraint 'chk_discount' is violated.
```

The first insert (50) succeeded; the second (150) was refused. Tidy up (same in both):

```
DROP TABLE tx_check;
```

Tier	What to say
Passes	"You'd validate the discount in the application code before saving it"
Strong	The <code>CHECK</code> constraint above, and why it's a stronger guarantee than application-level validation alone
Extra points	[+Validate] the real errors above, including PostgreSQL's <code>DETAIL</code> line with the exact rejected row · [+Business] application checks can be bypassed by a second application, a manual data fix, or a bug; a <code>CHECK</code> constraint can't be bypassed by anything that writes to the table: Consistency (Q71-051) enforced, not hoped for · [+Edge cases] MySQL before 8.0.16 accepted <code>CHECK</code> and ignored it. Always test that a constraint rejects a bad row

Likely follow-ups: What's the difference between a `CHECK` constraint and a `NOT NULL` constraint? Can a `CHECK` constraint look at another table? (No, *not directly*; a cross-table rule needs a trigger or a foreign key.)

Red flag: relying entirely on application code to enforce a rule the database itself could guarantee.

Learn it in: Chapter 12, section 12.13 (Step 3, and "When the database says no").

Q71-049 · Build a view for "each customer's total revenue," and explain what a view actually is · Core

Remember it as: A view isn't a copy of the data. It's a saved query with a name, re-run fresh every time you query it.

Answer in one line: CREATE VIEW saves a query under a name; querying the view re-runs that query against the live data every time, it doesn't store a snapshot.

Verified (riverstone_2025). Create the view (same in MySQL):

```
CREATE VIEW customer_revenue AS
SELECT c.customer_id, c.customer_name,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS revenue
FROM customers c
JOIN orders o      ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name;
```

Query it like a table:

```
SELECT * FROM customer_revenue ORDER BY revenue DESC LIMIT 3;
```

customer_id	customer_name	revenue
1	Sharma Hardware	502775.00
11	Harbour Traders	412680.50
7	Northgate Distributors	353088.50

(3 rows)

Then remove it, so riverstone_2025 stays as it was:

```
DROP VIEW customer_revenue;
```

Tier	What to say
Passes	"A view is like a virtual table" (true, doesn't explain the "re-run every time" mechanism)
Strong	+ a plain view has no stored data of its own; every SELECT against it runs the underlying query fresh, so it's always current, and it costs the full query every time
Extra points	[+Validate] the top three match what the underlying GROUP BY query returns when you run it directly (Q71-073), since that's literally what the view does · [+Trade-offs] a materialized view (CREATE MATERIALIZED VIEW, PostgreSQL only; MySQL has none) stores the result, so it's fast to query but stale until you run REFRESH MATERIALIZED VIEW: a trade between freshness and speed. In MySQL the equivalent is a summary table refreshed by a scheduled job · [+Business] a view is also a clean way to hide a complex join behind a simple name for less technical users, and to enforce row-level access (a view showing only a rep's own accounts) without repeating the logic in every query

Likely follow-ups: Can you INSERT into a view? (Sometimes, for simple views, with restrictions.) What does querying a view built on another view cost?

Red flag: believing a plain view stores a copy of the data at creation time.

Learn it in: Chapter 13, section 13.2 (views); Chapter 28, section 28.11 (materialized views).

Q71-050 · Demonstrate that ROLLBACK actually undoes a change, not just "cancels" it · Core

Remember it as: Nothing inside BEGIN...COMMIT is permanent until COMMIT. ROLLBACK makes it as if it never happened at all.

Answer in one line: A transaction's changes are only permanent once COMMIT runs; ROLLBACK reverts every change made since BEGIN, restoring the exact earlier state.

Verified, in riverstone_lab (same statements in both databases; MySQL also accepts START TRANSACTION for BEGIN):

```
CREATE TABLE tx_test (id INT, val VARCHAR(20));
INSERT INTO tx_test VALUES (1, 'original');
```

```
BEGIN;
UPDATE tx_test SET val = 'changed' WHERE id = 1;
SELECT * FROM tx_test;    -- inside the open transaction
ROLLBACK;
SELECT * FROM tx_test;    -- after ROLLBACK
DROP TABLE tx_test;
```

```
id | val
---+-----
1 | changed
(1 row)
```

```
id | val
---+-----
1 | original
(1 row)
```

MySQL:

```
+-----+-----+
| id  | val  |
+-----+-----+
| 1  | changed |
+-----+-----+
+-----+-----+
| id  | val  |
+-----+-----+
| 1  | original |
+-----+-----+
```

Tier	What to say
Passes	"ROLLBACK undoes changes" (correct, no demonstration of the mechanism)
Strong	+ explains that the UPDATE is visible <i>within</i> the same session immediately, but is not permanent until COMMIT, and ROLLBACK discards it entirely
Extra points	[+Validate] the live proof above: the same SELECT, before and after ROLLBACK, gives two different real answers on the same table · [+Business] this is <i>why</i> multi-step operations (move money between two accounts, place an order and reduce stock together) belong in one transaction: if step two fails, ROLLBACK guarantees step one never sticks around half-finished

Likely follow-ups: What does `COMMIT` do that makes a change durable? What happens if the connection drops before either `COMMIT` or `ROLLBACK` runs? (*The transaction is rolled back automatically.*)

Red flag: not knowing that changes are visible inside the same session before commit, and invisible to other sessions until commit.

Learn it in: Chapter 12, section 12.13 (Step 8, transactions).

Q71-051 • What does ACID stand for? Give one real example of each • Core

Remember it as: *Atomic: all or nothing. Consistent: rules always hold. Isolated: transactions don't see each other's half-finished work. Durable: once committed, it survives a crash.*

Answer in one line: Atomicity (a transaction fully succeeds or fully fails, no partial state), Consistency (every committed change leaves the data obeying its declared rules), Isolation (concurrent transactions don't see each other's uncommitted changes), Durability (once committed, a change survives a crash or power loss).

Tier	What to say
Passes	Can name what the letters stand for, can't give a concrete example of any of them
Strong	+ one real example each: Atomicity (Q71-050's transfer example: both the debit and the credit happen, or neither does) · Consistency (a <code>CHECK</code> constraint, Q71-048, refusing an invalid row) · Isolation (two people checking stock at once shouldn't both believe the last unit is theirs) · Durability (a committed order survives the database server crashing five seconds later)
Extra points	[+Edge cases] isolation comes in <i>levels</i> (Q71-056), so "isolated" isn't one fixed behavior but a range of trade-offs · [+Business] these four properties are why a relational database, not a plain file, is the default for anything involving money or stock, where a value must never silently go wrong

Likely follow-ups: Which of the four is hardest to guarantee at scale? Which systems deliberately relax some of them, and why?

Red flag: reciting the acronym with no example for any letter.

Learn it in: Chapter 12, section 12.13 (Step 8, transactions); Chapter 49, section 49.4 (ACID).

Beyond the book: concurrency in five minutes

The last four rapid-fire rows below (Q71-056 to Q71-059) go beyond the teaching chapters, so here is the short explanation they need. When two sessions work on the same table at once, the **isolation level** decides what one session can see of the other's unfinished work. Each database has a default. Check yours:

```
SHOW default_transaction_isolation;
```

```
default_transaction_isolation
-----
read committed
(1 row)
```

MySQL:

```
SELECT @@transaction_isolation;
```

```
+-----+
| @@transaction_isolation |
+-----+
| REPEATABLE-READ        |
+-----+
```

SHOW reads one of PostgreSQL's settings by name; in MySQL, @@ in front of a name reads a server setting (a *system variable*) the same way. So PostgreSQL starts every transaction at Read Committed, and MySQL at Repeatable Read.

A **dirty read** is the thing the weakest level allows. Picture two sessions, A and B, and an order whose status is 'Pending':

Step	Session A	Session B	What B sees
1	BEGIN; UPDATE orders SET status = 'Shipped' WHERE order_id = 10001;		
2		SELECT status FROM orders WHERE order_id = 10001;	Read Uncommitted: 'Shipped' (a dirty read). Read Committed or stricter: 'Pending'
3	ROLLBACK;		
4		the same SELECT again	'Pending': the "Shipped" B may have seen in step 2 never existed

Neither database allows a dirty read by default. Both let a plain SELECT in B go ahead at step 2 without waiting for A: each keeps the last committed version of the row for readers, an idea called **MVCC** (multi-version concurrency control).

Rapid-fire, §71.7 (Core; Q71-056 to Q71-059 Advanced)

#	Question	One-line answer	Extra point
Q71-052	ALTER TABLE ... ADD COLUMN with a default value on a huge table: what should you know?	PostgreSQL 11 and later add a column with a constant default instantly, without rewriting every row; older versions, and some other engines, rewrite the whole table, which can lock it for a long time	[+Business] check your engine and version before assuming a schema change on a huge production table is "instant" → Ch 28 §28.12 (changing a live database); the version detail is <i>beyond the book</i>
Q71-053	What's the difference between a PRIMARY KEY and a UNIQUE constraint?	A table has at most one PRIMARY KEY (which also forbids NULL); it can have many UNIQUE constraints, and each still allows multiple NULLs (Q71-013)	[+Edge cases] this is why an email UNIQUE constraint doesn't stop multiple blank emails, but adding NOT NULL to the UNIQUE column would. (A CHECK (email <> '') also blocks empty text, per Q71-022) → Ch 12 §12.13

#	Question	One-line answer	Extra point
Q71-054	What does NOT NULL enforce, and can you add it to a column that already has data?	It rejects any future NULL in that column; adding it to an existing column needs every current value to be non-NULL already, or the ALTER TABLE itself fails	[+Edge cases] run a check first, SELECT COUNT(*) FROM t WHERE col IS NULL, before adding NOT NULL to a live table → Ch 12 §12.13 (Step 9)
Q71-055	What's a materialized-view refresh strategy, in one sentence?	Refresh on a schedule (nightly, hourly), or refresh from a trigger or pipeline step after the source loads (PostgreSQL has no automatic refresh), chosen by how stale the data may get	[+Business] the same freshness-vs-cost trade as a Power BI scheduled refresh, one layer down → Ch 28 §28.11; Ch 46 (pipeline steps); Ch 16 §16.9 (Power BI refresh)
Q71-056	What's a transaction isolation level, in plain terms?	A setting for how much of another transaction's unfinished work you can see while your own transaction runs	[+Edge cases] from loosest to strictest: Read Uncommitted, Read Committed (PostgreSQL's default), Repeatable Read (MySQL InnoDB's default), Serializable (both checked above) → <i>Beyond the book</i> (the box above)
Q71-057	What's a "dirty read"?	Reading another transaction's uncommitted change, which might still be rolled back and never happen	[+Edge cases] only possible at Read Uncommitted; PostgreSQL doesn't implement that level separately and behaves as Read Committed even if you ask for it → <i>Beyond the book</i> (the box above)
Q71-058	What's a deadlock?	Two transactions each waiting for a lock the other one holds, so neither can ever proceed	[+Business] the database detects it and rolls back one of the two transactions rather than letting both hang forever; application code should catch that error and retry → <i>Beyond the book</i>
Q71-059	Does a SELECT ever get blocked by another transaction's write?	Under each engine's default level (Read Committed in PostgreSQL, Repeatable Read in MySQL), a plain SELECT isn't blocked: it reads the last committed version	[+Edge cases] the exception is a locking read, SELECT ... FOR UPDATE, which does wait for the writer. Plain reads don't wait because of MVCC → <i>Beyond the book</i> (the box above)

71.8 PostgreSQL vs. MySQL: where the dialects actually differ

Every pair below was run on both databases against the same Riverstone data.

Q71-060 • String aggregation: STRING_AGG vs. GROUP_CONCAT • Core

Remember it as: Same job, different name, different separator syntax.

PostgreSQL:

```
SELECT category,
       STRING_AGG(DISTINCT product_name, ' ' ORDER BY product_name) AS products
FROM products
GROUP BY category
ORDER BY category;
```

category	products
Furniture	Garden Chair
Industrial	Industrial Crate
Kitchen	Food Container Set, Lunch Box Set, Water Bottle 1L
Storage	Stackable Bin, Storage Box 10L, Storage Box 25L

(4 rows)

MySQL:

```
SELECT category,
       GROUP_CONCAT(DISTINCT product_name ORDER BY product_name SEPARATOR ', ') AS products
FROM products
GROUP BY category
ORDER BY category;
```

category	products
Furniture	Garden Chair
Industrial	Industrial Crate
Kitchen	Food Container Set, Lunch Box Set, Water Bottle 1L
Storage	Stackable Bin, Storage Box 10L, Storage Box 25L

Tier	What to say
Passes	Knows one function, not the other engine's name for it
Strong	Both function names and their (slightly different) syntax for the separator
Extra points	[+Validate] the same four lines from both engines, above · [+Edge cases] MySQL's <code>GROUP_CONCAT</code> has a length limit (<code>group_concat_max_len</code> , 1,024 bytes by default) and silently cuts off longer results: worth knowing before it bites in production

Learn it in: Chapter 14, section 14.4 (`STRING_AGG`, with MySQL's `GROUP_CONCAT` in the same paragraph). `group_concat_max_len` is *beyond the book*.

Rapid-fire, §71.8 (Core): dialect differences

#	Feature	PostgreSQL	MySQL	Extra point
Q71-061	NULL replacement	<code>COALESCE(x, 'default')</code>	<code>IFNULL(x, 'default')</code> or <code>COALESCE</code> (both work)	[+Edge cases] MySQL supports <code>COALESCE</code> too; <code>IFNULL</code> is a MySQL shorthand for the two-argument case → Ch 12 §12.7 (<code>COALESCE</code>); <code>IFNULL</code> is <i>beyond the book</i>
Q71-062	Auto-incrementing primary key	<code>GENERATED ALWAYS AS IDENTITY</code> (older code: <code>SERIAL</code>)	<code>AUTO_INCREMENT</code>	[+Business] copy-pasting a <code>CREATE TABLE</code> between engines without translating this is one of the most common cross-dialect errors → Ch 12 §12.13 (Steps 2 and 3)

#	Feature	PostgreSQL	MySQL	Extra point
Q71-063	Limiting and skipping rows	<code>LIMIT n OFFSET m</code>	<code>LIMIT m, n</code> (offset first!) or <code>LIMIT n OFFSET m</code> (also works)	[+Edge cases] MySQL's <code>LIMIT m, n</code> puts the offset <i>first</i> , a common source of swapped-number bugs when porting a query → Ch 12 §12.5 (<code>LIMIT</code>); <code>OFFSET</code> is <i>beyond the book</i>
Q71-064	Current date and time	<code>CURRENT_DATE</code> , <code>NOW()</code>	<code>CURRENT_DATE</code> (or <code>CURDATE()</code>), <code>NOW()</code>	[+Trade-offs] <code>CURRENT_DATE</code> and <code>NOW()</code> work in both; prefer <code>CURRENT_DATE</code> for portability. The real difference is date arithmetic → Ch 12 §12.16 (<code>DATEDIFF</code> and <code>INTERVAL</code>)
Q71-065	Case of table and column names	Unquoted names are folded to lower case	Table names are case-sensitive on Linux and case-insensitive on Windows and macOS by default	[+Edge cases] a query that works on a laptop can fail on a Linux production MySQL server purely because of table-name casing: use lower-case names everywhere → Ch 12 §12.13 (the "Naming" note)
Q71-066	Boolean type	Native <code>BOOLEAN</code> , <code>true/false</code>	<code>BOOLEAN</code> is an alias for <code>TINYINT(1)</code>	[+Edge cases] MySQL's "boolean" columns are really small integers, which is why 0/1 and <code>FALSE/TRUE</code> are interchangeable there (and why Q71-009 prints 1) → Ch 12 §12.13
Q71-067	Window functions	Full support since PostgreSQL 8.4	Full support since MySQL 8.0 (not in 5.7 or earlier)	[+Business] if a company runs an older MySQL, ask whether window functions are available before assuming this chapter's window questions (Q71-033 to Q71-040, Q71-042, Q71-045, Q71-046 and Q71-074) apply as written → Ch 13 §13.9

71.9 Query optimization and indexes

Q71-068 · What does `EXPLAIN` show, and what did it show on Riverstone's orders table? · *Advanced*

Remember it as: `EXPLAIN` tells you what the database plans to do, not what you assume it does.

Answer in one line: `EXPLAIN` shows the query planner's chosen strategy (a sequential scan, an index scan, a join method) and its estimated cost, before running the query.

Verified (riverstone_2025), on the 175-row orders table:

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 1;
```

```
          QUERY PLAN
-----
Seq Scan on orders (cost=0.00..4.19 rows=16 width=25)
  Filter: (customer_id = 1)
(2 rows)
```

Now add an index on `customer_id`, and refresh the table's statistics:

```
CREATE INDEX idx_orders_customer ON orders (customer_id);
ANALYZE orders;
```

`ANALYZE` refreshes the table statistics the planner uses; without it the planner may be working from stale row counts. Ask for the plan again:

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 1;
```

(Unchanged: the table is too small for the index to pay off.)

```
          QUERY PLAN
-----
Seq Scan on orders (cost=0.00..4.19 rows=16 width=25)
  Filter: (customer_id = 1)
(2 rows)
```

Remove the index, so the database is left as it was:

```
DROP INDEX idx_orders_customer;
```

MySQL: InnoDB builds an index for every foreign key automatically, so MySQL already has one on `orders.customer_id` and uses it (type is `ref`, key is `customer_id`) before you add anything.

`FORMAT=TRADITIONAL` asks for the plan as a table, which looks the same on every version; MySQL 9 otherwise prints the one-line tree that Chapter 28, section 28.13 reads (-> `Index lookup on orders using customer_id ...`). Ending the statement with `\G` instead of `;` makes the `mysql` client print each column on its own line, which is easier to read for a wide result like this one:

```
EXPLAIN FORMAT=TRADITIONAL SELECT * FROM orders WHERE customer_id = 1\G
```

```

***** 1. row *****
      id: 1
    select_type: SIMPLE
      table: orders
    partitions: NULL
      type: ref
possible_keys: customer_id
      key: customer_id
     key_len: 4
       ref: const
      rows: 16
  filtered: 100.00
    Extra: NULL

```

Tier	What to say
Passes	"EXPLAIN shows you the query plan"
Strong	+ explains what a sequential scan means (reads every row) versus an index scan (jumps straight to the matching rows)
Extra points	[+Validate] the real surprise above: even <i>with</i> an index, PostgreSQL's planner still chose a sequential scan on this table, and that's not a bug: on 175 rows, reading everything is cheaper than the extra work of using an index, and the planner knows it · [+Edge cases] this is the honest answer to "why isn't my index being used?": indexes pay off on large tables, not small ones, and a planner that ignores a small table's index is doing its job · [+Trade-offs] rows=16 is the planner's <i>estimate</i> , and it matches Sharma Hardware's 16 orders (Q71-034); EXPLAIN ANALYZE runs the query and adds the real counts and timings

Likely follow-ups: At what table size would you expect the planner to switch to the index? What's the difference between a sequential scan and an index scan in disk reads?

Red flag: assuming an index is always used just because it exists.

Learn it in: Chapter 28, sections 28.4–28.5 (and section 28.6 for indexes; Chapter 49 for storage at scale).

Rapid-fire, §71.9 (Advanced)

#	Question	One-line answer	Extra point
Q71-069	Which columns usually benefit most from an index?	Columns often used in WHERE, JOIN ... ON and ORDER BY, especially on large tables	[+Trade-offs] every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE must keep it up to date too): indexing everything is not free → Ch 28 §28.6
Q71-070	What's a composite index, and when does column order matter?	An index on several columns together; it helps most when a query filters on the <i>leading</i> column(s), in the order the index was built	[+Edge cases] an index on (customer_id, order_date) speeds up WHERE customer_id = X AND order_date > Y, but doesn't help a query filtering on order_date alone → Ch 28 §28.6
Q71-071	What's a covering index?	An index that holds every column a query needs, so the database answers from the index alone without reading the table rows	[+Business] one of the highest-impact, lowest-effort fixes for a frequently run report query → Ch 28 §28.6

71.10 Live-coding walk-throughs

Q71-072 · Walk-through: revenue by order, with a fan-out warning · Core

What they're really testing: whether you notice a join that silently multiplies rows before you trust the total it produces.

The setup, talked through live: "I need each order's revenue. Orders join to order_items for the line amounts. If this business also has invoices and payments tables, and I'm tempted to join through both to get to 'paid revenue,' I need to check first whether an invoice can have more than one payment, because that join would then count the order's revenue once per payment."

Verified (riverstone, the mini database): it has exactly this shape: order 5006's invoice has 2 payment rows. The wrong way, joining through invoices and payments before adding up the line items:

```
SELECT o.order_id,  
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS wrong_total  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN invoices i     ON o.order_id = i.order_id  
JOIN payments p     ON i.invoice_id = p.invoice_id  
WHERE o.order_id = 5006  
GROUP BY o.order_id;
```

```
order_id | wrong_total  
-----+-----  
    5006 |    29280.00  
(1 row)
```

The right way, adding up the line items on their own, with payments left out:

```
SELECT o.order_id,  
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS correct_total  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
WHERE o.order_id = 5006  
GROUP BY o.order_id;
```

```
order_id | correct_total  
-----+-----  
    5006 |    14640.00  
(1 row)
```

The order's revenue is ₹14,640. The fan-out join reports ₹29,280: exactly double, because its invoice has exactly 2 payments.

Extra-points moves demonstrated: [+Edge cases] expected the fan-out before running anything, from knowing an invoice can have several payments. [+Validate] proved it with a real doubled number on this exact order, not a hypothetical. [+Business] stated the fix plainly: compute revenue from order_items alone, and join to payments (if at all) only for payment-status questions, never in the same sum as revenue.

Likely follow-ups: How would you check for this kind of fan-out risk *before* writing the query, on a schema you've never seen? How would you fix a report that has already shipped with this bug?

Red flag: trusting a multi-join total without asking whether any joined table could have more than one matching row per order.

Learn it in: Chapter 12, section 12.10 (join fan-out); Chapter 13, section 13.8 (Pattern 10, checks before you trust a report).

Q71-073 · Walk-through: "give me our top customers": the clarifying-question round · Core

What they're really testing: whether you ask before you build, on a question that's genuinely ambiguous.

Talked through live: "'Top customers' could mean several things, and each needs a different query: by total revenue, by revenue this year only, by order count, or by average order value. I'll ask which, and while I wait I'll build the total-revenue version, since that's the most common meaning."

```
SELECT c.customer_id, c.customer_name,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)), 2) AS total_revenue
FROM customers c
JOIN orders o      ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Verified (riverstone_2025):

customer_id	customer_name	total_revenue
1	Sharma Hardware	502775.00
11	Harbour Traders	412680.50
7	Northgate Distributors	353088.50
3	Green Leaf Hotels	331388.75
5	Metro Mart	329297.50

(5 rows)

Extra-points moves demonstrated: **[+Clarify]** named the real ambiguity instead of guessing silently. **[+Simple first]** still produced a working, reasonable default answer while waiting, rather than stalling on the clarifying question. **[+Edge cases]** left out cancelled orders, a decision stated out loud rather than assumed. **[+Validate]** the first three match Q71-049's view exactly.

Likely follow-ups: Now by average order value instead. Now for a specific date range. Now leaving out one large outlier customer: how would you handle that request?

Red flag: picking one interpretation silently and never mentioning that the ambiguity existed.

Learn it in: Chapter 12, sections 12.9–12.10; Chapter 69, Move 1 (clarify first).

Q71-074 · Walk-through: does each customer's spend beat their own earlier average? · Advanced

What they're really testing: whether you can compare a row with the same customer's own history without a self-join, using a window frame most candidates have only seen in its default form.

Talked through live: "I need each customer-month's revenue, then the average of *that customer's own* earlier months only. A plain `AVG() OVER (PARTITION BY customer_id)` averages every month, future ones included. Adding `ORDER BY month` still includes the current row. I need a frame that ends at `1 PRECEDING`: everything before the current row, nothing after."

A common first attempt puts the window function straight into `WHERE`. That fails, for the reason in Q71-040: window functions run after `WHERE`.

```
SELECT customer_id, month, revenue
FROM (
  SELECT o.customer_id,
         DATE_TRUNC('month', o.order_date)::date AS month,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY 1, 2
) m
WHERE revenue > AVG(revenue) OVER (PARTITION BY customer_id ORDER BY month
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING);
```

ERROR: window functions are not allowed in WHERE

(MySQL's wording: `ERROR 3593 (HY000): You cannot use the window function 'avg' in this context.`) The fix: calculate the window in a CTE (`with_avg`) and filter the outer query, the same pattern as Q71-033 and Q71-035.

Verified (riverstone_2025):

```
WITH monthly AS (
  SELECT o.customer_id,
         DATE_TRUNC('month', o.order_date)::date AS month,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY 1, 2
),
with_avg AS (
  SELECT customer_id, month, revenue,
         AVG(revenue) OVER (
           PARTITION BY customer_id ORDER BY month
           ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
         ) AS prior_avg
  FROM monthly
)
SELECT customer_id, month,
       ROUND(revenue, 2) AS revenue,
       ROUND(prior_avg, 2) AS prior_avg
FROM with_avg
WHERE prior_avg IS NOT NULL
      AND revenue > prior_avg
```

```
ORDER BY customer_id, month
LIMIT 3;
```

customer_id	month	revenue	prior_avg
1	2025-03-01	64387.50	63656.25
1	2025-10-01	96825.00	34443.89
1	2025-11-01	82250.00	40682.00

(3 rows)

MySQL: the same `DATE_FORMAT` swap as in Q71-028 (`ROUND(x, 2)` already works in both); it returns the same three rows.

Extra-points moves demonstrated: **[+Edge cases]** caught the default-frame trap before writing any code, not after a wrong answer. **[+Validate]** customer 1's October revenue (₹96,825) against an earlier average of only ₹34,443.89: a real, large beat, and November's average checks by hand, $(9 \times 34,443.89 + 96,825) \div 10 = 40,682.00$. **[+Business]** "is this customer trending up against their own history" is a useful account-health signal, different from comparing customers with each other.

Likely follow-ups: What if you wanted a 3-month trailing average instead of all earlier months? How would this change using `LAG` instead of a frame clause?

Red flag: using the default window frame and not noticing the current row leaking into its own comparison average.

Learn it in: Chapter 13, section 13.6; Chapter 28, section 28.3 (frames that end before the current row).

Q71-075 · Walk-through: pivot monthly revenue by category into columns, live · Core

What they're really testing: whether conditional aggregation (`CASE` inside `SUM`) comes to mind, since standard SQL has no `PIVOT` keyword the way spreadsheets and a few database platforms do.

Talked through live: "There's no universal `PIVOT` in standard SQL. The standard trick is conditional aggregation: one `SUM(CASE WHEN category = X THEN revenue ELSE 0 END)` per category I want as a column, one column for each of the four categories."

Verified (riverstone_2025), the first three months:

```
SELECT DATE_TRUNC('month', o.order_date)::date AS month,
       ROUND(SUM(CASE WHEN p.category = 'Storage'
                      THEN oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)
                      ELSE 0 END), 2) AS storage,
       ROUND(SUM(CASE WHEN p.category = 'Kitchen'
                      THEN oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)
                      ELSE 0 END), 2) AS kitchen,
       ROUND(SUM(CASE WHEN p.category = 'Industrial'
                      THEN oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)
                      ELSE 0 END), 2) AS industrial,
       ROUND(SUM(CASE WHEN p.category = 'Furniture'
                      THEN oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)
                      ELSE 0 END), 2) AS furniture
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p    ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY 1
ORDER BY 1
LIMIT 3;
```

month	storage	kitchen	industrial	furniture
2025-01-01	172240.00	30400.00	0.00	0.00
2025-02-01	150231.50	54152.50	49280.00	0.00
2025-03-01	180785.00	49335.00	31500.00	16387.50

(3 rows)

MySQL: the same DATE_FORMAT swap as in Q71-028; it returns the same three rows.

Extra-points moves demonstrated: [+Edge cases] January's Industrial column shows 0, not a blank or an error: a real month with no Industrial sales, handled by the ELSE 0 in each CASE. And leave a category out and its revenue silently disappears from the row total. **[+Trade-offs]** this only works for a category list known in advance, written into the query; a changing set of categories needs pivoting in the application or engine-specific dynamic SQL. **[+Validate]** each row's four columns add up to that month's total in Q71-028: January 1,72,240 + 30,400 = 2,02,640 ✓; February 1,50,231.50 + 54,152.50 + 49,280 = 2,53,664 ✓; March 1,80,785 + 49,335 + 31,500 + 16,387.50 = 2,78,007.50 ✓, and March only reconciles because the Furniture column is there.

Likely follow-ups: What if the list of categories isn't known ahead of time? How would you do this in a BI tool instead of raw SQL?

Red flag: reaching for a non-standard PIVOT keyword without checking whether the engine supports it.

Learn it in: Chapter 13, section 13.8 (Pattern 9, pivot rows into columns); Chapter 11, section 11.5 (pivot tables) for the spreadsheet version.

Q71-076 • Walk-through: profile a table you've never seen before, in one query • Core

What they're really testing: whether you have a fast, repeatable first move for an unfamiliar table, rather than guessing or scrolling through raw rows.

Talked through live: "Before I write anything specific, I want a quick health check: the row count, how many NULLs in the columns that matter, the distinct count of what should be a key, and the date range. One query, several aggregates at once."

Verified (riverstone_2025), on the orders table:

```
SELECT
  COUNT(*)                                AS total_rows,
  COUNT(*) - COUNT(sales_rep_id)          AS null_sales_rep,
  COUNT(DISTINCT customer_id)             AS distinct_customers,
  MIN(order_date)                         AS earliest,
  MAX(order_date)                         AS latest,
  COUNT(*) FILTER (WHERE status = 'Cancelled') AS cancelled_count
FROM orders;
```

total_rows	null_sales_rep	distinct_customers	earliest	latest	cancelled_count
175	11	23	2025-01-02	2025-12-23	2

(1 row)

MySQL: there is no `FILTER`; count with `SUM(CASE ... THEN 1 ELSE 0 END)` instead:

```
SELECT
  COUNT(*)                                AS total_rows,
  COUNT(*) - COUNT(sales_rep_id)          AS null_sales_rep,
  COUNT(DISTINCT customer_id)             AS distinct_customers,
  MIN(order_date)                         AS earliest,
  MAX(order_date)                         AS latest,
  SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_count
FROM orders;
```

total_rows	null_sales_rep	distinct_customers	earliest	latest	cancelled_count
175	11	23	2025-01-02	2025-12-23	2

Extra-points moves demonstrated: [+Business] this one query answers, in seconds, what would otherwise take five: is the table roughly the size I expect, is there missing data to account for, does the date range match what I was told, are there statuses (like Cancelled) to filter in every later query.

[+Signpost] `COUNT(*) - COUNT(col)` as a NULL count and `COUNT(*) FILTER (WHERE ...)` as a conditional count are small idioms worth having memorized, not rebuilt each time. **[+Validate]** the 11 NULL `sales_rep_id` rows match the number Q71-002 found independently, a small consistency check that the profiling query itself can be trusted.

Likely follow-ups: How would you extend this to profile every column in a table automatically? What would you check differently for a table with no date column?

Red flag: starting with `SELECT * LIMIT 10` and eyeballing it as the only exploration step, with no aggregate health check at all.

Learn it in: Chapter 14, section 14.2 (profiling a new dataset); Chapter 13, section 13.8 (Pattern 10).

Common mistakes

Mistake	Symptom	Fix
NOT IN with a subquery that can contain NULL	Query silently returns zero rows	Filter NULLs in the subquery, or use NOT EXISTS / LEFT JOIN ... IS NULL
Filtering WHERE on an aggregate	Query errors	Use HAVING for aggregate conditions
LIMIT/OFFSET for "Nth highest"	Silently wrong on a tie	DENSE_RANK instead
Joining through a one-to-many table before aggregating	Totals inflate (fan-out), exactly Q71-072's doubled revenue	Aggregate the base fact table first; join only tables that don't fan out
Assuming an index is always used	Confusion when EXPLAIN shows a sequential scan anyway	Check the table size; small tables genuinely don't benefit
Forgetting ORDER BY inside a window function	LAG/LEAD/running totals become meaningless or change between runs	Give every window function an explicit row order, with a tie-breaker
Assuming PostgreSQL and MySQL syntax is interchangeable	A query that works on one engine errors or misbehaves on the other	Check the dialect differences (section 71.8) before porting

In the real world: the interview that hinged on one word

Deepak, interviewing for an Analytics Engineer role, is given a live SQL round: "Write a query showing each customer's total spend." He writes a clean, correct query joining customers to orders to order_items, summing revenue, in under three minutes.

The interviewer says: "Good. Now, what if I told you a customer can have order line items marked as returned, and 'total spend' should exclude those?" Deepak hadn't seen a `returned` flag in the schema he was shown, so instead of guessing, he asks: "Is there a status or flag column on order_items I should filter on, or does 'returned' show up somewhere else, like a separate returns table?" The interviewer, satisfied, describes a `returned` boolean column Deepak hadn't been shown, and Deepak adds one line: `WHERE returned = FALSE` (or `AND NOT oi.returned`, depending on how it's stored), explaining the change out loud as he types it.

What made this a strong round wasn't the original query, it was correct and unremarkable. It was that Deepak's first reaction to new information was a clarifying question, not a guess, and once given the real answer, resumed working with essentially no wasted motion, since the underlying query structure didn't need to change, only one filter needed to be added. That's Chapter 69's Move 1, live, under real pressure, and it's the single most common thing separating a "pass" from a "strong pass" in this specific kind of round.

Project

Goal: verified, real answers to the classic SQL problems, on your own data or Riverstone's.

Tools you'll need

PostgreSQL and **MySQL**, both free, both open source, installed as in Chapter 12, section 12.3 (the current releases: PostgreSQL 18 and MySQL 9.7 LTS, or 8.4 LTS). `psql` (PostgreSQL's command-line client) and the `mysql` client, or DBeaver for either. `EXPLAIN` (both engines) and `EXPLAIN ANALYZE` (runs the query and reports real timings, not just estimates) for reading query plans. Everything in this chapter was run on PostgreSQL 16 and MySQL 8.4 LTS, on Riverstone's `riverstone`, `riverstone_2025` and `riverstone_lab` databases, and run again on PostgreSQL 18 and MySQL 9.7 with the same results. One difference to know: MySQL 9.0 and later enforce the inline `REFERENCES` form that 8.4 ignores (Q71-047 avoids it).

1. Write and run the "customers with no orders" query (Q71-001), and reconcile the count against total customers minus customers with orders, the way this chapter did.
2. Demonstrate the `NOT IN NULL` trap yourself: find a nullable foreign key in your data, and show the same query returning zero rows with `NOT IN` and the correct count with `NOT EXISTS`.
3. Write one "top N per group" window-function query, and check whether any group returns fewer than N rows, reporting it honestly if so.
4. Run `EXPLAIN` on one of your own queries before and after adding an index, and report what actually changed, not what you expected to change. Drop the index afterwards.
5. Find one place in your own data (or Riverstone's) where a join could fan out unexpectedly, and prove the doubled or tripled total the way Q71-072 did.

REVIEW

Key terms

`LEFT JOIN` / `INNER JOIN` · anti-join · `NOT IN NULL` trap · `NOT EXISTS` · `WHERE` vs. `HAVING` · correlated subquery · scalar subquery · CTE (`WITH`) · recursive CTE · `EXISTS` vs. `IN` · `ROW_NUMBER` / `RANK` / `DENSE_RANK` · tie-breaker · `PARTITION BY` · `LAG` / `LEAD` · running total · default frame (`RANGE` vs. `ROWS`) · gaps-and-islands · fan-out (join cardinality) · `ON DELETE CASCADE` / `NO ACTION` · isolation level · dirty read · MVCC · `EXPLAIN` / `ANALYZE` · sequential scan vs. index scan · composite index · covering index · `STRING_AGG` / `GROUP_CONCAT` · `COALESCE` / `IFNULL` · `IDENTITY` / `AUTO_INCREMENT`

Final-week revision list

Q71-001, Q71-002, Q71-006, Q71-009, Q71-023, Q71-028, Q71-029, Q71-030, Q71-033, Q71-034, Q71-035, Q71-036, Q71-042, Q71-043, Q71-060, Q71-063, Q71-067, Q71-068, Q71-072, Q71-073.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 70, Excel/Sheets/VBA/BI Question Bank**, picks up the same Merge/Append and QUERY logic in spreadsheet form.
- **Chapter 77, Data Engineering & Data System Design Bank**, takes indexing and query optimization (section 71.9) to production scale.
- **Chapters 12, 13 and 28** teach the techniques this bank draws on (Chapter 14 for cleaning, Chapter 49 for ACID). Questions marked *Beyond the book* go further; each carries its own short explanation.

Chapter 72. Python & pandas Question Bank

Chapter at a glance

You will learn to: answer the Python and pandas questions that come up across screening calls, live-coding rounds, and take-homes for every data role · recognize the classic Python gotchas (mutable defaults, `is` vs `==`, late-binding closures) before they bite you live · write idiomatic, vectorized pandas instead of slow, easy-to-get-wrong loops · debug broken pandas code the way a live round actually tests you.

Before you start: Chapter 17 (Python from zero) and Chapter 18 (pandas), which teach almost everything here; Chapter 29 (classes, decorators, generators, keyword-only arguments) and Chapter 33 (generators and memory) for section 72.3; Chapter 69 for the answer tiers and the twelve extra-point tags. Chapter 71's SQL answers are the twins of this chapter's `merge` questions.

Time needed: about 4–6 hours to run every snippet yourself; 1 hour for a revision pass.

How this chapter is built. Same format as every question bank in Part 8 (Chapters 70–82): every core question leads with a "**Remember it as...**" hook, a one-line answer, and a compact tier table; rapid-fire sections are scan tables. Each question carries a level, **Warm-up**, **Core**, or **Advanced**, and within each group the questions run from easy to hard. **Every code snippet in this chapter was run**, on Python 3.11.15 with pandas 3.0.6; the book's recommended Python is 3.14 (Chapter 17, section 17.0), and where a version changes an output, the text says so. The order: language fundamentals and their gotchas first, then functions and iteration, then pandas from basic selection through to debugging and live coding.

Learn it in pointers name the chapter and section that teach each idea. Questions marked **Beyond the book** go further than Chapters 17–18 and carry their own short explanation, so you can learn the idea here.

A note on currency. This chapter is written against **pandas 3.0**, where two recent changes affect classic interview answers. First, **Copy-on-Write is always on** and can't be turned off, which retires the once-common "explain `SettingWithCopyWarning`" question (Q72-036). Second, **text columns now default to a dedicated `str` dtype**, not the older, more generic object dtype every pre-3.0 tutorial describes (Q72-038, Q72-051, Q72-054). Both the old behavior (which you may still meet on a team running an older pandas) and the current one are covered, clearly labeled.

In this chapter

LEARN	72.1 Python fundamentals: the gotchas that catch experienced candidates	116
	72.2 Predict the output: basic to advanced tricky questions	121
	72.3 Functions, generators, and decorators	127
	72.4 pandas fundamentals: Series, DataFrames, and selection	131
	72.5 pandas: grouping, joining, and reshaping	134
	72.6 pandas: performance, missing data, strings, and dates	138
	72.7 Debugging and live-coding walk-throughs	142
APPLY	Common mistakes · In the real world · Project	144
REVIEW	Key terms · Final-week revision list	146
NEXT	Where this leads	147

LEARN

72.1 Python fundamentals: the gotchas that catch experienced candidates

Q72-001 · Why does calling a function with a mutable default argument twice give a surprising result?

Level: Core · Beyond the book

Remember it as: A default argument is created once, when the function is defined, not fresh on every call. A mutable default remembers everything you've ever done to it.

Answer in one line: A default argument value is evaluated exactly once, at function definition time, and reused across every call, so a mutable default (like a list) accumulates changes across calls instead of starting fresh each time.

Beyond the book. Chapter 17, section 17.8 teaches default values (`discount_pct=0`) but not this trap. A list is **mutable**: `.append()` changes the list itself rather than making a new one. Python builds a default value once, when it runs the `def` line, and hands that same object to every call that leaves the argument out. With a number as the default, nothing can change it in place; with a list, every `.append()` lands in the one shared list.

```
def add_item(item, basket=[]):
    basket.append(item)
    return basket

r1 = add_item("crate")
print(r1)
```

```
['crate']
```

So far, as expected. Now call it a second time, with no `basket` again:

```
r2 = add_item("lid")
print(r1, r2, r1 is r2)
```

```
['crate', 'lid'] ['crate', 'lid'] True
```

- The second call returned `['crate', 'lid']`, not `['lid']`: the default list still held `"crate"` from the first call.
- **`r1` changed too**, after the fact, although nothing touched `r1` directly. That's because `r1` and `r2` are two names for the same list.
- **`r1 is r2`** asks "are these the very same object?" (is, Chapter 17, section 17.5; Q72-002 goes further). `True` proves there is one list, not two that happen to look alike.

The fix: use `None` as the default and build a fresh list inside the function, on every call.

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket

print(add_item("crate"))
print(add_item("lid"))
```

```
['crate']
['lid']
```

- **`basket=None`** makes the default something that can't be changed in place.
- **`if basket is None: basket = []`** runs on every call that leaves `basket` out, so each such call builds its own new list; `.append(item)` then adds to that list only, and `return` hands it back.
- Each call now returns a list with just its own item.

Tier	What to say
Passes	"You shouldn't use mutable defaults" (correct instinct, no demonstration of why)
Strong	The mechanism: the empty list is created once, at definition time, and silently shared by every call that doesn't pass its own <code>basket</code> ; the fix is <code>basket=None</code> , then <code>if basket is None: basket = []</code> inside the function
Extra points	[+Validate] the two cells above: <code>r1 is r2</code> is <code>True</code> , so both calls returned the exact same list object, and the fixed version returns <code>['lid']</code> on its own. [+Edge cases] the same trap applies to a <code>{}</code> or <code>set()</code> default

Likely follow-ups: Does this happen with immutable defaults like `basket=0`? (No: *immutable defaults can't be changed in place*, so there's nothing to accumulate.) Where else in Python does "created once, reused" show up? (Q72-004 and Q72-020.)

Red flag: not knowing why this specific bug happens, or claiming it's random behavior.

Learn it in: Beyond the book (defaults themselves: Chapter 17, section 17.8).

Q72-002 · What's the difference between `is` and `==`, and when does mixing them up cause a real bug?

Level: Warm-up

Remember it as: `==` asks "do these look the same?" `is` asks "are these literally the same object in memory?"

Answer in one line: `==` compares values for equality (and can be customized per type); `is` compares object identity, whether two names point to the exact same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print("a == b:", a == b)
print("a is b:", a is b)
print("a is c:", a is c)
```

```
a == b: True
a is b: False
a is c: True
```

- `a` and `b` hold the same values, so `==` is `True`; they are two separate lists, so `is` is `False`.
- `c = a` doesn't copy anything: it gives the one list a second name, so `a is c` is `True`.

Tier	What to say
Passes	" <code>is</code> checks identity, <code>==</code> checks equality" (correct, no example of the bug it causes)
Strong	The three-way comparison above, and a concrete case where confusing them bites: if <code>my_list</code> is <code>[]</code> : <code>is</code> always <code>False</code> , even for an empty list, because <code>[]</code> builds a brand-new list, and no existing list can be that new object. Write <code>if my_list == []</code> : or, better, <code>if not my_list</code> :
Extra points	[+Edge cases] small integers and short strings are cached by CPython (the standard Python you installed in Chapter 17), so <code>a = 5</code> ; <code>b = 5</code> ; <code>a is b</code> can return <code>True</code> purely as an implementation detail, not a language guarantee: never use <code>is</code> for values, only for <code>None</code> , <code>True</code> , and <code>False</code> . [+Validate] Python itself catches part of this: <code>n is 1000</code> makes it print <code>SyntaxWarning: "is" with a literal. Did you mean "=="?</code> . It gives no warning for <code>is []</code> , which is why that bug survives

Likely follow-ups: Why is `if x is None` the recommended style over `if x == None`? What's `__eq__`, and how does defining it change `==` for your own classes?

Red flag: using `is` to compare values (numbers, strings, lists) rather than reserving it for identity checks like `is None`.

Learn it in: Chapter 17, section 17.5 (`is None`, and what `is` asks); two names for one list is Beyond the book (Q72-001, Q72-026).

Q72-003 · What's the difference between a shallow copy and a deep copy, demonstrated on nested data?

Level: Core · Beyond the book

Remember it as: A shallow copy duplicates the outer box. A deep copy duplicates everything inside it too, all the way down.

Answer in one line: `.copy()` (or `copy.copy()`) creates a new outer container, but the *inner* objects it holds are still shared with the original; `copy.deepcopy()` copies everything, level by level, so nothing is shared.

Beyond the book. A dictionary holding a list doesn't hold a copy of the list, only a reference to it (the same "two names, one object" as Q72-002). `.copy()` makes a new dictionary with the same references inside. `copy.deepcopy`, from the standard library's `copy` module, also copies every list, dictionary, and object inside, however deep. Chapter 43, section 43.7 uses it to copy a whole model before changing it.

```
import copy
orig = {"items": [1, 2, 3]}
shallow = orig.copy()
deep = copy.deepcopy(orig)
shallow["items"].append(4)
print('orig["items"]:', orig["items"])
print('shallow["items"]:', shallow["items"])
print('deep["items"]:', deep["items"])
```

```
orig["items"]: [1, 2, 3, 4]
shallow["items"]: [1, 2, 3, 4]
deep["items"]: [1, 2, 3]
```

Appending through `shallow` changed `orig` as well, because both dictionaries point at the same inner list. `deep` has its own list and is untouched.

Tier	What to say
Passes	"Deep copy copies everything, shallow copy doesn't" (correct, no demonstration)
Strong	The nested-dictionary example above, showing the <i>original</i> data changing through a shallow copy of something you thought was safely duplicated
Extra points	[+Validate] the live proof: changing <code>shallow["items"]</code> silently changed <code>orig["items"]</code> too, exactly the kind of bug that's easy to miss in a larger program. [+Business] this trap shows up with nested configuration dictionaries or lists of records passed between functions: ask "does this container hold other mutable containers?" before reaching for a plain <code>.copy()</code>

Likely follow-ups: Does this matter for a flat dictionary with no nested containers? (No: a shallow copy is completely safe when nothing inside is itself mutable and shared.) What does `deepcopy` cost on a large nested structure? (Time and memory in proportion to everything inside it.)

Red flag: assuming `.copy()` always produces a fully independent object.

Learn it in: Beyond the book (`copy.deepcopy` in use: Chapter 43, section 43.7).

Q72-004 · Why does this loop of lambdas return the same value for every function, and how do you fix it?

Level: Advanced · **Beyond the book**

Remember it as: A closure remembers the variable, not its value at the time the closure was made. By the time you call it, the loop has already finished.

Answer in one line: A `lambda` created inside a loop refers to the *variable* `i`, not to its value at that point in the loop, so by the time any of the lambdas are called, the loop has finished and every one of them sees `i`'s final value.

Beyond the book. A `lambda` is a one-line function without a name (Chapter 18, section 18.6). A function created inside another piece of code can use that code's variables; it is then called a **closure**. It looks the variable up when it *runs*, not when it's created: that is **late binding**.

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

```
[2, 2, 2]
```

- The first line builds three small functions, each returning `i`.
- The second line calls each one. By then the loop is over and `i` is 2, so all three return 2, not 0, 1, 2 as intuition suggests.

Fixed, by capturing the value as a default argument, which is evaluated at once, when each lambda is made:

```
funcs_fixed = [lambda i=i: i for i in range(3)]
print([f() for f in funcs_fixed])
```

```
[0, 1, 2]
```

`lambda i=i:` gives each function its own parameter `i` whose default is the loop's *current* value. Defaults are evaluated once, at definition time (Q72-001), which is exactly the behavior needed here.

Tier	What to say
Passes	"Something about closures" (senses it's a known gotcha, can't explain it or fix it)
Strong	The variable-not-value explanation above, and the <code>lambda i=i:</code> fix, using the fact that default arguments <i>are</i> evaluated once, immediately
Extra points	[+Validate] both real outputs above: the broken <code>[2, 2, 2]</code> and the fixed <code>[0, 1, 2]</code> , same loop, one change. [+Signpost] this is the same "evaluated once vs. evaluated every time" distinction as Q72-001, showing up as a bug here and as a fix there

Likely follow-ups: Does this happen with a regular `for` loop and `def`, not just `lambda`? (Yes, the same *late binding* applies to any nested function.) What's a cleaner fix than the default-argument trick? (A small factory function, or `functools.partial`.)

Red flag: not recognizing this as a known, named Python behavior (late-binding closures), and guessing at random causes.

Learn it in: Beyond the book (`lambda`: Chapter 18, section 18.6).

Rapid-fire, §72.1

#	Question	One-line answer	Extra point
Q72-005	<i>Warm-up.</i> What's the difference between a list and a tuple?	Lists are mutable, tuples are immutable; tuples can be dictionary keys and set members (if everything inside them is hashable: <code>([1], 2)</code> isn't), lists can't	[+Business] a tuple signals "this shouldn't change," which is documentation as much as a technical constraint
Q72-006	<i>Warm-up.</i> What does Python's <code>None</code> represent, and how do you check for it?	The absence of a value; check with <code>is None</code> , <code>never == None</code>	[+Edge cases] <code>== None</code> usually works too, but <code>is None</code> can't be fooled by a class whose own <code>__eq__</code> changes what <code>==</code> means
Q72-007	<i>Core, Beyond the book.</i> What's the difference between <code>*args</code> and <code>**kwargs</code> ?	In a <code>def</code> , <code>*args</code> collects extra positional arguments into a tuple; <code>**kwargs</code> collects extra keyword arguments into a dictionary	[+Validate] run below: <code>describe(1, 2, 3, name="crate", qty=5)</code>
Q72-008	<i>Warm-up.</i> What's a list comprehension, and is it always faster than a loop?	<code>[expr for item in iterable if condition]</code> . Usually faster than the same loop, because it skips looking up and calling <code>.append</code> for every item, but not always and not by much: readability is the real reason to use one	[+Trade-offs] for big numeric work, vectorized pandas or NumPy beats both (Q72-047)
Q72-009	<i>Advanced.</i> What does <code>__init__</code> do, and is it a constructor?	It fills in a newly created object's attributes; the object itself is created just before, by <code>__new__</code>	[+Edge cases] almost nobody overrides <code>__new__</code> in data work (it's Beyond the book); knowing it exists is usually enough

Q72-007, run. The names `args` and `kwargs` are only a convention; the stars do the work:

```
def describe(*args, **kwargs):
    print("args =", args, "kwargs =", kwargs)

describe(1, 2, 3, name="crate", qty=5)
```

```
args = (1, 2, 3) kwargs = {'name': 'crate', 'qty': 5}
```

The three unnamed values arrive as the tuple `args`, and the two named ones as the dictionary `kwargs`. In a *call*, the stars work the other way round: `f(**settings)` unpacks a dictionary into keyword arguments, as Chapter 54, section 54.5 does.

Learn it in: Q72-005 Chapter 17, section 17.4 · Q72-006 section 17.5 · Q72-007 Beyond the book (`**` in a call: Chapter 54, section 54.5) · Q72-008 Chapter 17, section 17.6 · Q72-009 Chapter 29, section 29.5 (`__init__`).

72.2 Predict the output: basic to advanced tricky questions

The purest interview format: a short snippet, no explanation, just "what does this print, and why." The five core questions come first, easy to hard, then two rapid-fire tables, each easy to hard. Every one below was run; the printed output is real, not a remembered answer.

Q72-010 • `print(0.1 + 0.2 == 0.3)`: what prints, and why?

Level: Warm-up

Remember it as: Computers store most decimals as an approximation in binary, the same way $1/3$ has no exact end in decimal. 0.1 and 0.2 already aren't exactly what you typed, before you even add them.

Answer in one line: False: floating-point numbers are stored in binary, and most decimal fractions (0.1, 0.2, 0.3 included) have no exact binary form, so tiny rounding errors add up and `0.1 + 0.2` lands a hair away from the stored value of 0.3.

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

Tier	What to say
Passes	"Floating point is imprecise" (true, no mechanism, can't predict the actual behavior)
Strong	The binary-representation explanation above, and the right fix: compare within a small tolerance (<code>abs(a - b) < 1e-9</code> , or round both first) or use the decimal module for money, not <code>==</code>
Extra points	[+Validate] the real printed value above, <code>0.30000000000000004</code> , not just "it's not exactly 0.3". [+Business] this is why money is safer in decimal.Decimal or whole paise than in a raw float (the same point Chapter 17, section 17.3 makes, and why Chapter 12 stores money as NUMERIC), and why comparing two computed revenue totals with <code>==</code> in a test is a bug waiting to happen

Likely follow-ups: How would you safely compare two floats for "close enough"? Does this affect `int` arithmetic the same way? (No: Python's integers are exact, however large.)

Red flag: claiming this is a Python bug rather than how almost every language stores floating-point numbers (the IEEE 754 standard).

Learn it in: Chapter 17, section 17.3 ("Numbers with decimals are approximate"); Chapter 12, section 12.8 for the same test in SQL.

Q72-011 • `print(0 or 5)` and `print(3 and 5)`: what prints, and why?

Level: Core

Remember it as: `and/or` in Python don't return True/False. They return whichever actual operand decided the answer.

Answer in one line: `or` returns its first operand if it's truthy, otherwise its second; `and` returns its first operand if it's falsy, otherwise its second. Neither converts its result to a real boolean.

```
print(0 or 5)           # 5: 0 is falsy, so `or` returns its second operand
print(3 and 5)          # 5: 3 is truthy, so `and` returns its second operand
print([] or 'default')  # an empty list is falsy too
print(None and 'x')     # None is falsy, so `and` stops at it
```

```
5
5
default
None
```

Tier	What to say
Passes	"It returns True or False" (the single most common wrong assumption)
Strong	Correctly predicts all four outputs above, explaining that <code>or/and</code> return an actual operand, not a boolean
Extra points	[+Business] this is the mechanism behind the idiom <code>value = user_input or "default"</code> , using <code>or</code> on purpose to provide a fallback (Chapter 29's API client does it with <code>session or requests.Session()</code>). [+Edge cases] an empty list, an empty string, <code>0</code> , and <code>None</code> are all falsy and behave identically here, which can surprise someone expecting only <code>None/False</code> to count, and turns a genuine <code>0</code> into the fallback

Likely follow-ups: What's the output of `print(bool(0 or 5))`? What are *all* the falsy values in Python?

Red flag: assuming `and/or` always produce `True/False`.

Learn it in: Chapter 17, section 17.5 (`and`, `or`, truthiness); Chapter 29, section 29.9 (`or` as a fallback).

Q72-012 • `print(2 ** 3 ** 2)`: what prints, and why?

Level: Core

Remember it as: Most operators go left to right. The power operator is the odd one out: it goes right to left.

Answer in one line: 512: `**` is **right-associative** (a chain of them is worked from the right), so this is `2 ** (3 ** 2) = 2 ** 9 = 512`, not `(2 ** 3) ** 2 = 8 ** 2 = 64`.

```
print(2 ** 3 ** 2)
print(-2 ** 2)
```

```
512
-4
```

Tier	What to say
Passes	Guesses 64, assuming left to right like most other operators
Strong	Correctly computes 512, naming right-associativity as the reason
Extra points	[+Edge cases] <code>-2 ** 2</code> gives <code>-4</code> , not <code>4</code> , because the minus sign binds <i>less</i> tightly than <code>**</code> , so it's <code>-(2 ** 2)</code> , not <code>(-2) ** 2</code> : a second surprise in the same family. [+Validate] both real outputs above, together, since they're easy to mix up

Likely follow-ups: What's the full precedence order for arithmetic in Python? How would you force `(2 ** 3) ** 2`? (With the brackets, exactly as written.)

Red flag: confidently stating `**` works left to right like `+` or `*`.

Learn it in: Chapter 17, section 17.3 (`**` is power); the right-to-left rule is Beyond the book and explained above.

Q72-013 • `a = int("20", 4); print(a)`: what prints, and why?

Level: Core

Remember it as: The second argument to `int()` isn't a data type: it's the base the string should be read in. "20" in base 4 means 2 fours plus 0 ones.

Answer in one line: `int(string, base)` reads `string` as a number written in the given base, not base 10, so "20" in base 4 means $2 \times 4^1 + 0 \times 4^0 = 8$.

```
a = int("20", 4)
print(a)
```

8

Try changing the base: `int("20", 5)` is two fives, 10, and `int("20", 10)` is plain 20.

Tier	What to say
Passes	Guesses 20 (reading the string as if the base didn't matter), or guesses at random
Strong	Correctly computes 8 from place values: digit "2" is in the fours place, digit "0" in the ones place, $2 \times 4 + 0 \times 1 = 8$
Extra points	[+Edge cases] <code>int("0x1A", 16)</code> and <code>int("0x1A", 0)</code> both give 26: base 0 tells Python to <i>infer</i> the base from a prefix like <code>0x</code> (hexadecimal, base 16), <code>0o</code> (octal), or <code>0b</code> (binary) in the string itself. [+Business] this comes up whenever a system hands you IDs, hashes, or flags written in hex or binary as plain text and you need them back as integers

Likely follow-ups: What does `int("20")` give with no second argument? (20: base 10 is the default.) What happens with a digit that's invalid for the base, like `int("29", 4)`? (A `ValueError`, since 9 isn't a base-4 digit.)

Red flag: assuming the second argument changes the *output's* type or format rather than how the *input string* is read.

Learn it in: Chapter 17, section 17.3 (`int()`); Chapter 2, section 2.1 (binary). The `base` argument is Beyond the book and explained above.

Q72-014 • `grid = [[0]*3]*3; grid[0][0] = 1; print(grid)`: what prints, and why?

Level: Core • Beyond the book

Remember it as: `[[0]*3]*3` makes one inner list, then three references to that same list. Changing "one row" changes all of them at once.

Answer in one line: `[[1, 0, 0], [1, 0, 0], [1, 0, 0]]`: the outer `* 3` doesn't create three independent inner lists, it repeats three references to the *same* inner list, so changing one row through any of them changes what looks like every row.

Beyond the book. `*` on a list repeats what the list holds. `[0]*3` repeats a number, which is harmless. `[[0]*3]*3` repeats a *reference* to a list, the same "two names, one object" as Q72-001 and Q72-003.

```
grid = [[0]*3]*3
grid[0][0] = 1
print(grid)
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

The correct way to build three independent rows:

```
grid2 = [[0]*3 for _ in range(3)]
grid2[0][0] = 1
print(grid2)
```

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

The comprehension runs `[0]*3` three times, so it builds three separate lists. `_` is the conventional name for a loop variable you don't use.

Tier	What to say
Passes	Predicts <code>[[1, 0, 0], [0, 0, 0], [0, 0, 0]]</code> , the intuitive but wrong answer for the broken version
Strong	Correctly predicts all three rows changing, and explains it's the same bug family as Q72-003's shallow copy: <code>*</code> on a list of lists repeats references, not the lists they point to
Extra points	[+Validate] both real outputs above, the trap and its fix, side by side. [+Business] this is a common bug in anyone's first 2D grid, matrix, or game board in Python, and exactly the kind of thing worth a two-line check before trusting a larger program built on it

Likely follow-ups: Does the same trap apply to `[0, 0, 0] * 3`? (No: *the items are numbers, which can't be changed in place, so there's nothing to share by accident.*) How would you build a 2D NumPy array instead? (`np.zeros((3, 3))`), which has no such trap.)

Red flag: not recognizing this as the same "shared reference" mechanism as the mutable-default and shallow-copy traps.

Learn it in: Beyond the book (lists and comprehensions: Chapter 17, sections 17.4 and 17.6).

Rapid-fire, §72.2, part A: more predict-the-output, easy to hard

#	Snippet	Output	Why
Q72-015	<code>print(1 < 2 < 3)</code>	<code>True</code>	<i>Warm-up.</i> Python chains comparisons: this is <code>(1 < 2)</code> and <code>(2 < 3)</code> , with the middle 2 worked out only once
Q72-016	<code>print('ab' * 3)</code>	<code>ababab</code>	<i>Warm-up.</i> <code>*</code> on a string repeats it, the same repetition as <code>*</code> on a list, just on characters instead of items
Q72-017	<code>print(sum([]))</code> and <code>print(sum([], 10))</code>	<code>0</code> and <code>10</code>	<i>Warm-up.</i> <code>sum()</code> 's second argument is the starting value, 0 by default: useful for a possibly empty list without a separate empty check

#	Snippet	Output	Why
Q72-018	<pre>first, *rest = "hello"; print(first, rest)</pre>	<code>h ['e', 'l', 'l', 'o']</code>	<i>Core.</i> Unpacking works on any iterable, including a string (character by character); the starred name collects everything left over, as a list
Q72-019	<pre>c = int("1000"); d = int("1000"); print(c is d)</pre>	<code>False</code>	<i>Advanced, Beyond the book.</i> CPython keeps one shared copy of each small integer (about -5 to 256); 1000 is outside that cache, and building each 1000 at run time with <code>int()</code> stops Python from merging two identical literals into one constant, so these are two separate objects
Q72-020	<pre>class C: items = [] then c1 = C(); c1.items.append(1); c2 = C(); print(c2.items)</pre>	<code>[1]</code>	<i>Advanced, Beyond the book.</i> A mutable class attribute (set in the class body, not inside <code>__init__</code>) is shared by <i>every instance</i> : the class-level cousin of Q72-001's mutable default
Q72-021	<pre>print([x for x in range(5) if (y := x * 2) > 4])</pre>	<code>[3, 4]</code>	<i>Advanced, Beyond the book.</i> The walrus operator <code>:=</code> assigns <i>and</i> returns a value in one expression, here inside the filter; the list holds the <code>x</code> values (not <code>y</code>) that pass

Learn it in: Q72-015 Beyond the book · Q72-016 Chapter 17, section 17.3 (+ on strings) · Q72-017 section 17.4 (`sum`) · Q72-018 section 17.4 (unpacking; the star is Beyond the book) · Q72-019 Beyond the book · Q72-020 Chapter 29, section 29.5 (classes) · Q72-021 Beyond the book.

Rapid-fire, §72.2, part B: sequences, going deeper

Same quick format, narrowed to one theme: lists, tuples, strings, and the operations that show how they actually differ underneath.

#	Snippet	Output	Why
Q72-022	<pre>print(list(zip([1,2,3], [4,5])))</pre>	<code>[(1, 4), (2, 5)]</code>	<i>Warm-up.</i> <code>zip()</code> silently stops at the <i>shorter</i> input, with no warning or error: a quiet source of lost data when two "matching" lists aren't the same length
Q72-023	<pre>lst=[0,1,2,3,4,5]; print(lst[5:1:-1])</pre>	<code>[5, 4, 3, 2]</code>	<i>Core.</i> A negative step walks backward from index 5 down to, but not including, index 1; the stop is still left out, just approached from the other side
Q72-024	<pre>print([1,2,3] + (4,5))</pre>	<code>TypeError: can only concatenate list (not "tuple") to list</code>	<i>Core.</i> <code>+</code> between sequences needs matching types; mixing a list and a tuple is an error, not a silent conversion
Q72-025	<pre>print(sorted([(1,'b'), (1,'a'),(0,'z')], key=lambda x: x[0]))</pre>	<code>[(0, 'z'), (1, 'b'), (1, 'a')]</code>	<i>Core.</i> Python's sort is stable : items that tie on the key keep their original order, so <code>(1, 'b')</code> stays before <code>(1, 'a')</code> because it came first

#	Snippet	Output	Why
Q72-026	<code>a=[1,2]; b=a; a+=[3]; print(a, b, a is b)</code>	<code>[1, 2, 3] [1, 2, 3] True</code>	<i>Core, Beyond the book.</i> += on a list extends the <i>same</i> list in place; b sees the change because a and b are two names for one list
Q72-027	<code>c=[1,2]; d=c; c=c+[3]; print(c, d, c is d)</code>	<code>[1, 2, 3] [1, 2] False</code>	<i>Core, Beyond the book.</i> Plain + always builds a <i>new</i> list; pointing c at it leaves d on the original: the direct contrast with Q72-026
Q72-028	<code>t=([1,2], 3); t[0].append(99); print(t)</code>	<code>([1, 2, 99], 3)</code>	<i>Advanced, Beyond the book.</i> A tuple can't change <i>what it points to</i> , but the list t[0] points to can still be changed: you can't replace t[0], you can append to it

Learn it in: Q72-022 Chapter 17, section 17.6 (zip) · Q72-023 section 17.4 (slices) · Q72-024 section 17.4 (lists and tuples) · Q72-025 section 17.4 (sorted), Chapter 18, section 18.6 (lambda), and Chapter 33, section 33.7 (stable sorting) · Q72-026 to Q72-028 Beyond the book (Q72-001 and Q72-003 explain the shared reference).

72.3 Functions, generators, and decorators

Q72-029 · What's the difference between a function that returns a list and one that's a generator, and why does it matter for a large dataset?

Level: Core

Remember it as: A list builds and holds everything in memory at once. A generator hands you one value at a time and forgets it immediately after.

Answer in one line: A function using `return [...]` computes and stores every value at once; a function using `yield` produces values one at a time, on demand, using a small, constant amount of memory however many values there are.

Two ways to make the squares of the first n whole numbers:

```
def squares_list(n):
    return [i*i for i in range(n)]

def squares_gen(n):
    for i in range(n):
        yield i*i
```

A first, rough measure: `sys.getsizeof` gives the size of one object, in bytes.

```
import sys
print(sys.getsizeof(squares_list(1_000_000)))
print(sys.getsizeof(squares_gen(1_000_000)))
```

```
8448728
208
```

- `1_000_000` is one million; the underscores only make it easier to read.
- The list's 8,448,728 bytes (about 8.4 MB) are only its **slots**, one reference per item. `getsizeof` doesn't count the million numbers those slots point to.
- The generator's size is a small fixed amount, because it hasn't produced anything yet. Python 3.11 prints 208 here; 3.12 and 3.13 print 200.

For the real footprint, measure the peak memory while each version is summed, with `tracemalloc`, as Chapter 33, section 33.9 does:

```
import tracemalloc

tracemalloc.start()
total = sum(squares_list(1_000_000))
list_peak = tracemalloc.get_traced_memory()[1]
tracemalloc.reset_peak()
total = sum(squares_gen(1_000_000))
gen_peak = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()

print(f"list peak:      {list_peak / 1e6:.1f} MB")
print(f"generator peak: {gen_peak:,} bytes")
```

```
list peak:      40.4 MB
generator peak: 1,168 bytes
```

- `tracemalloc.start()` starts recording memory use; `get_traced_memory()` returns the current and the peak amount, in bytes, and `[1]` takes the peak.
- `reset_peak()` starts a new peak, so the second measurement is the generator's alone.
- With the numbers counted, the list needs about 40 MB at its peak and the generator a few hundred bytes: roughly 100,000 times less.

Tier	What to say
Passes	"Generators are more memory efficient" (true, no sense of the size of the difference)
Strong	The mechanism above, plus the catch: a generator can be read only once, while a list can be looped over again and again
Extra points	[+Validate] measured, not guessed: about 40 MB for the list against a few hundred bytes for the generator, and [+Limits] the honest caveat that <code>getsizeof</code> alone understates the list five times over. [+Scale] this is why reading a huge file or query result with a generator, or with pandas' <code>chunksize</code> (Q72-052), avoids running out of memory on data that would never fit in one list or DataFrame

Likely follow-ups: When would you *not* want a generator, even for a large sequence? (When you need the values twice, or need `len()` or indexing.) What's a generator expression, versus a generator function? (`(i*i for i in range(n))`): the same idea in round brackets, Chapter 33, section 33.2.)

Red flag: not knowing a generator can be consumed only once, or claiming it has no downsides.

Learn it in: Chapter 29, section 29.9 ("Step 5: a generator hands out the leads") and Chapter 33, section 33.9 (memory and `tracemalloc`).

Q72-030 · Write a decorator that times how long a function takes to run

Level: Advanced · Beyond the book

Remember it as: A decorator is a function that takes a function and returns a new function that wraps it: extra behavior added around the original, without changing the original's own code.

Answer in one line: `@decorator` above a function definition is shorthand for `function = decorator(function)`, and a typical decorator defines an inner `wrapper` function that does something extra, then calls the original inside it.

Beyond the book. Chapter 29, section 29.5 uses decorators (`@property`, `@classmethod`) and Chapter 33, section 33.8 uses `@lru_cache`; writing one is new. Three ideas make it work. A function can be passed around like any value. A function defined inside another function can use the outer one's names (a closure, Q72-004). And `*args`, `**kwargs` (Q72-007) let the wrapper accept and pass on whatever arguments the original takes.

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        t1 = time.perf_counter()
        print(f"{func.__name__} took {t1 - t0:.6f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))
```

- **timer(func)** receives the function being decorated and returns `wrapper` in its place.
- **wrapper(*args, **kwargs)** starts the stopwatch (`time.perf_counter()`, Chapter 18, section 18.1), calls the original with the same arguments, prints the time, and hands back the original's result unchanged.
- **@wraps(func)** keeps `slow_sum`'s own name and docstring on the wrapper (Q72-032).
- **@timer** above `def slow_sum` means `slow_sum = timer(slow_sum)`.

Now call it. Timings change on every run, so the first line of the output will differ on your machine:

```
result = slow_sum(1_000_000)
print("result:", result)
```

```
slow_sum took 0.011286s
result: 499999500000
```

The result is unchanged by the decorator: 499,999,500,000 is the sum of 0 to 999,999. And thanks to `@wraps`, the decorated function still knows its own name:

```
print(slow_sum.__name__)
```

```
slow_sum
```

Tier	What to say
Passes	Can describe what a decorator does in English, struggles to write one from scratch
Strong	The working <code>timer</code> decorator above, using <code>*args</code> , <code>**kwargs</code> (Q72-007) so it works on any function's signature, and <code>@wraps(func)</code>
Extra points	[+Validate] the timing line and the unchanged result (499,999,500,000) shown together, proving the decorator adds behavior without changing the return value. [+Business] a timing or logging decorator is one of the most common real uses in data code, wrapping pipeline steps to log how long each took

Likely follow-ups: What does `functools.wraps` do, and why add it? (Q72-032.) What if the decorated function takes arguments of its own? (*Already handled, by `*args`, `**kwargs` inside `wrapper`.*)

Red flag: a decorator without `*args`, `**kwargs`, which then works only on functions with one hard-coded signature.

Learn it in: Beyond the book (decorators in use: Chapter 29, section 29.5 and Chapter 33, section 33.8).

Rapid-fire, §72.3

#	Question	One-line answer	Extra point
Q72-031	<i>Advanced, Beyond the book.</i> What's a closure?	A function that "remembers" variables from the code it was defined in, even after that outer code has finished running	[+Signpost] Q72-004's late-binding bug is a closure behaving exactly as designed; the surprise is <i>when</i> the variable is read, not that it's remembered
Q72-032	<i>Advanced, Beyond the book.</i> What does <code>functools.wraps</code> do inside a decorator?	Copies the original function's name, docstring, and other details onto the wrapper, which would otherwise show the wrapper's own	[+Business] without it, error messages and documentation tools show wrapper instead of the real function's name, which makes tracebacks confusing
Q72-033	<i>Core.</i> What's the difference between a function and a method?	A method is a function defined inside a class; an ordinary method receives the instance (<code>self</code>) as its first argument (<code>@classmethod</code> receives the class, <code>@staticmethod</code> receives neither)	[+Edge cases] <code>@staticmethod</code> is Beyond the book: a plain function kept inside a class for tidiness
Q72-034	<i>Core.</i> What does <code>*</code> alone (with nothing after it) do in a function signature?	Every parameter after it must be passed by keyword, not by position	[+Business] used in well-designed APIs, like Chapter 29's <code>CrmClient</code> , so a caller can't pass several same-typed settings in the wrong order

Learn it in: Q72-031 and Q72-032 Beyond the book · Q72-033 Chapter 29, section 29.5 (methods, `self`, `@classmethod`) · Q72-034 Chapter 29, section 29.9 ("The robust client, as a class").

72.4 pandas fundamentals: Series, DataFrames, and selection

Every pandas cell from here on needs pandas imported once, as in Chapter 18:

```
import pandas as pd
print(pd.__version__)
```

3.0.6

Q72-035 · What's the difference between a pandas Series and a DataFrame, in one sentence?

Level: Warm-up

Remember it as: A *Series* is one labeled column. A *DataFrame* is a table of them, sharing a common row index.

Answer in one line: A **Series** is a one-dimensional labeled array (one column, with an index); a **DataFrame** is a two-dimensional table, a collection of Series sharing the same row index, each with its own name and data type.

Tier	What to say
Passes	"A DataFrame is like a table, a Series is like a column"
Strong	+ explicitly: <code>df["column"]</code> returns a Series; selecting several columns (<code>df[["a", "b"]]</code>) returns a DataFrame
Extra points	[+Edge cases] <code>df["column"]</code> (single brackets) returns a Series; <code>df[["column"]]</code> (double brackets) returns a one-column DataFrame: a common source of "why doesn't this method exist" errors, since Series and DataFrame methods overlap but aren't identical

Likely follow-ups: How would you turn a Series back into a one-column DataFrame? (`.to_frame()`)
What does a DataFrame's `.index` represent?

Red flag: not knowing the single-bracket vs. double-bracket distinction, or why it matters.

Learn it in: Chapter 18, section 18.1 ("DataFrames and Series").

Q72-036 · Does `SettingWithCopyWarning` still happen in current pandas, and what changed?

Level: Core

Remember it as: In pandas 3 every filtered table behaves like its own copy. Changing it never changes the original. To change the original, use `.loc` on it.

Answer in one line: Before pandas 3.0, filtering a DataFrame (`sub = df[condition]`) and then assigning to a column of `sub` might or might not change `df`, and pandas warned `SettingWithCopyWarning`; **pandas 3.0 turns on Copy-on-Write permanently**, so a filtered subset always behaves as its own copy and that warning is gone. Chained assignment (`df[mask]["col"] = x`) now never updates `df`, and pandas warns `ChainedAssignmentError: write df.loc[mask, "col"] = x`.

A three-row table:

```
df = pd.DataFrame({
    "customer": ["A", "B", "C"],
    "category": ["Storage", "Kitchen", "Storage"],
    "quantity": [3, 10, 2],
})
print(df)
```

	customer	category	quantity
0	A	Storage	3
1	B	Kitchen	10
2	C	Storage	2

Changing a filtered subset changes the subset only, with no warning:

```
sub = df[df["category"] == "Storage"]
sub["quantity"] = sub["quantity"] * 2
print(sub)
```

	customer	category	quantity
0	A	Storage	6
2	C	Storage	4

```
print(df)
```

	customer	category	quantity
0	A	Storage	3
1	B	Kitchen	10
2	C	Storage	2

`sub` has the doubled quantities, and `df` is untouched. That is Copy-on-Write: `sub` shares `df`'s data until one of them is changed, and then gets its own copy.

Chained assignment, filtering and assigning in one line, never reaches `df`:

```
df[df["category"] == "Storage"]["quantity"] = 99
print(df)
```

	customer	category	quantity
0	A	Storage	3
1	B	Kitchen	10
2	C	Storage	2

`df` is unchanged, and beneath the cell pandas prints a warning that starts `ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment`. The filter makes a new table, the `= 99` lands on that, and the new table is thrown away.

The one-step way to change the original is `.loc[rows, column]`:

```
df.loc[df["category"] == "Storage", "quantity"] = 99
print(df)
```

```
   customer category  quantity
0         A  Storage         99
1         B  Kitchen         10
2         C  Storage         99
```

Tier	What to say
Passes	Describes the old <code>SettingWithCopyWarning</code> trap as if it's still current behavior, without checking the pandas version
Strong	Distinguishes the versions: on an older pandas (before 3.0, still common in production code), the warning is real and the safe habit is <code>.copy()</code> when you mean a new table and <code>.loc</code> when you mean the original; on pandas 3.0+, a filtered subset is always its own copy, chained assignment never updates the original (and warns <code>ChainedAssignmentError</code>), and <code>.loc</code> is the one way to change <code>df</code>
Extra points	[+Validate] the real outputs above: <code>sub</code> changed and <code>df</code> provably untouched; the chained version warned and changed nothing; <code>.loc</code> changed rows 0 and 2. [+Business] confirm which pandas version a team runs before answering either way: this is exactly the kind of fast-changing fact to check, not recite

Likely follow-ups: What is Copy-on-Write, in one sentence? (*Tables share data until one is changed, and the change then happens on a private copy.*) What's the safe pattern on an older pandas you don't control? (`.copy()` for a new table, `.loc` for the original.)

Red flag: describing `SettingWithCopyWarning` as current behavior without saying which pandas version that applies to.

Learn it in: Chapter 18, section 18.4 (the chained-assignment cell and its "Watch out").

Rapid-fire, §72.4

#	Question	One-line answer	Extra point
Q72-037	Warm-up. <code>.loc</code> vs. <code>.iloc</code> ?	Label-based selection (<code>.loc[row_label, col_label]</code>) / position-based selection (<code>.iloc[row_number, col_number]</code>)	[+Edge cases] <code>.loc</code> 's slices <i>include</i> the end label; <code>.iloc</code> 's follow normal Python rules and exclude the end. With the default 0, 1, 2... index, <code>df.loc[0:2]</code> returns 3 rows and <code>df.iloc[0:2]</code> returns 2 (run below)
Q72-038	Warm-up. What does <code>df.dtypes</code> tell you, and why check it early?	The data type pandas chose for each column	[+Business] a numeric-looking column stored as text (<code>str</code> in pandas 3.0+, object in earlier versions) usually means non-numeric values mixed in, worth catching before any calculation (Q72-054)
Q72-039	Warm-up. How do you check for and count duplicate rows?	<code>df.duplicated().sum()</code> counts them; <code>df[df.duplicated()]</code> shows them; <code>df.drop_duplicates()</code> removes them	[+Edge cases] <code>.duplicated()</code> treats the <i>first</i> occurrence as the original by default; <code>keep="last"</code> or <code>keep=False</code> change that
Q72-040	Warm-up. What's <code>df.info()</code> useful for that <code>df.head()</code> isn't?	Row count, column types, and non-blank counts per column, all at once: a fast structural health check	[+Signpost] the pandas twin of Chapter 71's SQL table-profiling walk-through (Q71-076, section 71.10): one call, several facts about data you've never seen

Q72-037, run on the three-row df:

```
print(len(df.loc[0:2]), len(df.iloc[0:2]))
```

```
3 2
```

Learn it in: Q72-037 Chapter 18, section 18.4 (loc and iloc) · Q72-038 and Q72-040 section 18.3 · Q72-039 section 18.10.

72.5 pandas: grouping, joining, and reshaping

The questions in this section and the next use one small table of six orders and four customers, a toy built for the questions, not Riverstone's data. Build and print both first:

```
orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5, 6],
    "order_date": pd.to_datetime(["2025-01-05", "2025-01-06", "2025-01-06",
                                   "2025-01-07", "2025-01-08", "2025-01-09"]),
    "customer": ["Sharma Hardware", "Metro Mart", "Sharma Hardware",
                 "Coastal Foods", "Metro Mart", "Sharma Hardware"],
    "category": ["Storage", "Kitchen", "Cleaning", "Storage", "Kitchen", "Storage"],
    "quantity": [3, 10, 4, 5, 2, 2],
    "unit_price": [320.0, 85.0, 90.0, 980.0, 260.0, 320.0],
})
customers = pd.DataFrame({
    "customer": ["Sharma Hardware", "Metro Mart", "Coastal Foods", "Home Plus"],
    "city": ["Pune", "Mumbai", "Chennai", "Delhi"],
})
print(orders)
print(customers)
```

	order_id	order_date	customer	category	quantity	unit_price
0	1	2025-01-05	Sharma Hardware	Storage	3	320.0
1	2	2025-01-06	Metro Mart	Kitchen	10	85.0
2	3	2025-01-06	Sharma Hardware	Cleaning	4	90.0
3	4	2025-01-07	Coastal Foods	Storage	5	980.0
4	5	2025-01-08	Metro Mart	Kitchen	2	260.0
5	6	2025-01-09	Sharma Hardware	Storage	2	320.0

	customer	city
0	Sharma Hardware	Pune
1	Metro Mart	Mumbai
2	Coastal Foods	Chennai
3	Home Plus	Delhi

`pd.to_datetime` turns the date text into real dates (Chapter 18, section 18.9). Home Plus is in `customers` but has no orders, on purpose.

Q72-041 · Group the orders by customer, computing total revenue, order count, and average quantity in one call

Level: Core

Remember it as: `.agg()` with named outputs beats calling `.sum()`, `.count()`, and `.mean()` separately: one grouping step, self-documenting column names.

Answer in one line: `.groupby(col).agg(name=("column", "function"), ...)` computes several aggregations, on different columns if you like, from one grouping step, with output column names you choose.

```
orders["revenue"] = orders["quantity"] * orders["unit_price"]
print(orders.groupby("customer").agg(
    total_revenue=("revenue", "sum"),
    orders=("order_id", "count"),
    avg_qty=("quantity", "mean"),
))
```

	total_revenue	orders	avg_qty
customer			
Coastal Foods	4900.0	1	5.0
Metro Mart	1370.0	2	6.0
Sharma Hardware	1960.0	3	3.0

- The first line adds a `revenue` column, quantity times price, row by row.
- `total_revenue=("revenue", "sum")` reads "a column called `total_revenue`, made by summing `revenue`". `orders=("order_id", "count")` counts each customer's order IDs, and `avg_qty=("quantity", "mean")` averages their quantities.

Tier	What to say
Passes	Three separate <code>.groupby().sum()</code> , <code>.groupby().count()</code> , <code>.groupby().mean()</code> calls, then joining the results by hand: works, but three groupings and more code to keep in step
Strong	The single <code>.agg()</code> call above, with named outputs
Extra points	[+Validate] the output reconciles by hand: Sharma Hardware's three orders are $3 \times 320 + 4 \times 90 + 2 \times 320 = 1,960$, with quantities 3, 4, and 2 averaging 3.0; Metro Mart's are $10 \times 85 + 2 \times 260 = 1,370$. [+Business] named aggregation gives readable column names at once, where the older <code>.agg({"revenue": "sum"})</code> style leaves you renaming columns afterwards

Likely follow-ups: How would you group by two columns at once? What's the difference between `.agg()` and `.transform()`? (Q72-046.)

Red flag: running several separate `.groupby()` calls and merging the results by hand, instead of one `.agg()` call.

Learn it in: Chapter 18, section 18.6 (named aggregation).

Q72-042 · Merge an orders table with a customers table, and find any customer with zero orders

Level: Core

Remember it as: An inner join only shows what matches on both sides. Finding "nothing matched" needs a left join from the side you want to keep, checked for the rows that found no partner: the pandas version of Chapter 71's SQL LEFT JOIN / IS NULL pattern.

Answer in one line: `customers.merge(orders, on="customer", how="left", indicator=True)`, then keep the rows whose `_merge` column says `left_only`: customers with no matching order, exactly the SQL LEFT JOIN ... WHERE ... IS NULL of Chapter 71 (Q71-001).

```
m = customers.merge(orders, on="customer", how="left", indicator=True)
print(m[m["_merge"] == "left_only"][["customer", "city"]])
```

```
   customer  city
6  Home Plus  Delhi
```

- `how="left"` keeps every customer, with or without orders; customers with orders appear once per order.
- `indicator=True` adds a `_merge` column saying where each row came from: `both`, or `left_only` when no order matched.
- The filter keeps the `left_only` rows, and `[["customer", "city"]]` shows two columns. Home Plus is row 6 because the six rows for customers with orders come first.

The same answer, with the tables the other way round, is `how="right"` checked for a missing `order_id`:

```
merged_right = orders.merge(customers, on="customer", how="right")
print(merged_right[merged_right["order_id"].isna()][["customer", "city"]])
```

```
   customer  city
6  Home Plus  Delhi
```

Tier	What to say
Passes	<code>orders.merge(customers, on="customer")</code> with the default <code>how="inner"</code> : silently drops customers with no orders instead of surfacing them
Strong	The <code>how="left", indicator=True</code> version above, finding the one customer (Home Plus) with no orders
Extra points	[+Validate] also check that the plain inner merge keeps every order: its row count equals <code>orders</code> 's (run below: 6 and 6), so no order was dropped for lack of a known customer. [+Edge cases] the same fan-out risk as Chapter 71's SQL joins: a duplicate customer in <code>customers</code> would silently multiply the matching order rows, because <code>merge</code> follows the same join rules as SQL; <code>validate="many_to_one"</code> makes pandas stop instead

```
print(len(orders.merge(customers, on="customer")), len(orders))
```

```
6 6
```

Likely follow-ups: How would you do this with `how="outer"`, and what changes? (Unmatched rows from both sides appear, marked `left_only` or `right_only`.)

Red flag: using the default inner merge and never checking whether any expected rows disappeared.

Learn it in: Chapter 18, section 18.7 (merge, `indicator=True`, fan-out) and Chapter 71, section 71.1 for the SQL side.

Rapid-fire, §72.5

#	Question	One-line answer	Extra point
Q72-043	Core. <code>pivot_table</code> vs. <code>groupby</code> ?	<code>pivot_table</code> reshapes grouped results into a wide table (categories become columns) / <code>groupby</code> keeps results long	[+Validate] run below: each customer's spend lands under the right category column, zeros elsewhere
Q72-044	Warm-up. What does <code>fill_value=0</code> do in <code>pivot_table</code> , and why not leave the default?	Fills cells with no matching data (a customer-category pair that never happened) with 0 instead of NaN	[+Edge cases] a NaN left in a pivot meant for summing or charting can break later arithmetic; 0 is usually the right "this didn't happen"
Q72-045	Warm-up. What's <code>pd.concat()</code> for, and how is it different from <code>merge</code> ?	Stacks DataFrames, by rows (the default) or columns, by position, not by matching key values	[+Signpost] <code>concat</code> is pandas' UNION ALL (Chapter 71, Q71-006); <code>merge</code> is pandas' JOIN
Q72-046	Core. What does <code>.transform()</code> do that <code>.agg()</code> doesn't?	Returns one value per original row (each row gets its group's result), rather than one row per group	[+Business] useful for adding "this row's share of its group's total" directly onto the original, uncollapsed table

Q72-043, run:

```
print(orders.pivot_table(index="customer", columns="category",
                          values="revenue", aggfunc="sum", fill_value=0))
```

```
category
customer
Coastal Foods      0.0      0.0  4900.0
Metro Mart         0.0  1370.0     0.0
Sharma Hardware   360.0      0.0  1600.0
```

- `index="customer"` gives one row per customer, and `columns="category"` one column per category.
- `values="revenue"` with `aggfunc="sum"` puts the total revenue of each customer-category pair in its cell: Sharma Hardware's 1,600 in Storage is $3 \times 320 + 2 \times 320$.
- `fill_value=0` writes 0 where a pair never happened. Leave it out and those cells show NaN instead (Q72-044).

Learn it in: Q72-043 and Q72-044 Chapter 18, section 18.8 · Q72-045 section 18.7 · Q72-046 section 18.6 (`transform`).

72.6 pandas: performance, missing data, strings, and dates

Q72-047 · Why is `.apply()` with `axis=1` almost always slower than a vectorized operation, and how much slower in practice?

Level: Core

Remember it as: `.apply(axis=1)` is a Python for-loop in disguise. Vectorized pandas operations run in fast, compiled code underneath, on the whole column at once.

Answer in one line: `.apply(func, axis=1)` calls `func` once per row, in plain Python, as slow as an explicit loop; a vectorized operation (`df["a"] * df["b"]`) works on whole columns at once in compiled code, with no Python loop at all.

A table of 200,000 rows, built without NumPy:

```
big = pd.DataFrame({"a": range(200_000), "b": range(200_000)})
print(big.shape)
```

```
(200000, 2)
```

Now time both ways with Chapter 18's stopwatch, `time.perf_counter()`. Timings change on every run and every machine, so this cell's numbers will differ on yours:

```
import time

t0 = time.perf_counter()
r1 = big.apply(lambda row: row["a"] * row["b"], axis=1) # row by row
t1 = time.perf_counter()
r2 = big["a"] * big["b"]                               # vectorized
t2 = time.perf_counter()

print(f"apply:      {t1 - t0:.4f} s")
print(f"vectorized: {t2 - t1:.6f} s")
print(f"speedup:    {(t1 - t0) / (t2 - t1):.0f}x")
```

```
apply:      0.9891 s
vectorized: 0.001292 s
speedup:    766x
```

Fast is only useful if the answer is the same. Check every row:

```
r1 = big.apply(lambda row: row["a"] * row["b"], axis=1)
r2 = big["a"] * big["b"]
print((r1 == r2).all())
```

```
True
```

`r1 == r2` compares the two columns row by row, and `.all()` is `True` only if every one of the 200,000 rows matches.

Tier	What to say
Passes	"Vectorized code is faster" (true, no sense of <i>how much</i> , no explanation)
Strong	The compiled-code-versus-Python-loop explanation above, identifying <code>.apply(axis=1)</code> as a loop wearing a pandas costume
Extra points	[+Validate] measured, not asserted: several hundred times faster on the machine used for this book (766× in the run above; your number will differ), with every row checked to match. [+Business] on a large table this is the difference between a report that runs in a second and one that takes minutes, with no change to the logic

Likely follow-ups: Is `.apply()` ever the right choice? (Yes, when the per-row logic genuinely can't be vectorized: complex branching, or calling an outside function per row.) Is `.apply()` on a single column faster than the row-wise version? (Yes, but still a loop: look for a `.str`, `.dt`, `.map`, or arithmetic equivalent first.)

Red flag: reaching for `.apply(axis=1)` for row-wise arithmetic that could be a direct column operation.

Learn it in: Chapter 18, section 18.1 ("Whole columns, not loops"), section 18.5 (`.apply`, "a loop wearing a costume"), and section 18.16.

Q72-048 · Handle missing data in a numeric column: detect it, then decide between filling and dropping

Level: Core

Remember it as: Detect first, decide second. Filling and dropping are both valid, but only after you know how much is missing and why.

Answer in one line: `.isna().sum()` counts missing values; `.fillna(value)` replaces them (a mean, median, zero, or a forward fill, depending on context); `.dropna(subset=[...])` removes rows missing a required value. The right choice depends on *why* the data is missing, not on habit.

First, a copy of `orders` with one price blanked out, so there is something to find:

```
orders2 = orders.copy()
orders2.loc[2, "unit_price"] = None
print(orders2["unit_price"].isna().sum())
```

```
1
```

- `.copy()` makes `orders2` a separate table, so `orders` keeps its prices.
- `.loc[2, "unit_price"] = None` blanks the price at index 2, the third row. pandas stores the blank as NaN.
- `.isna().sum()` counts the blanks: one.

Filling with the mean of the prices that are there:

```
print(orders2["unit_price"].fillna(orders2["unit_price"].mean()).tolist())
```

```
[320.0, 85.0, 393.0, 980.0, 260.0, 320.0]
```

The blank at index 2 became 393.0, the mean of the five known prices: $(320 + 85 + 980 + 260 + 320) \div 5 = 1,965 \div 5 = 393$. `.tolist()` prints the column as a plain list.

Dropping instead:

```
print(orders2.dropna(subset=["unit_price"]).shape)
```

```
(5, 7)
```

`subset=["unit_price"]` drops only rows with a blank *price*; the result has 5 rows and all 7 columns.

Tier	What to say
Passes	"Use <code>.fillna()</code> or <code>.dropna()</code> " with no discussion of when each fits
Strong	+ explicitly: filling with a mean or median is reasonable when a value is plausibly missing at random and you need every row later; dropping is more honest when a blank means something you don't want to guess at, and losing a few rows doesn't bias the analysis
Extra points	[+Business] a silent fill can hide a real data-quality problem from everyone who reads the numbers later: pair a fill with a comment or a <code>was_imputed</code> column when it matters. [+Edge cases] the mean is computed from whatever rows exist <i>now</i> ; fill before you've settled the final row set and you bake in a slightly wrong mean

Likely follow-ups: When would forward fill (`.ffill()`) fit better than a mean? (A *reading that holds until it changes*, such as a price list over time.) How would you handle blanks in a text column? (A *label such as "(unknown)"*, as Chapter 18 does for missing reps.)

Red flag: filling every missing value with 0 by default, whatever the column means.

Learn it in: Chapter 18, sections 18.3 (`.isna()`), 18.5 (`.fillna()`), and 18.10 (cleaning); Chapter 14 for deciding what to do with blanks.

Rapid-fire, §72.6

#	Question	One-line answer	Extra point
Q72-049	<i>Warm-up.</i> Extract the first word of each customer name using <code>.str</code> ?	<code>df["customer"].str.split().str[0]</code>	[+Validate] run below: "Sharma Hardware" → "Sharma", "Metro Mart" → "Metro"
Q72-050	<i>Warm-up.</i> Get the day of the week from a date column?	<code>df["date"].dt.day_name()</code> , or <code>.dt.dayofweek</code> for a number from 0 (Monday) to 6	[+Validate] run below: 2025-01-05 → "Sunday", 2025-01-06 → "Monday"
Q72-051	<i>Core.</i> What does <code>.str.contains("text")</code> give for a missing value?	pandas 3.0+ (str dtype): False. Older pandas, or an object column: a missing value (None/NaN), and using that result as a filter raises <code>ValueError: Cannot mask with non-boolean array containing NA / NaN values</code>	[+Edge cases] pass <code>na=False</code> and both versions behave the same: a missing value counts as "no match"

#	Question	One-line answer	Extra point
Q72-052	Core. How would you process a file too large to fit in memory, using pandas?	<code>pd.read_csv(path, chunksize=100_000)</code> returns an iterator of DataFrame pieces, processed one at a time	[+Scale] the pandas version of Q72-029's generator idea: memory stays bounded however big the file is

Q72-049 and Q72-050, run on the first two orders:

```
print(orders["customer"].str.split().str[0].head(2).tolist())
print(orders["order_date"].dt.day_name().head(2).tolist())
```

```
['Sharma', 'Metro']
['Sunday', 'Monday']
```

Q72-051, the two behaviors side by side. The first Series gets pandas 3's `str` dtype; `dtype=object` makes the second behave like older pandas:

```
names = pd.Series(["Sharma Hardware", None, "Metro Mart"])
old_style = pd.Series(["Sharma Hardware", None, "Metro Mart"], dtype=object)
print(names.dtype, names.str.contains("Mart").tolist())
print(old_style.dtype, old_style.str.contains("Mart").tolist())
print(old_style.str.contains("Mart", na=False).tolist())
```

```
str [False, False, True]
object [False, None, True]
[False, False, True]
```

- **names** is an ordinary Series of text, so pandas 3 gives it the `str` dtype, and the missing name gives `False`.
- **dtype=object** forces the older, generic type: now the missing name gives `None`, which is neither `True` nor `False`, so `old_style[old_style.str.contains("Mart")]` would stop with the `ValueError` in the table.
- **na=False** says "count a missing value as no match", and the result is the same in both versions.

Learn it in: Q72-049 Chapter 18, section 18.5 (the `.str` accessor; `.str.split` and `.str.contains` are Beyond the book) · Q72-050 section 18.9 · Q72-051 Beyond the book · Q72-052 section 18.13 (`chunksize` on a database read; the same option works on `read_csv`).

72.7 Debugging and live-coding walk-throughs

Q72-053 · Fix this function: it's supposed to return each customer's share of total revenue, but it always returns 0 or 1

Level: Warm-up

Remember it as: `//` throws away everything after the decimal point, and a share is always below 1, so it becomes 0.

```
# BROKEN
def revenue_share(customer_revenue, total_revenue):
    return customer_revenue // total_revenue

print(revenue_share(1960, 8230))
```

```
0
```

What's wrong: `//` is whole-number (floor) division, not ordinary division. Any `customer_revenue` smaller than `total_revenue` floors to 0, and a customer who is the whole total gets 1: it can never return a fraction like 0.24.

```
# FIXED
def revenue_share(customer_revenue, total_revenue):
    return customer_revenue / total_revenue

print(revenue_share(1960, 8230))
```

```
0.23815309842041313
```

Tier	What to say
Diagnosis	<code>//</code> floors instead of dividing
Fix	A one-character change, <code>//</code> to <code>/</code>
Extra points	[+Validate] test with real numbers: Sharma Hardware's 1,960 against the toy table's total of 8,230 (Q72-041: 4,900 + 1,370 + 1,960) gives 0 broken and about 0.238 fixed. [+Edge cases] check <code>total_revenue == 0</code> separately either way: both <code>/</code> and <code>//</code> raise <code>ZeroDivisionError</code> on a zero total, and a customer total of zero is worth investigating on its own

Likely follow-ups: How is `/` in Python 3 different from Python 2's? (In Python 2, `/` on two whole numbers floored, like `//`.) How would the same bug look in pandas, on a whole column? (`df["revenue"] // total`, and the same fix.)

Red flag: not recognizing `//` as floor division at once, or hunting for something more complicated than the operator.

Learn it in: Chapter 17, section 17.3 (`//` is whole-number division).

Q72-054 · Walk-through: clean and validate a messy, realistic order export

Level: Core

What they're really testing: whether you have a repeatable process for a messy real file, not just a single trick.

Talked through live: "I'll check shape and dtypes first, look for duplicates, check for missing values column by column, and only then start fixing anything, since fixing before I understand what's wrong risks masking a bigger problem."

A deliberately messy small export:

```
messy = pd.DataFrame({
    "order_id": [1001, 1002, 1002, 1004, 1005],
    "customer": [" Sharma Hardware", "Metro Mart", "Metro Mart",
                "Coastal Foods", "Home Plus"],
    "quantity": ["3", "10", "10", "ten", None],
})
print("duplicate order_ids:", messy.duplicated(subset=["order_id"]).sum())
print("quantity dtype:", messy["quantity"].dtype)
print("missing quantities:", messy["quantity"].isna().sum())
print("names with a leading space:", messy["customer"].str.startswith(" ").sum())
```

```
duplicate order_ids: 1
quantity dtype: str
missing quantities: 1
names with a leading space: 1
```

- `.duplicated(subset=["order_id"])` marks each row whose `order_id` appeared earlier: order 1002 is there twice.
- **The dtype is str:** quantity is stored as text, not numbers, because one value is the word "ten".
- **One quantity is missing** (order 1005).
- `.str.startswith(" ")` finds the leading space in " Sharma Hardware", which would break an exact-match join.

The fix, applied in order:

```
clean = messy.drop_duplicates(subset=["order_id"]).copy()
clean["customer"] = clean["customer"].str.strip()
clean["quantity"] = pd.to_numeric(clean["quantity"], errors="coerce")
print(clean)
print(clean.dtypes)
```

```
   order_id  customer  quantity
0      1001 Sharma Hardware      3.0
1      1002   Metro Mart    10.0
3      1004 Coastal Foods      NaN
4      1005   Home Plus      NaN
order_id    int64
customer     str
quantity  float64
dtype: object
```

- `.drop_duplicates(subset=["order_id"])` keeps the first copy of order 1002. `.copy()` makes `clean` its own table on purpose; on pandas before 3.0 it also avoids the `SettingWithCopyWarning` of Q72-036 on the next two lines.
- `.str.strip()` removes the leading space.
- `pd.to_numeric(..., errors="coerce")` turns text into numbers, and anything it can't read, here "ten", into `NaN` instead of stopping. The missing quantity stays missing.
- **quantity is now float64** (3.0, 10.0): a column of whole numbers with a blank in it becomes decimal numbers, because `NaN` is a float.

To keep whole numbers alongside blanks, use pandas' nullable integer type, `"Int64"` with a capital I:

```
print(clean["quantity"].astype("Int64").tolist())
```

```
[3, 10, <NA>, <NA>]
```

`.astype("Int64")` converts the column to that type: 3 and 10 are whole numbers again, and the two blanks show as `<NA>`, pandas' own missing-value marker. `.tolist()` prints the column as a list.

Extra-points moves demonstrated: **[+Signpost]** diagnosed before fixing, in a stated order. **[+Validate]** every claim (a duplicate, a text dtype, a missing value, a leading space) backed by a check that printed a number, not assumed. **[+Edge cases]** `errors="coerce"` turned the genuinely bad value "ten" into a `NaN` that can be counted and reported, instead of crashing the whole pipeline.

Likely follow-ups: How would you decide whether to keep the first or the last copy of a duplicate order? What would you do with the two `NaN` quantities now? (Report them, with the order IDs, before deciding to fill or drop: Q72-048.)

Red flag: starting to "fix" a file (dropping rows, filling values) before running any checks on what's wrong with it.

Learn it in: Chapter 18, section 18.10 (cleaning in pandas) and Chapter 14; Chapter 71's table-profiling walk-through (Q71-076, section 71.10) for the same discipline in SQL.

APPLY

Common mistakes

Mistake	Symptom	Fix
A mutable default argument (<code>def f(x, items=[])</code>)	Calls silently accumulate state across unrelated calls	Use <code>items=None</code> , then if <code>items is None: items = []</code> inside
Using <code>is</code> to compare values	<code>if x is []</code> : never matches, even for an empty list	Keep <code>is</code> for <code>None/True/False</code> ; use <code>==</code> for values
Assuming <code>.copy()</code> fully isolates nested data	Changing a "copy" silently changes the original too	Use <code>copy.deepcopy()</code> when the container holds other mutable containers
Citing <code>SettingWithCopyWarning</code> as current pandas behavior with no version check	Confidently wrong on pandas 3.0+, where Copy-on-Write is always on	Say which pandas version the answer applies to

Mistake	Symptom	Fix
Chained assignment (<code>df[mask]["col"] = x</code>)	Nothing changes in <code>df</code> ; pandas 3 warns <code>ChainedAssignmentError</code>	<code>df.loc[mask, "col"] = x</code>
<code>.apply(axis=1)</code> for simple row-wise arithmetic	Hundreds to thousands of times slower than the vectorized version	Direct column operations (<code>df["a"] * df["b"]</code>) whenever possible
Inner-joining without checking for dropped rows	Missing customers or records go unnoticed	Compare row counts before and after; use <code>how="left"</code> with <code>indicator=True</code> when rows shouldn't disappear
Filling every missing value with 0 by habit	Real "unknown" values are treated as real zeros in later math	Decide fill vs. drop based on <i>why</i> the value is missing
"Fixing" a messy file before diagnosing it	Over-correcting, or missing the real problem	Check shape, dtypes, duplicates, and missing values first, in that order

In the real world: the report that "randomly" gave different numbers each run

Karan is asked to debug a weekly revenue report that sometimes produces a slightly different, lower total for the same underlying data, whenever the report ran more than once in the same session. He traces it to a helper function with a mutable default argument, exactly Q72-001's bug, used to accumulate a running list of "already counted" order IDs. The list was meant to reset for every report run, but never did, since the default list was created once, when the module was first imported, so every later run skipped the orders the earlier runs had counted.

The fix was the one-line change Q72-001 already teaches (`orders_seen=None` instead of `orders_seen=[]`), but the part that mattered in the interview retelling wasn't the fix, it was how Karan found it: rather than guessing, he added a single `print(len(orders_seen))` at the top of the function and ran the report twice in the same Python session, watching the count grow between runs when it should have reset to zero each time. That one line of evidence turned a vague "it's inconsistent somehow" into an exact, provable diagnosis in under two minutes.

The lesson the hiring manager wrote in her notes afterward: *"Didn't guess. Added one line to make the invisible state visible, then the bug explained itself."* That's Chapter 69's Move 7 (validate), applied to debugging instead of a finished answer, and it's exactly the difference this chapter's format tries to model in every question: don't just know the fix, show the evidence.

Project

Goal: find and fix a real instance of at least three of this chapter's core bugs, in your own code or a public repository.

Tools you'll need

Python and the book's virtual environment from Chapter 17, section 17.0 (the book recommends Python 3.14; this chapter's snippets were run on Python 3.11.15, and the generator sizes of Q72-029 also on 3.12 and 3.13), and **pandas** 3.0 or later (run on 3.0.6). `%timeit` in a Jupyter notebook, or the `time.perf_counter()` pattern used in this chapter, for any performance claim you make in an interview, rather than quoting a speedup from memory. **pandas' own release notes** are worth skimming before an interview at a company that uses pandas heavily, since behavior (like Copy-on-Write, Q72-036) does change between major versions.

1. Search your own past projects for a mutable default argument (`def f(x, y=[])` or `y={}`) and fix it, the way Q72-001 does.
2. Take one place in your own code that uses `.apply(axis=1)` and rewrite it as a vectorized operation; measure the real speedup and check the results match, the way Q72-047 does.
3. Pick a merge in your own work and verify the row count is what you expect, catching any silently dropped or duplicated rows (Q72-042).
4. Run `.isna().sum()` on a real dataset you have access to, and for each column with missing data, decide and justify fill vs. drop, in one sentence each.
5. Write one decorator of your own (logging, timing, or retry logic) and apply it to a real function you use regularly (Q72-030).

REVIEW

Key terms

mutable default argument · `is` vs. `==` · object identity · shallow copy · deep copy · closure · late binding · `*args` / `**kwargs` · keyword-only arguments (`*`) · list comprehension · generator · `tracemalloc` · decorator · `functools.wraps` · class attribute · walrus operator (`:=`) · pandas Series · pandas DataFrame · `.loc` vs. `.iloc` · Copy-on-Write · chained assignment · `ChainedAssignmentError` · `SettingWithCopyWarning` (legacy) · `groupby().agg()` · named aggregation · merge (join types) · `indicator=True` · `pivot_table` · `concat` · `.transform()` vs. `.agg()` · vectorization · `.apply(axis=1)` · `.isna()` / `.fillna()` / `.dropna()` · `.str` accessor · `.dt` accessor · `errors="coerce"` · nullable Int64 · `chunksize` (large-file reading)

Final-week revision list

Q72-001, Q72-002, Q72-003, Q72-004, Q72-029, Q72-030, Q72-035, Q72-036, Q72-041, Q72-042, Q72-047, Q72-048, Q72-053, Q72-054.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, already covered the join-cardinality and NULL-handling logic this chapter's pandas `merge` questions (section 72.5) reuse.
- **Chapter 72A, Data Structures & Algorithms Question Bank**, covers the classic algorithm problems (arrays, trees, graphs) this chapter leaves aside in favor of pandas-specific, data-role-shaped coding.
- **Chapter 74, Machine Learning Question Bank**, assumes this chapter's pandas fluency as a starting point, not something to re-teach.

Chapter 72A. Data Structures & Algorithms Question Bank

Chapter at a glance

You will practise: answering the core data structures and algorithms (DSA) questions that come up in data-role interviews, sized for those roles rather than a full software-engineering gauntlet · reasoning about time and space (Big-O) out loud, and backing the claim with a measurement · implementing and explaining the handful of structures and patterns that actually recur: hash maps and sets, two pointers and sliding windows, recursion and memoization, sorting and searching, linked lists, trees, and graphs.

Before you start: do Chapter 33 first (The Computer Science You Actually Need): it teaches almost every idea in this bank. You also need Chapter 17 (Python from zero), Chapter 29, section 29.5 (writing a class with `__init__` and `self`), and Chapter 69 (the three answer tiers and the twelve extra-point tags). Chapter 72's Python gotchas (mutable defaults, `is` versus `==`) come back here twice.

Time needed: 6–8 hours to run every snippet and answer each question aloud; 1 hour for a revision pass. Add 3–4 hours if the Chapter 33 sections named in the Learn-it-in lines are new to you.

How this chapter is built. Same format as the other question banks in Part 8: every core question leads with a "**Remember it as...**" hook, then a one-line answer, then a compact tier table (what **passes**, what's **strong**, and the **extra points**, tagged with Chapter 69's moves: **[+Clarify]**, **[+Edge cases]**, **[+Validate]** and so on). Rapid-fire sections are scan tables. **Every snippet was run, and every output shown is real**, on Python 3.11.15 with the standard library only; the book recommends Python 3.14 (Chapter 17, section 17.0), and none of these outputs depend on the version. Timings change on every run and every machine, so the cells that print them say so; run them yourself in a notebook (Chapter 17 set one up) and expect your numbers to differ, not the pattern.

Learn it in pointers name the chapter and section that teach each idea. Four topics go beyond Chapter 33: the two-pointer and sliding-window patterns, merge sort, linked lists, and binary search trees. Those questions are marked **Beyond the book** and carry a short primer you can learn from before the question itself.

In this chapter

LEARN	72A.1 Why data roles get asked this at all	149
	72A.2 Big-O: reasoning about time and space	149
	72A.3 Arrays, strings, and the two-pointer / sliding-window patterns	153
	72A.4 Hash maps and sets	157
	72A.5 Recursion and memoization	159
	72A.6 Sorting and searching	162
	72A.7 Linked lists	166
	72A.8 Trees and graphs	169
	72A.9 A live-coding walk-through: parentheses validation	174
APPLY	Common mistakes · In the real world · Project	175
REVIEW	Key terms · Final-week revision list	177
NEXT	Where this leads	177

LEARN

72A.1 Why data roles get asked this at all

Not every data role gets a DSA round. A Data Analyst or BA interview rarely does. A Data Engineer or ML Engineer interview, especially at a larger tech company, very often does, because the underlying skill (can you reason precisely about how a solution scales) is exactly what separates "a pipeline that works on the sample data" from "a pipeline that survives production volume." The bar is usually lower than a pure software-engineering interview: fewer obscure edge cases, more emphasis on getting a correct, reasonably efficient solution and explaining your reasoning clearly (Chapter 69's moves apply here as much as anywhere else in this part). Chapter 33, section 33.10 describes what that round looks like; this bank is the practice.

Levels and roles. Each question carries a level and the roles that usually ask it:

- **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds.
- **DE** data engineer · **AE** analytics engineer · **MLE** machine learning engineer · **DS** data scientist · **DA** data analyst.

Within each section the core questions run from easier to harder. If a DSA round is on your calendar for a first job, do every Fresher question first, then come back for the Mid ones.

72A.2 Big-O: reasoning about time and space

Q72A-001 · Explain Big-O notation to someone who's never seen it

Level: Fresher · **Roles:** DE, AE, MLE, DS, DA

Remember it as: Big-O answers one question: if the input doubles, does the work stay the same, double, or explode?

Answer in one line: Big-O describes how an algorithm's running time (or memory use) grows as the input size grows, ignoring constant factors and keeping only the dominant term as the input gets large.

The common orders, cheapest to most expensive:

Order	Name	Example	Steps for 5,000 items
$O(1)$	Constant	Dictionary lookup, list index access	1
$O(\log n)$	Logarithmic	Binary search	13
$O(n)$	Linear	A single loop through the data	5,000
$O(n \log n)$	Linearithmic	Merge sort, Python's <code>sorted()</code>	61,439
$O(n^2)$	Quadratic	A loop inside a loop over the same data	25,000,000
$O(2^n)$	Exponential	Naive recursive Fibonacci, trying every subset	2^{5000} , a 1,506-digit number: never finishes

The last column is plain arithmetic, and worth being able to do yourself:

```
import math

n = 5_000
print(f" $O(\log n)$ : {math.ceil(math.log2(n))} steps")
print(f" $O(n)$ : {n:,} steps")
print(f" $O(n \log n)$ : {n * math.log2(n):,.0f} steps")
print(f" $O(n^2)$ : {n * n:,} steps")
```

```
 $O(\log n)$ : 13 steps
 $O(n)$ : 5,000 steps
 $O(n \log n)$ : 61,439 steps
 $O(n^2)$ : 25,000,000 steps
```

- `math` is the standard library's maths module; `math.log2(n)` answers "how many times can you halve `n` before you reach 1?" (logarithms: Chapter 31, section 31.0). For 5,000 that's 12.3, so a binary search needs at most 13 looks; `math.ceil` rounds up to the next whole number.
- `{...:,}` and `{...:,.0f}` are the f-string formats from Chapter 17: thousands separators, and no decimals.

Tier	What to say
Passes	"It's how fast an algorithm is"
Strong	+ names the common orders (above) and what each looks like in code: one loop is $O(n)$, a loop inside a loop over the same data is $O(n^2)$
Extra points	[+Business] ties complexity to a real cost: an $O(n^2)$ report that's fine on 100 test rows can time out on 5 million production rows. [+Edge cases] Big-O is an upper bound on growth; unless you say otherwise, people assume you mean the worst case. Say "average case" explicitly when you mean it (a dictionary lookup is $O(1)$ on average, Q72A-003)

Likely follow-ups: What's the difference between time and space complexity? What's amortized complexity (Q72A-006)? Best case versus worst case for quicksort (Q72A-020)?

Red flag: confusing Big-O with measured runtime; can't name what $O(n^2)$ looks like in code.

Learn it in: Chapter 33, section 33.1 (and Chapter 48 for the same instinct at distributed scale).

Q72A-002 · Live demonstration: why the nested-loop duplicate check is dangerous

Level: Fresher · **Roles:** DE, AE, MLE, DS, DA

Remember it as: A nested loop over the same array is $O(n^2)$ hiding in plain sight. A set turns it into $O(n)$.

Answer in one line: Comparing every element with every later one is $O(n^2)$; remembering what you've seen in a set, whose membership check is $O(1)$ on average, does the same job in one $O(n)$ pass.

The two versions:

```
def has_dup_On2(arr):          #  $O(n^2)$ : compares every pair
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False

def has_dup_On(arr):           #  $O(n)$ : one pass, a set remembers what's been seen
    seen = set()
    for x in arr:
        if x in seen:
            return True
        seen.add(x)
    return False
```

- **has_dup_On2:** the outer loop picks position `i`; the inner loop starts at `i + 1`, so each pair is compared once, never an element with itself. It returns `True` at the first match.
- **has_dup_On:** one loop. `x in seen` asks the set, not a scan (Chapter 33, section 33.3 shows why that's instant); `seen.add(x)` records `x` for later.

Now some test data: 5,000 different whole numbers, so neither version can stop early (the worst case for both).

```
import random

random.seed(72)
data = random.sample(range(10_000_000), 5_000)
print(len(data), len(set(data)))
print(has_dup_On2(data), has_dup_On(data))
print(has_dup_On2(data + [data[0]]), has_dup_On(data + [data[0]]))
```

```
5000 5000
False False
True True
```

- **random.seed(72)** fixes the random generator's starting point, so you get the same 5,000 numbers every time you run it.

- `random.sample(range(10_000_000), 5_000)` draws 5,000 *different* numbers from 0 to 9,999,999. `len(set(data))` is 5,000 too, which proves there are no duplicates.
- The last line adds a copy of the first number at the end; both functions now find the duplicate. Always check that the fast version gives the *same answer* before you time it.

How many comparisons does the slow version make on the unique data? Each pair once:

```
n = len(data)
print(f"{n * (n - 1) // 2:,} comparisons, against n squared = {n * n:,}")
```

```
12,497,500 comparisons, against n squared = 25,000,000
```

That's $n(n-1)/2$, about 12.5 million: half of n^2 , and still $O(n^2)$, because Big-O drops the $\frac{1}{2}$. The table in Q72A-001 counts every ordered pair (25 million); the shape is the same.

Now time them, at 2,500 and at 5,000 items. Timings change on every run and every machine, so this cell's numbers will differ on yours:

```
import time

def seconds(func, arr):
    start = time.perf_counter()
    func(arr)
    return time.perf_counter() - start

for size in (2_500, 5_000):
    sample = data[:size]
    slow = seconds(has_dup_0n2, sample)
    fast = seconds(has_dup_0n, sample)
    print(f"n={size:,}: O(n^2) {slow:.4f} s    O(n) {fast:.6f} s    {slow / fast:,.0f}x")
```

```
n=2,500: O(n^2) 0.0719 s    O(n) 0.000183 s    393x
n=5,000: O(n^2) 0.2871 s    O(n) 0.000679 s    423x
```

- `seconds(func, arr)` runs any function once on `arr` and returns how long it took, using `time.perf_counter()`, Chapter 18's stopwatch. Passing a function as an argument (`has_dup_0n2`, with no brackets) hands over the function itself, not its result.
- `data[:size]` takes the first 2,500 or 5,000 numbers.
- Read the two rows against each other: doubling n multiplied the $O(n^2)$ time by four (0.0719 s to 0.2871 s), the measured shape of a quadratic, and a stronger answer than one speedup number. The $O(n)$ times stay under a millisecond, too small for a single run to time precisely.

Tier	What to say
Passes	"Use a set instead of nested loops"
Strong	+ explains <i>why</i> : <code>x in seen</code> is $O(1)$ on average, versus scanning the rest of the array for every element
Extra points	[+Validate] the timings above, on real data, with the answers checked equal first; doubling n quadruples the slow version. [+Business] checking for duplicate customer IDs or order IDs is everyday data cleaning, and on millions of rows the $O(n^2)$ version doesn't finish in any reasonable time

Likely follow-ups: What's the trade-off (a set uses more memory)? What if the input needs to stay in its original order?

Red flag: reaching for nested loops as a first instinct without considering a hash-based structure.

Learn it in: Chapter 33, sections 33.1 (the measured curves) and 33.2 (list versus set, timed).

Rapid-fire, 72A.2

Roles: DE, AE, MLE and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-003	Complexity of a dictionary (hash map) lookup?	$O(1)$ on average	[+Edge cases] worst case is $O(n)$ if almost every key collides; rare in practice with a good hash function	Fresher · 33.3
Q72A-004	Complexity of Python's built-in <code>sort()</code> ?	$O(n \log n)$	[+Edge cases] it's Timsort, which is $O(n)$ on data that's already sorted	Fresher · 33.7
Q72A-005	Time versus space trade-off, one example?	Memoization (see Q72A-018 below) spends memory on a cache to avoid repeated work	[+Business] it isn't free: caching everything can exhaust memory	Mid · 33.8, 33.9
Q72A-006	What's amortized complexity, in one example?	Appending to a Python list is $O(1)$ amortized: an occasional append copies the list into bigger storage ($O(n)$), but so rarely that the average per append stays constant	[+Edge cases] "amortized" is an average over many operations; any single call can still be slow	Mid · 33.1

72A.3 Arrays, strings, and the two-pointer / sliding-window patterns

Beyond the book: two pointers and sliding windows. Chapter 33, section 33.10 names both patterns; here is what they are. A **pointer** here is just a position in a list, held in a variable such as `l` or `r`. The **two-pointer** pattern keeps two positions and moves them towards each other (or both forwards) by a rule, so one pass does the job a pair of nested loops would. The **sliding-window** pattern keeps a running total for a fixed-width stretch of the list, the **window**, and moves it one place at a time: add the item that comes in, subtract the item that falls out, instead of adding the whole window up again. Q72A-008 and Q72A-009 trace each one step by step.

Q72A-007 · Find two numbers in an array that sum to a target (Two Sum)

Level: Fresher · **Roles:** DE, AE, MLE, DS

Remember it as: "What number would complete this?": check a hash map before you check the rest of the array.

Answer in one line: Loop once, and at each element check whether `target - element` has already been seen; a dictionary turns an $O(n^2)$ check of every pair into a single $O(n)$ pass.

```
def two_sum(nums, target):
    seen = {} # value -> the position where we saw it
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return None

print(two_sum([2, 7, 11, 15], 9))
print(two_sum([2, 7, 11, 15], 100))
```

```
[0, 1]
None
```

- **enumerate(nums)** (Chapter 17, section 17.6) gives each value with its position.
- **target - n in seen** asks "have I already met the number that completes this one?" It's a dictionary lookup, $O(1)$ on average; this is Chapter 33's dictionary index in miniature.
- **seen[n] = i** is stored *after* the check, so an element is never paired with itself.
- No pair adds up to 100, so the loop finishes and the function returns **None**.

Tier	What to say
Passes	Nested loop checking every pair ($O(n^2)$), correct, not the answer they want
Strong	The dictionary version above, $O(n)$ time, $O(n)$ space
Extra points	[+Clarify] ask whether the array is sorted: if so, two pointers from both ends work in $O(1)$ extra space. [+Edge cases] no valid pair (the None above), or several: which should be returned? [+Trade-offs] the dictionary version is faster but uses $O(n)$ extra memory; the two-pointer version uses $O(1)$ extra memory but needs sorted data, and sorting costs $O(n \log n)$

Likely follow-ups: What if you need all pairs, not just one? What if the array is sorted? Three Sum instead of two?

Red flag: jumping straight to nested loops with no mention of the hash-map alternative.

Learn it in: Chapter 33, sections 33.2 (the dictionary index) and 33.10 (the patterns).

Q72A-008 · Check whether a string is a palindrome

Level: Fresher · **Roles:** DE, AE, MLE, DS · **Beyond the book** (the two-pointer pattern)

Remember it as: *Two pointers walking toward each other from both ends: stop the moment they disagree.*

Answer in one line: Compare the first and last characters, then the second and second-to-last, and so on inwards; any mismatch means no, and meeting in the middle means yes.

```
def is_palindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return False
        l += 1
        r -= 1
    return True

print(is_palindrome("racecar"), is_palindrome("hello"), is_palindrome(""))
```

True False True

- `l, r = 0, len(s) - 1` puts one pointer on the first character and one on the last.
- `while l < r` keeps going until the pointers meet or cross; each step moves `l` right and `r` left.
- The empty string never enters the loop, so it counts as a palindrome.

Here is the walk for "racecar", printed by the same loop:

```
s = "racecar"
l, r = 0, len(s) - 1
while l < r:
    print(f"l={l} r={r} {s[l]} vs {s[r]} {'same' if s[l] == s[r] else 'different'}")
    l += 1
    r -= 1
print(f"l={l} r={r} pointers met: palindrome")
```

```
l=0 r=6 r vs r same
l=1 r=5 a vs a same
l=2 r=4 c vs c same
l=3 r=3 pointers met: palindrome
```

- `{'same' if s[l] == s[r] else 'different'}` is a one-line if inside the f-string; it prints *same* when the two characters match and *different* when they don't.
- The last `print` runs once the loop ends, showing where the pointers met.

Three comparisons for seven letters: the middle `e` never needs checking.

Tier	What to say
Passes	Reverse the string and compare (<code>s == s[::-1]</code>): correct, $O(n)$ extra space for the reversed copy
Strong	The two-pointer version above: $O(1)$ extra space, same $O(n)$ time, no copy needed
Extra points	[+Clarify] capitals, spaces and punctuation ("A man, a plan, a canal: Panama") need normalizing first; ask whether that's in scope. [+Trade-offs] the $O(1)$ -space version matters on huge strings; on interview-sized ones the slicing version is fine and clearer, so say you know both

Likely follow-ups: Ignore case and punctuation? Longest palindromic substring instead?

Red flag: not knowing the two-pointer alternative to slicing and reversing.

Learn it in: Beyond the book (the primer at the start of this section); slicing and `while`: Chapter 17, sections 17.4 and 17.6.

Q72A-009 · Maximum sum of any contiguous subarray of size k

Level: Mid · **Roles:** DE, AE, MLE, DS · **Beyond the book** (the sliding-window pattern)

Remember it as: Don't recompute the whole window every time: slide it. Subtract what falls out, add what comes in.

Answer in one line: Summing every window from scratch is $O(n \cdot k)$; a sliding window that updates the running sum (minus the element leaving, plus the element entering) is $O(n)$.

```
def max_sum_subarray(arr, k):
    window = sum(arr[:k])
    best = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i - k]
        best = max(best, window)
    return best

print(max_sum_subarray([2, 1, 5, 1, 3, 2], 3))
```

9

- `window = sum(arr[:k])` adds up the first window, positions 0 to k-1, once.
- `for i in range(k, len(arr)):` `arr[i]` is the item coming into the window and `arr[i - k]` the one falling out, so one addition and one subtraction move the window along.
- `best = max(best, window)` keeps the largest total seen so far.

The same loop, printing each step:

```
arr, k = [2, 1, 5, 1, 3, 2], 3
window = sum(arr[:k])
best = window
print(f"start: window {arr[:k]} sum {window}, best {best}")
for i in range(k, len(arr)):
    window += arr[i] - arr[i - k]
    best = max(best, window)
    print(f"i={i}: +{arr[i]} -{arr[i - k]} sum {window}, best {best}")
```

```
start: window [2, 1, 5] sum 8, best 8
i=3: +1 -2 sum 7, best 8
i=4: +3 -1 sum 9, best 9
i=5: +2 -5 sum 6, best 9
```

The best window is 5, 1, 3, found at `i=4`. One trap the function doesn't guard against:

```
print(max_sum_subarray([1, 2], 3))
```

3

With `k` bigger than the list, `arr[:k]` quietly returns the whole short list, the loop never runs, and the function answers 3 for a window of size 3 that doesn't exist.

Tier	What to say
Passes	Recompute the sum of each window of size k from scratch: correct, $O(n \cdot k)$
Strong	The sliding-window version above, $O(n)$, with the trace to show it
Extra points	[+Edge cases] guard with <code>if k <= 0 or k > len(arr): return None</code> , otherwise a short list returns a sum of fewer than k items, as the last cell shows. [+Signpost] the same subtract-outgoing, add-incoming idea handles a whole family of window problems, not just sums. [+Business] this is a trailing-window calculation, the idea behind a moving average, written as raw list logic instead of pandas' <code>.rolling()</code>

Likely follow-ups: What if k varies (a variable-size window)? Maximum item rather than sum? The shortest window that contains every character of a given set?

Red flag: recomputing the full window sum every step with no mention of the sliding update.

Learn it in: Beyond the book (the primer at the start of this section); Chapter 40, section 40.6 for the same trailing window with `.rolling()`.

Rapid-fire, 72A.3

Roles: DE, AE, MLE and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-010	Reverse an array in place?	Two pointers from each end: swap, move both inwards; $O(n)$ time, $O(1)$ space	[+Edge cases] odd length: the middle element never needs swapping, and the loop handles it	Fresher · primer above
Q72A-011	Find the missing number in a range 1 to n?	The sum of $1..n$, $n(n+1)/2$, minus the actual sum of the array; $O(n)$ time, $O(1)$ space	[+Trade-offs] a set-based approach also works but uses $O(n)$ extra space; the sum trick doesn't	Fresher · 33.2
Q72A-012	Merge two sorted arrays into one sorted array?	Two pointers, one per array; always take the smaller current element; $O(n + m)$	[+Business] this merge step is the core of merge sort (see Q72A-020 below) and of merging two sorted feeds	Mid · 33.7, primer above
Q72A-013	Group anagrams from a list of strings?	A dictionary keyed by each string's sorted letters; strings with the same key are anagrams	[+Edge cases] sorting each string's letters costs $O(k \log k)$ per string; a tuple of 26 letter counts as the key avoids that sort	Mid · 33.2

72A.4 Hash maps and sets

Q72A-014 · When would you reach for a set instead of a list, and why does it matter?

Level: Fresher · **Roles:** DE, AE, MLE, DS, DA

Remember it as: "Is X in here?": a list checks every element one by one; a set asks the hash table directly.

Answer in one line: A set's membership check (`x in my_set`) is $O(1)$ on average; a list's is $O(n)$, because it scans element by element: the mechanism behind Q72A-002's speedup.

Sets have one rule a list doesn't: everything in them must be **hashable** (Chapter 33, section 33.3).

```
print({(1, 2), (1, 2), (3, 4)})
try:
    {(1, [2])}
except TypeError as exc:
    print("TypeError:", exc)
```

```
{(1, 2), (3, 4)}
TypeError: unhashable type: 'list'
```

- The first set keeps one copy of (1, 2): sets drop duplicates.
- **try: ... except TypeError as exc:** (Chapter 17, section 17.10) runs the risky line, and if it raises a `TypeError`, prints the error's message (`exc`) instead of stopping.
- A tuple of numbers is hashable, so it can go in a set. A tuple *containing a list* isn't, because the list inside could change, and Python refuses it.

Tier	What to say
Passes	"Sets are faster for checking if something exists"
Strong	+ explains the $O(1)$ versus $O(n)$ mechanism, and that sets don't keep order or allow duplicates
Extra points	[+Business] deduplicating a customer or order list, and checking "has this ID been processed already", are the two most common reasons to reach for a set in data work. [+Edge cases] set elements must be hashable: a list or dict can't go in a set, and a tuple can only if everything inside it is hashable too (the <code>TypeError</code> above)

Likely follow-ups: Set versus dictionary: when do you need one over the other? What makes something "hashable" (Q72A-015)?

Red flag: not knowing why the membership check is faster, just that "sets are faster".

Learn it in: Chapter 33, sections 33.2 (list versus set, timed) and 33.3 (hashing).

Rapid-fire, 72A.4

Roles: DE, AE, MLE and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-015	What makes an object hashable in Python?	It has a working <code>__hash__</code> , and its hash never changes while it's in a set or used as a key. Built-in immutable values (str, int, a tuple of hashable values) qualify; list, dict and set don't. Instances of a plain class hash by identity (Q72A-026 uses that)	[+Edge cases] two equal objects must produce the same hash, or a set or dict silently misbehaves	Mid · 33.3
Q72A-016	Find the first non-repeating character in a string?	Count every character in one pass (<code>Counter</code>), then a second pass returns the first with count 1	[+Trade-offs] two $O(n)$ passes; a single-pass version is possible but usually less readable	Fresher · 33.2

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-017	Count the frequency of every word in a large text?	A dictionary, or <code>collections.Counter</code> , in one pass: $O(n)$	[+Business] it's the same counting that builds Chapter 41's bag of words (section 41.3)	Fresher · 33.2

72A.5 Recursion and memoization

Q72A-018 · Fibonacci, naive recursion versus memoized, and why the difference is so dramatic

Level: Mid · **Roles:** DE, MLE, DS

Remember it as: *Naive recursive Fibonacci recomputes the same small answers hundreds of thousands of times. Memoization is just "remember what you already solved."*

Answer in one line: The naive version calls itself twice per call and solves the same subproblems over and over, $O(2^n)$; caching each answer the first time it's computed makes it $O(n)$.

The naive version, with one extra line that counts how often each `n` is computed:

```
from collections import Counter

calls = Counter()

def fib_naive(n):
    calls[n] += 1
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

print(fib_naive(28))
print(f"{sum(calls.values()):,} calls; fib(2) was computed {calls[2]:,} times")
```

```
317811
1,028,457 calls; fib(2) was computed 196,418 times
```

- `calls[n] += 1` counts one more call for this `n`; a `Counter` starts every key at 0 (Chapter 33, section 33.2).
- `if n <= 1: return n` is the base case; the last line is the recursive case, which calls the function twice.
- Over a million calls to get one number, and `fib(2)` alone was worked out nearly 200,000 times. That duplicated work is the whole problem.

The memoized version, with the cache Python has built in:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

print(fib_memo(28))
print(fib_memo.cache_info())
```

```
317811
CacheInfo(hits=26, misses=29, maxsize=None, currsize=29)
```

- `@lru_cache(maxsize=None)` (Chapter 33, section 33.8) wraps the function so each call first looks up its argument in a dictionary of past results, and only runs the body for an `n` it hasn't seen. `maxsize=None` means it never forgets.
- `cache_info()` reports the work: 29 **misses** (each `n` from 0 to 28 computed once) and 26 **hits** (answers reused). Fifty-five calls instead of 1,028,457.

Now time both. Timings change on every run and every machine:

```
import time

start = time.perf_counter()
fib_naive(28)
naive_s = time.perf_counter() - start

fib_memo.cache_clear()          # start with an empty cache, or the timing is a pure lookup
start = time.perf_counter()
fib_memo(28)
memo_s = time.perf_counter() - start

print(f"naive: {naive_s:.4f} s    memoized: {memo_s:.6f} s    {naive_s / memo_s:,.0f}x")
```

```
naive: 0.1383 s    memoized: 0.000018 s    7,485x
```

- `start = time.perf_counter()` reads the stopwatch before each call, and `time.perf_counter() - start` after it gives the seconds taken, stored as `naive_s` and `memo_s`.
- `cache_clear()` empties the cache first; without it, the second call would find `fib_memo(28)` already stored and time a single lookup. The naive version still has its counting line, which slows it a little, but the ratio would be in the thousands without it too, and it widens fast as `n` grows, since one version is $O(2^n)$ and the other $O(n)$.

The memoized version still recurses `n` calls deep, and Python allows about 1,000:

```
fib_memo.cache_clear()
try:
    fib_memo(1500)
except RecursionError:
    print("RecursionError: fib_memo(1500) needs 1,500 calls at once")
```

RecursionError: fib_memo(1500) needs 1,500 calls at once

- **try / except RecursionError** (Chapter 17, section 17.10) catches the error and prints a line of our own. Python's own message starts "maximum recursion depth exceeded", as in Chapter 33, section 33.5; its ending varies with where the limit is hit, so the cell prints its own words instead.

That's the practical reason for the **bottom-up** version, which builds the answer from `fib(0)` upwards in a loop and keeps only the last two numbers:

```
def fib_bottom_up(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(fib_bottom_up(28))
print(len(str(fib_bottom_up(1500))), "digits, no recursion")
```

```
317811
314 digits, no recursion
```

- **a, b = 0, 1** holds `fib(0)` and `fib(1)`. **a, b = b, a + b** moves both one step along at once: the right-hand side is worked out in full before either name changes.
- **for _ in range(n):** `_` is the usual name for a loop variable you don't use.
- **len(str(fib_bottom_up(1500)))** turns the huge number into text and counts its characters, which is its number of digits.
- $O(n)$ time, $O(1)$ space, and no depth limit: `fib(1500)` has 314 digits and comes back instantly.

Tier	What to say
Passes	The naive recursive version: correct, exponential time
Strong	+ explains <i>why</i> it's slow (the same subproblems recomputed in different branches: 1,028,457 calls for $n = 28$) and fixes it with <code>@lru_cache</code>
Extra points	[+Validate] the call counts and <code>cache_info()</code> above, not a claimed speedup. [+Edge cases] the memoized recursion still goes n calls deep, so <code>fib_memo(1500)</code> raises <code>RecursionError</code> ; the bottom-up loop doesn't, and uses $O(1)$ space instead of $O(n)$. [+Edge cases] a hand-rolled <code>def fib(n, cache={})</code> works, but it's Chapter 72's mutable-default trap (Q72-001): the one dictionary survives between calls. Write <code>cache=None</code> and create the dictionary inside, or say you're keeping it on purpose. [+Business] any recursive calculation with overlapping subproblems is a memoization candidate, not just textbook Fibonacci

Likely follow-ups: What does the bottom-up version look like (above)? How do you spot a memoization opportunity? What's the space complexity of each version?

Red flag: saying "recursion is slow"; it isn't, inherently. Redundant recomputation is the problem.

Learn it in: Chapter 33, sections 33.8 (memoization, `lru_cache`, dynamic programming) and 33.5 (recursion and the 1,000-frame limit).

Rapid-fire, 72A.5

Roles: DE, MLE and DS.

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-019	What's a base case, and why does every recursive function need one?	The condition that returns an answer without calling the function again, so the recursion stops	[+Edge cases] with a missing or wrong base case the recursion never stops; Python halts it with <code>RecursionError</code> (the language-neutral name is a stack overflow), the recursive twin of Q70-041's never-ending macro loop	Fresher · 33.5

72A.6 Sorting and searching

Q72A-020 · Implement merge sort and explain why it's $O(n \log n)$

Level: Mid · **Roles:** DE, MLE, DS · **Beyond the book**

Remember it as: *Split in half until you can't, then merge back up while comparing. The "log n" is how many times you can halve; the "n" is the work at each level.*

Answer in one line: Merge sort splits the list in half repeatedly down to single items, then merges sorted halves back together; there are about $\log_2 n$ levels of splitting, and each level's merging touches all n items, so the total is $n \log n$.

Beyond the book: merge sort by hand. Chapter 33 uses `sorted()` and doesn't build a sort. Here is merge sort on four numbers. Split until every piece has one item (a one-item list is already sorted), then merge pairs of sorted pieces, each time taking the smaller front item:

```
[38, 27, 43, 3]
split:  [38, 27]      [43, 3]
split:  [38] [27]     [43] [3]
merge:  [27, 38]      [3, 43]      level 1: 4 items handled
merge:  [3, 27, 38, 43] level 2: 4 items handled
```

Two levels, because 4 halves twice to reach 1 ($\log_2 4 = 2$), and each level handles all 4 items: $2 \times 4 = 8$ steps of merging. For n items: $\log_2 n$ levels $\times n$ items each.

First the merge step on its own, the two-pointer walk of Q72A-012:

```
def merge(left, right):
    merged = []
    i = 0                # next unused item in left
    j = 0                # next unused item in right
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])    # one side is used up; copy what's left of the other
    merged.extend(right[j:])
    return merged

print(merge([27, 38], [3, 43]))
```

```
[3, 27, 38, 43]
```

- **i and j** are the two pointers, one per list. Each pass of the loop takes the smaller front item and moves that list's pointer on.
- **left[i] <= right[j]** uses **<=**, not **<**, so on a tie the item from the left list goes first. That keeps equal items in their original order, which makes merge sort **stable** (Q72A-024).
- **.extend(left[i:])** adds whatever remains once one list runs out; one of the two slices is always empty.
- **What if you change <= to <?** The output is the same for these numbers, but on a tie the right-hand item would go first, and the sort would no longer be stable.

Then the sort itself calls itself on each half and merges the results:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

print(merge_sort([38, 27, 43, 3]))
print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

```
[3, 27, 38, 43]
[3, 9, 10, 27, 38, 43, 82]
```

- **if len(arr) <= 1: return arr** is the base case: nothing to split.
- **mid = len(arr) // 2** finds the halfway point (**//** is whole-number division); the two recursive calls sort each half, and **merge** combines them.

Tier	What to say
Passes	Names merge sort as $O(n \log n)$ with no explanation of why
Strong	The implementation above, explaining the split-then-merge structure and the base case
Extra points	[+Signpost] "log n" is the number of times the list can be halved before reaching single items; "n" is the merging work at each of those levels; together $n \log n$. [+Trade-offs] merge sort needs $O(n)$ extra space and is stable. Quicksort is typically faster in practice, uses $O(\log n)$ stack space on average, is unstable, and degrades to $O(n^2)$ when pivots are chosen badly (for example, always the first element on already-sorted data); random or median-of-three pivots avoid that

Likely follow-ups: Quicksort instead: implement it? Why would you pick one over the other? Is Python's built-in sort stable?

Red flag: can't explain where the "log n" comes from.

Learn it in: Beyond the book (the primer above); `sorted()`, Timsort and stability: Chapter 33, section 33.7.

Q72A-021 · Implement binary search, and state its one hard requirement

Level: Fresher · **Roles:** DE, AE, MLE, DS

Remember it as: *Binary search only works on sorted data. That's not a footnote, it's the whole trick.*

Answer in one line: Look at the middle item of a sorted list; if the target is bigger, throw away the left half, if smaller the right half, and repeat: $O(\log n)$, about 20 looks for a million items.

The cell's last line searches an *unsorted* list. Before you run it, predict what it prints: the list does hold a 9.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

sorted_data = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(sorted_data, 9), binary_search(sorted_data, 4))
print(binary_search([9, 1, 13, 3, 11, 5, 7], 9))
```

```
4 -1
-1
```

- `binary_search(arr, target)` returns the position of `target` in `arr`, or `-1` if it isn't there.
- `while lo <= hi` keeps going while there is anything left to search.
- `lo` and `hi` mark the part of the list still worth searching; `mid = (lo + hi) // 2` is the middle position of that part.

- **if / elif / else:** found it, return the position; the middle value is too small, so the target can only be to its right (`lo = mid + 1`); otherwise it can only be to the left (`hi = mid - 1`).
- 9 is at position 4. 4 isn't in the list, so the loop ends with `lo > hi` and the function returns -1.
- **The last line is the trap:** the same numbers, unsorted. 9 is in the list (at position 0), but the search throws away the half it's in and answers -1, with no error. That's why the requirement matters.

Tier	What to say
Passes	The implementation, without stating the sorted-input requirement clearly
Strong	+ states explicitly: the list must be sorted, or binary search silently gives wrong answers rather than raising an error (the last line above)
Extra points	[+Edge cases] <code>(lo + hi) // 2</code> can overflow in languages with fixed-size integers; Python's integers are unbounded, but say you know the general issue. [+Business] a database index lookup is the same halving idea on a B-tree, which is why an index turns a slow table scan into a fast lookup. [+Simple first] in real Python code, <code>bisect</code> does this for you

Likely follow-ups: Find the first occurrence of a duplicated value? Search in a rotated sorted list?

Red flag: forgetting the sorted-input requirement.

Learn it in: Chapter 33, section 33.7 (binary search, `bisect`); Chapter 28, section 28.4 (B-tree indexes).

Rapid-fire, 72A.6

Roles: DE, AE, MLE and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72A-022	Find the kth largest element in an array?	<code>heapq.nlargest(k, nums)[-1]</code> , or keep a min-heap of size k for very large data	[+Trade-offs] sorting everything is $O(n \log n)$ and simplest to write; the heap is $O(n \log k)$, faster when k is small next to n	Mid · 33.7
Q72A-023	Bubble sort versus merge sort: why does one matter in interviews and not in practice?	Bubble sort compares neighbours and swaps them if they're in the wrong order, repeating up to n passes: $O(n^2)$. It's asked to see whether you can <i>reason</i> about why it's slow, not because anyone uses it	[+Business] production code uses the language's built-in sort (Timsort in Python), not a hand-rolled one; interviews test the reasoning, not the reinvention	Fresher · 33.7
Q72A-024	What's a stable sort, and when does stability matter?	Equal elements keep their original relative order after sorting	[+Edge cases] sorting orders by date and wanting ties left in their original row order needs a stable sort; an unstable one can silently scramble the ties	Fresher · 33.7

72A.7 Linked lists

Beyond the book: what a linked list is. A Python list keeps its items side by side in memory. A **linked list** keeps each item in its own small object, a **node**, which holds a value and a pointer to the next node, in an attribute called `next`. The last node's `next` is `None`. The list itself is just a reference to the first node, the **head**:

```
head
|
[1 | next] --> [2 | next] --> [3 | next] --> None
```

To reach the third item you start at the head and follow `next` twice: finding an item is $O(n)$, but adding or removing one where you already stand is $O(1)$, since only a pointer changes. Python programs rarely need one (a list or deque does the job), but interviews like them because every answer is about pointers. A node is a small class, written as in Chapter 29, section 29.5: `__init__` runs when you write `Node(5)`, `self` is the new node, and `self.val = val` stores a value on it.

```
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.next = nxt

def build(values):
    # [1, 2, 3] -> 1 -> 2 -> 3, returns the head
    head = None
    for v in reversed(values):
        head = Node(v, head)
    return head

def to_list(head):
    # walk the nodes and collect their values
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

three = build([1, 2, 3])
print(three.val, three.next.val, three.next.next.val, three.next.next.next)
print(to_list(three))
```

```
1 2 3 None
[1, 2, 3]
```

- `nxt=None` is the default for the pointer; it's named `nxt` because `next` is already a Python built-in.
- `build` makes the nodes back to front: each new node points at the one made before it, so the last one made is the head.
- `to_list` is the walk every linked-list answer uses: `while head:` keeps going until it reaches `None`, and `head = head.next` steps forward.
- The first printed line follows the pointers by hand: 1, then 2, then 3, then `None`.

Q72A-025 · Reverse a singly linked list

Level: Mid · **Roles:** DE, MLE · **Beyond the book**

Remember it as: *Three pointers walking together: remember where you came from, flip the arrow, step forward.*

Answer in one line: Walk the list once, turning each node's `next` to point at the node before it, while holding on to the rest of the list so it isn't lost: $O(n)$ time, $O(1)$ extra space.

```
def reverse_list(head):
    prev = None
    while head:
        nxt = head.next    # remember what's next before we overwrite it
        head.next = prev   # flip this node's arrow backwards
        prev = head        # move prev forward
        head = nxt         # move head forward
    return prev

print(to_list(reverse_list(build([1, 2, 3, 4, 5]))))
print(to_list(reverse_list(build([]))), to_list(reverse_list(build([7])))
```

```
[5, 4, 3, 2, 1]
[] [7]
```

The trace on three nodes, printed by the same loop, one row per pass (values shown instead of nodes):

```
def val(node):
    return None if node is None else node.val

head, prev, step = build([1, 2, 3]), None, 0
while head:
    step += 1
    nxt = head.next
    head.next = prev
    print(f"step {step}: node {head.val} now points to {val(prev)}; prev={head.val}, head=
        {val(nxt)}")
    prev, head = head, nxt
print("result:", to_list(prev))
```

```
step 1: node 1 now points to None; prev=1, head=2
step 2: node 2 now points to 1; prev=2, head=3
step 3: node 3 now points to 2; prev=3, head=None
result: [3, 2, 1]
```

After each step, one more arrow points backwards, and `nxt` has kept the rest of the list safe.

Tier	What to say
Passes	Describes the goal correctly, struggles to produce working pointer logic
Strong	The three-pointer version above, walked through out loud step by step
Extra points	[+Validate] trace it by hand on a three-node list before typing it, the table above. [+Edge cases] the empty list and a one-node list both work without special cases (second output line); say that you checked

Likely follow-ups: Reverse only part of the list (between positions m and n)? A recursive version instead of the loop?

Red flag: losing track of `head.next` before reassigning it, ending up with a list that loses its tail.

Learn it in: Beyond the book (the primer above); classes: Chapter 29, section 29.5.

Q72A-026 · Detect a cycle in a linked list

Level: Mid · **Roles:** DE, MLE · **Beyond the book**

Remember it as: *Fast and slow pointers: if there's a loop, the fast one eventually laps the slow one. If there isn't, fast just reaches the end first.*

Answer in one line: Move one pointer one node at a time and another two at a time; if they ever land on the same node the list loops, and if the fast one reaches `None` it doesn't (Floyd's cycle detection): $O(n)$ time, $O(1)$ extra space.

The simple version first, with a set of nodes already visited:

```
def has_cycle_set(head):
    seen = set()
    while head:
        if head in seen:
            return True
        seen.add(head)
        head = head.next
    return False
```

It works because a `Node` is hashable: instances of a plain class hash by identity, so the set asks "have I stood on *this very node* before?" (Q72A-015). Now the two-pointer version:

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

straight = build([1, 2, 3])
looped = build([1, 2, 3])
looped.next.next.next = looped          # node 3 now points back to node 1

print(has_cycle(straight), has_cycle(looped))
print(has_cycle_set(straight), has_cycle_set(looped))
```

```
False True
False True
```

- **`slow = fast = head`** starts both pointers at the head.
- **`while fast and fast.next`**: the fast pointer jumps two nodes, so both must exist; on a straight list it reaches the end and the loop stops.
- **`slow is fast`** asks "the very same node?", the identity test from Chapter 72's Q72-002, not "equal values".

- `looped.next.next.next = looped` makes the loop: the third node's `next` goes back to the first instead of to `None`. Both functions agree on both lists.

Tier	What to say
Passes	"Keep a set of visited nodes and check for repeats": correct, $O(n)$ extra space
Strong	The fast/slow pointer version (Floyd's cycle detection): $O(1)$ extra space, same $O(n)$ time
Extra points	[+Trade-offs] the set version is easier to write and reason about; interviewers usually want the two-pointer version because it's $O(1)$ space. [+Business] cycle detection matters in real systems: circular references in data, or a loop in a pipeline's task graph (Chapter 33, section 33.6 checks one)

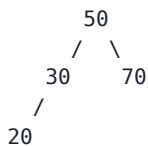
Likely follow-ups: Find where the cycle begins, not just whether one exists? A cycle in a directed graph instead of a linked list?

Red flag: the set version presented as the only option, with no mention of the $O(1)$ -space alternative.

Learn it in: Beyond the book (the primer above); cycles in graphs: Chapter 33, section 33.6.

72A.8 Trees and graphs

Beyond the book: binary search trees. Chapter 33, section 33.6 walks trees such as the org chart, where a node can have any number of children. A **binary tree** allows at most two, a **left** and a **right** child. A **binary search tree (BST)** adds one rule, the **invariant**: everything in a node's left subtree is smaller than the node, and everything in its right subtree is larger (or equal). Insert 50, 30, 70, 20 by hand: 50 becomes the root; 30 is smaller, so it goes left; 70 is larger, so it goes right; 20 is smaller than 50 (go left) and smaller than 30 (go left again):



Finding a value repeats the same choice at each node, so on a well-shaped tree a search takes about $\log_2 n$ steps, like binary search. An **in-order traversal** visits the left subtree, then the node, then the right subtree; on a BST that reads the values in sorted order (20, 30, 50, 70 here). The catch: insert values that are already sorted (20, 30, 50, 70) and each one goes right of the last, so the "tree" is a chain four deep and a search is $O(n)$. Q72A-027 measures that.

Q72A-027 · Build a binary search tree and do an in-order traversal

Level: Mid · **Roles:** DE, MLE, DS · **Beyond the book**

Remember it as: In-order traversal of a BST always visits nodes in sorted order: that's the whole point of a BST.

Answer in one line: Insert each value by walking down from the root (left if smaller, right if not) and attaching it where the walk runs out; an in-order traversal (left, node, right) then returns the values sorted.

```

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root

def inorder(root, out):
    if root:
        inorder(root.left, out)
        out.append(root.val)
        inorder(root.right, out)
    return out

root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, v)
print(inorder(root, []))

```

```
[20, 30, 40, 50, 60, 70, 80]
```

- **insert** is recursive (Chapter 33, section 33.5). The base case, an empty spot (`root is None`), makes the new node. Otherwise it goes left or right, and `root.left = insert(root.left, val)` reattaches whatever the call below returns, so the new node is hooked in at the right place.
- **inorder** does exactly "left subtree, this node, right subtree", adding values to `out` as it goes. The output is sorted, which is itself the check that the tree was built correctly.

Now the shape problem, measured. The **height** is the number of nodes on the longest path from the root down:

```

def height(node):
    if node is None:
        return 0
    return 1 + max(height(node.left), height(node.right))

chain = None
for v in [20, 30, 40, 50, 60, 70, 80]:          # the same values, already sorted
    chain = insert(chain, v)

print("mixed order, height:", height(root))
print("sorted order, height:", height(chain), inorder(chain, []))

```

```

mixed order, height: 3
sorted order, height: 7 [20, 30, 40, 50, 60, 70, 80]

```

The same seven values: inserted in a mixed order, the tree is three levels deep; inserted already sorted, it's a chain seven deep, a linked list in disguise, and every search walks it from end to end.

Tier	What to say
Passes	Can build the tree, struggles to explain what in-order traversal guarantees
Strong	+ states the invariant (left smaller, right larger), and that in-order traversal (left, node, right) therefore produces sorted output
Extra points	[+Validate] the sorted output is itself the check that the tree is right. [+Edge cases] inserting already-sorted data makes the tree a chain (height 7 instead of 3 above), $O(n)$ instead of $O(\log n)$. That's why balanced trees exist: in-memory libraries use red-black or AVL trees, and databases use B-trees (Chapter 28, section 28.4), which stay shallow by keeping many keys in each node

Likely follow-ups: Pre-order and post-order traversal, and what each is used for? How would you check whether a tree is balanced? Delete a node from a BST?

Red flag: not connecting in-order traversal to "produces sorted output" as the key insight.

Learn it in: Beyond the book (the primer above); recursion: Chapter 33, section 33.5; classes: Chapter 29, section 29.5.

Q72A-028 · Breadth-first search (BFS) versus depth-first search (DFS) on a graph

Level: Mid · **Roles:** DE, AE, MLE, DS

Remember it as: BFS spreads out level by level (a queue). DFS commits to one path and backtracks (a stack, or recursion).

Answer in one line: Both visit every node reachable from a start; BFS takes nodes from a first-in-first-out queue, so it goes level by level and finds shortest paths in an unweighted graph, while DFS takes them from a stack (or recurses), so it follows one route to the end before backing up.

```
from collections import deque

graph = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": ["E"], "E": []}

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append(nb)
    return order

def dfs(graph, node, visited=None):
    if visited is None:
        visited = []
    visited.append(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited)
    return visited

print("BFS:", bfs(graph, "A"))
print("DFS:", dfs(graph, "A"))
```

```
BFS: ['A', 'B', 'C', 'D', 'E']
DFS: ['A', 'B', 'D', 'E', 'C']
```

- **graph** is a dictionary of lists, as in Chapter 33, section 33.6: each node maps to its neighbours. D can be reached two ways (through B and through C).
- **visited = {start}** is a **set**, so each `nb not in visited` check is $O(1)$ and BFS is $O(V + E)$ for V nodes and E edges. A list here would make every check a scan (Q72A-014) and BFS $O(V^2)$.
- **queue.popleft()** takes from the front (Chapter 33, section 33.4), which is what makes the walk level by level: A, then B and C, then D, then E.
- **dfs** goes as deep as it can first: A, B, D, E, and only then back to C. It keeps **visited** as a list so the list is the visiting order, and it defaults to `None` rather than `[]` to avoid Chapter 72's mutable-default trap (Q72-001). On a big graph you'd keep a set for the checks as well.

Tier	What to say
Passes	Knows BFS and DFS are "two ways to traverse a graph"
Strong	+ BFS uses a queue (FIFO), explores level by level, and finds the <i>shortest</i> path in an unweighted graph; DFS uses a stack or recursion (LIFO) and goes as deep as possible before backtracking; visited is a set, so each check is $O(1)$ and the walk is $O(V + E)$
Extra points	[+Business] BFS's shortest-path property makes it the choice for "fewest hops" problems (shortest chain of connections, fewest steps through a process); DFS is the natural choice for "does <i>any</i> path exist" and for checking a pipeline's graph for cycles. [+Edge cases] both need a visited set on any graph with cycles, or they loop forever: Q70-041's never-ending loop, applied to graph traversal

Likely follow-ups: DFS without recursion (a list used as a stack)? Return the shortest path itself, not just the order? Detect a cycle in a directed graph?

Red flag: can't say which one (BFS) guarantees the shortest path in an unweighted graph.

Learn it in: Chapter 33, sections 33.6 (depth-first, breadth-first, cycles) and 33.4 (deque).

Q72A-029 · Level-order traversal of a tree, and why it's really BFS in disguise

Level: Mid · **Roles:** DE, MLE, DS

Remember it as: A tree is just a graph with no cycles and one path down from the root. BFS on it, level by level, is level-order traversal.

Answer in one line: Run BFS from the root with a queue, but before each round note how many nodes the queue holds: exactly those nodes form the current level.

```

from collections import deque

def level_order(root):
    out = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            if node:
                level.append(node.val)
                queue.append(node.left)
                queue.append(node.right)
        if level:
            out.append(level)
    return out

print(level_order(root))      # the BST from Q72A-027
print(level_order(None))

```

```

[[50], [30, 70], [20, 40, 60, 80]]
[]

```

- **for _ in range(len(queue))** is the key line: `len(queue)` is read once, before the loop starts, so the loop takes exactly the nodes of this level, even though it keeps adding their children to the back of the queue.
- **if node:** skips the `None` children of leaf nodes; **if level:** skips the empty round those `Nones` produce at the bottom.
- An empty tree (`None`) gives an empty list, with no special case.

Tier	What to say
Passes	Produces a flat list of values with no level grouping
Strong	The queue-based version above, grouping values level by level, recognized as BFS applied to a tree
Extra points	[+Signpost] explain the <code>for _ in range(len(queue))</code> line explicitly: it's what separates "this level" from "everything else in the queue", and it's easy to get subtly wrong. [+Edge cases] the empty tree, checked above

Likely follow-ups: Zigzag level order (alternating left-to-right and right-to-left)? The maximum width of any level?

Red flag: conflating this with a plain BFS that doesn't track level boundaries.

Learn it in: Chapter 33, section 33.6 (breadth-first, level by level: "who is two levels below the MD?"); the BST primer above.

72A.9 A live-coding walk-through: parentheses validation

Q72A-030 · Given a string of brackets, determine if every bracket is properly closed and nested

Level: Fresher · **Roles:** DE, AE, MLE, DS

Remember it as: A stack is "last opened, first closed": exactly how nested brackets have to resolve.

What they're really testing: whether you recognize the stack pattern immediately from "nested" and "matching", and handle the edge cases without being prompted.

```
def is_valid_parens(s):
    stack = []
    pairs = {")": "(", "]" : "[", "}": "{"}
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

for test in ["({[]})", "({[]})", "", ")))", ")((", "(":
    print(repr(test), is_valid_parens(test))
```

```
'({[]})' True
'({[]})' False
'' True
')))' False
')(' False
'(' False
```

- **stack** is a plain list used as a stack (Chapter 33, section 33.4): `.append` pushes onto the top and `.pop()` takes the top off.
- **pairs** maps each closing bracket to the opening bracket it needs.
- **not stack or stack.pop() != pairs[ch]**: if the stack is empty, or stops there (Chapter 17, section 17.5), so `.pop()` is never called on an empty list.
- **repr(test)** prints each string with its quotes, so the empty string shows as `''`.

Walking through it out loud, the way you'd say it live:

"I'll use a stack. Every time I see an opening bracket, I push it. Every time I see a closing bracket, I check two things: is the stack empty (a closing bracket with nothing open is automatically invalid), and does the top of the stack match this closing bracket's opener. If either check fails, I return false immediately. At the very end, if anything's still on the stack, there's an opener that was never closed, so I return `not stack`."

Extra-points moves demonstrated:

- **[+Edge cases]** the empty string is valid (`not stack` on an empty stack is `True`), and `"")"` fails at its first character, on the empty-stack check inside the loop.
- **[+Edge cases]** `"(("` gets through the loop but leaves two openers on the stack, so the final `not stack` catches it.
- **[+Validate]** all six cases above were run, not just claimed to work.
- **[+Business]** matching opens to closes is the same logic that checks JSON or HTML tags are balanced, not just an interview toy problem.

Likely follow-ups: What if other characters (letters, numbers) are mixed in and should be ignored?

What's the space complexity? Could you solve it without a stack?

Red flag: checking only whether the counts of opens and closes match; that misses ordering entirely, and `"("` would wrongly pass (it's `False` above).

Learn it in: Chapter 33, section 33.4 (stacks).

APPLY

Common mistakes

Mistake	Symptom	Fix
Nested loops over the same array	$O(n^2)$: fine on test data, far too slow at real volume	Reach for a dictionary or set first when checking "does this exist"
Keeping <code>visited</code> as a list in BFS or DFS	Every check is a scan, so the walk becomes $O(V^2)$	Use a set: <code>visited = {start}, visited.add(nb)</code>
Assuming binary search works on unsorted data	Silently wrong answers, no error	Confirm sorted input before reaching for binary search
Losing a pointer before reassigning it (linked lists)	The rest of the list gets silently dropped	Save <code>head.next</code> before overwriting it, every time
No <code>visited</code> tracking in graph traversal	Infinite loop on any graph with a cycle	Always track visited nodes on a graph; trees don't need it (no cycles) but graphs do
Recursion with no base case, or a wrong one	<code>RecursionError</code> (a stack overflow)	State the base case explicitly before writing the recursive case
A mutable default as a cache (<code>def f(n, cache={})</code>)	State leaks between calls; timings and results depend on what ran before	<code>@lru_cache</code> , or <code>cache=None</code> and create the dictionary inside
Treating Big-O as "how fast it feels"	Confidently wrong complexity claims	Trace what happens as <code>n</code> doubles, on paper, before answering; then measure
Reinventing a sort algorithm in real code	Slower and buggier than the built-in sort	Know how sorting works for interviews; use <code>sorted()</code> in practice

In the real world: the DSA question that was really a scale question

Karan, interviewing for a Data Engineer role, is asked: "Given two large lists of customer IDs, find the ones that appear in both." He recognizes it immediately as a set-intersection problem and writes a clean, $O(n + m)$ solution using Python sets in under two minutes.

The interviewer doesn't move on. "Good. Now: what if one of these lists has 200 million IDs and doesn't fit in memory?" This is a Senior-level follow-up, and Karan pauses, then reasons out loud: "Then I can't build a full Python set in memory. I'd sort both lists externally, with a merge sort variant designed for data that doesn't fit in RAM: sort chunks that do fit, write each to disk, then merge the sorted chunks. Then I'd walk through both sorted lists with two pointers, the same pattern as Q72A-012's sorted-array merge, advancing whichever pointer is behind. That's $O(n \log n)$ for the external sort plus $O(n + m)$ for the merge-walk, and it only ever needs a small, bounded amount of memory at once."

The follow-up wasn't really about algorithms at all: it was about whether Karan's algorithmic thinking survives contact with a real, unglamorous constraint (memory), which is the everyday reality of data engineering work. His answer to the "easy" version was necessary but not sufficient; the extra-point move that mattered was **[+Scale]**, reasoning about what changes when an assumption (it fits in memory) breaks.

Project

Goal: implement, verify, and explain your own version of the highest-value patterns from this chapter.

Tools you'll need

Python (Chapter 17, section 17.0), standard library only, as in Chapter 33: `collections.deque` for BFS queues, `collections.Counter` for counting, `heapq` for the kth-largest pattern, `functools.lru_cache` for memoization, `time.perf_counter()` for timings. This chapter's snippets were run on Python 3.11.15. Practice platforms such as LeetCode or HackerRank are useful for volume practice, though the questions in this bank are deliberately sized for data-role interviews, not the hardest tier those platforms offer.

1. Implement Two Sum, a sliding-window maximum, and a stack-based bracket validator, each with your own test cases, run and checked, not just written.
2. Time a nested-loop $O(n^2)$ approach against a set-based $O(n)$ approach at two sizes, n and $2n$, with at least 5,000 elements, the way Q72A-002 did. Report both speedups, and check that doubling n roughly quadruples the slow version's time.
3. Build a small binary search tree from a list of your own choosing, verify that in-order traversal returns it sorted, and compare its height with the tree you get from the same values sorted.
4. Pick one problem from this chapter and practise saying your solution out loud, Chapter 69-style: clarify, simple answer first, then edge cases and complexity.

Key terms

Big-O notation · time complexity · space complexity · $O(1)$ / $O(\log n)$ / $O(n)$ / $O(n \log n)$ / $O(n^2)$ / $O(2^n)$ · amortized complexity · hash map / set · hashable · two-pointer pattern · sliding-window pattern · recursion · base case · `RecursionError` · memoization · `lru_cache` · dynamic programming (top-down / bottom-up) · merge sort · quicksort · bubble sort · binary search · stable sort · heap · linked list · node · head · fast/slow pointers (Floyd's cycle detection) · binary search tree (BST) · BST invariant · tree height · in-order/pre-order/post-order traversal · breadth-first search (BFS) · depth-first search (DFS) · level-order traversal · stack (LIFO) · queue (FIFO)

Final-week revision list

Q72A-001, Q72A-002, Q72A-007, Q72A-008, Q72A-009, Q72A-014, Q72A-018, Q72A-020, Q72A-021, Q72A-025, Q72A-026, Q72A-027, Q72A-028, Q72A-030.

Where this leads

- **Looking back: Chapter 33, The Computer Science You Actually Need**, teaches the structures and patterns this bank tests; **Chapter 72, Python & pandas Question Bank**, covers the Python gotchas that came back here (mutable defaults, `is` versus `==`).
- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 74, Machine Learning Question Bank**, picks up "algorithm" in its other sense: the learning algorithms themselves, not data structures.
- **Chapter 77, Data Engineering & Data System Design Bank**, is where several of this chapter's patterns (the $O(n^2)$ lesson of Q72A-002 especially, which returns in Q77-035's deduplication design) show up again at real production scale.

Chapter 73. Statistics, Probability & Experimentation Bank

Chapter at a glance

You will learn to: answer probability puzzles that show up across every data role's interviews, and explain *why* the surprising answer is correct, not just what it is · reason correctly about distributions, p-values, and confidence intervals, including the ways almost everyone misinterprets them at first · design an A/B test's sample size before running it, and debug one that's already gone wrong · spot Simpson's paradox and correlation-masquerading-as-causation in real-looking data.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapter 21 (probability and distributions), Chapter 22 (confidence intervals, tests, A/B basics, confounders, Simpson's paradox, and regression basics in section 22.10), Chapter 30 (inference, power, experiment design, the sample-ratio check) and Chapter 31 (causal inference without experiments). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: about 8–10 hours for a first pass, running every cell and saying each answer aloud (the simulations take time to run and to understand); 1 hour for the final-week list.

How this chapter is built. Same format as every question bank in Part 8: every core question gives a memory hook ("Remember it as..."), a one-line answer, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, one per line, tagged the same way: **[+Validate]**, **[+Business]** and so on). Then come the likely follow-ups, the red flag, and where to learn it. Rapid-fire sections are scan tables. **Every number in this chapter comes from running the code shown** (NumPy, SciPy, statsmodels): simulations, test statistics, p-values, sample sizes. The order: probability warm-ups first, then the famous puzzles (they test the same basics from an angle), then distributions, testing, experiment design and debugging, causal reasoning, and two walk-throughs that put it all together.

Learn it in pointers name the chapter and section that teach each idea: Chapter 21 (probability and distributions), Chapter 22 (tests, A/B basics, confounders, Simpson's paradox), Chapter 30 (inference, power, experiment design, the sample-ratio check) and Chapter 31 (causal inference without experiments). The few ideas marked **Beyond the book** go further than those chapters and carry their own short explanation, so you can learn them here.

In this chapter

LEARN	73.0 The setup cell, and how to use this bank	179
	73.1 Core probability concepts	180
	73.2 Distributions	184
	73.3 Hypothesis testing fundamentals	186
	73.4 A/B test design	189
	73.5 A/B test debugging	191
	73.6 Causal inference and Simpson's paradox	194
	73.7 Live-coding and live-analysis walk-throughs	198
APPLY	Common mistakes · In the real world · Project	201
REVIEW	Key terms · Final-week revision list	203
NEXT	Where this leads	203

LEARN

73.0 The setup cell, and how to use this bank

Levels and roles. Each question carries a level and the roles that usually ask it:

- **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds.
- **DA** data analyst · **DS** data scientist · **PA** product analyst (the role that runs A/B tests most) · **BA** business analyst.

Within each section the core questions run from easier to harder. If you're preparing for a first analyst job, do every Fresher question first, then come back for the Mid ones.

The setup cell. Open a new notebook in your Chapter 17 environment and run this cell first. Every later cell in the chapter assumes it ran, and the cells run in order, like a notebook:

```
import numpy as np
from scipy import stats
import statsmodels.stats.api as sms

rng = np.random.default_rng(73)
```

This cell prints nothing; if it runs without an error, you're ready.

- `import numpy as np` loads NumPy (Chapter 18, section 18.1) for arrays and random numbers.
- `from scipy import stats` loads SciPy's statistics module: the distributions, `ttest_ind`, `chisquare`, `chi2_contingency` and `pearsonr`. You installed SciPy in Chapter 21, section 21.5.
- `import statsmodels.stats.api as sms` loads statsmodels' collection of tests and power tools under the short name `stats`: the proportion tests (`proportions_ztest`, `confint_proportions_2indep`) and the power calculators (`proportion_effectsize`, `NormalIndPower`) used in Q73-020 and Q73-036. You installed statsmodels in Chapter 22, section 22.2.
- `rng = np.random.default_rng(73)` makes a random number generator with seed 73 (Chapter 21, section 21.5), so your "random" numbers match the ones printed here. Each simulation cell below

starts the generator again with the same seed, so it prints the same answer even if you run it on its own.

73.1 Core probability concepts

Warm-ups, 73.1

Roles: DA, DS, PA and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-001	Independent vs. mutually exclusive events?	Independent: one doesn't affect the other's probability / mutually exclusive: they can't both happen	[+Edge cases] two mutually exclusive events with nonzero probability are automatically <i>not</i> independent: one happening tells you the other definitely didn't	Fresher · 21.6
Q73-002	What's the difference between $P(A \text{ and } B)$ and $P(A \text{ or } B)$?	$P(A) \times P(B)$ if independent / $P(A) + P(B) - P(A \text{ and } B)$ always	[+Edge cases] forgetting to subtract the overlap in "or" is the single most common probability arithmetic mistake	Fresher · 21.6
Q73-003	What's a conditional probability, in one sentence?	The probability of A, given that B is already known to be true: $P(A B)$	[+Business] almost every real business probability question is secretly conditional ("what's the chance this customer churns" really means "given what we know about them")	Fresher · 21.6
Q73-004	Expected value of a fair six-sided die roll?	$(1+2+3+4+5+6)/6 = 3.5$: a value the die itself can never actually show	[+Edge cases] expected value doesn't have to be a possible outcome; it's a long-run average, not a prediction for one roll	Fresher · 21.5

Q73-005 · Explain Bayes' theorem, and why a "99% sensitive" test can still be wrong most of the time

Level: Mid · **Roles:** DA, DS, PA

Remember it as: *A rare disease and an imperfect test: even a good test is mostly wrong about rare things, because there are so many more healthy people to false-positive on than sick people to correctly catch.*

Answer in one line: Bayes' theorem updates a prior probability using new evidence: $P(A|B) = P(B|A) \times P(A) / P(B)$, and it matters most when the thing you're testing for is rare, because the base rate can overwhelm even a very good test.

By hand first, with 1,000 people. A disease with 1% prevalence, a test that catches 99% of cases (its **sensitivity**) and wrongly flags 5% of healthy people (its **false-positive rate**). Out of 1,000 people, 10 are sick and 990 are healthy:

	Test positive	Test negative	Total
Sick	9.9	0.1	10
Healthy	49.5	940.5	990
Total	59.4	940.6	1,000

Of the 59.4 people who test positive, only 9.9 are sick: $9.9 \div 59.4 = 16.7\%$. Counting people like this (**natural frequencies**) is the easiest way to explain the answer aloud. The same in Python:

```
p_disease = 0.01          # prevalence: the base rate
p_pos_given_disease = 0.99 # sensitivity
p_pos_given_no_disease = 0.05 # false-positive rate

p_positive = p_pos_given_disease * p_disease + p_pos_given_no_disease * (1 - p_disease)
p_disease_given_positive = (p_pos_given_disease * p_disease) / p_positive
print(f"P(positive) = {p_positive:.4f}")
print(f"P(disease | positive) = {p_disease_given_positive:.4f}")
```

```
P(positive) = 0.0594
P(disease | positive) = 0.1667
```

- `p_positive` is the **Total** row of the table as a share: the sick who test positive plus the healthy who test positive (total probability, Chapter 21, section 21.7).
- `p_disease_given_positive` is Bayes' rule: the sick positives divided by all positives.

Someone testing positive on a test that catches 99% of cases and wrongly flags 5% of healthy people has the disease only **16.7%** of the time.

Tier	What to say
Passes	States the formula correctly, can't apply it to a concrete number
Strong	Works through the disease example above, arriving at 16.7%, and explains <i>why</i> it's so much lower than 99%: the 5% false-positive rate applies to the 99% of people who are healthy, a much bigger group than the 1% who are sick
Extra points	[+Validate] the 1,000-person table and the computed numbers agree [+Edge cases] calling this test "99% accurate" is itself the loose word: its accuracy (the share of everyone it gets right) is $0.99 \times 0.01 + 0.95 \times 0.99 = 95.0\%$, and neither number is the chance that a positive is sick [+Business] this is why a fraud or churn model's headline accuracy needs the base rate before it means anything: it's precision (Chapter 39, section 39.1), traced back to its probability root

Likely follow-ups: How would a higher-prevalence disease change the answer? What test characteristic matters more for a rare event, sensitivity or specificity?

Red flag: treating sensitivity as $P(\text{disease} \mid \text{positive})$: "it's 99% sensitive, so a positive means 99% likely".

Learn it in: Chapter 21, section 21.7 (Bayes' rule, "A worked Riverstone example").

Q73-006 · Walk through the Monty Hall problem, and prove your answer

Level: Mid · **Roles:** DA, DS

Remember it as: *The host's choice isn't random: he always avoids the car. That single fact is why switching wins twice as often.*

Answer in one line: Switching doors wins the car $\frac{2}{3}$ of the time; staying wins only $\frac{1}{3}$: because the host's forced choice (he always opens a goat door, never the car) concentrates the other $\frac{2}{3}$ probability onto the one remaining unopened door.

By hand first. Chapter 21, section 21.7 lists the three places the car can be, in a three-row table: staying wins in one row, switching in two. In an interview, draw that table first. Then offer a simulation as the check, 100,000 games. Before you run it, predict the two numbers it will print:

```
rng = np.random.default_rng(73)
wins_switch = wins_stay = 0
for _ in range(100_000):
    car = rng.integers(0, 3)
    choice = rng.integers(0, 3)
    remaining = [d for d in range(3) if d != choice and d != car]
    host_opens = rng.choice(remaining)
    switch_choice = [d for d in range(3) if d != choice and d != host_opens][0]
    wins_stay += (choice == car)
    wins_switch += (switch_choice == car)
print(f"stay wins:    {wins_stay / 100_000:.3f}")
print(f"switch wins: {wins_switch / 100_000:.3f}")
```

```
stay wins:    0.332
switch wins: 0.668
```

- `rng = np.random.default_rng(73)` starts the generator again with seed 73, so you get the numbers printed here. `wins_switch = wins_stay = 0` sets both counters to 0 in one line.
- `100_000` is 100000: Python ignores the underscore, which is there to make the number easy to read.
- `rng.integers(0, 3)` draws a whole number from 0 up to, but not including, 3: the three doors, numbered 0, 1 and 2 (Chapter 21, section 21.5 rolled a die the same way).
- `remaining` is a list comprehension with two conditions (Chapter 17, section 17.6): the doors that are neither your pick nor the car, which the host may open. There are two when you picked the car, one otherwise.
- `rng.choice(remaining)` picks one of those doors at random.
- `switch_choice` is the one door that is neither your pick nor the opened one; `[0]` takes it out of its one-item list.
- `wins_stay += (choice == car)` adds the comparison to the counter. When Python adds `True` or `False` to a number it counts them as 1 and 0, so the counter goes up only on a win.

Tier	What to say
Passes	States "switching is better" from memory, can't explain why
Strong	The reasoning: your original pick has a 1/3 chance of being right; the other two doors together have a 2/3 chance; once the host removes one (always a goat, never the car), that whole 2/3 collapses onto the single remaining door
Extra points	[+Validate] a 100,000-game simulation lands almost exactly on the theoretical 1/3 and 2/3 [+Edge cases] the key, often-missed detail: this only works because the host's action is <i>not</i> random: he knows where the car is and always avoids it; if he opened a random unopened door and it happened to reveal a goat, switching would no longer help

Likely follow-ups: What if there were 100 doors instead of 3? What changes if the host sometimes opens the car by accident?

Red flag: claiming it's 50/50 after one door is removed, the single most common wrong answer to this exact question.

Learn it in: Chapter 21, section 21.7 ("Two famous puzzles").

Q73-007 · The birthday problem: how many people need to be in a room before it's more likely than not that two share a birthday?

Level: Mid · **Roles:** DA, DS

Remember it as: It's not about matching one specific date: it's about how many pairs of people exist, and pairs grow far faster than people do.

Answer in one line: Just **23** people, computed as 1 minus the probability that all birthdays are different (the complement rule).

By hand first. Chapter 21, section 21.7 builds the answer step by step: the second person misses the first's birthday with probability 364/365, the third misses both with 363/365, and so on, and 1 minus the product is the chance of a match. Here the same loop becomes a function, so you can see the whole curve:

```
def birthday_prob(n):
    p_no_match = 1.0
    for i in range(n):
        p_no_match *= (365 - i) / 365
    return 1 - p_no_match

for n in (22, 23, 50, 70):
    print(f"{n} people: {birthday_prob(n):.4f}")
```

```
22 people: 0.4757
23 people: 0.5073
50 people: 0.9704
70 people: 0.9992
```

- `birthday_prob(n)` starts with `p_no_match = 1.0` and multiplies in one factor per person. `p_no_match *= x` is short for `p_no_match = p_no_match * x`, as `+=` is for adding (Chapter 17, section 17.6).
- `return 1 - p_no_match` is the complement rule (Chapter 21, section 21.6).

- The loop prints the curve: just under a half at 22 people, just over at 23.

Tier	What to say
Passes	Says 23, from memory, can't explain why
Strong	Explains the complement (1 minus the chance all birthdays differ) and the pairs insight: with 23 people there are $23 \times 22 / 2 = 253$ distinct pairs, each a chance for a shared birthday, and 253 chances is a lot even at low odds per pair
Extra points	[+Validate] the computed curve above: from under 50% at 22 people to over 99.9% at 70 [+Business] the same "count the chances, not the people" logic is behind multiple comparisons: 20 metrics at $\alpha = 0.05$ give 20 chances of a false alarm, 64% overall (Q73-028)

Likely follow-ups: What's the exact formula, not just the loop? How does the answer change for a 50% chance of *three* people sharing a birthday?

Red flag: answering close to 183 (half of 365), confusing "matching one specific date" with "any two people matching each other."

Learn it in: Chapter 21, section 21.7 ("Two famous puzzles"); the complement rule is section 21.6.

73.2 Distributions

Q73-008 · When would you use a Binomial distribution vs. a Poisson distribution?

Level: Fresher · **Roles:** DA, DS

Remember it as: *Binomial counts successes out of a known, fixed number of tries. Poisson counts events with no fixed "number of tries" at all, just a rate over time or space.*

Answer in one line: Binomial models the count of successes in a fixed number of independent yes/no trials (how many of 40 deliveries are late, how many of 100 leads convert); Poisson models the count of events in a fixed interval when there's no natural "number of trials," only an average rate (complaints per week, support tickets per hour, defects per metre of cable).

Tier	What to say
Passes	"Binomial is for two outcomes, Poisson is for counting things"
Strong	The trials-vs-rate distinction above, with a concrete example of each
Extra points	[+Edge cases] a step beyond Chapter 21: Poisson is the limiting case of Binomial as the number of trials grows very large and the per-trial probability gets very small, while their product (the expected count) stays fixed; the two aren't unrelated [+Business] Riverstone's 9.4 complaints a week (Chapter 21) is a textbook Poisson count, and so is "how many tickets will arrive in the next hour" for staffing; "what fraction of these 500 leads will convert" is Binomial

Likely follow-ups: What does the Poisson distribution's single parameter (λ) represent? When does Binomial start to look like a Normal distribution?

Red flag: treating the two as interchangeable, or not knowing which applies to a described scenario.

Learn it in: Chapter 21, section 21.5 (Binomial, Poisson).

Q73-009 · State the Central Limit Theorem in plain English, and say why it matters even if your underlying data isn't normally distributed

Level: Fresher · **Roles:** DA, DS, PA

Remember it as: *It doesn't matter what shape the data is. Average enough of it together, and the average's own distribution becomes bell-shaped anyway.*

Answer in one line: The Central Limit Theorem says that the distribution of a *sample mean* approaches a Normal distribution as sample size grows, regardless of the shape of the underlying data it's drawn from: which is why so much of statistics (t-tests, confidence intervals) can lean on Normal-distribution math even when real-world data (revenue, wait times) is skewed, not bell-shaped at all.

Tier	What to say
Passes	"Averages tend to be normal" (true, no mechanism, no "why it matters")
Strong	+ explicitly separates the individual data points (which can be any shape) from the <i>sample mean's</i> distribution (which becomes Normal), and connects this to why t-tests and z-tests are valid on skewed real-world data as long as the sample is reasonably large
Extra points	[+Business] this is why Riverstone's order values (right-skewed, Chapter 21, section 21.4) can still be compared between two groups with a t-test on the means (Chapter 30, section 30.3), even though the raw order values look nothing like a bell curve [+Edge cases] "large enough" depends on how skewed the underlying data is; a very heavy-tailed distribution needs a bigger sample before the CLT's approximation is trustworthy

Likely follow-ups: How large does a sample need to be for CLT to "kick in"? What happens to inference if you ignore CLT and the sample is too small?

Red flag: claiming the underlying *data* becomes normal, rather than the *sample mean's* distribution.

Learn it in: Chapter 21, section 21.8 (sampling and the central limit theorem); skew is section 21.4.

Rapid-fire, 73.2

Roles: DA and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-010	What does a distribution's variance measure?	The average squared distance of values from the mean: how spread out the data is	[+Edge cases] squaring means variance is in squared units (£ ² for revenue); standard deviation (variance's square root) is back in the original, interpretable units	Fresher · 21.2
Q73-011	Normal vs. log-normal distribution: when does each apply?	Normal: symmetric, bell-shaped data / log-normal: data whose <i>logarithm</i> is normal, common for values that can't go negative and are right-skewed (revenue, wait times)	[+Business] most real business "amount" data (order value, salary, session length) is closer to log-normal than Normal: check before assuming symmetry	Mid · beyond the book (skew 21.4; logs 31.0)

#	Question	One-line answer	Extra point	Level · learn it in
Q73-012	What's a uniform distribution, and give one real business example?	Every outcome in a range is equally likely	[+Business] a well-built A/B assignment: each user's hash bucket 0-99 is equally likely, which is what lets you check a 50/50 split (Q73-026); also <code>rng.uniform(1, 10, 200)</code> in Q73-032, where every account size from 1 to 10 is equally likely	Fresher · 21.5
Q73-013	What does "long-tailed" or "heavy-tailed" mean?	Extreme values occur more often than a Normal distribution would predict	[+Edge cases] heavy-tailed data can make sample means unstable and confidence intervals misleadingly narrow if treated as Normal without checking	Mid · 21.4

73.3 Hypothesis testing fundamentals

Q73-014 · Explain a p-value to a non-technical stakeholder, without saying "probability that the null hypothesis is true"

Level: Fresher · **Roles:** DA, DS, PA, BA

Remember it as: A p-value answers "how surprising is this result, if nothing real were going on?" It never answers "how likely is it that something real is going on?"

Answer in one line: A p-value is the probability of seeing a result at least as extreme as what you observed, if the null hypothesis were actually true: it's a statement about how surprising your data is under "nothing changed," not a statement about the probability that something changed.

Tier	What to say
Passes	"It tells you if the result is significant" (technically true, doesn't explain the mechanism, invites the classic misreading)
Strong	The correct definition above, explicitly distinguishing it from "probability the null hypothesis is true," which is the single most common misinterpretation
Extra points	[+Business] a non-technical framing that avoids the trap: "if our change actually did nothing, we'd see a result this extreme only 3% of the time by chance, which is why we're treating it as a real effect" [+Edge cases] a p-value says nothing about the <i>size</i> of an effect, only its surprise value under the null: a tiny, practically meaningless effect can still have a very small p-value with enough data

Likely follow-ups: What's the difference between statistical significance and practical significance? Why is $p < 0.05$ the conventional threshold, and is it always the right one?

Red flag: defining a p-value as "the probability the null hypothesis is true" or "the probability the result is due to chance," both common, both wrong.

Learn it in: Chapter 22, section 22.2 ("What a p-value is not"); Chapter 30, section 30.3 ("What the p-value does not say"); practical significance is Chapter 22, section 22.8.

Q73-015 · Demonstrate that underpowered tests miss real effects far more often than people expect

Level: Mid · Roles: DA, DS, PA

Remember it as: *A small sample doesn't just make you less sure: it makes you actively likely to miss a real, meaningful effect entirely.*

Answer in one line: With a real effect but too small a sample, a test's **Type II error rate** (missing a real effect) can be dramatically higher than most people's intuition suggests, and it falls sharply as sample size grows.

Verified, live: a true underlying difference of 0.3 standard deviations, tested with a t-test, 2,000 simulated experiments at each sample size:

```
rng = np.random.default_rng(73)
for n in [30, 200]:
    misses = 0
    for _ in range(2000):
        a = rng.normal(0, 1, n)
        b = rng.normal(0.3, 1, n)
        t, p = stats.ttest_ind(a, b)
        if p >= 0.05:
            misses += 1
    print(f"n={n} per group: Type II error rate = {misses / 2000:.3f}")
```

```
n=30 per group: Type II error rate = 0.800
n=200 per group: Type II error rate = 0.138
```

- `rng.normal(0, 1, n)` draws `n` values from a normal distribution with mean 0 and standard deviation 1: group A. Group B has mean 0.3, so a real difference of 0.3 standard deviations always exists.
- `stats.ttest_ind(a, b)` is the two-sample t-test (Chapter 22, section 22.2; Chapter 30, section 30.3). It returns the t statistic and the p-value, which `t, p = ...` unpacks. Without `equal_var=False` it's the ordinary t-test, which is fine here because both groups have the same spread.
- `if p >= 0.05: misses += 1` counts the experiments that fail to find the difference: each one is a Type II error.

With only 30 people per group, a real effect is **missed 80% of the time**: the test says "no significant difference" four times out of five, even though a real difference exists. At 200 per group, that drops to 13.8%.

Tier	What to say
Passes	"Bigger samples are more reliable" (true, no sense of <i>how much</i> more reliable)
Strong	Explains Type II error and power correctly, and that underpowered tests systematically under-detect real effects, not just "add noise"
Extra points	[+Validate] the simulated numbers above: an 80% miss rate falling to 14% from sample size alone, nothing else changed [+Business] a team that runs an underpowered A/B test and concludes "no effect" when there actually was one has made a real, costly, invisible mistake, not a safe, conservative call: this is exactly why sample-size planning (section 73.4, Q73-020) happens <i>before</i> a test runs, not after

Likely follow-ups: What's the relationship between power, effect size, sample size, and significance level? How do you choose a target power before running a test?

Red flag: treating "not statistically significant" as proof "there's no effect," rather than "we may not have had enough data to detect one."

Learn it in: Chapter 22, section 22.3 (Type II error and power); Chapter 30, section 30.7 (power and sample size).

Rapid-fire, 73.3

Roles: DA, DS and PA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-016	Type I vs. Type II error?	Rejecting a true null (a false alarm) / failing to reject a false null (a miss)	[+Business] which error costs more depends on context: a fraud check favours avoiding Type II (missing real fraud); an automatic account-freeze rule favours avoiding Type I (freezing an honest customer's account)	Fresher · 22.3
Q73-017	What's statistical power?	The probability of correctly detecting a real effect when one exists, i.e. $1 - \text{Type II error rate}$	[+Business] 80% power is the conventional target, meaning you accept a 1-in-5 chance of missing a real effect even when your test is well-designed	Fresher · 22.3, 30.7
Q73-018	t-test vs. z-test, when does it matter which you use?	t-test when the population standard deviation is unknown and estimated from the sample (the usual real-world case); z-test when it's known, or for proportions with large samples	[+Edge cases] with a large sample, the t-distribution converges to the normal distribution anyway, so the practical difference shrinks as n grows	Mid · 22.1, 22.2
Q73-019	What does a 95% confidence interval actually mean?	If you repeated the sampling process many times, 95% of the resulting intervals would contain the true population value	[+Edge cases] it does <i>not</i> mean "95% probability the true value is in this specific interval": the true value either is or isn't in any single interval; the 95% describes the <i>procedure's</i> long-run reliability	Fresher · 22.1

73.4 A/B test design

Q73-020 · Calculate the required sample size for an A/B test, before running it

Level: Mid · **Roles:** DA, DS, PA

Remember it as: You need four numbers before you can answer "how many": baseline rate, the smallest change worth detecting, your significance level, and your desired power.

Answer in one line: Sample size depends on the baseline conversion rate, the **minimum detectable effect (MDE)** you actually care about, the significance level (usually 0.05), and the desired power (usually 0.80): smaller effects and higher confidence both require larger samples.

Verified, live: baseline conversion 9.6%, and the business says the smallest lift worth acting on is 1.5 percentage points (to 11.1%), at 80% power and $\alpha = 0.05$:

```
effect_size = sms.proportion_effectsize(0.111, 0.096)
n_required = sms.NormalIndPower().solve_power(effect_size, power=0.8, alpha=0.05, ratio=1)
print(f"Cohen's h = {effect_size:.4f}")
print(f"Required sample size per group: {np.ceil(n_required):,.0f}")
```

```
Cohen's h = 0.0493
Required sample size per group: 6,466
```

- `sms.proportion_effectsize(0.111, 0.096)` turns the two rates into **Cohen's h**, an effect size for proportions built on an arcsine scale (Chapter 30, section 30.7). The rate you hope for goes first, the baseline second, as in Chapter 30.
- `sms.NormalIndPower()` is a power calculator for two independent groups. `.solve_power(...)` solves for whichever number you leave out: here the sample size, given `power=0.8`, `alpha=0.05`, and `ratio=1` (the second group is the same size as the first).
- `np.ceil` rounds **up**, because 6,465.6 people would leave the test slightly short of its power; `:.0f` prints a whole number with a thousands comma.

Chapter 22's hand formula, `sample_size_for_proportion(0.096, 1.5)` from section 22.3, gives 6,473 for the same inputs. The small difference is the effect-size scale (statsmodels works on Cohen's h, an arcsine transform). Either answer is fine in an interview; say which one you used.

Tier	What to say
Passes	"You need a big enough sample" with no method for computing it
Strong	Names the four required inputs, and correctly identifies that MDE is a <i>business</i> decision (how small a change is still worth caring about), not a purely statistical one
Extra points	[+Validate] the computed number above, 6,466 per group, checked against Chapter 22's formula, not a guessed round number [+Business] this number directly answers "how long will this test need to run": divide 6,466 by the expected daily traffic per arm. It's the most common follow-up question from a stakeholder, and this calculation answers it

Likely follow-ups: How does the required sample size change if the MDE gets smaller (harder to detect)? What if traffic is limited, how would you trade off test duration against detectable effect size?

Red flag: picking a sample size arbitrarily ("let's run it for two weeks") with no power calculation behind it.

Learn it in: Chapter 22, section 22.3 (designing an A/B test, the sample-size formula); Chapter 30, section 30.7 (power and sample size, Cohen's h).

Q73-021 · What's a Minimum Detectable Effect (MDE), and who should decide it?

Level: Mid · **Roles:** DA, DS, PA, BA

Remember it as: MDE isn't a statistics question. It's "how small a change would we actually act on?": a business question dressed in statistical clothing.

Answer in one line: The MDE is the smallest true effect size a test is designed to reliably detect; choosing too small an MDE demands an impractically large sample, while choosing too large a one risks missing real, meaningful improvements that are just below the threshold.

Tier	What to say
Passes	"It's the effect size you're testing for" (correct, misses who should set it and why)
Strong	+ explicitly: the MDE should come from the business ("would we actually change anything for a 0.5 percentage point lift? For 2 points?"), not be back-calculated from whatever sample size happens to be convenient
Extra points	[+Business] setting the MDE by working backward from "what sample size can we get in two weeks" instead of forward from "what change would we actually act on" is a common, subtle mistake: it optimizes for a schedule, not for a decision worth making [+Edge cases] an MDE set too small can make a test run for months chasing statistical significance on an effect too tiny to matter commercially even if proven real

Likely follow-ups: How would you choose an MDE with no historical baseline data at all? What happens if the observed effect is smaller than the MDE but still "significant"?

Red flag: treating MDE as a purely mathematical input with no connection to what the business would actually do differently.

Learn it in: Chapter 22, section 22.3; Chapter 30, sections 30.7 and 30.8 (designing an experiment).

Rapid-fire, 73.4

Roles: DA, DS and PA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-022	Why randomize which users see which variant?	Randomization is what makes the two groups comparable on everything else (known and unknown factors alike), so any observed difference can be attributed to the treatment	[+Edge cases] this is the entire logic of a controlled experiment versus an observational comparison (section 73.6): randomization is what earns the right to say "caused," not just "correlated"	Fresher · 22.3, 30.8

#	Question	One-line answer	Extra point	Level · learn it in
Q73-023	One-tailed vs. two-tailed test, when would you use each?	Two-tailed for a change in either direction (the book's default); one-tailed only when chosen before seeing the data, and only when a change in the other direction would lead to the same decision	[+Edge cases] switching to a one-tailed test after seeing the data, to make a marginal result "significant," is a form of p-hacking	Mid · 22.2
Q73-024	What's a holdout group, and why keep one even after shipping a winning test?	Beyond the book: a small group (say 5% of users) deliberately kept on the old version after launch, to measure the change's lasting effect against a true baseline	[+Business] without a holdout, a genuinely working feature's impact becomes impossible to separate from seasonal trends or other changes happening at the same time	Mid · beyond the book (design: 30.8)

73.5 A/B test debugging

Q73-025 · A dashboard shows a "statistically significant" result after checking it daily throughout the test. What's wrong, and prove it?

Level: Mid · **Roles:** DA, DS, PA

Remember it as: *Checking significance every day and stopping the moment it looks good is like flipping a coin until it lands on heads, then calling the coin biased.*

Answer in one line: Checking for significance repeatedly and stopping as soon as $p < 0.05$ appears ("peeking") inflates the true false-positive rate far above the nominal 5%, because each additional look is another chance for random noise to cross the threshold.

Verified, live: 2,000 simulated experiments with **no real effect at all** (both groups drawn from the identical distribution). Data arrives 20 per group at a time, and a t-test runs at every look from 40 to 200 per group: 9 looks in all:

```
rng = np.random.default_rng(73)
false_positives = 0
for _ in range(2000):
    data_a, data_b = [], []
    found_sig = False
    for _ in range(10):
        data_a.extend(rng.normal(0, 1, 20))
        data_b.extend(rng.normal(0, 1, 20))
        if len(data_a) >= 40:
            t, p = stats.ttest_ind(data_a, data_b)
            if p < 0.05:
                found_sig = True
                break
    false_positives += found_sig
print(f"false positive rate with repeated peeking: {false_positives / 2000:.3f}")
```

```
false positive rate with repeated peeking: 0.163
```


- `data_a.extend(...)` adds 20 new values to the list: one more batch of visitors per group. Ten batches take each group to 200.
- `if len(data_a) >= 40` skips a test on the very first batch, so the looks come at 40, 60, ..., 200 per group: 9 of them.
- `found_sig = True` and `break` stop the experiment at the first p below 0.05: that's the peeking.
- `false_positives += found_sig` counts `True` as 1, as in Q73-006.

With **no real effect whatsoever**, peeking and stopping at the first significant-looking result gives a false "win" **more than 3 times as often** as the stated 5% significance level promises. The exact rate moves a little with the seed (between 16% and 18% across ten seeds), and it grows with the number of looks: Chapter 22, section 22.4 found 14.3% with five looks and 23.9% with twenty.

Tier	What to say
Passes	"You shouldn't check too often" (correct instinct, no quantified reason)
Strong	Explains the mechanism: each check is another roll of the dice against a 5% threshold, and repeated rolls compound the chance of a false positive somewhere along the way
Extra points	[+Validate] the simulated number above: a false-positive rate of about 16% against a claimed 5%, from peeking alone, on data with zero true effect [+Business] the fix isn't "never check early," it's using a sequential testing method (group sequential design, or Bayesian methods designed for continuous monitoring) that's built to allow early looks without inflating the error rate: or committing to a single, pre-planned sample size and analysis date and holding to it

Likely follow-ups: What's a sequential testing correction, in concept? How would you explain this risk to a PM who wants a "quick check" on day 2 of a two-week test?

Red flag: not recognizing peeking as a real statistical problem, treating it as a minor process nitpick.

Learn it in: Chapter 22, section 22.4 ("Peeking"); Chapter 30, section 30.10 ("Peeking").

Q73-026 · An A/B test's traffic split is supposed to be 50/50, but you're seeing 5,200 vs. 4,800. Is that a problem?

Level: Mid · **Roles:** DA, DS, PA

Remember it as: A Sample Ratio Mismatch is a smoke alarm for the whole experiment, not a statistics problem to explain away.

Answer in one line: Run a chi-square goodness-of-fit test against the expected 50/50 split; if it's significant, you have a **Sample Ratio Mismatch (SRM)**, which almost always means something is broken in how users were assigned to groups, not real random variation, and the experiment's other results shouldn't be trusted until it's found and fixed.

Verified, live, two scenarios:

```
for observed in ([5200, 4800], [5100, 4900]):
    chi2, p = stats.chisquare(observed, f_exp=[5000, 5000])
    print(f"observed {observed}: chi2 = {chi2:.3f}, p = {p:.4f}")
```

```
observed [5200, 4800]: chi2 = 16.000, p = 0.0001
observed [5100, 4900]: chi2 = 4.000, p = 0.0455
```

- `stats.chisquare(observed, f_exp=[5000, 5000])` is the goodness-of-fit test from Chapter 30, section 30.9 (Step 2). `f_exp` is the list of **expected** counts: an exact 50/50 split of 10,000. Leave it out and SciPy assumes equal counts anyway, which is what Chapter 30 did; writing it makes the 50/50 assumption visible, and lets you test a 90/10 split the same way.
- The loop runs the test on both splits and prints the statistic and the p-value.

The 5,200/4,800 split is a real SRM ($p = 0.0001$): investigate before reading any result. The 5,100/4,900 split gives $p = 0.0455$: under 0.05, but not under the stricter alarm teams usually use (below), so it's worth a look at the assignment logs, not an alarm. Chapter 30's Riverstone website test (24,036 vs 23,250 visitors, $p = 0.0003$) is exactly the first case, and section 30.10 finds its cause.

Tier	What to say
Passes	"That's probably just random variation, it's close to 50/50"
Strong	Runs the chi-square test rather than eyeballing it, and correctly treats a significant result as a red flag about the experiment's plumbing, not a metric to report on
Extra points	[+Validate] the test itself: 5,200/4,800 is a significant SRM ($p = 0.0001$), while 5,100/4,900 is not under a strict rule ($p = 0.0455$): "looks close to 50/50" and "is statistically expected under 50/50" are different questions [+Edge cases] teams usually alarm on SRM at $p < 0.001$, not 0.05, because they check every experiment daily. At 0.05, one healthy test in 20 would trip the alarm (Q73-028's logic) [+Business] common real causes: a bug in the randomization code, a caching layer serving one variant more, bot traffic disproportionately hitting one arm, or a redirect that silently drops some users from one variant before they're even counted

Likely follow-ups: If you find an SRM, do you throw out the whole test? How would you debug which stage of the pipeline caused it?

Red flag: treating a skewed split as a minor curiosity rather than a signal the whole experiment's data may be unreliable.

Learn it in: Chapter 30, section 30.9 (Step 2, the sample-ratio check) and section 30.10 ("Sample-ratio mismatch: finding the cause").

Rapid-fire, 73.5

Roles: DA, DS and PA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-027	What's a novelty effect in an A/B test?	Users react to a change simply because it's <i>new</i> , not because it's actually better, and the effect fades over time	[+Business] a test that shows a strong early lift can be entirely novelty; running long enough to see the effect stabilize (or a holdout, Q73-024) separates the two	Mid · 30.10
Q73-028	Why is testing 20 different success metrics in one experiment a problem?	Multiple comparisons: testing many unrelated metrics multiplies the chance that at least one looks "significant" purely by chance	[+Validate] 20 metrics with no real effect, each tested at $\alpha = 0.05$: $P(\text{at least one false positive}) = 1 - 0.95^{20} = 0.642$; a simulation of 10,000 such experiments (seed 73) gives 0.639	Mid · 22.4

#	Question	One-line answer	Extra point	Level · learn it in
Q73-029	What's a Bonferroni correction, in one sentence?	Divide the significance threshold by the number of comparisons, so the <i>combined</i> false-positive risk across all of them stays at the original level	[+Trade-offs] simple and conservative, but it can make genuinely real effects harder to detect (lower power) when many comparisons are tested at once	Mid · 22.4
Q73-030	What's a network effect / interference problem in A/B testing?	When a treated user's behavior affects a control user's outcome (e.g., a social feature, a marketplace with shared inventory), violating the assumption that groups are independent	[+Business] standard A/B math assumes no spillover between groups; a marketplace or social feature test may need a different design (geographic or time-based splitting) to avoid contaminating the control group	Senior · 31.5

73.6 Causal inference and Simpson's paradox

Q73-031 · Construct a Simpson's paradox: show a sales rep who is better in every category yet worse overall

Level: Mid · **Roles:** DA, DS, PA, BA

Remember it as: When the group sizes are unbalanced across categories, "better in every group" and "better overall" can point in opposite directions at the same time.

Answer in one line: Simpson's paradox occurs when a trend present in every subgroup reverses when the subgroups are combined, because the subgroups are weighted very differently in size.

Verified, live. Two reps, easy and hard leads, each stored as (won, total):

```
rep_a = {"easy": (9, 10), "hard": (30, 100)}    # (won, total)
rep_b = {"easy": (80, 100), "hard": (2, 10)}
for kind in ["easy", "hard"]:
    won_a, total_a = rep_a[kind]
    won_b, total_b = rep_b[kind]
    print(f"{kind}: Rep A {won_a / total_a:.0%} ({won_a}/{total_a}), Rep B {won_b / total_b:.0%} ({won_b}/{total_b})")
```

```
easy: Rep A 90% (9/10), Rep B 80% (80/100)
hard: Rep A 30% (30/100), Rep B 20% (2/10)
```

- Each dictionary maps a lead type to a pair of numbers, (won, total). `won_a, total_a = rep_a[kind]` unpacks the pair into two names.
- `:.0%` prints a share as a whole percentage.

Now add up each rep's leads across both types:

```
won_a = rep_a["easy"][0] + rep_a["hard"][0]
total_a = rep_a["easy"][1] + rep_a["hard"][1]
won_b = rep_b["easy"][0] + rep_b["hard"][0]
total_b = rep_b["easy"][1] + rep_b["hard"][1]
print(f"Rep A overall: {won_a / total_a:.1%} ({won_a}/{total_a})")
print(f"Rep B overall: {won_b / total_b:.1%} ({won_b}/{total_b})")
```

```
Rep A overall: 35.5% (39/110)
Rep B overall: 74.5% (82/110)
```

- `rep_a["easy"][0]` is the first number of the easy pair, the leads won; `[1]` is the second, the total.

Rep A wins both categories (90% beats 80% on easy leads; 30% beats 20% on hard leads) **but Rep B's overall win rate is more than double Rep A's** (74.5% vs. 35.5%). The reversal happens because Rep A worked mostly hard leads (where everyone's rate is lower) and Rep B worked mostly easy leads (where everyone's rate is higher): the mix, not the skill, is driving the overall number.

Tier	What to say
Passes	Can define Simpson's paradox abstractly, can't construct or recognize a real example
Strong	The worked example above, correctly identifying that unequal group sizes (Rep A mostly on hard leads, Rep B mostly on easy ones) is the mechanism, not some statistical trick
Extra points	[+Validate] the checkable numbers above, both the subgroup rates and the reversed overall rates [+Business] this is exactly why comparing two sales reps', two ad campaigns', or two hospitals' raw overall success rates can be actively misleading if their underlying mix of harder and easier cases differs: always check the subgroup breakdown before ranking on an aggregate number

Likely follow-ups: How would you fix the comparison to be fair? (*Compare like-for-like within each category, or weight by a standardized mix: the same logic behind a standardized/risk-adjusted rate.*) Can you think of a real Simpson's paradox from your own work?

Red flag: not recognizing that the paradox is *always* about the mix/weighting, not about the individual category numbers being wrong.

Learn it in: Chapter 22, section 22.6 (Simpson's paradox; "Standardizing, by hand" answers the first follow-up).

Q73-032 · A strong correlation between website logins and revenue disappears almost entirely once you control for account size. What happened?

Level: Senior · **Roles:** DA, DS, PA

Remember it as: A confounder is a hidden variable pulling the strings on both sides of a correlation you're looking at.

Answer in one line: Account size is a **confounder**: bigger accounts naturally log in more *and* naturally generate more revenue, creating a strong correlation between logins and revenue that has nothing to do with logins actually driving revenue.

Verified, live. First, build 200 accounts where size drives both logins and revenue, and measure the raw correlation:

```
rng = np.random.default_rng(73)
account_size = rng.uniform(1, 10, 200)
logins = account_size * 2 + rng.normal(0, 2, 200)
revenue = account_size * 50000 + rng.normal(0, 20000, 200)

r_raw, _ = stats.pearsonr(logins, revenue)
print(f"raw correlation(logins, revenue) = {r_raw:.3f}")
```

```
raw correlation(logins, revenue) = 0.922
```

- `rng.uniform(1, 10, 200)` draws 200 account sizes, each equally likely to be anywhere from 1 to 10 (a uniform distribution, Chapter 21, section 21.5).
- `logins` is 2 per unit of size plus random noise; `revenue` is 50,000 per unit of size plus noise. `rng.normal(0, 2, 200)` is the noise: 200 values with mean 0 and standard deviation 2.
- `stats.pearsonr(logins, revenue)` returns Pearson's *r* and a *p*-value (Chapter 22, section 22.5); `r_raw, _ = ...` keeps *r* and throws the *p*-value away (`_` is the usual name for a value you don't need).

Revenue is built from size only, and logins from size only. Logins never enter the revenue formula, so any logins→revenue correlation must come through size.

Beyond the book: partial correlation. Chapter 22 doesn't teach it, and the idea fits in one sentence: partial correlation is the correlation of what's left in logins and revenue once account size is removed from both. Remove size with a straight line (section 22.10), keep what's left over (the **residuals**), and correlate those:

```
def partial_corr(x, y, z):
    slope_x, intercept_x = np.polyfit(z, x, 1)
    slope_y, intercept_y = np.polyfit(z, y, 1)
    resid_x = x - (slope_x * z + intercept_x)
    resid_y = y - (slope_y * z + intercept_y)
    r, _ = stats.pearsonr(resid_x, resid_y)
    return r

print(f"partial correlation, controlling for size = {partial_corr(logins, revenue,
    account_size):.3f}")
```

```
partial correlation, controlling for size = 0.029
```

- `np.polyfit(z, x, 1)` fits a straight line (a polynomial of degree 1) that predicts *x* from *z*, and returns its slope and intercept: the least-squares line of Chapter 22, section 22.10.
- `resid_x = x - (slope_x * z + intercept_x)` is each account's actual logins minus the logins its size predicts: the part of logins that size doesn't explain. `resid_y` does the same for revenue.
- `stats.pearsonr(resid_x, resid_y)` correlates the two leftovers.

The book's own tool gives the same verdict. A regression with both variables (Chapter 22, section 22.10, "More than one x"; Chapter 30, section 30.11) reads each coefficient "holding the others fixed":

```
import pandas as pd
import statsmodels.formula.api as smf

accounts = pd.DataFrame({"revenue": revenue, "logins": logins, "account_size": account_size})
model = smf.ols("revenue ~ logins + account_size", data=accounts).fit()
print(model.params.round(0))
print(f"p-value for logins: {model.pvalues['logins']:.2f}")
```

```
Intercept      2099.0
logins          248.0
account_size   49596.0
dtype: float64
p-value for logins: 0.68
```

- `pd.DataFrame({...})` puts the three arrays into one table, one column each, because `smf.ols` reads its columns by name.
- `smf.ols("revenue ~ logins + account_size", data=accounts).fit()` fits revenue on logins and account size together, exactly as in Chapter 30, section 30.11.
- `model.params` holds the intercept and the two coefficients; `.round(0)` rounds them to whole numbers. `model.pvalues['logins']` is the p-value for the logins coefficient.

The raw correlation of **0.92** (very strong) collapses to essentially **zero (0.03)** once account size is accounted for, in data that was built with *no direct causal link* between logins and revenue at all, only a shared cause. The regression says the same: with size held fixed, one extra login goes with only 248 more revenue, and that coefficient is nowhere near significant ($p = 0.68$), while size carries about 50,000 per unit (49,596), the number the data was built with.

Tier	What to say
Passes	"Correlation isn't causation" (true, doesn't diagnose <i>this specific</i> case)
Strong	Correctly names account size as a likely confounder and proposes checking a partial correlation or controlling for it in a regression
Extra points	[+Validate] the numbers above: a constructed example with zero true login→revenue effect, where the naive correlation would badly mislead anyone who stopped there [+Business] this is precisely the trap in a claim like "customers who log in more spend more, so let's push login frequency": the real driver (account size) isn't something a login-nudge campaign can change at all, and the campaign would likely show no revenue impact despite the strong raw correlation

Likely follow-ups: How would you actually test whether logins *cause* more revenue, not just correlate with it? What's the difference between controlling for a confounder and running a randomized experiment?

Red flag: citing "correlation isn't causation" as a complete answer without identifying what the actual confounder is or how to check for one.

Learn it in: Chapter 22, section 22.5 (confounders) and section 22.10 ("More than one x"); Chapter 30, section 30.11 ("Several variables at once"). Partial correlation is Beyond the book, explained above.

Rapid-fire, 73.6

Roles: DA, DS, PA and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q73-033	What's selection bias, with one business example?	The sample you're analyzing isn't representative of the population you're trying to draw conclusions about	[+Business] analyzing only customers who responded to a survey, and concluding "customers are satisfied," ignores the (likely more dissatisfied) customers who didn't bother responding at all	Fresher · 22.7
Q73-034	What's a natural experiment?	A real-world situation where something close to random assignment happens without anyone deliberately designing an experiment (a policy change hitting some regions and not others, or a threshold in a business rule)	[+Business] useful exactly when a true randomized test isn't ethical or possible, though weaker than a real randomized experiment since the "randomness" is only approximate	Mid · 31.3, 31.7
Q73-035	Why is "the data speaks for itself" a red flag phrase in a causal claim?	Data alone never distinguishes correlation from causation; that distinction always requires an assumption about the process that generated the data (randomization, a natural experiment, or a stated causal model)	[+Business] anyone claiming pure data-driven causal certainty with no discussion of confounders or study design is skipping the hardest, most important part of the analysis	Senior · 22.5, 31.9

73.7 Live-coding and live-analysis walk-throughs

Q73-036 · Walk-through: is this A/B test result real, worth shipping, and safe to trust?

Level: Senior · **Roles:** DA, DS, PA

What they're really testing: whether you run the full checklist (significance, effect size, SRM, power, practical relevance) rather than stopping at the first " $p < 0.05$."

The setup, talked through live: "I've got control at 480/5,000 conversions and treatment at 545/5,000. Before I say anything about shipping this, I want to check four things in order: is the split itself sane (no SRM), is the difference statistically significant, how big is the effect in business terms, and was the test big enough to trust a result this size."

Verified, live. First, the two-proportion z-test:

```
count = np.array([545, 480])    # conversions: treatment, control
nobs = np.array([5000, 5000])  # visitors: treatment, control
z_stat, p_value = sms.proportions_ztest(count, nobs)
print(f"control: {480 / 5000:.2%}")
print(f"treatment: {545 / 5000:.2%}")
print(f"z = {z_stat:.3f}, p = {p_value:.4f}")
```



```
control: 9.60%
treatment: 10.90%
z = 2.143, p = 0.0321
```

- `sms.proportions_ztest(count, nobs)` is the two-proportion z-test from Chapter 22, section 22.2: `count` holds the conversions and `nobs` the visitors, in the same order.
- The order matters for the sign. Treatment comes first, so a positive z means treatment converted more; swap the two and z becomes -2.143 with the same p-value.

Next, how big the lift is, with its uncertainty:

```
low, high = sms.confint_proportions_2indep(545, 5000, 480, 5000, method="wald")
print(f"lift: {(545 / 5000 - 480 / 5000) * 100:.1f} points, 95% interval {low * 100:+.2f} to {high * 100:+.2f} points")
```

```
lift: 1.3 points, 95% interval +0.11 to +2.49 points
```

- `confint_proportions_2indep(545, 5000, 480, 5000, method="wald")` gives the 95% interval for the first rate minus the second, with the textbook formula, as in Chapter 30, section 30.5. Multiplying by 100 turns shares into percentage points; `:.2f` prints the sign.

Last, was the test big enough? Power for the lift we saw, and the sample that 80% power would have needed:

```
observed_h = sms.proportion_effectsize(0.109, 0.096)
power = sms.NormalIndPower().power(observed_h, nobs1=5000, alpha=0.05, ratio=1)
needed = sms.NormalIndPower().solve_power(observed_h, power=0.8, alpha=0.05, ratio=1)
print(f"power at 5,000 per arm for a 1.3-point lift: {power:.2f}")
print(f"per arm for 80% power: {np.ceil(needed):,.0f}")
```

```
power at 5,000 per arm for a 1.3-point lift: 0.57
per arm for 80% power: 8,537
```

- `.power(observed_h, nobs1=5000, ...)` is the same calculator as Q73-020, run the other way: given the sample size, it returns the power.
- `.solve_power(...)` is Q73-020's call, for the lift this test actually saw.

Talked through, live: "p = 0.032, under the usual 0.05 threshold, so it's statistically significant. The lift is 1.3 percentage points, about a 13.5% relative improvement, which sounds meaningful, but I'd want to know the actual revenue or cost tied to a conversion before calling it worth shipping: statistical significance isn't the same question as 'is this worth the engineering cost to keep.' The split is exactly 5,000 vs. 5,000, so there's no SRM concern. And this is where it gets interesting: for a 1.3-point lift on 9.6%, 80% power needs about 8,500 per arm, and even the 1.5-point MDE of Q73-020 needs 6,466. We have 5,000, so power was only about 57% for the lift we saw. Either the test was stopped early, or it was never sized. A 'significant' result from an underpowered test is more likely to overstate the true lift, so I'd report the 95% interval for the difference, +0.11 to +2.49 points, and recommend running to the planned size before shipping."

Why "overstate"? Beyond the book, in one paragraph: when a test is too small, only the runs where chance pushed the lift *up* clear the significance bar. So among the significant results of small tests, the lift is, on average, larger than the truth, and it tends to shrink when the test is rerun at full size. This is sometimes called the **winner's curse**.

Extra-points moves demonstrated: **[+Signpost]** named a four-step checklist up front, then ran it in order. **[+Validate]** the real z-test, the interval, and the power (0.57 at 5,000 per arm), not an assumed conclusion. **[+Business]** explicitly separated "statistically significant" from "worth shipping," which are different questions with different owners. **[+Limits]** said plainly what an underpowered test can't show, and what would settle it.

Likely follow-ups: What if the sample sizes were very different between arms? How would you incorporate a cost estimate into the shipping decision?

Red flag: treating a single p-value as a complete answer to "should we ship this."

Learn it in: Chapter 30, section 30.9 (Riverstone's website test, end to end), section 30.7 (power and sample size), and section 30.5 (the interval for two proportions).

Q73-037 · Walk-through: a stakeholder shows you a chi-square test result and asks what it means

Level: Mid · **Roles:** DA, DS, PA, BA

What they're really testing: whether a chi-square test's actual question (independence between two categorical variables) is understood, not just its p-value.

Talked through live: "A chi-square test of independence checks whether two categorical variables are related at all, not the direction or size of the relationship, just whether the pattern of counts differs from what independence would predict."

Verified, live, on a segment-by-conversion table:

```
observed = np.array([[120, 80], [90, 110]]) # rows: segment; columns: converted, not converted
chi2, p, dof, expected = stats.chi2_contingency(observed, correction=False)
print(f"chi2 = {chi2:.3f}, p = {p:.4f}, dof = {dof}")
print("expected counts under independence:", expected.tolist())
```

```
chi2 = 9.023, p = 0.0027, dof = 1
expected counts under independence: [[105.0, 95.0], [105.0, 95.0]]
```

- `stats.chi2_contingency(observed, correction=False)` is the test from Chapter 22, section 22.2. It returns four things, which `chi2, p, dof, expected = ...` unpacks: the statistic, the p-value, the degrees of freedom, and the table of counts you'd expect if segment and conversion were unrelated.
- `correction=False` turns off **Yates' continuity correction**, a small, conservative adjustment SciPy applies to 2×2 tables by default. Chapter 22 turned it off too, so the result matches the hand sum and the two-proportion z-test. **What happens if you change it:** drop `correction=False`, and this table gives 8.431 ($p = 0.0037$) instead. In an interview, say which one you used.
- `expected.tolist()` prints the expected table as a plain list of rows.

Talked through, live: " $p = 0.0027$ says the segments and conversion aren't independent, real evidence of a relationship. But chi-square alone doesn't say *which* segment converts better or by how much, only that a relationship exists. I'd follow it with the actual conversion rates per segment to answer the business question, and I'd check the expected-counts table, shown here, to make sure no cell had too few expected observations for the test to be reliable, a common chi-square validity issue with small samples."

Extra-points moves demonstrated: [+Limits] said what chi-square can't tell you: "a relationship exists" (what it tests) is not "here's the relationship" (a separate follow-up). [+Edge cases] flagged the small-expected-count validity concern unprompted.

Likely follow-ups: What test would you use for two continuous variables instead? What's Cramér's V, and when would you report it alongside the p-value?

Red flag: interpreting a significant chi-square result as telling you the direction or size of an effect, which it doesn't.

Learn it in: Chapter 22, section 22.2 ("The chi-square test, by hand first"); Chapter 30, section 30.5 (chi-square and Cramér's V).

APPLY

Common mistakes

Mistake	Symptom	Fix
Reading a "99% sensitive" test as "99% chance you have it"	Wildly overestimating a positive result's reliability on a rare condition	Always ask for the base rate; run Bayes' theorem (Q73-005)
Defining a p-value as "probability the null is true"	Confidently wrong explanations to stakeholders	It's the probability of the <i>data</i> , assuming the null, never the reverse
Treating "not significant" as "no effect"	Shipping decisions based on an underpowered test's false negative	Check power before concluding there's no effect (Q73-015)
Skipping a sample-size calculation before running a test	A test that ends inconclusive, or runs far longer than needed	Compute required sample size from baseline, MDE, alpha, and power (Q73-020)
Checking significance daily and stopping at the first $p < 0.05$	A real, inflated false-positive rate far above the stated 5%	Pre-commit to a sample size and check date, or use a sequential method (Q73-025)
Ignoring a skewed traffic split as "probably fine"	Trusting results from a broken randomization pipeline	Run a chi-square SRM check before trusting anything else (Q73-026)
Ranking groups on an aggregate rate with very different underlying mixes	A real Simpson's paradox reversal going unnoticed	Always check subgroup rates before trusting an aggregate comparison (Q73-031)
Citing a strong correlation as if it were causal	Acting on a relationship driven entirely by a confounder	Look for (and control for) a plausible confounder before any causal claim (Q73-032)

In the real world: the A/B test that was actually broken plumbing

Farah, running a pricing experiment at a mid-size company, sees a strong, statistically significant lift in the treatment group after ten days and is ready to recommend shipping it. Before doing so, she runs the sample-ratio check (Q73-026), almost as a formality: and gets a real SRM: 5,340 users in control against 4,660 in treatment, a split that shouldn't happen by chance at that scale.

She doesn't ship the pricing change. She traces the mismatch and finds that the treatment variant's page loaded slightly slower, and a fraction of mobile users on slower connections were timing out before the assignment even completed, silently dropping out of the treatment group specifically. Those dropped users, it turns out, were disproportionately price-sensitive mobile shoppers, exactly the segment least likely to convert. The "lift" wasn't from the new pricing at all; it was from quietly excluding the users least likely to buy.

The fix took two days: a loading-performance bug, unrelated to pricing entirely. The re-run test, with the SRM resolved, showed no significant pricing effect at all. Farah's manager's reaction was the right one: relief that the SRM check happened before the change shipped, not after, when the "successful" pricing test would have been cited as evidence for a change that did nothing, while the real bug (a broken experience for a whole segment of mobile users) went unnoticed and unfixed.

Project

Goal: run, not just read, this chapter's core demonstrations on your own numbers.

Tools you'll need

Python with **SciPy** (`scipy.stats`) and **statsmodels** (`statsmodels.stats.api`), both free and both already in your Chapter 17 environment (installed in Chapter 21, section 21.5 and Chapter 22, section 22.2), used throughout this chapter for every simulation, test statistic, and sample-size calculation. **R** offers equivalent functionality (`prop.test`, `power.t.test`, `chisq.test`) and is common in some analytics teams; the underlying statistics are identical regardless of language. Everything in this chapter ran on Python 3.11.15 with NumPy 2.4.6, SciPy 1.17.1, statsmodels 0.15.0 and pandas 3.0.6 (the book recommends Python 3.14, Chapter 17, section 17.0); a recent SciPy and statsmodels should give the same numbers to three decimals.

1. Compute Bayes' theorem for a real base-rate scenario from your own work (a fraud flag, a churn flag, a quality-control test) and see how far the "accuracy" number is from the true positive-predictive value.
2. Run a real sample-size calculation for an A/B test you'd actually want to run, using your own baseline rate and a business-justified MDE.
3. Simulate the peeking problem yourself (or reason through why it happens) on a metric relevant to your own work.
4. Find (or construct) a Simpson's paradox in your own data: two groups, a category where the aggregate ranking reverses the subgroup rankings.

5. Take one correlation you've observed in your own work and name a plausible confounder, then say how you'd check whether it explains the relationship.

REVIEW

Key terms

Bayes' theorem · base rate · sensitivity · false-positive rate · natural frequencies · conditional probability · Monty Hall problem · birthday problem · independent vs. mutually exclusive events · expected value · Binomial distribution · Poisson distribution · Central Limit Theorem · variance · standard deviation · Normal / log-normal distribution · heavy-tailed distribution · p-value · null hypothesis · Type I error · Type II error · statistical power · t-test vs. z-test · confidence interval · minimum detectable effect (MDE) · Cohen's h · randomization · one-tailed vs. two-tailed test · holdout group · peeking (repeated significance testing) · Sample Ratio Mismatch (SRM) · novelty effect · multiple comparisons · Bonferroni correction · network effect (interference) · Simpson's paradox · confounder · partial correlation · residual · selection bias · natural experiment · Yates' continuity correction · winner's curse

Final-week revision list

Q73-005, Q73-006, Q73-007, Q73-008, Q73-009, Q73-014, Q73-015, Q73-016, Q73-017, Q73-019, Q73-020, Q73-021, Q73-025, Q73-026, Q73-028, Q73-031, Q73-032, Q73-036, Q73-037.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, already covered the mechanics of building the aggregated tables (retention counts, group-by rates) that feed many of this chapter's tests.
- **Chapter 74, Machine Learning Question Bank**, builds on this chapter's hypothesis testing and base-rate reasoning (Q73-005) for model evaluation.
- **Chapters 21 and 22** (probability, distributions, the two famous puzzles in section 21.7, and tests), **Chapter 30** (inference and experiments) and **Chapter 31** (causal inference without experiments) teach what this bank draws on; this chapter tests it, it doesn't re-teach it. The few ideas marked Beyond the book (log-normal data, holdout groups, partial correlation, the winner's curse) are explained where they appear.

Chapter 74. Machine Learning Question Bank

Chapter at a glance

You will learn to: answer the ML questions that come up across screening calls, live-coding rounds, and case interviews for Data Scientist, ML Engineer, and analytics-adjacent roles · explain bias and variance, not just define them · spot data leakage before a model's score fools you · read a confusion matrix, an ROC curve, and a calibration plot the way an interviewer actually wants · debug a model that "worked in training and broke in production."

Before you start: Part 4 (Chapters 35–43), which teaches every idea in this bank; Chapter 53, section 53.3 (early stopping) and Chapter 56 (monitoring, drift, training-serving skew) for section 74.6; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 6–8 hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and about an hour to run the four code demos yourself. Plus 1 hour for the final-week list.

How this chapter is built. Same format as every question bank in Part 8 (Chapters 70–82). Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, tagged the same way: [**+Clarify**], [**+Edge cases**], [**+Validate**] and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it (a bare number such as 37.2 means that section). Every **Learn it in** pointer names the section of Part 4 (or Chapter 53 or 56) that teaches the idea, and the numbers quoted are the ones you produced there. The four **Run it yourself** demos are complete cells: copy one into a fresh notebook and it prints what's shown (scikit-learn 1.9.1, the version installed in Chapter 35, section 35.9). Ideas no earlier chapter teaches are marked **Beyond the book** and carry their own short explanation and example.

Levels and roles. Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **DS** data scientist · **MLE** machine learning engineer · **DA** data analyst (and other analytics roles that work next to a model). Within each section the core questions run from easier to harder, and the sections do too: the basic-but-tricky questions come first.

A note on scope. This is an *interview* bank, testing whether you can explain and reason about ML concepts under time pressure, not a re-teaching of the algorithms themselves (that's Chapters 35–43 in full). Read this chapter after those, not instead of them.

In this chapter

LEARN	74.1 Basic-but-tricky ML questions	205
	74.2 Bias, variance, and the shape of a good model	207
	74.3 Metrics, calibration, and thresholds	211
	74.4 Data leakage	213
	74.5 Feature engineering	217
	74.6 Model debugging	219
	74.7 ML case studies and live-coding walk-throughs	220
APPLY	Common mistakes · In the real world · Project	222
REVIEW	Key terms · Final-week revision list	224
NEXT	Where this leads	224

LEARN

74.1 Basic-but-tricky ML questions

Q74-007 · Is a model with 95% accuracy always a good model?

Level: Fresher · **Roles:** DS, DA, MLE

Remember it as: Always ask the base rate before believing an accuracy number. Predicting "no" for everyone on a 5%-positive problem is already 95% accurate.

Answer in one line: No: on an imbalanced problem, a model that predicts the majority class for every single row can achieve very high accuracy while catching zero of the minority class it was actually built to find.

From Chapter 37, section 37.0: 484 of Riverstone's 5,000 accounts churned in 2025, a churn rate of 9.7%. So predicting "no churn" for every account is already **90.3%** accurate ($4,516 \div 5,000$).

Tier	What to say
Passes	"Accuracy isn't everything" (correct, no concrete demonstration)
Strong	Computes or cites the actual base-rate accuracy, showing how little a headline accuracy number proves on its own
Extra points	[+Clarify] "What's the positive rate?" before reacting to any accuracy figure [+Business] 90.3% is the real bar Riverstone's own churn model had to beat, and it's why Chapter 39 uses precision, recall, and PR-AUC instead of accuracy on any imbalanced problem

Likely follow-ups: What metrics would you ask for instead? How imbalanced does a problem need to be before accuracy becomes actively misleading?

Red flag: treating a single accuracy number as sufficient evidence of a good model with no mention of the base rate.

Learn it in: Chapter 37, section 37.0 (the 9.7% churn rate) and Chapter 39, sections 39.1–39.2.

Q74-008 · Can a model have a good ROC-AUC and still be a bad choice for the business?

Level: Mid · Roles: DS, DA

Remember it as: ROC-AUC measures whether a model ranks well. It says nothing about whether the threshold you'll actually use makes money.

Answer in one line: Yes: ROC-AUC measures a model's ability to rank positive cases above negative ones across every possible threshold, but says nothing about calibration, about performance at the one threshold you'll actually deploy, or about whether the business cost of a false positive versus a false negative makes that threshold worthwhile at all.

Tier	What to say
Passes	"AUC only tells you ranking, not the actual threshold" (correct, no elaboration)
Strong	Names the specific gaps: calibration (Q74-014), threshold-specific cost trade-offs, and PR-AUC's sensitivity to the base rate that plain ROC-AUC hides
Extra points	[+Business] Riverstone's lead model has an ROC-AUC of 0.823, which sounds strong, and its business value only became visible once a cost-based threshold was applied: working all 2,225 validation leads makes ₹10.8 lakh, working the 627 leads above the best threshold makes ₹24.5 lakh [+Limits] even that threshold gave way to capacity: the team can work about 504 leads in six months, so the real rule is "the top 504"

Likely follow-ups: What would you look at alongside AUC to judge business readiness? When is a lower-AUC but better-calibrated model the right choice?

Red flag: treating AUC as a complete summary of "is this model good."

Learn it in: Chapter 39, sections 39.2 (ROC and PR curves) and 39.5 (the profit curve and capacity).

Rapid-fire, 74.1

Roles: DS and DA for every row; MLE for Q74-011 and Q74-012.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-009	Does a higher R^2 always mean a better regression model?	No: <i>training</i> R^2 never goes down when you add a feature, even a useless one. Judge with held-out R^2 or adjusted R^2 (below), which charges for each extra feature	[+Validate] Chapter 37 reports <i>test</i> R^2 , which can fall: 0.9546 with one feature, 0.9593 with many	Mid · 37.1; adjusted R^2 is Beyond the book
Q74-010	Is a model with zero training error a good sign?	Usually a red flag: it often means the model has memorized the training set rather than learned a pattern that generalizes	[+Evidence] an unlimited decision tree on the churn accounts: training AUC 1.000, validation 0.557	Fresher · 37.6
Q74-011	Do you always need to scale features before training a model?	Depends on the algorithm: distance- and penalty-based methods (k-NN, k-means, SVM, regularized linear models) need it; tree-based methods (random forest, gradient boosting) generally don't	[+Evidence] k-NN on the churn accounts: AUC 0.765 scaled, 0.633 unscaled, because revenue in rupees swamps every distance	Fresher · 37.4

#	Question	One-line answer	Extra point	Level · learn it in
Q74-012	Is a random forest always better than a single decision tree?	No, but it's very often more stable: a single tree is high-variance and sensitive to its exact training sample; averaging many trees reduces that variance, at some cost in interpretability	[+Trade-offs] you give up a tree you can print and read (section 37.6's depth-2 tree) for a score you can trust more	Fresher · 37.7

Beyond the book: adjusted R^2 . Chapter 37 judges regression on held-out rows, which is the better habit. Some interviewers still ask for **adjusted R^2** , which penalizes R^2 for the number of features p given the number of rows n :

$$\text{adjusted } R^2 = 1 - (1 - R^2) \times (n - 1) \div (n - p - 1)$$

A model with R^2 0.90 on $n = 50$ rows using $p = 10$ features: $1 - 0.10 \times 49 \div 39 = 1 - 0.126 = \mathbf{0.874}$. Add a useless eleventh feature that nudges R^2 to 0.901 and adjusted R^2 *falls*, to $1 - 0.099 \times 49 \div 38 = \mathbf{0.872}$: the penalty for the extra feature outweighs the gain.

74.2 Bias, variance, and the shape of a good model

Q74-001 · Explain the bias-variance trade-off, and show it on a real model

Level: Fresher · **Roles:** DS, MLE

Remember it as: Too simple, and the model can't learn the pattern even from the training data (high bias). Too complex, and it memorizes the training data's noise instead of the real pattern (high variance).

Answer in one line: **Bias** is error from a model too simple to capture the real pattern, showing up as poor performance on *both* training and held-out data; **variance** is error from a model too sensitive to the specific training sample, showing up as a large gap between strong training performance and weak held-out performance.

From Chapter 37, section 37.6: decision trees of growing depth on Riverstone's churn accounts (3,000 training, 1,000 validation):

<code>max_depth</code>	Train AUC	Validation AUC	Leaves
1	0.696	0.651	2
2	0.759	0.708	4
3	0.791	0.752	8
4	0.829	0.748	16
5	0.865	0.750	29
6	0.886	0.733	48
8	0.934	0.672	97

<code>max_depth</code>	Train AUC	Validation AUC	Leaves
10	0.969	0.609	161
14	0.994	0.543	230
None (no limit)	1.000	0.557	277

Depth 1 is high bias: both scores are low and close together. The unlimited tree is high variance: perfect on training, barely better than a coin on validation. Depth 3 is the sweet spot on this data, with 4 and 5 close behind.

Tier	What to say
Passes	Defines both terms correctly, can't demonstrate the trade-off with numbers
Strong	The sweep above: depth 1 underfits (both scores low and close), unlimited depth overfits (perfect training, a large gap to validation), and the best validation score sits in between, at depth 3
Extra points	[+Evidence] these are measured numbers from your own work, not a textbook sketch [+Signpost] name the second tool: section 37.10's learning curves show logistic regression limited by bias (training and CV scores meet at about 0.81 and stay there) and boosting by variance (training 0.945 against CV 0.836, still closing as rows are added) [+Business] this is why "just add more model complexity" isn't a free lunch

Likely follow-ups: How would you diagnose which one your model has, without a controlled sweep like this? What's the relationship between bias-variance and a learning curve?

Red flag: defining the terms correctly but being unable to say which symptom (both scores low vs. a train-validation gap) points to which problem.

Learn it in: Chapter 37, sections 37.0 ("Two ways to be wrong"), 37.6 (depth and overfitting), and 37.10 (bias, variance, and learning curves).

Q74-002 · Why can a model reach perfect training AUC on pure noise, with enough features and too few samples?

Level: Mid · **Roles:** DS, MLE

Remember it as: Give a model enough dimensions relative to how much data it has, and it can always find some combination that happens to separate the training rows perfectly, real pattern or not.

Answer in one line: With enough features relative to the sample size, a flexible model can find a combination of features that perfectly separates the training labels by chance alone, even when the features carry zero real signal: the **curse of dimensionality** meeting overfitting.

Run it yourself. 200 rows of pure noise, with a target that has no relationship to any feature. The function below builds one such dataset, splits it 70/30, fits a logistic regression with almost no penalty, and returns the training and test AUC:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

def noise_run(n_features, seed):
    rng = np.random.default_rng(seed)
    X = rng.normal(0, 1, (200, n_features))
    y = rng.integers(0, 2, 200)
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=seed)
    model = LogisticRegression(C=1_000_000, max_iter=10_000).fit(X_tr, y_tr)
    train_auc = roc_auc_score(y_tr, model.predict_proba(X_tr)[: , 1])
    test_auc = roc_auc_score(y_te, model.predict_proba(X_te)[: , 1])
    return train_auc, test_auc

for n_features in [5, 50, 200]:
    train_auc, test_auc = noise_run(n_features, seed=0)
    print(f"{n_features:>3} features: train AUC {train_auc:.3f}    test AUC {test_auc:.3f}")
```

```
5 features: train AUC 0.620    test AUC 0.576
50 features: train AUC 0.894    test AUC 0.498
200 features: train AUC 1.000    test AUC 0.578
```

How it works:

- `np.random.default_rng(seed)` is a seeded random generator (Chapter 21), so every run with the same seed draws the same numbers. `rng.normal(0, 1, (200, n_features))` fills a 200-row table with standard normal noise; `rng.integers(0, 2, 200)` draws 200 random 0s and 1s, the target. Nothing links them.
- `train_test_split(..., test_size=0.3, random_state=seed)` keeps 140 rows for training and 60 for testing (Chapter 36, section 36.3).
- `LogisticRegression(C=1_000_000, ...)`: `C` is the inverse of the penalty strength (Chapter 37, section 37.3), so a million means almost no regularization. `max_iter=10_000` gives the optimizer room to finish.
- `roc_auc_score` is Chapter 36's AUC; `[: , 1]` keeps the probability of class 1.

With 200 noise features and only 140 training rows, the model scores a **perfect 1.000** on the rows it learned from. But one test set of 60 rows is small, and a single test AUC swings a lot by chance: 0.576 and 0.578 aren't signs of learning. Before you run the next cell, predict: averaged over 20 different random datasets, where will the test AUC land?

```

for n_features in [5, 50, 200]:
    runs = np.array([noise_run(n_features, seed) for seed in range(20)])
    print(
        f"{n_features:>3} features: mean train AUC {runs[:, 0].mean():.3f}    "
        f"mean test AUC {runs[:, 1].mean():.3f} "
        f"(range {runs[:, 1].min():.2f} to {runs[:, 1].max():.2f})"
    )

```

```

5 features: mean train AUC 0.595    mean test AUC 0.524 (range 0.32 to 0.69)
50 features: mean train AUC 0.860    mean test AUC 0.519 (range 0.37 to 0.69)
200 features: mean train AUC 1.000    mean test AUC 0.499 (range 0.32 to 0.72)

```

How it works: the list comprehension runs `noise_run` for seeds 0 to 19, and `np.array` stacks the 20 (train, test) pairs into a 20-by-2 array. `runs[:, 0]` is the training column and `runs[:, 1]` the test column; `.mean()`, `.min()` and `.max()` summarize each.

Reading it. The training AUC climbs with the number of features, from 0.595 to a perfect 1.000 at every one of the 20 seeds with 200 features. The test AUC stays at about 0.5, a coin toss, whatever the number of features, and any single run can land anywhere from about 0.3 to 0.7. More features bought a perfect training score on data with no signal at all.

Tier	What to say
Passes	"More features can cause overfitting" (true, no demonstration of the mechanism or size of the effect)
Strong	Explains the mechanism: with enough dimensions, there's almost always <i>some</i> direction in feature space that happens to line up with the training labels by chance
Extra points	[+Validate] perfect training AUC on data built to have zero signal, and a test AUC that averages 0.5 [+Edge cases] a single test AUC on 60 rows can read 0.58 or 0.32 by luck; average over several splits (or cross-validate) before calling it a trend [+Business] a model with hundreds of engineered features and only a few thousand rows needs regularization and honest held-out validation, not a high training score, before anyone trusts it

Likely follow-ups: How does regularization (L1/L2) address this specifically? What sample-to-feature ratio would make you nervous?

Red flag: treating a high training score alone as evidence a model is good.

Learn it in: Chapter 37, sections 37.2 (ridge, lasso, and elastic net) and 37.10; Chapter 36, section 36.5 (cross-validation).

Rapid-fire, 74.2

Roles: DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-003	What's regularization, in one sentence?	A penalty added to the loss that discourages large model weights, trading a little training fit for better generalization	[+Signpost] name the three kinds (ridge, lasso, elastic net) and the one setting that controls the strength (α)	Fresher · 37.2

#	Question	One-line answer	Extra point	Level · learn it in
Q74-004	L1 vs. L2 regularization: what's the practical difference?	L1 (lasso) can shrink coefficients exactly to zero, performing feature selection; L2 (ridge) shrinks them toward zero but rarely to exactly zero	[+Evidence] Chapter 37's lasso path (Figure 37.2): 14 features kept at $\alpha = 0.001$, 5 at 0.01, 1 at 0.2, with almost the same accuracy at 5 as at 14	Fresher · 37.2
Q74-005	What's a learning curve, and what does a persistent gap between train and validation lines tell you?	A plot of training and validation score against training-set size; a large gap signals high variance, and if the validation line is still rising, more data should help	[+Edge cases] two lines that meet early and stay flat mean high bias: more data won't help (logistic regression on the churn accounts, gap 0.013)	Mid · 37.10
Q74-006	Does adding more training data fix high bias?	No: high bias means the model itself is too simple for the pattern; more data of the same kind won't change that, a more expressive model or better features will	[+Evidence] four engineered features lifted logistic regression's cross-validated AUC on the churn accounts from 0.807 to 0.849; more rows hadn't moved it	Mid · 37.10

74.3 Metrics, calibration, and thresholds

Q74-013 · Build a confusion matrix by hand and derive precision, recall, and F1 from it

Level: Fresher · **Roles:** DS, DA, MLE

Remember it as: Precision reads across the "predicted positive" row. Recall reads down the "actually positive" column. They answer different questions and usually trade against each other.

Answer in one line: Given TP, FP, FN, TN: **precision** = $TP/(TP+FP)$ (of what you flagged, how much was right), **recall** = $TP/(TP+FN)$ (of what was actually positive, how much did you catch), **F1** = the harmonic mean of the two.

From Chapter 39, section 39.1: Riverstone's lead model on 2,225 validation leads, at a threshold of 0.1:

```
TP=100  FP=386  FN=46  TN=1693
precision = 100/486 = 0.206
recall    = 100/146 = 0.685
F1        = 2 × 0.206 × 0.685 / (0.206 + 0.685) = 0.317   (0.316 unrounded)
```

Tier	What to say
Passes	States the formulas correctly, makes an arithmetic error applying them
Strong	The correct worked calculation above, matching a real model's real confusion matrix
Extra points	[+Edge cases] layouts differ: Chapter 39's table puts predictions on the rows, but scikit-learn's <code>confusion_matrix</code> puts the <i>actual</i> classes on the rows, so there precision reads <i>down</i> the predicted-positive column. Always say which axis is which before reading one [+Business] at this threshold only 1 in 5 flagged leads is won (20.6% precision), but the model catches over two-thirds of all wins (68.5% recall); whether that's right depends on what a wasted call costs against a missed win, which is Chapter 39's cost-based threshold

Likely follow-ups: What happens to precision and recall as you raise the threshold? Why can't you maximize both at once, in general?

Red flag: computing precision and recall with the numerator and denominator swapped, a very common slip under time pressure.

Learn it in: Chapter 39, section 39.1.

Q74-014 · What does it mean for a model's probabilities to be "well calibrated," and why can a model with good ranking still be poorly calibrated?

Level: Mid · **Roles:** DS, MLE

Remember it as: Ranking asks "did the model put the right cases higher than the wrong ones?"

Calibration asks "when it says 70%, does that group of cases actually convert 70% of the time?" Different questions, no guaranteed relationship.

Answer in one line: A model is well calibrated if, among all the cases it gives a certain probability (say, 70%), roughly that share turn out positive; a model can rank cases well (a good AUC) while being badly calibrated, if its probabilities are systematically too high or too low (Naive Bayes says 94% where the truth is 27%).

From Chapter 39, section 39.4, two models on the same 2,225 validation leads, where 6.6% were won:

Model	ROC-AUC	Mean predicted probability	Log loss
Logistic regression	0.823	6.9%	0.1948
Naive Bayes	0.806	18.0%	0.4830

Naive Bayes ranks reasonably well (0.806 AUC) while being **dramatically overconfident**: its top tenth of leads is given a 94% chance of being won, and only 27% of them are.

Tier	What to say
Passes	"Calibration means the probabilities are accurate" (vague, no example of the ranking/calibration gap)
Strong	The real example above: a model can have a good AUC and still be badly, provably miscalibrated
Extra points	[+Business] a rep told "94% chance" who wins one in four stops trusting the model entirely [+Evidence] isotonic calibration fixed it: log loss from 0.483 to 0.202, with the ranking almost unchanged (ROC-AUC 0.806 to 0.802)

Likely follow-ups: How do you check calibration in practice? What's the difference between Platt scaling and isotonic regression?

Red flag: assuming a high AUC implies trustworthy probabilities.

Learn it in: Chapter 39, section 39.4.

Rapid-fire, 74.3

Roles: DS and DA for every row; MLE for Q74-015 and Q74-018.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-015	ROC-AUC vs. PR-AUC: when does the choice matter most?	They tell a similar story on balanced data; on imbalanced data, PR-AUC reflects what users actually experience (precision) far better than ROC-AUC, which stays high	[+Evidence] the same lead model scores ROC-AUC 0.823 and PR-AUC 0.302; a random model would score 0.5 and 0.066 (the base rate)	Mid · 39.2
Q74-016	What's the harm in always using 0.5 as a classification threshold?	It's an arbitrary default with no link to the cost of a false positive vs. a false negative. On Riverstone's lead data it flags only 7 of 2,225 leads, catching 4 of 146 wins; its 93.5% accuracy is no better than predicting "no" for everyone (93.4%)	[+Business] set the threshold from the costs (break-even at 0.05 for the leads), then cap it by team capacity	Fresher · 39.1, 39.5
Q74-017	MAE vs. RMSE: what's the difference, and when do they disagree the most?	Mean absolute error weights every error equally; root mean squared error punishes large errors far more (it squares them), so they diverge most when a few large errors exist	[+Evidence] Riverstone's revenue model: MAE ₹92,933, RMSE ₹2,32,567, because one account is off by about ₹37 lakh	Fresher · 39.3
Q74-018	What's the difference between macro and weighted F1 for a multi-class problem?	Macro averages each class's F1 equally regardless of class size; weighted averages them by each class's support (number of examples), so a poor score on a rare class matters less in the weighted version	[+Business] per-class F1 of 0.9, 0.8 and 0.2 with 800, 150 and 50 examples: macro 0.633, weighted 0.850. Report macro (or the per-class table) when the rare class is the one you care about	Mid · 39.1 ("More than two classes")

74.4 Data leakage

Q74-019 · Demonstrate target leakage: build a model with and without a leaked feature, and show the difference

Level: Mid · **Roles:** DS, MLE

Remember it as: *If a feature couldn't have existed at the moment you're predicting, it doesn't belong in the model, however good it makes the score look.*

Answer in one line: **Target leakage** happens when a feature is influenced by, or nearly a copy of, the outcome you're trying to predict, giving the model information it would never have at prediction time and inflating its score in a way that won't hold up in production.

Run it yourself. First, a dataset with one honest feature, `signal`, that drives the outcome, and one leaked feature: the outcome itself plus a little noise, like a status field that's only set *after* the outcome is known.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(74)
n = 2_000
signal = rng.normal(0, 1, n)
y = (signal + rng.normal(0, 0.5, n) > 1).astype(int)
leaked = y + rng.normal(0, 0.1, n)
X = pd.DataFrame({"signal": signal, "leaked": leaked})
print(f"{n:,} rows, {y.mean():.1%} positive")
```

```
2,000 rows, 19.6% positive
```

How it works:

- `rng = np.random.default_rng(74)` seeds the generator, so you get exactly these rows.
- `signal` is 2,000 draws of standard normal noise. The outcome `y` is 1 when `signal` plus some extra noise (`rng.normal(0, 0.5, n)`) is above 1, so `signal` predicts `y` well but not perfectly. `.astype(int)` turns True/False into 1/0.
- `leaked` is `y` plus a tiny bit of noise (standard deviation 0.1): almost a copy of the answer.
- `pd.DataFrame({...})` puts the two features in one table with named columns.

Now train the same random forest twice, once on `signal` alone and once with `leaked` added. Before you run it, predict: how much will the test AUC rise, and which feature will the forest say matters most?

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=74, stratify=y
)
for cols in [["signal"], ["signal", "leaked"]]:
    forest = RandomForestClassifier(
        n_estimators=200, min_samples_leaf=5, random_state=74
    ).fit(X_tr[cols], y_tr)
    auc = roc_auc_score(y_te, forest.predict_proba(X_te[cols])[:, 1])
    importances = forest.feature_importances_.round(2)
    print(f'{' + '.join(cols):<16} test AUC {auc:.3f} importances {importances}')
```

```
signal          test AUC 0.932 importances [1.]
signal + leaked test AUC 1.000 importances [0.29 0.71]
```

How it works:

- `stratify=y` keeps the 19.6% positive rate the same in the training and test rows (Chapter 36, section 36.3).
- The loop fits one forest per feature list. `X_tr[cols]` picks just those columns. `n_estimators=200` trees, `min_samples_leaf=5` and `random_state=74` are the kind of settings Chapter 37, section 37.7 uses.
- `feature_importances_` has one number per feature, adding up to 1 (section 37.7); `.round(2)` keeps two decimals. With one feature it's `[1.]`.
- `' + '.join(cols)` writes the feature list as "signal + leaked"; `[:16]` pads it to 16 characters so the columns line up.

Reading it. One noisy copy of the answer pushes the test AUC from 0.932 to a **perfect 1.000**, and takes 71% of the importance away from the feature that really drives the outcome. A score that's suspiciously perfect plus one dominant feature: that's the profile of a leak, not of a powerful predictor.

Tier	What to say
Passes	"Leakage is when future information gets into training" (correct, no demonstration)
Strong	The worked comparison above, and the diagnostic instinct it teaches: a suspiciously high score (especially near-perfect) plus one feature dominating importance is the classic leakage fingerprint
Extra points	[+Validate] the measured jump from 0.932 to 1.000 and the 0.29/0.71 importance split, not an assumed pattern [+Evidence] the same thing on Riverstone's leads: adding <code>has_quote</code> , filled in only for leads already going well, lifts validation AUC from 0.823 to 0.978

Likely follow-ups: Name three real-world sources of target leakage beyond a literal copy of the label. How would you catch this without already suspecting a specific column?

Red flag: treating a near-perfect score as good news rather than the first thing to investigate.

Learn it in: Chapter 36, section 36.7 (three leaky features and one contaminated encoding, each added one at a time with its AUC, and the leakage checklist); Chapter 37, section 37.7 (random forests and `feature_importances_`).

Q74-020 · What's temporal leakage, and why is a random train/test split dangerous for time-ordered data?

Level: Mid · **Roles:** DS, MLE

Remember it as: A model trained on the future to predict the past will look brilliant and mean nothing. Always split by time when time is part of the story.

Answer in one line: **Temporal leakage** happens when information from *after* the prediction moment sneaks into training, most commonly through a random (not time-based) train/test split on time-series or event data, letting the model learn from outcomes that hadn't happened yet at the point it's meant to predict.

Tier	What to say
Passes	"You should split time series data by time, not randomly" (correct, no reasoning given)
Strong	Explains that a random split puts rows from <i>after</i> a test row's date in the training set, so features that quietly encode "what eventually happened" leak through
Extra points	[+Evidence] Riverstone's lead model scored <i>lower</i> on a random split (0.791) than on the proper time split (0.823), the <i>opposite</i> of what leakage usually does [+Limits] the honest explanation: it wasn't leakage, it was a real shift in the lead mix over time. A lower score isn't automatically "more honest"; you have to know <i>why</i> the numbers differ

Likely follow-ups: How would you build a time-respecting cross-validation scheme, not just a single split? What's a rolling-origin backtest?

Red flag: assuming every score difference between a random and a time-based split is explained by leakage.

Learn it in: Chapter 36, sections 36.3 (split by time) and 36.10 ("Back to the random split"); Chapter 40, section 40.7 (backtesting).

Rapid-fire, 74.4

Roles: DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-021	What's target encoding leakage?	Encoding a categorical feature with the target's mean computed on the same rows being scored, silently baking outcome information into the feature	[+Evidence] company win rates computed on all leads lift validation AUC from 0.823 to 0.845, on company names that carry no real signal; TargetEncoder inside the pipeline gives the honest 0.823	Mid · 36.7 ("Contamination")
Q74-022	Why is fitting a scaler or encoder on the full dataset before splitting a leakage risk?	Its learned values (mean, standard deviation, category list, category win rates) are computed partly from test rows, so a little of the test set shapes training	[+Evidence] fitting the <i>target encoder</i> before cross-validation inflated AUC from 0.78 to 0.82; a scaler fitted on all rows leaks far less, but the fix is the same pipeline	Mid · 36.9
Q74-023	What's the fastest sanity check for suspected leakage in a model that scores too well?	Check what the score does as you remove features one at a time, especially any that are suspiciously predictive alone, or recorded at or after the outcome moment	[+Signpost] start with the one feature carrying a large share of the importance, especially one that "sounds like" the answer; permutation importance is the more reliable way to find it	Mid · 36.7; 39.8
Q74-024	Can leakage happen even with a properly time-ordered split?	Yes: a feature aggregated "as of today" that includes same-day or future data (a rolling window without a shift, say) leaks even inside a correctly time-split pipeline	[+Limits] leaks aren't always dramatic: Chapter 40's leaky rolling means made the forecast look only half a point better (WAPE 8.9% against an honest 9.4%)	Mid · 40.6 (the .shift(1) before .rolling() rule) and its exercise 10

74.5 Feature engineering

Q74-025 · A categorical feature has 500 unique values. What are your options, and what's the trade-off for each?

Level: Mid · **Roles:** DS, MLE

Remember it as: One-hot blows up your feature count. Target encoding risks leakage. Frequency or hashing keeps it small and safe, at the cost of some information.

Answer in one line: One-hot encoding (500 new columns, sparse and unwieldy for a high-cardinality feature), target encoding (compact, but a real leakage risk unless it's fitted inside the pipeline, Q74-021), frequency encoding (replace each category with how often it appears: simple and safe, but loses category-specific signal), or feature hashing (a fixed number of columns, at the cost of occasional collisions between unrelated categories).

Beyond the book: frequency encoding and feature hashing. Chapter 36 teaches one-hot and target encoding; these two are new here. **Frequency encoding** replaces each category with its share of the rows. **Feature hashing** runs each category through a hash function (Chapter 45, section 45.5) and uses the result to pick one of a fixed number of columns, so 500 categories, or 50,000, still make, say, 32 columns. Two different categories can land in the same column: a **collision**. Here are both on four company names:

```
import pandas as pd
from sklearn.feature_extraction import FeatureHasher

company = pd.Series(["Lotus Kitchenware", "Metro Mart", "Lotus Kitchenware", "Sharma Hardware"])
share = company.value_counts(normalize=True)
print("frequency:", company.map(share).to_list())

hasher = FeatureHasher(n_features=8, input_type="string", alternate_sign=False)
hashed = hasher.transform([name] for name in company).toarray()
print("hashed column for each row:", hashed.argmax(axis=1))
```

```
frequency: [0.5, 0.25, 0.5, 0.25]
hashed column for each row: [7 7 7 5]
```

How it works:

- `value_counts(normalize=True)` gives each name's share of the rows instead of its count (Chapter 18); `.map(share)` looks up each row's name in that table, and `.to_list()` prints the result as a plain list. Lotus Kitchenware is half the rows, so it becomes 0.5.
- `FeatureHasher(n_features=8, ...)` will make 8 columns. `input_type="string"` says each row is a list of text values, which is why each name is wrapped in its own list, `[name]`. `alternate_sign=False` keeps every entry at +1 (by default some are -1, a trick that makes collisions cancel out on average).
- `.transform(...)` returns a sparse matrix (mostly zeros, stored compactly); `.toarray()` turns it into an ordinary NumPy array. Each row has a single 1, in the column its name hashed to, and `.argmax(axis=1)` finds that column: the position of the largest value in each row (Chapter 35, section 35.8).

Reading it. Two different companies, Lotus Kitchenware and Metro Mart, landed in the same column, 7: a collision, so the model can't tell them apart. **What happens if you change it:** give the hasher 16 columns instead of 8.

```
hasher16 = FeatureHasher(n_features=16, input_type="string", alternate_sign=False)
print(hasher16.transform([[name] for name in company]).toarray().argmax(axis=1))
```

```
[ 7 15  7  5]
```

hasher16 is the same FeatureHasher call with `n_features=16`, and the same `.transform(...)`, `.toarray()` and `.argmax(axis=1)` on the four company rows. Metro Mart moves to column 15 and the collision is gone. More columns mean fewer collisions, at the cost of a wider table; with 500 real categories, a few collisions are the price of a small, fixed table.

Tier	What to say
Passes	Names one-hot encoding as the only option, without acknowledging it scales poorly with cardinality
Strong	Names at least two alternatives with the trade-off for each, as above
Extra points	[+Trade-offs] the right choice depends on the model: tree-based models cope with compact encodings (target or frequency) well; linear and distance-based models need the number of columns actively managed [+Edge cases] a category that appears in test data but never in training: one-hot gives it all zeros (<code>handle_unknown="ignore"</code>), target encoding falls back to the overall mean, and hashing still gives it a column

Likely follow-ups: How would you safely do target encoding without leaking? What's the risk of a rare category appearing in test data but not training data?

Red flag: treating one-hot encoding as the universal default regardless of cardinality.

Learn it in: Chapter 36, sections 36.4 (`OneHotEncoder`), 36.6 ("Categories": one-hot, ordinal, and the high-cardinality row) and 36.7 (target encoding inside the pipeline). Frequency encoding and hashing are new here.

Rapid-fire, 74.5

Roles: DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-026	Why log-transform a skewed numeric feature like quantity or revenue before modeling?	It compresses a long right tail, reduces the pull of extreme values on linear-model coefficients and distances, and often makes the relationship with the target closer to linear	[+Edge cases] use <code>np.log1p</code> (the log of $1 + x$) so zeros are allowed, and cap impossible values first	Fresher · 36.6
Q74-027	What's an interaction feature, and when do you need one explicitly?	A feature built from two others (a product, a ratio, a flag for "both") to capture an effect that depends on both together; needed explicitly for linear models, which don't discover interactions the way trees can	[+Evidence] hospitality leads in August–October are won 8.9% of the time against 6.4% in other months: an interaction of segment and month	Mid · 36.6 ("Dates and times")

#	Question	One-line answer	Extra point	Level · learn it in
Q74-028	Should missing values always be imputed with the mean?	No: sometimes the fact that a value is <i>missing</i> is itself informative (worth a separate missing-indicator flag), and imputing blindly can hide that signal	[+Evidence] leads with no quantity are won 5.71% of the time against 8.06% when it's given, because marketplace forms let buyers skip the box	Fresher · 36.8
Q74-029	What's the risk of engineering features "as of today" instead of "as of the prediction moment"?	It silently recreates temporal leakage (Q74-020): a feature aggregated from data that didn't exist yet at the real prediction time	[+Edge cases] use the time an event was <i>logged</i> , not when it's said to have happened; backdated entries leak	Mid · 36.6 ("Aggregates from activity logs")

74.6 Model debugging

Q74-030 · A model scores well in offline evaluation but performs noticeably worse once deployed. Walk through how you'd debug it

Level: Senior · **Roles:** MLE, DS

Remember it as: "It worked in training" and "it works in production" are different claims. The gap is almost always a difference between what the model saw then and what it's seeing now.

Answer in one line: Check, in order: (1) **training-serving skew**, the same feature computed differently in training and in production (different code, different timing, different defaults for blanks); (2) **data drift**, the inputs have shifted since training; (3) **feature availability**, a feature present in training but missing or late at serving time; (4) **label delay**, the outcomes arrive weeks later, so live accuracy can't be measured yet (monitor the inputs and predictions until it can); and (5) **silent pipeline bugs**, a join, a default value, or a type conversion behaving differently in production.

Tier	What to say
Passes	"Maybe the data changed" (a reasonable first guess, no structured process)
Strong	The five-category checklist above, with a proposed order to check them in
Extra points	[+Evidence] the churn forest's top features (days since the last order, then late-payment days) are the first ones to recompute in production and compare against their training distributions [+Limits] input drift isn't model decay: in Chapter 56's six months, new lamps pushed PSI to 4.4 while recall held (96.8% → 96.5%); then a new mould cut recall to 84.4% with no input alarm at all. Monitor outcomes, not just inputs [+Simple first] the fastest check is comparing a production feature's distribution with its training-time distribution for the same population, before touching the model

Likely follow-ups: How would you set up ongoing monitoring to catch this before a stakeholder notices? What's the difference between concept drift and data drift?

Red flag: jumping to "retrain the model" as the first response, before diagnosing what actually changed.

Learn it in: Chapter 56, sections 56.1 (data and concept drift), 56.7 (monitoring in three layers), 56.8 (drift, measured) and 56.11 (training-serving skew and feature stores); Chapter 39, section 39.10 (model cards); Chapter 37, section 37.7 (the churn forest's feature importances).

Rapid-fire, 74.6

Roles: DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q74-031	Your model's feature importances put a customer ID or row-order column near the top. What does that suggest?	A strong sign of leakage or a pipeline bug (an unshuffled split, an ID that tracks signup date, duplicates across train and test), not a real predictive relationship	[+Business] an ID should never be predictive on its own; if it looks like it is, something upstream is wrong	Mid · 36.7 (leakage checklist)
Q74-032	Training loss keeps falling but validation loss starts rising partway through training. What's happening, and what do you do?	Overfitting in progress; use early stopping (keep the model from the epoch with the best validation score) rather than training to the lowest possible training loss	[+Evidence] Chapter 43's churn network: validation AUC peaks at 0.773 around epoch 200 and slides to 0.730 by epoch 500 while training loss keeps falling	Mid · 43.5 and its exercise 8; 53.3, 53.6 (early_stopping=True)
Q74-033	A model performs very differently across two customer segments. Is that automatically a problem?	Not automatically, but it needs investigating: check whether it reflects a real difference in the segments' base rates (calibrated correctly for both) or an unfair, uncalibrated gap	[+Clarify] "Different how: in ranking, in calibration, or in who gets flagged?" Each has its own fix	Senior · 39.9
Q74-034	You retrain a model monthly and its performance quietly degrades over several cycles. What would you check first?	Whether the data itself has drifted (a changing lead mix, a new customer segment, a process change upstream) before assuming the modeling approach has stopped working	[+Evidence] the marketplace's share of Riverstone's leads rose from 30.1% (2023) to 44.5% (2025), and its win rate halved in 2025	Mid · 36.3; 56.9

74.7 ML case studies and live-coding walk-throughs

Q74-035 · Case study: a vendor pitches a "91% accurate" deep-learning churn model. Walk through how you'd evaluate the claim, live

Level: Mid · **Roles:** DS, DA

What they're really testing: whether you reach for the base rate and the right metrics before being impressed by a headline number, under real conversational pressure.

Talked through live: "First question: what's the actual churn rate in their data? If it's anywhere near 9%, predicting 'no churn' for everyone would already be about 91% accurate, so that number alone tells me almost nothing yet."

From Chapter 37, section 37.0: Riverstone's churn rate is 9.7% (484 of 5,000 accounts), so predicting "no churn" for everyone is **90.3%** accurate.

Talked through, live: "Their 91% is barely above the do-nothing baseline of 90.3%. I'd ask for AUC and log loss, compared against that same baseline, not accuracy, and I'd ask what their model was trained on. 'Deep learning' on its own tells me nothing about whether it beats a well-tuned simpler model: on our own churn accounts, a small neural network scored a test AUC of 0.818, against 0.797 for logistic regression and 0.821 for gradient boosting."

Extra-point moves shown: **[+Clarify]** asked for the base rate before reacting to the headline number. **[+Validate]** the real comparison, not an assumed one. **[+Business]** proposed the concrete next step (ask for AUC and log loss against the baseline) rather than just expressing skepticism.

Likely follow-ups: What if the vendor's AUC actually did beat your own model's? What questions would you ask about their training data before trusting it?

Red flag: being impressed by a percentage with no base-rate context.

Learn it in: Chapter 43's "In the real world" (a conference talk's 91% accuracy claim, with the base-rate check as the deciding question) and Chapter 35's "In the real world" (a vendor's 91% lead score, turned into a pilot with a pass mark).

Q74-036 · Case study: which algorithm would you reach for first, and how would you know if you're wrong?

Level: Senior · **Roles:** DS, MLE

What they're really testing: whether "it depends" gets followed by an actual decision process, not left as a dodge.

Talked through live: "For a first pass on a new tabular business problem, I'd start with logistic regression as a fast, interpretable baseline, then try gradient boosting as the strong default, and I'd know I picked wrong if boosting doesn't clearly beat the baseline on held-out data. That would suggest the relationships in the data are mostly additive, and the added complexity isn't earning its keep."

From Chapter 37, sections 37.11 and 37.12, test AUC, each scored once:

Data	Plain logistic regression	Tuned gradient boosting	Logistic + engineered features
Churn accounts (37.11)	0.797	0.838	0.850
Leads (37.12)	0.838	0.830	—

Talked through, live: "On churn, boosting beat the *plain* baseline by 0.04, but a logistic model with engineered features beat boosting, and it's explainable. On leads, boosting didn't win at all; the signal is mostly additive. That's why I always run the baseline, and a *good* baseline, before trusting the complex model."

Extra-point moves shown: **[+Trade-offs]** named a concrete decision rule ("did the complex model clearly beat the baseline?") instead of a vague "try a few things." **[+Evidence]** cited two different, real outcomes from the same body of work, showing the answer depends on the data rather than reciting "boosting always wins." **[+Validate]** added a third data point: in Chapter 43, a small neural network on the churn accounts landed between logistic regression and boosting (validation AUC 0.764, 0.773, 0.786).

Likely follow-ups: How would you decide "clearly beat," in numbers? What would make you skip straight to a more complex model without trying logistic regression first?

Red flag: naming one algorithm as a universal answer with no acknowledgment that it depends on the data.

Learn it in: Chapter 37, sections 37.10 ("Fixing bias with features"), 37.11 (tuning and the one-time test) and 37.12 (lead scoring, baseline vs boosting); Chapter 43, section 43.5 (the honest comparison).

APPLY

Common mistakes

Mistake	Symptom	Fix
Reporting accuracy alone on an imbalanced problem	A headline number barely above the base rate	Always state the base rate; use precision, recall, PR-AUC
Treating high AUC as proof of trustworthy probabilities	A confidently wrong "94% chance" shown to a stakeholder	Check calibration separately from ranking (Q74-014)
A suspiciously perfect or near-perfect score	The model has learned the target from a leaked feature	Check feature importance for one dominant column; question the top feature specifically
A random train/test split on time-ordered data	An offline score that doesn't hold up once deployed	Split by time; backtest with rolling origins
Jumping to "retrain the model" when production performance drops	The real cause (a pipeline bug, drifted inputs) goes unaddressed	Check training-serving skew and data drift before retraining
Assuming more features always helps	Perfect training score, poor test score, especially with few rows	Check the rows-to-features ratio; regularize; validate on held-out data
Judging a model on one small test set	A "trend" that's really luck (0.58 one run, 0.32 the next)	Cross-validate, or average several splits, before comparing
Picking a complex model without a baseline	No way to tell whether the complexity was worth it	Always run the simplest reasonable baseline first, and compare

In the real world: the feature importance that gave away the bug

Priyanka, a Data Scientist candidate, is shown a model with a suspiciously strong 0.97 AUC and asked to find what's wrong. Rather than guessing, she asks to see the feature importance table first, and immediately flags the top feature: `days_since_status_change`, contributing nearly half the model's total importance.

She reasons out loud: "That column name suggests it's updated *when* the status changes, which for a churn model probably means it gets updated right around, or even after, the churn event itself. If that's true, the model isn't predicting churn, it's detecting that churn has already been recorded." The interviewer confirms: the feature was indeed set at the moment an account was marked churned, a close cousin of this chapter's own leaked-feature demonstration.

The interviewer's note afterward: "Went straight to feature importance, and reasoned about the specific column name rather than asking to see more code. Diagnosed the leak from the name and the number alone, before we even opened the data." That's this chapter's whole method in miniature: a suspicious score plus a dominant feature is the fingerprint, and naming *why* a specific feature is suspicious, not just that leakage is possible in general, is what separates a strong answer from a correct-sounding one.

Project

Goal: reproduce this chapter's core demonstrations on your own model or data.

Tools you'll need

scikit-learn, **pandas**, and **NumPy**, the same stack used throughout Chapters 35–43 (scikit-learn is installed in Chapter 35, section 35.9). No new tools: `FeatureHasher` is part of scikit-learn.

1. Run your own bias-variance sweep (varying maximum depth, regularization strength, or a similar complexity setting) on a real model, and find where the sweet spot actually sits, not where you'd guess it sits.
2. Deliberately leak a feature into a model you control (a noisy copy of the target, or a feature computed after the outcome) and measure exactly how much the score inflates.
3. Take a model you've built with a random split and rebuild it with a time-based split if your data has any time order; compare the two honestly.
4. Pick one of your own models and write a two-sentence "how I'd debug this in production" plan, using the five-category checklist of Q74-030, before you need it for real.

Key terms

bias-variance trade-off · underfitting · overfitting · regularization (L1/L2) · learning curve · curse of dimensionality · adjusted R^2 · base rate · confusion matrix · precision · recall · F1 score · macro and weighted F1 · ROC-AUC · PR-AUC · calibration · target leakage · temporal leakage · target encoding leakage · training-serving skew · data drift · concept drift · label delay · feature importance · early stopping · one-hot encoding · frequency encoding · feature hashing · hash collision · interaction feature · missing-value indicator

Final-week revision list

Q74-001, Q74-002, Q74-007, Q74-008, Q74-013, Q74-014, Q74-015, Q74-016, Q74-019, Q74-020, Q74-021, Q74-025, Q74-030, Q74-031, Q74-035, Q74-036.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 79, GenAI, LLM & MLOps Question Bank**, takes Q74-030's monitoring and skew questions into feature stores and LLM-based systems.
- **Chapters 35–43** are where every technique this bank draws on was taught in full; this chapter tests them, it doesn't re-teach them. When an interview pushes past "the model predicts X" into "does changing X actually cause the outcome?", go back to **Chapter 30** (experiments) and **Chapter 31** (causal inference).

Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates

Chapter at a glance

You will learn to: structure an ambiguous business case the way an interviewer actually wants, instead of jumping straight to an answer · diagnose a metric that moved, systematically, not by guessing at causes · design a KPI dashboard that answers real decisions, not just displays numbers · size a market or estimate a quantity with a defensible structure, not a guessed final number.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 3 (KPIs, dashboards), 4 (percentage points, estimation), 5 (precise questions, issue trees, MECE), 22 (A/B design, Simpson's paradox), 23 (KPI trees, diagnosing a change, guardrails) and 24 (turning an ask into a question), with a few links to Chapters 15, 16, 25 and 30. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 2½–3½ hours to read and drill every question once, out loud; 2–3 hours more for the project.

How this chapter is built. Same format as every question bank in Part 8: a memory hook ("Remember it as..."), a one-line answer, and a tier table (**passes**, **strong**, **extra points**, tagged with Chapter 69's moves: **[+Clarify]**, **[+Signpost]** and so on), then follow-ups, the red flag, and where to learn it. Rapid-fire rows end with a level and the section that teaches the idea (a bare number such as 23.10 means that section). There is no code: a case is judged on structure and reasoning. Every number in a worked example is either an assumption, labelled as one, or a figure with its source, and the arithmetic is shown step by step. The chapter moves from the general case framework (section 75.1) through metric diagnosis (75.2), metrics and dashboards (75.3) and product decisions (75.4) to guesstimates (75.5) and two full cases (75.6). Guesstimates need only Chapter 4, so a first-job candidate can do section 75.5 straight after 75.1.

Levels and roles. Fresher: screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **DA** data analyst · **BA** business analyst · **PA** product analyst · **BI** BI developer. Do every Fresher question first.

In this chapter

LEARN	75.1 The general case framework	226
	75.2 Diagnosing a metric that moved	228
	75.3 Designing metrics and dashboards	230
	75.4 Product sense and decision cases	233
	75.5 Guesstimates: structured estimation	234
	75.6 Full cases, talked through live	238
APPLY	Common mistakes · In the real world · Project	239
REVIEW	Key terms · Final-week revision list	241
NEXT	Where this leads	241

75.1 The general case framework

Q75-001 · You're given a vague, one-sentence business question in an interview. Walk through your first sixty seconds

Level: Fresher · **Roles:** DA, BA, PA

Remember it as: *Restate, clarify, structure, then answer, in that order, every single time, regardless of how simple the question sounds.*

Answer in one line: Restate the question in your own words to confirm you understood it, ask one or two clarifying questions that would genuinely change your approach, lay out a structure (a short list of the areas you'll investigate) before touching any specifics, and only then start filling in the structure with actual reasoning.

Worked example, applied to "Our support team wants a way to know which customers to prioritize. How would you help them?":

"Prioritize for what, exactly, retention risk, upsell potential, or urgency of a live issue? And is this a one-time list or a standing, updated view? I'll assume retention risk, updated regularly, unless you say otherwise. I'd look at this in three parts: what signals predict risk, how we'd turn those into a single prioritized list, and how support would actually act on it day to day."

Tier	What to say
Passes	Restates the question, then starts answering with a sensible but unstructured list
Strong	Restates, asks one genuinely important clarifying question, states an assumption, and previews a structure before diving in
Extra points	[+Clarify] the clarifying question changes the actual approach, not a generic "what do you mean" [+Signpost] the three-part preview above gives the interviewer a map of where the answer is going, so they can redirect early if it's the wrong direction, rather than sitting through five minutes before finding out

Likely follow-ups: What if the interviewer won't answer your clarifying question? (State your assumption explicitly and proceed; don't stall waiting for permission.) How do you know when you've spent too long clarifying and not enough time actually answering?

Red flag: either no clarification at all, or clarifying so much it eats most of the available time without ever reaching a structured answer.

Learn it in: Chapter 24 §24.1 (turning an ask into a question) and Chapter 5 §5.2 (from a vague request to a precise question). **Practise it with:** Chapter 69, Move 1 (clarify) and Move 3 (signpost).

Q75-002 · What's the difference between a "diagnose a metric" case and a "should we build this" case, and why does mixing up the structure hurt you?

Level: Mid · **Roles:** DA, BA, PA

Remember it as: A metric-drop case works backward from a number to a cause. A product-decision case works forward from a goal to a recommendation. Using the wrong shape wastes the whole case.

Answer in one line: A metric-diagnosis case starts from an observed change and works backward through possible causes (data issue, external factor, internal change, segment-specific shift) to find what actually happened; a product-decision case starts from a goal or user need and works forward through options, trade-offs, and success metrics to reach a recommendation: using a diagnostic structure on a decision question (or vice versa) produces an answer that technically covers ground but never actually answers what was asked.

Tier	What to say
Passes	Describes both kinds of case sensibly, but doesn't say which structure each one needs
Strong	Explicitly identifies which of the two shapes the question is (or names a case that's genuinely a hybrid) before choosing a structure
Extra points	[+Signpost] naming the case type out loud in the first thirty seconds ("this sounds like a diagnostic case, so I'll work backward from the number") itself signals structured thinking to the interviewer, before any content has even been delivered

Likely follow-ups: What's a case that's genuinely a hybrid of both? How would you handle a case where the interviewer's question doesn't clearly signal which type it is?

Red flag: applying a rigid, memorized framework to every case regardless of fit, producing a technically thorough but off-target answer.

Learn it in: Chapter 5 §5.1 (descriptive, diagnostic, predictive and prescriptive questions). Both shapes are worked in this chapter: diagnostic cases in section 75.2, product decisions in section 75.4.

Rapid-fire, 75.1

Roles: DA, BA and PA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q75-003	What's a MECE structure?	Mutually Exclusive, Collectively Exhaustive: a breakdown where categories don't overlap and together cover the whole space	[+Business] a non-MECE structure (say, "large customers" and "enterprise customers" as separate buckets) leads to double-counting or gaps that quietly undermine the whole analysis	Fresher · 5.4
Q75-004	Why state an assumption out loud instead of silently picking one?	It lets the interviewer correct you immediately if you assumed wrong, rather than watching you build fifteen minutes of analysis on the wrong foundation	[+Assume] "I'll assume we mean billed revenue; tell me if not": one sentence that invites the correction before it costs you anything	Fresher · 5.5
Q75-005	How much time should structuring take relative to the whole case?	As a rule of thumb, roughly 10–15% (one to two minutes of a fifteen-minute case): enough to set direction, not so much that there's no time left to actually work through it	[+Edge cases] over-structuring (a beautiful framework with no time left to fill it in) scores worse than a slightly rougher structure that's actually worked through	Mid · 69.2

#	Question	One-line answer	Extra point	Level · learn it in
Q75-006	What's the risk of a purely qualitative answer with zero numbers, in a case that clearly has data available?	It signals an inability or unwillingness to quantify, which is usually exactly what the interviewer is testing for in a data-adjacent role	[+Business] even a rough, clearly-labeled estimate ("if I had to guess, maybe 20% of the drop") beats no number at all	Fresher · 4.9

75.2 Diagnosing a metric that moved

Q75-007 · "Weekly active users dropped 15% last week. Walk me through how you'd figure out why."

Level: Fresher · **Roles:** DA, PA, BA

Weekly active users (WAU) counts the distinct users who used the product at least once in the week: the same idea as Chapter 3's active customers, counted weekly.

Remember it as: *Verify the number is real before you go hunting for a business reason it might not need.*

Answer in one line: Check data integrity first (a tracking bug, a reporting change, a timezone or date-boundary artifact), then check for an external or seasonal explanation (a holiday, a known outage, a competitor event), then look for an internal cause (a recent release, a pricing change, a removed feature), and only then drill into which segment of users actually drove the drop.

Worked structure, applied out loud:

"First, I'd confirm the 15% is measured the same way it always is; a logging change or a shifted week boundary can produce a number that looks like a real business drop but isn't one. Assuming it's real, I'd check for anything external that week, a holiday, an outage, a competitor launch, before assuming it's something we did. Then I'd check what changed internally: a release, a pricing change, a removed feature, in roughly that order of likelihood. Finally, I'd segment the drop by platform, geography, and user tenure to see if it's broad or concentrated, since a drop concentrated in one segment points to a very different cause than a broad, even one."

Tier	What to say
Passes	Checks one or two plausible causes (a release, a holiday) and segments by platform, but never asks whether the number itself is right
Strong	The four-stage funnel above: verify the data, check external factors, check internal changes, then segment
Extra points	[+Validate] explicitly proposes checking whether the drop is broad or concentrated in one segment, since that single fact rules out most hypotheses at once [+Business] ties the investigation to a concrete next action at each stage, not just a list of things to "look into"

Likely follow-ups: The drop turns out to be concentrated entirely in one country. What does that suggest, and what would you check next? How would you distinguish a real user-behavior change from a measurement artifact, concretely?

Red flag: guessing a specific root cause before verifying the number and ruling out the broad categories.

Learn it in: Chapter 23 §23.11 (a general method for diagnosing a KPI move: confirm it's real, walk down the tree, check for a mix effect) and Chapter 5 §5.4 (issue trees and MECE). **Practise it with:** Chapter 69, Move 5 (edge cases) and Move 7 (validate), and section 69.4's worked revenue drop.

Q75-008 · "Revenue is up 10% year over year, but our finance team is worried. Why might a rising number still be bad news?"

Level: Mid · **Roles:** DA, BA, PA

Remember it as: A single headline number can hide a mix shift, a one-time event, or a sustainability problem underneath a technically true "growth" story.

Answer in one line: Revenue growth can mask a worse underlying picture in several ways: it might be driven by a small number of large, unrepeatable deals rather than broad healthy demand; it might be coming at the cost of margin (revenue up, profit down); it might be masking customer churn (fewer, bigger customers replacing many smaller ones, a riskier concentration); it might be price-driven (list-price rises masking falling units or fewer active customers); or it might not be keeping pace with a cost base or a target that's growing even faster.

Tier	What to say
Passes	Names one reason (usually "profit could be down") without a way to check it
Strong	Names at least two of the mechanisms above with a concrete business reason each matters
Extra points	[+Business] ties each mechanism to a specific follow-up metric that would confirm or rule it out: margin trend for the profit concern, customer count and concentration for the mix-shift concern, cohort retention for the churn-masking concern, units and the active-customer trend for the price concern

Likely follow-ups: Which of these would you check first, and why? How would you present this nuance to a stakeholder who's happy about the headline number?

Red flag: treating "revenue is up" as inherently good news with no interrogation of what's underneath it.

Learn it in: Chapter 23 §23.10–23.11 (the KPI tree: active customers × orders per customer × average order value, and walking a change down it) and Chapter 4 §4.10 (a headline number without its context). Section 75.3 below meets the same mix problem again, in dashboard design.

Rapid-fire, 75.2

Roles: DA, PA and BA for every row.

#	Scenario	Structure to reach for	Extra point	Level · learn it in
Q75-009	Conversion rate on the checkout page dropped after a redesign	Data-first: confirm the drop wasn't caused by an analytics tagging change in the redesign itself before assuming the design is at fault	[+Edge cases] a redesign that changes page structure very often breaks event tracking at the same time it changes the actual experience, tangling two separate questions together	Fresher · 23.11
Q75-010	Customer support ticket volume spiked	Segment by ticket category first: a spike concentrated in one issue type points to a specific bug or change; a broad spike across categories points to something upstream (an outage, a billing run)	[+Business] ticket <i>volume</i> alone doesn't say whether it's a new problem or a known one just getting reported more, worth checking against a "first time this issue" flag if one exists	Mid · 5.4
Q75-011	Average order value is flat, but total revenue fell	The drop is in order <i>count</i> , not order value. Split order count into active customers × orders per customer: fewer customers points to acquisition or churn; the same customers ordering less often points to frequency or retention. Pricing and basket size are ruled out	[+Signpost] the same decomposition as Chapter 23's KPI tree (section 23.10, Figure 23.3), walked in section 23.11, just applied live under interview pressure	Mid · 23.10, 23.11
Q75-012	A metric looks fine in aggregate but a stakeholder insists something's wrong in their area	Always check for Simpson's paradox before dismissing the stakeholder: an aggregate can hide a real, reversed trend in a specific subgroup	[+Evidence] the stakeholder closest to the data is often right about a localized problem an aggregate view genuinely can't show; practise it with Chapter 73, Q73-031	Mid · 22.6

75.3 Designing metrics and dashboards

Q75-013 · "Design a KPI dashboard for a customer support team's manager." Walk through your approach

Level: Mid · **Roles:** DA, BI, BA

Remember it as: Ask what decision the dashboard needs to support before picking a single metric. A dashboard with no decision behind it is just numbers on a wall.

Answer in one line: Start from the decisions the manager actually needs to make (staffing that day, which tickets need escalation, whether the team is meeting its service commitments), then pick the smallest set of metrics that supports those decisions, rather than starting from "what data do we have" and displaying all of it.

Two support terms used below: **average handle time** (AHT) is the average minutes an agent spends on one ticket, and a **service-level agreement** (SLA) is the promised time to respond to or resolve a ticket.

Worked structure:

"I'd ask what decisions this manager makes weekly: probably staffing levels, escalation priority, and reporting up to their own leadership. For staffing, I'd want ticket volume trend and average handle time, shown next to repeat-contact rate so a push for speed can't hide unresolved tickets. For escalation, open tickets by age and by severity, so anything aging past a threshold is visible immediately. For reporting up, a service-level metric like % of tickets resolved within target time, trended over time, not just a snapshot. I'd deliberately leave off vanity metrics like total tickets ever handled, since it doesn't drive any decision this specific manager makes day to day."

Tier	What to say
Passes	A sensible list of support metrics (volume, handle time, SLA) not tied to decisions
Strong	The decisions-first structure above, with metrics explicitly tied to staffing, escalation, and reporting
Extra points	[+Business] explicitly excludes a plausible-sounding but decision-irrelevant metric, which signals real judgment, not just metric-generation [+Edge cases] flags that a single "tickets resolved" count can hide the exact Simpson's-paradox-style mix problem from Q75-012, so an SLA breakdown by ticket type is more honest than one blended number

Likely follow-ups: How would this dashboard differ for a frontline agent instead of a manager? What's one leading indicator (predicts a future problem) versus one lagging indicator (reports a past one) you'd include, and why have both?

Red flag: a dashboard design that's really just "every metric I can think of related to the topic," with no filter for what's actually decision-relevant.

Learn it in: Chapter 3 §3.6 (dashboards and the decisions they serve), Chapter 15 §15.2 (start from the question) and Chapter 16 §16.7 (designing the report page); the handle-time guardrail is Chapter 23 §23.12. **Practise it with:** Chapter 70 §70.8 (Q70-069/070), the same "does this serve a real decision" discipline applied to an already-built dashboard instead of a from-scratch design.

Q75-014 · What's the difference between a leading and a lagging indicator, and why does a good dashboard need both?

Level: Fresher · **Roles:** DA, BA, PA, BI

Remember it as: A lagging indicator tells you what already happened. A leading indicator gives you a chance to change what happens next.

Answer in one line: A **lagging indicator** (revenue, churned customers, resolved tickets) reports an outcome after it's already occurred, useful for accountability and reporting; a **leading indicator** (website traffic, open pipeline, ticket backlog growth rate) signals a likely future outcome early enough to still act on it: a dashboard built entirely on lagging indicators can only ever confirm a problem after it's too late to prevent it. To trust a leading indicator, check that it moves *before* the outcome across several periods, then test it: does intervening on it change the outcome?

Tier	What to say
Passes	Defines the terms correctly, can't give a concrete pair from the same domain
Strong	A matched pair from one domain: e.g. churned customers (lagging) vs. a declining product-usage trend among still-active customers (leading, since it predicts churn before it happens)
Extra points	[+Business] a dashboard's real value often comes specifically from its leading indicators, since the lagging ones mostly confirm what a team already suspects by the time the number moves

Likely follow-ups: Name a leading indicator for churn that isn't usage frequency. How would you validate that a proposed leading indicator actually predicts the lagging outcome, rather than just correlating with it?

Red flag: unable to distinguish the two, or naming a lagging indicator as an example of a leading one.

Learn it in: Chapter 3 §3.5 (leading and lagging) and Chapter 23 §23.12 (pairing a leading metric with a guardrail); for checking that it predicts, Chapter 22 §22.5 (correlation, causation, and confounders) and Chapter 31 (causal inference without experiments). **Practise it with:** Chapter 73, Q73-032.

Rapid-fire, 75.3

Roles: DA, BA, PA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q75-015	What's a vanity metric?	A number that looks impressive and trends upward easily, but doesn't connect to a real business decision or outcome (total signups ever, cumulative downloads)	[+Business] the test: "if this number changed, would anyone actually do anything differently?": if not, it's a vanity metric	Fresher · 23.12
Q75-016	Why is a single blended average often a worse metric than a segmented breakdown?	It can hide Simpson's-paradox-style reversals (Q75-012) and mask the fact that different segments need entirely different actions	[+Validate] show the segmented table next to the blended number, so the reader can see whether the segments agree with the total	Mid · 22.6
Q75-017	What's a North Star metric, and what's the risk of picking the wrong one?	A single number, upstream of revenue, that captures whether customers are getting value (Riverstone: active customers). It is the one branch leadership watches weekly, not a replacement for the KPI tree, and it needs a guardrail because any single target can be gamed. The wrong one (engagement time instead of a genuinely valuable outcome) can hit its target while hurting the business	[+Business] a North Star metric should be hard to "game" without also delivering real value, or it quietly becomes a target people optimize around instead of through	Mid · 23.10, 23.12
Q75-018	How would you decide the right refresh frequency for a dashboard?	Match it to how often the decision it supports actually gets made: a daily staffing decision needs daily data; a quarterly strategy review doesn't need real-time numbers	[+Trade-offs] fresher data costs more: in Power BI it's Import with a scheduled refresh against DirectQuery; practise it with Chapter 70, Q70-063	Mid · 16.2

75.4 Product sense and decision cases

Q75-019 · "Should we build a loyalty program?" How do you approach a yes/no product decision with no data given upfront?

Level: Senior · **Roles:** PA, BA, DA

Remember it as: A product decision case isn't "yes" or "no." It's "here's what would have to be true for yes to be right, and here's how I'd check."

Answer in one line: Frame the decision around the conditions that would make it worthwhile (does the target segment actually respond to loyalty incentives, is the expected retention or spend lift big enough to cover the program's cost, is there a cheaper alternative that achieves the same goal), rather than trying to produce a confident yes/no with no data in hand.

Worked structure:

"I'd start with what problem this is meant to solve: is retention the issue, average order value, or something else? Assuming retention, I'd want to know our current churn rate and whether we have any evidence, even anecdotal, that price or reward sensitivity is a real churn driver for our customers, versus something a loyalty program wouldn't fix, like product availability. Then I'd size it: rough cost of running the program against a plausible retention lift, even a rough one, to see if the economics could plausibly work before recommending we actually build it. I'd end with a recommendation to pilot on one segment first, rather than a company-wide commitment, given how much of this is still genuinely uncertain."

Tier	What to say
Passes	Lists pros and cons of loyalty programs and leans yes, with no sizing or pilot
Strong	The conditions-for-yes framing above, ending in a specific, proportionate recommendation (a pilot, not a full commitment)
Extra points	[+Trade-offs] explicitly separates "would this help" from "is this the best use of the same budget," since a case that only evaluates one option in isolation misses cheaper alternatives that might solve the same problem [+Business] the pilot recommendation matches the actual level of confidence the analysis supports, rather than overclaiming certainty either direction

Likely follow-ups: How would you design the pilot to actually produce a clear answer? What metric would tell you the pilot worked?

Red flag: a confident yes/no answer with no acknowledgment of what's actually unknown.

Learn it in: Chapter 22 §22.3 (designing an A/B test) and Chapter 30 §30.7–30.8 (power and sample size; designing an experiment, with a primary metric and guardrails): exactly what a "pilot" here would need to be built well. **Practise it with:** Chapter 73 §73.4–73.5 (A/B test design and debugging).

Rapid-fire, 75.4

Roles: PA, BA and DA for every row.

#	Scenario	Key move	Extra point	Level · learn it in
Q75-020	"Would you prioritize feature A or feature B?" with limited detail on either	Ask for the goal each feature serves and roughly how many users each would affect, before comparing; don't guess at relative value with no basis	[+Signpost] reach for a simple, stated framework, for example RICE = (Reach × Impact × Confidence) ÷ Effort, so a feature scores higher when it helps more people, more, with more certainty, for less work (see the note below)	Mid · 25.9, 25.12
Q75-021	"A competitor just launched a feature we don't have. Should we build it too?"	Ask whether the feature addresses a real, evidenced need of your own users, not just competitive parity, before recommending building it	[+Edge cases] matching a competitor feature-for-feature with no evidence it matters to your own users is a common, costly trap	Mid · 5.3
Q75-022	"How would you know if a new feature is successful, before launch?"	Define the success metric and a rough target <i>before</i> building it, not after seeing what the number happens to be	[+Validate] fix the smallest effect worth detecting before launch, the same discipline as an A/B test's minimum detectable effect; practise it with Chapter 73, Q73-021	Mid · 22.3, 30.8
Q75-023	"Our product has high engagement but low revenue. What would you investigate?"	Check whether the engaged users are the same users being monetized, or whether engagement and monetization are happening in two different, disconnected populations	[+Business] a mismatch between who's engaged and who pays is one of the most common, fixable product-strategy problems this kind of question is fishing for	Senior · 5.4, 23.10

Beyond the book: RICE (Q75-020). RICE is the product-team version of the ranking score in Chapter 25: section 25.9 ranks gaps by hours saved × error weight ÷ build effort, and section 25.12 ranks automations on frequency, time, error cost and effort. RICE scores a feature on **Reach** (how many users it touches in a period), **Impact** (how much it helps each one, on a small scale such as 1 to 3), **Confidence** (how sure you are of those two, as a percentage) and **Effort** (person-weeks of work). Effort divides, because the same benefit for less work should rank higher. A feature that reaches 1,000 users a quarter, with impact 2, confidence 80% and effort 2 person-weeks, scores $(1,000 \times 2 \times 0.8) \div 2 = 800$. The same feature at 10 person-weeks scores $1,600 \div 10 = 160$. Multiply by effort instead of dividing and you get 3,200 against 16,000: the expensive feature wins, which is the opposite of what the score is for.

75.5 Guesstimates: structured estimation

Every number in a guesstimate is an assumption you choose and say out loud; the interviewer is grading the chain, not the facts. Where a real figure exists, the worked answers below give it with its source, so you can see how close a sensible assumption lands.

Q75-024 · Estimate the number of gyms in Mumbai. Walk through your structure, not just a final number

Level: Fresher · **Roles:** DA, BA, PA

Remember it as: A guesstimate is judged on the chain of reasonable numbers, not on how close the final answer lands.

Answer in one line: Say which Mumbai you mean, then break the target quantity into a chain of estimable factors (population → the fraction of that population who are gym-goers → the average size of a gym), state a defensible round number for each link, and multiply through, showing the full chain rather than a single guessed final number.

Worked calculation:

```
Greater Mumbai population (BMC area): ~12,500,000
share who regularly go to a gym: ~3%
→ gym-going population: 12,500,000 × 3% = 375,000
average members per gym: ~500
→ estimated number of gyms: 375,000 ÷ 500 = 750
```

The geography is itself an assumption to state: this chain uses **Greater Mumbai**, the area the city corporation (BMC) runs, not the wider metropolitan region around it, which has millions more people. The 2011 census counted 12,442,373 people there (Census of India 2011, Primary Census Abstract, ward-level tables), so 1.25 crore is a sensible round number; it has grown since, which is one more reason to keep only one or two significant figures. The 3% and the 500 are assumptions.

Tier	What to say
Passes	A chain of two factors with round numbers but no justification or sanity check
Strong	The full chain above, with each factor justified in one sentence ("3% feels right for a mix of gym-goers and non-gym-goers across income levels in a big city")
Extra points	[+Assume] names the geography first: "I'll take Greater Mumbai, the BMC area, about 1.25 crore people, not the whole metropolitan region" [+Validate] sanity-checks the final number against something known: "750 gyms across the BMC's 24 wards is about 31 per ward, or one gym for roughly every 17,000 people, which feels plausible for a dense city, not obviously too high or too low" [+Edge cases] flags the single most uncertain link in the chain (probably the 3% gym-going rate) as the one worth refining first if more precision were needed, rather than treating every link as equally solid

Likely follow-ups: Which of your assumptions would you want real data to check first? How would the estimate change for a smaller city?

Red flag: a single number with no visible chain, or a chain so long and granular it becomes a guess wearing a structure's clothing.

Learn it in: Chapter 4 §4.9 (the four-step estimation method: break it down, write each assumption with a round number, multiply, sanity-check). **Practise it with:** Chapter 69, Move 2 (state assumptions) and Move 7 (validate).

Q75-025 · Estimate the total addressable market, in rupees, for plastic storage containers sold to households across India

Level: Mid · **Roles:** DA, BA, PA

The **total addressable market** (TAM) is the total yearly spend on a category if every possible buyer is counted, whoever they buy from.

Remember it as: Top-down (population → adoption → spend) and bottom-up (built from the supply side: sellers × units each sells × price) should land in the same rough neighborhood: if they don't, that gap itself is the interesting finding.

Answer in one line: Top-down, chain the number of households through the share who buy this category each year and the average annual spend per buying household; then cross-check with a bottom-up estimate built from the supply side (the number of shops selling the category × containers each sells a year × average price, or the sum of known brands' sales), rather than trusting one method alone.

Worked calculation, top-down:

Indian households:	~300,000,000 (30 crore)
share buying storage containers a year:	~15%
→ buying households:	$300,000,000 \times 15\% = 45,000,000$
average spend per buying household:	~₹400 a year
→ market size:	$45,000,000 \times ₹400$ $= ₹18,00,00,00,000$ (₹1,800 crore)

Every link is an assumption. A quick check on the first: 30 crore households at about 4.5 people each is about 135 crore people; the 2011 census counted 121 crore (1,210,854,977), and the population has grown since, so the order of magnitude fits.

Worked calculation, bottom-up:

shops selling household plastics:	~500,000 (5 lakh)
containers each sells a year:	~250 (about 5 a week)
→ containers sold a year:	$500,000 \times 250 = 125,000,000$
average price per container:	~₹150
→ market size:	$125,000,000 \times ₹150$ $= ₹18,75,00,00,000$ (₹1,875 crore)

The two routes land within about 4% of each other (₹1,875 crore against ₹1,800 crore). They agree on units too: ₹400 a year at ₹150 a container is about 2.7 containers per buying household, or about 12 crore containers a year, close to the bottom-up 12.5 crore.

Tier	What to say
Passes	A top-down chain with round numbers, stated as if it were precise, with no cross-check
Strong	The full top-down chain above, explicitly stated as top-down, and said in the unit the audience uses: "about ₹1,800 crore"
Extra points	[+Validate] adds the bottom-up estimate from the supply side and compares the two, in rupees and in units [+Business] notes this is a <i>category</i> size, not any one company's addressable share, and ties it to Riverstone (Chapter 3): Riverstone sells to retailers, hospitality and wholesalers, not to households, but its retail customers sell into exactly this household market, so the figure bounds what Riverstone's retail segment could grow into [+Signpost] states the answer in the unit the interviewer uses: ₹1,800 crore, not "₹18 billion" or a string of zeros (Chapter 4 §4.9)

Likely follow-ups: How would Riverstone's unit sales plus an assumed market share cross-check the 15%? What's the biggest risk in this estimate being wrong by an order of magnitude?

Red flag: presenting a single top-down estimate as if it were precise, with no acknowledgment of the wide uncertainty band around it.

Learn it in: Chapter 4 §4.9 (the four-step estimation method, with the storage-box example this question scales up, and lakh and crore). **Practise it with:** Chapter 69, Move 2 (state assumptions) and Move 7 (validate).

Rapid-fire, 75.5

Roles: DA, BA and PA for every row. Every number in this table is an assumption.

#	Guesstimate	Structure to reach for	Extra point	Level · learn it in
Q75-026	How many piano tuners are there in a city of 5 million?	Pianos owned → tunings needed per piano per year → tunings one tuner can do per year	[+Signpost] the classic Fermi-problem format: every link should be a number you could defend in one sentence, not a fact you'd need to look up	Fresher · 4.9
Q75-027	Daily revenue of a mid-size coffee shop	Customers per day (estimate from seating capacity and turnover, or foot traffic) × average ticket size	[+Validate] 300 customers a day × ₹150 average ticket = ₹45,000 a day, a number worth sanity-checking against a rough rent-and-staff cost estimate for the same shop	Fresher · 4.9
Q75-028	Number of WhatsApp messages sent in India in one day	India's population → % of population on WhatsApp → average messages sent per active user per day	[+Edge cases] "average" hides a skew (heavy users send far more than the median user); worth naming that the estimate is more uncertain because of it, rather than pretending the average is a clean single number	Mid · 4.9
Q75-029	How many gas stations does a country need?	Vehicles on the road → average fuel consumption → stations' typical daily throughput capacity	[+Signpost] this specific guesstimate is really a supply-demand balance question in disguise, not a pure counting one, worth naming explicitly	Senior · 4.9

75.6 Full cases, talked through live

Q75-030 · Full case, talked through live: "Our email open rate dropped from 25% to 18% last month. What happened?"

Level: Mid · **Roles:** DA, PA

What they're really testing: whether the full structure (verify, external, internal, segment) actually gets executed end to end under time pressure, not just described in the abstract.

Talked through live, start to finish:

"So that's 7 percentage points, or a 28% relative drop, month on month; is that against last month only, or also the same month last year? First, I'd confirm the drop is measured consistently, same tracking pixel, same definition of 'open,' since email open-rate tracking is notoriously fragile, particularly around any change to how images load by default in major email clients, which can silently affect open tracking with nothing about the actual email changing at all. Assuming that checks out, I'd look for anything external: did a major email provider change its spam filtering or default image-blocking that month? Then internal: did we change our subject lines, sending time, sender name, or list composition (a big new batch of colder, less-engaged contacts added to the list would mechanically drop the average even with individually unchanged behavior)? Finally, I'd segment by list, by device, and by send time to see whether the drop is broad or concentrated. If it's concentrated in one email client, that points straight back to a client-side default-image-blocking change, not anything we did."

Extra-points moves demonstrated: **[+Clarify]** restated the drop with its base and period, in points and as a relative change ($25 - 18 = 7$ points; $7 \div 25 = 28\%$), before using it. **[+Edge cases]** named a specific, well-known confound (email clients changing how they load images by default) rather than a generic "maybe it's a tracking issue." **[+Signpost]** ran the full verify → external → internal → segment funnel completely, out loud, in order. **[+Business]** connected the list-composition hypothesis to a concrete mechanism (a mechanical average effect from adding colder contacts) rather than just naming it as a possibility.

Likely follow-ups: The drop turns out to be broad, not concentrated in one client or segment. What does that change about your next hypothesis? How would you recommend recovering the open rate?

Red flag: listing possible causes without ever proposing how to actually distinguish between them.

Learn it in: Chapter 4 §4.2 (percentage points and percent change), Chapter 23 §23.11 (diagnosing a change, including the mix check) and Chapter 5 §5.4 (issue trees). **Practise it with:** section 75.2 above, and Chapter 69, Move 5.

Q75-031 · Full case, talked through live: "Design a single metric to measure whether Riverstone's new self-service reorder portal is succeeding"

Level: Senior · **Roles:** PA, BA, DA

What they're really testing: whether a single metric can be chosen and defended, or whether the answer collapses into an undifferentiated list of "things you could measure."

Talked through live, start to finish:

"The portal's real goal is probably reducing manual reorder friction for repeat customers, not just 'more orders.' I'd propose **% of eligible repeat orders placed through the self-service portal, without needing a call or email to a rep**, as the single metric, since it directly measures adoption of the thing being built, not a proxy for it. I'd deliberately avoid 'total orders' as the top metric, since that's driven by far more than this one feature and wouldn't isolate the portal's actual contribution. I'd pair it with one guardrail metric, order accuracy or error rate through the portal, so we're not accidentally rewarding adoption that comes at the cost of more mistakes."

Extra-points moves demonstrated: [+Business] explicitly rejected a plausible-sounding but poorly-isolated metric (total orders) and explained why. [+Edge cases] added a guardrail metric unprompted, anticipating that optimizing the primary metric alone could create a hidden downside. [+Signpost] the chosen metric is a *rate* (self-service share of eligible orders), not a raw count, which is what makes it comparable over time regardless of overall business growth.

Likely follow-ups: How would you define "eligible" precisely? What would make you recommend killing the portal rather than iterating on it?

Red flag: naming several metrics with no commitment to a single primary one, or picking a metric that's really measuring overall business health rather than this specific feature.

Learn it in: Chapter 23 §23.12 (a rate instead of a count, and pairing a metric with a guardrail) and Chapter 30 §30.8 (a primary metric and its guardrail metrics). **Practise it with:** section 75.3 above (North Star metrics, Q75-017) and Chapter 70 §70.8 (Q70-069/070).

APPLY

Common mistakes

Mistake	Symptom	Fix
Answering before clarifying or structuring	A technically correct answer to the wrong specific question	Restate, clarify, preview a structure, in that order, every time
Guessing a root cause before verifying the metric itself	Chasing a business explanation for what's actually a tracking bug	Always check data integrity first in a diagnostic case
A dashboard built from "all available data"	A wall of numbers nobody actually uses to decide anything	Start from the decisions the dashboard needs to support
A confident yes/no on a product decision with no data given	Overclaiming certainty the case doesn't support	Frame it as "what would have to be true," recommend a pilot where genuinely uncertain
A guesstimate with no visible reasoning chain	Impossible for the interviewer to assess whether the thinking was sound	Show every link in the chain, one defensible number at a time
Naming a metric with no guardrail against gaming it	A metric that technically improves while the real goal suffers	Pair a primary metric with at least one guardrail

Mistake	Symptom	Fix
Treating every case with the same rigid memorized framework	A technically thorough but off-target answer	Identify the case type (diagnostic vs. decision) before choosing a structure

In the real world: the case that hinged on one clarifying question

Kabir, interviewing for a Business Analyst role, is given the case: "Our biggest customer's order volume dropped 40% last quarter. What do you do?" He starts to ask his usual battery of clarifying questions, then stops and asks one specific one instead: "Is this customer still ordering at all, just less, or have they effectively gone dark?"

The interviewer's answer changes everything: the customer had placed exactly one small order the entire quarter, down from a steady weekly cadence. Kabir immediately shifts his structure from a "why did volume decline gradually" diagnostic (checking pricing, product mix, competitor activity) to a "why did this relationship nearly stop" one (checking for a specific triggering event: a contract dispute, a champion leaving the customer's organization, a competitor winning an exclusive deal). He asks two more targeted questions and correctly guesses, out loud, that the most likely explanation is a champion departure, exactly what the interviewer had in mind.

The interviewer's note afterward: *"Most candidates ask five generic clarifying questions and get five generic answers. Kabir asked one that actually changed his whole approach, and used the answer immediately."* The lesson isn't "ask fewer questions." It's that a clarifying question earns its place only if the answer would genuinely change what you do next, exactly Chapter 69's Move 1, and this case shows what it looks like when that discipline is followed for real, under pressure, rather than performed as a checklist item.

Project

Goal: run this chapter's full method on cases of your own choosing.

Tools you'll need

No software specific to this chapter. A notebook or whiteboard to sketch a structure before speaking is the single most useful habit; interviewers consistently rate a candidate who visibly organizes their thinking (even three bullet points jotted down) higher than one who reasons entirely out loud with no visible structure.

1. Pick a real metric from your own work that changed recently, and run the full verify → external → internal → segment funnel on it, even if you already know the answer, to practice the structure itself.
2. Design a one-page dashboard for a real team you're familiar with, starting explicitly from the decisions it needs to support, and cut anything that doesn't serve one.
3. Take a real "should we do X" question from your own workplace and write the "what would have to be true" framing for it, ending in a specific, proportionate recommendation.

4. Practice three guesstimates out loud, timed to under three minutes each, showing your full chain of reasoning, not just a final number.

REVIEW

Key terms

MECE (Mutually Exclusive, Collectively Exhaustive) · diagnostic case · product-decision case · weekly active users · leading indicator · lagging indicator · North Star metric · vanity metric · guardrail metric · average handle time · service-level agreement (SLA) · RICE · Fermi estimation · total addressable market (TAM) · top-down vs. bottom-up estimate · pilot (as a decision-testing step)

Final-week revision list

Q75-001, Q75-002, Q75-007, Q75-008, Q75-013, Q75-014, Q75-017, Q75-019, Q75-024, Q75-025, Q75-030, Q75-031.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against, especially Moves 1 (clarify), 3 (signpost), and 7 (validate).
- **Chapter 76A, Data Analyst & Data Scientist Question Bank**, and **Chapter 76B, Business Analyst Question Bank**, continue from this chapter's case reasoning: 76A into project narratives, 76B into requirements and process questions.
- **Chapter 70, section 70.8**, already covers dashboard critique from the other direction, judging one that's already built rather than designing one from scratch.
- **Chapter 73, Statistics, Probability & Experimentation Bank**, is where a "pilot" proposed in any of this chapter's cases would actually need to be designed properly before running.
- **Chapters 3, 4, 5, 23 and 24** teach every technique this bank draws on, in full; this chapter tests them, it doesn't re-teach them.

Chapter 76A. Data Analyst & Data Scientist Question Bank

Chapter at a glance

You will learn to: answer the role-specific questions a Data Analyst or Data Scientist interview actually asks, distinct from the skill-testing banks elsewhere in this part (SQL, Python, stats, ML) · walk an interviewer through a real project end to end, at the right level of depth for each role · handle the "what's the difference between a DA and a DS" question, and the harder version, "which one are you actually best suited for" · survive a portfolio deep-dive without getting caught flat-footed on a detail you glossed over.

Before you start: Chapter 7, §7.2 (the tracks and the ten roles) · Chapter 8, §8.5 (decoding a job description) · Chapter 24, §24.4–24.8 (bottom line up front, the memo, handling pushback) · Chapter 25, §25.1 (the business analyst on a data team) · Chapters 36–39 (the lead-scoring project behind §76A.3) · Chapter 44 (the churn call list behind §76A.6) · Chapter 69 (the three answer tiers and the twelve extra-point moves).

Time needed: 2–2.5 hours to read and drill every question aloud; 3–4 hours for the project (two five-line summaries, one defended metric, and rehearsed walkthroughs).

How this chapter is different from Chapters 70–75. Those chapters test whether you *can do the work* (write SQL, debug pandas, reason about a p-value). This chapter tests whether you can *talk about the work you've done*, structure a project narrative, and demonstrate the specific judgment a DA or DS role requires beyond raw technical skill. Same format throughout: "**Remember it as...**" hooks, tier tables with extra points tagged by Chapter 69's moves ([+Clarify], [+Validate], [+Limits] and so on), and scan tables for rapid-fire.

Levels and roles. Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **DA** data analyst · **DS** data scientist · **BI** BI developer. Each **Learn it in** line names the section that teaches the skill; where a later bank drills it, a **Practise it with** pointer follows.

In this chapter

LEARN	76A.1 What the two roles actually do, and how interviewers probe the line between them	243
	76A.2 Walking through a Data Analyst project, end to end	245
	76A.3 Walking through a Data Science project, end to end	247
	76A.4 Stakeholder communication for DA and DS specifically	248
	76A.5 Portfolio deep-dives and handling scrutiny	250
	76A.6 Full scenarios, talked through live	252
APPLY	Common mistakes · In the real world · Project	253
REVIEW	Key terms · Final-week revision list	254
NEXT	Where this leads	255

76A.1 What the two roles actually do, and how interviewers probe the line between them

Q76A-001 · What's the actual difference between a Data Analyst and a Data Scientist, in a way that isn't just "DS does machine learning"?

Level: Fresher · **Roles:** DA, DS

Remember it as: A Data Analyst answers questions that already exist. A Data Scientist often has to decide which question is even worth asking, and builds something that keeps answering it automatically.

Answer in one line: A **Data Analyst** primarily investigates specific, often one-off business questions using existing data, producing reports, dashboards, and recommendations for human decision-makers; a **Data Scientist** more often builds systems (models, algorithms) that make or support decisions repeatedly and automatically, and spends more time on problem framing, experimentation, and statistical rigor before any model exists at all.

Tier	What to say
Passes	"DA does reporting, DS does machine learning" (true often enough, misses the deeper distinction)
Strong	The one-off-question vs. repeatable-system distinction above, with a concrete example of each: a DA answering "why did Q3 sales drop in the East region" once, versus a DS building a churn model that scores every customer every day going forward
Extra points	+ [+Business] the overlap is large and growing, especially at smaller companies where one person does both; naming that honestly, rather than presenting a rigid boundary, signals real experience over a textbook answer + [+Edge cases] a DA role at a company without a dedicated DS function often includes real modeling work, and a DS role at a data-mature company can be dominated by dashboarding and ad-hoc analysis, so the title alone doesn't fully predict the day-to-day

Likely follow-ups: Which parts of your own experience map more to one role than the other? Where do you see yourself wanting to grow, toward more DA-style or more DS-style work?

Red flag: a rigid, textbook boundary with no acknowledgment of real-world overlap, or being unable to place your own experience on either side of it.

Learn it in: Chapter 7, §7.2 (the tracks and the ten roles this whole book is built around) and Chapter 25, §25.1 (the four-role table: BA, DA, DS, DE).

Q76A-002 · An interviewer asks "why do you want this DA role and not a DS role, given your background touches both?" How do you answer honestly without talking yourself out of the job?

Level: Fresher · **Roles:** DA, DS

Remember it as: This question is testing self-awareness and genuine fit, not loyalty. A specific, honest reason beats a generic "I love both equally."

Answer in one line: Give a specific, honest reason tied to what you actually enjoy or want more of right now (closer to business stakeholders, faster feedback loops, breadth over depth, or the reverse for a DS answer), not a vague "I'm passionate about data" that could apply to either role equally.

Worked example:

"I've done modeling work and enjoyed it, but what energizes me more day to day is the moment right after I hand someone an answer and watch them actually act on it, that feedback loop is faster and more visible in DA work than in a lot of DS work, where the payoff can be months away. That's a real, specific reason I'm choosing this role now, not a consolation prize."

Tier	What to say
Passes	"I like both, but this role sounded interesting" (generic, doesn't actually answer why this one specifically)
Strong	The specific, honest framing above, tied to a real preference, not a rehearsed line
Extra points	+ [+Trade-offs] naming a genuine trade-off (faster feedback vs. deeper technical depth) rather than pretending one role is strictly better shows the kind of self-aware judgment interviewers are actually screening for here

Likely follow-ups: What would make you consider a DS role again in the future? What's one thing from DS work you'd want to bring into a DA role?

Red flag: an answer so generic it could be copy-pasted into an interview for the opposite role with no changes.

Learn it in: Chapter 8, §8.9 (pick your door, then learn backward) and Chapter 7, §7.8 (where you are right now).

Rapid-fire, 76A.1

Roles: DA and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-003	Where does a Data Analyst's work typically end and a Data Engineer's begin?	A DA typically works with data already made accessible (a warehouse, a clean table); a DE builds and maintains the pipelines that get it there in the first place	[+Edge cases] at a small company with no data engineer, the analyst often builds the first pipelines too; say so if that was you	Fresher · 7.2, 25.1
Q76A-004	Is "Business Intelligence Analyst" the same role as "Data Analyst"?	Largely overlapping: "BI analyst" is a common title for the role Chapter 7 calls the BI developer, leaning more heavily toward dashboards and reporting tools, while DA sometimes implies more ad-hoc, code-based analysis	[+Business] titles vary enormously by company; always check the actual JD's must-have list rather than assuming from the title alone	Fresher · 7.2, 8.5
Q76A-005	What's the most common reason a DA candidate is passed over for being "too DS," or vice versa?	Over-indexing the interview on technical depth (model internals, algorithm choice) when the role is really about stakeholder-facing analysis, or the reverse: under-demonstrating statistical rigor in a DS interview by staying purely descriptive	[+Business] matching your answer's depth and focus to the actual role, not just showcasing everything you know, is itself part of what's being evaluated	Mid · 8.3, 8.5

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-006	Does a Data Scientist need to know SQL as well as a Data Analyst does?	Yes, in almost every real DS role; the depth of Python/ML skill differs sharply between the two roles, but SQL fluency is close to a shared baseline	[+Evidence] name one real query you wrote to build a model's training table (a join and a date cut-off), not just "I know SQL"	Fresher · 7.2; Ch 12-13 (practise: Ch 71)

76A.2 Walking through a Data Analyst project, end to end

Q76A-007 · "Walk me through a data analysis project you're proud of." Give a strong, structured answer

Level: Mid · **Roles:** DA

Remember it as: *Business question first, method second, impact last, and the impact needs a number, even a rough one.*

Answer in one line: Structure the narrative as: the business question and why it mattered, what data and method you used (briefly, not a code walkthrough unless asked), what you found, and critically, what changed as a result, ideally with a quantified impact.

Worked example:

"Riverstone's sales team was manually pulling lead data weekly to decide who to call, taking about three hours every Monday. I built a prioritized call list instead: I pulled two years of closed leads and worked out the win rate for each lead source and company size. The gaps were large: referral leads were won about 19% of the time and marketplace leads about 3%, and bigger companies won more often too. So I built a simple scoring rule the sales lead could act on without needing a model explained to them: call referral, trade-fair and partner leads first. It cut their weekly prep time from three hours to about twenty minutes, and conversion on the leads called first rose from about 7% to about 12%, against the same number of leads called first under the old first-come-first-served order. The part I'm most proud of isn't the analysis itself, it's that I checked back a month later and it was still being used daily, which isn't always true of analysis work."

Tier	What to say
Passes	Names the business problem and the method, but the impact is vague ("it helped the team"), with no number
Strong	The business-question-first, quantified-impact structure above
Extra points	+ [+Business] the closing line, "I checked back a month later," directly answers the unspoken follow-up every interviewer wants to ask but often doesn't: did this actually stick, or was it a one-time deliverable nobody used again + [+Validate] the specific before/after numbers (three hours to twenty minutes, 7% to 12%) give the interviewer something concrete to probe further, rather than a vague "it really helped"

Likely follow-ups: What would you do differently if you started this project again today? What was the hardest part, and how did you get past it?

Red flag: a project narrative that never states a business impact, or states one with no number attached at all.

Learn it in: Chapter 24, §24.4–24.6 (bottom line up front, one message per slide, the one-page memo); Chapter 27, §27.10 (telling the story, in two minutes and in ten); Chapter 36, §36.10 (the "referrals first" rule; its exercise 5 works out the win rate by source). **Practise it with:** Chapter 69, §69.4 (one question, all three tiers).

Rapid-fire, 76A.2

Roles: DA and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-008	An interviewer asks "what was the hardest technical challenge in that project?" What are they actually testing?	Whether you can go one level deeper than the polished summary, showing real problem-solving under a specific constraint, not just reciting the project again	[+Business] have one specific, technical sticking point ready for every project in your portfolio, not just the smooth headline version	Mid · 27.10
Q76A-009	How would you handle being asked to present the same project to a technical panel and then to a business stakeholder, back to back?	Keep the core findings identical; adjust the depth of method explanation and vocabulary for each audience, exactly Chapter 24's storytelling discipline	[+Signpost] tell each audience at the start what you'll cover and how long it will take, then keep to it	Mid · 24.4–24.7
Q76A-010	What if your proudest project doesn't have a clean, positive outcome to report?	Report it honestly, including what you learned from a result that didn't pan out; a thoughtful account of a project that taught you something is often more convincing than a suspiciously perfect success story	[+Limits] interviewers are often more interested in how you handle an ambiguous or negative result than in a flawless win	Fresher · 27.8
Q76A-011	Should you mention tools and technologies by name in a project walkthrough, or focus purely on the business story?	Both, but in order: lead with the business story, mention tools briefly and naturally as part of the "how," not as a checklist recited up front	[+Business] a walkthrough that opens with a tool list ("I used Python, pandas, and Tableau") before explaining what problem it solved reads as technical-first, not business-first	Fresher · 27.10

76A.3 Walking through a Data Science project, end to end

Q76A-012 · "Walk me through an ML project you've built." Give a strong, structured answer

Level: Senior · **Roles:** DS

Remember it as: Problem framing and evaluation choice matter as much as the model itself. A DS project narrative that jumps straight to "I used gradient boosting" skips the parts that actually show judgment.

Answer in one line: Structure the narrative as: the business problem and why it needed a model (not just a rule or a report), how the target and evaluation metric were chosen and why, what approach was tried and what it was compared against, and what happened once it was in front of real users or real data, including any honest limitation.

Worked example: the same lead problem as Q76A-007, one step further. The analyst's rule became the baseline the model had to beat.

"The business problem was deciding which leads the inside sales desk should call first. The team already called referral, trade-fair and partner leads first, and the question was whether a model could do better than that rule. I framed it as binary classification: won within 90 days, predicted 24 hours after the lead arrives, using only what's in the CRM by then. Logistic regression beat the rule clearly: on the untouched test set it scored 0.838 AUC against the rule's 0.727. Then I tried tuned gradient boosting, to see whether the extra complexity earned its keep. It didn't: boosting scored 0.830 on the same test set, and logistic had the better log loss, so I kept the simpler, better-calibrated model. Then I set the call threshold by cost, not at 0.5: working a lead costs about ₹1,500, and a won lead is worth about ₹30,000 in first-year margin, so any lead with better than about a 5% chance of winning is worth a call. The honest limitation: it's a static model, so a shift in the lead mix between retrainings wouldn't be caught automatically; that monitoring is the next thing I'd build."

Tier	What to say
Passes	Explains the problem and the model and gives an AUC, but no baseline and no link from the metric to a business decision
Strong	The full structure above: framing, baseline-vs-complex comparison, evaluation choice tied to real cost, and an honest limitation
Extra points	+ [+Business] naming the cost-based threshold as the thing that turned a score into a call list, not the raw AUC, shows the exact judgment a DS interview is probing for: knowing that a good metric and a useful model aren't automatically the same thing + [+Trade-offs] reporting that the complex model lost, and keeping the simpler one, is itself the strongest signal of judgment + [+Limits] volunteering a real limitation (a static model with no drift check) unprompted, rather than waiting to be asked, signals honest self-assessment instead of oversell

Likely follow-ups: How would you detect the model going stale between retrainings? (Drift monitoring: Chapter 56, §56.8.) Why do you think boosting didn't win here? (Chapter 37, §37.12: the signal is mostly additive, and there were only 576 wins in training.)

Red flag: a project narrative dominated by algorithm names with no mention of why that algorithm, what it was compared against, or how success was actually measured.

Learn it in: Chapter 36, §36.1 (framing: the 90-day target and the prediction moment) and §36.10 (baselines, and the rule's 0.727); Chapter 37, §37.12 (lead scoring, baseline vs boosting); Chapter 39, §39.5 (choosing the threshold by business cost). **Practise it with:** Chapter 74, §74.7 (ML case studies).

Rapid-fire, 76A.3

Roles: DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-013	"What would you do if you had ten times more data for this project?" What's this really testing?	Whether you understand what's actually limiting your current model, more data helps most with variance/overfitting problems, not with a fundamentally wrong feature set or framing	[+Clarify] ask whether "more data" means more rows or more columns: more rows help a model that overfits; only new information helps a model that's missing signal	Senior · 37.10 (practise: Q74-001)
Q76A-014	How do you decide when a DS project is "done" versus needing more iteration?	When the marginal improvement from further work no longer justifies its cost against the business need, not when the model can't technically be improved any further at all	[+Business] "done" is a business judgment call as much as a technical one; naming that explicitly is itself a strong answer	Senior · 39.5, 44.6
Q76A-015	An interviewer asks you to defend a modeling choice you'd actually reconsider today. How do you answer honestly?	Say so directly: explain the original reasoning, then state plainly what you'd do differently now and why, rather than defending a choice you no longer believe in	[+Limits] say what changed your mind (a later result, a problem you missed), so the change reads as learning, not guessing	Mid · 69.3 (Move 10)
Q76A-016	Should a DS candidate be able to explain their model to a non-technical stakeholder, or is that a DA-specific skill?	It's expected of both: a DS who can't translate model output into a decision a business person can act on is significantly less valuable regardless of model quality	[+Business] end the explanation with the decision it supports ("so this account goes on Monday's call list"), not with the score	Mid · 39.8, 44.5

76A.4 Stakeholder communication for DA and DS specifically

Q76A-017 · A stakeholder asks you to make a dashboard "simpler" with no further detail. As a DA, how do you respond, live?

Level: Mid · **Roles:** DA, BI

Remember it as: "Simpler" almost always means "I can't find the thing I actually need," not "there's too much visual complexity."

Answer in one line: Ask what they're specifically trying to find or decide when they open it, since "simpler" is usually a proxy for "I can't quickly find what I need," a findability problem, not necessarily a raw information-density problem.

Worked example:

"I'd ask: 'When you say simpler, is it that there's too much on the page, or that it's hard to find the one number you actually check first?' Those need very different fixes. If it's the second, reorganizing what's prominent versus what's secondary might solve it without removing any real information at all."

Tier	What to say
Passes	Asks one question, then removes the least-used charts
Strong	The diagnostic question above, distinguishing a findability problem from a genuine information-overload one
Extra points	+ [+Clarify] one specific question ("too much on the page, or can't find the number you check first?") before touching the dashboard, the same instinct Chapter 76B's Q76B-001 applies to "make the report better": a vague complaint needs one specific clarifying question before any change is made

Likely follow-ups: What if simplifying for this stakeholder makes the dashboard worse for a different stakeholder who uses the same one?

Red flag: treating "simpler" as a literal instruction to delete content, with no diagnosis of the real underlying complaint.

Learn it in: Chapter 24, §24.1 (turning an ask into a question); Chapter 25, §25.3 (from a vague ask to a written requirement); Chapter 15, §15.2 (start from the question); Chapter 16, §16.7 (designing the report page). **Practise it with:** Chapter 76B, Q76B-001, and Chapter 70, §70.8 (Q70-069/070, dashboard critiques).

Rapid-fire, 76A.4

Roles: DA and DS, except where a row names one.

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-018	A DS candidate is asked to explain a model's prediction to someone who's never heard of a p-value. What's the winning move? (DS)	Skip statistical vocabulary entirely; explain in terms of the specific factors driving <i>this</i> prediction, using SHAP-style "this account scored high risk because of X and Y," concrete and case-specific	[+Simple first] one plain sentence first, then the two or three factors, and numbers only if they ask	Mid · 39.8, 44.5
Q76A-019	How would a DA and a DS each respond differently to "can you just change the number"?	The underlying discipline (diagnose before responding) is identical; a DA more often faces this on a reporting number, a DS more often faces a variant of it as "can you make the model say what we expect"	[+Trade-offs] offer an honest alternative they can accept, such as a clearly labelled different cut, or a note explaining the dip	Mid · 24.8 (practise: Q76B-033)

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-020	What's the risk of a DS presenting only the model's strongest result and omitting a weaker segment's performance? (DS)	It's a form of the same overclaiming problem Chapter 39 warns against; a stakeholder who later discovers the omitted weak segment loses trust in every future result, not just that one	[+Limits] put the weak segment in the summary yourself, with what you'd do about it	Senior · 39.9, 44.7
Q76A-021	How would you push back on a stakeholder who wants a specific chart type you think is misleading for their data?	Show, don't just tell: build the alternative chart alongside the requested one and let the clearer version make the case, rather than only arguing abstractly against the original request	[+Trade-offs] say what the requested chart does well before what it hides, so the stakeholder isn't made wrong in front of others	Mid · 15.2, 15.12, 16.7 (practise: 70.8, Q70-069/070)

76A.5 Portfolio deep-dives and handling scrutiny

Q76A-022 · An interviewer picks one number from your resume ("you say this raised conversion from 7% to 12%") and asks you to defend it in detail. What do they actually want?

Level: Senior · **Roles:** DA, DS

Remember it as: A resume number is a claim. The interviewer is checking whether you actually understand how you measured it, not whether the number itself was impressive.

Answer in one line: They want to know exactly how that number was measured (against what baseline, over what time period, with what confounders considered), since a candidate who can't explain their own headline metric in detail either didn't do the analysis rigorously or is inflating the claim.

Worked example: the resume line is "raised conversion on prioritized leads from 7% to 12%", from the project in Q76A-007.

"That figure compares the leads we called first under the scoring rule with the same number of leads called first under the old first-come-first-served order, the quarter before. Comparing the top-scored leads with *all* leads would have flattered it, because top-ranked leads convert better by construction. I also checked conversion across all leads, not just the ones called first, and it rose too, so the gain wasn't just a reshuffle of who got called first. And I checked seasonality: the same quarter a year earlier was flat. What I can't rule out without a holdout is that the reps simply tried harder on the flagged list. The clean test would be to randomly keep half the reps on the old order for a month and compare."

Tier	What to say
Passes	Can restate the number but can't explain the comparison baseline or any confounders considered
Strong	The full explanation above: the baseline, a like-for-like comparison, and at least one confound checked; and it says whether the change is in points or percent: 5 percentage points, about 71% relative (Chapter 4, §4.2)
Extra points	+ [+Validate] naming the selection trap yourself (top-ranked leads would beat the average even if the ranking changed nothing) before the interviewer asks "but could it have been X instead," shows the rigor was applied before the metric was ever put on a resume

Likely follow-ups: How would you know if that 5-point rise wasn't statistically significant, just noise? What would make you distrust your own number here?

Red flag: unable to state the exact baseline a resume metric was measured against, or visible defensiveness rather than genuine engagement with the scrutiny.

Learn it in: Chapter 4, §4.2 (percentage points and percent change); Chapter 22, §22.5 (correlation, causation, and confounders) and §22.7 (selection bias); Chapter 30, §30.8 (designing an experiment); Chapter 31, §31.2 (why the before-and-after comparison misleads). **Practise it with:** Chapter 69, §69.3 (Move 7, validate the result).

Rapid-fire, 76A.5

Roles: DA and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-023	An interviewer finds a small inconsistency between two things you said about the same project. How do you handle it live?	Acknowledge it directly and clarify which version is accurate, rather than doubling down or getting flustered; a small, honestly-corrected inconsistency is far less damaging than visible defensiveness about it	[+Limits] say which version is right and why the other slipped ("earlier I gave the validation number; the test number is this one")	Fresher · 69.3 (Move 10)
Q76A-024	What's the risk of over-preparing a polished, memorized project narrative?	It can sound rehearsed and brittle under a genuine follow-up question that deviates from the memorized script, revealing the narrative was recited rather than truly understood	[+Business] know the underlying facts cold; don't memorize the exact wording	Fresher · 27.10
Q76A-025	An interviewer asks about a project on your resume from three years ago, in specific detail. How do you handle gaps in your memory honestly?	Say plainly what you remember clearly and what's genuinely fuzzy after this long, rather than fabricating confident-sounding detail you don't actually have	Red flag: confidently inventing specifics for a memory gap is far more damaging if caught than an honest "I don't recall that exact detail, but here's what I do remember"	Fresher · 69.3 (Move 10)

#	Question	One-line answer	Extra point	Level · learn it in
Q76A-026	Should you bring a laptop with your actual project code to an interview, in case you're asked to show it?	Generally yes if the interview is portfolio-focused and it's practical, but only code and data you own or that is public (portfolio projects, synthetic data). Never show an employer's code, data or dashboards without permission; say "I can't show that, but here's the same technique on public data"	[+Business] if you do, make sure it actually runs before the interview, nothing undercuts credibility faster than a live demo failing	Fresher · 27.9

76A.6 Full scenarios, talked through live

Q76A-027 · Full scenario, talked through live: an interviewer says "your resume says you 'used machine learning to improve retention.' What does that actually mean?"

Level: Senior · **Roles:** DS

What they're really testing: whether a vague, resume-friendly phrase can be unpacked into the real, specific work behind it, live, without stalling.

Talked through live, start to finish:

"Fair to push on that, it's a compressed phrase covering a few specific things. Concretely: I built a churn prediction model, gradient boosting, trained on one year of account activity, predicting whether an account places no order in the following year. The 'improve retention' part specifically means: I turned the scores into a monthly call list of 40 accounts for the sales team, ranked by expected margin at risk (churn probability × last year's revenue × margin) rather than probability alone, since a large account with a modest risk score is worth more attention than a small account with a high one. What I can't fully claim is that retention definitively improved *because of the model*; what I can say is the accounts on the list were called, and a good share of them didn't churn, but proving causation would need a proper holdout test I haven't run yet, and I'd say that honestly if asked."

Extra-points moves demonstrated: **[+Signpost]** unpacked the vague phrase into its specific, real components rather than restating it more confidently. **[+Business]** distinguished expected-value ranking from probability-alone ranking, exactly Chapter 44's own capstone lesson. **[+Limits]** stated the actual causal limitation honestly rather than overclaiming the resume phrase's implied certainty.

Likely follow-ups: How would you design that holdout test to actually prove causation? What would you change about how you'd phrase this on a resume, now that you've unpacked it?

Red flag: doubling down on the vague resume phrase instead of engaging with the specific unpacking the question is asking for.

Learn it in: Chapter 44, §44.3–44.4 (from probability to rupees, and the call list) and Chapter 30, §30.8 (designing the holdout test). **Practise it with:** Chapter 73, §73.4 (A/B test design).

Common mistakes

Mistake	Symptom	Fix
A project narrative that opens with tools, not the business problem	Interviewer struggles to follow why the work mattered	Business question first, method second, impact last
No quantified impact in a project walkthrough	A vague "it really helped the team" with nothing to probe further	Always have at least a rough number, honestly caveated if it's rough
A rigid DA-vs-DS boundary with no acknowledgment of overlap	Sounds like a textbook definition, not real experience	Name where your own work actually sits, including the overlap
Defending a resume metric with no memory of its actual baseline	Signals the number may have been inflated or poorly measured	Know exactly how every headline number on your resume was calculated
A DS narrative dominated by algorithm names	Misses the framing, evaluation, and judgment an interviewer actually wants to see	Lead with problem framing and evaluation choice, not the model name
Doubling down when caught in a small inconsistency	Reads as defensiveness, damages trust more than the original slip	Acknowledge it plainly and move on
A memorized, over-polished project script	Breaks under a genuine follow-up that deviates from the script	Know the facts; don't memorize the wording

In the real world: the resume line that became the whole interview

Kavita, interviewing for a Data Analyst role after a career transition, has one line on her resume: "Automated a weekly report, saving the team 3 hours a week." The interviewer spends nearly ten minutes on that single line, not because it's the most impressive thing on the page, but because it's specific and checkable, and specific claims are exactly what a good interviewer knows how to probe.

She's asked: which report, what was manual about it before, how exactly was "3 hours" measured, did anyone verify the automated version was actually correct before it replaced the manual one, and what happened the one time it broke. Because she'd genuinely done the work and understood it, not just summarized it for a resume, she has a real, specific answer to each question, including an honest one about the time it broke: a source file's format changed upstream without warning, and she'd since added a validation check specifically because of that failure.

The interviewer's note afterward: *"The resume line was modest. The depth behind it was not. Ten minutes on one bullet point told us more than the rest of the resume combined."* The lesson generalizes: every single claim on a resume should be able to survive exactly this kind of sustained, specific scrutiny, not because interviewers are trying to catch you out, but because depth on a real claim is one of the fastest, most reliable signals of real work versus a polished summary of someone else's.

Project

Goal: prepare two full project narratives, one DA-style and one DS-style (or two of whichever type matches your own background), to the standard this chapter models.

Tools you'll need

No new tools beyond what earlier chapters in this part already cover. The one habit worth building specifically for this chapter: after finishing any real project, write a five-line summary immediately (business question, method, finding, impact number, one honest limitation) while it's fresh, so months later in an interview you're recalling a summary you already wrote carefully once, not reconstructing the whole thing from memory under pressure.

1. For each project, write the five-line summary described above: business question, method, finding, quantified impact, honest limitation.
2. Practice defending your single most impressive resume metric out loud: state its exact baseline, a like-for-like comparison, and one confound you checked, and say whether the change is in points or percent.
3. Write your honest, specific answer to "why this role and not the other one," using Q76A-002's structure.
4. Have someone else interrupt your project walkthrough with an unexpected follow-up question, and practice answering it without falling back on a memorized script.

REVIEW

Key terms

Data Analyst vs. Data Scientist (role distinction) · business-question-first narrative structure · quantified impact · baseline (for a claimed metric) · confound (in a resume claim) · problem framing (DS) · cost-based threshold · honest limitation · portfolio deep-dive · resume scrutiny

Final-week revision list

Q76A-001, Q76A-002, Q76A-007, Q76A-012, Q76A-017, Q76A-022, Q76A-027.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 76B, Business Analyst Question Bank**, covers the same "diagnose before responding" and stakeholder-pushback discipline from the BA angle.
- **Chapters 36–39** (lead scoring) and **Chapter 44** (the churn call list) are the projects behind this chapter's DS examples; if you did them, you can tell these stories as your own practice work.
- **Chapter 81, Behavioral, HR & Offer Conversations**, is where this chapter's project-narrative skills extend into full STAR-format behavioral stories.

Chapter 76B. Business Analyst Question Bank

Chapter at a glance

You will learn to: answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews · tell a BRD, FRD, and SRS apart without hesitating · write a user story with real acceptance criteria, not a vague wish · map a process the way a BA actually would, with decision points and swimlanes · handle "can you just change the number?" and other stakeholder pushback without folding or getting defensive.

Before you start: Chapter 24 (all of it: turning an ask into a question, business rules, stakeholders, and pushback) and Chapter 25 (all of it: the business analyst track), which between them teach every technique this bank tests; Chapter 26, section 26.10 (Agile, Scrum, and the backlog); Chapter 3, section 3.2 (order 5001's journey from enquiry to cash, which the worked examples use); and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 2.5–3 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row), plus 4–6 hours for the project: two interviews, a swimlane map, a story, and a requirement.

How this chapter is built. Same format as every question bank in Part 8. Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, tagged the same way: **[+Clarify]**, **[+Edge cases]**, **[+Validate]** and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it (a bare number such as 25.8 means that section). This chapter has no code to execute; a requirements document, a process map, and a user story are judged on completeness and clarity, so every worked example here is a full, concrete artifact, not a description of one. The worked examples use Riverstone's order-to-cash process, the same one Chapter 25 maps, with the same steps, requirement numbers, and figures.

Levels and roles. Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **BA** business analyst · **DA** data analyst · **BI** BI developer. The basic-but-tricky questions come first; if you're aiming at your first BA job, start with section 76B.1 and the Fresher questions, then work through the rest.

In this chapter

LEARN	76B.1 Basic-but-tricky BA questions	257
	76B.2 Requirements gathering	259
	76B.3 User stories and acceptance criteria	261
	76B.4 Process mapping	263
	76B.5 SDLC and where a BA fits	265
	76B.6 Gap analysis and UAT	267
	76B.7 Stakeholder management and pushback	269
	76B.8 Full scenarios, talked through live	270
APPLY	Common mistakes · In the real world · Project	271
REVIEW	Key terms · Final-week revision list	273
NEXT	Where this leads	273

LEARN

76B.1 Basic-but-tricky BA questions

Q76B-007 · What's the difference between a BRD, an FRD, and an SRS?

Level: Fresher · **Roles:** BA, DA

Remember it as: BRD is for the business, FRD is for the build, SRS is for the engineers, each one more technical and more detailed than the last.

Answer in one line: A **BRD** (Business Requirements Document) captures the business need and goals, written for sponsors, executives, and finance, readable by someone who will never log in; an **FRD** (Functional Requirements Document) translates them into numbered functional requirements, process flows, business rules, and data definitions, written for developers and testers (detailed enough that two developers reading it build the same thing); an **SRS** (Software Requirements Specification) is the most technical of the three, detailed enough for a vendor to build from without talking to you, adding interfaces, non-functional requirements, data models, constraints, and error handling.

Tier	What to say
Passes	Knows all three acronyms exist, can't distinguish their actual audience or level of detail
Strong	The audience-and-detail-level distinction above, with a one-line example of the same requirement expressed at each level
Extra points	[+Edge cases] not every organization uses all three; some combine BRD and FRD into one document, or skip a formal SRS entirely and go straight to user stories (section 76B.3) for agile teams: naming that this varies by organization shows real experience, not just memorized definitions

Likely follow-ups: Which of the three would a project sponsor sign off on? Which would a developer actually build from?

Red flag: treating the three as interchangeable synonyms.

Learn it in: Chapter 25, section 25.7 (the three documents, and who reads each).

Q76B-008 · Is a user story the same thing as a requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: A user story is a conversation starter with acceptance criteria attached. A requirement can be a much heavier, more formal document. They're related, not identical.

Answer in one line: Not exactly: a **user story** is a short, deliberately incomplete description of a need from a user's perspective ("As a [role], I want [goal], so that [benefit]"), meant to prompt a conversation and get refined collaboratively with the team; a **requirement** in the traditional sense is meant to be complete and unambiguous on its own, without needing further conversation to be understood.

Tier	What to say
Passes	Treats "user story" and "requirement" as interchangeable words for the same thing
Strong	The completeness distinction above: a user story is intentionally a placeholder for a conversation, not a finished specification
Extra points	[+Business] this is exactly why a user story needs acceptance criteria (Q76B-013) attached before it's actually "done": the criteria are what turn the conversation starter into something testable, closing the gap between the two

Likely follow-ups: In what kind of project would you use formal requirements documents instead of user stories, or both together?

Red flag: treating a bare user story ("As a user, I want a better dashboard, so that I can see my data") as if it were already a complete, actionable requirement.

Learn it in: Chapter 25, section 25.8 (use cases and user stories).

Rapid-fire, 76B.1

Roles: BA for every row; DA for Q76B-010 and Q76B-011.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-009	Use case vs. user story?	A use case describes a full interaction between an actor and a system, including alternate and exception flows, in more formal, structured detail / a user story is a short, single-sentence expression of a need	[+Business] use cases suit complex, multi-step system interactions; user stories suit iterative, conversation-driven agile work	Fresher · 25.8
Q76B-010	What's an assumption, in a requirements document, and why document it explicitly?	Something taken as true without confirmation, stated openly so it can be challenged or validated before it silently becomes a wrong foundation for the whole project	[+Assume] Chapter 69's Move 2 (state assumptions), the identical discipline applied to formal documentation instead of a live answer	Fresher · 25.7 (what a BRD holds); 69.3, Move 2

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-011	What's the difference between "in scope" and "out of scope," and why does an explicit out-of-scope list matter?	In scope is what the project will deliver; out of scope is what it explicitly will not, stated up front to prevent scope creep and manage expectations before, not after, stakeholders assume otherwise	[+Business] an unstated out-of-scope item is the single most common source of a stakeholder feeling blindsided at delivery	Fresher · 25.7; 24.1 (question 3)
Q76B-012	What's a stakeholder register?	A structured list of everyone with an interest in the project, their role, their influence, and how and when they need to be communicated with: the power-interest grid written down as a table	[+Business] distinguishing high-influence, high-interest stakeholders (manage closely) from low-influence, low-interest ones (monitor only) is the core of practical stakeholder prioritization	Fresher · 24.3

76B.2 Requirements gathering

Q76B-001 · A stakeholder says "make the report better." How do you turn that into an actual requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: "Better" is a feeling, not a requirement. Your job is to find the specific, checkable thing hiding behind the vague word.

Answer in one line: Ask specifically what problem the current report causes them, day to day, and work backward from that concrete pain point to a testable requirement, rather than guessing at what "better" might mean.

Worked example, a real elicitation dialogue:

Analyst: "When you say 'better,' can you walk me through the last time the current report let you down?"

Stakeholder: "Last Tuesday I needed last week's numbers by region, and I had to open it in Excel and build a pivot table myself, it took twenty minutes."

Analyst: "So the requirement isn't 'better,' it's specifically: the report needs a region-level breakdown, viewable without any manual rework, available same-day."

Tier	What to say
Passes	Asks what "better" means, gets "faster, clearer", and writes that down with one follow-up question
Strong	Asks for a specific recent example of the pain, then restates it as a testable requirement in the stakeholder's own words
Extra points	[+Business] the fix here isn't asking more questions in general, it's asking for a <i>specific instance</i>, since people are far better at describing one real moment than defining an abstract concept like "better" [+Validate] restating the requirement back to the stakeholder in one sentence and getting a "yes, exactly" confirms it was captured correctly before any work starts

Likely follow-ups: What if the stakeholder can't recall a specific example? How would you prioritize this requirement against three other vague asks from different stakeholders?

Red flag: documenting "make the report better" as the requirement itself, or guessing at a specific fix with no elicitation at all.

Learn it in: Chapter 24, section 24.1 (turning an ask into a question) and Chapter 25, section 25.3 (the five moves from a vague ask to a written requirement).

Q76B-002 · What's the difference between a business requirement, a functional requirement, and a non-functional requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: *Business requirement: why we're doing this. Functional requirement: what the system must do. Non-functional requirement: how well it must do it.*

Answer in one line: A **business requirement** states the underlying business goal ("reduce time spent reconciling orders"); a **functional requirement** states what the system must specifically do to support that goal ("the system shall flag any order with a quantity mismatch between the PO and the invoice"); a **non-functional requirement** states a quality constraint on how it does that ("the flag must appear within 2 seconds of the invoice being uploaded").

Tier	What to say
Passes	Can define one of the three, conflates the other two
Strong	All three definitions above, with a matched example showing how one business requirement can produce several functional and non-functional requirements underneath it
Extra points	[+Edge cases] a common real mistake: writing a functional requirement that's actually a design decision in disguise ("the system shall use a dropdown menu" describes a UI choice, not a requirement; "the system shall let a user select from a predefined list of statuses" is the actual requirement, implementation-neutral) [+Edge cases] names a data-specific non-functional requirement unprompted: freshness ("no more than two hours old during working hours") or reconciliation ("monthly revenue reconciles to the ERP to the rupee"), the ones that get dropped and then cause an incident

Likely follow-ups: Give a non-functional requirement that isn't about speed. Why does mixing requirement types in one document cause problems later?

Red flag: writing an implementation detail (a specific button, a specific dropdown) as if it were the requirement itself.

Learn it in: Chapter 25, section 25.5 (the three kinds, and the table of non-functional requirements specific to data).

Rapid-fire, 76B.2

Roles: BA for every row; DA for Q76B-004 to Q76B-006.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-003	What's a business rule, and how does it differ from a requirement?	A business rule is a statement of how the business works, independent of any system. Two kinds matter to a BA: constraints ("orders over ₹5 lakh need manager approval") and definitions ("active customer = a non-cancelled order in the trailing 90 days"). A requirement says what a system must do, often to enforce or compute one	[+Business] business rules usually outlive any one system; requirements are tied to the specific project building or changing that system	Mid · 24.2; 23.13 (the six-part template)
Q76B-004	What's elicitation, as a formal BA term?	The structured process of drawing out requirements from stakeholders, via interviews, workshops, document analysis, observation, or surveys, rather than requirements simply being handed over complete	[+Business] different elicitation techniques suit different situations: a workshop surfaces conflicting stakeholder views fast; direct observation catches things nobody thinks to mention because it's "just how we do it"	Fresher · 25.1 (the four verbs); 25.3
Q76B-005	Why interview more than one stakeholder for the same process?	Different roles see different parts of the process and often disagree about how it actually works, even for something they'd all claim to understand the same way	[+Evidence] a sales rep and a finance controller frequently describe the exact same order-approval process differently, and the gap between their descriptions is itself useful information	Fresher · 24.3; 25.4 ("the room starts talking")
Q76B-006	What's a requirements traceability matrix, in one sentence?	A table linking each requirement back to its business need and forward to the test case that verifies it, so nothing gets built without a reason or shipped without being checked	[+Business] it's the tool that answers "why are we building this" and "how do we know it worked" for every single requirement, at once	Mid · 25.7

76B.3 User stories and acceptance criteria

Q76B-013 · Write a complete user story with acceptance criteria for: "sales reps want to know which of their accounts are at risk of churning"

Level: Mid · **Roles:** BA, DA, BI

Remember it as: A user story with no acceptance criteria is a wish. Acceptance criteria are what turn a wish into something a developer can build and a tester can check off.

Answer in one line: The story states the role, the goal, and the benefit in one sentence; the acceptance criteria list the specific, testable conditions that must be true for the story to be considered done, including the actual rule, never a word like "recently" or "the defined threshold".

Worked example, the at-risk list Chapter 25 specifies (its FR-01 to FR-03):

User story: As a sales rep, I want to see which of my accounts are at risk of not ordering again, so that I can call them before they go quiet.

Acceptance criteria:

- **AC-1 (scope).** Given I am a logged-in sales rep, when I open the at-risk list, then I see only accounts where I am the assigned rep.
- **AC-2 (the rule).** Given an account with at least four days of orders whose time since its last order is more than 1.5 times its own average gap between orders, when the list is generated, then that account appears on it with an "At Risk" flag.
- **AC-3 (edge case).** Given an account with fewer than four days of orders, including one with a single order or none, when the list is generated, then it is not flagged and is counted separately as "too new to judge".
- **AC-4 (empty state).** Given I have no at-risk accounts, when I open the list, then I see a clear "no accounts currently at risk" message, not an empty or broken-looking screen.
- **AC-5 (reconciliation).** Given the list is refreshed, when the twelve-month revenue it shows for an account is compared with the ERP's figure for the same months, then the two agree to the rupee.

The rule in AC-2 isn't hypothetical. Run against Riverstone's 24 key accounts as of 31 December 2025, it flags 3 of them, and 7 are too new to judge (Chapter 25, section 25.6). That's why AC-3 exists: seven accounts that no rule is watching is a finding in itself.

Tier	What to say
Passes	A story plus one happy-path criterion, no edge case
Strong	The story plus Given/When/Then criteria for the scope, the actual rule, and at least one edge case
Extra points	[+Edge cases] the empty state (AC-4) is exactly the kind of edge case a strong BA adds unprompted: what happens when there's <i>nothing</i> to show is easy to forget and easy for a developer to leave broken if it's never specified [+Validate] a reconciliation criterion (AC-5), the fourth kind of criterion for data, and the one people new to data work miss [+Limits] ties the list to a real technical constraint: it's only as fresh as its data, which may be up to one working day old (Chapter 25's NFR-01), so a flag doesn't change the instant a customer orders

Likely follow-ups: How would you prioritize this story against three others in a sprint? What would make you split this into two smaller stories instead of one? What changes in AC-2 if the flag comes from a churn model's score instead of a rule?

Red flag: acceptance criteria so vague they can't actually be tested ("the dashboard should work well", or "flag accounts above the defined threshold" with no threshold).

Learn it in: Chapter 25, section 25.8 (Figure 25.3's story and its criteria, and the reconciliation criterion AC-5) and section 25.6 (the rule, and the query that counts it).

Rapid-fire, 76B.3

Roles: BA for every row; DA and BI for Q76B-014 and Q76B-016.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-014	What does "Given/When/Then" mean in acceptance criteria?	Given (the starting context) / When (the action taken) / Then (the expected result), a structured format (called Gherkin) for writing testable criteria	[+Business] this structure is what lets a QA engineer write a test case almost directly from the acceptance criteria with no further translation	Fresher · 25.8
Q76B-015	What's the INVEST mnemonic for a good user story?	Independent, Negotiable, Valuable, Estimable, Small, Testable	[+Edge cases] a story that fails "Independent" (it can't be built without another story finishing first) often signals it should be resequenced or the dependency made explicit	Mid · 25.8; 26.10
Q76B-016	Why should a user story avoid specifying a particular UI element?	The story should describe the need, not the implementation, so the team retains freedom to choose the best actual design; specifying a "dropdown" locks in a decision that isn't the BA's or the story's to make	[+Signpost] name the rule behind it: it's Q76B-002's line between a requirement and a design decision, applied to a story	Fresher · 25.8; 25.5
Q76B-017	What's a spike, in agile terminology?	A time-boxed research or investigation task, done specifically because a story can't be sized or written properly until an open technical question is answered first	[+Business] a spike is the honest alternative to guessing at a story's size or acceptance criteria when genuine uncertainty exists	Mid · 26.10; 25.2

76B.4 Process mapping

Q76B-018 · Map Riverstone's order-to-cash process at a high level, and explain what a swimlane adds that a simple flowchart doesn't

Level: Mid · **Roles:** BA, DA

Remember it as: A flowchart shows what happens. A swimlane shows what happens AND whose job it is, which is usually the more useful question in a real business process.

Answer in one line: A basic flowchart shows the sequence of steps; a **swimlane diagram** adds horizontal or vertical lanes, one per role or department, so each step visibly sits inside the lane of whoever's responsible for it, immediately surfacing handoffs between departments, which is usually exactly where a real process breaks down.

Worked example, order-to-cash for Riverstone's order 5001, swimlane by role (the numbers are Chapter 3's ten steps):

Customer lane: 1 enquiry

Sales lane: 2 visit and agree terms → 3 quote → 4 order typed into the ERP

Warehouse lane: 5 stock check and reserve → 6 pick, pack, ship → 8 signed proof of delivery scanned

Finance lane: 7 invoice → 9 payment matched to the invoice → 10 monthly sales report

Tier	What to say
Passes	A flowchart with the right steps in order, with a note on who does each, but no lanes and no handoffs called out
Strong	The swimlane version above, correctly separating steps by which team owns them
Extra points	[+Business] the handoff <i>between</i> lanes (sales → warehouse, warehouse → finance) is almost always where delays and errors concentrate in a real process, an insight a plain flowchart hides entirely by not distinguishing ownership at all [+Evidence] on order 5001, five of the nine transitions cross a lane, and each crossing is where a data-quality problem is born: re-typed product codes at step 4, missing delivery dates at step 8 [+Edge cases] a real map would also show what happens on an exception path (a rejected order, a payment dispute), not just the clean happy path shown above

Likely follow-ups: Where in this process would you look first for a delay-reduction opportunity, and why? What would a BPMN version of this add beyond a plain swimlane diagram?

Red flag: a process map showing only the happy path, with no exception handling at all.

Learn it in: Chapter 25, section 25.4 (Figure 25.1, the swimlane of order 5001, and the lane table under it); the ten steps are Chapter 3, section 3.2.

Rapid-fire, 76B.4

Roles: BA for every row; DA for Q76B-021 and Q76B-022.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-019	What's a decision point (or gateway) in a process map?	A point where the process branches based on a condition (approved vs. rejected, in-stock vs. backorder), shown as a diamond in most notations	[+Edge cases] every decision point needs every branch mapped, including the "unhappy" ones; a map that only shows the approved path is incomplete	Fresher · 25.4
Q76B-020	What does BPMN stand for, and what does it add over a generic flowchart?	Business Process Model and Notation, a standardized symbol set (specific shapes for events, gateways, tasks, and messages between participants) that makes a process map unambiguous across different tools and readers	[+Business] standardization matters most when a process map needs to be handed to an external vendor or IT team who wasn't in the room when it was drawn	Mid · 25.4

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-021	What's the difference between an "as-is" and a "to-be" process map?	As-is documents the process exactly as it currently, actually works, warts and all; to-be documents the proposed, improved version	[+Edge cases] skipping the as-is map and jumping straight to to-be is a common mistake: you can't credibly propose an improvement to a process you never actually documented as it really runs	Fresher · 25.4
Q76B-022	Why might the as-is process, once actually mapped, surprise the people who work in it every day?	People executing a process day to day often only know their own step; mapping the whole thing end to end frequently reveals workarounds, redundant approvals, or handoffs nobody had the full picture of before	[+Evidence] this is one of the most consistently reported "aha" moments in real BA work: the map itself is often the deliverable that creates the most value, before any redesign happens at all	Mid · 25.4

76B.5 SDLC and where a BA fits

Q76B-023 · Walk through the phases of the SDLC, and say specifically what a BA does in each

Level: Mid · **Roles:** BA

Remember it as: A BA's role doesn't stop after requirements are written. It runs the whole length of the SDLC, in a different form at every stage.

Answer in one line: Across the six phases, the BA does all of **Requirements** (elicitation, process mapping, gap analysis, writing and agreeing the requirements), defends the requirement in **Design** when the design quietly drops part of it, stays available in **Build** to answer the questions that would otherwise be guessed at, writes or reviews the UAT plan and prepares the users in **Test**, facilitates UAT in **Deploy** while real users run it, then confirms sign-off and hands over, and in **Maintain** triages whether an issue is a genuine defect or a new requirement in disguise. The phase names vary by company; the sequence doesn't.

Tier	What to say
Passes	Names the SDLC phases correctly, describes the BA's role as "writing requirements" with no involvement in the other phases
Strong	The phase-by-phase involvement above, correctly identifying that a BA's role changes shape but doesn't disappear after requirements are handed off
Extra points	[+Business] the triage role at the maintenance stage (is this a bug, or a new requirement someone's calling a bug) is a genuinely common, underappreciated part of the job, and naming it specifically signals real experience rather than a textbook answer [+Edge cases] the BA facilitates UAT but doesn't drive it: if the BA drives the mouse, the test measures the BA, not whether the product meets the users' need

Likely follow-ups: How does a BA's role differ in a Waterfall project versus an Agile one? What happens when a developer's question during the build phase reveals the original requirement was actually ambiguous?

Red flag: describing the BA role as ending once requirements are signed off.

Learn it in: Chapter 25, section 25.2 (the six phases and the BA's part in each) and section 25.10 (who runs UAT).

Rapid-fire, 76B.5

Roles: BA for every row; DA and BI for Q76B-024 to Q76B-026.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-024	Waterfall vs. Agile, from a BA's perspective?	Waterfall: requirements are finalized once, up front, before development starts / Agile: requirements evolve iteratively across sprints, with the BA continuously refining the backlog	[+Business] most real organizations run a hybrid, not a pure version of either, worth naming rather than presenting the two as a strict binary	Fresher · 25.2; 26.10
Q76B-025	What's the difference between the product backlog and the sprint backlog, and who owns each?	The product backlog is everything not yet started, in priority order, owned and ordered by the Product Owner (the person accountable for what gets built and in what order); the sprint backlog is what the team pulled into this sprint, plus its plan, owned by the developers who'll do it. The BA feeds the product backlog with refined, testable stories	[+Edge cases] a BA and a Product Owner role can overlap significantly or be entirely separate people depending on the organization	Mid · 26.10; 25.2
Q76B-026	What's a Definition of Done, and why does a team need to agree on one explicitly?	A team-wide, explicit checklist of what "finished" means for any story (code reviewed, tested, documentation updated, and so on), agreed once so it doesn't get silently redefined story by story	[+Business] without an explicit Definition of Done, "done" quietly means something different to the developer, the BA, and the stakeholder, and that gap surfaces late, expensively	Mid · 26.10; 25.8
Q76B-027	What's a change request, and why does even a small one need a formal process?	A formally logged request to modify already-approved scope, requirements, or a signed-off deliverable; a formal process (however lightweight) protects against silent scope creep and keeps a record of who approved what	[+Business] the process doesn't need to be heavy for a small change, but skipping it entirely for "just this once" is exactly how scope creep starts	Mid · 25.11 and its Common mistakes ("agreeing a scope change in a corridor"); 63.9

76B.6 Gap analysis and UAT

Q76B-028 · What's a gap analysis, and walk through one for Riverstone's order-to-cash process

Level: Mid · **Roles:** BA, DA

Remember it as: Gap analysis is just "where are we now" minus "where do we need to be," made explicit and specific enough to act on.

Answer in one line: A **gap analysis** compares the current (as-is) state against the desired (to-be) state, identifying the specific differences that need to be closed and the root cause of each, each one becoming a candidate requirement or improvement initiative.

Worked example, the delivery step (step 8) of Riverstone's order-to-cash process:

As-is: A driver brings back a paper proof of delivery. Someone at Bhiwandi Main scans it, emails it to finance, and changes the order to *Delivered* by hand, days later or not at all. The ERP holds no delivery date.

To-be: The delivery is recorded in the ERP when the customer signs, with the date and time.

Gap: Nothing records the delivery when it happens; the ERP hears about it later, from a scanned email.

Root cause: Not "the warehouse is slow". The only record of the delivery is a piece of paper at the customer's site: the driver has no device, so the fact travels back on the truck before anyone can type it in.

Resulting requirement: FR-14: the delivery status and date shall be recorded in the ERP within one hour of the proof of delivery being signed. It depends on NFR-09: a delivery recorded where there is no mobile signal shall reach the ERP within one hour of the signal returning, and none shall be lost.

Tier	What to say
Passes	Defines gap analysis correctly in the abstract, can't apply it to a concrete example
Strong	The full worked example above: as-is, to-be, the specific gap, its root cause, and the requirement it produces
Extra points	[+Business] the gap analysis doesn't stop at identifying the difference, it names the <i>root cause</i> (no device where the delivery happens, not "people are slow"), which is what actually makes the resulting requirement solve the real problem rather than a symptom of it [+Validate] asks "why" once more than feels necessary: telling the warehouse to scan faster would leave the date arriving days late anyway, so FR-14 can't work without NFR-09, the requirement that the recording survive a missing signal

Likely follow-ups: How would you prioritize this gap against several others found in the same analysis? What would you check before assuming a device for every driver is actually the right fix?

Red flag: a gap analysis that identifies a difference but never traces it to an actionable, specific requirement, or that names the symptom as the root cause.

Learn it in: Chapter 25, section 25.9 (Figure 25.4 is this row, and the ranking table scores it against two other gaps); the step itself is Chapter 3, section 3.2, and section 3.7 lists what goes wrong there.

Q76B-029 · What's UAT, who does it, and why can't developers or QA alone replace it?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: UAT isn't "does the system work." It's "does the system actually do what the business needed," and only the business can really answer that.

Answer in one line: **User Acceptance Testing** is the final validation stage, done by real business users (not developers or QA), checking whether the delivered system actually meets the original business need, not just whether it's technically functioning correctly, since a system can pass every technical test while still failing to solve the real problem it was built for.

Tier	What to say
Passes	"UAT is when users test the system before launch" (correct, misses <i>why</i> it can't be skipped or delegated to QA)
Strong	The distinction above: QA verifies the system works as specified; UAT verifies the specification itself was actually right, which only the people with the original business need can judge
Extra points	[+Business] a system can pass 100% of its QA test cases and still fail UAT, because the test cases were built from a requirement that, it turns out, didn't fully capture what the business actually needed, exactly the kind of gap Q76B-001's elicitation discipline is meant to prevent upstream [+Validate] for a data product, UAT has a second job: reconcile at least one complete period against a source the business already trusts (last month's closed figures, the ERP's own report), at more than one level of aggregation, because a total can match while the breakdown doesn't

Likely follow-ups: What would you do if UAT reveals the original requirements were wrong, this late in the project? How would you structure a UAT test plan?

Red flag: treating UAT as a redundant, box-checking repeat of QA testing.

Learn it in: Chapter 25, section 25.10 (UAT, and the two halves of a data UAT plan).

Rapid-fire, 76B.6

Roles: BA for every row; DA and BI for Q76B-030.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-030	What's a UAT test case, and how does it differ from a QA test case?	Written from the business user's real workflow and language, checking business outcomes / QA test cases are written from the technical specification, checking system behavior	[+Validate] well-written acceptance criteria (Q76B-013) are already UAT test cases: they can be handed to a user as they stand	Mid · 25.10; 25.8
Q76B-031	What happens if UAT fails close to a planned go-live date?	The defect gets triaged: is it a genuine bug (fix and retest), a missed requirement (assess impact and possibly delay), or a misunderstanding of scope (clarify and possibly defer to a later phase)	[+Business] the temptation to launch anyway and "fix it after" needs to be weighed honestly against the real cost of the specific gap found, not treated as automatically acceptable or automatically a blocker	Senior · 25.10

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-032	Who signs off on UAT completion?	Typically the business stakeholder or sponsor who owns the requirement, not IT or the project manager, since sign-off is specifically confirming the business need was met	[+Business] this sign-off is the formal moment accountability for "does this actually work for the business" transfers, worth taking seriously rather than as a formality	Mid · 25.10

76B.7 Stakeholder management and pushback

Q76B-033 · A stakeholder says "can you just change the number to make it look better?" How do you respond, live?

Level: Mid · **Roles:** BA, DA, BI

Remember it as: *Don't refuse and don't comply silently. Ask what's actually driving the request, then respond to that, not to the literal words.*

Answer in one line: Ask what's behind the request (are they worried the number looks bad to their own boss, do they suspect the number itself is wrong, is there missing context that would explain it) before either agreeing or refusing, since the literal request is almost never actually what they need.

Worked example, talked through live:

"I'd say: 'I want to understand what's driving that, can you tell me more? If the number's wrong, I'll absolutely fix it, and I'd want to find the root cause. If it's right but looks concerning, I can add context that helps explain it rather than changing the number itself, since changing a correct number isn't something I can do.' Most of the time what they actually want is help explaining a real number, not a fabricated one, and naming that directly usually resolves it without an awkward confrontation."

Tier	What to say
Passes	Explains politely that the number is correct and can't be changed, and offers to double-check it, but doesn't ask what's driving the request or offer an alternative
Strong	The diagnostic question above, distinguishing "the number might be wrong" from "the number is right but needs context" before responding either way
Extra points	[+Business] offering a legitimate alternative (adding context, a footnote, a caveat) gives the stakeholder something real they can use, rather than leaving them with nothing after a flat refusal, which is often what actually causes the request to escalate or repeat

Likely follow-ups: What if the stakeholder insists after you've explained you can't change a correct number? How would you document this exchange, if at all?

Red flag: quietly changing a correct number to avoid conflict: altering a correct figure is an integrity failure, not a stakeholder-management choice. A defensive, unhelpful flat refusal with no attempt to

solve the real underlying concern is the lesser failure.

Learn it in: Chapter 24, section 24.8 (the decision flow for "can you just change the number?", Figure 24.4).

Rapid-fire, 76B.7

Roles: BA, DA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-034	Two stakeholders give you contradicting requirements. What do you do?	Don't silently pick one; bring both stakeholders together (or relay both positions clearly) so the conflict is resolved explicitly, with a documented decision, not quietly absorbed by the BA	[+Business] an undocumented, silently-resolved conflict tends to resurface later when the stakeholder who "lost" notices the outcome	Mid · 24.3 (RACI: who is accountable); 24.8; 25.11
Q76B-035	A powerful stakeholder wants a feature that conflicts with the actual data or user needs. How do you push back?	Lead with evidence, not opinion: show the specific data or user feedback that conflicts with the request, and frame it as a shared problem to solve together, not a personal disagreement	[+Signpost] bottom line up front, one message: Chapter 24's storytelling discipline applied to a difficult conversation instead of a report	Senior · 24.4; 24.8
Q76B-036	What's the risk of only communicating with the most senior stakeholder on a project?	Requirements end up shaped by whoever has the most authority to speak up, not necessarily whoever has the most accurate day-to-day knowledge of the actual process	[+Edge cases] the people closest to the work know the exceptions and the workarounds; that's Q76B-005's reason for interviewing more than one person	Mid · 24.3; 25.4
Q76B-037	How would you communicate a delay to a stakeholder who's expecting on-time delivery?	As early as possible, with the specific reason, the revised timeline, and what's being done about it, not a vague "it's taking longer than expected" with no further detail	[+Business] early, specific bad news is almost always received better than late, vague bad news	Fresher · 24.7

76B.8 Full scenarios, talked through live

Q76B-038 · Full scenario, talked through live: elicit requirements for automating Riverstone's payment matching

Level: Senior · **Roles:** BA, DA

What they're really testing: whether a full elicitation-to-requirement cycle can be run start to finish, live, not just described as a concept. The process is step 9 of order-to-cash: a finance assistant reads each bank statement line and works out which invoice it pays.

Talked through live, start to finish:

"First, I'd ask finance to walk me through exactly how they match a payment today, step by step, including what they do when a line doesn't match, since the exception path usually matters more than the happy path. Say they tell me: they read each line on the bank statement, look up that customer's open invoices in the ERP, and record the payment against the one it pays; when the amount doesn't equal any open invoice, or the reference is as thin as 'SHARMA HW JAN', they work it out by eye or ask sales. I'd ask about the awkward cases by name: a customer paying one invoice in two instalments, or one transfer covering two invoices. I'd ask how often a line can't be matched cleanly, since if exceptions are rare, automation removes most of the work, and if they're common, it doesn't. Say it's about 15% of lines. At 15% exceptions, automation handles about 85% of lines unattended, which is worth building, as long as the other 15% go to a named person. I'd then ask what 'match' actually means precisely, exact to the paisa, or within some tolerance for rounding, since 'close enough' matching is a very different, harder requirement than exact matching. From there I'd draft a first-pass requirement: 'The system shall compare each imported bank line with the customer's open invoices and record a match when customer and amount agree within ₹1; any other line shall go to [named person] for review; every automatic match shall record the rule and time that produced it,' and take that back to finance to confirm before writing it up formally."

Extra-points moves demonstrated: **[+Clarify]** asked about the exception path, not just the happy path, unprompted. **[+Business]** asked for the exception frequency specifically because it changes the case for automating this at all, and said what 15% implies. **[+Scale]** asked for an audit trail and a named exception owner unprompted, the two additions most often forgotten when a person's job is automated. **[+Validate]** ended by proposing to confirm the draft requirement with the stakeholder before treating it as final, rather than writing it up unilaterally.

Likely follow-ups: How would you write the acceptance criteria for this requirement? What would change about your approach if this were a brand-new process instead of automating an existing manual one?

Red flag: drafting a requirement after asking only about the happy path, with no question about what happens when something doesn't match.

Learn it in: Chapter 25, section 25.3 (the five moves, run live here) and section 25.12 (specifying an automation: BR-05, BR-06, FR-15 to FR-17, and the three additions); the step is Chapter 3, section 3.2, step 9.

APPLY

Common mistakes

Mistake	Symptom	Fix
Documenting a vague ask as the requirement itself	Nobody can tell what "done" actually means	Ask for a specific example of the pain, restate as a testable requirement

Mistake	Symptom	Fix
Treating BRD/FRD/SRS or use case/user story as interchangeable	Wrong document for the audience, confusion downstream	Know the audience and detail level each one is for
A user story with no acceptance criteria	The story can't actually be tested or marked done	Always attach Given/When/Then criteria, including edge cases
A process map showing only the happy path	Exception handling gets built as an afterthought, or not at all	Map every branch at every decision point, including rejections
Assuming the BA's job ends once requirements are signed off	The delivered system quietly drifts from the original need	Stay involved through design, build, and UAT
Skipping the as-is map and jumping straight to to-be	A proposed improvement that doesn't actually address the real current process	Always document as-is first, however tempting it is to skip
Silently complying with (or flatly refusing) "can you just change the number"	Either damaged data integrity or an unresolved, escalating stakeholder relationship	Diagnose what's actually driving the request before responding

In the real world: the requirement that was actually three requirements

Farah, now a data analyst whose first project turned out to be requirements work, is asked to gather requirements for "a way to see which customers haven't ordered recently." She resists the urge to write that sentence down as the requirement and instead asks three different stakeholders, separately, what they'd actually do with that information.

Sales wants it to prioritize outreach calls, and only cares about customers likely to still be recoverable. Finance wants it for a different reason entirely: flagging dormant accounts that still have open balances for a bad-debt review, and closing their credit limits, with no interest in whether they're "recoverable." Customer success wants it for a third reason: proactively checking in with accounts before they become a support escalation, regardless of order recency alone.

What looked like one requirement was genuinely three, each needing a different definition of "haven't ordered recently," a different output format, and a different owner. Farah's actual deliverable ends up being three separate, precisely scoped requirements instead of one vague one, and the project's initial one-week estimate for "the recency report" becomes a more honest three-week estimate for three real features. Her manager's note: *"The original ask would have shipped something nobody could actually use well. Three real requirements, properly scoped, took longer to define but were faster to build right the first time."*

Project

Goal: run this chapter's full method on a real process, start to finish.

Tools you'll need

No software specific to this chapter. A whiteboard or a simple diagramming tool (Lucidchart, draw.io, or even PowerPoint) for process maps; a shared document for BRDs, FRDs, and user stories; Chapter 11's spreadsheet skills (the same ones Chapter 25, section 25.7 uses for a traceability matrix) for a requirements traceability matrix in practice.

1. Pick a real, recurring process you're familiar with (at work or elsewhere), and interview at least two people involved in different steps of it.
2. Map it as-is, including at least one exception path, using a swimlane format.
3. Write one user story with full Given/When/Then acceptance criteria for a specific improvement to that process.
4. Draft the requirement, in one sentence, that a gap analysis of the same process would produce.
5. Write the "can you just change the number" response you'd actually give, for a real or hypothetical version of that request in your own context.

REVIEW

Key terms

elicitation · business requirement · functional requirement · non-functional requirement · business rule · requirements traceability matrix · stakeholder register · BRD · FRD · SRS · user story · use case · INVEST · Given/When/Then (Gherkin) · reconciliation criterion · Definition of Done · spike · swimlane diagram · handoff · BPMN · decision point (gateway) · as-is vs. to-be · gap analysis · root cause · SDLC · product backlog vs. sprint backlog · Product Owner · change request · UAT · UAT sign-off

Final-week revision list

Q76B-001, Q76B-002, Q76B-007, Q76B-008, Q76B-013, Q76B-018, Q76B-023, Q76B-028, Q76B-029, Q76B-033, Q76B-038.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against, especially Move 1 (clarify) throughout sections 76B.2 and 76B.8.
- **Chapter 75, Product Sense, Metrics, Case Studies & Guesstimates**, already covers case-structure and stakeholder reasoning from the diagnostic-case angle; this chapter covers the same discipline from the documentation and process side.
- **Chapters 24 and 25** teach every technique this bank tests, and Chapter 26, section 26.10 the Agile vocabulary; each question's "Learn it in" names the section.
- **Chapter 78, Automation & Integration Question Bank**, takes automation from the requirement (Q76B-038) to the build: choosing the tool, retries, idempotency, and alerting.

Chapter 77. Data Engineering & Data System Design Bank

Chapter at a glance

You will learn to: answer the pipeline, data-modeling, and system-design questions that come up in Data Engineer interviews · design a batch or streaming pipeline live, under time pressure, the way an interviewer actually wants · reason correctly about idempotency, partitioning, and schema evolution, not just define the terms · walk through a full system design case end to end, stating trade-offs out loud.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Part 5 (Chapters 45–52: ingestion, pipelines, data quality, distributed compute, storage, streaming, activation and deployment) and Chapter 28 (query plans, grain, the star schema, slowly changing dimensions). They also link to Chapter 12, section 12.13 (the upsert), Chapter 33 (queues and graphs), Chapter 61 (partitioning and sharding), and the two banks this one leans on, Chapter 71 (SQL) and Chapter 72A (complexity). This chapter tests those skills; it doesn't teach them again, with one exception: Q77-018 builds topological sort step by step, because no earlier chapter does.

Time needed: 3–4 hours for a first pass (about 10 minutes per core question, including running its code, and 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list.

How this chapter is built. Same format as the other question banks in Part 8 (Chapters 70 onward): every core question leads with a **"Remember it as..."** hook, then a one-line answer, then a compact tier table (what **passes**, what's **strong**, and the **extra points**, tagged with Chapter 69's moves: **[+Trade-offs]**, **[+Edge cases]**, **[+Validate]** and so on). Rapid-fire sections are scan tables: question, one-line answer, one extra point, level, and where to learn it. **Every SQL result shown was run on PostgreSQL 16** in `riverstone_lab`, the practice database you created in Chapter 12, section 12.13; each demo drops its tables at the end. **Every Python output is real**, from Python 3.11.15 with the standard library only (the book recommends Python 3.14, Chapter 17, section 17.0; nothing here depends on the version). The sections run from pipeline fundamentals to modeling, reliability, orchestration, scale and quality, and end with full system-design walk-throughs.

Levels and roles. Each core question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **DE** data engineer · **AE** analytics engineer · **MLE** machine learning engineer · **ARCH** data architect. **Learn it in** pointers name the chapter and section where each idea is taught, mostly Part 5 (Chapters 45–52) and Chapter 28 (data modeling); where another bank drills it, a **Practise it with** pointer follows.

In this chapter

LEARN	77.1 Pipeline fundamentals	275
	77.2 Basic-but-tricky data engineering questions	277
	77.3 Data modeling for pipelines and warehouses	280
	77.4 Orchestration and scheduling	283
	77.5 Scale, partitioning, and performance	288
	77.6 Data quality and monitoring	291
	77.7 Full system design walk-throughs	293
APPLY	Common mistakes · In the real world · Project	295
REVIEW	Key terms · Final-week revision list	297
NEXT	Where this leads	297

LEARN

77.1 Pipeline fundamentals

Q77-001 · ETL vs. ELT: what's the actual difference, and when does it matter which you pick?

Level: Fresher · **Roles:** DE, AE

Remember it as: ETL transforms before loading, in a separate processing step. ELT loads raw data first and transforms inside the warehouse itself, using the warehouse's own compute.

Answer in one line: ETL (Extract, Transform, Load) transforms data in a separate processing layer before it ever reaches the destination system; ELT (Extract, Load, Transform) loads raw data into the destination (usually a modern cloud warehouse) first, then transforms it there using the warehouse's own compute power, which became the dominant pattern once cloud warehouses got cheap and powerful enough to make that practical.

Tier	What to say
Passes	"ETL and ELT do the same three steps in a different order" (technically true, misses why the order matters)
Strong	+ explains the practical driver: ELT keeps a full copy of raw data in the warehouse (useful for reprocessing or auditing later), and pushes transformation compute onto the warehouse rather than a separate processing cluster
Extra points	+ [+Trade-offs] ELT's "load everything raw first" approach means raw, unvalidated data briefly exists in the warehouse before transformation, a real consideration for compliance-sensitive data + [+Business] the choice is rarely purely technical anymore; it's often driven by which the team's existing tooling (dbt, Airflow, a specific cloud warehouse) is already built around

Likely follow-ups: Name a scenario where ETL is still clearly the better choice today. What's reverse ETL, and how does it relate to either pattern?

Red flag: treating the two as interchangeable synonyms.

Learn it in: Chapter 32, §32.1 (ETL versus ELT) and Chapter 45, §45.1 (where company data comes from, and the warehouse's layers). Reverse ETL: Chapter 51.

Q77-002 · Batch vs. streaming: how do you decide which a new pipeline actually needs?

Level: Fresher · **Roles:** DE, MLE

Remember it as: Ask how fresh the answer actually needs to be, in business terms, before assuming streaming is the more impressive, more correct choice.

Answer in one line: **Batch** processing runs on a schedule (hourly, daily), processing accumulated data in one pass, simpler to build and debug; **streaming** processes events continuously as they arrive, needed when a decision genuinely can't wait for the next batch window, and the right choice depends entirely on how fresh the downstream decision actually needs to be, not on which technology is more modern.

Tier	What to say
Passes	"Streaming is more real-time and modern, so it's generally better"
Strong	The freshness-driven decision above, with a concrete example each: a daily sales summary report is a natural batch case; a fraud-detection system blocking a transaction in real time is a natural streaming case
Extra points	+ [+Business] streaming systems are meaningfully harder to build, test, and debug correctly (exactly-once semantics, out-of-order events, windowing) than batch ones; choosing streaming for a problem that doesn't actually need sub-minute freshness is real, avoidable added complexity, not a sign of engineering sophistication

Likely follow-ups: What's micro-batching, and where does it sit between the two? How would you migrate an existing batch pipeline to streaming without a risky big-bang rewrite?

Red flag: recommending streaming by default without asking what freshness the actual decision requires.

Learn it in: Chapter 50, §50.1 (what "real-time" actually means) and §50.8 (when Riverstone would not build a stream).

Rapid-fire, 77.1

Roles: DE and AE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-003	What's a data warehouse vs. a data lake vs. a lakehouse?	Structured, schema-on-write, optimized for analytics queries / raw, schema-on-read, stores any format cheaply / a hybrid combining a lake's flexible storage with a warehouse's structured querying and transaction support	[+Trade-offs] a lakehouse is a lake plus a table format (Delta Lake, Iceberg) that gives plain files transactions, schema changes and time travel	Fresher · 49.1, 49.5, 49.7; star schema 28.9
Q77-004	What's a data contract?	An explicit agreement between a data producer and consumer about a dataset's schema, format, and update cadence, so downstream pipelines don't silently break when the source changes	[+Business] the DE-team equivalent of an API contract between two services	Mid · 47.7

#	Question	One-line answer	Extra point	Level · learn it in
Q77-005	What's the difference between a full load and an incremental load?	A full load reprocesses the entire source every run; an incremental load processes only new or changed records since the last run, using a timestamp or change-tracking mechanism	[+Trade-offs] full loads are simpler and self-healing but don't scale to large sources; incremental loads scale but need careful handling of updates and deletes, not just new rows	Fresher · 45.3–45.4
Q77-006	What's CDC (Change Data Capture)?	A technique for detecting and capturing row-level changes (inserts, updates, deletes) in a source database, often by reading its transaction log, to drive incremental pipelines without querying the whole source table	[+Business] CDC avoids the "how do I know what changed" problem incremental loading otherwise has to solve with timestamps, which can miss deletes entirely	Mid · 45.6

77.2 Basic-but-tricky data engineering questions

Q77-007 · Your pipeline runs on a schedule, but if it fails and gets re-run, it duplicates every row it already loaded. What's the fix?

Level: Fresher · **Roles:** DE, AE

Remember it as: A pipeline that can safely run twice with the same input and produce the same result is idempotent. One that can't, isn't safe to retry, ever.

Answer in one line: Make the load step **idempotent**: use an upsert (insert-or-update) keyed on a unique identifier instead of a plain insert, so re-running the same load with the same data updates existing rows instead of duplicating them.

Verified, in riverstone_lab. First the table. The primary key on `order_date` is what makes "the same day" detectable:

```
CREATE TABLE daily_summary (
  order_date    DATE PRIMARY KEY,
  total_revenue NUMERIC(12,2) NOT NULL
);
```

Now the load step. ₹20,900 is Riverstone's real net revenue on 5 January 2025 (one order, in `riverstone_2025`):

```
INSERT INTO daily_summary VALUES ('2025-01-05', 20900.00)
ON CONFLICT (order_date)
DO UPDATE SET total_revenue = EXCLUDED.total_revenue;

SELECT * FROM daily_summary;
```

```
order_date | total_revenue
-----+-----
2025-01-05 |      20900.00
(1 row)
```

- **ON CONFLICT (order_date)** names the unique column to check. If a row for that date already exists, PostgreSQL doesn't insert; it runs the `DO UPDATE` part instead.
- **EXCLUDED** means "the row you tried to insert" (Chapter 12, section 12.13), so `total_revenue = EXCLUDED.total_revenue` copies the new value onto the existing row.

The job fails later that night and gets re-run. Run the exact same statement a second time, then count the rows:

```
INSERT INTO daily_summary VALUES ('2025-01-05', 20900.00)
ON CONFLICT (order_date)
DO UPDATE SET total_revenue = EXCLUDED.total_revenue;

SELECT COUNT(*) FROM daily_summary;
```

```
count
-----
      1
(1 row)
```

One row, not two. **What if the table had no primary key?** Try the same upsert on a copy without one, and predict what happens before you run it:

```
CREATE TABLE daily_summary_nokey (
  order_date    DATE,
  total_revenue NUMERIC(12,2) NOT NULL
);

INSERT INTO daily_summary_nokey VALUES ('2025-01-05', 20900.00)
ON CONFLICT (order_date)
DO UPDATE SET total_revenue = EXCLUDED.total_revenue;
```

```
ERROR:  there is no unique or exclusion constraint matching the ON CONFLICT specification
```

`ON CONFLICT` only works when the conflict column is declared unique: without the key, the database has no way to tell that two rows are "the same day". Clean up:

```
DROP TABLE daily_summary, daily_summary_nokey;
```

Tier	What to say
Passes	"Add a check to skip rows that already exist" (a partial fix, misses the case where a value genuinely needs updating on retry)
Strong	The upsert pattern above, explaining that idempotency means safe <i>retry</i> , not just avoiding duplicates on first run, and that the upsert needs a unique key on the business key (here, the date)
Extra points	+ [+Validate] the real proof above: the same statement run twice leaves exactly one row, not two + [+Business] idempotency is what makes "just re-run the failed job" a safe, boring operational response instead of a risky one that needs careful manual cleanup first

Likely follow-ups: How would you make a multi-step pipeline (not just one insert) idempotent end to end? What's the difference between idempotency and exactly-once processing?

Red flag: a fix that only prevents duplicates on a clean re-run, not one that's genuinely safe to retry

after a partial failure mid-load.

Learn it in: Chapter 12, §12.13 (step 7, the upsert); Chapter 45, §45.5 (why upserts make loads safe to repeat); Chapter 46, §46.4 (idempotency, proven with a failure test).

Q77-008 · What's the difference between "at-least-once," "at-most-once," and "exactly-once" delivery, and which is actually achievable?

Level: Mid · **Roles:** DE, MLE

Remember it as: *At-least-once can duplicate. At-most-once can lose data. Exactly-once, in the strict sense, usually isn't really achievable end to end, only effectively achievable by combining at-least-once delivery with idempotent processing on the receiving end.*

Answer in one line: **At-least-once** guarantees a message is delivered, possibly more than once (safe against loss, risks duplication); **at-most-once** guarantees no duplication, possibly at the cost of losing a message entirely (safe against duplication, risks loss); true **exactly-once** delivery across a genuinely distributed system is famously difficult and, in most real systems, achieved in practice by combining at-least-once delivery with idempotent processing downstream (Q77-007), not by a delivery mechanism that's magically exactly-once on its own.

Tier	What to say
Passes	Defines at-least-once and at-most-once correctly, but treats exactly-once as a setting you switch on
Strong	The three-way distinction above, and specifically names idempotent processing as the real mechanism behind most practical "exactly-once" claims
Extra points	+ [+Signpost] this is precisely why Q77-007's upsert pattern matters beyond just convenience: it's what turns an honestly at-least-once delivery guarantee into an effectively exactly-once <i>outcome</i> + [+Trade-offs] the other route, used by Spark checkpoints and Kafka transactions (Chapter 50, §50.3), commits the output and the read position together in one transaction: real exactly-once, but narrower and slower

Likely follow-ups: Which would you choose for a financial transaction log, and why? What's a real system that claims exactly-once, and how does it actually achieve it under the hood?

Red flag: treating "exactly-once" as a simple guarantee some systems just have and others don't, with no understanding of how it's actually achieved.

Learn it in: Chapter 50, §50.3 (delivery guarantees, and the duplicates you will get) and Chapter 51, §51.4 (making writes idempotent, and proving it).

Rapid-fire, 77.2

Roles: DE and AE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-009	What's schema drift, and why is it dangerous in an automated pipeline?	A source system's schema changing (a renamed column, a new field, a changed data type) without the downstream pipeline being updated to match, often causing a silent failure or silently wrong data rather than a loud error	[+Business] a pipeline that fails loudly on schema drift is far safer than one that silently adapts (or silently drops the new column), which can hide the problem for weeks	Mid · 45.8; contracts 47.7

#	Question	One-line answer	Extra point	Level · learn it in
Q77-010	What's backfilling, and why does it need special care?	Reprocessing historical data, usually after a bug fix or a new metric definition; needs care because it can be expensive at scale, and because downstream reports or dashboards using the old values need a plan for the transition, not just silently different historical numbers	[+Edge cases] a backfill that changes historical numbers without any announcement is a fast way to lose a stakeholder's trust in a dashboard	Mid · 46.5
Q77-011	Why can't you always trust a source system's own "last modified" timestamp for incremental loading?	Clock skew between systems, a source that doesn't update the timestamp on every field change, or a source that allows backdated edits can all make a naive timestamp-based incremental load silently miss real changes	[+Edge cases] no timestamp ever shows a deleted row, because nothing is left to carry one; CDC (Q77-006) reads deletes from the log	Mid · 45.4–45.6
Q77-012	What's a dead-letter queue?	A separate holding location for messages or records that failed processing repeatedly, so they don't block the rest of the pipeline and can be investigated separately	[+Business] without one, a single malformed record can silently halt an entire stream or repeatedly retry and fail forever	Fresher · 50.7 (the dead-letter path)

77.3 Data modeling for pipelines and warehouses

Q77-013 · Implement Slowly Changing Dimension (SCD) Type 2 for a customer whose segment changes, and explain why Type 1 wouldn't preserve history

Level: Mid · **Roles:** DE, AE

Remember it as: Type 1 overwrites, so the past is gone. Type 2 closes the old row and opens a new one, so you can always ask "what did we believe was true on this date."

Answer in one line: SCD Type 1 overwrites the old value in place, losing all history of what it used to be; SCD Type 2 keeps every historical version as its own row, with validity date ranges (and usually a current-flag), so you can query the dimension as it looked at any point in the past, not just as it looks today.

Verified, in riverstone_lab. The change is a real one: riverstone_2025's customer_changes table (Chapter 28, section 28.1) records Harbour Traders, customer 11, moving from Retail to Wholesale on 1 September 2025. First the dimension, holding the customer's first version:

```
CREATE TABLE customer_history (  
    customer_key    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id     INTEGER NOT NULL,  
    customer_name   VARCHAR(100) NOT NULL,  
    segment         VARCHAR(30) NOT NULL,  
    valid_from      DATE NOT NULL,  
    valid_to        DATE NOT NULL DEFAULT '9999-12-31',  
    is_current      BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
INSERT INTO customer_history (customer_id, customer_name, segment, valid_from)  
VALUES (11, 'Harbour Traders', 'Retail', '2025-04-09');
```

- **customer_key** is the surrogate key: the database numbers the versions 1, 2, 3... by itself, as in Chapter 28, section 28.10. **customer_id** is the business key, and it repeats, once per version.
- **valid_from and valid_to** form a half-open period: the version is true from **valid_from** up to, but not including, **valid_to**. So **valid_to is the first day the row is no longer true**, and 9999-12-31 means "still true". 9 April 2025 is the day Harbour Traders signed up.

Now the change: close the old version and open the new one, inside one transaction, so nobody ever sees the customer with no current row:

```
BEGIN;  
  
UPDATE customer_history  
SET valid_to   = '2025-09-01',  
    is_current = FALSE  
WHERE customer_id = 11  
    AND is_current;  
  
INSERT INTO customer_history (customer_id, customer_name, segment, valid_from)  
VALUES (11, 'Harbour Traders', 'Wholesale', '2025-09-01');  
  
COMMIT;  
  
SELECT * FROM customer_history ORDER BY customer_key;
```

customer_key	customer_id	customer_name	segment	valid_from	valid_to	is_current
1	11	Harbour Traders	Retail	2025-04-09	2025-09-01	f
2	11	Harbour Traders	Wholesale	2025-09-01	9999-12-31	t

(2 rows)

`BEGIN` and `COMMIT` make the two statements one transaction (Chapter 12, section 12.13, step 8): either both happen or neither does. The `UPDATE` closes version 1 by setting its `valid_to` to the change date and `is_current` to `FALSE`; `AND is_current` makes sure only the open version is touched. The `INSERT` adds version 2, and its `customer_key`, `valid_to` and `is_current` take their defaults. The old row closes on the same date the new one opens. Check the two days either side of the change with the same half-open test a fact table join uses (`date >= valid_from AND date < valid_to`):

```
SELECT segment FROM customer_history
WHERE customer_id = 11
  AND DATE '2025-08-31' >= valid_from
  AND DATE '2025-08-31' < valid_to;

SELECT segment FROM customer_history
WHERE customer_id = 11
  AND DATE '2025-09-01' >= valid_from
  AND DATE '2025-09-01' < valid_to;
```

```
segment
-----
Retail
(1 row)

segment
-----
Wholesale
(1 row)
```

Every day falls in exactly one version: no gap, no overlap. Clean up:

```
DROP TABLE customer_history;
```

Tier	What to say
Passes	Can define Type 1 vs Type 2 in the abstract, can't produce a working implementation
Strong	The working two-statement pattern above: close the current row, insert the new one, both inside the same transaction, with a surrogate key for each version
Extra points	+ [+Validate] the real output above shows both the historical (Retail, closed) and current (Wholesale, open) rows coexisting, which is the entire point, and the two date checks show the half-open periods leave no gap + [+Business] this is exactly why "what was this customer's segment when they placed order X" is answerable with Type 2 history and not with Type 1, which would show only ever the current segment regardless of when the order was placed, silently misattributing every historical order to the wrong segment

Likely follow-ups: What's SCD Type 3, and when would you use it instead of Type 2? How would you handle a fact table that needs to join to the *correct historical version* of a Type 2 dimension, not just the current one?

Red flag: implementing this with a plain `UPDATE` that overwrites the segment in place, silently losing history despite believing Type 2 was implemented.

Learn it in: Chapter 28, §28.9–28.10 (dimensional modeling; slowly changing dimensions), and Chapter 32, §32.9 (dbt snapshots do Type 2 for you).

Rapid-fire, 77.3

Roles: DE and AE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-014	Star schema vs. snowflake schema?	A star schema's dimension tables are fully denormalized (flat); a snowflake schema further normalizes dimensions into sub-tables, trading some query simplicity for reduced redundancy	[+Trade-offs] dimension tables are small, so a star's repetition costs little, and BI tools such as Power BI work best with a star: default to it	Fresher · 28.9, 16.3 (practise: 70.7)
Q77-015	What's a surrogate key, and why use one instead of a natural key?	A system-generated, meaningless identifier (often a sequential integer) used as a table's primary key, instead of a real-world identifier like an email or SSN that could change, be reused, or vary in format across sources	[+Business] a surrogate key is exactly what makes SCD Type 2 workable: the natural business key (customer_id) repeats across history rows, but each row still needs its own unique surrogate key	Fresher · 28.7, 28.10
Q77-016	What's a fact table's grain, and why must it be decided before building anything else?	The level of detail one row represents (one order line, one daily summary, one customer-month); every measure and every join in the model depends on the grain being consistent and explicit	[+Edge cases] mixing grains in one fact table (some rows per-order, some pre-aggregated per-day) is a common, subtle modeling error that silently breaks any SUM across the whole table	Mid · 28.8
Q77-017	What's a junk dimension?	A single dimension table combining several small, low-cardinality flags or indicators that don't deserve their own separate dimension tables (for Riverstone's order lines, the order status and a yes/no "discounted" flag), reducing the number of tables a fact table needs to join against	[+Trade-offs] trades a slightly less "clean" model for meaningfully fewer joins in every query against the fact table	Mid · 28.9 (junk dimensions)

77.4 Orchestration and scheduling

Q77-018 · Given this pipeline's task dependencies, determine a valid execution order

Level: Mid · Roles: DE, AE

Remember it as: A pipeline's tasks form a graph, not a list. The valid order is whatever a topological sort of that graph says it is, and there can be more than one right answer.

Answer in one line: Build a dependency graph (which tasks must finish before which others can start) and run a **topological sort**, an ordering in which every task comes after all the tasks it depends on; any order that respects every dependency edge is valid, and there's often more than one correct answer, not a single "the" order.

Topological sort, built one step at a time. No earlier chapter builds this algorithm, so here it is, cell by cell, in a notebook. The standard method is **Kahn's algorithm**: count how many tasks each task is waiting for, start with the tasks that wait for nothing, and each time a task finishes, tell the tasks that were waiting for it. First, the pipeline as a dictionary. Each key is a task; its list holds the tasks it needs first:

```
dag = {
    "extract_orders": [],
    "extract_customers": [],
    "clean_orders": ["extract_orders"],
    "join_orders_customers": ["clean_orders", "extract_customers"],
    "compute_daily_summary": ["join_orders_customers"],
    "load_to_warehouse": ["compute_daily_summary"],
}
print(len(dag), "tasks")
```

6 tasks

A task's **in-degree** is the number of tasks it's waiting for, which is simply the length of its list. A dictionary comprehension (Chapter 17, section 17.7) builds all six counts:

```
in_degree = {task: len(needs) for task, needs in dag.items()}
for task, count in in_degree.items():
    print(f"{task:22} {count}")
```

```
extract_orders      0
extract_customers   0
clean_orders        1
join_orders_customers 2
compute_daily_summary 1
load_to_warehouse   1
```

`{task:22}` pads each name to 22 characters so the counts line up. The dictionary says who each task *needs*; the algorithm also has to know the opposite, who is *waiting for* each task. Build that map, the task's **children**:

```
children = {task: [] for task in dag}
for task, needs in dag.items():
    for parent in needs:
        children[parent].append(task)
for task, kids in children.items():
    print(f"{task:22} -> {kids}")
```

```
extract_orders      -> ['clean_orders']
extract_customers   -> ['join_orders_customers']
clean_orders        -> ['join_orders_customers']
join_orders_customers -> ['compute_daily_summary']
compute_daily_summary -> ['load_to_warehouse']
load_to_warehouse   -> []
```

The first line starts every task with an empty list. The loop then reads each dependency backwards: `clean_orders` needs `extract_orders`, so `clean_orders` is a child of `extract_orders`. Now put every task with in-degree 0, the tasks that can start straight away, in a queue:

```
from collections import deque

ready = deque(task for task, count in in_degree.items() if count == 0)
print(list(ready))
```

```
['extract_orders', 'extract_customers']
```

`deque` is the queue from Chapter 33, section 33.4: first in, first out, with `popleft()` taking from the front. The expression inside `deque(...)` is a comprehension without the square brackets; it hands the queue each task whose count is 0. The main loop takes the next ready task, adds it to the order, and lowers each child's count by one. A child whose count reaches 0 has nothing left to wait for, so it joins the queue:

```
order = []
while ready:
    task = ready.popleft()
    order.append(task)
    for child in children[task]:
        in_degree[child] -= 1
        if in_degree[child] == 0:
            ready.append(child)
for step, task in enumerate(order, start=1):
    print(step, task)
```

```
1 extract_orders
2 extract_customers
3 clean_orders
4 join_orders_customers
5 compute_daily_summary
6 load_to_warehouse
```

Why does `extract_orders` come before `extract_customers`? Only because it was put in the dictionary first (dictionaries keep the order you insert keys in) and the queue is first in, first out. Put `extract_customers` first in `dag` and it comes out first. Both orders are valid, and that's the point of the question: nothing makes one extract wait for the other, so they could also run in parallel.

Last, the same steps as one function, with a cycle check. If the graph has a cycle, the tasks in it never reach in-degree 0, so the order comes out shorter than the task list:

```
def topo_order(dag):
    in_degree = {task: len(needs) for task, needs in dag.items()}
    children = {task: [] for task in dag}
    for task, needs in dag.items():
        for parent in needs:
            children[parent].append(task)
    ready = deque(task for task, count in in_degree.items() if count == 0)
    order = []
    while ready:
        task = ready.popleft()
        order.append(task)
        for child in children[task]:
            in_degree[child] -= 1
            if in_degree[child] == 0:
                ready.append(child)
    if len(order) < len(dag):
        raise ValueError("cycle found: no valid order exists")
    return order

print(topo_order(dag) == order)
```

True

The function builds its own counts, because the loop above used up the first ones (every count is 0 now). `raise ValueError(...)` stops with an error, as in Chapter 29, section 29.2. Now break the graph on purpose: make `extract_orders` wait for `load_to_warehouse`, which itself waits, step by step, for `extract_orders`. Before you run it, predict which tasks could still start:

```
looped = dict(dag)
looped["extract_orders"] = ["load_to_warehouse"]
try:
    topo_order(looped)
except ValueError as err:
    print("error:", err)
```

error: cycle found: no valid order exists

`dict(dag)` makes a copy, so the original `dag` is untouched. Only `extract_customers` could start; the other five tasks wait on each other in a loop and never reach 0.

Tier	What to say
Passes	Describes the dependencies in English, produces an order by eyeballing it rather than a systematic method
Strong	Names topological sort explicitly and produces a genuinely valid order, acknowledging more than one valid order can exist (the two extracts could run in either order, or in parallel)
Extra points	+ [+Validate] the real, algorithmically-produced order above, not a guessed one, and a cycle check that fails loudly + [+Business] this is exactly what an orchestrator such as Dagster or Airflow does automatically from a declared DAG: the engineer states dependencies, the scheduler determines (and often parallelizes) a valid execution order, rather than the engineer hard-coding a sequence

Likely follow-ups: How would you detect a cycle in a dependency graph, meaning the DAG isn't actually valid? Which of these tasks could safely run in parallel?

Red flag: producing an order that violates a stated dependency, or being unable to name the standard algorithm for this problem.

Learn it in: Chapter 46, §46.2 (pipelines as graphs, and why they must be acyclic); Chapter 33, §33.4 (queues and deque) and §33.6 (walking graphs, cycles). Topological sort itself is built in this question.

Practise it with: Chapter 72A, Q72A-028 (BFS and DFS).

Rapid-fire, 77.4

Roles: DE and AE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-019	What does DAG stand for, and why must a pipeline's dependency graph be acyclic?	Directed Acyclic Graph; a cycle (task A depends on B, which depends on A) has no valid execution order at all, since neither could ever go first	[+Edge cases] a cycle is easy to create by accident (a load that waits for the report that reads it); Q77-018's <code>topo_order</code> catches it, because tasks in a loop never reach in-degree 0	Fresher · 46.2 (practise: Q72A-028)
Q77-020	What's a sensor (or a "wait for" task) in an orchestration tool?	Something that reacts to an event outside the schedule. In Airflow a sensor is a task that <i>waits</i> for a condition (a file to arrive, an upstream job to finish) before the rest of the pipeline proceeds; in Dagster (Chapter 46, §46.6) a sensor <i>watches</i> for an event and starts a run or sends an alert	[+Business] sensors are how a pipeline safely depends on something outside its own DAG, like a third-party data delivery with no guaranteed exact timing	Mid · 46.2, 46.6, 46.8
Q77-021	What's the risk of a pipeline with a single point of failure at its very first task?	Every single downstream task, however independent it looks, silently can't run if that one upstream task fails, making the whole day's pipeline as fragile as its weakest single link	[+Business] designing for partial success (independent tasks can still run and deliver partial value even if one branch fails) is often worth the added complexity for a pipeline with many independent downstream consumers	Mid · 46.10; 61.6–61.7
Q77-022	What's backpressure, in a streaming context?	A mechanism for a slower downstream consumer to signal an upstream producer to slow down, preventing the consumer from being overwhelmed by data arriving faster than it can process. With a log such as Kafka (Chapter 50), producers aren't slowed down: the consumer reads at its own pace, and the gap shows up as consumer lag , which is why section 50.7 tells you to monitor lag	[+Business] without backpressure handling, a slow consumer either falls further and further behind or crashes outright under load it can't keep up with	Mid · 50.2, 50.7 (lag)

77.5 Scale, partitioning, and performance

Q77-023 · A table has grown to hundreds of millions of rows and queries filtering on date are slow. Walk through partitioning as a fix, and prove it works

Level: Mid · Roles: DE, AE

Remember it as: A partitioned table lets the database skip entire partitions it knows can't contain a match, instead of scanning rows one at a time to find out.

Answer in one line: **Range partitioning** by date splits a large table into smaller physical sub-tables, each holding a specific date range; a query filtering on date lets the query planner skip (prune) any partition that can't possibly contain a match, dramatically reducing how much data actually needs to be scanned.

Verified, in riverstone_lab. A small events table, partitioned by month:

```
CREATE TABLE events_partitioned (  
    id            INTEGER GENERATED ALWAYS AS IDENTITY,  
    event_date    DATE NOT NULL,  
    payload       TEXT  
) PARTITION BY RANGE (event_date);  
  
CREATE TABLE events_2025_01 PARTITION OF events_partitioned  
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');  
CREATE TABLE events_2025_02 PARTITION OF events_partitioned  
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
```

This is PostgreSQL's own partitioning syntax, which no earlier chapter uses:

- **PARTITION BY RANGE (event_date)** declares the parent table. It holds no rows itself; it says which column decides where each row goes.
- **PARTITION OF events_partitioned FOR VALUES FROM (a) TO (b)** creates a child table, a **partition**, for the dates from a (included) up to b (excluded): the same half-open rule as Q77-013, so 1 February belongs to February only.
- On **INSERT** into the parent, each row is **routed** to its child automatically.

Load 1,000 events into each month, then count what landed in each partition:

```
INSERT INTO events_partitioned (event_date, payload)  
SELECT DATE '2025-01-01' + (n % 31), 'event ' || n  
FROM generate_series(1, 1000) AS n;  
  
INSERT INTO events_partitioned (event_date, payload)  
SELECT DATE '2025-02-01' + (n % 28), 'event ' || n  
FROM generate_series(1, 1000) AS n;  
  
SELECT COUNT(*) AS january_rows FROM events_2025_01;  
SELECT COUNT(*) AS february_rows FROM events_2025_02;
```

```

janeury_rows
-----
      1000
(1 row)

february_rows
-----
      1000
(1 row)

```

`generate_series(1, 1000)` (Chapter 13, section 13.8) makes the numbers 1 to 1,000, one row each, called `n`. `n % 31` is the remainder after dividing by 31 (0 to 30), and adding a whole number to a `DATE` moves it forward that many days, so the rows spread across January's 31 days. `'event ' || n` joins text to the number. The rows went into the parent, and every one was routed to the right month. Now collect statistics and ask for the plan:

```

ANALYZE events_partitioned;

EXPLAIN SELECT * FROM events_partitioned WHERE event_date = '2025-01-15';

```

```

                                QUERY PLAN
-----
Seq Scan on events_2025_01 events_partitioned (cost=0.00..19.50 rows=32 width=17)
  Filter: (event_date = '2025-01-15'::date)
(2 rows)

```

`ANALYZE` counts the rows and samples their values, the statistics the planner uses to estimate (Chapter 28, section 28.5). The numbers in brackets are those estimates; here only the partition name matters. The query plan touches **only the January partition**, `events_2025_01`; the February partition is never scanned at all, real, confirmed partition pruning.

What if the query doesn't filter on the partition key?

```

EXPLAIN SELECT * FROM events_partitioned WHERE payload = 'event 7';

```

```

                                QUERY PLAN
-----
Append (cost=0.00..39.01 rows=2 width=17)
-> Seq Scan on events_2025_01 events_partitioned_1 (cost=0.00..19.50 rows=1 width=17)
   Filter: (payload = 'event 7'::text)
-> Seq Scan on events_2025_02 events_partitioned_2 (cost=0.00..19.50 rows=1 width=17)
   Filter: (payload = 'event 7'::text)
(5 rows)

```

Both partitions are scanned, and `Append` glues their results together: with no date in the filter, the planner can't rule out either month. Clean up (dropping the parent drops its partitions too):

```

DROP TABLE events_partitioned;

```


Tier	What to say
Passes	"Partitioning makes queries faster" (true, no proof of the actual mechanism)
Strong	The pruning explanation above, correctly identifying that the <i>query planner</i> is what decides to skip irrelevant partitions, not the query itself doing anything different
Extra points	+ [+Validate] the real query plan above shows only one partition's name (<code>events_2025_01</code>) in the plan at all, direct proof of pruning, not an assumed benefit + [+Business] partitioning also makes bulk operations dramatically cheaper: retiring an entire month's stale data is <code>ALTER TABLE events_partitioned DETACH PARTITION events_2025_01</code> (then archive it, or <code>DROP TABLE</code>) instead of a slow, logged <code>DELETE</code> scanning and removing millions of individual rows

Likely follow-ups: How would you choose the partition granularity (daily vs. monthly vs. yearly)? What happens to a query that doesn't filter on the partition key at all?

Red flag: claiming partitioning always speeds up every query, without the "the query must filter on the partition key" caveat.

Learn it in: Chapter 49, §49.6 (partitioning and file sizing); Chapter 48, §48.5 (partition pruning: don't open boxes you don't need); Chapter 28, §28.5 (reading query plans with `EXPLAIN`). **Practise it with:** Chapter 71, §71.9 (query optimization and indexes).

Rapid-fire, 77.5

Roles: DE, AE and ARCH for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-024	What's sharding, and how is it different from partitioning?	Table partitioning (as in Q77-023) splits a table into pieces inside one database; sharding spreads the data across <i>separate database servers</i> , adding a routing layer to know which shard holds which data. In Spark and Kafka (Chapters 48 and 50), "partition" also means a piece of data that can live on a different machine: the word depends on the system	[+Trade-offs] sharding solves a scale problem partitioning alone can't (a single machine's storage/compute limit), at the cost of real added complexity: cross-shard queries and joins become much harder	Mid · 61.5; Spark and Kafka partitions 48.2, 50.2
Q77-025	What's a materialized view, and when does it help at scale?	A pre-computed, physically stored query result, refreshed periodically, trading data freshness for query speed on an expensive aggregation run frequently	[+Trade-offs] the answer is stale between refreshes, and each refresh re-runs the whole query; PostgreSQL's <code>REFRESH MATERIALIZED VIEW CONCURRENTLY</code> lets people keep reading during a refresh, but needs a unique index	Mid · 28.11 (practise: Q71-049, Q71-055)
Q77-026	Why does <code>SELECT *</code> become a bigger problem at scale than it is on a small table?	Pulling every column, including large or unused ones, multiplies I/O cost linearly with row count; on a small table the waste is invisible, on a billion-row table it's a real, measurable cost	[+Edge cases] on a columnar format, <code>SELECT *</code> throws away the main advantage: every column's data is read (Q77-027)	Fresher · 49.2 (see also Q71-015, <code>SELECT *</code> with <code>GROUP BY</code>)

#	Question	One-line answer	Extra point	Level · learn it in
Q77-027	What's a columnar storage format (like Parquet), and why does it suit analytical queries specifically?	Stores each column's values together on disk, rather than each row together; an analytical query that only needs 3 of a table's 50 columns reads only those 3 columns' data, not the other 47	[+Business] this is the storage-layer reason a data warehouse or lakehouse built on columnar formats is dramatically faster for typical GROUP BY/aggregation-heavy analytical queries than a row-oriented format built for fast single-row lookups	Fresher · 49.2-49.3

77.6 Data quality and monitoring

Q77-028 · Design a data quality check that would have caught a real Riverstone data problem: a pipeline that silently produced negative revenue values

Level: Fresher · **Roles:** DE, AE

Remember it as: A data quality check needs to run automatically, on every load, and fail loudly, not depend on someone noticing a wrong number weeks later.

Answer in one line: Build automated, scheduled checks (not manual, occasional ones) that assert specific, testable properties of the data on every pipeline run, revenue should never be negative, order counts shouldn't drop to zero unexpectedly, a key column shouldn't suddenly have nulls, and fail the pipeline run loudly (alerting someone) rather than silently loading bad data.

Worked example, a concrete check:

"A simple assertion: `SELECT COUNT(*) FROM daily_summary WHERE total_revenue < 0` should always return 0; if it doesn't, fail the pipeline run and alert before the bad data ever reaches a dashboard. I'd also add a volume check: today's row count shouldn't be, say, more than 50% below the trailing 7-day average, which catches a source system silently sending a truncated file, a different kind of failure the negative-revenue check wouldn't catch at all."

Tier	What to say
Passes	Names one sensible check (e.g. no negative revenue) but runs it by hand, after the fact
Strong	At least one concrete, automatable assertion (the negative-revenue check above), run on every pipeline execution, not manually
Extra points	+ [+Edge cases] names a <i>second</i> , different kind of check (a volume/row-count anomaly check) unprompted, since a single check type only catches one failure mode, and real pipeline failures come in more than one shape + [+Edge cases] computes the baseline from days that passed their checks, or uses the same weekday over the last four weeks, so a run of bad days can't drag the baseline down with it: after a week of zero-row days, a plain trailing average is near zero too, and the check stops firing + [+Business] ties the check to failing the pipeline loudly, not just logging a warning that nobody reads until the damage is already visible downstream

Likely follow-ups: How would you decide the right threshold for a volume-anomaly check without too many false alarms? What's the difference between a data quality check and a schema validation check?

Red flag: proposing manual, human-driven spot-checks as the primary quality control for an automated pipeline.

Learn it in: Chapter 47, §47.2–47.3 (tests in the pipeline, a check library) and §47.5 (volume, and "don't learn your baseline from broken days"); Chapter 14, §14.10 (checks that must return zero). See also Chapter 36, §36.8's missing-value discipline (the modeling-side cousin of the same "detect before you trust" habit).

Rapid-fire, 77.6

Roles: DE and AE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q77-029	What's the difference between data quality monitoring and data observability?	Data quality monitoring checks specific, predefined rules against the data itself; data observability more broadly tracks the health of the whole pipeline system (freshness, volume, schema, lineage) to catch problems you didn't think to write a specific rule for	[+Business] observability is meant to catch the failure mode nobody anticipated; quality checks catch the ones you did	Mid · 47.5
Q77-030	What's data lineage, and why does an on-call engineer need it at 2am?	A traceable record of where a piece of data came from and what transformed it along the way; at 2am, debugging a wrong number starts with lineage, tracing backward from the bad number to find exactly which upstream step introduced it	[+Evidence] without lineage, debugging a wrong number in a large pipeline can mean manually checking every step in order, instead of jumping straight to the likely source	Mid · 47.6
Q77-031	Why does a monitoring setup need a heartbeat, an alert when a pipeline or monitor has gone silent, and not just failure alerts?	A heartbeat (or dead-man's switch) is a signal that must keep arriving on time; its absence is the alert. A monitoring system that itself silently stops running produces no alerts at all, which can be mistaken for "everything's fine" when it actually means nobody would know if something broke	[+Business] Chapter 47's freshness check is a heartbeat for data: it fires when the newest data is older than the previous working day, even though nothing "failed"	Mid · 47.5 (freshness)
Q77-032	What's the cost of alert fatigue, and how does it actually cause outages?	Too many low-value or false-positive alerts trains the on-call team to ignore or mute them, so a real, important alert gets missed in the noise exactly when it matters most	[+Business] a smaller number of carefully-tuned, high-signal alerts is more reliable in practice than a comprehensive but noisy set that gets tuned out	Fresher · 47.9, 20.8

77.7 Full system design walk-throughs

Q77-033 · Design a daily sales pipeline for Riverstone, end to end

Level: Mid · **Roles:** DE, AE

What they're really testing: whether a complete, coherent pipeline design comes out under time pressure, touching every stage, not just the interesting parts.

Talked through live, start to finish:

"Source: the transactional order database. Extraction: a scheduled incremental pull each night, using an updated-at timestamp or CDC if available, rather than a full reload of the whole orders table every time. Landing: raw extracted data lands in a staging area first, unmodified, so I always have an unaltered copy to reprocess from if a downstream transformation bug is found later. Transformation: clean and join orders with customers and products, compute daily aggregates, handling the exact fan-out risk Chapter 71 already covers if this joins through any one-to-many relationship carelessly. Load: upsert into the daily_summary fact table, keyed on date, so a re-run is safe (Q77-007). Quality checks: the negative-revenue and volume-anomaly checks from Q77-028, run automatically on the new data before anyone can read it, and only then published: write-audit-publish. Orchestration: an Airflow-style DAG with the dependency order Q77-018 worked through, scheduled to run after the source database's own nightly maintenance window closes, with a sensor task waiting for that to complete rather than a fixed clock time that could run too early. Monitoring: alert on failure, alert on the monitor itself going silent (Q77-031), and a lineage-traceable record of exactly which run produced which row in the final table."

Extra-points moves demonstrated: **[+Signpost]** covered every stage (extract, land, transform, load, quality, orchestrate, monitor) explicitly, not just the transformation logic, and tied the stages to answers already given in this chapter (Q77-007, Q77-018, Q77-028, Q77-031), showing the concepts as one coherent system, not isolated trivia. **[+Business]** justified the staging-area decision with a specific reason (safe reprocessing), not just "it's best practice."

Likely follow-ups: How would this change if the source database couldn't support CDC at all? What's your rollback plan if a bug is found in the transformation logic three days after it shipped?

Red flag: a design that only covers the transformation SQL, with no mention of scheduling, quality checks, or failure handling.

Learn it in: Chapter 46, §46.3–46.7 (Riverstone's daily pipeline, built); Chapter 47, §47.4 (write-audit-publish); this chapter's §77.1–77.6, pulled together.

Q77-034 · Design a real-time pipeline for monitoring sensor data from Riverstone's plant machines

Level: Senior · **Roles:** DE, MLE, ARCH

What they're really testing: whether the candidate correctly identifies this as a genuinely streaming problem (Q77-002) and reasons about the specific challenges streaming introduces that batch doesn't have.

Talked through live, start to finish:

"First, I'd confirm this genuinely needs streaming: if the goal is alerting on equipment failure within seconds, not next-day reporting, then yes, batch wouldn't meet the actual business need here, unlike the daily sales case. Ingestion: sensors publish to a message queue (Kafka or similar), which decouples the sensors from whatever's consuming their data and buffers against a consumer temporarily falling behind. Processing: a stream processor consumes the queue, computing rolling windows (say, a 5-minute rolling average temperature per machine) rather than reacting to every single raw reading, which would be noisy. Out-of-order handling: sensor data over a real network can arrive out of order or late; I'd use event time, not processing time, for the windows, with a defined lateness tolerance (a watermark), rather than assuming perfectly ordered arrival. Alerting: when a rolling average crosses a threshold, publish an alert event, itself handled idempotently (Q77-007) so a reprocessed or retried alert doesn't spam the on-call team twice for the same real event. Storage: raw sensor events also land in cheap, durable storage (a data lake) for later batch analysis and model training, alongside the real-time alerting path, since the two paths serve genuinely different purposes."

Extra-points moves demonstrated: **[+Clarify]** explicitly validated that streaming was the right call before designing for it, rather than assuming. **[+Edge cases]** named out-of-order and late-arriving data unprompted, a genuinely common real pitfall specific to streaming that a batch-only background can easily miss. **[+Business]** justified the dual raw-storage-plus-real-time-path design with the different purposes each actually serves.

Likely follow-ups: How would you size the rolling window, and what trade-off does that involve? What happens if the message queue itself goes down, does data get lost?

Red flag: a design that processes each raw sensor reading individually with no windowing, or no acknowledgment that streaming data can arrive out of order.

Learn it in: Chapter 50, §50.5 (event time, windows, and watermarks) and §50.8 (Riverstone's plant monitoring case), plus §77.1–77.2 of this chapter.

Q77-035 · Design a system to detect and merge duplicate customer records at scale

Level: Senior · **Roles:** DE, AE, ARCH

What they're really testing: whether "just check if the names match" gets recognized as inadequate for a real, messy production dataset, and a genuinely scalable approach is proposed instead.

Talked through live, start to finish:

"Exact-match deduplication (Chapter 71's GROUP BY approach) catches identical records, but real duplicates are messier: 'Sharma Hardware' vs. 'Sharma Hardware Pvt Ltd,' a trailing space, a different phone number format for the same company. I'd approach this in two stages. First, blocking: instead of comparing every pair of records against every other (which is $O(n^2)$ and doesn't scale, Chapter 72A's exact lesson), group records into smaller candidate 'blocks' likely to contain real duplicates, say by the first few characters of the normalized name plus the same city (the same match-key idea as Chapter 14's), so only records within the same block get compared pairwise. Second, within each block, score candidate pairs on similarity (normalized name distance, matching phone or email, matching address) and flag pairs above a threshold for either automatic merge (very high confidence) or human review (medium confidence). I'd deliberately avoid fully automatic merging at anything but very high confidence, since an incorrectly merged pair of genuinely different customers is a much more damaging, harder-to-undo mistake than leaving two real duplicates unmerged a little longer."

Extra-points moves demonstrated: **[+Scale]** explicitly named the $O(n^2)$ scaling problem a naive pairwise-comparison approach would hit, and proposed blocking as the specific fix, directly reusing Chapter 72A's own complexity reasoning. **[+Business]** made an explicit, justified risk trade-off (false-merge cost vs. missed-duplicate cost) rather than optimizing purely for automation.

Likely follow-ups: What similarity algorithm would you actually use for the name comparison? How would you handle a customer who insists two "duplicate" records are actually two genuinely different but related businesses?

Red flag: proposing a pairwise comparison across the entire dataset with no blocking strategy, ignoring the scaling problem entirely.

Learn it in: Chapter 14, §14.4 (exact and fuzzy duplicates, match keys, and why a wrong merge is worse than a missed one); Chapter 71, Q71-041 (exact-match dedup); Chapter 33, §33.1 and Chapter 72A, Q72A-002 (the $O(n^2)$ cost this design specifically avoids).

APPLY

Common mistakes

Mistake	Symptom	Fix
A load step using plain INSERT with no conflict handling	Re-running a failed pipeline duplicates every row	Use an idempotent upsert keyed on a unique identifier
Choosing streaming by default for a problem that doesn't need it	Real, avoidable added complexity with no business benefit	Ask how fresh the downstream decision genuinely needs to be first
SCD Type 1 used where history actually matters	Historical reports silently reflect only the current state	Use Type 2, closing old rows instead of overwriting them
A dependency graph hard-coded as one linear sequence	Independent tasks run needlessly serially, wasting time	Declare true dependencies; let the orchestrator determine (and parallelize) a valid order

Mistake	Symptom	Fix
Partitioning on a column queries don't actually filter on	No pruning benefit at all, despite the added complexity	Partition on the column your queries actually filter by
Manual, occasional data quality spot-checks	A bad number reaches a dashboard before anyone notices	Automated, scheduled checks that fail the pipeline loudly
A dedup design comparing every record pair with no blocking	Doesn't scale past a small dataset	Block first, compare only within blocks

In the real world: the pipeline that failed silently for three weeks

Arjun, a Data Engineer, is asked in an interview to describe a real production incident. He describes a pipeline that had been quietly loading zero rows for a source that had actually stopped sending data three weeks earlier, an upstream vendor had changed their export format without notice, and the pipeline's extraction step had been silently succeeding (technically, "zero rows extracted" isn't an error) and loading nothing, night after night, with no alert firing because nothing had technically failed.

He walks through exactly what he changed: a volume-anomaly check (Q77-028's second check), alerting if a day's row count dropped more than 50% below the trailing average, which would have caught this within a day instead of three weeks. He's asked the natural follow-up: why hadn't a check like that existed before the incident? His honest answer: "Because the team, including me, had only ever built checks for data that looked *wrong*, never for data that looked *absent*. It's an easy blind spot: a pipeline that runs successfully and loads nothing doesn't look like a failure from the outside, it looks exactly like a quiet day."

The interviewer's note: "*Didn't just describe the fix, explained the actual blind spot in the team's thinking that let it happen in the first place.*" That's the difference this chapter's whole method is built around: a correct technical fix is table stakes; naming the reasoning gap that let the problem exist unnoticed is what shows real production experience.

Project

Goal: apply this chapter's method to a real or realistic pipeline of your own.

Tools you'll need

PostgreSQL 16 and **Python**, the same environment Chapters 71 and 72A use; every SQL result and every topological-sort output in this chapter was produced in them. In real production work: Airflow, Dagster, or Prefect for orchestration; Kafka or a managed equivalent for streaming; dbt for transformation; a cloud warehouse (Snowflake, BigQuery, Redshift) or lakehouse (Databricks) for storage and compute.

1. Implement SCD Type 2 for one real dimension in your own data (or Riverstone's), and prove the historical version is queryable, the way Q77-013 did.

2. Make one real pipeline step idempotent using an upsert pattern, and prove a duplicate re-run doesn't duplicate rows.
3. Write two automated data quality checks for a real dataset: one checking a specific value constraint, one checking for an unexpected volume anomaly.
4. Pick one of this chapter's system design questions (daily pipeline, streaming sensors, or deduplication) and write your own full design for a different, real scenario from your own work.

REVIEW

Key terms

ETL vs. ELT · batch vs. streaming · data warehouse vs. data lake vs. lakehouse · data contract · CDC (Change Data Capture) · idempotency · upsert (`ON CONFLICT, EXCLUDED`) · at-least-once / at-most-once / exactly-once delivery · schema drift · backfilling · dead-letter queue · SCD Type 1/2/3 · surrogate key · half-open validity period · fact table grain · junk dimension · DAG (Directed Acyclic Graph) · topological sort · Kahn's algorithm · in-degree · sensor (orchestration) · backpressure · consumer lag · partitioning · partition pruning · sharding · materialized view · columnar storage · data observability · data lineage · heartbeat (dead-man's switch) · alert fatigue · write-audit-publish · watermark · blocking (deduplication)

Final-week revision list

Q77-001, Q77-002, Q77-007, Q77-008, Q77-013, Q77-018, Q77-023, Q77-028, Q77-033, Q77-034, Q77-035.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, already covers the join-cardinality, indexing (section 71.9), and materialized-view foundations this chapter builds directly on.
- **Chapter 72A, Data Structures & Algorithms Question Bank**, supplies the complexity reasoning ($O(n^2)$ scaling, walking graphs with BFS and DFS) this chapter's design questions reuse explicitly.
- **Chapter 78, Automation & Integration Question Bank**, applies this chapter's idempotency, retry and monitoring answers to business automations and integrations.
- **Part 5 of this book** (data engineering proper) teaches every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.

Chapter 78. Automation & Integration Question Bank

Chapter at a glance

You will learn to: choose the right automation approach for a given problem, macro, script, low-code tool, or RPA, instead of defaulting to whichever one you know best · design report and alert automations that actually get read · reason correctly about APIs, webhooks, and reverse ETL · build retry and idempotency logic that survives a flaky upstream system · investigate an automation that silently stopped working, the way an interviewer actually wants.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 19–20 (spreadsheet and report automation), 45 and 51 (APIs, webhooks, reverse ETL), 58 and 63 (intelligent automation, automation architecture), with retries from Chapter 29, section 29.9 and the circuit breaker from Chapter 57, section 57.8. This chapter tests those skills; it doesn't teach them again.

Time needed: 2–2.5 hours for a first pass (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 30 minutes to run the four code demos yourself); 30 minutes for the final-week list.

How this chapter is built. Same format as Chapters 70–77: every core question leads with a "Remember it as..." hook, a one-line answer, and a compact tier table whose extra points carry Chapter 69's tags ([+Edge cases], [+Business] and so on). Rapid-fire sections are scan tables.

Every Python pattern in this chapter was actually executed, not described from memory: webhook idempotency, rate limiting, retry-with-backoff, and upsert-based CRM sync are all shown with their real output. The order: choosing a tool first, then the basic-but-tricky questions, then the specific automation types (reports, APIs, sync), then failure handling, then a full design case pulling it together.

Levels and roles. Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **AUT** automation (macro, script and integration) roles · **DA** data analyst · **BA** business analyst · **BI** BI developer · **DE** data engineer.

Learn it in pointers name the chapter and section that teach each idea, mainly Chapters 19–20 (spreadsheet and report automation), 45 and 51 (APIs, webhooks, reverse ETL), 57–58 and 63 (automation at scale). In the rapid-fire tables, the last column gives the level and the section numbers.

In this chapter

LEARN	78.1 Choosing the right automation approach	299
	78.2 Basic-but-tricky automation questions	300
	78.3 Report and alert automation	302
	78.4 APIs, webhooks, and reverse ETL	303
	78.5 Failure handling and monitoring for automations	307
	78.6 Full design case	311
APPLY	Common mistakes · In the real world · Project	313
REVIEW	Key terms · Final-week revision list	314
NEXT	Where this leads	315

LEARN

78.1 Choosing the right automation approach

Q78-001 · A stakeholder wants a manual weekly task automated. Walk through how you'd choose between a macro, a script, a low-code tool, and RPA

Level: Mid · **Roles:** AUT, DA, BA

Remember it as: The right tool depends on how the task touches its systems: inside one app (macro); across systems with APIs, calling them yourself (script); across systems with APIs that already have ready-made connectors, clicking them together (low-code); or across systems with no API at all, mimicking clicks (RPA). Each one is a step further from "clean" and a step closer to "works on anything."

Answer in one line: A **macro** (VBA, Apps Script) suits automation entirely inside one application; a **script** (Python, calling APIs directly) suits automation across systems that expose clean APIs; a **low-code tool** (Zapier, Power Automate) suits cross-system automation where a visual builder and pre-built connectors save real development time; **RPA** (robotic process automation, simulating clicks and keystrokes) is the last resort for systems with no API and no low-code connector at all, brittle but sometimes the only option.

Tier	What to say
Passes	Names the four options but matches them to tasks only loosely
Strong	The four-way decision framework above, correctly matching each option to what it's actually suited for
Extra points	+ [+Business] RPA should be treated as a genuine last resort, not a default: it's the most fragile option, since it depends on a UI's exact layout never changing, and breaks immediately if a vendor redesigns their interface + [+Edge cases] the same task can move between categories over time: an RPA solution built because no API existed should be revisited once a real API becomes available, rather than left in place purely out of inertia

Likely follow-ups: When would you choose a low-code tool over writing a script, even though you're capable of writing the script? What's the maintenance cost difference between these four options?

Red flag: defaulting to RPA or a macro for a task that a clean, available API would handle far more reliably.

Learn it in: Chapter 63, section 63.3 (choosing the right tool for the job); Chapter 20, sections 20.1 (the automation ladder), 20.3 (choosing the delivery tool) and 20.10 (low-code automation); Chapter 51,

section 51.10 and Chapter 58, section 58.1 (RPA); macros themselves: Chapter 19. **Practise it with:** Chapter 70, sections 70.5–70.6 (the same platforms, tested there for syntax, here for when to reach for them at all).

Rapid-fire, 78.1

Roles: AUT, DA and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q78-002	What's the main maintenance risk of a low-code automation platform?	The logic lives inside a third-party tool's UI, often without version control, making it harder to review changes, roll back, or hand off to another person than a plain script would be	[+Trade-offs] the speed of building in a low-code tool is a real, genuine advantage; the maintainability trade-off is the honest cost of that speed	Fresher · 20.10, 63.3
Q78-003	Why might a company deliberately choose a script over a low-code tool even when both could do the job?	Version control, code review, and testing infrastructure the team already has for scripts, plus avoiding a recurring per-automation cost that many low-code platforms charge	[+Business] the "cheaper to build" option (low-code) and the "cheaper to maintain at scale" option (a well-engineered script) are often different tools	Mid · 20.3, 63.3
Q78-004	What's the single biggest failure mode of an RPA-based automation?	Any change to the target application's UI, even a cosmetic one, silently breaks the automation, since it's simulating a specific sequence of clicks tied to a specific layout	[+Evidence] an RPA bot that clicks "the third button from the left" breaks the moment a fourth button gets added anywhere before it	Fresher · 51.10, 58.1
Q78-005	How would you decide whether a task is even worth automating in the first place?	Weigh the time saved per run times how often it runs against the time it would take to build and maintain the automation, including the cost of it eventually breaking	[+Business] a task done once a year, however tedious, is rarely worth automating; the same task done daily almost always is	Fresher · 63.2, 20.14

78.2 Basic-but-tricky automation questions

Q78-023 · Does automating a broken manual process actually fix it?

Level: Fresher · **Roles:** AUT, DA, BA

Remember it as: *Automating a bad process just makes it fail faster and more consistently. Fix the process, then automate it, not the other way around.*

Answer in one line: No: automating a process that's fundamentally flawed (wrong logic, unclear ownership, missing edge-case handling) usually just executes the same flaw faster and more consistently, and can make the underlying problem *harder* to notice, since a human doing the task manually might catch an obviously wrong result before acting on it, while an automation won't.

Tier	What to say
Passes	Says the process should be reviewed first, without saying why
Strong	Explicitly names the risk: automation removes the human sanity-check step that was, often invisibly, catching errors before
Extra points	+ [+Business] the right sequence is almost always: understand and fix the process first, <i>then</i> automate the corrected version, not automate first and hope problems surface on their own

Likely follow-ups: How would you know, before automating, whether a manual process has a hidden human sanity-check step worth preserving? What would you add to the automated version to replace that lost check?

Red flag: treating "we automated it" as inherently synonymous with "we improved it."

Learn it in: Chapter 20, section 20.2 (map the flow before you automate); Chapter 58, section 58.10 (when not to automate). **Practise it with:** Chapter 76B, §76B.4 (Q76B-021: as-is vs. to-be).

Rapid-fire, 78.2

Roles: AUT, DA and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q78-024	Is "no-code" the same thing as "no technical skill required"?	No: a no-code tool removes the need to write syntax, but building a genuinely reliable automation in one still requires the same logical thinking (edge cases, error handling) a script would need	[+Business] a poorly-designed no-code automation fails in exactly the same ways a poorly-designed script does, just without a stack trace to help debug it	Fresher · 20.10
Q78-025	Why might a "quick win" automation built by a single enthusiastic employee become a long-term liability?	If it's undocumented and only that one person understands it, the automation becomes a single point of failure the moment they leave or go on leave, exactly the same risk as an undocumented process	[+Evidence] this is one of the most common real sources of "nobody knows why this stopped working" incidents	Fresher · 20.13, 63.8
Q78-026	Should every manual task with any repetition at all be automated eventually?	No: some tasks are rare enough, or variable enough in their exact steps, that the automation's build and maintenance cost never pays back the time saved	[+Business] ask Chapter 58's four questions in order: volume, variability, whether the rules can be written down, and the cost of an error; a task that fails the last one stays manual even at high volume	Fresher · 58.2, 58.10
Q78-027	What's the risk of over-customizing an automation for one specific stakeholder's exact preferences?	It becomes fragile and hard to extend to a second stakeholder with slightly different needs, and often duplicates logic that should have been one shared, parameterized design	[+Scale] with Q78-006's one-template, per-recipient-filter design, a second stakeholder is a new row in the recipient table, not a copy of the automation	Mid · 20.12

78.3 Report and alert automation

Q78-006 · Design an automated daily MIS email sent to 200 managers, each seeing only their own team's numbers

Level: Mid · **Roles:** AUT, DA, BI

Remember it as: One template, one query per recipient (or one query, filtered per recipient), not 200 separately hand-built reports.

Answer in one line: Build one parameterized report template and one data pull that includes every manager's team, then loop over the recipient list, filtering the data to each manager's own scope and rendering the same template with their specific numbers, rather than maintaining 200 separate report definitions.

Worked structure:

"One query pulls every manager's team performance in a single pass, tagged by manager ID. A loop iterates over the 200 managers, filters that single dataset down to each manager's rows, renders a shared HTML template with their specific numbers substituted in, and sends it via a transactional email service, not a personal inbox, so delivery is tracked and rate-limited sensibly. I'd log every send (recipient, timestamp, success or failure) so a manager who says 'I never got mine' can be checked against a real record instead of a guess, and I'd build in a retry for any individual send that fails, without blocking the other 199 emails from going out on time."

Tier	What to say
Passes	One template, but with a hand-maintained recipient list and no failure isolation
Strong	The one-template, one-filtered-pull, per-recipient-loop structure above
Extra points	+ [+Edge cases] an individual send failure (a bad email address, a temporary mail server issue) shouldn't block the other 199, exactly the "partial failure isolation" principle from Chapter 77's pipeline design, applied here to email delivery instead of data loading + [+Business] the send log is what turns "a manager complains they didn't get it" from a debugging mystery into a two-second lookup

Likely follow-ups: How would you handle a manager who should see two teams' data, not one? What would you monitor to know the whole batch actually completed successfully?

Red flag: a design that fails the entire batch if any single recipient's send fails.

Learn it in: Chapter 20, sections 20.5–20.6 (the report as an email, sending mail from code safely) and 20.12 (recipients and confidentiality: per-recipient filtering); logging every run: section 20.11; Chapter 19, section 19.7 (the VBA and Outlook version). **Practise it with:** Chapter 70, section 70.5 (the same pattern, tested for syntax).

Rapid-fire, 78.3

Roles: AUT, DA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q78-007	Why does an automated report need a "no news" state, not just "here's today's numbers"?	If a report silently sends nothing when there's genuinely nothing to report (versus when the pipeline feeding it failed), the recipient can't tell the difference between "all quiet" and "broken"	[+Edge cases] send a short "no sales were recorded" note on an empty day, so a missing email always means a broken job; see also Q77-031	Fresher · 20.8, 20.11
Q78-008	What's the risk of sending an automated alert on every single anomaly, however small?	Alert fatigue (Chapter 77, Q77-032): recipients start ignoring the automation entirely, including the one alert that actually mattered	[+Business] tune thresholds deliberately conservative for anything that emails a human directly, rather than flagging every minor fluctuation	Fresher · 20.8, 47.9
Q78-009	Should a report automation retry immediately on failure, or wait?	Usually wait, with backoff (Q78-017): an immediate retry against a system that's down or overloaded often just adds to the load causing the failure in the first place	[+Edge cases] retry only errors that might go away (a timeout, a busy server); a wrong password fails the same way every time	Fresher · 45.9, 46.6; §78.5
Q78-010	How would you let a stakeholder unsubscribe from an automated report without filing an IT ticket?	Build a simple self-service preference mechanism (a link, a form, a shared settings sheet) rather than requiring a developer to manually edit a recipient list every time	[+Business] the absence of self-service unsubscribe is a common, avoidable source of stakeholders quietly filtering the automation to spam instead of asking for changes	Mid · 20.12

78.4 APIs, webhooks, and reverse ETL

Q78-011 · What's the difference between polling an API and receiving a webhook, and when does each make sense?

Level: Mid · Roles: AUT, DE

Remember it as: Polling is you repeatedly asking "anything new?" A webhook is the other system calling you the moment something happens, so you never have to ask.

Answer in one line: **Polling** means repeatedly calling an API on a schedule to check for new data, simple to build but wasteful (most calls find nothing new) and inherently delayed by the polling interval; a **webhook** means the source system calls your endpoint the moment an event happens, near-instant and efficient, but requires the source to support webhooks at all and requires you to run a reliably available endpoint to receive them.

Tier	What to say
Passes	"Webhooks are more real-time" (true, no discussion of the trade-off or when polling is still the right choice)
Strong	The efficiency-vs-availability trade-off above, with a concrete case for each: polling suits a source with no webhook support, or where a few minutes of delay genuinely doesn't matter; webhooks suit anything needing near-real-time reaction
Extra points	+ [+Edge cases] a webhook can be delivered more than once for the same event (network retries on the sender's side), so the receiving endpoint needs idempotent processing (Q78-012), exactly the same discipline Chapter 77 tests for pipeline loads + [+Business] a webhook receiver going down, even briefly, can silently lose events forever unless the sender has its own retry/redelivery mechanism, worth confirming before relying on webhooks alone for anything critical

Likely follow-ups: How would you design a fallback in case your webhook receiver goes down for an hour? (A *scheduled reconciliation sync underneath the webhook*: Chapter 51, section 51.7.) What's a webhook signature, and why does an endpoint need to verify one?

Red flag: treating webhook delivery as guaranteed and exactly-once, with no idempotency handling.

Learn it in: Chapter 51, sections 51.6 (webhooks: pushing instead of polling, and HMAC signatures) and 51.7 (reconciliation as the safety net); delivery guarantees: Chapter 50, section 50.3. **Practise it with:** Chapter 77, Q77-008.

Q78-012 · Build a webhook handler that safely processes a "lead created" event, even if the same event gets delivered twice

Level: Mid · **Roles:** AUT, DE

Remember it as: Every webhook payload should carry its own unique event ID. Check the ID before processing; record it only once processing has succeeded, ideally in the same transaction.

Answer in one line: Check the incoming event's unique ID against a record of already-processed events before doing any work; if it's already been seen, acknowledge success and skip processing entirely, rather than reprocessing (and potentially duplicating downstream effects like creating a second CRM record).

Verified, live. A set of processed IDs stands in for the database table, and a list stands in for the CRM. Before you run it, predict what the second call prints, and how many leads the CRM holds at the end:

```
processed_ids = set()
crm_leads = []

def process_webhook(event_id, payload):
    if event_id in processed_ids:
        return "skipped (already processed)"
    crm_leads.append(payload)          # the real work: create the lead
    processed_ids.add(event_id)        # record the ID only after the work succeeded
    return f"processed: {payload}"

print(process_webhook("evt_123", "lead created"))
print(process_webhook("evt_123", "lead created")) # the same event, delivered twice
print("leads in the CRM:", len(crm_leads))
```

```

processed: lead created
skipped (already processed)
leads in the CRM: 1

```

How it works.

- `processed_ids` is a set, so `event_id in processed_ids` is an instant membership check (Chapter 33, section 33.2).
- `crm_leads.append(payload)` is the work the event asks for. Only after it succeeds does `processed_ids.add(event_id)` record the ID. If the work raised an error, the ID would never be recorded, so the sender's retry would be processed properly instead of skipped and lost.
- The second call finds `evt_123` already recorded and returns without touching the CRM: one lead, not two.

Tier	What to say
Passes	Knows duplicates can happen and suggests checking the database for an existing record
Strong	The event-ID-check pattern above, correctly skipping a genuine duplicate, with the ID recorded only after the work succeeds
Extra points	+ [+Validate] the real output above: the second identical delivery is correctly skipped, and the CRM holds one lead, not two + [+Business] in production, <code>processed_ids</code> would be a persistent store (a database table), not an in-memory set, since a set held only in memory is lost the moment the process restarts, silently reopening the exact duplicate-processing risk this pattern exists to close + [+Edge cases] two copies arriving at the same moment can both pass an in-memory check; a <code>UNIQUE</code> constraint on <code>event_id</code> in the database closes that race

Likely follow-ups: What HTTP status code should the webhook handler return, and why does that matter to the sender's retry behavior? How would you handle a webhook payload that fails processing partway through? (*Don't record its ID, so the sender's retry processes it again; that's safe only if the work itself can be repeated.*)

Red flag: an in-memory-only duplicate check presented as a complete, production-ready solution with no mention of its persistence gap.

Learn it in: Chapter 51, sections 51.6 (webhooks: "expect duplicates") and 51.3–51.4 (idempotency keys, and proving a write is idempotent). **Practise it with:** Chapter 77, Q77-007 (idempotent upserts, the same principle applied to a database load instead of an event handler).

Rapid-fire, 78.4

Roles: AUT and DE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q78-013	What's reverse ETL, and how is it different from normal ETL/ELT?	Normal ETL/ELT moves data <i>into</i> a warehouse for analysis; reverse ETL moves already-transformed, warehouse-computed data (like a churn score) <i>out</i> to operational tools (a CRM, an email platform) so business teams can act on it directly in the tools they already use	[+Business] reverse ETL is what turns a model's output (Chapters 36–39) into something a sales rep actually sees inside their CRM, not just a table in a warehouse only analysts can query	Fresher · 51.1–51.4

#	Question	One-line answer	Extra point	Level · learn it in
Q78-014	Why does a reverse ETL sync into a CRM need to be an upsert, not a plain insert?	Re-running the sync (daily, say) would otherwise create a duplicate CRM record for every lead on every single run, instead of updating the existing one's score	[+Validate] Q78-028 runs it: lead_101's new score updates the existing record (72 → 80) rather than creating a second one, and the count stays at 3 after a new and an existing lead are synced, and again after a re-run	Mid · 51.4
Q78-015	What's an API rate limit, and why does an integration need to respect it proactively?	A cap on how many requests a system will accept in a given time window; exceeding it usually gets your requests rejected or your access temporarily blocked, which is worse for the automation than simply pacing itself	[+Validate] the limiter below allows 3 calls in a 1-second window, rejects the 4th and 5th, and allows the next call once the window has moved on	Mid · 29.9, 45.9
Q78-016	What's an API key vs. OAuth, at a level useful for an integration decision?	An API key is a single, simple secret string identifying the calling application; OAuth is a more complex flow granting scoped, revocable, user-specific access without ever sharing the user's actual password	[+Business] OAuth is the right choice whenever an integration needs to act <i>on behalf of a specific user</i> , not just as an application in general	Mid · 51.3, 52.6

Worked example: a rate limiter (Q78-015)

The plan: remember the times of the calls you allowed; before each new call, forget any that are a full window old; allow the call only if fewer than the cap remain. The times are passed in as numbers of seconds, so the run comes out the same every time; a real limiter passes the clock's current reading instead.

```
from collections import deque

MAX_CALLS = 3      # at most 3 calls...
WINDOW = 1.0       # ...in any 1-second window
recent = deque()    # times of the calls allowed so far

def allow(now):
    while recent and now - recent[0] >= WINDOW:
        recent.popleft()
    if len(recent) < MAX_CALLS:
        recent.append(now)
        return True
    return False

for t in [0.0, 0.1, 0.2, 0.3, 0.4, 1.05]:
    print(f"call at {t:.2f}s:", "allowed" if allow(t) else "rejected")
```

```
call at 0.00s: allowed
call at 0.10s: allowed
call at 0.20s: allowed
call at 0.30s: rejected
call at 0.40s: rejected
call at 1.05s: allowed
```

How it works.

- `deque()` is the double-ended queue from Chapter 33, section 33.4: `append` adds a time at the right, and `popleft()` removes the oldest from the left.
- `MAX_CALLS` and `WINDOW` are the cap: 3 calls per 1.0 second.
- The `while` loop drops every remembered call that is `WINDOW` or more seconds older than `now`; `recent` and `...` stops it when the deque is empty.
- `len(recent) < MAX_CALLS` asks whether there's room. If there is, the call is remembered and allowed (`True`); if not, it's rejected (`False`) and not remembered.
- At 1.05 s, the call made at 0.00 s is more than a second old, so it's dropped; two remain, and the new call fits.

In an integration, "rejected" means *wait*, not *give up*: the script sleeps until the oldest call leaves the window, then sends. A server that rejects you anyway answers 429, and Chapter 45, section 45.9 retries it after the `Retry-After` wait.

78.5 Failure handling and monitoring for automations

Q78-017 · An automation calls a flaky third-party API that sometimes fails transiently. Build retry logic that doesn't hammer the failing service

Level: Mid · **Roles:** AUT, DE

Remember it as: *Retrying immediately just adds load to a service that's already struggling. Waiting longer after each failure gives it room to recover.*

Answer in one line: Use **exponential backoff**: retry after a short delay, and double that delay after each subsequent failure, up to a maximum number of attempts, so a transient blip gets a quick retry while a sustained outage doesn't get hammered with rapid-fire requests that make the underlying problem worse.

Verified, live, in three cells. First, a stand-in for the flaky API: it fails on its first two calls and works on the third. A dictionary keeps the call count, so the function can change it:

```
import time

state = {"calls": 0}

def flaky():
    state["calls"] += 1
    if state["calls"] <= 2:
        raise ConnectionError("service unavailable")
    return "data received"
```

- `import time` gives `time.sleep()`, which the next cell uses to wait.
- `state["calls"] += 1` counts each call. The first two raise `ConnectionError`, Python's built-in error for a failed connection; the third returns a result.

Next, the retry function. It prints what it does, so you can watch it:

```
def call_with_retry(func, max_attempts=5, base_delay=0.01):
    for attempt in range(1, max_attempts + 1):
        try:
            result = func()
            print(f"succeeded on attempt {attempt}")
            return result
        except ConnectionError:
            if attempt == max_attempts:
                raise
            delay = base_delay * (2 ** (attempt - 1))
            print(f"attempt {attempt} failed, retrying in {delay:.3f}s")
            time.sleep(delay)
```

- `func` is the call to protect, passed in without brackets so that `call_with_retry` can call it. `max_attempts=5` caps the tries; `base_delay=0.01` is the first wait in seconds (tiny here; a real API gets a second or more).
- `range(1, max_attempts + 1)` counts the attempts 1 to 5.
- `except ConnectionError:` catches only that transient error. Anything else (a wrong password, a missing record) isn't caught, so it fails fast instead of being retried.
- `if attempt == max_attempts: raise` gives up on the last attempt. A bare `raise`, with nothing after it, re-throws the error that just happened, so the caller sees the real failure.
- `2 ** (attempt - 1)` is 1, 2, 4, 8..., so the delay doubles after each failure: 0.010 s, then 0.020 s, then 0.040 s.

Before you run it, predict how many lines it prints:

```
print(call_with_retry(flaky))
```

```
attempt 1 failed, retrying in 0.010s
attempt 2 failed, retrying in 0.020s
succeeded on attempt 3
data received
```

Reading it. Two failures, each followed by a longer wait than the last, then success on the third attempt; the last line is `flaky()`'s own result, passed back through. **What happens if you change it:** give it a service that never recovers, and only three attempts:

```
def always_down():
    raise ConnectionError("service unavailable")

try:
    call_with_retry(always_down, max_attempts=3)
except ConnectionError as err:
    print("gave up:", err)
```

```
attempt 1 failed, retrying in 0.010s
attempt 2 failed, retrying in 0.020s
gave up: service unavailable
```

- `always_down()` raises `ConnectionError` on every call: a service that never comes back.
- `call_with_retry(always_down, max_attempts=3)` allows three attempts instead of five.
- `try: ... except ConnectionError as err:` catches the error that `call_with_retry` finally re-throws; `as err` names it, so `print("gave up:", err)` can show its message.

The third failure isn't retried: `raise` hands it to the caller, which reports it. Without the cap, this loop would run forever.

Tier	What to say
Passes	Adds a fixed delay between retries, with a cap
Strong	The exponential backoff pattern above, with a maximum attempt cap so it eventually gives up rather than retrying forever, and retries only transient errors (429, 503, timeouts), not 400, 401 or 404
Extra points	+ [+Validate] the real runs above: two failures, each with a longer wait than the last, then success on the third attempt; and a clean give-up when the service never recovers + [+Edge cases] a real implementation should add jitter (a small random amount added to each delay) so that if many clients are retrying the same recovering service at once, they don't all retry at exactly the same synchronized moment and cause a new spike

Likely follow-ups: What's the difference between a transient error worth retrying and a permanent one that isn't? How would you decide the maximum number of attempts?

Red flag: a retry loop with no delay at all, or no maximum attempt limit, risking an infinite retry loop.

Learn it in: Chapter 29, section 29.9 (exponential backoff with jitter); Chapter 45, section 45.9 (which errors to retry, and `Retry-After`); Chapter 46, section 46.6 (retry policies and timeouts); why a retry must be idempotent: Chapter 46, section 46.4.

Q78-018 · An automation that's run reliably for months silently stopped working last week. Walk through how you'd find out, and how you'd prevent it happening silently again

Level: Mid · **Roles:** AUT, DA, DE

Remember it as: *"It stopped running" and "it ran but did nothing useful" are two different failures, and the second one is far more dangerous because nothing looks broken from the outside.*

Answer in one line: First check whether it's actually still executing at all (a scheduler log, a "last run" timestamp) versus running but silently failing partway through or producing empty/wrong output; then check what changed around the time it stopped (an expired credential, an upstream API change, a permissions change); then, regardless of the specific cause found, add a monitoring check so the next silent failure surfaces immediately instead of a week later.

Worked structure:

"I'd check the scheduler first: did it even attempt to run? If yes, I'd check its logs for the actual error, most commonly an expired API token or OAuth credential, a changed API response format, or a permissions change on the target system. If it ran with no error but produced no visible effect, I'd suspect a silent logic issue, like a filter condition that now matches zero rows because of an upstream data change. Either way, once I've found and fixed the specific cause, I'd add a monitoring check (a freshness or heartbeat check, Chapter 47, section 47.5 and Q77-031, applied here to an automation instead of a data pipeline): alert if this automation hasn't successfully completed within its expected window, rather than relying on someone noticing its absence."

Tier	What to say
Passes	Investigates and fixes the immediate cause with no mention of preventing a future silent recurrence
Strong	The two-stage diagnosis above (did it run at all vs. did it run but do nothing useful), plus a monitoring fix to close the loop
Extra points	+ [+Business] explicitly distinguishes "stopped executing" from "executed but silently did nothing," since the second is a genuinely harder, more dangerous failure mode to notice: Chapter 46's bad morning (section 46.7), when a load reported "0 new" as if it were a normal quiet day, now applied to a business-facing automation instead of a data pipeline

Likely follow-ups: What's the most common real-world cause of an automation silently failing after months of working fine? How would you design the monitoring alert to avoid false positives on a legitimately quiet day (a holiday, say)?

Red flag: fixing the immediate cause with no monitoring added to catch the next occurrence automatically.

Learn it in: Chapter 20, section 20.11 (making an automation trustworthy: log every run, alert on failure); Chapter 47, section 47.5 (freshness); Chapter 63, section 63.7 (ownership and runbooks).

Practise it with: Chapter 77, Q77-028 and Q77-031.

Rapid-fire, 78.5

Roles: AUT, DA and DE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q78-019	What's the most common real cause of a previously-working automation breaking without any code change?	An expired credential, API key, or OAuth token; these have finite lifespans and their expiry is rarely tied to any code deployment, so nothing in version control explains the sudden failure	[+Business] tracking credential expiry dates proactively (a calendar reminder, a monitoring check) avoids being surprised by this specific, very common failure mode	Fresher · 51.3, 52.6

#	Question	One-line answer	Extra point	Level · learn it in
Q78-020	Why should an automation log its own successful runs, not just its failures?	Without a positive "last successful run" record, there's no way to distinguish "silently broken for a week" from "just ran successfully an hour ago"	[+Validate] a separate check reads the "last successful run" time and alerts when it's too old, so the monitor doesn't depend on the automation remembering to complain; see also Q77-031	Fresher · 20.11, 47.5
Q78-021	What's a circuit breaker pattern, in automation terms?	After a certain number of consecutive failures calling an external system, stop trying entirely for a cooldown period, rather than continuing to retry a system that's clearly down	[+Trade-offs] trades a short window of "giving up early" against not making a struggling system's outage worse by continuing to hammer it	Mid · 57.8, 58.6, 61.6
Q78-022	Should an automation's failure alert go to an individual person or a team channel?	Generally a team channel or shared on-call rotation, not one individual, so the automation doesn't silently go unmonitored the day that one person is on leave	[+Business] a single point of alerting failure is exactly as fragile as a single point of automation failure	Mid · 63.7

On Q78-021: Chapter 57 (section 57.8, exercise 12) described a circuit breaker for calls to a model provider; Chapter 58 (section 58.6) built one that stops a whole batch when too many items fail; Chapter 61 (section 61.6) sets it beside timeouts and retries.

78.6 Full design case

Q78-006, Q78-018 and Q78-001 are also full design cases; practise them aloud as well.

Q78-028 · Design case: sync lead scores into the CRM without creating duplicates

Level: Senior · **Roles:** AUT, DE

What they're really testing: whether the upsert discipline gets applied correctly to a live, operational sync target, not just a data warehouse table.

Talked through live: "Each lead needs a stable external ID shared between the scoring system and the CRM, either the CRM's own record ID if the scoring system already knows it, or an agreed common key like email. The sync checks for an existing record by that ID: if found, update its score field; if not, create a new record. This is exactly the upsert pattern from idempotent loads, just targeting a CRM's API instead of a database table."

Verified, live. A dictionary stands in for the CRM, keyed by lead ID:

```
existing_crm_records = {"lead_101": {"score": 72}, "lead_102": {"score": 55}}

def sync_lead_score(lead_id, score):
    if lead_id in existing_crm_records:
        old = existing_crm_records[lead_id]["score"]
        existing_crm_records[lead_id]["score"] = score
        return f"updated {lead_id}: {old} -> {score}"
    existing_crm_records[lead_id] = {"score": score}
    return f"created {lead_id} with score {score}"

print(sync_lead_score("lead_101", 80))
print(sync_lead_score("lead_103", 60))
print("records in the CRM:", len(existing_crm_records))
```

```
updated lead_101: 72 -> 80
created lead_103 with score 60
records in the CRM: 3
```

How it works.

- `existing_crm_records` is the CRM: each key is the stable external ID, each value the record.
- `if lead_id in existing_crm_records:` is the "look up by external ID" step. If the lead exists, the function keeps the `old` score for the message and overwrites the score field: an update.
- Otherwise it adds a new record: a create. Update-or-create is the **upsert**.
- `len(existing_crm_records)` counts the records: 3, because `lead_101` was updated in place, not added a second time.

Now the test that matters for a daily sync: run the same writes again.

```
print(sync_lead_score("lead_101", 80))
print(sync_lead_score("lead_103", 60))
print("records in the CRM:", len(existing_crm_records))
```

```
updated lead_101: 80 -> 80
updated lead_103: 60 -> 60
records in the CRM: 3
```

The second run changes nothing and adds nothing: the sync is safe to repeat.

Extra-points moves demonstrated: **[+Signpost]** explicitly connected this to the upsert pattern of idempotent loads rather than treating it as a new problem. **[+Edge cases]** named the stable-external-ID requirement as the actual precondition for this working at all, not an afterthought. **[+Validate]** the real record count (3, not 4), after both runs, proves no duplicate was created.

Likely follow-ups: What would you do if the CRM's API doesn't support a native upsert operation, only separate create and update calls? (*Look the lead up by its ID, then PATCH the one field or create the record, with an idempotency key on each write: Chapter 51, sections 51.3–51.4.*) How would you handle a lead that exists in the CRM but was deleted from the source scoring system?

Red flag: a sync design using a plain "create" call on every run, with no check for an existing record

first.

Learn it in: Chapter 51, sections 51.3–51.5 (writing through an API, idempotent writes, system of record). **Practise it with:** Chapter 77, Q77-007 (the database version).

APPLY

Common mistakes

Mistake	Symptom	Fix
Defaulting to RPA or a macro regardless of fit	A fragile automation where a clean API would have worked	Match the tool to how the task actually touches its systems (§78.1)
An immediate, no-delay retry loop against a flaky API	Adds load to an already-struggling service, can worsen an outage	Exponential backoff with a maximum attempt cap
A webhook handler with no duplicate-event check	A duplicate delivery creates a duplicate downstream record	Check a unique event ID before processing, persisted, not in-memory only
A CRM sync using plain "create" every run	Duplicate records pile up on every sync cycle	Upsert by a stable external ID
No monitoring for an automation's own silent failure	A break goes unnoticed for days or weeks	Alert on absence of a successful run, not just on explicit errors
Automating a broken process as-is	The flaw executes faster and more consistently, harder to notice	Fix the process first, then automate the corrected version
One person quietly owning an undocumented "quick win" automation	A single point of failure when that person is unavailable	Document ownership and logic; don't let critical automation live in one person's head

In the real world: the automation that broke because of a renamed column

Meera is asked in an interview to describe debugging a real automation failure. She describes a weekly commission-calculation automation that had run correctly for over a year, then started producing wrong numbers, not failing outright, just quietly wrong, which took two pay cycles to notice because nobody was checking the automation's output against a manual calculation anymore, exactly the trust that made the automation valuable in the first place also made its silent failure harder to catch.

She traces it to a source spreadsheet where someone had renamed a column from "Discount %" to "Discount Percentage," and her script had been reading that column by its exact name. The rename didn't break the script outright, since the script's error handling silently defaulted a missing column to zero rather than crashing, meaning discounts were being read as zero for every single row for two full pay cycles, quietly inflating every calculated commission.

The interviewer's follow-up: what did she change? Her answer: two things, not one. First, the immediate fix: read the column by name, and fail loudly with a clear message if that name isn't found, instead of silently defaulting to zero. Second, and the part she says mattered more: "I stopped treating

'the automation ran without an error' as proof it worked correctly. Now it validates its own output against a rough expected range before sending anything downstream, so a silently wrong result gets caught the same day, not two pay cycles later." That's this chapter's whole method: the fix for the specific bug matters less than the systemic change that catches the *next* silent failure automatically.

Project

Goal: apply this chapter's method to a real automation of your own.

Tools you'll need

Python for scripted automation and API integration; **VBA/Apps Script** (Chapter 19) for in-application macros; common low-code platforms (Zapier, Power Automate, Make; Chapter 20, section 20.10) for cross-system automation without custom code; a scheduler (cron, Chapter 20, section 20.7, or an orchestrator such as Dagster, Chapter 46) for anything running on a timer. Every code pattern in this chapter uses only Python's standard library (`time` and `collections`), so it runs in the environment Chapter 17 set up, with nothing new to install.

1. Pick one manual, repetitive task from your own work and run it through Q78-001's decision framework: which of the four approaches actually fits, and why.
2. Add retry-with-backoff logic to one real script you have that calls an external system, and test that it actually waits longer between successive failures.
3. Add a duplicate-event or duplicate-record check to one integration you maintain, and prove (the way Q78-012 and Q78-028 did) that running it twice doesn't create a duplicate.
4. Write the monitoring check you'd add to catch your most critical automation silently failing, and say specifically what "silently failing" would look like for that one.

REVIEW

Key terms

macro vs. script vs. low-code vs. RPA · report automation · alert fatigue · polling vs. webhook · webhook idempotency · reverse ETL · API rate limit · API key vs. OAuth · exponential backoff · jitter · circuit breaker · silent failure · monitoring (absence of success) · upsert (in an integration context) · single point of failure (undocumented automation)

Final-week revision list

Q78-001, Q78-006, Q78-011, Q78-012, Q78-013, Q78-017, Q78-018, Q78-023, Q78-028.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 77, Data Engineering & Data System Design Bank**, already covers idempotency, delivery guarantees, and monitoring from the data-pipeline angle; this chapter applies the identical discipline to business-facing automation and integrations. Retries with backoff are taught in Chapter 45, section 45.9 and Chapter 29, section 29.9.
- **Chapter 63, section 63.3, and Chapter 20, sections 20.1, 20.3 and 20.10**, teach the tool choice behind §78.1, and Chapter 51, section 51.10 and Chapter 58, section 58.1 cover RPA; **Chapter 70, sections 70.5–70.6**, test the VBA/Apps Script syntax this chapter's framework helps decide when to actually reach for.
- **Chapter 76B, §76B.4 (Q76B-021)**, tests the as-is-before-to-be discipline this chapter's Q78-023 reuses for automation; Chapter 20, section 20.2 teaches it.

Chapter 79. GenAI, LLM & MLOps Question Bank

Chapter at a glance

You will learn to: answer the LLM fundamentals, RAG design, and evaluation questions that come up in GenAI-adjacent Data Scientist, ML Engineer, and AI Engineer interviews · reason correctly about embeddings, chunking, and retrieval, not just name the components · explain how a model actually survives production (registries, serving, monitoring, retraining) · design a RAG assistant and a model-serving platform end to end, stating trade-offs out loud.

Before you start: Chapters 54–57 (Generative AI and LLMs; RAG, agents and evaluation; MLOps; LLMOps), which teach every idea in this bank; Chapter 35, section 35.2 for the dot product and cosine similarity; Chapter 41, section 41.7 for word embeddings; Chapter 74 for the classical-ML side of monitoring; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 3½–4½ hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, 20 minutes for each design case, and about 45 minutes to run the four code demos yourself. Plus 45 minutes for the final-week list.

How this chapter is built. Same format as the other question banks. Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, tagged the same way: **[+Clarify]**, **[+Edge cases]**, **[+Validate]** and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it (a bare number such as 54.2 means that section). The four **Run it yourself** demos (cosine similarity, chunking, a drift statistic, and a cost estimate) are complete: copy a demo's cells, in order, into a fresh notebook and they print what's shown (Python 3.11, NumPy 2.4.6, SciPy 1.17.1; NumPy is installed in Chapter 18, SciPy in Chapter 21, section 21.5). Prompting technique, agent design, and vendor behaviour can't be "run" the way a retrieval score can, so they are reasoned through, with a pointer to the chapter that measured them. Ideas no earlier chapter teaches are marked **Beyond the book** and carry their own short explanation.

In this chapter

LEARN	79.1 LLM fundamentals	317
	79.2 Basic-but-tricky GenAI questions	320
	79.3 Retrieval-Augmented Generation (RAG)	322
	79.4 Agents and tool use	325
	79.5 Evaluating AI applications	327
	79.6 MLOps: making models survive production	328
	79.7 LLMOps	331
	79.8 Full design cases	333
APPLY	Common mistakes · In the real world · Project	335
REVIEW	Key terms · Final-week revision list	337
NEXT	Where this leads	338

Levels and roles. Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **AIE** AI engineer (builds products on top of AI models) · **DS** data scientist · **MLE** machine learning engineer.

A note on currency. GenAI tooling and model capabilities change fast. This chapter teaches durable ideas (how RAG works, why chunking matters, what a drift statistic measures) rather than specific model names or context-window sizes, which go stale within weeks. The only prices used, in Q79-033, are Chapter 54's price tiers, checked 29 September 2026. Verify any current specifics against the providers' own pages before an actual interview.

LEARN

79.1 LLM fundamentals

Q79-001 · What's a token, and why does a model's context window get measured in tokens, not words or characters?

Level: Fresher · **Roles:** AIE, DS, MLE

Remember it as: A token is roughly a chunk of a word, not a whole word, which is exactly why "context window" numbers look bigger than you'd expect from word count alone.

Answer in one line: A **token** is the basic unit a language model actually processes, often a word, part of a word, or punctuation mark, produced by a tokenizer that breaks text into these sub-word pieces; models measure context (and pricing) in tokens because that's the model's native unit of computation, and it doesn't map cleanly to word count, since a rare or long word can split into several tokens while a common short word is one.

Tier	What to say
Passes	"A token is basically a word" (a rough approximation, misses the sub-word splitting that actually matters for the answer)
Strong	Explains that a token is often a sub-word unit, and that this is precisely why token counts and word counts diverge, especially for uncommon words, code, or non-English text
Extra points	[+Business] this divergence matters practically: rough ratios are commonly cited (about four characters, or about 0.75 English words, per token; Chapter 54 prices with the four-characters rule), but they break down for code, product codes, technical jargon, or other languages, and any cost or context-length estimate built on a clean word-to-token ratio should be treated with real caution

Likely follow-ups: Why does the same context window "fit less" for code than for plain English text? How does tokenization affect a model's ability to do character-level tasks, like counting letters in a word?

Red flag: treating "tokens" and "words" as interchangeable when estimating cost or context usage.

Learn it in: Chapter 54, section 54.2 (tokens, and byte-pair encoding by hand).

Q79-002 · What's an embedding, and why does cosine similarity, not Euclidean distance, dominate as the comparison metric for them?

Level: Mid · **Roles:** AIE, DS, MLE

Remember it as: Cosine similarity asks "do these two vectors point in the same direction," ignoring how long each vector is. For meaning-based comparison, direction is usually what actually matters, not magnitude.

Answer in one line: An **embedding** is a dense numeric vector representing a piece of text (or an image, audio, etc.) such that semantically similar inputs produce vectors that are close together in that vector space; **cosine similarity** measures the angle between two vectors, ignoring their length, which tends to reflect things like text length or intensity rather than meaning, so the angle is a more reliable meaning-similarity signal than raw distance.

Run it yourself. A query and three tiny three-number "embeddings": a document about returns, a document about something else, and a third that points exactly the way the returns document does but is twice as long.

```
import numpy as np

query = np.array([0.9, 0.1, 0.0])      # a question about returns
doc_a = np.array([0.85, 0.15, 0.05])  # a document about returns
doc_b = np.array([0.1, 0.2, 0.95])    # a document about something else
doc_c = np.array([1.7, 0.3, 0.1])     # doc_a's direction, twice as long

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

for name, doc in [("doc_a", doc_a), ("doc_b", doc_b), ("doc_c", doc_c)]:
    cosine = cosine_sim(query, doc)
    distance = np.linalg.norm(query - doc)
    print(f"{name}: cosine {cosine:.3f}    distance {distance:.3f}")
```

```
doc_a: cosine 0.996    distance 0.087
doc_b: cosine 0.124    distance 1.246
doc_c: cosine 0.996    distance 0.831
```

How it works:

- `np.array([...])` turns each list into a NumPy array (Chapter 18), so arithmetic works on all three numbers at once.
- `np.dot(a, b)` is the dot product: multiply matching numbers and add them up (Chapter 35, section 35.2). By hand for the query and `doc_a`: $0.9 \times 0.85 + 0.1 \times 0.15 + 0 \times 0.05 = 0.765 + 0.015 + 0 = 0.78$.
- `np.linalg.norm(v)` is the vector's length, the square root of the sum of its squares: $\sqrt{(0.9^2 + 0.1^2 + 0^2)} = 0.9055$ for the query and 0.8646 for `doc_a`. Dividing the dot product by both lengths gives the cosine: $0.78 \div (0.9055 \times 0.8646) = 0.78 \div 0.7829 \approx 0.996$.
- `np.linalg.norm(query - doc)` is the Euclidean distance: subtract the vectors, then take the length of the difference (Chapter 35, section 35.1). The loop stores each number under a name, then prints both.
- `:.3f` prints three decimals.

Reading it. `doc_a` and `doc_c` get the same cosine, 0.996, because they point the same way; only their lengths differ. Euclidean distance treats them very differently (0.087 against 0.831) purely because `doc_c` is longer. That is the case for cosine: a longer or more emphatic text shouldn't count as less similar in meaning.

Tier	What to say
Passes	"Embeddings capture meaning as numbers" (correct, no explanation of the actual comparison mechanism)
Strong	The direction-versus-length distinction above, correctly identifying why cosine similarity is the standard choice for comparing embeddings
Extra points	[+Validate] the computed scores above: 0.996 for a related pair and 0.124 for an unrelated one, a large separation, and the same 0.996 for a vector twice as long [+Edge cases] many embedding models return unit-length vectors (Chapter 54, section 54.8 scales its own to length 1); for those, cosine similarity equals the dot product, and ranking by Euclidean distance gives the same order, so the choice mostly matters when vector lengths differ [+Simple first] it's the same cosine similarity Chapter 35, section 35.2 used to compare customers' product mixes; embeddings are one of its highest-leverage uses

Likely follow-ups: What happens to retrieval quality if two genuinely different documents happen to produce very similar embeddings? How would you visualize a set of embeddings to sanity-check they're capturing meaningful structure?

Red flag: not knowing why cosine similarity, specifically, is the standard choice over a plain distance.

Learn it in: Chapter 35, section 35.2 (dot product and cosine similarity); Chapter 41, section 41.7 and Chapter 54, section 54.8 (embeddings).

Rapid-fire, 79.1

Roles: AIE, DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-003	What's a context window, in plain terms?	The maximum amount of text (measured in tokens) a model can consider at once, including both the prompt and its own generated response	[+Business] a conversation or document exceeding the context window must be truncated or summarized somehow, or the model simply can't see the parts that don't fit	Fresher · 54.4
Q79-004	What's temperature, in an LLM's generation settings?	A setting controlling how random or deterministic the model's next-token choice is; low temperature favors the most likely next token consistently, high temperature allows more varied, less predictable output	[+Business] a factual extraction task usually wants low temperature (consistency); a creative writing task often wants it higher	Fresher · 54.5
Q79-005	What's the difference between fine-tuning and prompting (few-shot examples) as ways to adapt a model's behavior?	Fine-tuning updates the model's own weights on new examples, a heavier, more permanent change; prompting adapts behavior at inference time through instructions and examples in the prompt itself, with no weight changes at all	[+Trade-offs] fine-tuning can achieve a more consistent behavior change for a narrow task, at real cost and complexity; prompting is far cheaper to iterate on and is often sufficient	Mid · 54.9
Q79-006	What's a multimodal model?	A model that can process and/or generate more than one type of input or output, text and images together, for instance, rather than being restricted to text alone	[+Business] a scanned purchase order or a photo of a damaged delivery is exactly the input a text-only model can't read	Fresher · 54.10

79.2 Basic-but-tricky GenAI questions

Q79-007 · If an LLM "hallucinates" a confident, plausible-sounding wrong answer, is that a bug in the model?

Level: Fresher · **Roles:** AIE, DS, MLE

Remember it as: A language model is fundamentally predicting a plausible next token, not looking up a verified fact. Hallucination isn't a malfunction, it's the expected behavior of that mechanism when it has no grounding to lean on.

Answer in one line: Not exactly a "bug" in the traditional sense: an LLM generates text by predicting statistically plausible continuations, and when it lacks reliable information about a specific fact, it can still generate fluent, confident-sounding text that's simply wrong, since fluency and factual accuracy are not the same thing the model is optimizing for; this is precisely why grounding techniques like RAG (section 79.3) exist, to give the model real, retrieved information to base its answer on instead of relying purely on its trained-in, sometimes-wrong "memory."

Tier	What to say
Passes	"Hallucination means the model makes things up sometimes" (true, no explanation of the underlying mechanism)
Strong	The plausibility-vs-accuracy distinction above, connecting it directly to why RAG and grounding are the standard mitigation
Extra points	[+Business] a system with no grounding and no way to say "I don't know" will always eventually hallucinate on a question outside its reliable knowledge; the fix isn't a "smarter" model alone, it's an architecture (retrieval, citations, a refusal threshold) that gives the model a real alternative to guessing

Likely follow-ups: How would you measure a system's hallucination rate in practice? What's the risk of a RAG system hallucinating even *with* retrieved context available?

Red flag: treating hallucination as a simple defect that better prompting alone fully solves.

Learn it in: Chapter 54, section 54.11 (limits and risks); Chapter 55, section 55.6 (grounding, citations, and refusal).

Rapid-fire, 79.2

Roles: AIE, DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-008	Does a bigger context window mean you should always stuff in as much context as possible?	No: more context costs money and time, and models use information buried in the middle of a long prompt worse than information at the start or end, so answer quality can drop even when the relevant text is technically present	[+Business] relevant, well-curated context (via good retrieval, section 79.3) usually beats simply maximizing context volume	Mid · 54.4, 55.1
Q79-009	Is prompt engineering a durable, transferable skill, or is it fragile to the specific model?	Some general principles transfer (clarity, structure, examples), but specific prompt wording that works well on one model version can behave differently on another, an ongoing maintenance cost worth planning for	[+Scale] a production system built on carefully tuned prompts needs a regression test on its golden set whenever the provider ships a new model version, exactly like code needs tests when a dependency updates	Mid · 57.4
Q79-010	Can you fully prevent prompt injection in a system that takes user input and feeds it to an LLM?	Not with complete certainty using prompting alone; layered defenses (instructions separated from data, output validation, limiting what the model is allowed to actually do) reduce risk but don't eliminate it	[+Limits] say plainly that no defense closes it fully, then name the layer that matters most: model output never triggers an action without validation	Mid · 54.11; 55.10 (injection through your own documents); governance in Chapter 64

#	Question	One-line answer	Extra point	Level · learn it in
Q79-011	Is an LLM-based system deterministic, given the exact same input twice?	Not necessarily, even at temperature 0, due to factors like non-deterministic floating-point computation, request batching, and model versions changing under the same name; treat outputs as close-to-deterministic, not guaranteed identical	[+Edge cases] some current models don't accept a temperature setting at all; pin the model version, store the output you validated, and prefer rule-based or semantic checks to exact-match tests	Mid · 54.5, 54.11

79.3 Retrieval-Augmented Generation (RAG)

Q79-012 · Walk through RAG end to end, and implement a simple chunking function

Level: Mid · **Roles:** AIE, DS, MLE

Remember it as: Retrieve first, then generate. The model only answers using what was actually found, not purely from what it happened to memorize during training.

Answer in one line: RAG breaks source documents into **chunks**, converts each chunk into an **embedding**, stores them in a **vector index**; at query time, the query is embedded and compared against that index to **retrieve** the most relevant chunks, which are then inserted into the model's prompt as grounding context before it **generates** an answer, ideally with **citations** back to the source chunks used.

Run it yourself. A fixed-size chunking function with overlap, run on one bullet of Riverstone's returns policy (policy_returns.md in Chapter 55's corpus):

```
def chunk_text(text, chunk_size=60, overlap=15):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks

policy = ("Goods ordered in error may be returned within 15 days if unused "
        "and in original packaging. A restocking fee of 10% applies, "
        "and freight both ways is charged to the customer.")
print(f"{len(policy)} characters")
for i, chunk in enumerate(chunk_text(policy)):
    print(f'chunk {i}: "{chunk}"')
```

```
173 characters
chunk 0: "Goods ordered in error may be returned within 15 days if unu"
chunk 1: " 15 days if unused and in original packaging. A restocking f"
chunk 2: " A restocking fee of 10% applies, and freight both ways is c"
chunk 3: " both ways is charged to the customer."
```

How it works:

- `start` is where the next chunk begins. Each pass takes `text[start:end]`, 60 characters, then moves `start` on by `chunk_size - overlap = 45`, so each chunk repeats the last 15 characters of the one before.
- Slicing past the end of a string just stops at the end, which is why the last chunk is shorter (38 characters); the `while` loop ends once `start` passes the length.
- `overlap` must be smaller than `chunk_size`. If they were equal, `start` would never move and the loop would run forever. Try changing it to `overlap=0`: you get three chunks instead of four, and the word "unused" is cut into "unu" and "sed" with neither chunk holding it whole.
- Python joins strings written side by side into one, so the brackets around `policy` only let the text run over three lines. `enumerate` numbers the chunks from 0.

Reading it. The 15 overlapping characters, " 15 days if unu", appear at the end of chunk 0 and the start of chunk 1: that redundancy is what keeps a fact from vanishing at a cut. Notice too that the cuts land mid-word ("unu", "restocking f", "is c"): character chunking doesn't know what a word or a sentence is. And chunk 0 carries "15 days" without the 10% restocking fee, so an answer built from chunk 0 alone would be incomplete, the failure this chapter's real-world story is about.

Tier	What to say
Passes	Names the RAG pipeline stages in the right order, can't produce working chunking logic
Strong	The working overlap-chunking function above, correctly explaining <i>why</i> overlap exists: without it, a sentence split exactly at a chunk boundary can lose meaning that only makes sense with the text on the other side of the cut
Extra points	[+Validate] the output above shows the overlap in action (" 15 days if unu" on both sides of the first cut), and shows its limit: chunk 0 still lacks the fee [+Edge cases] fixed-size chunking is simple but cuts words and sentences in half; chunking by sentences or by document sections, with the document's title attached to each chunk, is the usual refinement, and Chapter 55 measures all three rather than choosing by taste

Likely follow-ups: How would you choose chunk size for a specific document type (a legal contract vs. a product FAQ)? What's reranking, and why is it often added after the initial retrieval step?

Red flag: describing RAG's stages with no understanding of why chunking or overlap specifically matters.

Learn it in: Chapter 55, section 55.3 (chunking: fixed, sentence, and section strategies).

Q79–013 · What's reranking, and why does an extra step after initial vector retrieval improve answer quality?

Level: Mid · **Roles:** AIE, DS, MLE

Remember it as: *The first retrieval pass is fast but approximate. Reranking is a slower, more careful second look at just the top candidates, worth the extra cost because there are far fewer of them to examine closely.*

Answer in one line: Initial retrieval (vector search, keyword search, or both) is fast but approximate, built to quickly narrow thousands or millions of chunks down to a top handful; a **reranker**, typically a more expensive model that reads the question and each candidate chunk together and scores

relevance directly (a **cross-encoder**), then re-scores just that smaller candidate set more carefully, often surfacing a better final ranking than the fast first pass alone.

Tier	What to say
Passes	"Reranking makes the results better" (true, no explanation of why a two-stage approach specifically)
Strong	The speed-vs-accuracy two-stage trade-off above: cheap-and-broad first pass, expensive-and-precise second pass on a much smaller set
Extra points	[+Validate] measure it before keeping it: the crude word-count reranker in Chapter 55's exercise 11 made recall@1 worse (0.87 to 0.82), because re-sorting by a weaker score undoes a good ranking [+Business] this two-stage pattern (fast filter, then expensive precise scoring on a small candidate set) recurs well beyond RAG, wherever a precise method doesn't scale to the full dataset but is affordable on a pre-filtered handful

Likely follow-ups: What would you check if reranking made results measurably worse on a specific query? How would you decide how many candidates to pass into the reranking stage?

Red flag: treating reranking as an optional polish step rather than understanding the two-stage cost/accuracy trade-off it exists to solve.

Learn it in: Chapter 55, section 55.5 (hybrid search, measuring it, and what a reranker adds).

Rapid-fire, 79.3

Roles: AIE, DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-014	Why does a RAG system need citations, not just an answer?	Citations let a user (or an automated evaluator) verify the answer actually traces back to real, retrieved source material, rather than trusting the model's fluency alone	[+Validate] a citation is also a test hook: an evaluation can check that the cited source is the one that holds the answer	Fresher · 55.6
Q79-015	What happens if the retrieval step finds no genuinely relevant chunks for a query?	A well-designed system should recognize this and respond with an honest "I don't have information on that" rather than generating a plausible-sounding answer from the model's untethered training knowledge	[+Edge cases] the refusal needs a threshold on the absolute retrieval score (a normalized top score is always 1.0), chosen on questions the documents can and can't answer	Mid · 55.6; Q79-007
Q79-016	Why might a RAG system need to periodically re-embed and re-index its documents?	If the underlying embedding model is updated, or the source documents change meaningfully, the existing vector index can become stale or inconsistent with newly embedded content	[+Scale] this is a real, recurring maintenance cost of a RAG system, not a one-time setup task; a superseded document left in the index is how an assistant quotes a dead policy	Mid · 55.10, 55.11

#	Question	One-line answer	Extra point	Level · learn it in
Q79-017	What's a vector database, and how is it different from a traditional relational database for this use case?	A store built for fast approximate-nearest-neighbour search over high-dimensional vectors, a query pattern ordinary relational indexes aren't built for at scale; though extensions such as pgvector add this to PostgreSQL, keeping vectors in a table beside your data, often the right first choice	[+Simple first] at Riverstone's scale, a NumPy array and a dot product are enough; an approximate-nearest-neighbour index earns its place at millions of chunks	Mid · 55.4, and Chapter 55's project tools (pgvector, dedicated vector databases); relational indexes: Chapter 71, section 71.9

79.4 Agents and tool use

Q79-018 · What's the difference between a simple prompt-and-response LLM call and an "agent"?

Level: Mid · **Roles:** AIE, DS, MLE

Remember it as: A single call answers once. An agent can decide to call a tool, look at the result, and decide again, in a loop, until it's actually done.

Answer in one line: A single LLM call takes a prompt and returns one response; an **agent** wraps an LLM in a loop that lets it decide to call external tools (a search, a calculator, a database query, an API), observe the result, and decide on a next action, potentially several times, before producing a final answer, giving it the ability to take multi-step actions rather than just generate text once.

Tier	What to say
Passes	"An agent is an LLM that can use tools" (correct, no mention of the decision loop that makes multi-step behavior possible)
Strong	The observe-decide-act loop above, correctly distinguishing single-shot generation from iterative, tool-using behavior
Extra points	[+Business] an agent's added capability comes with added failure modes: it can call the wrong tool, misinterpret a tool's result, or loop unproductively, all failure modes a single prompt-response call simply doesn't have, which is exactly why evaluating and guardrailing agents (section 79.5) is a genuinely harder problem than evaluating a single response [+Trade-offs] most business problems are a script with one or two tools; an agent earns its cost only when the number of steps can't be known in advance

Likely follow-ups: What's the risk of an agent getting stuck in a loop, and how would you guard against it? How would you decide whether a task actually needs an agent versus a single well-designed prompt?

Red flag: treating "agent" as just a marketing term with no specific technical distinction from a single LLM call.

Learn it in: Chapter 55, sections 55.7 (tools) and 55.8 (agents, without the hype).

Rapid-fire, 79.4

Roles: AIE and MLE for every row; DS for Q79-021.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-019	Why does an agent typically need a maximum number of steps or a timeout?	Without one, a genuinely stuck agent (repeatedly calling the same tool, or oscillating between two actions) can loop indefinitely, wasting cost and never returning a result	[+Edge cases] give it a step limit, a token budget, and a timeout; the step limit is the same idea as a retry cap (Q78-017), while a circuit breaker (Q78-021) is the related cross-run version that stops calling a failing service for a while	Fresher · 55.8 (step limits)
Q79-020	What's the risk of giving an agent access to a tool with real side effects (sending an email, modifying a database)?	An agent's decision to call that tool is itself a probabilistic model output, not a guaranteed-correct decision; a wrong or hallucinated tool call with real side effects can cause real, hard-to-undo damage	[+Business] make tools read-only by default, whitelist them per context, and put a human confirmation in front of any irreversible action	Mid · 55.7; Chapter 58, section 58.5
Q79-021	How would you debug an agent that's producing a wrong final answer despite calling the right tools?	Trace through the full sequence: what each tool actually returned, and how the agent interpreted that result at each step, since the failure could be in tool selection, tool execution, or the agent's interpretation of a correct tool result	[+Edge cases] look first for a tool that returned an error or an empty result the agent treated as an answer; it's the same "trace backward through the steps" method as data lineage in Chapter 77 (Q77-030)	Mid · Chapter 57, section 57.10 (traces)
Q79-022	What's the difference between a single agent and a multi-agent system?	A single agent handles the whole task itself, looping through tool calls; a multi-agent system splits the task across several specialised agents that pass work between them, trading simplicity for narrower, better-scoped roles	[+Trade-offs] more agents means more coordination, more model calls, and more places for a hand-off to go wrong	Senior · Beyond the book (below)

Beyond the book: multi-agent systems. Chapter 55 builds one agent: one loop, one model, a set of tools. A **multi-agent system** puts an **orchestrator** agent in charge, which splits a task and hands each part to a specialised agent with its own instructions and tools (one searches the documents, one looks up orders, one drafts the reply), then combines their results. Each specialist is easier to prompt and test on its own, but every hand-off is another model call and another place for a misunderstanding. A fair interview answer: start with one agent and a step limit, and split only when one agent's instructions or tool list have grown too large to test.

79.5 Evaluating AI applications

Q79-023 · How do you evaluate an LLM-based application's output quality, given there's often no single "correct" answer to check against?

Level: Mid · Roles: AIE, DS, MLE

Remember it as: A golden set gives you known-good answers to check against. An LLM-as-judge extends that same idea to open-ended answers a simple string match can't score.

Answer in one line: Build a **golden set** of representative queries with known-good expected answers (or key facts that must appear) to check against systematically; for genuinely open-ended output where an exact match doesn't make sense, use **LLM-as-judge** (a separate model call scoring the output against a written rubric) alongside sampled **human review**, since no single method alone reliably catches every failure mode.

Tier	What to say
Passes	Suggests a fixed test set of questions, but scores it only by exact match against one expected answer
Strong	The golden-set-plus-LLM-as-judge-plus-human-review combination above, correctly noting no single method is sufficient alone
Extra points	[+Edge cases] an LLM-as-judge has its own failure modes (it can be fooled by fluent-but-wrong answers, or have its own biases about what "good" looks like), so it needs calibration against human judgment: score about 50 answers by hand once, check the judge agrees, and re-check after any model change [+Scale] a golden set needs to be actively maintained and expanded as new failure modes are discovered in production, and run as a build step on every prompt or model change, not built once and left static

Likely follow-ups: How would you build an initial golden set with no existing production data to draw from? What's the risk of over-optimizing a system specifically to score well on its own golden set?

Red flag: relying entirely on spot-checking outputs by eye, with no systematic, repeatable evaluation method.

Learn it in: Chapter 55, section 55.9 (evaluating the whole application); Chapter 57, section 57.3 (the golden set as a build step).

Rapid-fire, 79.5

Roles: AIE, DS and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-024	What's a guardrail, in the context of an LLM application?	A check (before or after the model's generation) that catches and blocks unsafe, off-topic, or otherwise unacceptable input or output before it does harm	[+Business] guardrails are a defense layer separate from prompting; relying on prompting alone to prevent bad output is fragile, the same lesson as Q79-010's prompt-injection limits	Fresher · 55.10

#	Question	One-line answer	Extra point	Level · learn it in
Q79-025	Why might an evaluation metric that looks good in aggregate still hide a real problem?	The same Simpson's-paradox-style risk (Chapter 73, Q73-031): an overall-good average score can mask a specific query type or user segment the system handles consistently poorly	[+Edge cases] break the golden-set score out by question type before trusting the average	Mid · 55.9; Q73-031
Q79-026	What's a regression test for an LLM-based system, specifically?	A saved set of prompts and their previously-acceptable outputs, re-run whenever the underlying model, prompt, or retrieval logic changes, to catch a quality regression before it reaches production	[+Validate] it only works if prompts are versioned like code, so a failing run points at exactly what changed	Mid · 57.2–57.4
Q79-027	Why is human review still necessary even with a good automated evaluation pipeline in place?	Automated methods (golden sets, LLM-as-judge) can miss failure modes nobody anticipated when designing them; sampled human review is what catches the genuinely novel problem the automated system wasn't built to look for	[+Close] close the loop: every problem a reviewer or a customer finds becomes a new golden-set case	Mid · 55.9; Chapter 57, section 57.11

79.6 MLOps: making models survive production

Q79-028 · How do you detect whether a production model's input data has drifted from its training data, and what does drift tell you?

Level: Mid · **Roles:** MLE, DS

Remember it as: "It looks a little different" isn't drift detection. A distance between two distributions (PSI, or the KS statistic *D*), with a threshold calibrated on a stable period, is. And drift says the world changed, not that the model got worse.

Answer in one line: Compare the training-time distribution of each important feature with its recent production distribution using a distance: the **population stability index (PSI)** on bucketed values (Chapter 56's main metric, with rules of thumb of 0.1 for "watch" and 0.25 for "investigate", checked against a stable period) or the **Kolmogorov-Smirnov statistic *D*** for a continuous feature; alert on the size of the distance, not on a p-value, and treat drift as an early warning, with outcomes as the verdict.

Run it yourself. First, a training sample and two production samples of 1,000 values each, one with no real change and one whose mean has moved from 50 to 58:

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(79)
training = rng.normal(50, 10, 1000)
prod_same = rng.normal(50, 10, 1000)      # no real shift
prod_drifted = rng.normal(58, 10, 1000)   # the mean has moved from 50 to 58

for name, recent in [("no shift", prod_same), ("mean 50 -> 58", prod_drifted)]:
    result = stats.ks_2samp(training, recent)
    print(f"{name:<14} D = {result.statistic:.3f}, p = {result.pvalue:.1e}")
```

```
no shift      D = 0.029, p = 7.9e-01
mean 50 -> 58 D = 0.317, p = 8.3e-45
```

How it works:

- `np.random.default_rng(79)` is a random generator with a **seed**, 79: the same seed always draws the same numbers, so your output matches this one (Chapter 21).
- `rng.normal(50, 10, 1000)` draws 1,000 values from a normal distribution with mean 50 and standard deviation 10 (Chapter 21).
- `stats.ks_2samp(a, b)` compares two samples. `.statistic` is **D**, the largest vertical gap between the two samples' cumulative distributions: 0 means identical, 1 means no overlap (Chapter 56, section 56.8 works it out by hand). `.pvalue` is the test's p-value (Chapter 22).
- `:.1e` prints a number in scientific notation with one decimal (Chapter 35): 7.9e-01 is 0.79, and 8.3e-45 is 0.000...083 with 44 zeros after the point.

With 1,000 values each, D separates the cases clearly: 0.029 against 0.317. Now the trap. Two samples of 200,000 values whose means differ by 0.3, three hundredths of a standard deviation, a difference no model would notice. Before you run the cell, predict: which of D and p will call this a big difference?

```
big_training = rng.normal(50, 10, 200_000)
big_recent = rng.normal(50.3, 10, 200_000)  # a shift of 0.03 standard deviations
result = stats.ks_2samp(big_training, big_recent)
print(f"D = {result.statistic:.3f}, p = {result.pvalue:.1e}")
```

```
D = 0.016, p = 3.2e-23
```

`big_training` and `big_recent` come from the same `rng.normal` as before, now 200,000 values each (200_000; the underscore only makes it readable), and `stats.ks_2samp` and the `print` line work exactly as in the first cell: `result.statistic` is D and `result.pvalue` is p. The p-value calls this tiny shift overwhelmingly "significant", because with enough rows almost any difference is. D stays small, 0.016, below even the no-shift sample's 0.029 above. **With thousands of rows, alert on the size of D or PSI, never on the p-value**, which is what Chapter 56, section 56.8 does.

Tier	What to say
Passes	Watches only the model's accuracy, which needs labels and arrives late
Strong	Monitors input drift (PSI or the KS statistic) as the early warning, and outcomes (or a cheap proxy, with a sampled audit for the truth) as the verdict: drift says the world changed, not that the model got worse
Extra points	[+Evidence] drift and decay are different things: in Chapter 56, Riverstone's new lamps pushed brightness PSI up to 4.4 while recall held (96.8% before, 96.5% during), and the new mould then cut recall to 84.4%, about 12 points, with no new drift signal; that's why you need both layers [+Validate] calibrate the threshold on held-out stable weeks (Riverstone's noise floor for brightness was PSI 0.006–0.013) rather than trusting 0.1 blindly [+Limits] drift is a <i>leading</i> signal of change (Chapter 75, Q75-014), not of harm: it can't see a model quietly missing one kind of defect, so outcome monitoring, the lagging signal, still decides whether it mattered

Likely follow-ups: What would you do once drift is detected: retrain immediately, or investigate first? How would you monitor drift for a categorical feature instead of a continuous one?

Red flag: treating a drift alarm (or a tiny p-value) as proof the model is broken, or monitoring only one of the two layers.

Learn it in: Chapter 56, sections 56.7 (monitoring, in three layers) and 56.8 (drift, measured).

Rapid-fire, 79.6

Roles: MLE and DS for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-029	What's a model registry, and why does a team need one?	A central index of every trained model version, the run that produced it, its metrics, and an alias such as production pointing at the version in use, so "which exact model is live right now" always has one clear answer	[+Business] without one, a team can lose track of which model version produced a specific prediction; with one, a rollback is moving the alias back	Fresher · 56.2, 56.3
Q79-030	What's a feature store, and what problem does it solve?	A shared service that computes each feature once and serves the same values to training and to production, so a feature can't be computed one way in training and another in the service (training-serving skew : the same feature computed differently in training and in production)	[+Trade-offs] excellent when many models share features; for one or two models, a scikit-learn pipeline packaged with the model applies the same fitted steps at serving time and does the same job	Mid · Chapter 36, section 36.9 (one pipeline for training and serving); 56.4, 56.11; Q74-030
Q79-031	How should you decide when to retrain a model?	A schedule is the simplest default; add a performance-floor trigger (late, because it needs labels), treat drift as a prompt to investigate rather than an automatic retrain, and above all retrain on known business events (a new product, supplier, or machine)	[+Business] the best trigger comes from the business, not a monitor: Riverstone's rule is "retrain monthly, and immediately on any event the plant tells us about"	Mid · 56.9

#	Question	One-line answer	Extra point	Level · learn it in
Q79-032	What's the difference between online and batch model serving?	Online serving responds to individual requests in real time (a single prediction per API call); batch serving scores a large dataset all at once on a schedule, with no per-request latency requirement	[+Business] the choice depends on whether a downstream decision genuinely needs a fresh, on-demand prediction or can work from a periodically refreshed batch of scores, the same freshness-driven logic as Q77-002's batch-vs-streaming decision	Fresher · 56.5

79.7 LLMOps

Q79-033 · Estimate the daily cost of an LLM-powered feature, and explain what levers you'd pull if it's too expensive

Level: Mid · **Roles:** AIE, MLE

Remember it as: Cost scales with tokens in, tokens out, and calls per day, so any lever that reduces one of those three reduces cost proportionally.

Answer in one line: Cost per call is input tokens times the input price plus output tokens times the output price (output is usually several times dearer), multiplied by the number of calls per day; to reduce cost, shrink the prompt (better retrieval, less unnecessary context), shrink the expected output length, cache repeated queries, or route simpler queries to a cheaper, smaller model.

Run it yourself. Prices are quoted in US dollars per million tokens, as in Chapters 54 and 57. This uses Chapter 54's workhorse tier (\$2 in, \$10 out) and the volume-tier prices Chapter 57 uses (\$0.75 in, \$3.75 out), both from price tiers checked on 29 September 2026, for a call with 500 tokens in and 150 out, 2,000 times a day:

```
USD_TO_INR = 88.0    # rupees per US dollar, as in Chapter 57; check today's rate
```

```
def estimate_cost(tokens_in, tokens_out, price_in_per_m, price_out_per_m):
    cost_in = tokens_in / 1_000_000 * price_in_per_m
    cost_out = tokens_out / 1_000_000 * price_out_per_m
    return cost_in + cost_out
```

```
tiers = [("workhorse", 2.00, 10.00), ("volume", 0.75, 3.75)]
for tier, price_in, price_out in tiers:
    per_call = estimate_cost(500, 150, price_in, price_out)
    per_day = per_call * 2000
    rupees = per_day * USD_TO_INR
    print(f"{tier:<9} per call ${per_call:.6f};",
          f"per day ${per_day:.2f} (₹{rupees:,.0f})")
```

```
workhorse per call $0.002500; per day $5.00 (₹440)
volume    per call $0.000937; per day $1.88 (₹165)
```


How it works:

- `USD_TO_INR` holds the exchange rate in one named place, as Chapter 57 does; it is the only rupee assumption.
- `estimate_cost` divides each token count by a million (`1_000_000`), multiplies by the price per million, and adds the input and output costs. By hand for the workhorse tier: $500 \div 1,000,000 \times \$2 = \$0.001$ in, plus $150 \div 1,000,000 \times \$10 = \$0.0015$ out, is $\$0.0025$ a call.
- `tiers` lists each tier's name and prices, and the loop unpacks each (tier, input price, output price) triple; `rupees` converts the day's dollars. `print` given two strings prints them with a space between. `:.6f` prints six decimals, because a single call costs a fraction of a cent; `:,.0f` prints whole rupees.

Output is under a quarter of the tokens and 60% of the workhorse cost. Routing to the volume tier cuts the bill by more than 60%, if its answers pass the same evaluation (Q79-036).

Tier	What to say
Passes	Knows cost is per token, but multiplies one blended price by an average length and ignores the input/output price split
Strong	The token-based cost formula above, correctly separating input and output token pricing, and naming at least two concrete cost-reduction levers
Extra points	[+Evidence] Riverstone's own order-email pipeline, metered in Chapter 57, costs about ₹0.08 (₹0.079) per email, under ₹90 a month, and Chapter 55's support assistant about 13–14 paise a question: at this scale the staff time around the model costs far more than the model [+Business] caching deserves special mention: a cache hit costs nothing, and for a feature with real repetition it can be the highest-leverage lever, but it saves only the share of traffic that repeats (17% on a normal day of Riverstone's order emails), so measure the hit rate on your own logs

Likely follow-ups: How would you decide which queries are safe to route to a cheaper model versus which need the more capable one? What's the trade-off of caching for a use case where answers need to reflect very current information?

Red flag: no structured cost model at all, or ignoring caching as a lever entirely.

Learn it in: Chapter 54, section 54.12 (price tiers); Chapter 57, sections 57.5 (tokens, cost, and the meter) and 57.6 (caching).

Rapid-fire, 79.7

Roles: AIE and MLE for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q79-034	Why does an LLM application need prompt versioning, the same way code needs version control?	A prompt change can meaningfully change output quality or behavior; without versioning, there's no way to know exactly which prompt produced a given historical output, or to safely roll back a change that made things worse	[+Validate] pair every prompt version with its golden-set score, as Q79-026's regression test does	Mid · 57.2–57.4

#	Question	One-line answer	Extra point	Level · learn it in
Q79-035	What's tracing, in an LLM application context?	Recording the full sequence of steps (retrieval, tool calls, model calls, intermediate outputs) behind a single final response, so a specific bad output can be debugged step by step instead of treated as an unexplainable black box	[+Edge cases] log the IDs of the retrieved chunks with each answer, or you can't tell a retrieval failure from a generation failure (the same trace-the-steps discipline as Q79-021)	Mid · 57.10; Chapter 55 (retrieved chunks carry their document name as the citation)
Q79-036	What's the risk of routing a query to a cheaper, smaller model purely based on cost, with no quality check?	The cheaper model may produce a meaningfully worse answer for that specific query's complexity, silently degrading the user's experience in exchange for a cost saving that may not be worth it	[+Business] any routing strategy, including a fallback to a cheaper model when the main one fails, needs its own evaluation (section 79.5) proving the quality trade-off is acceptable	Mid · 57.8 (fallback model)
Q79-037	Why is prompt injection a genuinely different security problem from a traditional SQL injection?	Traditional injection exploits a rigid parser with a fixable grammar; prompt injection exploits a model's flexible, natural-language interpretation, which has no equally rigid, fully closeable boundary between "instructions" and "data" the way a parameterized SQL query does	[+Limits] so the defense is containment, not a fix: validate output, keep tools read-only, and monitor injection attempts	Senior · 54.11; 55.10; 57.10; Q79-010

79.8 Full design cases

Q79-038 · Design a RAG-based product-support assistant for Riverstone, end to end

Level: Senior · **Roles:** AIE, MLE, DS

What they're really testing: whether a complete RAG system design comes out under time pressure, covering ingestion, retrieval, generation, and evaluation, not just the retrieval step alone.

Talked through live, start to finish:

"Source content: product manuals, past support tickets, and the returns policy, ingested and chunked (Q79-012), each chunk embedded and stored in a vector index. At query time: a customer's question is embedded, the index returns the top-k most similar chunks, and a reranker (Q79-013) narrows those to the genuinely most relevant few before they're inserted into the prompt as grounding context. The model generates an answer citing which source chunks it used, and if retrieval finds nothing sufficiently relevant, the system explicitly says it doesn't have a confident answer rather than guessing (Q79-015), routing to a human agent instead. For evaluation, I'd build a golden set from real historical support tickets with known-good resolutions, checked with LLM-as-judge plus sampled human review (Q79-023), and I'd add a guardrail blocking any generated answer that promises something outside Riverstone's actual policies, like an unauthorized discount. For cost, I'd cache answers to genuinely repeated common questions (Q79-033), keyed on the normalised question plus the policy-document version, and never for questions that depend on the customer's own order; the cache is cleared when a source document changes."

Extra-points moves demonstrated: **[+Signpost]** covered ingestion through evaluation and cost, in order, not just the retrieval mechanics. **[+Edge cases]** explicitly handled the no-good-retrieval case with a human handoff rather than letting the model guess, and kept live order data out of the cache. **[+Business]** named a guardrail tied to a real business risk (an unauthorized discount promise), not a generic safety placeholder.

Likely follow-ups: How would you handle a question that spans multiple product manuals? What would make you decide this needs an agent (able to look something up dynamically) rather than pure RAG?

Red flag: a design that only covers retrieval and generation, with no mention of evaluation, guardrails, or the no-good-answer case.

Learn it in: sections 79.3 and 79.5 above, pulled together; Chapter 55 builds this assistant, and Chapter 57, section 57.6 covers caching and its limits.

Q79-039 · Design a model-serving platform that supports both real-time and batch predictions for multiple models

Level: Senior · **Roles:** MLE, DS

What they're really testing: whether the online-vs-batch distinction (Q79-032) gets applied to a genuine shared-infrastructure design, not treated as two unrelated systems.

Talked through live, start to finish:

"A model registry (Q79-029) is the shared source of truth for every model version and its deployment status, used by both serving paths. Each model is packaged with its preprocessing pipeline and its metadata, such as its threshold, so the feature logic used in training is the logic used at serving time, avoiding training-serving skew; a feature store (Q79-030) takes over that job if several models share features, but for one or two models the packaged pipeline does the same job. For real-time serving: models loaded behind a low-latency API. For batch serving: a scheduled job scores a full dataset on a cadence appropriate to that model's use case (Chapter 77's batch pipeline pattern applies directly here). Both paths log predictions with the exact model version used, feeding the monitoring (Q79-028): input drift as the early warning, and outcomes, a proxy plus a sampled audit, as the verdict. A retraining trigger (Q79-031) fires on a baseline schedule and on known business events; a drift alarm opens an investigation, and a confirmed drop in performance triggers the retrain."

Extra-points moves demonstrated: **[+Signpost]** laid out the shared pieces first (registry, packaged feature logic, monitoring), then each serving path, rather than designing two disconnected systems. **[+Trade-offs]** chose a packaged pipeline over a feature store until several models share features. **[+Business]** connected monitoring directly to concrete retraining triggers, including business events, closing the loop rather than leaving monitoring as a dashboard nobody acts on.

Likely follow-ups: How would you handle two models needing genuinely different feature computation logic within the same shared feature store? What would you monitor differently for the batch path versus the real-time path?

Red flag: designing the two serving paths as entirely separate systems with no shared registry, feature logic, or monitoring.

Learn it in: Chapter 56, sections 56.4 (packaging), 56.5 (serving), 56.7–56.9 (monitoring, drift, retraining) and 56.11 (how far up the ladder to climb); Chapter 77 (the batch pipeline pattern this design's batch path reuses).

APPLY

Common mistakes

Mistake	Symptom	Fix
Assuming a bigger context window means "always add more context"	Higher cost and latency, and sometimes worse answers despite more information present	Curate relevant context via good retrieval, don't maximize volume
No grounding, relying purely on the model's trained-in knowledge	Confident, fluent, sometimes wrong answers (hallucination)	Use RAG to ground answers in real, retrieved, current information
Chunking with no overlap	Meaning split across a chunk boundary gets lost	Use overlapping chunks, or chunk by sentences or sections

Mistake	Symptom	Fix
Monitoring only one layer: accuracy alone, or drift alone	Accuracy alone finds problems weeks late; drift alone alarms on harmless change and stays silent on real decay	Watch inputs as the early warning and outcomes (a proxy plus a sampled audit) as the verdict (Q79-028)
Alerting on a drift test's p-value	With thousands of rows, every tiny shift is "significant" and the alert fires daily	Alert on the size of PSI or D, with a threshold calibrated on a stable period
No cost model for an LLM feature before shipping it	An unpleasant cost surprise once usage scales up	Estimate cost per call from prices per million tokens, and multiply by expected volume before launch
An agent given side-effect tools with no human-in-the-loop check	A wrong or hallucinated tool call causes real, hard-to-undo damage	Read-only tools by default; require confirmation for high-stakes, irreversible actions
Evaluating only on an aggregate score	A real, segment-specific failure mode hides behind a good average	Check performance broken out by query type or user segment

In the real world: the RAG assistant that confidently cited a document that didn't say what it claimed

Priyanka, a Data Scientist, is asked in an interview to describe debugging a real RAG system issue. She describes a customer-support assistant that had started giving customers an incorrect return window, and the response even included a citation to a real, correctly retrieved policy document, which made the error harder to spot at first: the citation looked legitimate, so nobody initially suspected the retrieved content itself.

She traces it to the chunking boundary: the policy document's actual text read "returns are accepted within 30 days, except for electronics, which have a 14-day window," and a fixed-size chunk had split exactly between those two clauses, so the retrieved chunk for an electronics question contained only "returns are accepted within 30 days," the exception clause landing entirely in the next chunk, which wasn't retrieved for that specific query. The citation was technically accurate, that chunk really did come from the real policy document, but the chunk itself no longer contained the complete, correct information.

The interviewer's follow-up: how did she fix it, and how did she make sure it wouldn't happen again the same way? Her answer: she moved to chunking on sentence and paragraph boundaries instead of a fixed character count, so a chunk wouldn't split a sentence's meaning apart from its own exception clause, and she added a golden-set test case specifically covering the electronics exception, so a future chunking or retrieval change that reintroduced this exact failure would be caught automatically before shipping, not discovered again by a customer complaint.

The interviewer's note: *"A citation being real doesn't mean the retrieved content is complete. She caught a failure mode a simpler 'is this grounded' check would have completely missed."* That's the deeper lesson this whole chapter is built around: grounding and citations reduce hallucination risk, but they don't eliminate every way a RAG system can still confidently produce a wrong answer.

Project

Goal: apply this chapter's verifiable pieces to a real or realistic scenario.

Tools you'll need

Python with **NumPy** (Chapter 18) and **SciPy** (Chapter 21, section 21.5), used for every computable claim in this chapter (cosine similarity, chunking, the KS drift statistic, the cost estimate); the outputs shown came from Python 3.11, NumPy 2.4.6 and SciPy 1.17.1. For step 3, Chapter 56's `psi` function (section 56.8). In real GenAI and MLOps work: a vector store (pgvector or a dedicated vector database), an orchestration framework for agents, MLflow or a similar tool for the model registry and tracking, and an LLM observability or tracing platform. Vendor and model specifics change quickly; verify current details before relying on them in an actual interview.

1. Run the chunking function from Q79-012 on a real document of your own, and inspect every chunk boundary to check whether it splits a fact from its condition, the way this chapter's real-world story describes.
2. Compute cosine similarity between a real query and a few candidate documents (using Chapter 54's embeddings, or any embedding source available to you) and confirm the ranking matches your own judgment of relevance.
3. Run Q79-028's drift check on two real or simulated datasets of your own, one with a genuine shift and one without. Report D and PSI for each, then repeat with ten times the rows and watch what happens to the p-value while D and PSI stay put.
4. Estimate the daily cost of a hypothetical LLM feature for your own context with Q79-033's formula and prices per million tokens, and name the one cost lever you'd pull first if it came in too expensive.

REVIEW

Key terms

token · context window · temperature · fine-tuning vs. prompting · embedding · cosine similarity · unit-length vector · hallucination · RAG (Retrieval-Augmented Generation) · chunking (fixed-size, sentence, section) · overlap · vector database · reranking · cross-encoder · citation/grounding · refusal · agent · tool use · step limit · multi-agent system · orchestrator · golden set · LLM-as-judge · guardrail · regression test (LLM) · model registry · feature store · training-serving skew · drift detection · population stability index (PSI) · KS statistic (D) · retraining trigger · online vs. batch serving · prompt versioning · tracing (LLM) · prompt injection · caching (LLM cost)

Final-week revision list

Q79-001, Q79-002, Q79-007, Q79-012, Q79-013, Q79-018, Q79-023, Q79-028, Q79-033, Q79-038, Q79-039.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 74, Machine Learning Question Bank**, already covers training-serving skew and drift-adjacent concepts (Q74-030) from the classical-ML angle; this chapter extends the same discipline to LLM-based systems.
- **Chapter 77, Data Engineering & Data System Design Bank**, supplies the batch pipeline pattern Q79-039's batch-serving path reuses; **Chapter 78** (Q78-021) and **Chapter 57, section 57.8** describe the circuit breaker, and Chapter 58, section 58.6 builds one for a whole batch.
- **Chapters 54-57 of this book** teach every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.
- **Chapter 80, Architecture & Leadership Question Bank**, comes next: the same interview discipline, applied to whole-platform design and to leading the people who build it.

Chapter 80. Architecture & Leadership Question Bank

Chapter at a glance

You will learn to: reason about distributed-systems trade-offs the way a senior interview actually probes them · make and defend a build-vs-buy or architecture decision with real trade-offs stated, not a confident guess · answer governance, security, and cost questions at the level an architect owns them · handle leadership scenarios (influencing without authority, saying no, presenting to a board) live · walk through a full architecture design case end to end.

Before you start: Part 7 (Chapters 60–67), which teaches almost every idea in this bank; Chapter 20, section 20.14 and Chapter 23, section 23.9 for the time-saving and payback arithmetic in Q80-016; Chapter 24, sections 24.7 and 24.8 for presenting to executives and handling pushback; and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Time needed: 2–3 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 15 minutes to run Q80-016's cells yourself), plus 2–3 hours for the project: an ADR, an ROI, a pushback, and a roadmap.

A scope note. This chapter is for senior IC and architect-level interviews specifically (an **IC**, individual contributor, is a senior engineer or scientist who leads work without managing people), distinct from the Data Analyst/Data Scientist/Data Engineer focus of most of this part. Readers targeting DA/DS/DE roles can treat this chapter as optional, forward-looking material for a later career stage.

How this chapter is built. Same format as every question bank in Part 8. Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, tagged the same way: **[+Clarify]**, **[+Edge cases]**, **[+Validate]** and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it (a bare number such as 61.4 means that section). Most of this chapter is judgment and trade-off reasoning rather than executable code; where a claim is computable (the ROI in Q80-016), it's actually run and shown with real numbers.

Levels and roles. Each question carries a level and the roles that usually ask it. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. There are no Fresher questions: this bank is for a later stage of a career. **DE** senior data engineer · **DS** senior data scientist or ML engineer · **ARC** data or solutions architect · **LEAD** head of data, or an engineering or analytics manager. A question asked of DE or DS comes up in senior IC rounds; one asked only of ARC or LEAD belongs to architect and lead rounds.

In this chapter

LEARN	80.1 Distributed systems trade-offs	340
	80.2 Basic-but-tricky architecture questions	341
	80.3 Data architecture decisions	343
	80.4 Security, governance, and cost	344
	80.5 Leadership scenarios	348
	80.6 Full architecture design cases	349
APPLY	Common mistakes · In the real world · Project	351
REVIEW	Key terms · Final-week revision list	353
NEXT	Where this leads	353

LEARN

80.1 Distributed systems trade-offs

Q80-001 · Explain the CAP theorem, and give a real example of a system choosing each side of the trade-off

Level: Mid · **Roles:** DE, ARC

Remember it as: You can't have all three of Consistency, Availability, and Partition tolerance at once during an actual network partition. Partition tolerance isn't optional in a real distributed system, so the real choice is between Consistency and Availability.

Answer in one line: The CAP theorem states that during a network partition, a distributed system must choose between **Consistency** (every read sees the latest write, or an error) and **Availability** (every request gets a response, even if it might be stale); since partitions are a fact of real networked systems, not an edge case to design away, the practical choice most systems actually face is CP vs. AP.

Tier	What to say
Passes	Recites the three letters without explaining that partition tolerance is effectively mandatory, not a genuine third option
Strong	Correctly frames the real trade-off as CP vs. AP, with one concrete example of each: a banking system favoring consistency (reject a transaction rather than risk showing a stale balance) versus a social media feed favoring availability (show slightly stale content rather than an error)
Extra points	[+Business] the right choice is a business decision tied to what's actually worse for that specific system: a stale bank balance is a real, serious problem; a slightly stale "like count" is not, and the architecture should reflect that difference deliberately, not by accident [+Edge cases] even banks choose AP in places: an ATM that can't reach the bank may still allow a capped withdrawal and reconcile it later. The business sets a <i>limit</i> on the risk of acting on stale data rather than demanding CP everywhere

Likely follow-ups: What's eventual consistency, and where does it fit relative to strict CP or AP? How would you explain this trade-off to a non-technical executive asking "why can't we just have both"?

Red flag: claiming a real system can be fully consistent, available, and partition-tolerant simultaneously, with no trade-off at all.

Learn it in: Chapter 61, section 61.2 (the CAP theorem, and Riverstone's CP order writes and AP dashboard reads).

Rapid-fire, 80.1

Roles: DE and ARC for every row; LEAD for Q80-003.

#	Question	One-line answer	Extra point	Level · learn it in
Q80-002	What's eventual consistency?	A guarantee that, given no new writes, all replicas of a piece of data will <i>eventually</i> converge to the same value, without promising exactly when	[+Business] acceptable for use cases where a brief window of staleness is genuinely harmless (a view count), not for ones where it isn't (an account balance)	Mid · 61.4 (the consistency spectrum)
Q80-003	What's a single point of failure, architecturally?	Any component whose failure alone brings down the whole system, with no redundancy or fallback	[+Evidence] name one you found: Chapter 61's container-by-container failure analysis found Riverstone's warehouse to be the platform's one true single point of failure	Mid · 61.7; drilled at pipeline level in Q77-021
Q80-004	What's the difference between horizontal and vertical scaling?	Vertical (scaling up): making one machine bigger (more CPU/RAM); horizontal (scaling out): adding more machines and distributing load across them	[+Trade-offs] vertical scaling is simpler but has a hard ceiling; horizontal scaling has no such ceiling but adds real distributed-systems complexity	Mid · 61.5 ("Scaling up and scaling out")
Q80-005	What's a leader-follower (primary-replica) architecture, and what's its main trade-off?	One node accepts writes and propagates them to read-only replicas; simplifies write consistency but makes the leader a potential bottleneck and single point of failure unless failover is handled carefully, and reads from followers can be stale (replication lag: the eventual consistency of section 61.4)	[+Edge cases] automatic failover needs the surviving copies to agree on one new leader (consensus, leader election); two leaders both accepting writes is the failure to design against	Senior · 61.4; 61.5 (replication; "Who is in charge? Quorum and consensus"); 61.7

80.2 Basic-but-tricky architecture questions

Q80-024 · The warehouse is backed up every night. Is it still a single point of failure?

Level: Mid · **Roles:** DE, ARC

Remember it as: A backup protects the data. It doesn't keep the system running. Durability and availability are different promises.

Answer in one line: Yes: a backup means the data survives a failure (**durability**), not that the service keeps answering (**availability**); while the one warehouse is down, and until a restore finishes, everything that reads from it stops, so it's still a single point of failure, and only a second copy that can take over removes that.

Tier	What to say
Passes	"No, we have backups", or a bare "yes" with no reason
Strong	Separates durability (backups, versioned copies: the data survives) from availability (a second copy that can take over: the service survives), and says what stops while a restore runs
Extra points	[+Evidence] Chapter 61's failure analysis found exactly this at Riverstone: the warehouse is the platform's one true single point of failure, and in Chapter 52 a network fault, not lost data, was enough to stop the dashboard and the Flash [+Trade-offs] a standby copy with automatic failover costs money and needs the copies to agree which one is in charge (consensus); for a small team, a restore that has actually been tested and timed may be the right first step. Say which you'd pick, and why [+Validate] a backup nobody has ever restored is a hope, not a plan: restore it on a schedule and time how long it takes

Likely follow-ups: How long would a restore take, and who decides whether that's acceptable? What would a second copy cost, and what new failure would it add?

Red flag: treating "we have backups" as the answer to an availability question.

Learn it in: Chapter 61, sections 61.5 (replication, for availability and durability; quorum and consensus) and 61.7 (the warehouse as the platform's one true single point of failure).

Rapid-fire, 80.2

Roles: DE and ARC for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q80-025	Is eventual consistency a bug?	No: it's a deliberate trade (faster, more available reads, in exchange for a window where copies disagree), but only if something actually makes the copies converge	[+Edge cases] "it'll converge eventually" is only true if a sync, a retry or a reconciliation check makes it happen; Riverstone's CRM syncs are reconciled after every run (Chapter 51, section 51.7)	Mid · 61.4 (and its "Watch out")
Q80-026	What does PACELC add to CAP? Give an example with no partition.	Even with a healthy network, a system trades latency against consistency: waiting for the very latest write costs time on every request	[+Evidence] Riverstone's semantic layer is recomputed on a schedule, not on every write: it chooses low latency, and its numbers can lag the latest order	Senior · 61.3
Q80-027	Is partitioning the same as sharding?	No: partitioning splits one table into pieces by a key, still stored in one place and read by one engine; sharding puts those pieces on different servers, each storing and answering for only its share	[+Evidence] Riverstone's sensor archive is partitioned by month, not sharded; it would become sharding if Bhiwandi Main's and Chakan Pune's readings were served by two separate databases	Mid · 61.5
Q80-029	Does a second copy of a service always make it more available?	Not always: a copy that keeps no data of its own is easy to add, but some pieces must stay single, and copies of data bring back the consistency question	[+Edge cases] Riverstone's scheduler must stay one: two copies would each run the 6:30 schedule and send two Flashes	Senior · 61.5; 61.7

80.3 Data architecture decisions

Q80-006 · A team wants to build a custom data quality framework in-house instead of buying an existing tool. How do you evaluate that decision?

Level: Senior · **Roles:** DE, ARC, LEAD

Remember it as: *Build gives you exactly what you need and full control, forever, including forever maintaining it. Buy gives you speed now and someone else's roadmap later.*

Answer in one line: Weigh the **total cost of ownership** (TCO: what it costs to get the thing, plus what it costs to run and maintain it for as long as you keep it) on both sides, not just initial cost: building in-house means ongoing engineering time to build *and* maintain it indefinitely, but total control and no vendor lock-in; buying means faster time-to-value and someone else's ongoing investment, at the cost of licensing fees, less customization, and dependency on the vendor's own roadmap and continued existence.

Tier	What to say
Passes	A confident "build" or "buy" recommendation with no stated reasoning for why
Strong	The total-cost-of-ownership framing above, with both options scored on the same criteria: time to a working solution, fit, maintenance over the years you'll keep it, cost at current scale, and lock-in
Extra points	[+Business] the deciding factor is often whether the capability is genuinely core to the business's competitive differentiation (worth owning) or a solved, common problem plenty of vendors already do well (usually not worth reinventing); naming that distinction explicitly is a stronger answer than a generic cost comparison alone [+Evidence] Riverstone made this call twice and landed on different sides: ADR-018 kept data quality as SQL tests inside the pipeline rather than buying an observability platform (yet), while Chapter 66's scorecard for a data catalog had buying win on four of five dimensions

Likely follow-ups: How would you structure a pilot to test a vendor tool before fully committing? What's vendor lock-in, and how would you architect around minimizing it even when buying?

Red flag: comparing only sticker price without accounting for the ongoing maintenance burden of a build decision.

Learn it in: Chapter 66, section 66.5 (build versus buy: a managed data catalog, scored honestly); Chapter 60, section 60.5 (ADR-018).

Rapid-fire, 80.3

Roles: DE and ARC for every row; LEAD for Q80-009 and Q80-010.

#	Question	One-line answer	Extra point	Level · learn it in
Q80-007	What's a lambda architecture, and what problem does it solve?	Runs both a batch layer (accurate, complete, slower) and a speed/streaming layer (fast, approximate) in parallel, merging their results in a serving layer, so a system gets both eventual accuracy and low-latency freshness	[+Trade-offs] the cost is real complexity: maintaining two separate processing paths that must produce reconcilable results	Senior · 62.2 (Lambda and Kappa on one scenario)

#	Question	One-line answer	Extra point	Level · learn it in
Q80-008	What's a kappa architecture, and how does it simplify on lambda?	Treats everything as a stream, including reprocessing historical data by replaying the stream from the beginning, avoiding lambda's need to maintain two separate codebases for batch and speed layers	[+Trade-offs] simpler to maintain, but requires a streaming platform capable of efficient full replay; not every system's a good fit	Senior · 62.2
Q80-028	Is Kappa always simpler than Lambda?	Simpler code (one processing path instead of two), not simpler operations: Kappa needs a log that keeps months of history and can replay it quickly and reliably	[+Trade-offs] Riverstone runs the Lambda shape on purpose: its daily correction job is one more asset on the Dagster it already runs, while Kappa would need an always-on, replayable log for a four-person team	Senior · 62.2
Q80-009	What's an Architecture Decision Record (ADR), and why write one?	A short, versioned document capturing a specific architecture decision, the context that led to it, the options considered, and the reasoning, so a future engineer (including a future version of the same decision-maker) understands <i>why</i> , not just <i>what</i>	[+Business] without one, a "why did we do it this way" question six months later has no answer beyond someone's memory, which fades or leaves the company	Mid · 60.4; 67.3 ("Architecture decision records as institutional memory")
Q80-010	What's a C4 diagram, and what problem does it solve that a single architecture diagram doesn't?	A layered set of diagrams (Context, Containers, Components, Code) at increasing levels of detail, letting different audiences see the appropriate level without one diagram trying to serve everyone at once	[+Business] an executive needs the Context level; an engineer joining the team needs Components or Code; one diagram trying to do both usually serves neither well	Mid · 60.2

80.4 Security, governance, and cost

Q80-011 · How would you approach a "Responsible AI" review for a new model before it ships?

Level: Senior · **Roles:** DS, ARC, LEAD

Remember it as: A responsible AI review isn't a single "is this ethical" checkbox. It's a structured check across specific, nameable risks: *bias, privacy, explainability, and misuse*.

Answer in one line: Structure the review around specific, checkable dimensions: **fairness** (does it perform consistently across relevant segments, Chapter 39, section 39.9's fairness checks), **privacy** (does training or inference expose personal data inappropriately), **explainability** (can a decision be explained to someone affected by it), and **misuse potential** (could this capability be used for harm beyond its intended purpose), rather than one vague "is this responsible" judgment call.

Tier	What to say
Passes	"You'd check if the model is fair and doesn't have bias" (true, incomplete, treats it as a single dimension)
Strong	The four-dimension structure above, with a concrete check named for at least two of them
Extra points	[+Evidence] Chapter 39, section 39.9's segment-level fairness checks are exactly the concrete mechanism behind the fairness dimension here, and Chapter 64's audit shows why the mechanism matters: the lead score put East 14 points below West without ever using region, because company size acted as a proxy, so the fix went into how the sales team uses the score, not into the model's code

Likely follow-ups: Who should own sign-off on a responsible AI review, and why not just the model's own builder? How would you handle a model that passes every technical fairness check but still produces outcomes stakeholders are uncomfortable with?

Red flag: treating "responsible AI" as a single vague value judgment rather than a structured set of specific, checkable risks.

Learn it in: Chapter 39, section 39.9 (fairness checks); Chapter 64, sections 64.4 (privacy by design), 64.7 (a fairness audit, worked) and 64.8 (model governance and sign-off).

Rapid-fire, 80.4

Roles: ARC and LEAD for every row; DE for Q80-013 and Q80-014.

#	Question	One-line answer	Extra point	Level · learn it in
Q80-012	What's data governance, at the level an architect owns it?	The policies, standards, and accountability structures defining who can access, modify, and be responsible for specific data, distinct from the technical implementation of any one system	[+Signpost] name its four working parts: a catalog, lineage, a named owner for every governed dataset, and policies that are actually enforced	Senior · 64.6
Q80-013	What's the principle of least privilege?	Grant only the minimum access genuinely needed for a role or system to do its job, nothing more, reducing the damage any single compromised credential or account can do	[+Business] this applies as much to a service account calling an internal API as to a human user's login	Mid · 64.2; 64.3
Q80-014	What's segregation of duties, in a data platform context?	No single person or system should have unchecked end-to-end control over a sensitive process (approving and executing the same financial transaction, say), reducing both fraud risk and the impact of an honest mistake	[+Evidence] Chapter 64's model card for the lead score leaves its approver line empty until someone other than the builder signs it: the person who built a model can't approve it for production	Senior · 63.9; 64.8
Q80-015	What does "showback" mean in cloud cost management, and how is it different from "chargeback"?	Showback reports each team's cloud cost to them for visibility, without actually billing their budget for it; chargeback actually bills the cost to that team's budget	[+Business] showback is often a gentler first step toward cost accountability before an organization is ready for full chargeback	Mid · 65.4 (budgets, tagging, and showback)

Q80-016 · Calculate the ROI of automating a specific manual task, and use it to justify (or reject) building it

Level: Senior · **Roles:** DE, ARC, LEAD

Remember it as: A *payback period in months, computed from real numbers, is a far stronger justification than "this will save the team a lot of time."*

Answer in one line: Estimate the annual value of time saved (hours saved per run × runs per year × a fully-loaded hourly cost), compare it against the one-time build cost, and compute the **payback period** (one-time cost ÷ monthly net saving; the months until the build has paid for itself); if it's short relative to the automation's expected useful life, it's justified, and if not, it isn't, regardless of how tedious the manual task feels.

Run it yourself. Take a weekly task that costs three hours of someone's time each run, and a quote of ₹60,000 to automate it. First, the yearly value of the time saved, at Riverstone's loaded staff cost:

```
hours_saved_per_run = 3
runs_per_year = 52
hourly_cost = 300    # Riverstone's loaded staff cost per hour, the same as Chapter 20
annual_savings = hours_saved_per_run * runs_per_year * hourly_cost
print(f"Annual savings: ₹{annual_savings:,.0f}")
```

```
Annual savings: ₹46,800
```

How it works:

- `hours_saved_per_run`, `runs_per_year` and `hourly_cost` are the three drivers, one per line, so each can be questioned on its own. `hourly_cost = 300` is the loaded cost (salary plus overheads) that Chapter 20, section 20.14 used for the Flash; use your own company's figure in a real interview.
- `annual_savings` multiplies the three drivers: $3 \times 52 \times 300$.
- `print(f"...")` prints it with `:,.0f`, which adds thousands separators and drops the decimals (Chapter 17's f-strings).

Now the payback. Before you run the next cell, predict: is it under a year or over?

```
build_cost = 60_000    # the one-time quote to build it, in rupees
payback_months = build_cost / (annual_savings / 12)
print(f"Payback period: {payback_months:.1f} months")
```

```
Payback period: 15.4 months
```

How it works: `60_000` is Python's way of writing 60000 with a separator you can read; the underscore changes nothing about the number. `annual_savings / 12` is the saving per month, and dividing the one-time cost by it gives the months until the build has paid for itself, the same shape as Chapter 23, section 23.9's customer payback ($CAC \div \text{monthly gross profit}$). `:.1f` keeps one decimal place.

The follow-up every interviewer asks: what about the automation's own upkeep? What happens if you change the calculation to include two hours of maintenance a month? The saving shrinks and the payback stretches:

```
maintenance_hours_per_month = 2
net_annual = annual_savings - maintenance_hours_per_month * 12 * hourly_cost
net_payback_months = build_cost / (net_annual / 12)
print(f"Net annual savings: ₹{net_annual:,.0f}")
print(f"Payback, net of maintenance: {net_payback_months:.1f} months")
```

```
Net annual savings: ₹39,600
Payback, net of maintenance: 18.2 months
```

How it works: `maintenance_hours_per_month * 12 * hourly_cost` is the yearly cost of looking after the automation ($2 \times 12 \times 300 = ₹7,200$), and `net_annual` takes it off the gross saving. `net_payback_months` is the same division as before, on the net figure.

Reading it. At ₹300 an hour this is not an easy yes: the build pays for itself only after about a year and a half, net of upkeep. If the task will still exist in three years, the automation saves ₹1,18,800 against a ₹60,000 build, a gain of ₹58,800; if the process is due to change next year, it never pays back. That's the conversation the number lets you have.

Tier	What to say
Passes	A qualitative "this will definitely save time" with no actual number attached
Strong	The real calculation above, net of ongoing maintenance, giving a concrete payback period a business can actually evaluate
Extra points	[+Business] an 18-month payback is a genuinely different conversation from a 5-month one: the question becomes how long the task will exist, not whether it's tedious; worth having the honest number either way, not just when it's flattering [+Assume] say which inputs are measured (the hours, timed before and after, as Chapter 20 insists) and which are assumptions (the maintenance hours), as Chapter 63's ROI table does [+Edge cases] a good payback doesn't override a high risk: in Chapter 63, straight-through PO intake had better labour economics than the assisted mode and was still rejected, because its wrong orders cost far more than it saved

Likely follow-ups: How would you estimate the "fully loaded hourly cost" figure honestly? How long does the task have to keep existing for this build to be worth it?

Red flag: justifying an automation investment with no quantified payback calculation at all.

Learn it in: Chapter 20, section 20.14 (hours saved $\times$ loaded rate); Chapter 23, section 23.9 (the payback period); Chapter 63, section 63.2 (value, effort and risk); Chapter 66, section 66.3 (the honest business case).

80.5 Leadership scenarios

Q80-017 · A senior stakeholder wants you to build something you believe is technically the wrong approach. How do you push back, live?

Level: Senior · **Roles:** DE, DS, ARC, LEAD

Remember it as: *Lead with the specific risk, in their terms, not a technical objection in yours. And always come with an alternative, not just a "no."*

Answer in one line: Frame the pushback in terms of the *business* risk their preferred approach creates (cost, timeline, maintainability, a specific failure mode), not a purely technical objection that doesn't connect to what they actually care about, and always propose a concrete alternative rather than just a disagreement.

Worked example:

"I'd say: 'I want to flag a real risk with this approach before we commit. This choice will likely cost us significantly more to maintain within a year as we scale, based on [specific reason]. Here's an alternative that gets us the same business outcome with a lower long-term cost. I'm glad to move forward with your original approach if there's a constraint I'm not seeing, but I wanted to make sure this trade-off was visible before we're locked in.'"

Tier	What to say
Passes	Either silently complies despite disagreeing, or pushes back in purely technical language the stakeholder can't evaluate
Strong	The business-risk framing above, paired with a concrete alternative, not just an objection
Extra points	[+Business] explicitly leaving room for the stakeholder to have context you don't ("if there's a constraint I'm not seeing") respects their position while still making the risk fully visible, rather than framing it as a pure technical-authority standoff

Likely follow-ups: What if the stakeholder proceeds with their original approach anyway? How would you document this disagreement for the record without it reading as insubordination?

Red flag: either silent compliance or a purely technical objection with no business framing and no alternative offered.

Learn it in: Chapter 67, section 67.4 ("Saying no"); Chapter 24, section 24.8 (handling pushback).

Rapid-fire, 80.5

Roles: ARC and LEAD for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q80-018	What's "influencing without authority," and why does an architect need it specifically?	Getting other teams to adopt a technical direction or standard without having any formal management authority over them, relying instead on credibility, clear reasoning, and genuine relationship-building	[+Evidence] in Chapter 67, Meera won the sales team over to data contracts with their own duplicated-macro incident, not an email announcing a standard	Senior · 67.4 ("Influencing without authority")
Q80-019	How would you present a technical architecture decision to a board with no technical background?	Lead with the business outcome and risk in plain language, keep technical detail available only if asked, and anchor the decision in cost, risk, and timeline, the terms a board actually evaluates decisions in	[+Business] lead with the number a skeptic would ask for first, and keep a conditional saving out of the total until its condition is shown to hold	Senior · 67.3 ("Presenting to a board"), built on 24.7
Q80-020	What would your first 90 days look like as a newly hired architect at a company you don't know yet?	Listen and map the current state before proposing changes: understand existing systems, meet key stakeholders, identify quick wins that build credibility, and avoid a sweeping redesign proposal before genuinely understanding why things are the way they are	[+Signpost] name the three phases: listen (weeks 1-4), diagnose (weeks 5-8), earn credibility with one small, visible fix (weeks 9-12)	Senior · 67.6
Q80-021	How would you handle two senior engineers on your team who strongly disagree about a technical direction?	Get both positions fully understood first, look for what each is actually optimizing for underneath the disagreement, and make a clear, explained decision rather than letting the disagreement stay unresolved or quietly picking a side without reasoning	[+Business] an unresolved technical disagreement between two senior people tends to quietly split a team's execution, even if nobody says so out loud	Senior · 67.4 (leading people); 67.5 (judgment)

80.6 Full architecture design cases

Q80-022 · Design case: go back to the Riverstone of Parts 2 and 3, which runs everything off its ERP's database, and architect its next three years of data infrastructure

Level: Senior · **Roles:** ARC, LEAD, DE

What they're really testing: whether a multi-year architecture roadmap is grounded in actual current constraints and growth signals, not a greenfield fantasy design ignoring where the company actually is today. The setting is Riverstone before Part 5 was built: orders and customers in the ERP's PostgreSQL database, the reports and spreadsheets of Part 2, and a Daily Sales Flash sent by a scheduled Python script (Chapter 20). No warehouse, no orchestrator, no models yet.

Talked through live, start to finish:

"I'd start by understanding today's actual pain points and growth trajectory, not designing in a vacuum: what's the current data volume, what's genuinely slow or fragile today, and what's the realistic growth rate. Assuming meaningful growth is expected, I'd propose a staged path, not a single big-bang migration: first, introduce a proper data warehouse alongside the existing operational database, so analytics workloads stop competing with production traffic (row storage for running the business, columnar for understanding it: Chapter 49, section 49.2). Second, once volume genuinely justifies it, introduce the pipeline and orchestration patterns from Chapter 46 to automate what's currently manual. Third, only once there's a genuine multi-team, multi-source scale problem, consider a lakehouse (Chapter 49, section 49.5) or a data mesh (Chapter 62, section 62.4, and only after section 62.6's readiness check passes), resisting the temptation to build for a scale Riverstone doesn't have yet just because it's architecturally interesting."

Extra-points moves demonstrated: **[+Business]** explicitly resisted over-engineering for a scale the company doesn't have yet. **[+Signpost]** proposed a staged roadmap, and connected each stage to a specific technique already covered in this book rather than inventing new concepts wholesale. **[+Scale]** tied each stage to a growth trigger (volume, then several teams and sources) rather than a date or a single fixed end-state design.

Check your roadmap against what happened. Parts 5 to 7 are what Riverstone actually did: ingestion into a warehouse (Chapters 45 and 49), an orchestrator (Chapter 46; ADR-002 chose Dagster), and then, once it was all running, a design document that described it as one system (Chapter 60, section 60.5). Compare your three stages with that document's containers and ADR index: which of your stages did Riverstone take, in what order, and which did it rightly not take yet? (Chapter 62, section 62.6 scored it not ready for a data mesh.)

Likely follow-ups: What would make you accelerate this roadmap, or slow it down? How would you sequence this against the team's existing day-to-day workload?

Red flag: a single, final-state architecture diagram with no staged path or connection to current actual constraints.

Learn it in: Chapter 60, sections 60.5 (the design document) and 60.6 (evolving a design under real constraints); Chapter 62, section 62.6 (the mesh readiness check); Chapter 66, sections 66.1 (a data strategy on one page) and 66.2 (maturity, scored honestly). The orchestration half is drilled in Chapter 77.

Q80-023 · Design case: your team's biggest automation just failed silently for a week, causing real financial impact. As the architect, how do you respond, and what changes structurally?

Level: Senior · **Roles:** ARC, LEAD

What they're really testing: whether the response goes beyond fixing the immediate bug to a genuine structural/governance change, matching the seniority of the role being interviewed for.

Talked through live, start to finish:

"Immediate: fix the specific failure and communicate the actual financial impact honestly to stakeholders, no minimizing it. Then, at the architecture level: this failure mode (silent, no alert) suggests our automation governance has a gap, not just this one automation. I'd propose an automation inventory (Chapter 63, section 63.8) if one doesn't already exist, since 'how many critical automations do we even have, and does each have monitoring' is often the real unanswered question after an incident like this. I'd establish a standard that every business-critical automation must have a defined owner and a minimum monitoring bar (freshness and volume checks, Chapter 47, section 47.5) before it's allowed into that category, and I'd run this as a genuine retrospective with the team, not just a top-down mandate, since the people closest to the work usually know which other automations share this same risk."

Extra-points moves demonstrated: [+Business] distinguished the immediate technical fix from the structural, governance-level change a senior role is actually being evaluated on. [+Evidence] connected to a named organizational tool (an automation inventory) rather than a vague "we'll be more careful" response.

Likely follow-ups: How would you prioritize which existing automations to audit first, given limited time? How would you get buy-in for the new standard without it feeling like blame directed at whoever built the original automation?

Red flag: a response that only fixes the immediate bug, with no structural or governance-level change proposed.

Learn it in: Chapter 63, sections 63.7 (ownership, runbooks, and support) and 63.8 (the automation inventory); Chapter 47, section 47.5 (freshness and volume monitoring); Chapter 78, Q78-018 (the identical incident, examined there at the individual-debugging level instead of the architectural-response level).

APPLY

Common mistakes

Mistake	Symptom	Fix
Claiming a system can be fully consistent, available, and partition-tolerant	Reveals a misunderstanding of the CAP theorem's actual constraint	Frame the real trade-off as CP vs. AP during an actual partition
Answering an availability question with "we have backups"	The data survives, but the service still stops until the restore finishes	Separate durability from availability; name what takes over while the main copy is down
Comparing build vs. buy on sticker price alone	Underestimates a build decision's true long-term cost	Compare total cost of ownership on both sides
A single architecture diagram trying to serve every audience	Too detailed for executives, too vague for engineers	Use layered diagrams (C4) matched to each audience
Treating "responsible AI" as one vague judgment call	Real, specific risks (fairness, privacy, misuse) go unchecked	Structure the review around named, checkable dimensions

Mistake	Symptom	Fix
Justifying an automation investment with no ROI number	A budget owner has nothing concrete to evaluate	Calculate a real payback period, net of maintenance, before proposing the build
Pushing back on a stakeholder in purely technical language	The stakeholder can't evaluate a risk they can't understand	Frame pushback in business-risk terms, with an alternative offered
A sweeping architecture proposal in an architect's first weeks	No credibility yet, and often misses real existing constraints	Listen and map the current state before proposing major change

In the real world: the standard that came from one bad incident

In his previous role, Siddharth inherited a team where a critical nightly automation had failed silently for a week, costing real money before anyone noticed. Interviewing for a Data Architect role a year later, he describes the incident and is asked the pointed follow-up: "Everyone says 'we'll add monitoring' after an incident like this. What did you actually do differently?"

His answer: he didn't just add monitoring to that one automation. He ran a genuine inventory across the team, discovering fourteen other business-critical automations with no monitoring at all, several of which nobody could confidently name an owner for. Rather than mandating a fix top-down, he brought the finding back to the team as a shared problem: "We have fifteen single points of failure like the one that just cost us real money. Which of these worry you most?" The team, not Siddharth alone, prioritized the list, and the resulting standard, every business-critical automation needs a named owner and a minimum monitoring bar before launch, came from the team's own prioritization, not an imposed mandate.

The interviewer's note: *"Turned one incident into a systemic finding, and built the fix as something the team owned, not something handed down."* That's the difference this chapter's leadership sections are built around: a senior technical response fixes the bug; an architectural and leadership response fixes the condition that made the bug possible, and does it in a way the team actually adopts rather than resents.

Project

Goal: apply this chapter's frameworks to a real or hypothetical architecture decision.

Tools you'll need

No software specific to this chapter. A whiteboard or diagramming tool for C4 diagrams and architecture sketches; a shared document for Architecture Decision Records; a spreadsheet for ROI and unit-economics calculations, the same skills Chapters 10–11 taught (and Chapter 70 drills), or the Jupyter notebook from Chapter 17 for Q80–016's cells.

1. Write a one-page ADR for a real technical decision you've made or observed, including the context, options considered, and reasoning, not just the final choice.

2. Calculate a real ROI/payback period for one automation or system you're familiar with, using Q80-016's method, net of maintenance.
3. Draft the business-risk-framed pushback you'd give for a real or hypothetical technical disagreement in your own context.
4. Sketch a staged, three-stage architecture roadmap for a system you know, tied to specific growth triggers rather than a single end-state design.

REVIEW

Key terms

CAP theorem · CP vs. AP · PACELC · eventual consistency · durability vs. availability · single point of failure · horizontal vs. vertical scaling · partitioning vs. sharding · leader-follower (primary-replica) replication · failover · individual contributor (IC) · build vs. buy · total cost of ownership (TCO) · lambda architecture · kappa architecture · Architecture Decision Record (ADR) · C4 diagram · Responsible AI review · data governance · principle of least privilege · segregation of duties · showback vs. chargeback · ROI/payback period · influencing without authority · first-90-days plan (architect)

Final-week revision list

Q80-001, Q80-006, Q80-009, Q80-011, Q80-016, Q80-017, Q80-020, Q80-022, Q80-023, Q80-024.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 77 and Chapter 78** already cover the technical single-point-of-failure and monitoring concepts this chapter's leadership scenarios examine at the organizational and governance level instead.
- **Chapter 24's storytelling discipline** (sections 24.7 and 24.8) underlies this chapter's board-presentation and stakeholder-pushback questions.
- **Chapters 60–67 of this book** (Part 7: Architecture, Governance & Leadership) teach the techniques this bank draws on, with Chapter 20 (section 20.14) and Chapter 23 (section 23.9) for the time-saving and payback arithmetic; this chapter tests them rather than re-teaching them.
- **Chapter 81, Behavioral, HR & Offer Conversations**, comes next: the leadership stories you rehearsed here (the pushback, the incident that became a standard) are the raw material for its behavioral answers.

Chapter 81. Behavioral, HR & Offer Conversations

Chapter at a glance

You will learn to: answer any behavioral question using the STAR method without sounding scripted · build a small story bank from your own experience that covers most behavioral themes with five to eight real stories, not forty different ones · ask questions that actually reveal something about the role, not filler · answer "what's your expected CTC?", read an offer's breakup, and negotiate without either accepting the first number or overplaying a weak hand.

Before you start: Chapter 8, §8.6 (reading salary figures; CTC and in-hand pay) · Chapter 68 (the hiring process gate by gate, and §68.6 on pay before the offer gate) · Chapter 69 (the three answer tiers and the twelve extra-point tags) · Chapter 76A (talking about your own projects). The worked examples reuse your portfolio projects from Chapters 27 and 44.

Time needed: 2–3 hours to read and say every core answer aloud once; 4–6 hours for the project (building your own story bank, three STAR outlines, and timed practice).

How this chapter is built. Same format as Chapters 70–80: every core question leads with a "Remember it as..." hook, a one-line answer, and a compact tier table whose extra points carry Chapter 69's tags ([+Business], [+Limits] and so on). Rapid-fire sections are scan tables. There is no code in this chapter; every worked example is a full, concrete STAR answer, not a description of what one should contain, and every negotiation script is written out in full. These questions are asked of every role and at every level, so the questions carry no level or role labels; where your level changes the answer (a first job, for example), the answer says so.

A note on reuse. Worked examples reuse the book's recurring people and projects (Farah, the sales executive who moved into data in Chapters 8, 9 and 27; your Chapter 27 and Chapter 44 portfolio projects), so you can see a real story told well. Where an example is a composite written for this chapter, it says so. Your own story bank must hold only your own experiences.

In this chapter

LEARN	81.1 The STAR method	355
	81.2 Building a story bank	356
	81.3 Classic behavioral questions, worked in full	358
	81.4 Basic-but-tricky HR questions	360
	81.5 Questions to ask interviewers	362
	81.6 Handling offers	363
	81.7 Handling a hostile follow-up, live	366
APPLY	Common mistakes · In the real world · Project	367
REVIEW	Key terms · Final-week revision list	368
NEXT	Where this leads	369

81.1 The STAR method

Q81-001 · What's the STAR method, and what's the most common way candidates get it wrong?

Remember it as: *Situation and Task set the scene quickly. Action is where most of your answer's time should actually go. Result needs a number, even a rough one.*

Answer in one line: **Situation** (brief context), **Task** (what you specifically needed to accomplish), **Action** (what you actually did, step by step, the part that should take up most of the answer), **Result** (the outcome, quantified where possible); the most common failure is spending 80% of the answer on Situation and Task and rushing through Action and Result, exactly backwards from where the interviewer's attention actually needs to go.

Worked example, your Chapter 44 capstone told as STAR:

Situation: "In my capstone portfolio project, built on a practice dataset of a supplies distributor's business accounts, the question was which at-risk accounts a sales team should call in a month when it only had time for 40 calls."

Task: "I needed a call list that reflected which accounts were actually worth a call, not just which were most likely to leave."

Action: "I found that ranking by churn probability alone missed large accounts with only moderate risk. So I ranked by expected margin at risk instead: probability times revenue times margin. I dropped accounts where a ₹1,500 call couldn't pay for itself, and compared my list with a probability-only list from the same model: they shared only 10 of their 40 accounts. Then I wrapped the steps in two small functions, so the list can be rebuilt with one line for a different number of calls."

Result: "With the same 40 calls, the value-ranked list targeted about ₹12.0 lakh of expected margin at risk, against ₹3.9 lakh for the list ranked by probability alone: about three times as much. That's what the model expects, not money saved, so I recommended a small experiment (Chapter 30) to measure how many calls actually keep an account."

Tier	What to say
Passes	All four letters present, but Action is one vague sentence ("I built a model") and Result has no number
Strong	The full worked example above: brief Situation/Task, detailed Action, quantified Result
Extra points	+ [+Validate] the Result says exactly what the number is (expected margin at risk, from the model) and what it isn't (money saved), which is exactly the kind of checkable claim Chapter 76A's resume-scrutiny discipline rewards + [+Limits] recommending the experiment that would measure the real effect, unprompted, shows you know where your own evidence stops

Likely follow-ups: What would you do differently if you did this project again? What was the hardest part of the Action you just described?

Red flag: an Action section so brief it could apply to almost any project, with no specific technical or

judgment detail at all.

Learn it in: Chapter 44, §44.3–44.4 (expected margin at risk and the call list) and Chapter 76A, Q76A-007 (the same business-question-first, quantified-impact structure, here formalized as STAR).

Rapid-fire, 81.1

#	Question	One-line answer	Extra point
Q81-002	How long should a STAR answer take, out loud?	Roughly 1.5 to 2.5 minutes; shorter risks feeling thin, longer risks losing the interviewer's attention or eating time needed for follow-ups. About 90 seconds is fine in a round heavy with follow-ups	[+Business] practicing out loud with a timer, not just reading a written answer silently, is the only reliable way to know your actual pacing
Q81-003	Is it acceptable to use a story from outside your professional work (a class project, a portfolio project, volunteering) for a behavioral question?	Yes, especially early in your career, as long as the story genuinely demonstrates the behavior being asked about with real specificity, and you say plainly what kind of project it was	[+Edge cases] a thin professional story loses to a detailed, honest non-professional one every time
Q81-004	What if you genuinely can't think of a story fitting the exact question asked?	Pick your closest real story and be honest about the fit ("this isn't a perfect match, but it's the closest real example I have"), rather than forcing or fabricating a better-fitting one	[+Limits] saying the fit isn't perfect is Chapter 69's Move 10 (admit limits honestly, §69.3), and it earns trust for the rest of the answer
Q81-005	Should every STAR answer end with what you learned?	Not mandatory, but a brief, genuine reflection often strengthens an answer, especially for a story where the result wasn't a clean win	[+Business] a forced, generic "I learned to always communicate more" adds nothing; a specific, real lesson does

81.2 Building a story bank

Q81-006 · You have limited time to prepare. How do you build a story bank that covers most behavioral questions without preparing forty separate stories?

Remember it as: Most behavioral questions are really asking about one of seven underlying themes. Map five to eight of your own real stories to those themes, and most questions become "which story do I already have for this," not "what do I say."

Answer in one line: Identify the small set of themes that cover the large majority of behavioral questions (conflict/disagreement, failure/mistake, ambiguity, influencing without authority, pressure/high stakes, going above and beyond, receiving critical feedback), and for each, prepare one real, detailed story of your own, so five to eight well-prepared stories can flexibly answer dozens of specific question phrasings.

The one rule of a story bank. Every story in your bank must be something you did. The book's projects (Chapter 27's portfolio, Chapter 44's capstone) count as your own portfolio work, so use them, but always call them portfolio or practice projects, never work you did at a company. Never retell someone else's incident as yours: a follow-up question about a detail you weren't there for exposes it in seconds, and a candidate caught doing it fails outright.

Worked example: how Farah (the sales executive who moved into data, Chapters 8, 9 and 27) mapped *her own* experiences. Five stories cover all seven themes, because two of them serve twice:

- **Conflict/disagreement:** the call list in Chapter 44's real-world story, where she questioned a large account flagged for late payments, because she knew the payments were late during the customer's plant shutdown and were normal again; the list stayed, and she opened the call with the shutdown instead of the payments
- **Failure/mistake:** her first portfolio finding in Chapter 27 ("early-season buyers are worth more"), which fell apart when the comparison group was fixed; she kept both versions in the project and said so in the README
- **Receiving critical feedback:** the same Chapter 27 story, told from Anita's one question, "worth more than whom?", and what Farah did with it
- **Ambiguity:** her requirements project in Chapter 76B, where "a way to see which customers haven't ordered recently" turned out to be three requirements for three teams, and a one-week estimate became an honest three weeks
- **Influencing without authority:** the Monday at-risk customer email she built as a sales executive in Chapter 8, with no analyst title, which in its first month caught two customers she would have missed; at the sales review Vikram asked whether his team could have the same email
- **Going above and beyond:** the same Chapter 8 story, told from how she built the email while holding a full-time sales job, and asked Anita for Friday afternoons on the at-risk customer list rather than waiting to be offered them
- **Pressure/high stakes:** the pricing test in Chapter 73 that looked like a clear win after ten days; she ran the sample-ratio check before recommending it, found 5,340 users in control against 4,660 in treatment, and didn't ship

Tier	What to say
Passes	Prepares one all-purpose story and tries to force it to answer every question, regardless of fit
Strong	A theme-to-story mapping like the one above, five to eight of your own stories covering the seven themes
Extra points	+ [+Business] one real story can often serve two or three themes depending on which part you emphasize: Farah's Chapter 27 story answers "a failure," "critical feedback," and "attention to detail," told three slightly different ways

Likely follow-ups: Which of your prepared stories would you use for "tell me about a time you influenced someone without authority"? What's your backup story if an interviewer says "give me a different example" after your first one?

Red flag: forcing an obviously mismatched story onto a question rather than acknowledging a closer story would be better if available.

Learn it in: Chapter 76A's whole portfolio-narrative discipline, applied here to behavioral stories instead of technical project walkthroughs; Chapter 27, §27.10 (telling the story, in two minutes and in ten).

Rapid-fire, 81.2

#	Question	One-line answer	Extra point
Q81-007	Should your story bank favor recent stories or your single most impressive one, regardless of age?	Generally recent and relevant over impressive-but-old; a recent story also signals current skill level and is easier to speak about with fresh, specific detail	[+Edge cases] a genuinely exceptional older story is still worth having ready as a backup, just not as your default
Q81-008	How many stories does a well-prepared candidate typically need?	Five to eight well-developed stories, flexibly told, usually covers the large majority of behavioral interviews	[+Business] quality and specificity of preparation beats sheer quantity of half-prepared stories every time
Q81-009	Should you write out full scripts for each story, word for word?	No, per Chapter 76A's own guidance (Q76A-024): know the facts cold, don't memorize the exact wording, or the story breaks under a genuine follow-up that deviates from the script	[+Edge cases] write each story as bullet-point facts (the numbers, the dates, who was involved), not sentences, so every retelling is fresh
Q81-010	What's the risk of reusing the exact same story for two different questions in the same interview round?	If the same interviewer or panel hears it twice, it can read as having a thin, limited set of real experiences	[+Business] know which stories you've already used in a given interview loop, especially across multiple rounds with overlapping panelists

81.3 Classic behavioral questions, worked in full

Q81-011 · "Tell me about a time you disagreed with a decision at work." Give a full worked STAR answer

Remember it as: This question wants to see how you disagree, not whether you were right. A story where you were graciously wrong, or only half right, is often stronger than one where you were vindicated.

Worked example (a composite written for this chapter, to show the shape of a strong answer; your version must be something you did):

Situation: "Our sales head wanted the daily sales email to reach managers by 7 a.m. I argued for 9 a.m., because the overnight data load sometimes finished late."

Task: "We needed a send time the managers could both use and trust."

Action: "I showed her two recent mornings when a 7 a.m. email would have gone out on incomplete data. She pointed out something I hadn't asked about: the managers plan their day at 8, so a 9 a.m. email would arrive after every decision it was meant to inform. We agreed on 7:30, and I added a check that puts a 'data incomplete' banner at the top of the email whenever the load hasn't finished."

Result: "The banner appeared twice in the first quarter, and on both mornings nobody acted on a wrong number. I was half right: the data risk was real, but so was her deadline, and I hadn't asked about it before arguing."

Tier	What to say
Passes	A vague "I disagreed and eventually we found a compromise" with no specific mechanism or outcome
Strong	The full worked example above, with a specific reason for the disagreement, what the other person knew that you didn't, and a real, checkable outcome
Extra points	+ [+Business] the disagreement is settled by evidence on both sides (two bad mornings; the managers' 8 a.m. planning) rather than by force of personality or hierarchy, which is exactly the kind of disagreement-handling most technical roles want + [+Limits] "I was half right" names your own part of the problem plainly, which is far more convincing than a story in which everyone eventually agreed with you

Likely follow-ups: How did she react in the moment, before you had the two mornings to point to? Have you ever pushed back and been completely wrong?

Red flag: a story where "disagreement" is really just "I was right and everyone eventually agreed," with no genuine tension or risk described.

Learn it in: Chapter 24, §24.8 (handling pushback, calmly and with reasons) and Chapter 47, §47.5 (freshness checks, and in its real-world story the status banner).

Q81-012 · "Tell me about a time you failed." Give a full worked STAR answer, including how to choose which failure to share

Remember it as: Choose a real failure with a real, honest lesson, not a humble-brag disguised as a weakness ("I work too hard").

Worked example (a composite written for this chapter; the numbers are illustrative):

Situation: "Early in a project, I built a duplicate-detection query for our customer list and reported zero duplicate customers."

Task: "That number fed a decision about whether to clean up the CRM, so it needed to be right."

Action: "I had used an exact match and hadn't checked it a second way. A week later, on a related task, I noticed two 'different' customers with the same phone number. Nobody else had caught it. I re-ran the check after normalizing the names with `TRIM` and `LOWER`, found the near-duplicates that trailing spaces and capital letters had hidden, and told the manager the same day, with the corrected figure."

Result: "The real count was 37 duplicates in 1,200 customer records, about 3%, so the clean-up plan changed. Since then, whenever I report a duplicate check, I say what it does and doesn't catch."

Tier	What to say
Passes	A "failure" that's actually a strength in disguise, or a genuine failure with no real lesson or change in behavior afterward
Strong	A real, specific mistake, honestly described, with the correction, a number, and a concrete change in behavior that followed
Extra points	+ [+Limits] stating that <i>nobody caught the error but you</i> is a specific, honest detail that makes the story more credible, not less, since it shows genuine self-review rather than only being accountable when caught + [+Close] telling the manager the same day, with the corrected number, closes the loop on everyone who saw the wrong one

Likely follow-ups: How did the manager react to the correction? Has that caveat-stating habit come up again since?

Red flag: choosing a "failure" so mild or so clearly a backhanded strength that it doesn't actually answer the question asked.

Learn it in: Chapter 14, §14.4 (duplicates, exact and fuzzy) and §14.5 (normalizing text with `LOWER(TRIM())`); Chapter 69, §69.3, Move 10 (admit limits honestly). **Practise it with:** Chapter 71, Q71-074, where a first attempt fails and the answer says so plainly before fixing it.

81.4 Basic-but-tricky HR questions

Q81-013 · "What's your biggest weakness?" How do you answer honestly without either self-sabotaging or giving an obviously fake answer?

Remember it as: A fake weakness ("I'm too much of a perfectionist") is instantly recognizable and costs more credibility than an honest one ever would.

Answer in one line: Name a real, specific weakness, ideally one you're actively working on with a concrete example of how, rather than a backhanded-compliment weakness or a vague, unfalsifiable one.

Worked example:

"I used to under-communicate progress on long projects, assuming people would ask if they needed an update, which sometimes left a stakeholder anxious mid-project with no idea whether things were on track. I've built a habit of a short, scheduled check-in message at fixed points now, even when there's nothing dramatic to report, specifically because I know this is a real gap for me, not something I've fully solved."

Tier	What to say
Passes	"I'm a perfectionist" or "I work too hard" (transparently a strength dressed up as a weakness)
Strong	A real, specific, believable weakness with a concrete mitigation habit
Extra points	+ [+Limits] naming the weakness as ongoing ("I know this is a real gap for me, not something I've fully solved") rather than claiming it's fully fixed reads as more honest and more self-aware than a tidy "and now I've completely overcome it" ending

Likely follow-ups: Can you give a specific recent example of this weakness actually showing up? How would a former colleague describe this same weakness in you?

Red flag: an obviously strategic "weakness" that's actually a strength, or a weakness so vague it can't be evaluated at all.

Learn it in: Chapter 69, §69.3, Move 10 (admit limits honestly).

Rapid-fire, 81.4

#	Question	One-line answer	Extra point
Q81-014	"Tell me about yourself." What structure works best?	A brief present-past-future arc: what you do now, the relevant path that got you here, and what you're looking for next, in under two minutes	[+Business] this isn't a full career history; it's a focused pitch relevant to the specific role
Q81-015	"Why are you leaving your current role?" How do you answer without badmouthing your employer?	Frame it in terms of what you're moving toward, not what you're escaping; a specific, positive reason for this role beats any negative reason for leaving the last one	Red flag: speaking negatively about a current or former employer, however justified it might feel, reads as a risk to any future employer
Q81-016	How do you explain a gap in your work history honestly?	State plainly what the gap was for (caregiving, a health matter, further study, a genuine job search), briefly, then move the conversation forward to what you did with the time or what you're ready for now. You never owe details of a health matter; "a health matter, now resolved" is enough	[+Business] most interviewers are far less concerned about a gap itself than about how directly and confidently a candidate discusses it
Q81-017	"Where do you see yourself in five years?" What's this actually testing?	Whether your stated trajectory is remotely compatible with what this specific role can actually offer, not a literal prediction held against you later	[+Business] an answer wildly mismatched to the role (a five-year plan to manage a team, for an individual-contributor-only role) is a genuine signal worth addressing honestly rather than glossing over

#	Question	One-line answer	Extra point
Q81-018	What if you're asked something personal and irrelevant to the job (marriage plans, family plans, religion)?	You can redirect politely to what matters for the job ("I'm fully able to commit to the role's hours and travel"). You don't have to answer	[+Edge cases] the question itself tells you something about the employer; note it when you weigh the offer

81.5 Questions to ask interviewers

Q81-019 · What makes a question to an interviewer strong versus weak, and give three strong examples for a data role

Remember it as: A strong question could only be answered by someone actually in this specific role at this specific company, not by reading the company's website.

Answer in one line: A weak question is answerable from public information (the company's about page, a quick search) or is so generic it reveals nothing was actually prepared; a strong question is specific to the role, the team, or something you learned during the interview itself, and genuinely helps you decide if this is the right fit.

Three worked examples:

- "What does the data quality situation actually look like day to day here, is it mostly clean, or is a meaningful share of your time spent on cleanup and validation?"
- "You mentioned the team is planning a shift toward more self-service analytics. What's actually driving that, and what would make it a success a year from now?"
- "What's a recent decision this team made where the data pointed one way and the actual call ended up going a different direction? What happened?"

Tier	What to say
Passes	"What's the company culture like?" or "What do you like about working here?" (generic, could be asked at any company)
Strong	At least one question specific to something mentioned earlier in the interview, showing active listening, not a pre-written list recited regardless of context
Extra points	+ [+Business] the third example above is a genuinely revealing question: how an organization actually handles data-versus-judgment tension tells you more about its real data culture than almost anything else you could ask directly

Likely follow-ups: (This question doesn't have a standard follow-up; it usually ends the interview.)

Red flag: having no questions prepared at all, or only asking about compensation and benefits at this stage.

Learn it in: Chapter 69, §69.3, Move 1 (clarify first), the same instinct for a genuinely useful question, applied here to evaluating the employer instead of a technical problem.

Rapid-fire, 81.5

#	Question	One-line answer	Extra point
Q81-020	Is it appropriate to ask about work-life balance directly in a first interview?	Generally yes, framed constructively ("what does a typical week look like") rather than as a red flag about your own commitment	[+Business] how the question is received is itself useful information about the company
Q81-021	Should you ask different questions to a peer interviewer versus a hiring manager?	Yes: a peer can speak to day-to-day reality better than strategy; a hiring manager can speak to team direction and expectations better than daily workflow	[+Business] match the question to who can actually answer it well
Q81-022	Is it ever appropriate to ask about the reason a role is open (backfill vs. new headcount)?	Yes, and it's a genuinely useful question: a backfill invites "what happened to the person before me," which can surface real information about role fit or team dynamics	[+Edge cases] ask this respectfully, not as an interrogation
Q81-023	What's the risk of asking zero questions at the end of an interview?	Reads as a lack of genuine interest or preparation, regardless of how strong the rest of the interview went	[+Business] always have at least two or three ready, even if some get naturally answered earlier in the conversation

81.6 Handling offers

Watch out: general guidance, not advice. This section is general guidance on conversations, not financial, tax or legal advice. Read the whole offer letter before signing. Ask HR in writing about anything unclear: how the variable pay has actually been paid out, whether a joining bonus must be paid back if you leave early, any service bond, and the notice period. For tax or contract questions, ask a qualified professional. Pay norms and the rules on provident fund, gratuity and tax change; check current sources, as Chapter 8, §8.6 showed.

Q81-024 · "What's your expected CTC?" And how do you read the offer's breakup when it arrives?

Remember it as: *Negotiate the fixed part. Compare offers on monthly in-hand pay, not on the CTC headline.*

Answer in one line: Give the recruiter a range you can defend (from two salary sources, for your city and experience, as Chapter 68, §68.6 showed), and when the offer comes, ask for the full **breakup**: how much of the **CTC** (cost to company, Chapter 8, §8.6) is **fixed pay**, how much is **variable pay** that isn't guaranteed, and how much is the employer's own contributions that never reach your monthly pay.

Worked example, an illustrative breakup (the numbers are made up for this example; real offers are structured in many different ways):

Part of the CTC	₹ a year	Reaches your bank account?
Fixed salary	5,10,000	Yes, every month, before your deductions
Target variable pay	60,000	Only if paid; not guaranteed
Employer's provident fund (PF) contribution	21,600	No: it goes to your retirement account
Gratuity	8,400	No: paid only after a qualifying period of service
CTC	6,00,000	

The headline says ₹6 lakh, but the monthly fixed gross is $\text{₹}5,10,000 \div 12 = \text{₹}42,500$. Your **in-hand** pay is lower still, after your own PF contribution, income tax and any other deductions, which depend on your tax regime and the rules that year.

"Thank you, I'm glad to have the offer. Could you share the full breakup? Of the ₹6 lakh, how much is fixed, and how has the variable part actually been paid out in the last couple of years? [After reviewing:] Based on my research on comparable roles in this city, I was hoping the fixed component could be closer to ₹5.6 lakh. Is there flexibility there?"

Tier	What to say
Passes	Names one number with no source, and compares offers on the CTC headline
Strong	A defended range at the recruiter call; at the offer, asks for the breakup and negotiates the fixed part, as in the script above
Extra points	+ [+Clarify] asking how the variable part has actually been paid out turns a "target" into evidence + [+Validate] working out the monthly fixed gross yourself (₹42,500 here) before comparing two offers, rather than trusting the headline

Likely follow-ups: What if they insist on a single number rather than a range? What if the fixed part can't move but the variable can?

Red flag: accepting or comparing offers on the CTC headline alone, without ever asking what's fixed.

Learn it in: Chapter 8, §8.6 (reading salary data; CTC and in-hand pay) and Chapter 68, §68.6 (pay before the offer gate, and setting your expected CTC).

Q81-025 · You receive an offer below what you expected. Walk through how you'd respond, live

Remember it as: Don't accept or reject on the spot. Ask for a little time, within any deadline the offer states, then come back with a specific, justified counter, not just "can you do better."

Answer in one line: Thank them genuinely, ask for time to consider (typically a few days; if the offer has a stated deadline, respect it and ask only for what you need, "could I confirm by Friday?"), and if the number is genuinely below expectations, come back with a specific counter-number backed by a reason (market data, a competing offer, the value of a specific skill), rather than a vague "I was hoping for more."

Worked example:

"Thank you so much for the offer, I'm genuinely excited about the role. I'd like a few days to review the full details, is that alright? [After reviewing:] I really want to make this work. Based on my research on comparable roles and the specific skills this role needs, I was expecting the fixed pay to be closer to [specific number]. Is there flexibility there?"

Tier	What to say
Passes	Either accepts immediately without asking for time, or asks for more money with no specific number or justification
Strong	The ask-for-time, then-specific-justified-counter structure above
Extra points	+ [+Business] genuine enthusiasm ("I'm genuinely excited") stated alongside the negotiation signals you're negotiating in good faith toward accepting, not shopping the offer around purely as leverage, which most recruiters can tell the difference between and respond to differently

Likely follow-ups: What if they say the number is firm with no flexibility at all? How would you negotiate non-salary elements (start date, remote work, title) if the fixed pay truly can't move?

Red flag: accepting the first number without even a brief pause to consider it, when negotiation was clearly available.

Learn it in: Chapter 8, §8.6 (reading salary data; CTC and in-hand pay) and Chapter 68, §68.1 gate 7, §68.3 (the recruiter call) and §68.6 (pay before the offer gate). This conversation is the final gate that process was building toward.

Rapid-fire, 81.6

#	Question	One-line answer	Extra point
Q81-026	Should you disclose a competing offer's exact number when negotiating?	Optional, and there's a real trade-off: it's often a strong, concrete anchor, but disclosing it also removes ambiguity that sometimes worked in your favor	[+Edge cases] never state a competing number you don't actually have; it's easily and damagingly disprovable
Q81-027	Is it acceptable to negotiate a first job offer, or only for experienced hires?	Generally yes, though the room to negotiate is often narrower for entry-level roles with more standardized bands	[+Edge cases] campus and bulk-hiring offers are often fixed bands; negotiate the start date or location instead, or accept gracefully
Q81-028	What should you verify before accepting an offer beyond the fixed pay?	The CTC breakup (fixed vs variable, employer PF, gratuity, any joining bonus and whether it must be paid back if you leave early), the start date and how it fits your notice period, location or remote work, and role scope matching what was actually discussed in interviews	[+Business] a role whose actual day-to-day scope drifted from what interviews described is a mismatch worth catching before accepting, not after starting
Q81-029	How do you decline an offer gracefully, keeping the relationship intact?	Thank them specifically and genuinely, give a brief, honest reason without over-explaining, and decline promptly once decided rather than letting it drag	[+Business] the data and tech industry is smaller than it looks; a gracious decline preserves a relationship worth having later

#	Question	One-line answer	Extra point
Q81-030	Should you state your current CTC when asked?	Answer truthfully if asked: employers may ask for payslips later, for example during background verification. You can lead with your expected fixed pay, based on market data, rather than your current number	[+Evidence] "my expectation is based on two salary sources for this role and city" moves the talk from your past pay to the role's market
Q81-031	Your notice period is long. What do you say?	State it accurately, and ask whether they can wait or would buy it out (pay your current employer to let you leave early). Never resign before you have a written offer	[+Edge cases] a notice period you understate at the recruiter call becomes a broken promise at the offer stage
Q81-032	Is it OK to accept one offer and keep interviewing elsewhere?	Be honest with yourself and with them: decide before you accept. Accepting and then backing out (reneging) burns bridges in a small industry	[+Business] if another process is close to an offer, ask the first company for a few days before you accept, rather than accepting and withdrawing later
Q81-033	Your current employer makes a counter-offer to keep you. Should you take it?	A counter-offer here is your current employer's offer of more pay to stay. Weigh it against why you started looking: if the reason wasn't pay, more pay rarely fixes it	[+Trade-offs] decide on the whole reason for moving (work, learning, growth), not the last number on the table

81.7 Handling a hostile follow-up, live

Q81-034 · Full scenario, talked through live: an interviewer asks a rapid-fire follow-up to a behavioral answer that seems designed to find a flaw

What they're really testing: composure and honesty under a harder-than-expected follow-up, not whether the original story was flawless.

Talked through live, start to finish (Farah, telling her Chapter 73 pricing-test story):

Interviewer: "You said the pricing test result turned out to be a data bug, not a real effect. Didn't that waste everyone's time investigating something that wasn't real?"

Farah, live: "It's a fair challenge, and I'd actually argue the opposite: the investigation is exactly why we didn't ship a pricing change on a false result. The sample-ratio check took minutes; tracing the bug took two days. Against that, we'd have shipped a price change on a result that was measuring a loading bug, and the re-run showed no significant pricing effect. Two days against a wrong decision that nobody would have questioned afterwards: I'd make that trade on every test."

Extra-points moves demonstrated: **[+Signpost]** reframed the challenge directly ("I'd argue the opposite") rather than getting defensive or conceding a point that wasn't actually true. **[+Business]** compared the cost of checking (minutes, then two days) with the cost of a wrong decision, rather than just asserting the investigation was worthwhile.

Likely follow-ups: What if the interviewer keeps pushing, suggesting you should have trusted the initial result faster? How would you know when a challenge like this has a legitimate point buried in it, versus being purely a pressure test?

Red flag: getting visibly rattled or conceding a point that isn't actually true just to end the confrontation.

Learn it in: Chapter 69, §69.1, the collaboration dimension (how you handle being pushed matters as much as the original answer's content); Chapter 73's real-world story (the same incident, in its data-analysis framing).

APPLY

Common mistakes

Mistake	Symptom	Fix
A STAR answer with a thin Action section	Interviewer can't tell what you specifically did	Spend the majority of the answer's time on Action, with real detail
Forcing one all-purpose story onto every question	Obvious mismatch between question and story	Build a small story bank mapped to common themes
Telling someone else's story as your own	Falls apart at the first detailed follow-up	Use only your own experiences; call portfolio projects portfolio projects
A "weakness" that's really a disguised strength	Instantly recognizable, costs credibility	Name a real, specific, ongoing weakness with a concrete mitigation
Badmouthing a current or former employer	Reads as a future risk to the new employer	Frame leaving in terms of what you're moving toward
Asking only generic, website-answerable questions	Signals low preparation or interest	Ask something specific to the role or something said earlier in the interview
Comparing offers on the CTC headline	Accepting the "bigger" offer and getting less in hand	Ask for the breakup; compare fixed pay and monthly in-hand pay
Accepting an offer on the spot with no consideration time	Loses easy, expected negotiation room	Ask for a little time, within any stated deadline, even if you plan to accept
Getting defensive under a challenging follow-up	Reads as fragility under pressure	Reframe with evidence and reasoning, calmly

In the real world: the behavioral answer that was really a technical answer in disguise

Arjun, preparing for a Data Engineer interview, initially treats behavioral prep as a separate, lesser task from his technical prep, memorizing generic answers about "teamwork" and "communication" with no real specificity. In a mock interview, when asked "tell me about a time you found a bug that could have caused real damage," his prepared generic answer falls flat, vague, no real detail, nothing an interviewer could actually probe.

His mock interviewer points him back to a story he'd already told in the *technical* portion of the same mock session: the silently-failing pipeline that went undetected for three weeks because "zero rows extracted" wasn't technically an error. That story, already detailed and already understood cold from his technical prep, turns out to be a far stronger answer to "tell me about a mistake" or "tell me about identifying a risk" than anything he'd separately prepared as a "behavioral" story.

The lesson he took from it, and the one this chapter is built around: **your technical stories are usually your best behavioral stories too**, just told with the emphasis shifted from *what* you built to *how* you handled the situation, the people involved, and what you learned. Preparing them once, thoroughly, and being ready to tell the same real story from either angle is far more efficient, and far more convincing, than maintaining two separate, shallower sets of stories for "technical" and "behavioral" rounds.

Project

Goal: build your own story bank to this chapter's standard.

Tools you'll need

No software specific to this chapter. A document listing your five to eight story-bank stories, each with a one-line theme tag and a bullet-point STAR outline (not a full script), reviewed and refreshed as new real experiences accumulate.

1. List five to eight real stories from your own experience (work, study, or your portfolio projects, labelled as such), tagging each with the behavioral theme(s) it could answer. Check that all seven themes in Q81-006 are covered.
2. Write a full STAR outline (bullet points, not a script) for your three strongest stories, each with a number in its Result.
3. Practice your "tell me about yourself" answer out loud, timed, and cut it if it runs past two minutes.
4. Write three specific questions you'd ask an interviewer for a real role you're targeting, each one unanswerable from the company's public website alone.
5. Write your expected-CTC range with its two sources, and draft the exact words you'd use to ask for time and for the breakup before responding to a real or hypothetical offer.

REVIEW

Key terms

STAR method (Situation, Task, Action, Result) · story bank · behavioral theme · self-aware weakness · gap explanation · offer negotiation · expected CTC · CTC breakup · fixed vs variable pay · in-hand pay (Chapter 8) · notice period · counter-offer (from your current employer)

Final-week revision list

Q81-001, Q81-006, Q81-011, Q81-012, Q81-013, Q81-019, Q81-024, Q81-025, Q81-034.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 76A, Data Analyst & Data Scientist Question Bank**, already covers project-narrative structure in full; this chapter formalizes the same discipline as STAR for behavioral-specific questions.
- **Chapter 68, How Data Hiring Works**, covers everything before this chapter's offer conversation; this chapter is the process's final gate.
- **Chapter 82, Take-Home Assignments & Mock Interviews**, is where this chapter's stories and Chapter 69's technical moves get rehearsed together, end to end, under realistic time pressure.

Chapter 82. Take-Home Assignments & Mock Interviews

Chapter at a glance

You will learn to: judge whether a take-home assignment is fair before you start it · complete a take-home the way a strong candidate actually would, not just technically correctly, but with the judgment and communication a reviewer is scoring · sit through a realistic mock interview and hear what a strong answer and a weaker one sound like at the same turn · read interviewer scoring notes and understand what actually moved the score.

Before you start: Chapter 69 (the five-dimension rubric and the twelve extra-point tags). The take-homes use SQL from Chapters 12 and 13, the lead model from Chapters 36, 37 and 39, and the pipeline ideas from Chapters 45–47. The mocks draw on the banks in Chapters 71, 74, 76A, 76B, 77 and 81. This chapter rehearses those skills; it doesn't teach them again, and each section says where each one is taught.

Time needed: 2–3 hours to read the chapter and run every query and cell; about 10–12 hours if you also do all three take-homes against the clock (2 + 3 + 2 hours), compare them with the scoring tables, and record one mock.

How this chapter is built. Three complete take-homes and three scored mock interviews, one each for Data Analyst, Data Scientist and Data Engineer. Business analysts can adapt the Data Analyst take-home using Chapter 76B's elicitation questions; automation and BI candidates can adapt the Data Engineer one. Each take-home gives the assignment as a candidate would receive it, a model submission, a scoring table and what a weak submission looks like. **Every model submission was actually run:** the SQL on PostgreSQL 16 against the Riverstone databases, the Python on Python 3.11.15 with scikit-learn 1.9.1 (the book recommends Python 3.14, Chapter 17, section 17.0; nothing here depends on the version), and every output shown is the real one. The mocks are condensed transcripts: each long answer is written out once, in full, and each mock shows a weaker answer next to the stronger one at two of its turns, with Chapter 69's scores and tags.

In this chapter

LEARN	82.0 Before you start: is this a fair take-home?	371
	82.1 Take-home assignment: Data Analyst	371
	82.2 Take-home assignment: Data Scientist	376
	82.3 Take-home assignment: Data Engineer	381
	82.4 Mock interview: Data Analyst (entry level)	386
	82.5 Mock interview: Data Scientist (mid level)	388
APPLY	82.6 Mock interview: Data Engineer (mid level)	390
	Common mistakes · Project	392
REVIEW	Key terms · Final-week revision list	393
NEXT	Where this leads	393

82.0 Before you start: is this a fair take-home?

A take-home is unpaid work, so judge it before you begin. Chapter 8 promised this chapter would show what a fair one looks like.

A fair take-home has:

- a stated time limit, usually no more than about 4 hours;
- synthetic or public data, or a sample prepared for the exercise;
- a clear deliverable ("a one-page summary plus the queries"), and a named person to send questions to.

Warning signs:

- an open-ended brief such as "build our churn feature" or "redesign our pipeline";
- the company's real client data, or a problem that looks like their live backlog;
- no time limit, or an estimate of several days of unpaid work.

What to do:

- Ask for scope clarification **in writing**, early: "Should I assume X, or Y?" A reviewer reads that as good judgment, not weakness.
- Say how long you spent, at the top of your submission.
- It's fine to deliver the time-boxed version with a "what I'd do next" list, exactly as the Data Engineer submission in section 82.3 does. Over-delivering on a fair take-home rarely changes the result; over-delivering on an unfair one teaches the company that you'll work for free.

All three assignments below pass this test: each has a time limit, practice data from this book, and a one-page deliverable.

82.1 Take-home assignment: Data Analyst

The assignment, as given

"Riverstone's sales leadership wants to understand which product categories are driving growth and which need attention. Using the attached order data, prepare a short analysis (a one-page summary plus supporting queries/code) answering: which categories are performing best and worst, what's driving the difference, and one specific recommendation. You have 2 hours."

Practice data: `riverstone_2025` (Chapter 13, section 13.1): all of 2025, 24 customers, 175 orders, 173 of them not cancelled.

Model submission

Step 1: the core numbers. Revenue, orders and customers by category, leaving out cancelled orders:

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 2) AS revenue,
       COUNT(DISTINCT o.order_id) AS orders,
       COUNT(DISTINCT o.customer_id) AS customers
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category
ORDER BY revenue DESC;
```

category	revenue	orders	customers
Storage	2297973.50	142	23
Kitchen	1208030.00	85	16
Industrial	795830.00	23	5
Furniture	33637.50	2	2

(4 rows)

- The revenue formula is Chapter 12's (section 12.9): quantity × price charged × (1 – discount). `ROUND(..., 2)` keeps two decimals; without it PostgreSQL prints the raw result of the division, with twenty decimal places.
- `COUNT(DISTINCT ...)` counts each order and each customer once, however many lines they have (the fan-out trap, section 12.10).
- **Check it reconciles:** the four revenues add up to ₹43,35,471, Riverstone's 2025 revenue (Chapter 10). The order counts add up to 252, more than the 173 orders, because most orders contain more than one category. So "orders" here means *orders that include this category*.
- Industrial's 23 orders come from only **5 customers**.

Step 2: one level deeper, checking whether Industrial's smaller order count is offset by more spend per order. A first attempt averaged per order *line*, not per order: silently the wrong grain. The fix is to add up each order's spend in the category first, then average those order totals:

```
WITH order_category_revenue AS (
  SELECT o.order_id,
         p.category,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS order_revenue
  FROM orders AS o
  JOIN order_items AS oi ON o.order_id = oi.order_id
  JOIN products AS p ON oi.product_id = p.product_id
  WHERE o.status <> 'Cancelled'
  GROUP BY o.order_id, p.category
)
SELECT category,
       ROUND(AVG(order_revenue), 2) AS avg_category_spend_per_order,
       COUNT(*) AS n_orders
FROM order_category_revenue
GROUP BY category
ORDER BY avg_category_spend_per_order DESC;
```

category	avg_category_spend_per_order	n_orders
Industrial	34601.30	23
Furniture	16818.75	2
Storage	16182.91	142
Kitchen	14212.12	85

(4 rows)

- The CTE (Chapter 13, section 13.2) makes one row per order and category; the outer query averages those rows. Averaging order-level totals, not lines, is the grain rule from Chapter 12, section 12.12.
- The column name says what it is: the spend on *this category's items* in an order that includes it, not the size of the whole basket.
- **Check it agrees with Step 1:** ₹22,97,973.50 ÷ 142 = ₹16,182.91 ✓, and ₹7,95,830 ÷ 23 = ₹34,601.30 ✓.

Step 3: growth, which is what the question actually asked. Revenue by quarter, and the second half of the year against the first. This is the quarterly pivot from Chapter 12's exercise 32, with one column added:

```
WITH sales_lines AS (
  SELECT p.category,
         EXTRACT(QUARTER FROM o.order_date) AS qtr,
         oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100) AS line_revenue
  FROM orders AS o
  JOIN order_items AS oi ON o.order_id = oi.order_id
  JOIN products AS p ON oi.product_id = p.product_id
  WHERE o.status <> 'Cancelled'
)
SELECT category,
       ROUND(SUM(CASE WHEN qtr = 1 THEN line_revenue ELSE 0 END), 0) AS q1,
       ROUND(SUM(CASE WHEN qtr = 2 THEN line_revenue ELSE 0 END), 0) AS q2,
       ROUND(SUM(CASE WHEN qtr = 3 THEN line_revenue ELSE 0 END), 0) AS q3,
       ROUND(SUM(CASE WHEN qtr = 4 THEN line_revenue ELSE 0 END), 0) AS q4,
       ROUND(100.0 * SUM(CASE WHEN qtr >= 3 THEN line_revenue ELSE 0 END)
             / SUM(CASE WHEN qtr <= 2 THEN line_revenue ELSE 0 END) - 100, 0)
       AS h2_vs_h1_pct
FROM sales_lines
GROUP BY category
ORDER BY q4 DESC;
```

category	q1	q2	q3	q4	h2_vs_h1_pct
Storage	503257	342422	451179	1001116	72
Kitchen	133888	219336	393660	461146	142
Industrial	80780	147560	275450	292040	149
Furniture	16388	17250	0	0	-100

(4 rows)

- `EXTRACT(QUARTER FROM o.order_date)` gives 1 to 4. Each `SUM(CASE WHEN qtr = 1 ...)` adds only that quarter's lines: one pivot column per quarter.
- `h2_vs_h1_pct` divides July–December revenue by January–June revenue: $100 \times H2 \div H1 - 100$ is the percentage change. With one year of data, half-on-half is the fairest growth measure available; there's no 2024 in this database to compare with.

- Kitchen and Industrial both grew steadily, quarter after quarter. Storage's growth is mostly one quarter: Q4 is more than twice Q3. Furniture sold nothing after June.

Step 4: revenue isn't profit. Chapter 12's revenue-versus-profit query (section 12.10), with profit per order and the average discount added:

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)
                - oi.quantity * p.unit_cost), 0) AS gross_profit,
       ROUND(100.0 * SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)
                - oi.quantity * p.unit_cost)
             / SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 1) AS margin_pct,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)
                - oi.quantity * p.unit_cost)
             / COUNT(DISTINCT o.order_id), 0) AS profit_per_order,
       ROUND(AVG(oi.discount_pct), 1) AS avg_discount_pct
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category
ORDER BY gross_profit DESC;
```

category	gross_profit	margin_pct	profit_per_order	avg_discount_pct
Storage	619174	26.9	4360	4.8
Kitchen	401580	33.2	4724	1.7
Industrial	108330	13.6	4710	9.2
Furniture	8138	24.2	4069	2.5

(4 rows)

- `gross_profit` is revenue minus quantity × unit_cost, and `margin_pct` divides total profit by total revenue, both exactly as in section 12.10.
- `profit_per_order` divides the category's gross profit by the orders that include it (Step 1's `orders`).
- `avg_discount_pct` is the average discount across the category's order lines.

Reading it. Industrial orders are more than twice as large, but at a 13.6% margin they earn about the same gross profit per order as Kitchen (₹4,710 against ₹4,724). Industrial is 18% of revenue but under 10% of gross profit (₹1,08,330 of ₹11,37,221), and it carries the deepest discounts.

The one-page summary:

2025 category performance

Growth: Kitchen and Industrial are driving growth. Both more than doubled from the first half of 2025 to the second (Kitchen +142%, Industrial +149%), rising every quarter. Storage is still the largest category (₹22.98 lakh of ₹43.35 lakh) and grew 72%, but most of that came in Q4 alone, so I'd check whether Q4 was a one-off before calling it a trend. Furniture sold nothing after June.

What's driving the difference: Industrial orders are large (₹34,601 of Industrial items per order that includes them, against ₹14,212–₹16,183 for Kitchen and Storage), but they carry the deepest discounts (9.2% on average) and a 13.6% margin. So an Industrial order earns about the same gross profit as a Kitchen order (₹4,710 against ₹4,724). Kitchen grows almost as fast at a 33.2% margin, which makes it the most valuable growth in the business.

Recommendation: Before funding any push to win more Industrial customers, review the discounts on crate orders (Chapter 12's revenue-versus-profit query already flagged them). Industrial volume is only worth growing if those discounts are controlled. In the meantime, put new sales effort behind Kitchen, where growth already comes with a healthy margin, and track both against this same breakdown next quarter.

Caveats: one year and 24 customers. Industrial's revenue comes from 5 customers, so a single lost account would change its picture. Furniture's 2 orders are too few to judge it either way; I'm flagging it rather than ranking it or dismissing it.

How this would be scored

Dimension	What a strong submission does
Correctness	Numbers reconcile (Step 1 adds up to the year's revenue; Step 2 agrees with Step 1); SQL is real and runs without error
Structure & communication	One-page summary is scannable and leads with the finding, not the method
Depth & edge cases	Goes past the headline ranking: fixes the grain, measures growth over time, and checks profit, not just revenue
Business judgment	Ends in one specific, proportionate recommendation, not five vague ones, and ties it to margin
Collaboration	States the caveats (one year, five Industrial customers, two Furniture orders) explicitly and doesn't overclaim

What a weak submission looks like: the revenue table alone, with a generic "Storage is doing well, Furniture needs improvement" observation, no growth measure, no recommendation and no caveat. Technically not wrong, just thin: exactly Chapter 69's core lesson applied to a whole assignment instead of one question. A *plausible-but-wrong* submission is worse: "Industrial orders are worth double, so win more Industrial customers", which reads revenue as profit.

Learn it in: Chapter 12, sections 12.9–12.10 (the revenue formula, joins, revenue versus profit) and 12.12 (the grain of an average); Chapter 13, sections 13.2 (CTEs) and 13.6 (time windows and growth).

Practise it with: Chapter 71, Q71–072 (revenue by order, with a fan-out warning).

82.2 Take-home assignment: Data Scientist

The assignment, as given

"We want to predict which leads are likely to convert, using the attached lead data. Build a model, evaluate it properly, and explain in one page how you'd use it in production, including any limitation you'd flag before it ships. You have 3 hours."

Practice data: Riverstone's CRM export from Chapter 36, rebuilt by `lead_data.py`, a copy of Chapter 37's helper, in `companion/ch82/`. Open a Jupyter notebook in that folder and run each block below as its own cell, top to bottom. If `companion/crm/` is empty, first build the data with `python generate_riverstone_crm.py` in the companion folder (Chapter 36).

Model submission

Step 1: load the data and look at the split.

```
import sys
import warnings

import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import confusion_matrix, log_loss, roc_auc_score

sys.path.append(".")
from lead_data import LEAD_CATS, LEAD_NUMS, load_leads, make_model

warnings.filterwarnings("ignore", category=UserWarning)

train, valid, test = load_leads()
X_cols = LEAD_CATS + LEAD_NUMS
for name, part in [("train", train), ("valid", valid), ("test", test)]:
    print(
        f"{name:<5} {len(part):>5,} leads, {part['won'].mean():.1%} won, "
        f"{part['created_at'].min():%b %Y} to {part['created_at'].max():%b %Y}"
    )
```

```
train 7,291 leads, 7.9% won, Jan 2023 to Dec 2024
valid 2,225 leads, 6.6% won, Jan 2025 to Jun 2025
test 1,185 leads, 8.8% won, Jul 2025 to Oct 2025
```

- `load_leads()` removes duplicate enquiries, builds Chapter 36's features and splits **by date** (Chapter 36, section 36.3): the model learns from 2023–2024, is chosen on the first half of 2025, and is tested once on later leads. `LEAD_CATS + LEAD_NUMS` are its thirteen feature columns.
- `sys.path.append(".")` and `warnings.filterwarnings(...)` are the same two lines as in Chapter 39, section 39.1.

- `{part['created_at'].min():%b %Y}` formats a date as a short month and year.
- **The base rate matters:** only 6.6% of validation leads were won. Every later number is read against it.

Step 2: a baseline and a challenger, compared on validation. The baseline is Chapter 36's logistic regression; the challenger is gradient boosting with the settings Chapter 37's random search chose (section 37.12):

```
boosting = HistGradientBoostingClassifier(
    learning_rate=0.014,
    max_depth=3,
    max_iter=454,
    min_samples_leaf=67,
    l2_regularization=0.015,
    random_state=37,
)
candidates = {
    "logistic regression": make_model(LEAD_CATS, LEAD_NUMS),
    "gradient boosting": make_model(LEAD_CATS, LEAD_NUMS, boosting),
}
y_valid = valid["won"].to_numpy()
for label, model in candidates.items():
    model.fit(train[X_cols], train["won"])
    p = model.predict_proba(valid[X_cols])[:, 1]
    print(
        f"{label:<20} ROC-AUC {roc_auc_score(y_valid, p):.3f}    "
        f"log loss {log_loss(y_valid, p):.4f}"
    )
```

```
logistic regression  ROC-AUC 0.823    log loss 0.1948
gradient boosting    ROC-AUC 0.826    log loss 0.1964
```

- `make_model(LEAD_CATS, LEAD_NUMS)` is Chapter 36's pipeline with logistic regression; passing `boosting` as the third argument swaps the model and keeps the same preparation.
- The five settings are the ones Chapter 37's search picked (sections 37.8 and 37.11), rounded to three decimals: `max_iter=454` trees, each at most `max_depth=3` levels deep, each correcting only a small step of the error so far (`learning_rate=0.014`), no leaf with fewer than `min_samples_leaf=67` leads, and a gentle penalty on large leaf values (`l2_regularization=0.015`). `random_state=37` fixes boosting's random choices, so the run repeats exactly.
- `candidates` is a dictionary from name to model; `.items()` gives each pair in turn. Each model learns from `train` and is scored on `valid`.
- **ROC-AUC** measures ranking and **log loss** the honesty of the probabilities (Chapter 39, sections 39.2 and 39.4). Boosting ranks a hair better (0.826 against 0.823) and its probabilities are a hair worse. That's a tie, and a tie goes to the simpler, explainable model.

Step 3: choose the threshold by cost, not 0.5. Working a lead costs about ₹1,500, and a won lead is worth about ₹30,255 of margin; both are Chapter 39's stated assumptions (section 39.5):

```
p_valid = candidates["logistic regression"].predict_proba(valid[X_cols])[0, 1]
WORK_COST = 1500 # ₹ to work one lead properly (Chapter 39, section 39.5)
WIN_VALUE = 30255 # ₹ margin from one won lead (Chapter 39, section 39.5)

def profit_at(threshold):
    worked = p_valid >= threshold
    wins = y_valid[worked].sum()
    return WIN_VALUE * wins - WORK_COST * worked.sum(), worked.sum(), wins

grid = np.linspace(0.0, 0.5, 501)
profits = [profit_at(t)[0] for t in grid]
best = grid[np.argmax(profits)]
for label, t in [("work every lead", 0.0), ("best threshold", best)]:
    profit, worked, wins = profit_at(t)
    print(f"{label:<16} {t:.3f}: {worked:>5,} worked, {wins} wins, profit ₹{profit:,.0f}")
```

```
work every lead  0.000: 2,225 worked, 146 wins, profit ₹1,079,730
best threshold   0.077:   627 worked, 112 wins, profit ₹2,448,060
```

- `p_valid` is the logistic model's probability of a win for each validation lead.
- `profit_at` is Chapter 39's function: every worked lead costs ₹1,500, every win among them earns ₹30,255. It returns the expected profit, the leads worked and the wins.
- `np.linspace(0.0, 0.5, 501)` tries every threshold from 0 to 0.5 in steps of 0.001, and `np.argmax` finds the one with the most profit.
- Working the 627 leads above 0.077 makes about ₹24.5 lakh of expected profit, against ₹10.8 lakh for working all 2,225: about 2.3 times the profit from 28% of the effort.

Step 4: what the chosen threshold does, next to the default.

```
for t in [0.5, best]:
    tn, fp, fn, tp = confusion_matrix(y_valid, (p_valid >= t).astype(int)).ravel()
    print(
        f"threshold {t:.3f}: TP {tp:>3}  FP {fp:>3}  FN {fn:>3}  TN {tn:>4}  "
        f"precision {tp / (tp + fp):.3f}  recall {tp / (tp + fn):.3f}"
    )
```

```
threshold 0.500: TP   4  FP   3  FN 142  TN 2076  precision 0.571  recall 0.027
threshold 0.077: TP 112  FP 515  FN  34  TN 1564  precision 0.179  recall 0.767
```

- `(p_valid >= t).astype(int)` turns probabilities into 0/1 predictions; `confusion_matrix(...).ravel()` returns the four counts in scikit-learn's order, `tn`, `fp`, `fn`, `tp` (Chapter 39, section 39.1).
- At the default 0.5 the model flags 7 leads and finds 4 of the 146 wins. At 0.077 it finds 112 of them (recall 0.767), and about 1 worked lead in 6 is won (precision 0.179), which is expected when only 6.6% of leads convert.

Step 5: the team's capacity. Chapter 39 found the team can work about 4 leads a day, 504 in the 126 working days of the period. That's fewer than 627:

```
budget = 126 * 4 # working days in Jan-Jun 2025 × leads the team can work a day
top = np.sort(p_valid)[:,-1][:budget]
profit, worked, wins = profit_at(top.min())
print(f"capacity {budget}: work leads scoring above {top.min():.3f}")
print(f"{worked} worked, {wins} wins, profit ₹{profit:,.0f}")
```

```
capacity 504: work leads scoring above 0.097
504 worked, 102 wins, profit ₹2,330,010
```

- `np.sort(p_valid)[:,-1]` sorts the scores from highest to lowest (`[:,-1]` reverses the order), and `[:budget]` keeps the top 504.
- The lowest of those, `top.min()`, is the threshold that fills the team's capacity exactly.
- Working the top 504 still makes about ₹23.3 lakh. Chapter 39 compared this with the old rule of thumb (work referrals, trade fairs and partners: ₹17.4 lakh from a similar 531 leads).

Step 6: the test set, once. Refit both candidates on every lead before July 2025 and score the later leads:

```
before_july = pd.concat([train, valid])
y_test = test["won"].to_numpy()
for label, model in candidates.items():
    model.fit(before_july[X_cols], before_july["won"])
    p = model.predict_proba(test[X_cols])[:, 1]
    print(
        f"{label:<20} test ROC-AUC {roc_auc_score(y_test, p):.3f}    "
        f"log loss {log_loss(y_test, p):.4f}"
    )
```

```
logistic regression  test ROC-AUC 0.838    log loss 0.2369
gradient boosting    test ROC-AUC 0.830    log loss 0.2422
```

- `pd.concat([train, valid])` stacks the two tables, so the final models learn from every lead before July 2025.
- The loop is Step 2's, reusing the same two `candidates`: `.fit` retrains each on the larger table, and `roc_auc_score` and `log_loss` score it on the test leads, which neither model has seen.
- On the test leads, logistic regression is ahead on both measures. The decision from Step 2 stands.

The one-page summary:

Model: logistic regression on Chapter 36's thirteen features, split by date. I compared it with tuned gradient boosting: on validation they tied (ROC-AUC 0.823 against 0.826), and on the held-out test leads logistic regression was ahead (0.838 against 0.830) with better-calibrated probabilities. I'm shipping the simpler model: it's as accurate, faster, and easy to explain to the sales team.

Evaluation: at the default 0.5 threshold the model is nearly useless: it flags 7 of 2,225 leads. Working the 627 leads that score above 0.077 gives an estimated ₹24.5 lakh of expected profit on the January–June leads, against ₹10.8 lakh for working all 2,225: about 2.3 times the profit from 28% of the effort. The team can work only about 504 leads in that period, so in practice I'd work the top 504 by score (a threshold of about 0.097): ₹23.3 lakh.

Production use: score new leads nightly and hand the sales team a ranked call list sized to their capacity, not a raw score. I'd rank by expected value (probability × deal value, Chapter 44's lesson) once we have a per-lead deal estimate, such as the quoted value or company size. Today every lead uses the same ₹30,255, so ranking by probability is equivalent, and that's the first data gap I'd close.

Limitations I'd flag before shipping: the model learns from 2023–2024 patterns; if lead sources or the market shift (Chapter 36 showed the marketplace share rising), retrain it, and watch how many leads each day's threshold flags. The ₹1,500 cost and the ₹30,255 win value are assumptions to agree with finance, not facts; the profit figures are expected values from the model, not money earned, and should be confirmed with a small trial before anyone quotes them.

How this would be scored

Dimension	What a strong submission does
Correctness	A real baseline-versus-challenger comparison on the same split, chosen on validation and tested once; every number comes from one model and one dataset
Structure & communication	Model choice → evaluation → production plan → limitations, in that order
Depth & edge cases	Goes past AUC into a cost-based threshold and the team's capacity, showing the metric-to-decision translation
Business judgment	Chooses the simpler model when the complex one doesn't win, and sizes the list to what the team can actually work
Collaboration	States real limitations unprompted and doesn't oversell the model as a finished, permanent solution

What a weak submission looks like: reports only AUC, picks boosting because it's "more advanced", recommends a 0.5 threshold with no cost reasoning, and has no limitations section at all: technically functional, missing every dimension beyond raw correctness.

Learn it in: Chapter 36, sections 36.3 (a split by date) and 36.10 (baselines); Chapter 37, section 37.12 (baseline versus boosting on the leads); Chapter 39, sections 39.1 (the confusion matrix), 39.2 (ROC-AUC) and 39.5 (the cost threshold and capacity); Chapter 44, section 44.4 (ranking by value, not probability). **Practise it with:** Chapter 74, Q74-008 (a good AUC can still be a bad choice), Q74-015 (ROC-AUC versus PR-AUC) and Q74-016 (the 0.5 threshold).

82.3 Take-home assignment: Data Engineer

The assignment, as given

"Design and implement a small pipeline that loads daily order data into a summary table, safely re-runnable if it fails partway through, with at least one automated data quality check. You have 2 hours."

Practice data: the Riverstone mini database (12 orders, January to March 2026), loaded into `riverstone_lab`, the practice database from Chapter 12, section 12.13, so that nothing you do here touches the `riverstone` database itself. If you dropped `riverstone_lab` at the end of that section, create it again with `CREATE DATABASE riverstone_lab;`. Then, connected to `riverstone_lab`, run `riverstone_setup.sql` from the companion files (Appendix E) with *Execute SQL Script*, as in section 12.3. It creates the mini database's seven tables in the lab.

Model submission

Step 1: the summary table, created once. The table's own rules are the first quality check:

```
CREATE TABLE IF NOT EXISTS daily_summary (  
    order_date    DATE PRIMARY KEY,  
    total_revenue NUMERIC(12,2) NOT NULL CHECK (total_revenue >= 0),  
    order_count   INTEGER      NOT NULL CHECK (order_count > 0)  
);
```

- **IF NOT EXISTS** makes the setup safe to run twice. A plain `CREATE TABLE` stops the second time with `ERROR: relation "daily_summary" already exists`; with `IF NOT EXISTS`, PostgreSQL prints a notice and carries on.
- **PRIMARY KEY** on `order_date` allows one row per day, so a duplicate day can never be stored.
- **NOT NULL** and the two **CHECK** rules (Chapter 12, section 12.13) reject a missing or negative revenue, and a day with no orders. A load that breaks them fails with an error instead of storing a wrong number.

Run the same statement a second time:

```
NOTICE:  relation "daily_summary" already exists, skipping
```

Step 2: the load, safe to repeat. Replace a whole window of days inside one transaction:

```
BEGIN;

DELETE FROM daily_summary
WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31';

INSERT INTO daily_summary (order_date, total_revenue, order_count)
SELECT o.order_date,
       SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)),
       COUNT(DISTINCT o.order_id)
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
   AND o.order_date BETWEEN '2026-01-01' AND '2026-03-31'
GROUP BY o.order_date;

COMMIT;
```

- **BEGIN ... COMMIT** is a transaction (Chapter 12, section 12.13): the **DELETE** and the **INSERT** are kept together or undone together. If anything fails in between, the table keeps its previous contents.
- **The DELETE** clears every day in the window; **the INSERT** rebuilds them from the source. Chapter 46, section 46.4 proved this delete-and-insert pattern with a failure test.
- The window here is the whole of the mini database, the first quarter of 2026. A daily job would reload the last few days, so that late changes (a cancellation, a corrected line) are picked up.

Check what arrived:

```
SELECT COUNT(*) AS days, SUM(order_count) AS orders, SUM(total_revenue) AS revenue
FROM daily_summary;
```

```
days | orders | revenue
-----+-----+-----
  11 |    11 | 323930.00
(1 row)
```

Eleven days, one order each: the twelve orders less the cancelled one. ₹3,23,930 is the same total as Chapter 12's revenue-versus-profit query (section 12.10). ✓

Now run the whole load block (**BEGIN** to **COMMIT**) a second time, and count again:

```
SELECT COUNT(*) AS days, SUM(order_count) AS orders, SUM(total_revenue) AS revenue
FROM daily_summary;
```

```
days | orders | revenue
-----+-----+-----
  11 |    11 | 323930.00
(1 row)
```

The same eleven days and the same total: a second run changes nothing.

Step 3: two failure tests. A re-run of the same data is the easy case. The two that catch real pipelines out are a correction to data already loaded, and a bad row.

Test 1, a correction. The customer cancels order 5012, the only order on 15 March:

```
UPDATE orders SET status = 'Cancelled' WHERE order_id = 5012;
```

Run the load block again, then count:

```
SELECT COUNT(*) AS days, SUM(order_count) AS orders, SUM(total_revenue) AS revenue
FROM daily_summary;
```

```
days | orders | revenue
-----+-----+-----
  10 |      10 | 297710.00
(1 row)
```

15 March has gone, and the total fell by that order's ₹26,220. This is why the load deletes the window first. An upsert (`INSERT ... ON CONFLICT (order_date) DO UPDATE`, Chapter 12, section 12.13) would also survive a re-run without duplicates, but it only touches the days its `SELECT` returns. A day that loses all its orders returns no row, so the upsert would leave 15 March at its old ₹26,220 forever.

Test 2, a bad row. A new order arrives with a typing slip: a discount of 150% instead of 15%:

```
INSERT INTO orders VALUES (5013, 1, '2026-03-20', 'Pending', 3);
INSERT INTO order_items VALUES (21, 5013, 101, 10, 450.00, 150);
```

Run the load block again:

```
ERROR: new row for relation "daily_summary" violates check constraint
"daily_summary_total_revenue_check"
DETAIL: Failing row contains (2026-03-20, -2250.00, 1).
```

The line's revenue is $10 \times ₹450 \times (1 - 1.5) = -₹2,250$, and the `CHECK` rule refuses it. The load **fails loudly**, and because the `DELETE` ran inside the same transaction, it is undone too:

```
SELECT COUNT(*) AS days, SUM(order_count) AS orders, SUM(total_revenue) AS revenue
FROM daily_summary;
```

```
days | orders | revenue
-----+-----+-----
  10 |      10 | 297710.00
(1 row)
```

The previous good table is still there, untouched. Fix the source, and the same load succeeds:

```
UPDATE order_items SET discount_pct = 15 WHERE order_item_id = 21;
```

Run the load block once more, then count:

```
SELECT COUNT(*) AS days, SUM(order_count) AS orders, SUM(total_revenue) AS revenue
FROM daily_summary;
```

days	orders	revenue
11	11	301535.00

(1 row)

Eleven days again: 20 March is in, with $10 \times ₹450 \times 0.85 = ₹3,825$.

Step 4: a data test after every load. A data test is a query that returns the rows that *break* a rule; zero rows means it passed (Chapter 47, section 47.3). This one compares every day in the summary with the source, in both directions:

```
SELECT s.order_date, s.total_revenue, src.order_date AS source_date, src.revenue AS source_revenue
FROM daily_summary AS s
FULL OUTER JOIN (
    SELECT o.order_date,
           ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 2) AS revenue
    FROM orders AS o
    JOIN order_items AS oi ON o.order_id = oi.order_id
    WHERE o.status <> 'Cancelled'
    GROUP BY o.order_date
) AS src ON s.order_date = src.order_date
WHERE s.total_revenue IS DISTINCT FROM src.revenue;
```

order_date	total_revenue	source_date	source_revenue
------------	---------------	-------------	----------------

(0 rows)

- The subquery in brackets is the load's revenue by day, straight from the source, rounded to two decimals like the table's `NUMERIC(12,2)` column.
- **FULL OUTER JOIN** keeps days from both sides (Chapter 12, section 12.10: the reconciliation join), so a day missing from the summary or a stale day left in it both show up.
- **IS DISTINCT FROM** (Chapter 28, section 28.10) is "not equal" that also treats a NULL on one side as a difference; a plain `<>` would silently skip those rows.
- Zero rows: every day matches its source. The pipeline's scheduler runs this after each load and **fails the run if any row comes back**, the way Chapter 47's `run_tests` does.

Clean up when you're done:

```
DROP TABLE daily_summary;
```

`DROP TABLE` removes the practice table (Chapter 12, section 12.13). The mini database's tables can stay in the lab for the next time you practice.

Step 5: the one-page design note:

Idempotency: the load deletes and re-inserts a window of days inside one transaction, so running it twice gives the same table, and a correction to data already loaded (a cancelled order, a fixed line) is picked up on the next run. I chose this over an upsert (`ON CONFLICT ... DO UPDATE`) because an upsert never removes a day that has lost all its orders. One statement, or one transaction, is atomic: a failure partway through leaves the table unchanged, so there's nothing to clean up. What makes a *second* run safe is the delete-and-insert.

Data quality: two layers, and both can fail. The table's own rules (`NOT NULL`, `CHECK`) stop a missing or negative revenue at write time, which also covers a `NULL` discount, since a `NULL` anywhere in the revenue formula makes the line's revenue `NULL` (Chapter 12, section 12.7). After each load, a data test reconciles every day with the source and fails the run on any mismatch. In a real deployment I'd add a volume check (today's order count far below the trailing average, Chapter 47, section 47.5), because a truncated source file passes both of these: fewer orders, all of them valid.

Scale: each run recomputes the whole window, which is fine at Riverstone's size. At scale I'd reload only the last few days, sized to how late changes arrive.

What I'd add given more time: a raw/staging layer (Chapter 45's raw → staging layers), so extracted data lands unmodified before transformation, letting a transformation bug be fixed and reprocessed without re-extracting from the source; and an explicit dependency declaration if this joins with other scheduled loads, rather than assuming execution order.

How this would be scored

Dimension	What a strong submission does
Correctness	The load genuinely re-runs safely: no duplicates, no stale days, and a bad row can't reach the table
Structure & communication	Table, load, tests and design note clearly separated, not tangled together
Depth & edge cases	Tests the hard cases (a correction and a bad row), not just a second run, and names a <i>second</i> check type (volume) unprompted
Business judgment	Explains <i>why</i> a pipeline should fail loudly rather than silently store a wrong number
Collaboration	States what's missing given more time, rather than presenting the 2-hour version as fully production-ready

What a weak submission looks like: a plain `INSERT` with no conflict handling, which fails the core "safely re-runnable" requirement entirely; or a quality check that only counts bad rows and logs a warning, so nothing ever fails.

Learn it in: Chapter 12, section 12.13 (constraints, transactions, the upsert); Chapter 45, section 45.1 (raw and staging layers); Chapter 46, section 46.4 (idempotency, proven with a failure test); Chapter 47, sections 47.3 (data tests) and 47.5 (volume checks). **Practise it with:** Chapter 77, Q77-007 (the re-run that duplicates rows) and Q77-028 (a check that would have caught negative revenue).

82.4 Mock interview: Data Analyst (entry level)

A 20-minute mock for an entry-level Data Analyst role, condensed. Interviewer notes appear in brackets, with Chapter 69's tags. At two turns, a weaker answer to the same question follows, with its score. The scoring table at the end uses Chapter 69's scale, 1 (weak) to 4 (outstanding).

Interviewer: Let's start with something practical. Write a query to find customers who haven't placed an order in the last 90 days.

Candidate: Sure. Two quick checks first. When you say "last 90 days," is that from today, or from the most recent order date in the data? If this is historical data, "today" might not mean what I'd assume. And should a cancelled order count as activity, and do you want customers who've never ordered at all?

[Interviewer note: **[+Clarify]** caught a real ambiguity most candidates miss, plus two scope questions that change the answer.]

Interviewer: From the most recent date in the data. Cancelled orders don't count, and yes, include customers who've never ordered.

Candidate: Got it.

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders AS o
  WHERE o.customer_id = c.customer_id
        AND o.status <> 'Cancelled'
        AND o.order_date > (SELECT MAX(order_date) FROM orders) - INTERVAL '90 days'
);
```

The inner `SELECT MAX(order_date)` is the latest date in the data, and `- INTERVAL '90 days'` moves 90 days back from it. I used `NOT EXISTS` on purpose. The obvious version is `NOT IN` with a subquery, but if that subquery ever returns a NULL customer ID, the whole query silently returns zero rows, because "x is not in a list containing NULL" can never be confirmed true. `NOT EXISTS` asks "is there no recent order for this customer?", and a NULL can't spoil it. It also keeps customers with no orders at all, which you said you want.

[Interviewer note: **[+Edge cases]** chose the safe form and explained why the alternative fails, which is Chapter 69's definition of a 4 on Correctness.]

A weaker answer to the same turn: writes `WHERE c.customer_id NOT IN (SELECT customer_id FROM orders WHERE ...)` straight away, with no NULL filter and no question about cancelled orders. It happens to work on this table, because `orders.customer_id` is never NULL, but asked "could this ever return nothing?", the candidate can't say why. [Correctness 2: correct for the simple case only.]

Interviewer: Let's say this returns 40 customers. Sales wants to know which to call first. How would you prioritize?

Candidate: I'd rank by revenue at risk, not just recency: the probability they've stopped buying times their value, if there's a model, or at minimum their revenue over the last year as a proxy if there isn't. Ranking by "most overdue" alone would put a small customer who's very overdue ahead of a large account that's only slightly overdue.

[Interviewer note: **[+Business]** connects a simple query to a real prioritization decision without being asked.]

A weaker answer to the same turn: "I'd call the most overdue first." Reasonable, but it ties the list to nothing the business values. [Business judgment 2: mentions the business, doesn't tie the answer to a decision.]

Interviewer: Last one. Walk me through a project where an analysis you did led to an actual decision.

Candidate (about 90 seconds, which suits a round with this many follow-ups; Chapter 81, Q81-002):

Situation: "In my portfolio project, on a practice dataset from a supplies distributor, the key accounts manager asked whether deeper discounts would grow Retail. Customers on discounts of 5% or more placed 14.8 orders in 2025, against 9.3 for everyone else."

Task: "I had to say whether that gap was a reason to discount harder."

Action: "I built a customer-year table from the order lines and reconciled it to the year's total before trusting it. Then I split the discount bands by segment, and the headline fell apart: 662 of the 681 discounted customers were Wholesale. Inside Wholesale, where customers are comparable, the shallowest discounts placed 14.1 orders and the deepest 13.6, no real difference. And Wholesale runs at an 18.6% margin against 30.2% for Retail."

Result: "My memo recommended no, not on this evidence, and proposed a two-quarter test on a random group of Retail customers instead, which is the only way to answer a cause-and-effect question and far cheaper than a policy change. It was a portfolio project, so the decision was my recommendation. What I'd do differently: write the analysis plan, with the segment split in it, before the first query."

[Interviewer note: **[+Evidence]** quantified and specific; **[+Limits]** says plainly that it was portfolio work, and what it would change. Most of the time went on Action, where it belongs.]

Scoring

Dimension	Score	Why
Correctness	4	Correct query, and explained why the <code>NOT IN</code> alternative fails with a <code>NULL</code>
Structure & communication	4	Signposted: scope questions, then the query, then why <code>NOT IN</code> fails; the STAR answer put its time into Action and Result
Depth & edge cases	4	Raised the <code>NULL</code> trap, cancelled orders and never-ordered customers unprompted
Business judgment	3	Revenue-at-risk framing ties the call list to a decision; no attempt to size the revenue involved
Collaboration	4	Clarified before writing anything and adapted the query to the answers

Overall: extra-points level (mostly 4s). Hire. Not flawless in a dramatic way, just consistently a step past "correct" at every turn. For a mid-level analyst role, expect the interviewer to push further: the same list with a window function (Chapter 13, section 13.8, Pattern 6, at-risk customers), and how you'd check it against what sales already knows.

Learn it in: Chapter 12, section 12.12 (NOT EXISTS, and why NOT IN fails with NULLs); Chapter 13, section 13.8 (Pattern 6, at-risk customers); Chapter 27, sections 27.7 and 27.10 (the memo, and telling the story); Chapter 81, section 81.1 (STAR). **Practise it with:** Chapter 71, Q71-002.

82.5 Mock interview: Data Scientist (mid level)

A 25-minute mock for a mid-level Data Scientist role, condensed, in the same format.

Interviewer: Tell me about a model you're proud of.

Candidate: "It's a portfolio project: lead scoring on a supplies distributor's CRM data, about 7,300 training leads from two years, where 6.6% of the next half-year's leads were won. The question was which leads the sales team should work. I split by date, so the model was always judged on leads that came after the ones it learned from. I compared logistic regression with tuned gradient boosting: they tied on validation, and on the test leads logistic regression was slightly ahead, 0.838 against 0.830, with better-calibrated probabilities. So I kept the simpler model. Then I set the threshold from costs, not 0.5: about ₹1,500 to work a lead and about ₹30,000 of margin from a win. The team can work only about 504 leads in six months, so I sized the list to that. Working the top 504 by score gave an expected ₹23.3 lakh, against ₹17.4 lakh for the old rule of thumb at about the same effort. The limitation I flagged: those are expected values from the model, not money earned, so I recommended a small trial before anyone quoted them."

*[Interviewer note: **[+Simple first]** baseline before the complex model; **[+Business]** quantified at the team's real capacity; **[+Limits]** separated expected from earned. Chapter 76A, Q76A-007 structure, unprompted.]*

Interviewer: You mentioned a test AUC of 0.838. Walk me through what that number actually means, and whether it's "good."

Candidate: AUC is the probability that the model ranks a randomly chosen won lead above a randomly chosen lost one. 0.838 means it gets that order right about 84% of the time: solid, not exceptional. Whether it's good enough depends on the decision it feeds, not on the number alone. Here the base rate is 6.6%, so the number sales actually feels is precision: on the 504-lead list, about one worked lead in five is won. Pairing the model with a cost-based threshold mattered more than the AUC itself.

*[Interviewer note: correct definition, and **[+Business]** resists treating the number as meaningful without the decision and the base rate.]*

*A weaker answer to the same turn: "It means the model is 84% accurate." That confuses ranking with accuracy; on these leads, predicting "lost" for everyone is already 93.4% accurate.
[Correctness 1: wrong.]*

Interviewer: What would you do if a stakeholder asked you to make the model "more accurate" with no further detail?

Candidate: I'd ask what's actually driving that request. Is a specific kind of error causing a real problem: missed high-value leads, or too many false positives wasting sales time? "More accurate" could mean better precision, better recall, or a better AUC, and those often trade off against each other, so I'd want to know which error matters more before doing anything.

[Interviewer note: **[+Clarify]** the same diagnose-before-acting instinct as Chapter 76B's "make the report better" question (Q76B-001), applied to a model instead of a report.]

Interviewer: Your model's feature importance shows a customer ID column ranking unusually high. What's your first reaction?

Candidate: That's a red flag, not a finding. An ID shouldn't predict anything on its own. Two likely mechanisms. Either IDs were assigned in sequence, so the ID is really a proxy for signup date, and with a random split the model learns era effects it can't use on future leads. Or the same customer's rows sit on both sides of the split, so the model recognizes customers instead of learning a pattern. I'd drop the ID, split by time or by customer (Chapter 36), and see whether the AUC falls.

[Interviewer note: **[+Edge cases]** two real leakage mechanisms, and **[+Validate]** a test that tells them apart, not just "that's suspicious."]

A weaker answer to the same turn: "Great, the ID carries signal, so I'd keep it in." [Correctness 1 and Depth & edge cases 1: treats a leak as a feature.]

Scoring

Dimension	Score	Why
Correctness	4	Accurate AUC definition; both leakage mechanisms real, with the test that separates them
Structure & communication	4	The project story hit every beat unprompted: question, split, baseline, threshold, limitation
Depth & edge cases	4	Named two specific leakage mechanisms, not a vague "that's odd"
Business judgment	4	Quantified the model's value at the team's real capacity, against the rule it would replace
Collaboration	3	Strong throughout; could have asked one more clarifying question on the "more accurate" request before answering as fully as they did

Overall: extra-points level (mostly 4s). Hire, with one small, realistic note: even a very strong candidate doesn't need a 4 everywhere. A fresher's version of this interview would stop at the confusion matrix and the threshold; a senior candidate would also be asked how to monitor the model and design the trial (Chapter 30).

Learn it in: Chapter 36, sections 36.3 and 36.7 (splits and leakage); Chapter 39, sections 39.2 and 39.5 (AUC, the cost threshold, capacity); Chapter 76A, Q76A-007 (telling a project story). **Practise it with:** Chapter 74, Q74-016, Q74-020 and Q74-031.

82.6 Mock interview: Data Engineer (mid level)

A 25-minute mock for a mid-level Data Engineer role, condensed. This one is a borderline pass, so the scoring ends with the interviewer's hire decision and the reasoning behind it.

Interviewer: Walk me through how you'd design a pipeline that loads daily sales data, and make it safe to re-run if it fails.

Candidate: "Four stages. First, extract yesterday's orders and order lines from the source system and land them unchanged in a raw layer, with the load date on every row, so I can always reprocess without going back to the source. Second, transform in staging: fix types, drop cancelled orders, apply the revenue formula. Third, load the summary table with an upsert keyed on the date, so if the job is retried, the day's row is updated instead of duplicated. Fourth, checks after every load: revenue can't be negative or missing, and today's order count shouldn't be far below the trailing average, because a truncated file passes every row-level check. A scheduler runs the stages in order, retries a failed step, and alerts a person when a check fails. And I'd monitor freshness against when people need the numbers: if a morning report goes out at 7:30, the summary must be complete well before that."

[Interviewer note: **[+Signpost]** complete, staged answer, not just the load step; **[+Edge cases]** the volume check, unprompted.]

Interviewer: An order from last Tuesday is cancelled today. What does your summary table show for Tuesday tomorrow morning?

Candidate: The upsert updates Tuesday's row with the new total.

Interviewer: Does tomorrow's run read Tuesday at all?

Candidate: No, you're right: it only loads yesterday, so Tuesday keeps its old total. I'd reload a window, say the last seven days, on every run. And if a day loses *all* its orders, the upsert has no row to update it with, so I'd delete and re-insert the window inside one transaction instead.

[Interviewer note: reached the right design, but only after a hint. Late corrections are the classic edge case of this task. **[+Trade-offs]** once prompted, the upsert-versus-delete-and-insert reasoning was sound.]

A stronger answer to the same turn: the delete-and-insert window from the start, with the reason, as in section 82.3's design note. [Depth & edge cases 4.]

Interviewer: Your pipeline ran successfully last night, no errors, but this morning the numbers look wrong. Walk me through debugging this.

Candidate: First I'd separate two possibilities: did it process the right data and compute something wrong, or did it succeed technically while processing something incomplete, like a source file that arrived truncated? I'd check the row counts it processed against a trailing average first, since a silent volume drop is a common cause of "ran fine, wrong numbers" that a pure success-or-failure check wouldn't catch. If the volume looks normal, I'd trace the transformation logic for a recent change.

[Interviewer note: **[+Signpost]** two hypotheses, cheapest check first, not a random guess.]

A weaker answer to the same turn: "I'd rerun it and see if the numbers change." A rerun of an idempotent pipeline on the same bad input gives the same bad output. [Depth & edge cases 1: no hypothesis at all.]

Interviewer: Say you find the issue: a source column got renamed upstream, and your pipeline was silently defaulting the missing column to zero instead of erroring. What do you change, beyond fixing this one instance?

Candidate: Two things, not just the immediate fix. I'd change the load to fail loudly if an expected column is missing, rather than defaulting silently: that's the actual root cause, not the rename itself. And I'd add a step that checks the pipeline's own output against a rough expected range before the run counts as successful, so a similar silent failure is caught the same day, not whenever someone notices the numbers look off.

*[Interviewer note: **[+Business]** separated the specific bug from the systemic gap that let it go unnoticed, exactly the depth this question is fishing for.]*

Interviewer: How would you decide whether this pipeline should be batch or streaming?

Candidate: By how fresh the downstream decision needs to be, not by which is more modern. A daily sales summary doesn't need sub-minute freshness, so batch is the right, simpler choice here. I'd only reach for streaming if there were a real decision that couldn't wait for the next batch window.

*[Interviewer note: **[+Trade-offs]** correctly resisted recommending streaming as the more impressive answer.]*

Scoring

Dimension	Score	Why
Correctness	3	Re-runs are safe and batch is the right call; the first design would have kept stale totals after a late correction
Structure & communication	3	Clear, staged design and a two-hypothesis debugging plan
Depth & edge cases	2	Needed a hint to see that a late cancellation never reaches yesterday-only loads
Business judgment	3	Fail-loudly root cause, and freshness tied to when the report is needed; batch because the decision is daily
Collaboration	3	Took the hint without defensiveness and adapted the design straight away

Overall: borderline pass (mostly 3s, one 2). Hire, with a note. The interviewer's reasoning: "The debugging and root-cause answers are what this job needs every week. The late-correction gap is teachable, and the candidate closed it within one hint. For a senior role, the same gap would be a no-hire: seniors are expected to raise late-arriving changes themselves." A fresher would pass comfortably with the staged design and the batch answer alone; for them the root-cause answer would be an extra point.

Learn it in: Chapter 45, section 45.1 (raw and staging layers) and 45.8 (when the source changes shape); Chapter 46, sections 46.4–46.6 (idempotency, backfills, retries and alerting); Chapter 47, section 47.5 (volume and freshness); Chapter 50, section 50.1 (what "real-time" actually means).
Practise it with: Chapter 77, Q77-002, Q77-007 and Q77-009.

APPLY

Common mistakes

Mistake	Symptom	Fix
Starting an unbounded take-home without asking anything	Days of unpaid work, and a submission that answers a different question from the one the reviewer meant	Check it against section 82.0; ask your scope questions in writing, and state the time you spent
A take-home with no summary, just code and output	Reviewer has to reverse-engineer the finding themselves	Always include a one-page summary leading with the answer
A take-home with no stated limitation	Reads as overconfident or unaware of real gaps	State at least one honest limitation, even under time pressure
Reporting revenue as if it were profit, or one model's numbers under another's name	The reviewer checks one figure, finds it doesn't match, and stops trusting the rest	Reconcile every number to its source query or cell, and say which model and which data each one comes from
A mock answer that's technically correct but thin	Passes on correctness, scores low on depth and business judgment	Practice adding at least one unprompted extra-point move per answer
Treating every mock question as isolated	Missed chances to connect answers into a coherent story	Notice when a later question connects to an earlier answer, and say so
Defensiveness under a "why did this go wrong" follow-up	Reads as fragility, costs the collaboration dimension	Engage with the question directly, as problem-solving, not a personal challenge

Project

Goal: complete one of this chapter's three take-homes yourself, from scratch, before reading the model submission again.

Tools you'll need

PostgreSQL 16 with `riverstone_2025` and `riverstone_lab` (Chapters 12 and 13), and **Python** in the book's virtual environment with `scikit-learn` (installed in Chapter 35) and the Chapter 82 companion folder. A timer, for practicing take-homes and mock answers under the same time pressure a real interview or assignment imposes.

1. Check the assignment against section 82.0, and write down the scope questions you'd ask.
2. Set a timer matching the stated time limit, and attempt the assignment cold.
3. Compare your submission with this chapter's scoring table, dimension by dimension, not just "was I right."

4. Record yourself doing one of the three mock interviews, playing both roles if needed, and compare your answers with the scored transcript and its weaker answers.
5. Identify the one dimension (correctness, structure and communication, depth and edge cases, business judgment, collaboration) you scored lowest on, and practice that one deliberately on your next three questions from any earlier chapter in this part.
6. *Business analysts*: before touching any data, rewrite section 82.1's assignment as a list of requirements questions for the sales head, using Chapter 76B, Q76B-001.

REVIEW

Key terms

take-home assignment · fair take-home · model submission · mock interview · interviewer scoring notes · one-page summary (take-home) · cost-based threshold (reused) · idempotent load (reused) · data test (reused) · volume-anomaly check (reused)

Final-week revision list

- Section 82.0: the three marks of a fair take-home, and the three warning signs.
- Section 82.1: Steps 2 to 4 (the grain of an average, growth by half-year, profit per order), and the one-page summary.
- Section 82.2: why the simpler model shipped; the cost threshold (0.077) and the capacity limit (504 leads).
- Section 82.3: the two failure tests, and why delete-and-insert beats an upsert for a window of days.
- Sections 82.4–82.6: each weaker answer, and the one move that would have lifted it.

NEXT

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric every scoring table in this chapter is built from.
- **Every other chapter in Part 8** supplied the questions these take-homes and mocks draw on: Chapter 71 (SQL), Chapter 74 (machine learning), Chapter 76A (telling a project story), Chapter 76B (requirements), Chapter 77 (pipeline design) and Chapter 81 (STAR answers). The skills themselves were taught earlier: Chapters 12 and 13 (SQL), Chapters 36, 37, 39 and 44 (the lead model and ranking by value), and Chapters 45–47 (pipelines and data quality).
- This is the last chapter in Part 8, Be Interview Ready. The reader who has worked through this part end to end has, in effect, already sat through the mocks in this chapter once, for real, one question at a time. **Chapter 83, The Long Game**, closes the book.

The whole book

Analyst to Architect comes as four books. **Book 1, Theory**, gives you the ideas, with no software (Parts 0 and 1). **Book 2, Practical**, teaches the analyst's tools, hands on (Parts 2 and 3). **Book 3, Implementation**, builds real systems (Parts 4 to 7, then the closing chapter). **Book 4, Be Interview Ready**, is Part 8. Books 1 to 3 share one page count; Book 4 has its own. Each book's contents, after this map, lists the sections of its chapters.

How to Use This Book	Book 1	iii
Part 0 — First Principles: Data from Zero	Book 1	1
Chapter 1. What Is Data?	Book 1	2
Chapter 2. How Computers Store, Move and Protect Data	Book 1	25
Chapter 3. How a Business Runs on Data	Book 1	50
Chapter 4. Numbers Without Fear	Book 1	74
Chapter 5. Thinking Like an Analyst	Book 1	98
Chapter 6. Planning Your Learning	Book 1	120
Part 1 — The Map	Book 1	133
Chapter 7. The Data Landscape	Book 1	134
Chapter 8. The Career Tree: How Skills Unlock Roles	Book 1	160
Chapter 9. How Expertise Actually Forms	Book 1	187
Part 2 — The Analyst	Book 2	207
Chapter 10. Spreadsheet Fundamentals: Excel & Google Sheets	Book 2	208
Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets	Book 2	262
Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script	Book 2	338
Chapter 12. Databases & SQL Foundations	Book 2	390
Chapter 13. SQL for Real Analysis	Book 2	516
Chapter 14. Data Cleaning & Preparation	Book 2	575
Chapter 15. Data Visualization Principles	Book 2	641
Chapter 16. Business Intelligence with Power BI	Book 2	688
Chapter 17. Python from Zero	Book 2	736
Chapter 18. Python for Analysts: pandas & Automation	Book 2	797
Chapter 20. Automating Reports & Delivering Insights	Book 2	872
Chapter 21. Descriptive Statistics & Probability	Book 2	918
Chapter 22. Statistics Without Fooling Yourself	Book 2	962
Chapter 23. Business Acumen, KPIs & Metrics	Book 2	1014
Chapter 24. Requirements, Storytelling & Stakeholders	Book 2	1054
Chapter 25. The Business Analyst Track	Book 2	1076
Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants	Book 2	1107

Chapter 27. Capstone: Your Analyst Portfolio	Book 2	1158
Part 3 — Advanced Analytics & Analytics Engineering	Book 2	1196
Chapter 28. Advanced SQL, Performance & Data Modeling	Book 2	1197
Chapter 34. The Command Line, Linux & Networking Basics	Book 2	1302
Chapter 29. Python as Software, Not Scripts	Book 2	1342
Chapter 32. Analytics Engineering with dbt	Book 2	1409
Chapter 33. The Computer Science You Actually Need	Book 2	1456
Chapter 30. Inference & Experiments	Book 2	1495
Chapter 31. Causal Inference Without Experiments	Book 2	1549
Part 4 — Machine Learning & Data Science	Book 3	1597
Chapter 35. The Math Under the Models	Book 3	1598
Chapter 36. The Machine Learning Workflow & Feature Engineering	Book 3	1654
Chapter 37. Supervised Learning Algorithms	Book 3	1701
Chapter 38. Unsupervised Learning	Book 3	1770
Chapter 39. Evaluation, Tuning, Interpretation & Honesty	Book 3	1819
Chapter 40. Time Series & Forecasting	Book 3	1873
Chapter 41. NLP Foundations	Book 3	1924
Chapter 42. Recommender Systems & Ranking	Book 3	1970
Chapter 43. A First Look at Deep Learning	Book 3	2009
Chapter 44. Capstone: An End-to-End Data Science Project	Book 3	2057
Part 5 — Data Engineering, Integration & Scale	Book 3	2077
Chapter 45. Data Ingestion & Integration	Book 3	2078
Chapter 46. Pipelines & Orchestration	Book 3	2124
Chapter 47. Data Quality, Observability & Contracts	Book 3	2171
Chapter 48. Big Data & Distributed Compute	Book 3	2215
Chapter 49. Storage, Warehouses & Lakehouses	Book 3	2251
Chapter 50. Streaming & Real-Time	Book 3	2286
Chapter 51. Data Activation: Reverse ETL, APIs & System Integration	Book 3	2321
Chapter 52. The Cloud, Containers & Infrastructure as Code	Book 3	2359
Part 6 — Production ML, Generative AI & MLOps	Book 3	2409
Chapter 53. Deep Learning in Depth	Book 3	2410
Chapter 54. Generative AI & Large Language Models	Book 3	2463
Chapter 55. Building AI Applications: RAG, Agents & Evaluation	Book 3	2511
Chapter 56. MLOps: Making Models Survive Production	Book 3	2562
Chapter 57. LLMOps	Book 3	2606
Chapter 58. Intelligent Automation	Book 3	2643

Chapter 59. Industry Case Studies	Book 3	2682
Part 7 — Architecture, Governance & Leadership	Book 3	2698
Chapter 60. Designing Whole Systems	Book 3	2699
Chapter 61. Distributed Systems & Trade-offs	Book 3	2724
Chapter 62. Data Architecture Patterns	Book 3	2749
Chapter 63. Automation Architecture & Governance	Book 3	2772
Chapter 64. Security, Privacy, Governance & Responsible AI	Book 3	2796
Chapter 65. FinOps: The Economics of Data Platforms	Book 3	2829
Chapter 66. Data Strategy, Maturity & Building Data Teams	Book 3	2853
Chapter 67. The Architect as Leader	Book 3	2874
Closing		
Chapter 83. The Long Game	Book 3	2888
Part 8 — Be Interview Ready	Book 4	1
Chapter 68. How Data Hiring Works	Book 4	2
Chapter 69. The Extra-Points Method	Book 4	27
Chapter 70. Excel, Google Sheets, VBA & BI Question Bank	Book 4	45
Chapter 71. SQL Question Bank	Book 4	71
Chapter 72. Python & pandas Question Bank	Book 4	115
Chapter 72A. Data Structures & Algorithms Question Bank	Book 4	148
Chapter 73. Statistics, Probability & Experimentation Bank	Book 4	178
Chapter 74. Machine Learning Question Bank	Book 4	204
Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates	Book 4	225
Chapter 76A. Data Analyst & Data Scientist Question Bank	Book 4	242
Chapter 76B. Business Analyst Question Bank	Book 4	256
Chapter 77. Data Engineering & Data System Design Bank	Book 4	274
Chapter 78. Automation & Integration Question Bank	Book 4	298
Chapter 79. GenAI, LLM & MLOps Question Bank	Book 4	316
Chapter 80. Architecture & Leadership Question Bank	Book 4	339
Chapter 81. Behavioral, HR & Offer Conversations	Book 4	354
Chapter 82. Take-Home Assignments & Mock Interviews	Book 4	370